

Litmus Attribute Reference

Product Version 26.10

May 2026

© 2026 Cadence Design Systems, Inc.
Printed in the United States of America.

Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

Open SystemC, Open SystemC Initiative, OSCI, SystemC, and SystemC Initiative are trademarks or registered trademarks of Open SystemC Initiative, Inc. in the United States and other countries and are used with permission.

Trademarks: Trademarks and service marks of Cadence Design Systems, Inc. (Cadence) contained in this document are attributed to Cadence with the appropriate symbol. For queries regarding Cadence's trademarks, contact the corporate legal department at the address shown above or call 800.862.4522.

All other trademarks are the property of their respective holders.

Restricted Permission: This publication is protected by copyright law and international treaties and contains trade secrets and proprietary information owned by Cadence. Unauthorized reproduction or distribution of this publication, or any portion of it, may result in civil and criminal penalties. Except as expressly permitted below, this publication may not be copied, reproduced, modified, published, uploaded, posted, transmitted, or distributed in any way, including automated processes, training or services involving a large language model, foundation model, deep machine learning, generative artificial intelligence, or any other similar process or technology, without prior written permission from Cadence. Unless otherwise agreed to by Cadence in writing, customers are granted permission to print one (1) hard copy of this publication subject to the following conditions:

1. The publication may be used only in accordance with a written agreement between Cadence and its customer.
2. The publication may not be modified in any way.
3. Any authorized copy of the publication or portion thereof must include all original copyright, trademark, and other proprietary notices and this permission statement.
4. The information contained in this document cannot be used in the development of like products or software, whether for internal or external use, and shall not be used for the benefit of any other party, whether or not for consideration.

Disclaimer: Information in this publication is subject to change without notice and does not represent a commitment on the part of Cadence. Except as may be explicitly set forth in such agreement, Cadence does not make, and expressly disclaims, any representations or warranties as to the completeness, accuracy or usefulness of the information contained in this document. Cadence does not warrant that use of such information will not infringe any third party rights, nor does Cadence assume any liability for damages or costs of any kind that may result from use of such information. Cadence is committed to using respectful language in our code and communications. We are also active in the removal and replacement of inappropriate language from existing content. This product documentation may however contain material that is no longer considered appropriate but still reflects long-standing industry terminology. Such content will be addressed at a time when the related software can be updated without end-user impact.

Restricted Rights: Use, duplication, or disclosure by the Government is subject to restrictions as set forth in FAR52.227-14 and DFAR252.227-7013 et seq. or its successor.

Contents

About This Manual	4
Audience	4
Conventions Used in This Manual	4
1	7
General Attributes	7
Itms	8
2	25
Root Attributes	25
add_filler	26
add_reinforce_pg	29
add_reinforce_pg_critical_path_slack	29
add_reinforce_pg_irdrop_hierarchical_block	30
add_reinforce_pg_irdrop_hierarchical_top_rail_analysis_directory	30
add_reinforce_pg_respect_defined_nets	31
add_reinforce_pg_respect_routes	31
add_reinforce_pg_respect_stdcell_geometry	32
add_route_vias	33
add_stripes	34
add_stripes_allow_jog	36
add_stripes_allow_non_preferred_dir	37
add_stripes_area	37
add_stripes_blocks_without_same_net	38
add_stripes_break_at	39
add_stripes_continue_on_no_selection	41
add_stripes_detailed_log	41
add_stripes_domain_offset_from_core	42
add_stripes_extend_to_closest_target	42
add_stripes_extend_to_first_ring	43
add_stripes_ignore_block_check	43
add_stripes_ignore_block_ring_during_break	44
add_stripes_ignore_drc	44
add_stripes_ignore_non_default_domains	45

add_stripes_in_cell_allow_shift	45
add_stripes_in_cell_only	46
add_stripes_keep_pitch_after_snap	47
add_stripes_mask_color_balance	48
add_stripes_max_extension_distance	50
add_stripes_max_via_size	50
add_stripes_merge_with_all_layers	51
add_stripes_mesh_via	52
add_stripes_offset_from_core	53
add_stripes_opt_stripe_for_routing_track	54
add_stripes_orthogonal_offset	55
add_stripes_orthogonal_only	57
add_stripes_over_row_extension	57
add_stripes_partial_set_through_domain	58
add_stripes_remove_floating_stapling	58
add_stripes_remove_floating_stripe_over_block	59
add_stripes_remove_stripe_under_ring	59
add_stripes_respect_routes	60
add_stripes_route_over_rows_only	61
add_stripes_rows_without_stripes_only	61
add_stripes_same_size_stack_vias	61
add_stripes_skip_via_on_pin	62
add_stripes_skip_via_on_wire_shape	62
add_stripes_spacing_from_block	63
add_stripes_spacing_type	64
add_stripes_split_long_via	64
add_stripes_split_vias	65
add_stripes_split_wire_spacing	65
add_stripes_split_wire_weight	66
add_stripes_split_wire_width	66
add_stripes_stacked_via_bottom_layer	67
add_stripes_stacked_via_top_layer	67
add_stripes_stapling_extend_to_minimum_spacing	68
add_stripes_stapling_nets_style	68
add_stripes_stapling_shift	71
add_stripes_stop_at_closest_target	72
add_stripes_stop_at_last_wire_for_area	72

add_stripes_stripe_min_length	73
add_stripes_stripe_min_width	73
add_stripes_switch_cell_name	74
add_stripes_switch_layer_overlap_length	74
add_stripes_trim_antenna_back_to_shape	75
add_stripes_trim_antenna_max_distance	76
add_stripes_trim_stripe	77
add_stripes_use_exact_spacing	78
add_stripes_use_point_to_point_route	78
add_stripes_use_stripe_width	79
add_stripes_via_using_exact_crossover_size	79
add_stripes_width_file	80
add_target_pg	80
add_target_pg_allow_weak_connect	81
add_target_pg_max_extension_distance	81
add_target_pg_pins	82
add_target_pg_pins_group_distance	83
add_target_pg_respect_routes	85
add_target_pg_share_resource	86
add_tieoffs	86
add_tieoffs_cells	87
add_tieoffs_create_hports	88
add_tieoffs_honor_dont_touch	88
add_tieoffs_honor_dont_use	89
add_tieoffs_max_distance	89
add_tieoffs_max_fanout	90
add_tieoffs_module_prevention	91
add_tieoffs_prefix	91
add_tieoffs_report_hports	92
add_well_taps	93
add_well_taps_abut_boundary_tap_distance	94
add_well_taps_antenna_tap_break_cell	95
add_well_taps_antenna_tap_left_cell	95
add_well_taps_antenna_tap_right_cell	95
add_well_taps_antenna_tap_cell	96
add_well_taps_antenna_tap_pitch	96
add_well_taps_antenna_tap_right_top_edge_cell	97

add_well_taps_avoid_abutment	97
add_well_taps_avoid_vertical_well_abutment	97
add_well_taps_block_boundary_only	98
add_well_taps_bottom_tap_cell	98
add_well_taps_bottom_termination_cell	99
add_well_taps_cell	99
add_well_taps_channel_offset	100
add_well_taps_check_channel	100
add_well_taps_column_cells	100
add_well_taps_create_rows	101
add_well_taps_disable_check_zone_at_boundary	101
add_well_taps_in_row_offset	102
add_well_taps_insert_cells	102
add_well_taps_replace_intersection_cell	103
add_well_taps_rule	104
add_well_taps_site_offset	104
add_well_taps_tap_function_cells	105
add_well_taps_tap_termination_alignment	105
add_well_taps_termination_align	106
add_well_taps_termination_cells	106
add_well_taps_top_tap_cell	107
add_well_taps_vertical_boundary_spacing	107
add_well_taps_top_termination_cell	108
add_well_taps_well_cut_cells	108
bscan	109
dft_boundary_cell_module_prefix	109
dft_jtag_instance_name	109
dft_jtag_module_name	110
check_ac_limit	111
check_design	134
check_drc	136
check_drc_area	137
check_drc_check_illegal_trim_shapes	137
check_drc_check_only	138
check_drc_check_reverse	139
check_drc_check_routing_halo	139
check_drc_check_routing_halo_corner	140

check_drc_disable_rules	140
check_drc_enable_post_passive_fill	141
check_drc_exclude_pg_net	142
check_drc_ignore_cell_blockage	142
check_drc_ignore_fill_wire	143
check_drc_ignore_non_rectangle_shapes	143
check_drc_ignore_trial_route	144
check_drc_inside_via_def	144
check_drc_layer_range	145
check_drc_limit	145
check_drc_max_wrong_way_halo	146
check_drc_ndr_spacing	146
check_drc_quiet	147
check_drc_report	147
check_drc_short_only	148
check_drc_trim_length	148
check_drc_uncolored	149
check_drc_use_min_spacing_on_block_obs	150
cpu	150
power_intent_insert_auto_ls	150
cts	152
clock_trees	156
clock_tree_source_groups	157
cts_add_port_driver_max_distance_from_port	157
cts_add_port_driver_max_distance_from_port_rows	158
cts_adjacent_rows_legal	159
cts_allow_non_std_clock_gate_inputs	160
cts_allow_post_route_sizing_stack_via_rule_update	161
cts_balance_mode	161
cts_blackbox_default_driver_base_pin	162
cts_blackbox_default_load_base_pin	163
cts_buffer_cells	163
cts_buffer_cells_leaf	165
cts_buffer_cells_top	165
cts_cell_density	166
cts_cell_halo_mode	168
cts_cell_halo_rows	169

cts_cell_halo_sites	170
cts_cell_halo_x	171
cts_cell_halo_y	172
cts_clock_gating_cells	173
cts_clock_gate_movement_limit	175
cts_clock_source_cells	175
cts_clock_tree_source_group_clock_trees	176
cts_clock_tree_source_input_max_transition_time	177
cts_cloning_effort	178
cts_cloning_inst_name_suffix	178
cts_cloning_inst_name_suffix_source_group_assignment	179
cts_declone_clock_gates	180
cts_declone_clock_logic	180
cts_delay_cells	181
cts_detailed_cell_warnings	182
cts_exit_if_no_placeable_area	183
cts_exit_if_skew_target_over_constrained	184
cts_exit_if_stage_delay_sigma_target_over_constrained	184
cts_fix_clock_sinks	185
cts_flexible_htree_allow_mixed_sink_polarity	186
cts_flexible_htree_enable_metrics_reporting	186
cts_flexible_htree_placement_legalization_effort	187
cts_force_target_max_transition_time	187
cts_ignore_problematic_skew_as_result_of_dont_touch_nets	188
cts_inst_name_prefix	189
cts_inverter_cells	190
cts_inverter_cells_leaf	191
cts_inverter_cells_top	192
cts_load_capacitance_cells	193
cts_logic_cells	194
cts_manage_power_intent_violations	195
cts_max_fanout	196
cts_max_instance_height	196
cts_max_source_to_sink_net_length	197
cts_max_source_to_sink_net_length_leaf	198
cts_max_source_to_sink_net_length_top	199
cts_max_source_to_sink_net_length_trunk	200

cts_mixed_fanout_net_type	201
cts_move_clock_gates	202
cts_move_logic	202
cts_net_name_prefix	203
cts_pin_insertion_delay_histogram_bin_size	204
cts_post_route_enable	205
cts_post_route_repair_drv	206
cts_post_route_enable_routing_eco	206
cts_post_route_repair_skew_by_buffering	207
cts_primary_delay_corner	208
cts_primary_reporting_skew_groups	209
cts_primary_reporting_skew_groups_log_min_max_sinks	209
cts_repair_congestion	210
cts_repair_drv_by_buffering	211
cts_report_skew_groups_only_with_targets	212
cts_route_clock_tree_nets	213
cts_route_type_flexible_htree	214
cts_route_type_flexible_htree_backside	215
cts_route_type_leaf	215
cts_route_type_top	216
cts_route_type_trunk	217
cts_routing_preferred_layer_effort	218
cts_safety_tmr_isolation_route_type	219
cts_size_clock_gates	220
cts_size_clock_sources	221
cts_size_logic	221
cts_skew_group_report_columns	222
cts_skew_group_report_histogram_bin_size	224
cts_spec_config_base_pin_trace_through_to	225
cts_spec_config_create_clock_tree_source_groups	226
cts_spec_config_create_generator_skew_groups	227
cts_spec_config_create_generator_skew_groups_name_prefix	228
cts_spec_config_create_reporting_only_skew_groups	228
cts_spec_config_trace_multi_output_cells_with_single_clock_output	230
cts_target_max_capacitance	231
cts_target_max_capacitance_leaf	232
cts_target_max_capacitance_top	233

cts_target_max_capacitance_trunk	234
cts_target_max_stage_delay_sigma	235
cts_target_max_transition_time	235
cts_target_max_transition_time_leaf	236
cts_target_max_transition_time_top	237
cts_target_max_transition_time_trunk	238
cts_target_skew	239
cts_timing_connectivity_based_skew_groups_balance_master_clocks	240
cts_timing_connectivity_based_skew_groups	241
cts_timing_connectivity_info	242
cts_top_fanout_threshold	244
cts_update_clock_latency	245
cts_use_inverters	245
cts_use_receiver_model_capacitance_for_drv	246
flexible_htrees	247
skew_groups	248
cwd	250
cwd_setup_file	250
delaycal	251
design	269
dp	274
apply_booth_encoding	274
control_logic_optimization	275
dp_analytical_opt	276
dp_area_mode	277
dp_csa	277
dp_csa_factorize	278
dp_opt_power_opt	279
dp_opt_power_opt_scaling	279
dp_opt_toggle_skew_threshold_for_booth_encoding	280
dp_rename	280
dp_sharing	281
dp_speculation	282
dp_ungroup_during_syn_map	283
eco	285
extract_rc	294
flip_chip	308

flip_chip_allow_layer_change	309
flip_chip_allow_routed_bump_edit	310
flip_chip_auto_pairing_file	311
flip_chip_bottom_layer	312
flip_chip_bump_use_octagon_shape	312
flip_chip_cell_connection_type	313
flip_chip_check_bump_access_directions	314
flip_chip_compaction	316
flip_chip_connect_bump_to_block	316
flip_chip_connect_bump_to_bump	317
flip_chip_connect_power_cell_to_bump	317
flip_chip_constraint_file	319
flip_chip_drop_via_on_all_shapes	319
flip_chip_drop_via_on_power_mesh	321
flip_chip_extra_config	322
flip_chip_finger_direction	322
flip_chip_finger_max_width	323
flip_chip_finger_min_width	324
flip_chip_finger_target_mesh_layer_range	324
flip_chip_honor_bump_connect_target_constraint	325
flip_chip_ignore_pad_type_check	325
flip_chip_lower_layer_route_width	326
flip_chip_lower_layer_prevent_diagonal_routing	327
flip_chip_merge_nearby_pin	327
flip_chip_multi_pad_routing_style	328
flip_chip_multiple_connection	330
flip_chip_pg_mesh_direction	332
flip_chip_pg_mesh_main_width	332
flip_chip_pg_mesh_max_width	333
flip_chip_pg_mesh_width	334
flip_chip_prevent_diagonal_wire_access_bumps	335
flip_chip_prevent_via_under_bump	335
flip_chip_prevent_via_under_bump_extension	336
flip_chip_route_bump_cluster	338
flip_chip_route_pg_style	339
flip_chip_route_style	340
flip_chip_route_width	340

flip_chip_top_layer	341
flip_chip_via_abut_bump	342
flip_chip_wire_to_pad_honor_min_spacing	343
flow	344
flowtool	356
fv	358
design_verification_effort	359
verification_directory_naming_style	359
write_verification_files	361
generate_special_via	362
generate_special_via_accuracy_effort	364
generate_special_via_add_pin_to_pin_vias	365
generate_special_via_align_merged_stack_via_metal	365
generate_special_via_allow_via_expand	366
generate_special_via_allow_wire_shape_change	366
generate_special_via_area_only	368
generate_special_via_bot_enclosure	369
generate_special_via_check_signal_routes	369
generate_special_via_create_double_row_cut_via	370
generate_special_via_create_max_row_cut_via	371
generate_special_via_cut_class_preference	371
generate_special_via_cut_spacing	373
generate_special_via_disable_via_merge	373
generate_special_via_enable_check_drc	374
generate_special_via_extend_out_wire_end	375
generate_special_via_full_cut_via_only	376
generate_special_via_hookup_auto_set_viacell	376
generate_special_via_hookup_contact_max	377
generate_special_via_hookup_contact_pg_track	379
generate_special_via_hookup_fixed_grid	380
generate_special_via_hookup_min_distance	382
generate_special_via_hookup_preplace_fix	383
generate_special_via_hookup_rail_pair	383
generate_special_via_hookup_via_distance	384
generate_special_via_hookup_via_style	385
generate_special_via_hookup_via_rule	389
generate_special_via_hookup_virtual_trim_grid	389

generate_special_via_ignore_design_boundary	390
generate_special_via_ignore_drc	391
generate_special_via_ignore_rule_enclosure	392
generate_special_via_inherit_wire_status	392
generate_special_via_keep_existing_via	393
generate_special_via_keep_fixed_via	393
generate_special_via_mincut_preference	394
generate_special_via_opt_cross_via	395
generate_special_via_opt_via_on_routing_track	396
generate_special_via_parameterized_via_only	397
generate_special_via_partial_overlap_threshold	397
generate_special_via_preferred_vias_only	399
generate_special_via_prefix	400
generate_special_via_reference_boundary	400
generate_special_via_respect_stdcell_geometry	401
generate_special_via_respect_wire_ndr	401
generate_special_via_rule_preference	402
generate_special_via_set_via_expand_dir	402
generate_special_via_snap_via_center_to_grid	403
generate_special_via_split_long_via_global_grid	406
generate_special_via_symmetrical_via_only	407
generate_special_via_top_enclosure	408
generate_special_via_use_track_offset	409
generate_special_via_use_trim_metal_enclosure	409
get_db Category Attributes	409
hdl	411
dp_ungroup_separator	415
hdl_allow_inout_constant_port_connect	416
hdl_allow_instance_name_conflict	416
hdl_allow_positional_connections_for_pg_inst	417
hdl_append_generic_ports	417
hdl_array_naming_style	419
hdl_async_set_reset	420
hdl_auto_async_set_reset	421
hdl_auto_sync_set_reset	421
hdl_bus_wire_naming_style	422
hdl_case_mux_threshold	423

hdl_case_sensitive_instances	423
hdl_convert_one_bit_vector_to_scalar	424
hdl_convert_onebit_vector_wire_to_scalar	425
hdl_create_label_for_unlabeled_generate	427
hdl_decimal_parameter_name	427
hdl_delete_transparent_latch	428
hdl_early_redundant_logic_removal	429
hdl_enable_proc_name	429
hdl_enable_real_support	430
hdl_error_on_blackbox	430
hdl_error_on_latch	431
hdl_error_on_logic_abstract	432
hdl_error_on_negedge	432
hdl_exclude_parameter_in_cell_search	433
hdl_ff_keep_explicit_feedback	433
hdl_ff_keep_feedback	434
hdl_file_row_column_consider_modules_as_leaf_insts	435
hdl_flatten_complex_port	436
hdl_flatten_complex_port_in_bottom_up_flow	436
hdl_generate_index_style	437
hdl_generate_separator	438
hdl_ignore_pragma_names	438
hdl_index_mux_threshold	439
hdl_inout_assign	439
hdl_inout_wand_wor_assign	440
hdl_instance_array_naming_style	442
hdl_interface_separator	442
hdl_keep_first_module_definition	443
hdl_language	443
hdl_latch_keep_feedback	444
hdl_link_from_any_lib	445
hdl_max_loop_limit	445
hdl_max_map_to_mux_control_width	446
hdl_max_memory_address_range	447
hdl_max_recursion_limit	447
hdl_module_definition_resolution	448
hdl_nc_compatible_module_link	449

hdl_new_inout_assign	451
hdl_overwrite_command_line_macros	451
hdl_parameter_module_name	452
hdl_parameter_naming_style	453
hdl_preserve_async_set_reset_priority_logic	453
hdl_preserve_dangling_output_nets	454
hdl_preserve_supply_nets	454
hdl_preserve_sync_control_logic	456
hdl_preserve_sync_set_reset	456
hdl_preserve_unused_flop	457
hdl_preserve_unused_latch	459
hdl_preserve_unused_registers	462
hdl_primitive_input_multibit	465
hdl_record_naming_style	465
hdl_register_array_naming_style	466
hdl_register_naming_style	466
hdl_register_record_naming_style	467
hdl_rename_cdn_flop_pins	467
hdl_rename_cdn_latch_pins	468
hdl_report_case_info	469
hdl_resolve_instance_with_lib_cell	470
hdl_resolve_parameterized_instance_with_structural_module	470
hdl_sv_module_wrapper	473
hdl_sync_set_reset	473
hdl_track_filename_row_col	474
hdl_track_module_elab_memory_and_runtime	475
hdl_unconnected_value	476
hdl_use_block_prefix	486
hdl_use_case_generate_prefix	486
hdl_use_current_dir_before_hdl_search_path	487
hdl_use_cw_first	487
hdl_use_default_parameter_values_in_name	488
hdl_use_for_generate_prefix	488
hdl_use_if_generate_prefix	489
hdl_use_port_default_value	490
hdl_verilog_defines	491
hdl_vhdl_assign_width_mismatch	491

hdl_vhdl_case	493
hdl_vhdl_environment	493
hdl_vhdl_preferred_architecture	493
hdl_vhdl_range_opt	494
hdl_vhdl_read_version	495
hdl_vhdl_strict_compliance	495
init_blackbox_for_undefined	496
init_hdl_search_path	497
input_assert_one_cold_pragma	498
input_assert_one_hot_pragma	499
input_asynchro_reset_blk_pragma	499
input_asynchro_reset_pragma	500
input_case_cover_pragma	501
input_case_decode_pragma	501
input_map_to_mux_pragma	502
input_pragma_keyword	503
input_synchro_enable_blk_pragma	503
input_synchro_enable_pragma	504
input_synchro_reset_blk_pragma	504
input_synchro_reset_pragma	505
proto_hdl	506
script_begin	506
script_end	507
synthesis_off_command	507
synthesis_on_command	508
hier	510
assemble_design_generic_hier	510
ilm	511
init	515
lec	519
clp_enable_1801_hierarchical_bbox	520
wcdc_clock_dom_comb_propagation	521
write_clp_lib_statetable	521
write_do_lec_add_no_black_box_retime_subdesign	522
write_do_lec_analyze_abort	522
write_do_lec_analyze_setup	523
write_do_lec_auto_analyze	524

write_do_lec_black_box_ilm_modules	525
write_do_lec_compare_threads	526
write_do_lec_composite_compare	526
write_do_lec_dft_constraint_file	527
write_do_lec_gzip_lec_json	528
write_do_lec_hier_append_string	528
write_do_lec_hier_comp_threshold	529
write_do_lec_hier_compare_string	529
write_do_lec_hier_prepend_string	530
write_do_lec_lib_statetable	531
write_do_lec_low_power_analysis	531
write_do_lec_multithread_license_list	532
write_do_lec_no_dft_constraints	532
write_do_lec_parallel_threads	533
write_do_lec_post_add_no_translate_modules	533
write_do_lec_run_hier_check_non_equivalent	534
write_do_lec_set_cdn_syn_root	535
write_do_lec_uniquify	535
write_do_lec_use_lec_model	536
write_do_lec_use_smart_lec	537
write_do_lec_verbose	538
mesg	538
mesg_severity_downgrade	539
report_library_message_summary	539
metric	541
oa_read_write	548
opt	565
auto_partition	572
auto_ungroup	573
auto_ungroup_max_threshold	574
avoid_tied_inputs	575
boundary_optimize_constant_hpins	575
boundary_optimize_equal_opposite_hpins	577
boundary_optimize_feedthrough_hpins	578
boundary_optimize_invert_hpins	580
boundary_optimize_invert_hpins_rename_nets	582
combinational_sequential_merge_message_threshold	583

delete_flops_on_preserved_net	584
delete_hier_insts_on_preserved_net	584
delete_unloaded_insts	585
delete_unloaded_seqs	586
double_cell_search_pattern	587
drc_first	588
drc_max_cap_first	589
drc_max_fanout_first	590
drc_max_trans_first	592
enable_strict_percent_control	593
fix_min_drcs	594
map_clock_tree	594
map_drc_first	595
map_latch_allow_async_decomp	596
map_prefer_non_inverted_clock_line	597
map_respect_rtl_clk_phase	597
merge_combinational_hier_instances	599
merge_non_scan_to_scan_flops	600
minimize_uniquify	600
multibit_allow_unused_bits	601
multibit_area_power_scoring	602
multibit_aware_seq_mapping	602
multibit_aware_seq_mapping_higher_priority	603
multibit_debug	604
multibit_invert_clock_phase	606
multibit_mapping_effort_level	606
multibit_predefined_allow_unused_bits	607
multibit_unused_input_value	608
opt_activity_refresh_args	609
opt_activity_refresh_flow	610
opt_activity_refresh_stages	611
opt_adaptive_subd1_flow	612
opt_add_always_on_feed_through_buffers	612
opt_add_insts	613
opt_add_ports	614
opt_add_repeater_report_failure_reason	614
opt_all_end_points	615

opt_allow_floating_outputs	616
opt_allow_only_cell_swapping	617
opt_allow_multi_bit_on_flop_with_sdc	617
opt_area_recovery	618
opt_area_recovery_setup_target_slack	619
opt_boundary_optimize_invert_hpins_rename_extension	620
opt_clock_traverse_by_level	621
opt_clone_insts_list	621
opt_concatenate_default_and_user_prefixes	622
opt_consider_routing_congestion	623
opt_constant_inputs	624
opt_constant_nets	624
opt_delete_insts	625
opt_detail_drv_failure_reason	625
opt_detail_drv_failure_reason_max_num_nets	626
opt_dont_use_qn_sequential_pins	627
opt_down_size_insts	627
opt_drv	628
opt_drv_margin	629
opt_drv_with_miller_cap	629
opt_duplicate_cte_constrained_hport	630
opt_early_hold_fixing	630
opt_enable_always_on_type_in_remove_assign	631
opt_enable_clock_pulse_width_checks	632
opt_enable_data_to_data_checks	632
opt_enable_drv_failure_reason_warn	633
opt_enable_pdv2_clock_opt_flow	634
opt_enable_restructure	634
opt_enable_targeted_synthesis	635
opt_fix_fanout_load	635
opt_flop_pins_report	636
opt_flops_report	637
opt_hier_add_antenna_cell	637
opt_hier_opt_stage	638
opt_high_effort_cells	638
opt_hold_allow_overlap	639
opt_hold_allow_resize	640

opt_hold_allow_setup_tns_degradation	640
opt_hold_ignore_path_groups	641
opt_hold_cells	641
opt_hold_on_excluded_clock_nets	642
opt_hold_slack_threshold	643
opt_hold_target_slack	643
opt_honor_density_screen	644
opt_honor_fences	645
opt_icg_enable_pin_rebuffering	645
opt_leakage_to_dynamic_ratio	646
opt_max_density	647
opt_max_length	647
opt_move_insts	648
opt_multi_bit_combinational_mode	649
opt_multi_bit_combinational_opt	649
opt_multi_bit_combinational_merge_timing_effort	650
opt_multi_bit_combinational_split_timing_effort	651
opt_multi_bit_flop_merge_bank_label_inference	652
opt_multi_bit_flop_merge_timing_effort	652
opt_multi_bit_flop_opt	653
opt_multi_bit_flop_name_prefix	654
opt_multi_bit_flop_name_separator	654
opt_multi_bit_flop_name_suffix	656
opt_multi_bit_flop_reorder_bits	656
opt_multi_bit_flop_split_post_cts	657
opt_multi_bit_flop_split_post_route	658
opt_multi_bit_flop_split_report_failure_reason	659
opt_multi_bit_flop_split_timing_effort	659
opt_multi_bit_merge_scan_head	660
opt_multi_bit_split_scan_head	660
opt_multi_bit_unused_bits	661
opt_multi_bit_unused_bit_count	662
opt_multibit_allow_shift_register_head_flop_merge	663
opt_new_inst_prefix	663
opt_new_net_prefix	664
opt_one_pass_lec	664
opt_pin_swapping	665

opt_podv2_flow_effort	665
opt_post_route_allow_overlap	666
opt_post_route_area_reclaim	667
opt_post_route_art_flow	667
opt_post_route_check_antenna_rules	668
opt_post_route_drv_recovery	669
opt_post_route_fix_clock_drv	669
opt_post_route_fix_glitch	670
opt_post_route_fix_si_transitions	671
opt_post_route_hold_recovery	671
opt_post_route_report_si_transitions	672
opt_post_route_setup_recovery	673
opt_power_effort	674
opt_pre_route_ndr_aware	676
opt_preserve_all_sequential	676
opt_preserve_hpin_function	677
opt_print_ports_nets_preserved_for_callback	677
opt_remove_redundant_insts	678
opt_report_multi_bit_unmerged_reasons	679
opt_resize_flip_flops	679
opt_resize_level_shifter_and_iso_insts	680
opt_resize_power_switch_insts	681
opt_retime_move_mux_loop_with_register	681
opt_retime_register_naming_suffix	682
opt_route_opt_recovery	683
opt_sequential_genus_restructure_report_failure_reason	684
opt_setup_target_slack	685
opt_skew	685
opt_skew_apply_delay_limits_to_full_flow	686
opt_skew_ccopt	687
opt_skew_post_route	688
opt_skew_pre_cts	688
opt_skew_delay_pre_cts	689
opt_skew_macro_only	689
opt_skew_max_allowed_delay	690
opt_skew_min_allowed_delay	691
opt_skew_no_boundary	692

opt_target_based_opt_file	692
opt_target_based_opt_hold_file	693
opt_target_based_opt_file_only	694
opt_tied_inputs	694
opt_time_design_compress_reports	695
opt_time_design_expanded_view	696
opt_time_design_num_paths	696
opt_time_design_report_net	697
opt_time_design_report_path_type	697
opt_time_design_vertical_timing_summary	698
opt_tns	699
opt_unfix_clock_insts	699
opt_use_multi_output_gates	700
opt_verbose	701
optimize_constant_0_flops	703
optimize_constant_1_flops	704
optimize_constant_across_preserved	705
optimize_constant_feedback_seqs	705
optimize_constant_latches	707
optimize_merge_flops	708
optimize_merge_latches	709
optimize_seq_x_to	710
optimize_yield	710
percent_control_tolerance_for_map	711
phys_early_skew_flow	711
preserve_combinational_loop_ports_nets	712
propagate_constant_from_timing_model	712
proto_feasible_target	713
proto_feasible_target_adjust_slack_pct	714
proto_feasible_target_threshold	715
proto_feasible_target_threshold_clock_pct	715
retime_async_reset	716
retime_effort_level	717
retime_optimize_reset	718
retime_verification_flow	718
syn_exact_match_sequential_async_control	719
st_launch_wait_time	720

timing_library_ui_respect_preserve	721
timing_library_use_max_cap_look_up_table	721
tns_critical_range	722
opt_signoff	724
partition_floorplan	777
partition_floorplan_export	778
partition_floorplan_import	778
place	780
power	787
power_intent	889
precision	894
enable_ui_precision	894
program	896
read_physical	898
report_obj	900
report_obj_display_limit	900
route	901
extract_keep_fill_wires	905
route_add_antenna_inst_prefix	905
route_adjust_auto_via_weight	906
route_allow_inst_overlaps	906
route_allow_pin_as_feedthru	907
route_antenna_cell_name	908
route_antenna_diode_insertion	908
route_concurrent_minimize_via_count_effort	909
route_connect_to_bumps	910
route_design_high_freq_bus_mixed_layer	911
route_design_top_level_design	911
route_detail_add_passive_fill_only_on_layers	912
route_detail_allow_passive_fill_only_in_layers	912
route_detail_antenna_eco_list_file	913
route_detail_auto_stop	914
route_detail_check_mar_on_cell_pin	914
route_detail_end_iteration	915
route_detail_fix_antenna	915
route_detail_fix_antenna_on_secondary_pg_nets	916
route_detail_fix_antenna_with_gate_array_filler_mode	917

route_detail_merge_abutting_cut	917
route_detail_min_length_for_spread_wire	918
route_detail_min_length_for_widen_wire	919
route_detail_min_slack_for_opt_wire	920
route_detail_no_taper_in_layers	920
route_detail_no_taper_on_output_pin	921
route_detail_on_grid_only	922
route_detail_post_route_litho_repair	923
route_detail_post_route_spread_wire	924
route_detail_post_route_swap_via	925
route_detail_post_route_via_pillar_effort	926
route_detail_post_route_wire_widen	927
route_detail_post_route_wire_widen_rule	927
route_detail_postroute_via_priority	928
route_detail_search_and_repair	929
route_detail_signoff_effort	929
route_detail_stub_routing_in_first_layer	930
route_detail_use_multi_cut_via_effort	931
route_diode_insertion_for_clock_nets	932
route_disable_route_rule_on_via_pillar_to_special_net_wire	932
route_eco_ignore_existing_route	933
route_enable_mixed_size_tsv_via	934
route_enable_route_rule_si_limit_length	934
route_enforce_route_rule_on_special_net_wire	935
route_extra_via_enclosure	936
route_fix_clock_nets	936
route_high_freq_constraint_groups	937
route_high_freq_match_report_file	938
route_high_freq_num_reserved_layers	938
route_high_freq_remove_floating_shield	939
route_high_freq_search_repair	939
route_high_freq_shield_trim_length	940
route_honor_exclusive_region	940
route_honor_power_domain	941
route_ignore_antenna_top_cell_pin	941
route_ignore_follow_pin_shapes	942
route_interposer_allow_diagonal_trunk	943

route_interposer_control_options	943
route_interposer_interlayer_shielding_layers	944
route_interposer_interlayer_shielding_nets	944
route_interposer_interlayer_shielding_offsets	945
route_interposer_interlayer_shielding_widths	946
route_interposer_same_layer_shielding_net	946
route_interposer_same_layer_shielding_width_spacing	947
route_interposer_trunk_routing_layers	948
route_interposer_trunk_routing_width_spacing	948
route_number_fail_limit	949
route_number_info_limit	949
route_number_thread	950
route_number_warning_limit	951
route_process_node	951
route_rc_extraction_corner	952
route_relaxed_route_rule_spacing_to_power_ground_nets	952
route_reserve_space_for_multi_cut	953
route_reverse_direction	954
route_route_clock_nets_first	955
route_selected_net_only	955
route_shield_crosstie_offset	956
route_shield_length_threshold	956
route_shield_report_skip_status	957
route_shield_stripe_layer_range	957
route_shield_tap_cell_insertion	958
route_shield_tap_cell_name	958
route_skip_analog	959
route_strictly_honor_1d_routing	960
route_strict_honor_route_rule	960
route_stripe_layer_range	961
route_third_party_data	962
route_tieoff_to_shapes	962
route_top_level_design	964
route_trim_pull_back_distance_from_boundary	964
route_trunk_with_cluster_target_size	965
route_use_auto_via	966
route_via_weight	967

route_with_eco	969
route_with_litho_driven	969
route_with_si_driven	970
route_with_timing_driven	970
route_with_trim_metal	971
route_with_via_in_pin	972
route_with_via_only_for_block_cell_pin	973
route_with_via_only_for_stdcell_pin	973
route_trial_max_print_ignored_pad_nets	975
route_special	976
safety	987
safety_dcls_isolate_halo_type	988
safety_dcls_isolate_type	988
safety_dcls_route_types	989
safety_flow_enable	990
safety_midas_enable	990
safety_ser_cells	991
safety_ser_type	992
safety_tmr_isolate_clock	992
safety_tmr_isolate_reset	993
safety_tmr_isolate_scan_enable	993
safety_tmr_spacing	994
safety_tmr_spacing_x	995
safety_tmr_spacing_y	995
safety_tmr_well_tap_cells	996
si	996
ssyn	1015
pbs_db_directory	1015
pbs_mim_flow	1016
system	1017
tech	1027
timing	1029
timing_library	1169
aocv_library	1171
bank_based_multibit_inferencing	1172
disable_when_checks	1172
enable_library_pins_sorting_in_mmmc	1172

honor_valid_location	1173
ignore_pin_error_in_test_cell_function	1173
ignore_scan_combinational_arcs	1174
ignore_sigma_arc_mismatch	1174
init_design_mmmc_skip_inactive	1175
large_cell_arc_threshold	1175
link_library	1175
mark_async_pin_using_timing_arcs	1176
mark_macro_as_power_switch_cell	1176
mark_valid_lp_cell_as_usable	1176
multibit_allow_async_phase_map	1177
multibit_auto_exclude_registers_with_exceptions	1179
multibit_cells_from_different_busses	1180
multibit_preserve_inferred_instances	1180
multibit_preserved_net_check	1181
multibit_seqs_instance_naming_style	1181
multibit_seqs_members_naming_style	1183
multibit_split_string	1184
operating_conditions	1186
opt_multibit_combinational_name_concatenate_string	1187
opt_multibit_seqs_name_concatenate_string	1187
opt_use_default_related_pg_pin_for_always_on	1188
opt_use_next_state_type_only_to_assign_sync_control	1188
override_library_max_drc	1188
parse_lib_moments_table	1189
reload_when_for_macro_cell	1189
socv_library	1190
support_internal_pg_pins	1190
support_master_slave_flop	1191
support_multi_seq_elements	1191
support_multi_seq_scan_latch	1191
syn_exact_match_sequential_sync_control	1192
target_library	1192
timing_enable_set_reset_latch_preset_clear_arcs	1193
timing_library_allow_invalid_primary_power_pins_lib_cell	1193
timing_library_allow_multiple_sync_control	1194
timing_library_ignore_attribute_check_during_pin_convert	1194

timing_library_infer_cap_range_from_c1cn_dynamic_pin_cap_model	1195
timing_library_infer_cap_range_from_dynamic_pin_cap_model	1195
timing_library_limit_messages	1195
timing_library_look_up_drv_per_frequency	1196
timing_library_multibit_force_merge_combinational_cells	1197
timing_library_multibit_force_merge_iso_cells	1198
timing_library_multibit_force_merge_sequential_cells	1199
timing_library_multibit_use_combinational_cells	1200
timing_library_respect_async_control_priority	1201
timing_library_sequential_in_out_phase_opt	1201
timing_library_support_3d_table_power_arc	1203
timing_library_support_combinational_clock	1204
timing_library_support_merge_libs	1204
timing_library_support_serial_scan_in_multibit_cell	1205
timing_library_support_transparent_latch_group	1205
timing_library_suppress_library_messages	1205
timing_library_use_sequential_test_cell	1206
treat_non_seq_arc_cell_as_unusable	1206
turbo_lib_loading	1206
use_area_from_lef	1207
use_compatibility_based_grouping	1207
use_main_cell_output_function_for_test_cell	1208
use_multibit_cells	1208
use_multibit_iso_cells	1210
use_multibit_seq_and_tristate_cells	1211
use_scan_seqs_for_non_dft	1212
write_db	1214
write_def	1215
write_netlist	1216
3	1218
Rule Attributes	1218
rule_filter	1219
rule_instance	1222
rule_occurrence	1228
4	1233
SVA Attributes	1233

ltms_sva_assertion_in_module_limit	1236
ltms_sva_assertion_limit	1236
ltms_sva_assertion_prefix	1236
ltms_sva_assertion_suffix	1236
ltms_sva_cgc_file_prefix	1236
ltms_sva_cgc_file_suffix	1237
ltms_sva_cgc_use_edge_sampling	1237
ltms_sva_clock_selection	1237
ltms_sva_disable_sdc_comment_processing	1237
ltms_sva_file_prefix	1238
ltms_sva_file_suffix	1238
ltms_sva_generate_comment	1238
ltms_sva_include_per_property_comment	1238
ltms_sva_inc_buf	1239
ltms_sva_mcp_assertion_limit	1239
ltms_sva_mcp_assertion_prefix	1239
ltms_sva_mcp_assertion_suffix	1239
ltms_sva_mcp_file_prefix	1239
ltms_sva_mcp_file_suffix	1240
ltms_sva_mcp_module_prefix	1240
ltms_sva_mcp_module_suffix	1240
ltms_sva_mcp_reset_signal_name	1240
ltms_sva_mcp_skip_single_cycle	1241
ltms_sva_module_prefix	1241
ltms_sva_module_suffix	1241
ltms_sva_num_paths_per_exception	1241
ltms_sva_path_based_sampling_exhaustive	1241
ltms_sva_path_based_sampling_opt_level	1242
ltms_sva_reset_signal_name	1242
ltms_sva_sca_clock_define	1242
ltms_sva_sca_clock_name	1242
ltms_sva_sca_config_pin_exclude	1243
ltms_sva_sca_config_pin_include	1243
ltms_sva_sca_enable_sca_text	1243
ltms_sva_sca_fail_string	1243
ltms_sva_sca_file_extension	1243
ltms_sva_sca_file_prefix	1244

ltms_sva_sca_file_suffix	1244
ltms_sva_sca_id	1244
ltms_sva_sca_module_name	1244
ltms_sva_sca_reset_define	1244
ltms_sva_sca_reset_polarity_define	1245
ltms_sva_sca_reset_polarity_value	1245
ltms_sva_sca_reset_signal_name	1245
ltms_sva_sca_tag	1245
ltms_sva_sca_top_inst_define	1246
ltms_sva_sca_top_inst_name	1246
ltms_sva_sca_use_user_defined_clock	1246
ltms_sva_sca_valid_string	1246
ltms_sva_sdt_clock_define	1246
ltms_sva_sdt_clock_name	1247
ltms_sva_sdt_file_prefix	1247
ltms_sva_sdt_file_suffix	1247
ltms_sva_sdt_id	1247
ltms_sva_sdt_label	1247
ltms_sva_sdt_module_name	1248
ltms_sva_sdt_property_macro_name	1248
ltms_sva_sdt_reset_define	1248
ltms_sva_sdt_tag	1248
ltms_sva_sdt_top_define	1249
ltms_sva_sdt_top_inst_name	1249
ltms_sva_sdt_use_user_defined_clock	1249
ltms_sva_sfp_assertion_limit	1249
ltms_sva_sfp_assertion_prefix	1250
ltms_sva_sfp_assertion_suffix	1250
ltms_sva_sfp_cycle	1250
ltms_sva_sfp_file_prefix	1250
ltms_sva_sfp_file_suffix	1250
ltms_sva_sfp_module_prefix	1251
ltms_sva_sfp_module_suffix	1251
ltms_sva_sfp_reset_signal_name	1251
ltms_sva_traverse_through_mux_select_pin	1251
5	1252

CPC Attributes	1252
timing_disable_promotion_clock_prefix	1253
timing_disable_promotion_clock_group_prefix	1253
timing_enable_constant_promotion_to_top_driver	1253
timing_promote_boundary_path_exceptions	1253
timing_promote_cgc_enable_source_invert_and_multiply_by	1253
timing_promote_cgc_source_pin_with_master	1254
timing_promote_clock_to_top_port	1254
timing_promote_derived_clocks_for_exceptions	1254
timing_promote_global_block_constraints	1254
timing_promote_in_out_delay_over_combinational_cone	1255
timing_promote_inherit_clock_relationships	1255
timing_promote_input_transition_as_annotated_transition	1255
timing_promote_internal_set_input_delay	1255
timing_promote_internal_set_output_delay	1255
timing_promote_inverted_clock_pair_to_top	1256
timing_promote_set_load_over_combinational_cone	1256

About This Manual

This manual provides a reference of the attributes available to the user when using the Litmus software with the Stylus Common user interface.

Audience

This manual is written for experienced designers of digital integrated circuits. Such designers must be familiar with design planning, placement and routing, block implementation, chip assembly, and design verification. Designers must also have a solid understanding of UNIX and Tcl/Tk programming.

Conventions Used in This Manual

This section describes the typographic and syntax conventions used in this manual.

<code>text</code>	Indicates text that you must type exactly as shown. For example: <code>analyze_connectivity -analyze all</code>
<code>text</code>	Indicates information for which you must substitute a name or value. In the following example, you must substitute the name of a specific file for <code>configfile</code> : <code>wroute filename configfile</code>
<code>text</code>	Indicates the following: • Text found in the graphical user interface (GUI), including form names, button labels, and field names • Terms that are new to the manual, are the subject of discussion, or need special emphasis • Titles of manuals
<code>[]</code>	Indicates optional arguments. In the following example, you can specify none, one, or both of the bracketed arguments: <code>command [-arg1] [arg2 value]</code>

[]	Indicates an optional choice from a mutually exclusive list. In the following example, you can specify any of the arguments or none of the arguments, but you cannot specify more than one: <pre>command [arg1 arg2 arg3 arg4]</pre>
{ }	Indicates a required choice from a mutually exclusive list. In the following example, you must specify one, and only one, of the arguments: <pre>command {arg1 arg2 arg3}</pre>
{ [] [] }	Indicates a required choice of one or more items in a list. In the following example, you must choose one argument from the list, but you can choose more than one: <pre>command {[arg1] [arg2] [arg3]}</pre>
{ }	Indicates curly braces that must be entered with the command syntax. In the following example, you must type the curly braces: <pre>command arg1 {x y}</pre>
...	Indicates that you can repeat the previous argument.
.	Indicates an omission in an example of computer output or input.
<i>Command - Subcommand</i>	Indicates a command sequence, which shows the order in which you choose commands and subcommands from the GUI menu. In the following example, you choose <i>Power</i> from the menu, then <i>Power Planning</i> from the submenu, and then <i>Add Rings</i> from the displayed list: <i>Power - Power Planning - Add Rings</i> This sequence opens the <i>Add Rings</i> form..

General Attributes

- [Itms](#)

Itms

- [Itms_adjust_precision](#)
- [Itms_clock_regexp_matching](#)
- [Itms_const_allowed_driving_cell](#)
- [Itms_const_attr_val_display_limit](#)
- [Itms_const_case_analysis_check_skip_transition](#)
- [Itms_const_clk_check_skip_false_path](#)
- [Itms_const_clock_conv_include_logical_exclusivity](#)
- [Itms_const_clock_mapping_for_unloaded_clock](#)
- [Itms_const_combinational_path_delay_condition](#)
- [Itms_const_consider_bidirectional_clock_exception_as_exclusive](#)
- [Itms_const Consolidate occurrences](#)
- [Itms_const_constraint_analysis_type](#)
- [Itms_const_generated_clock_enable_launch_clock_based_source_path_count](#)
- [Itms_const_harmonic_cycle_threshold](#)
- [tms_const_hier_logic_transition_condition](#)
- [Itms_const_hier_wire_transition_condition](#)
- [Itms_const_input_delay_condition](#)
- [Itms_const_input_delay_ratio](#)
- [Itms_const_io_check_skip_unconstrained_path](#)
- [Itms_const_joint_clock_mapping_for_master_generated_clocks](#)
- [Itms_const_liberty_generated_clock_enable_source_path_type_check](#)
- [Itms_const_max_capacitance](#)
- [Itms_const_max_clock_limit_on_clock_pin](#)
- [Itms_const_max_clock_transition](#)
- [Itms_const_max_input_transition](#)

- ltms_const_min_capacitance
- ltms_const_min_clock_transition
- ltms_const_min_input_transition
- ltms_const_multicycle_path_enable_literal_based_cycle_count_check
- ltms_const_multicycle_path_multiplier_difference
- ltms_const_output_delay_condition
- ltms_const_output_delay_ratio
- ltms_const_report_all_witnesses
- ltms_const_skip_clock_on_mux_select
- ltms_const_strict_clock_mapping_for_internal_clock
- ltms_const_verbose_val_display_limit
- ltms_const_virtual_clock_naming_style
- ltms_enable_audit_timing_flow
- ltms_enable_conflicting_design_ref
- ltms_enable_design_clocks_opt_sync_grp
- ltms_enable_elaborated_netlist_flow
- ltms_enable_exc17_classification_from_exc1
- ltms_enable_exc_clk1_flp7_optimization
- ltms_enable_full_file_name
- ltms_enable_graph_based_ignored_exceptions_flow
- ltms_enable_reg2out_bfs_phase_transformation
- ltms_enable_reg2out_clock_exception_reporting
- ltms_enable_sample_path_points
- ltms_enable_stage_profiling
- ltms_exclude_fp_async_logical_physical_clockpairs
- ltms_hier_compare_propagated_clock_latency
- ltms_max_path_to_explore

- [ltms_max_sequential_depth_to_explore](#)
- [ltms_nominal_gate_delay](#)
- [ltms_nominal_wire_delay](#)
- [ltms_report_ports_with_unconstrained_points](#)
- [ltms_sdc_version_min_limit](#)
- [ltms_skip_top_path_description](#)
- [ltms_timing_const_delay_path_to_consider_io_delay](#)
- [ltms_timing_path_based_undefined_clock_uncertainty](#)

ltms_adjust_precision

`ltms_adjust_precision {true/false}`

Default: false

Data_type: Boolean

This attribute enable `ui_precision`.

ltms_clock_regexp_matching

`ltms_clock_regexp_matching string`

Default: " "

Data_type: string

Specifies the regular expression used during clock mapping to match top-level clock to corresponding block-level clock.

ltms_const_allowed_driving_cell

`ltms_const_allowed_driving_cell string`

Default: " "

Data_type: string

Specifies allowed driving cells. This attribute configures the checking for the rules: `CONST_LINT_ITR4`.

ltms_const_attr_val_display_limit

ltms_const_attr_val_display_limit *integer*

Default: no_value

Data_type: integer

Specify limit to restrict display rule attribute data.

ltms_const_case_analysis_check_skip_transition

ltms_const_case_analysis_check_skip_transition {true|false}

Default: false

Data_type: boolean

Specify whether case analysis checks to skip transition values.

ltms_const_clk_check_skip_false_path

ltms_const_clk_check_skip_false_path {true|false}

Default: false

Data_type: boolean

Specifies whether clock check (CONST_HIER_CLK6) to skip false path.

ltms_const_clock_conv_include_logical_exclusivity

ltms_const_clock_conv_include_logical_exclusivity {true|false}

Default: false

Data_type: boolean

Specifies whether to include logically exclusive clock pairs.

ltms_const_clock_mapping_for_unloaded_clock

ltms_const_clock_mapping_for_unloaded_clock {true|false}

Default: false

Data_type: boolean

Specify whether to enable clock mapping for a clock that has no sink.

ltms_const_combinational_path_delay_condition

`ltms_const_combinational_path_delay_condition` *string*

Default: " "

Data_type: string

Specify the condition used to check the combinational path delay. The attribute configures the checking for the rules: CONST_DGN_CMB3.

ltms_const_consider_bidirectional_clock_exception_as_exclusive

`ltms_const_consider_bidirectional_clock_exception_as_exclusive` {true|false}

Default: false

Data_type: boolean

Consider bidirectional inter-clock false paths as mutually exclusive clocks. This attribute configures the rules: CONST_LINT_GRP1.

ltms_const Consolidate occurrences

`ltms_const Consolidate occurrences` {true|false}

Default: false

Data_type: boolean

Specify whether to consolidate rule occurrences. The attribute configures the checking for the rules: CONST_CLK_DEF1, CONST_CLK_DEF6.

ltms_const_constraint_analysis_type

`ltms_const_constraint_analysis_type` *string*

Default: " "

Data_type: string

Specify the analysis type of layout. The attribute defines layout for checking of the rules:
CONST_LINT_CTR4, CONST_LINT_CTR5, CONST_LINT_HFN1, CONST_LINT_LAT2

ltms_const_generated_clock_enable_launch_clock_based_source_path_count

`ltms_const_generated_clock_enable_launch_clock_based_source_path_count {true|false}`

Default: false

Data_type: boolean

Specify the generated clock source path count to be launch clock based instead of path based. The attribute configures the checking for the rules: CONST_CLK_DEF4

ltms_const_harmonic_cycle_threshold

`ltms_const_harmonic_cycle_threshold integer`

Default: no_value

Data_type: integer

Specify harmonic cycle threshold for a pair of correlated clocks.

tms_const_hier_logic_transition_condition

`ltms_const_hier_logic_transition_condition string`

Default: " "

Data_type: String

Specifies logic transition condition for hierarchical check.

ltms_const_hier_wire_transition_condition

`ltms_const_hier_wire_transition_condition string`

Default: " "

Data_type: String

Specifies wire transition condition for hierarchical check.

ltms_const_input_delay_condition

ltms_const_input_delay_condition *integer*

Default: " "

Data_type: string

Specifies the condition used to check the input delay. This attribute configures the checking for the rules: CONST_IO_IDL5.

ltms_const_input_delay_ratio

ltms_const_input_delay_ratio *double*

Default: no_value

Data_type: double

Specify the ratio with respect to the clock period used by ltms_const_input_delay_condition attribute. The attribute configures the checking for the rules: CONST_IO_IDL5.

ltms_const_io_check_skip_unconstrained_path

ltms_const_io_check_skip_unconstrained_path {true|false}

Default: false

Data_type: boolean

Specify whether IO checks to skip unconstrained path.

ltms_const_joint_clock_mapping_for_master_generated_clocks

ltms_const_joint_clock_mapping_for_master_generated_clocks {true|false}

Default: false

Data_type: boolean

Specify the clock mapping condition of block generated and its master clock by ltms_const_joint_clock_mapping_for_master_generated_clocks attribute.

ltms_const_liberty_generated_clock_enable_source_path_type_check

ltms_const_liberty_generated_clock_enable_source_path_type_check {true|false}

Default: false

Data_type: boolean

Specifies whether to enable source path type check for Liberty generated clock. This attribute configures the checking for the rules: CONST_CLK_DEF40.

ltms_const_max_capacitance

ltms_const_max_capacitance double

Default: no_value

Data_type: double

Specifies the maximum value for the capacitance characterization range. This attribute configures the checking for the rules: CONST_LINT_OLD1.

ltms_const_max_clock_limit_on_clock_pin

ltms_const_max_clock_limit_on_clock_pin integer

Default: no_value

Data_type: integer

Specifies the maximum number of clocks reaching a clock pin. This attribute configures the checking for the rules: CONST_CLK_DEF45.

ltms_const_max_clock_transition

ltms_const_max_clock_transition double

Default: no_value

Data_type: double

Specifies the maximum value for the clock transition characterization range. This attribute configures the checking for the rules: CONST_LINT_CTR2, CONST_LINT_CTR3, CONST_LINT_CTR4.

ltms_const_max_input_transition

ltms_const_max_input_transition double

Default: no_value

Data_type: double

Specifies the maximum value for the input transition characterization range. This attribute configures the checking for the rules: CONST_LINT_ITR1.

ltms_const_min_capacitance

ltms_const_min_capacitance double

Default: no_value

Data_type: double

Specifies the minimum value for the capacitance characterization range. This attribute configures the checking for the rules: CONST_LINT_OLD1.

ltms_const_min_clock_transition

ltms_const_min_clock_transition double

Default: no_value

Data_type: double

Specifies the minimum value for the clock transition characterization range. This attribute configures the checking for the rules: CONST_LINT_CTR2, CONST_LINT_CTR3, CONST_LINT_CTR4.

ltms_const_min_input_transition

ltms_const_min_input_transition double

Default: no_value

Data_type: double

Specifies the minimum value for the input transition characterization range. This attribute configures the checking for the rules: CONST_LINT_ITR1.

ltms_const_multicycle_path_enable_literal_based_cycle_count_check

ltms_const_multicycle_path_enable_literal_based_cycle_count_check {true|false}

Default: false

Data_type: boolean

Specify whether to enable literal based cycle count check for multicycle path. The attribute configures the rule: CONST_HIER_EXC7

False: Timing intent based check which considers required time.

True: Literal based check which considers cycle count only.

ltms_const_multicycle_path_multiplier_difference

ltms_const_multicycle_path_multiplier_difference *string*

Default: " "

Data_type: string

Specify the difference between setup and hold multipliers for multicycle paths.

at_least_one: Setup multiplier is at least one greater than hold multiplier.

one: Setup multiplier is exactly one greater than hold multiplier.

ltms_const_output_delay_condition

ltms_const_output_delay_condition *string*

Default: " "

Data_type: string

Specifies the condition used to check the output delay. This attribute configures the checking for the rules: CONST_IO_ODL5.

ltms_const_output_delay_ratio

ltms_const_output_delay_ratio *double*

Default: no_value

Data_type: double

Specify the ratio with respect to the clock period used by ltms_const_output_delay_condition attribute.

The attribute configures the checking for the rules: CONST_IO_IDL5.

ltms_const_report_all_witnesses

```
ltms_const_report_all_witnesses {true|false}
```

Default: false

Data_type: boolean

Specify whether to report all the witnesses for a given rule occurrence.

The attribute configures the checking for the rules: CONST_LINT_REF1, CONST_LINT_REF4, CONST_CLK_DEF1, CONST_CLK_DEF6, CONST_CLK_DEF45, CONST_IO_ITR10, CONST_CG_CHK4.

ltms_const_skip_clock_on_mux_select

```
ltms_const_skip_clock_on_mux_select {true|false}
```

Default: false

Data_type: Boolean

Specifies whether to skip clock on mux select.

ltms_const_strict_clock_mapping_for_internal_clock

```
ltms_const_strict_clock_mapping_for_internal_clock {true|false}
```

Default: false

Data_type: boolean

Specify whether clock mapping for block level internal clock to skip top level propagated clock.

ltms_const_verbose_val_display_limit

```
ltms_const_verbose_val_display_limit string
```

Default: no_value

Data_type: string

Specifies limit to restrict display rule verbose message data.

ltms_const_virtual_clock_naming_style

`ltms_const_virtual_clock_naming_style` *string*

Default: " "

Data_type: string

Specifies allowed virtual clock naming style. This attribute configures the checking for the rules: CONST_LINT_CLK9.

ltms_enable_audit_timing_flow

`ltms_enable_audit_timing_flow` {*true|false*}

Default: false

Data_type: Boolean

This attribute enable audit timing flow.

ltms_enable_conflicting_design_ref

`ltms_enable_conflicting_design_ref` {*true|false*}

Default: false

Data_type: boolean

Enables conflicting design ref pin in CONST_SCA_CHK1

ltms_enable_design_clocks_opt_sync_grp

`ltms_enable_design_clocks_opt_sync_grp` {*true|false*}

Default: false

Data_type: boolean

Specify whether to enable design synchronous clocks groups in CONST_IO_IDL13/ODL13.

ltms_enable_elaborated_netlist_flow

ltms_enable_elaborated_netlist_flow {true|false}

Default: false

Data_type: Boolean

This attribute enable elaborated netlist flow.

ltms_enable_exc17_classification_from_exc1

ltms_enable_exc17_classification_from_exc1 {true|false}

Default: false

Data_type: boolean

Specifies enabling of EXC17 classification from EXC1 for hierarchical exception checks.

ltms_enable_exc_clk1_flp7_optimization

ltms_enable_exc_clk1_flp7_optimization {true|false}

Default: false

Data_type: boolean

Specifies enabling of CONST_EXC_CLK1 & CONST_EXC_FLP7 signoff checks optimization.

ltms_enable_full_file_name

ltms_enable_full_file_name {true|false}

Default: false

Data_type: boolean

Specify whether to enable display of full file name in 2 SDC compare report.

ltms_enable_graph_based_ignored_exceptions_flow

ltms_enable_graph_based_ignored_exceptions_flow {true|false}

Default: false

Data_type: Boolean

Specifies whether to enable advanced ignored exceptions handling flow.

ltms_enable_reg2out_bfs_phase_transformation

```
ltms_enable_reg2out_bfs_phase_transformation {true|false}
```

Default: false

Data_type: Boolean

This attribute enable BFS phase transformation for reg2out exception checking.

ltms_enable_reg2out_clock_exception_reporting

```
ltms_enable_reg2out_clock_exception_reporting {true|false}
```

Default: false

Data_type: boolean

Specify whether to enable data display of stage wise processing.

ltms_enable_sample_path_points

```
ltms_enable_sample_path_points {true|false}
```

Default: false

Data_type: boolean

Specify whether to enable sample path start/end points support in CONST_EXC_OLP*.

ltms_enable_stage_profiling

```
ltms_enable_stage_profiling {true|false}
```

Default: false

Data_type: boolean

Specify whether to enable data display of stage wise processing.

ltms_exclude_fp_async_logical_physical_clockpairs

ltms_exclude_fp_async_logical_physical_clockpairs {true|false}

Default: false

Data_type: boolean

Specify whether to suppress violations for false path when startpoint and endpoint clocks have an asynchronous/logically_exclusive/physically_exclusive relationship.

The attribute configures the checking for the rules: CONST_EXC_FLP10

ltms_hier_compare_propagated_clock_latency

ltms_hier_compare_propagated_clock_latency {true|false}

Default: false

Data_type: boolean

Controls the behavior of the CONST_HIER_LAT1 partitioning check, specifically in scenarios involving propagated clocks and clock latency settings.

When set to **false**, the check **ignores** cases where:

- A block clock (`bclk`) is mapped to a top-level propagated clock (`tclk`),
- `set_clock_latency` is defined for `bclk` in the block SDC,
- But **not** defined for `tclk` at the top level.

This allows flexibility in hierarchical clock latency validation when block-level constraints are applied independently of top-level mappings.

ltms_max_path_to_explore

ltms_max_path_to_explore integer

Default: no_value

Data_type: Integer

Specifies maximum path to explore.

ltms_max_sequential_depth_to_explore

`ltms_max_sequential_depth_to_explore integer`

Default: no_value

Data_type: integer

Specifies maximum sequential depth to explore during graph traversal.

ltms_nominal_gate_delay

`ltms_nominal_gate_delay double`

Default: no_value

Data_type: integer

Specifies a unit gate delay value (default: 0ps).

ltms_nominal_wire_delay

`ltms_nominal_wire_delay double`

Default: no_value

Data_type: integer

Specifies an average lumped wire delay value (default: 0ps).

ltms_report_ports_with_unconstrained_points

`ltms_report_ports_with_unconstrained_points {true|false}`

Default: false

Data_type: boolean

Specify whether to report ports without `set_input_delay/set_output_delay` going to an unconstrained endpoint/coming from an unconstrained startpoint.

The attribute configures the checking for the rules: CONST_IO_IDL1/CONST_IO_ODL1

ltms_sdc_version_min_limit

`ltms_sdc_version_min_limit string`

Default: " "

Data_type: string

Specifies minimum limit for SDC version.

ltms_skip_top_path_description

```
ltms_skip_top_path_description {true|false}
```

Default: false

Data_type: boolean

Specifies disabling of top path description attribute top_from/top_through/top_to in HIER_EXC.

ltms_timing_const_delay_path_to_consider_io_delay

```
ltms_timing_const_delay_path_to_consider_io_delay {true|false}
```

Default: false

Data_type: boolean

Specify whether timing delay path to be considered set by

ltms_timing_const_delay_path_to_consider_io_delay attribute.

The attribute configures the checking for the rules:

CONST_IO_IDL1/CONST_IO_ODL1/CONST_IO_IDL13/CONST_IO_ODL13.

ltms_timing_path_based_undefined_clock_uncertainty

```
ltms_timing_path_based_undefined_clock_uncertainty {true|false}
```

Default: false

Data_type: boolean

Specify whether undefined clock uncertainty is based on timing path.

Root Attributes

- [add_filler](#)
- [add_reinforce_pg](#)
- [add_route_vias](#)
- [add_stripes](#)
- [add_target_pg](#)
- [add_tieoffs](#)
- [add_well_taps](#)
- [bscan](#)
- [check_ac_limit](#)
- [check_design](#)
- [check_drc](#)
- [cpi](#)
- [cts](#)
- [cwd](#)
- [delaycal](#)
- [design](#)
- [dp](#)
- [eco](#)
- [extract_rc](#)
- [flip_chip](#)
- [flow](#)
- [flowtool](#)
- [fv](#)
- [generate_special_via](#)
- [get_db Category Attributes](#)
- [hdl](#)
- [hier](#)
- [ilm](#)
- [init](#)
- [lec](#)
- [mesg](#)
- [metric](#)
- [oa_read_write](#)
- [opt](#)
- [opt_signoff](#)
- [partition_floorplan](#)
- [place](#)
- [power](#)
- [power_intent](#)
- [precision](#)
- [program](#)
- [read_physical](#)
- [report_obj](#)
- [route](#)
- [route_special](#)
- [safety](#)
- [si](#)
- [ssyn](#)
- [system](#)
- [tech](#)
- [timing](#)
- [timing_library](#)
- [write_db](#)
- [write_def](#)

- [write_netlist](#)

add_filler

The root attributes in the `add_fillers` category are:

- [add_fillers_cells](#)
- [add_fillers_check_trim_rule](#)
- [add_fillers_double_height_filler_insertion](#)
- [add_fillers_enable_restore_filler_flow](#)
- [add_fillers_prefix](#)
- [vt_swap_out_dir](#)
- [vt_swap_out_file](#)
- [vt_swap_to_fix_filler_violations](#)

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `add_fillers` category, use the following command:

```
get_db -category add_fillers
```

add_fillers_cells

```
add_fillers_cells {{list-of-cells1}} {list-of-cells2} ...}
```

Default: ""

Data_type: string

Specifies the filler cells to add with `add_fillers`. Enclose the filler cell names in quotation marks (") or curly braces ({ }). You can specify multiple cell lists at one time, using curly braces ({ }) to separate each one.

add_fillers_check_trim_rule

```
add_fillers_check_trim_rule {true | false}
```

Default: false

Data_type: bool, read/write

Controls trim rule checking during the filler insertion. When set to true, the trim rule is checked, and runtime increases.

add_fillers_double_height_filler_insertion

```
add_fillers_double_height_filler_insertion {true | false}
```

Default: false

Data_type: bool, read/write

Enables double height filler during filler insertion.

add_fillers_enable_restore_filler_flow

```
add_fillers_enable_restore_filler_flow {true | false}
```

Default: true

Data_type: bool, read/write

When this option is set to `false`, fillers are not saved in the memory (which is default i.e the save and restore filler insertion flow is default ON).

add_fillers_prefix

```
add_fillers_prefix prefix
```

Default: ""

Data_type: string

Specifies the prefix for the added filler cells.

vt_swap_out_dir

`vt_swap_out_dir` *vtSwapReportFileDirectory*

Default: Current directory `./`

Data_type: string, read/write

Specifies the directory of the vt swap report file,

vt_swap_out_file

`vt_swap_out_file` *fileName*

Default: ""

Data_type: string, read/write

Specifies the file name of the vt swap report.

vt_swap_to_fix_filler_violations

`vt_swap_to_fix_filler_violations` { true | false }

Default: false

Data_type: string, read/write

Changes the VT of the functional cell to fix filler gaps.

add_reinforce_pg

The root attributes in the `add_reinforce_pg` category are:

- `add_reinforce_pg_critical_path_slack`
- `add_reinforce_pg_irdrop_hierarchical_block`
- `add_reinforce_pg_irdrop_hierarchical_top_rail_analysis_directory`
- `add_reinforce_pg_respect_defined_nets`
- `add_reinforce_pg_respect_routes`
- `add_reinforce_pg_respect_stdcell_geometry`

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `add_reinforce_pg` category, use the following command:

```
get_db -category add_reinforce_pg
```

add_reinforce_pg_critical_path_slack

```
add_reinforce_pg_critical_path_slack slack
```

Default: 0.0

Data_type: double, read/write

Specifies a slack threshold value (in nanoseconds) for critical nets during timing-driven PG stripe reinforcement. It means that `add_reinforce_pg` honors all the violation paths with slack value lesser than the specified `add_reinforce_pg_critical_path_slack` value, since all such associated nets are considered critical.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```
- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_reinforce_pg_irdrop_hierarchical_block

```
add_reinforce_pg_irdrop_hierarchical_block block
```

Default: ""

Data_type: string, read/write

Reports the IR drop violations on the specified list of hierarchical block(s) to fix IR drop. The names of the specified hierarchical blocks should be instantiations of the same master block, which is the loaded block. If you specify the * value of this parameter, all the instantiations of the loaded block are considered.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_reinforce_pg_irdrop_hierarchical_top_rail_analysis_directory

```
add_reinforce_pg_irdrop_hierarchical_top_rail_analysis_directory {directory_name}
```

Default: ""

Data_type: string, read/write

Reads the rail analysis information of full chip available in the specified directory for the hierarchical flow blocks. It collects the IR drop information of the specified block and uses it to fix the IR drop.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_reinforce_pg_respect_defined_nets

`add_reinforce_pg_respect_defined_nets {file_name}`

Default:

Data_type: string, read/write

When this attribute is used, `add_reinforce_pg` respects the user-defined nets listed in the specified file, which are critical in stripe generation. This helps in reserving space around the specified nets, and the newly added PG wires are DRC clean with these nets. This attribute is independent of the `add_reinforce_pg_respect_routes` attribute. However, when it is used together with the `add_reinforce_pg_respect_routes` attribute, `add_reinforce_pg` respects all of them.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

`set_db attribute_name value`

- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

add_reinforce_pg_respect_routes

`add_reinforce_pg_respect_routes {none | fixed | fixed_and_clock | all}`

Default: fixed_and_clock

Data_type: enum {none fixed fixed_and_clock all}, read/write

Specifies the route types to honor check of pre-existing routes during stripe generation. This attribute always honor all covers and PGs, but does not honor the trail routes, Following are the attribute values:

- `none`: Specifies that no route types are respected.
- `fixed`: Respects fixed regular wires.
- `fixed_and_clock`: Respects fixed and routed clocks.
- `all`: Respects fixed and routed clocks along with the routed signals.

The following table presents the support of the `add_reinforce_pg_respect_routes` attribute values with various statuses of net:

Target PG Honoring a Wire			add_reinforce_pg_respect_routes Values			
Net Use	Status		none	fixed	fixed_and_clock	all
P/G			Yes	Yes	Yes	Yes
Non-P/G	Cover		Yes	Yes	Yes	Yes
	Fixed		No	Yes	Yes	Yes
	Routed	Clock	No	No	Yes	Yes
		Signal	No	No	No	Yes
	Unknown		No	No	No	No

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

add_reinforce_pg_respect_stdcell_geometry

`add_reinforce_pg_respect_stdcell_geometry {none sequential all}`

Default: all

Data_type: enum {none sequential all}, read/write

Specifies the cell pin types to be respected in via generation. This attribute ignores the standard cell pins to control DRC violation checking on standard cell pins in via generation.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_route_vias

The root attributes in the `add_route_vias` category are:

- [add_route_vias_auto](#)
- [add_route_vias_ndr_only](#)

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `add_route_vias` category, use the following command:

```
get_db -category add_route_vias
```

add_route_vias_auto

```
add_route_vias_auto {true | false}
```

Default: `false`

Enables auto via generation when design is loaded. The option is of type bool.

add_route_vias_ndr_only

```
add_route_vias_ndr_only {true | false}
```

Default: `false`

Generates vias for non default rules only.

add_stripes

The root attributes in the `add_stripes` category are:

- `add_stripes_allow_jog`
- `add_stripes_allow_non_preferred_dir`
- `add_stripes_area`
- `add_stripes_blocks_without_same_net`
- `add_stripes_break_at`
- `add_stripes_continue_on_no_selection`
- `add_stripes_detailed_log`
- `add_stripes_domain_offset_from_core`
- `add_stripes_extend_to_closest_target`
- `add_stripes_extend_to_first_ring`
- `add_stripes_ignore_block_check`
- `add_stripes_ignore_block_ring_during_break`
- `add_stripes_ignore_drc`
- `add_stripes_ignore_non_default_domains`
- `add_stripes_in_cell_allow_shift`
- `add_stripes_in_cell_only`
- `add_stripes_keep_pitch_after_snap`
- `add_stripes_mask_color_balance`
- `add_stripes_max_extension_distance`
- `add_stripes_max_via_size`
- `add_stripes_merge_with_all_layers`
- `add_stripes_mesh_via`
- `add_stripes_offset_from_core`
- `add_stripes_opt_stripe_for_routing_track`

- `add_stripes_orthogonal_offset`
- `add_stripes_orthogonal_only`
- `add_stripes_over_row_extension`
- `add_stripes_partial_set_through_domain`
- `add_stripes_remove_floating_stapling`
- `add_stripes_remove_floating_stripe_over_block`
- `add_stripes_remove_stripe_under_ring`
- `add_stripes_respect_routes`
- `add_stripes_route_over_rows_only`
- `add_stripes_rows_without_stripes_only`
- `add_stripes_same_size_stack_vias`
- `add_stripes_skip_via_on_pin`
- `add_stripes_skip_via_on_wire_shape`
- `add_stripes_spacing_from_block`
- `add_stripes_spacing_type`
- `add_stripes_split_long_via`
- `add_stripes_split_vias`
- `add_stripes_split_wire_spacing`
- `add_stripes_split_wire_weight`
- `add_stripes_split_wire_width`
- `add_stripes_stacked_via_bottom_layer`
- `add_stripes_stacked_via_top_layer`
- `add_stripes_stapling_extend_to_minimum_spacing`
- `add_stripes_stapling_nets_style`
- `add_stripes_stapling_shift`
- `add_stripes_stop_at_closest_target`
- `add_stripes_stop_at_last_wire_for_area`

- [add_stripes_stripe_min_length](#)
- [add_stripes_stripe_min_width](#)
- [add_stripes_switch_cell_name](#)
- [add_stripes_switch_layer_overlap_length](#)
- [add_stripes_trim_antenna_back_to_shape](#)
- [add_stripes_trim_antenna_max_distance](#)
- [add_stripes_trim_stripe](#)
- [add_stripes_use_exact_spacing](#)
- [add_stripes_use_point_to_point_route](#)
- [add_stripes_use_stripe_width](#)
- [add_stripes_via_using_exact_crossover_size](#)
- [add_stripes_width_file](#)

To set the value of any root attribute, use the [set_db](#) command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the [get_db](#) command:

```
get_db attribute_name
```

To get the current value of all attributes in the `add_stripes` category, use the following command:

```
get_db -category add_stripes
```

add_stripes_allow_jog

```
add_stripes_allow_jog {none|{[padcore_ring ][block_ring ]}}
```

Default: padcore_ring block_ring

Data_type: enum* {none padcore_ring block_ring}, read/write

Specifies the type that stripe can jog to connect.

- If set to `padcore_ring` and/or `block_ring`, the stripes are allowed to be on the same/different metal layer to connect to `padcore_ring/block_ring`.
- If set to `none`, the stripe is not allowed to jog on any layer. A straight/direct connection (no jog)

with switch layer is allowed to make the connection.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_allow_non_preferred_dir

```
add_stripes_allow_non_preferred_dir {none | blockring| corering | padring}
```

Default: padring

Data_type: enum* {none blockring corering padring stripe}, read/write

Specifies whether to allow non-preferred direction to jog to target. When set to `none`, the non-preferred direction jogging is disabled.

Note:

- 0 or `false` is not a legal value in this parameter.
- There are five available values in this parameter.
- The value `none` is exclusive to other option values.
- The default value is `padring`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_area

```
add_stripes_area string
```

Default: ""

Data_type: string, read/write

Specifies the coordinates of the rectangular and/or rectilinear design area(s) in which the stripes must be created. It appends the specified area to previous ones.

Note:

- If `add_stripes_area` conflicts with `add_stripes -area`, `add_stripes -area` being on higher priority, `add_stripes_area` is ignored.
- If stripes are only to be created in the area in current command, reset the previous areas by one of the following three ways:

```
set_db add_stripes_area -reset
```

```
set_db add_stripes_area {}
```

```
set_db add_stripes_area ""
```

- When this attribute is used with `add_stripes_domain_offset_from_core true`, the `add_stripes -start_offset` value for specified area is measured from core on using `add_stripes_area`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_blocks_without_same_net

```
add_stripes_blocks_without_same_net {none | block_ring | selected_block  
| unassigned_bump | overlap_ringpin | outer_ring | outside_ringmacro  
| blocks_without_same_net}
```

Default: none

Data_type: enum* {none block_ring selected_block unassigned_bump overlap_ringpin outer_ring outside_ringmacro blocks_without_same_net}, read/write

Specifies the stripe break point.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_break_at

```
add_stripes_break_at {none|{[block_ring] [selected_block] [unassigned_bump]  
[overlap_ringpin] [outer_ring][outside_ringmacro]}}
```

Default: none

Data_type: enum* {none block_ring selected_block unassigned_bump overlap_ringpin outer_ring outside_ringmacro blocks_without_same_net}, read/write

Specifies the break point of stripes.

If any of the

options block_ring, selected_block, unassigned_bump, overlap_ringpin, outer_ring **or** outside_ringmacro is specified, the stripe breaks when it encounters the specified structure that are not in the same wire group.

block_ring

Stripes break and connect to the block rings and ring pins if they are encountered.

Note: This attribute applies only to block rings that surround a block, and does not apply to block rings that surround rows or standard cells. If the block is a ring macro, the stripe will break at the ring pin inside the block ring.

selected_block

Stripes stop at the block boundary when the block does not contain a blockage.

Stripes stop at the block boundary when the block contains a blockage and spacing from the block boundary to the edge of the blockage is greater than the minimum spacing value of that layer.

Stripes stop at the minimum spacing away from the blockage when the spacing from the block boundary to the edge of the blockage is less than the minimum spacing of that layer.

Note: If the selected block has block ring, stripes break and connect to the block ring.

unassigned_bumps

Stripes break at unassigned bumps. It prevents the unassigned bumps from being ignored and shorted with the power and ground nets by the stripes created in the same metal layer.

overlap_ringpin

Prevents stripes from overlapping with same direction ring pins of a different net.

Note: This option is only valid if the ring pin is more than half the length of the macro in the direction of the ring pin.

outer_ring

When a stripe encounters nested block rings or ring pins that are not in the same wire group, the stripe breaks at the outer ring. If not specified, the stripe breaks at the innermost ring.

Note: This attribute works only in conjunction with the following attribute values of

add_stripes_break_at:

- block_ring
- selected_block

outside_ringmacro

Stripes break at a small distance outside the block if the selected block has ring pins.

Note: This attribute is available only if you also specify `add_stripes_break_at selected_block`

blocks_without_same_net

Breaks stripes at blocks, where no pin has same net as the nets specified in the `add_stripes` command.

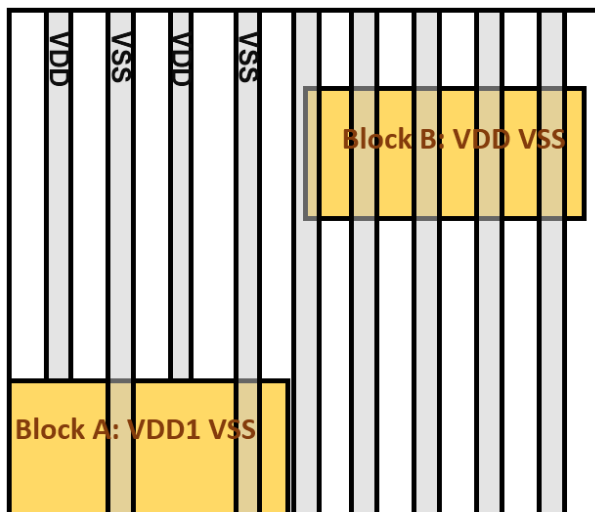
Example

Block A has power supply VDD1 and VSS, Block B has power supply VDD and VSS, commands are as follows:

```
set_db add_stripes_break_at blocks_without_same_net
```

```
add_stripes -nets {VDD VSS}...
```

In the above commands, VDD breaks at Block A, VSS goes through Block A, VDD and VSS will go through Block B. Refer to the screen below:



This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_continue_on_no_selection

```
add_stripes_continue_on_no_selection {true | false}
```

Default: false

Data_type: bool, read/write

If the design has no power domain, or no fences are selected, `add_stripes` ignores the `-over_power_domain` parameter and generates stripes extending to core ring.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_detailed_log

```
add_stripes_detailed_log {true | false}
```

Default: false

Data_type: bool, read/write

Setting this attribute will make the `add_stripes` output an extensive log or a detailed log.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```


add_stripes_domain_offset_from_core

```
add_stripes_domain_offset_from_core {true | false}
```

Default: false

Data_type: bool, read/write

Creates stripes over a power domain to align with the global stripes in your design by enabling you to use the offset that was used for the global stripes as the offset for the power domain stripes. This attribute uses the offset from the core, which is the default for global stripes.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_extend_to_closest_target

```
add_stripes_extend_to_closest_target {none | stripe | ring | same_dir_stripe  
| area_boundary}
```

Default: none

Data_type: enum* {ring stripe same_dir_stripe area_boundary none}, read/write

Extends the stripe to the specified target. This attribute can work in conjunction with `add_stripes_max_extension_distance`.

Following is the description of the attribute values:

- `none`: Restricts the stripe from extending to any target.
- `stripe`: Allows the stripe to extend to the nearby stripe.
- `ring`: Allows the stripe to extend to the nearby ring.
- `same_dir_stripe`: Allows the stripe to extend to the stripe in the same direction.
- `area_boundary`: Allows the stripe to extend to certain area boundary.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_extend_to_first_ring

```
add_stripes_extend_to_first_ring {true | false}
```

Default: false

Data_type: bool, read/write

If set to `true`, stripe antennas are extended to the closest ring. If set to `false`, stripes stop at the boundary specified by the `-extend_to` attribute of the `add_stripes` command.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_ignore_block_check

```
add_stripes_ignore_block_check {true | false}
```

Default: false

Data_type: bool, read/write

If set to `false`, when a stripe encounters a ring, the software does not check whether that ring is surrounding a block. If set to `true`, when a stripe encounters a ring, the software automatically checks whether a block is enclosed within the ring.

Note: This attribute only works in conjunction with the `-break_at` attribute of the `add_stripes` command.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_ignore_block_ring_during_break

```
add_stripes_ignore_block_ring_during_break {true | false}
```

Default: false

Data_type: bool, read/write

If set to `false`, breaks the stripe outside the block if the stripe cannot go over the block.

If set to `true`, breaks the stripe at the block ring if it cannot go over the block.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_ignore_drc

```
add_stripes_ignore_drc {true | false}
```

Default: false

Data_type: bool, read/write

Setting this attribute creates stripes ignoring DRCs but honoring user defined values. This attribute should be used only for prototyping.

When the `add_reinforce_pg` command is run, the value of the `add_stripes_ignore_drc` attribute is automatically set to `false`, irrespective of its value set before by the `set_db` command. However, its initial value is restored once the `add_reinforce_pg` command is terminated.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_ignore_non_default_domains

```
add_stripes_ignore_non_default_domains {true | false}
```

Default: false

Data_type: bool, read/write

Creates global stripes over domains without breaking. If you do not want the stripes to break at the non-default domains, set this variable to `true` before using the `add_stripes` command.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_in_cell_allow_shift

```
add_stripes_in_cell_allow_shift {true | false}
```

Default: false

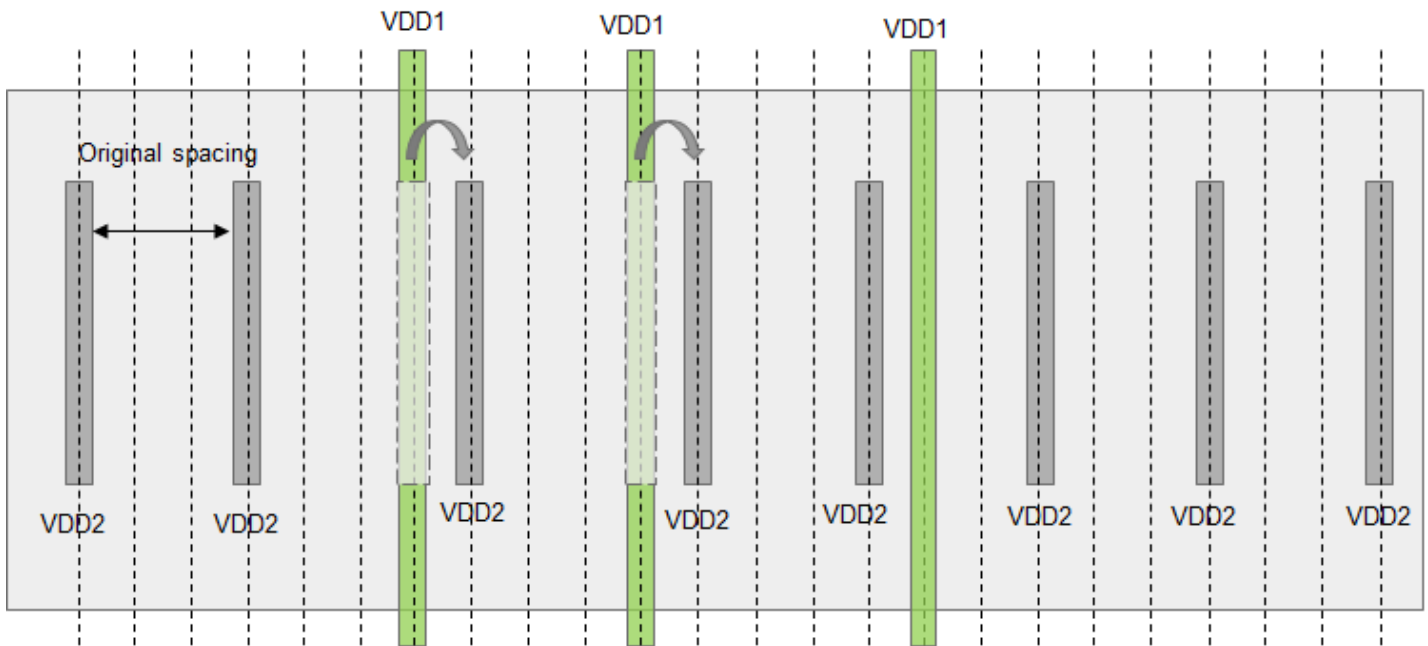
Data_type: bool, read/write

Allows to shift the newly created pillars when meeting DRC, if there is already an existing `special_wire` on the given track. This attribute works only when the `add_stripes_in_cell_only` attribute is set to `true`.

When the `add_stripes_in_cell_allow_shift` attribute is set to `false`, the pillar is removed when meeting violation. When set to `true`, the newly created pillar is shifted to right among the three tracks when meeting violation. The next pillar's position depends on the final position of the current pillar.

The distance between these pillars should satisfy the user specified spacing. The spacing between the two pillars should not be smaller than the user-specified spacing.

The following example shows that using the `add_stripes_in_cell_allow_shift` attribute, the newly created pillars of `VDD2` are shifted right to avoid DRC with `VDD1`.



This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_in_cell_only

```
add_stripes_in_cell_only {true | false}
```

Default: false

Data_type: bool, read/write

Control whether the stripe will be generated inside selected or specified instances. If no instance is selected or without the `add_stripes -master` parameter, no stripe will be generated.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_keep_pitch_after_snap

`add_stripes_keep_pitch_after_snap {true | false}`

Default: false

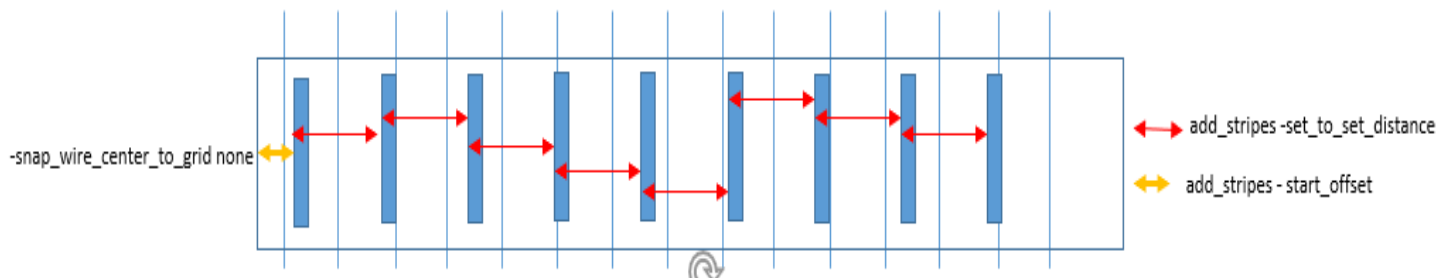
Data_type: bool, read/write

When set to `true`, the distance between two PG sets for snapping is set to greater than or equal to the pitch value specified through the `add_stripes -set_to_set_distance` parameter. When set to `false`, the newly created PG sets are snapped from the current location to the nearest location valid for the `add_stripes_keep_pitch_after_snap` attribute. The average pitch of the newly generated stripe is approximately equal to the specified `add_stripes -set_to_set_distance` parameter value.

This attribute works with the `add_stripes -snap_wire_center_to_grid` parameter. The behavior of the `add_stripes_keep_pitch_after_snap` attribute under different scenarios is explained below.

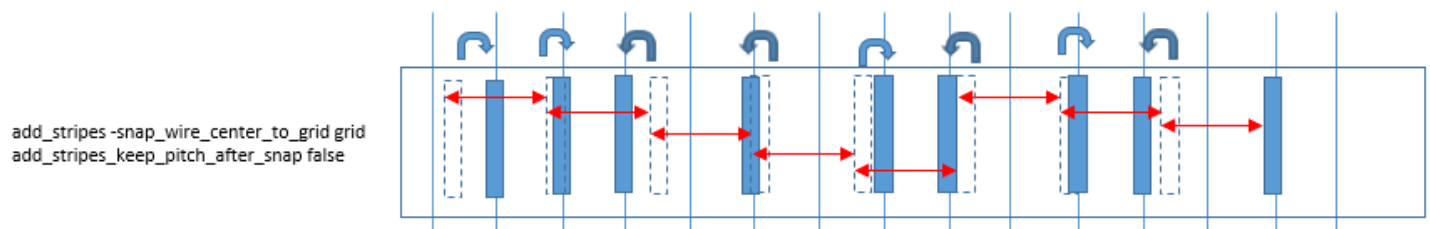
Scenario 1:

When the `add_stripes -snap_wire_center_to_grid` parameter is set to `none`, the `add_stripes_keep_pitch_after_snap` attribute does not work. See the example below for understanding:



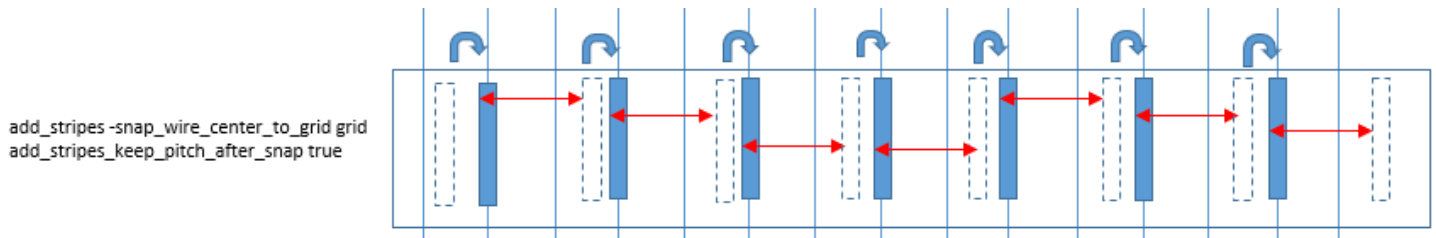
Scenario 2:

When the `add_stripes -snap_wire_center_to_grid` parameter is set to `grid`, and the `add_stripes_keep_pitch_after_snap` attribute is set to `false`, all the locations for snapping are decided based on the start location. Post then, the stripes are snapped to the nearest track. The average distance between of newly generated stripes is approximately equal to the specified pitch (`add_stripes -set_to_set_distance` parameter value). See the example below for understanding:



Scenario 3:

When the `add_stripes -snap_wire_center_to_grid` parameter is set to `grid`, and the `add_stripes_keep_pitch_after_snap` attribute is set to `true`, the location for snapping is decided based on the previous location. Post then, the stripes are snapped to the next track. See the example below for understanding:



Note:

- The first stripe of the design always snaps to the next track to meet the `add_stripes -start_offset` value. To start the first stripe snapping at the nearest track, set the `add_stripes -allow_snapping_override_custom_spacing` parameter to 1.
- By default, the `add_stripes_keep_pitch_after_snap` attribute is turned on only at the N10 nodes.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

add_stripes_mask_color_balance

`add_stripes_mask_color_balance {same_color | alternate_color | none}`

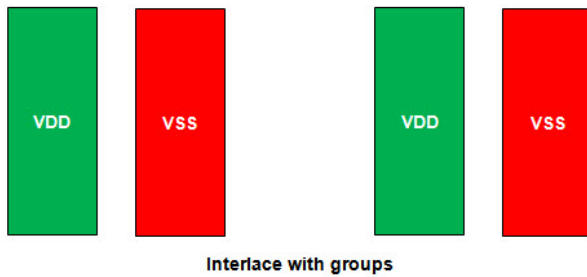
Default: none

Data_type: enum {same_color alternate_color none}, read/write

Specifies addition of VDD/VSS stripes with balanced color.

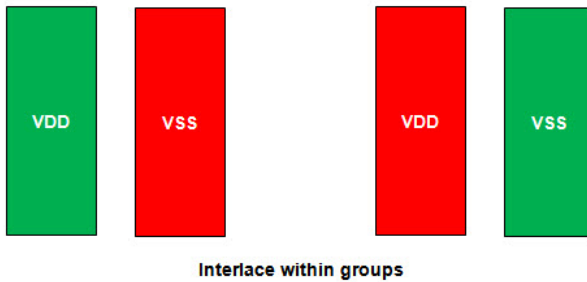
`same_color`

Corresponds to interlace with groups.



`alternate_color`

Corresponds to interlace within the group.



To achieve color balancing, the color of stripes occupying even number tracks is picked alternatively.

Note: The stripes location will always be either on full grid/track or half grid/track:

- If the stripes occupy odd number of tracks, they must be on full grid.
- If the stripes occupy even number of tracks, they must be on half grid.

To achieve color balance, the stripes color will be as mentioned below:

- If the stripes occupy odd number of tracks, the color is opposite to the center track color.
- If the stripes occupy even number of tracks, the color can be any of red or green and will keep alternating.

Only two nets are supported.

The attribute must be specified along with `add_stripes_width_file`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```


add_stripes_max_extension_distance

`add_stripes_max_extension_distance distance`

Default: 2.14748e+09

Data_type: double, read/write

Sets the maximum extension distance between stripes to nearby stripes in the same layer and the same direction.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_max_via_size

`add_stripes_max_via_size {shape width% height% target_penetration%}`

Default: ""

Data_type: string, read/write

The maximum size of a crossover via, specified as a percentage of a full-size via. You must specify three values for each shape that you specify.

The shape values are:

- Pad
via size between stripe and pad pin/ring
- Ring
via size between stripe and core ring
- Stripe
via size between stripes
- Blockring
via size between stripe and block ring
- Blockpin
via size between stripe and block pin, including ring pin

- Cover
via size between stripe and the macro which has CLASS COVER defined in LEF
- Selected
via size between stripe and selected objects
- No shape
via size between stripe and wires with shape None
- Standardcell
via size between stripe and standardcell pin

The three values you must specify for each shape are:

- The maximum width for the crossover via, as a percentage of a full-size via
- The maximum height for the crossover via, as a percentage of a full-size via
- The target penetration, which specifies a smaller size for vias connecting to targets of the specified shape

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_merge_with_all_layers

```
add_stripes_merge_with_all_layers {true | false}
```

Default: false

Data_type: bool, read/write

Merges a stripe to a block ring or pin on all layers. This attribute is similar to the `merge_stripes_value` attribute of the `add_stripes` command, except that it merges a stripe to all layers instead of only the stripe layer.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_mesh_via

```
add_stripes_mesh_via {true | false}
```

Default: false

Data_type: bool, read/write

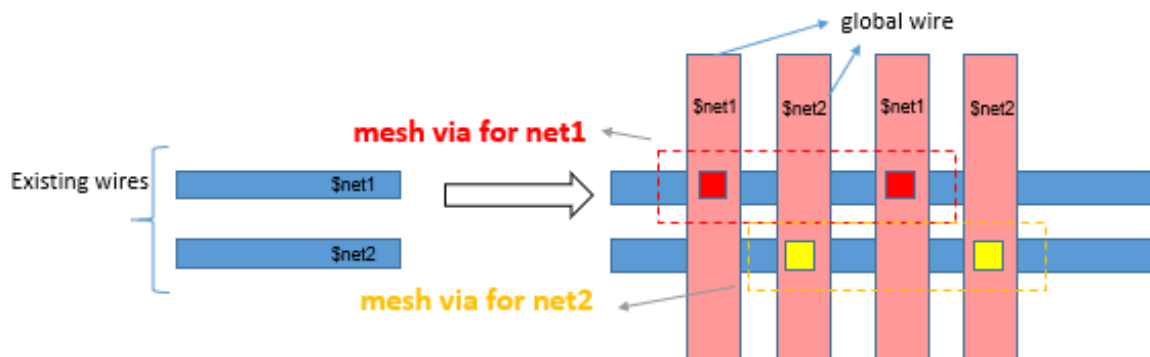
Enables to use mesh via pillars to reduce memory.

Example 1:

The following commands use mesh via pillars with the help of the `-mesh_via` parameter:

```
set_db add_stripes_mesh_via 1
add_stripes -nets {$net1 $net2 $net1 $net2} ...
```

Following is the output:

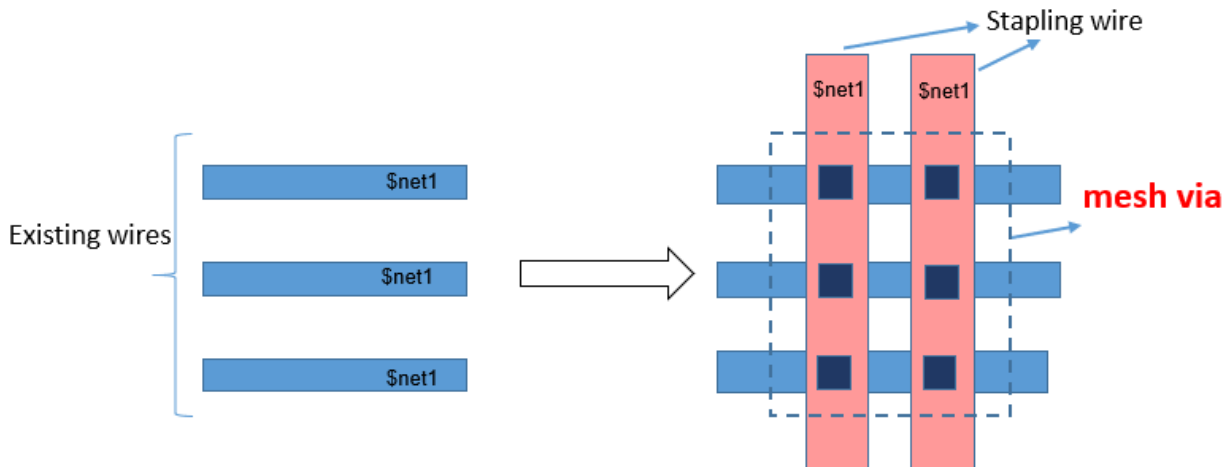


Example 2:

The following commands generate mesh via automatically in one stapling stripe:

```
set_db add_stripes_mesh_via 1
add_stripes -stapling{ ... } -nets {$net1 ...$net1...} ...
```

Following is the output:



This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

add_stripes_offset_from_core

`add_stripes_offset_from_core {true | false}`

Default: true

Data_type: bool, read/write

If set to `true`, the first or last stripe is offset from the core area of the design. If set to `0`, the first or last stripe is offset from core ring, pad ring, or design boundary, depending on the attributes specified with the `add_stripes` command.

Note: This attribute does not affect the offset if you specify the `add_stripes` command with any of the following attributes:

- `-over_power_domain 1`
- `-extend_to all_domains`
- `-area`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_opt_stripe_for_routing_track

```
add_stripes_opt_stripe_for_routing_track {shift | none}
```

Default: none

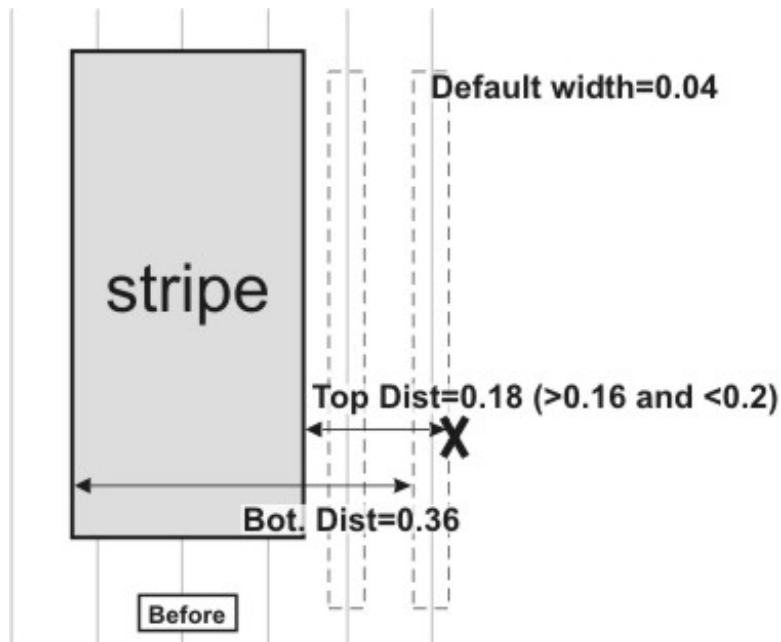
Data_type: enum* {shift none}, read/write

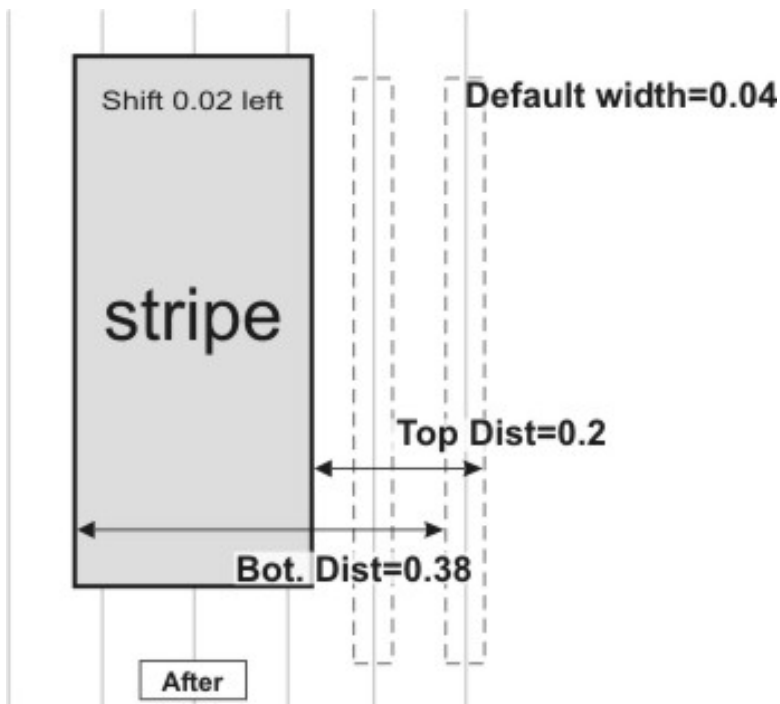
Specifies the location setting to move stripe center line to avoid the forbidden spacing. If both sides cannot be fixed at the same time, one side is always fixed.

When set to this attribute to the `shift` value, `add_stripes` automatically shifts stripes to preserve routing resource.

Example

For the rule below,
FORBIDDENSPACING 0.16 0.2 WIDTH 0.05 PRL 0;





After shifting the stripe 0.02 left, Top Dist. and Bot. Dist. are ≥ 0.2 , and one track is saved.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_orthogonal_offset

```
add_stripes_orthogonal_offset {[{edge1 orthogonal_offset1} {edge2 edge3  
orthogonal_offset2} {edge4 edge5 ... orthogonal_offset3 } ...]} [all orthogonal_offset]}
```

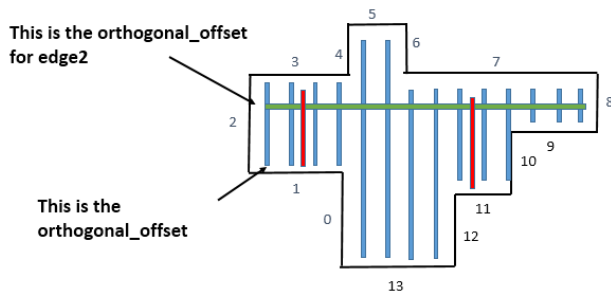
Default: {none}

Data_type: string, read/write

Defines `orthogonal_offset` of stripes for every edge of block level designs, blocks or standard cells for specifying the offsets for edges orthogonally. Edge number starts from the left bottom vertical edge. When this attribute is set for the second time, all the `orthogonal_offset` values are reset firstly.

Example 1:

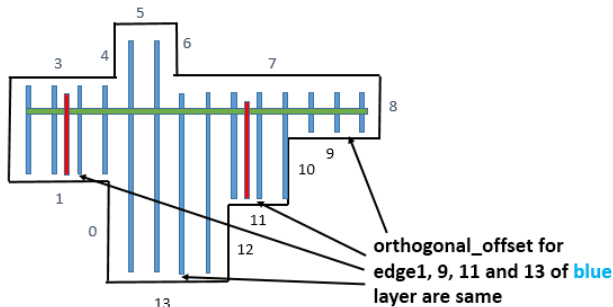
```
set_db add_stripes_orthogonal_offset {{1 orthogonal_offset1} {2 orthogonal_offset2} {3
orthogonal_offset3} {5 orthogonal_offset5} {7 orthogonal_offset7} {8
orthogonal_offset8} {9 orthogonal_offset9} {11 orthogonal_offset11} {13
orthogonal_offset13}}
add_stripes -layer green ...
add_stripes -layer blue ...
set_db add_stripes_orthogonal_offset {{1 orthogonal_offset1} {3 orthogonal_offset3} {7
orthogonal_offset7} {11 orthogonal_offset11}}
add_stripes -layer red ...
```



Example 2:

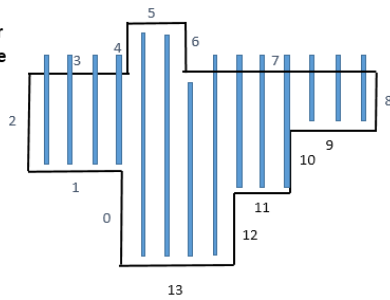
The following commands ensure that the stripes are generated in the block with a specified orthogonal offset:

```
set_db add_stripes_orthogonal_offset {{1 9 11 13 orthogonal_offset1} {2
orthogonal_offset2} {3 orthogonal_offset3} {5 orthogonal_offset5} {7
orthogonal_offset7} {8 orthogonal_offset8}}
add_stripes -layer green ...
add_stripes -layer blue ...
set_db add_stripes_orthogonal_offset {{1 orthogonal_offset1} {3 orthogonal_offset3} {7
orthogonal_offset7} {11 orthogonal_offset11}}
add_stripes -layer red ...
```



```
set_db add_stripes_orthogonal_offset {{3 7 orthogonal_offset3} {1 5 9 11 13
orthogonal1}}
add_stripes -layer blue ...
```

The orthogonal_offset for
edge3, edge7 are negative



This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_orthogonal_only

```
add_stripes_orthogonal_only {true | false}
```

Default: true

If you do not specify this attribute or if you specify it with a value of 1, the software creates vias for orthogonal wires and pin intersections.

Data_type: bool, read/write

Creates vias *only* for orthogonal wires and pin intersections.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_over_row_extension

```
add_stripes_over_row_extension {true | false}
```

Default: false

Data_type: bool, read/write

Extends the stripes to cover the followpin at the row end.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_partial_set_through_domain

```
add_stripes_partial_set_through_domain {true | false}
```

Default: false

Data_type: bool, read/write

Specifies whether to go over the power domain if domain contains the specified net.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_remove_floating_stapling

```
add_stripes_remove_floating_stapling {true | false}
```

Default: true

Data_type: bool, read/write

Removes those floating stapling stripes that are not connected to the power stripe/follow pin, when the stapling stripes are generated using a reference layer. This attribute does not remove floating stapling stripes when they are added through the user-specified offset and pitch.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_remove_floating_stripe_over_block

```
add_stripes_remove_floating_stripe_over_block {true | false}
```

Default: `true`

Data_type: bool, read/write

If set to `true`, stripe fragments that start and end inside the same block or ring macro are removed. If set to `0`, stripe fragments are not removed.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_remove_stripe_under_ring

```
add_stripes_remove_stripe_under_ring {true | false}
```

Default: `false`

Data_type: bool, read/write

Removes stripe segments, which have overlapped area and are in the same direction as the existing ring. The stripe can be in a different layer to the existing ring.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_respect_routes

add_stripes_respect_routes {none | fixed | fixed_and_clock | all}

Default: fixed_and_clock

Data_type: enum {none | fixed fixed_and_clock all}, read/write

Specifies the route types to ignore DRC check during stripe generation. This attribute always honor all covers and PGs, but does not honor the trail routes, Following are the attribute values:

- **none:** Specifies that no route types are respected.
- **fixed:** Respects fixed regular wires.
- **fixed_and_clock:** Respects fixed and routed clocks.
- **all:** Respects fixed and routed clocks along with the routed signals.

The following table presents the support of the `add_stripes_respect_routes` attribute values with various statuses of net:

Target PG Honoring a Wire		add_stripes_respect_routes Values			
Net Use	Status	none	fixed	fixed_and_clock	all
P/G		Yes	Yes	Yes	Yes
Non-P/G	Cover	Yes	Yes	Yes	Yes
	Fixed	No	Yes	Yes	Yes
	Routed	Clock	No	Yes	Yes
	Signal				
	Unknown	No	No	No	No

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

add_stripes_route_over_rows_only

```
add_stripes_route_over_rows_only {true | false}
```

Default: false

Data_type: bool, read/write

If set to `true`, route stripes only over standard cells rows. If set to `0`, stripes are not limited to being routed over rows within this boundary.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_rows_without_stripes_only

```
add_stripes_rows_without_stripes_only {true | false}
```

Default: false

Data_type: bool, read/write

If set to `true`, global stripes are generated as specified by other attributes. Then stripes are also generated over row clusters that do not already contain stripes. If set to `false`, stripes are generated as specified by other attributes and not over row clusters without stripes.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_same_size_stack_vias

```
add_stripes_same_size_stack_vias {true | false}
```

Default: false

Data_type: bool, read/write

If set to `true`, the software always generates same-sized stacked vias. If set to `false`, the software trims the vias as needed to meet the DRC rules.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_skip_via_on_pin

```
add_stripes_skip_via_on_pin {[Pad] [Block] [Cover] [Standardcell] [Physicalpin]}
```

Default: Standardcell

Data_type: enum* {pad block cover standardcell physicalpin}, read/write

Prevents vias from being generated on the specified types of pins.

If you specify `-skip_via_on_pin {}`, the power planning software makes connections to all five categories of pins. If you do not specify this attribute, vias are generated for block pins, pad pins, and cover macro pins, but are not generated for standard cell pins.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_skip_via_on_wire_shape

```
add_stripes_skip_via_on_wire_shape { [Blockring] [Stripe] [Padring] [Ring] [Noshape] }
```

Default: noshape

If you do not specify this attribute, vias can be generated on all wire shapes *except* no-shape wires.

Data_type: enum* {blockring stripe followpin corewire blockwire iowire padring ring fillwire noshape}, read/write

Prevents vias from being generated on the specified wire shapes.

If you specify `-skip_via_on_wire_shape {}`, the power planning software makes connections to all five categories of wire shapes.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_spacing_from_block

`add_stripes_spacing_from_block float`

Default: 0.0

Data_type: double, read/write

If the value is less than 0, over-the-block stripes end the specified distance from the boundary.

If the value is greater than 0, global stripes break at the specified user units from the obstructed blocks.

If the value is equal to 0, the stripe offset is 0.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_spacing_type

`add_stripes_spacing_type {enum {edge_to_edge | center_to_center}}`

Default: `edge_to_edge`

Data_type: `enum {edge_to_edge center_to_center}`, read/write

Specifies the edge-to-edge spacing between stripes in each set.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_split_long_via

`add_stripes_split_long_via { threshold_value step_value offset_value height_value }`

Default: `0 0 -1 -1`

If you do not specify this attribute, the software does not split vias.

Data_type: string, read/write

Splits vias longer than the specified length (`threshold_value`, in micrometers) into smaller vias with specified center-to-center spacing (`step_value`, in micrometers), left/bottom end offset (`offset_value`, in micrometers) and vertical/horizontal height (`height_value`, in micrometers) values . This is independent of the `add_stripes_split_vias` attribute.

Note: If `offset_value` is specified as negative, the vias will be placed symmetrically on both ends. The default value of `offset_value` is `-1` .

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_split_vias

```
add_stripes_split_vias {true | false}
```

Default: false

Data_type: bool, read/write

Specifies whether two or more partial vias can be created at the crossover between a stripe and any target (pin or wire) if an obstruction (macro obstruction; wire or pin on a different net) prevents the creation of a full-size via. This can be useful, for example, if several metal blockages partially obstruct the area where a full-size via would normally be placed. This attribute permits the creation of partial vias that would not violate the process rules, in the open areas. A value of `1` allows multiple partial vias to be created. A value of `0` prevents multiple partial vias from being created.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_split_wire_spacing

```
add_stripes_split_wire_spacing floating_point_value
```

Default: 0.0

If you omit this attribute, or set its value to `0`, the software uses the spacing rule in the LEF file.

Data_type: double, read/write

Specifies the spacing between each split wire.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```


add_stripes_split_wire_weight

`add_stripes_split_wire_weight integer_0_to_10`

Default: 0.0

If you omit this attribute, or set its value to 0, no I/O wires are deleted when the above connections are made.

Data_type: double, read/write

When split wires from a stripe on one layer attempt to make a target connection on another layer but are obstructed by I/O wires, some I/O wires might be deleted so that approximately n of 10 split stripe wires complete the target connection.

Note: When you specify this option, the software deletes the fewest I/O wires to accommodate approximately n of 10 split stripe wires.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_split_wire_width

`add_stripes_split_wire_width floating_point_value`

Default: 0.0

If you omit this attribute, or set its value to 0, the software uses the maximum width value in the LEF file.

Data_type: double, read/write

Specifies the width of each split wire.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_stacked_via_bottom_layer

```
add_stripes_stacked_via_bottom_layer layername
```

Default: `bottomLayer`

The lowest metal layer in the design.

Data_type: string, read/write

Specifies the lowest layer in which vias can be stacked.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_stacked_via_top_layer

```
add_stripes_stacked_via_top_layer layername
```

Default: `topLayer`

The highest metal layer in the design.

Data_type: string, read/write

Specifies the highest layer in which vias can be stacked.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_stapling_extend_to_minimum_spacing

`add_stripes_stapling_extend_to_minimum_spacing {true | false}`

Default: false

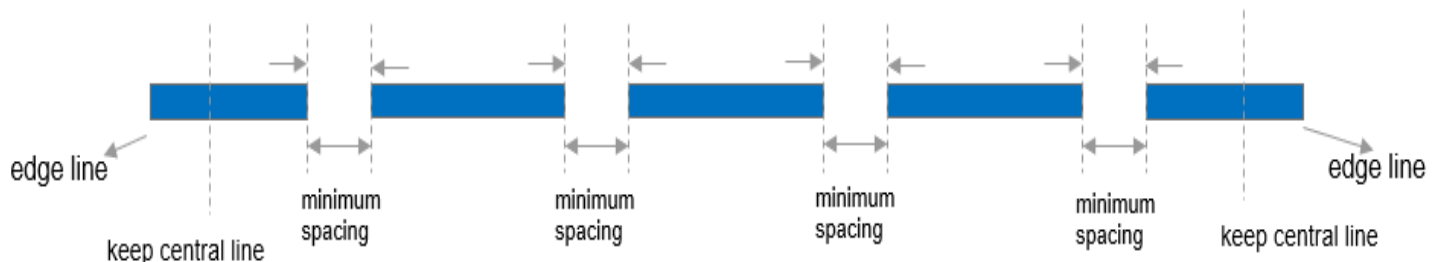
Data_type: bool, read/write

Automatically extends the newly created adjacent stapling to satisfy the end-to-end minimum spacing requirement, which will also be free from DRCs. This attribute works only with the stapling wires.

When set to `false`, the length of the newly added stripe follows the length defined through the `add_stripes -stapling` parameter. The following figure presents the placing of stripes when this attribute is set to `false`:



When set to `true`, the newly added stripes are extended automatically. The wire on the edge with only one end will have end-to-end spacing with the newly generated stripes. Only the end-to-end side is extended and the edge line end is not extended. The extended stripe still remains within the die area. The following figure presents the placing of the extended stripes when this attribute is set to `true`:



This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_stapling_nets_style

`add_stripes_stapling_nets_style {end_to_end | side_to_side | side_to_side_full_nets}`

Default: end_to_end

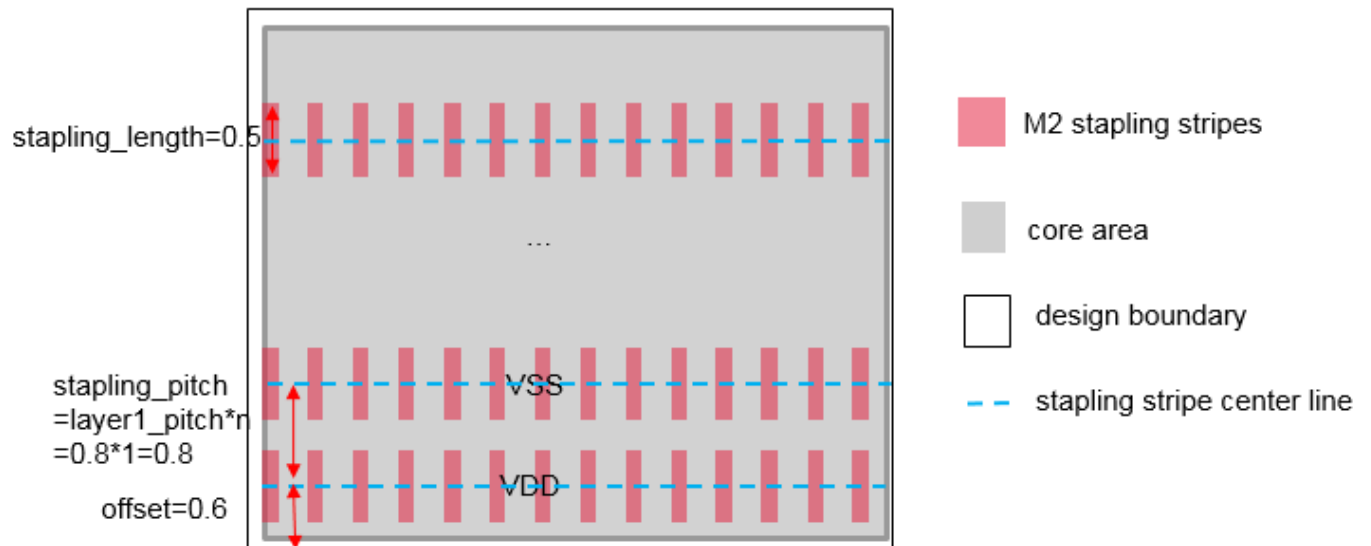
Data_type: enum {end_to_end side_to_side side_to_side_full_nets}, read/write

Specifies the stapling style, if multiple nets are used. Please note that:

- When `end_to_end` (default value) is specified, nets are arranged in end direction. Refer to the following examples:

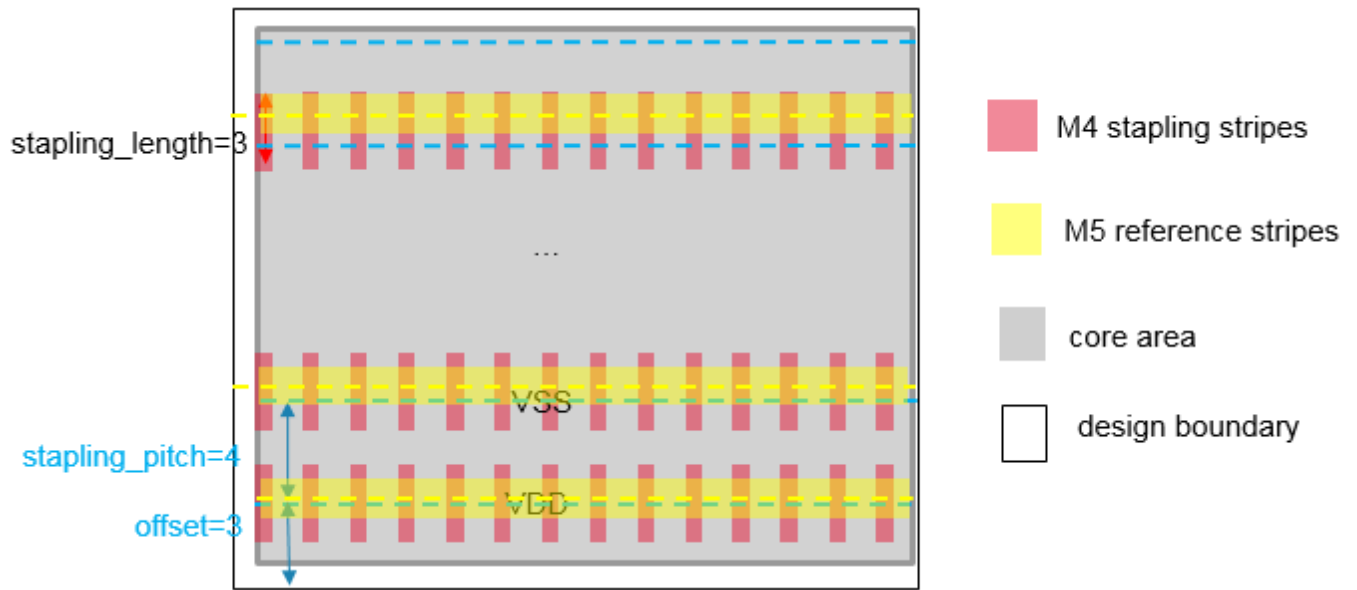
Example 1

```
add_stripes -stapling {0.5 0.6 0.8:1} -nets {VDD VSS} -layer metal2 -width 0.1 -
direction vertical -set_to_set_distance 0.5
```



Example 2

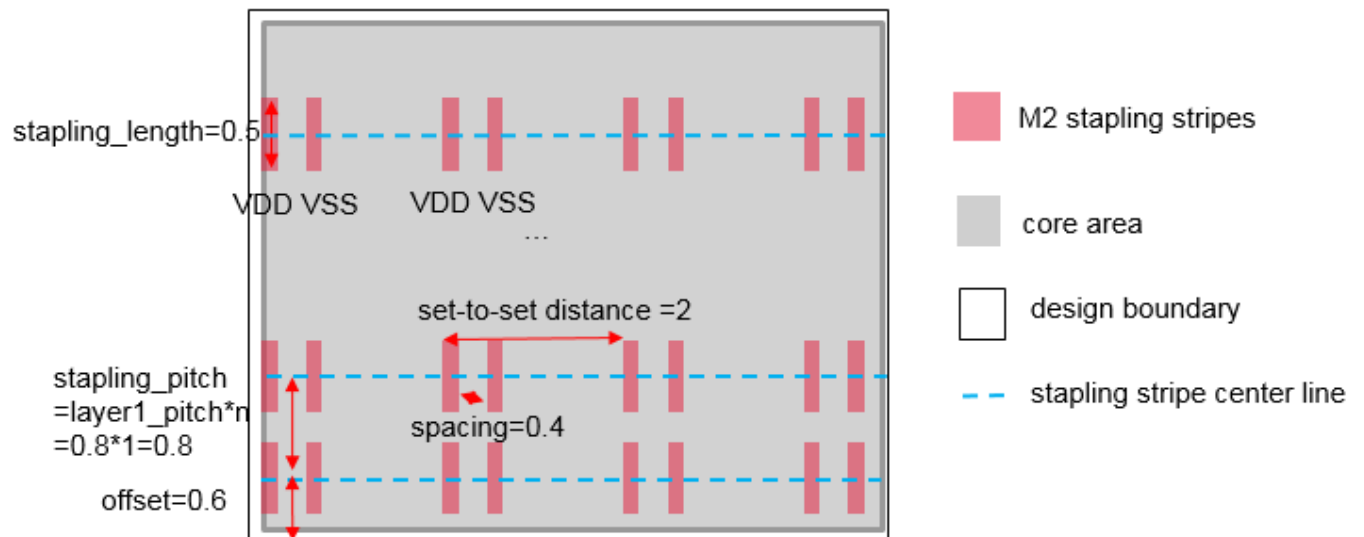
```
set metal3_track_offset 3
set metal3_pitch 0.8
add_stripes -stapling '3 $metal3_track_offset $metal3_pitch:5 metal5' -nets {VDD
VSS} -layer metal4 -width 0.5 -direction vertical -set_to_set_distance 2
```



- When `side_to_side` is specified, nets are arranged in the side direction. Refer to the following example:

Example 3

```
add_stripes -stapling {0.5 0.6 0.8:1} -nets {VDD VSS} -spacing 0.4 -layer metal2 -
width 0.1 -direction vertical -set_to_set_distance 2
```



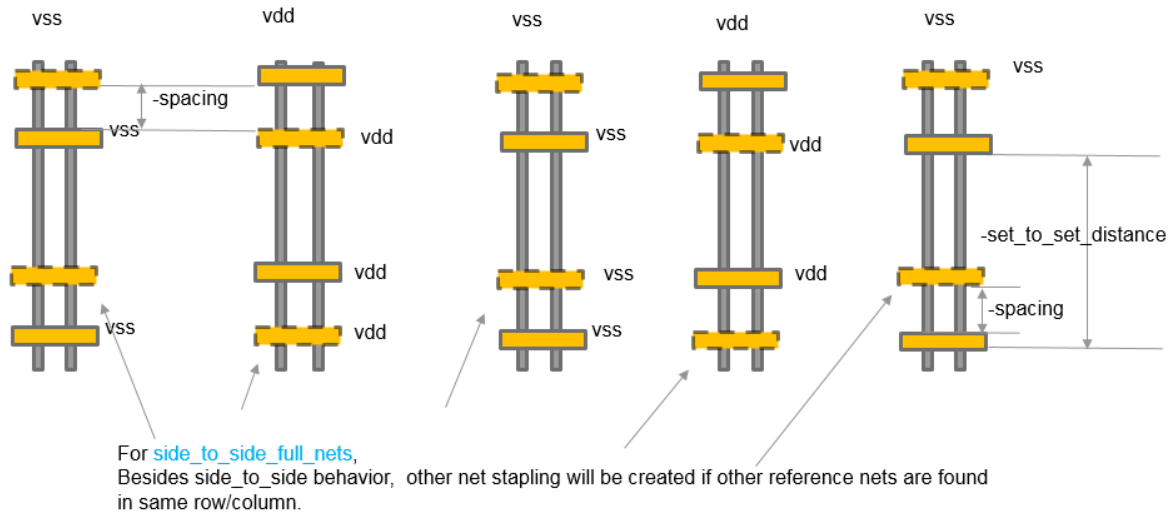
- When `side_to_side_full_nets` is specified, nets are arranged in the side direction. Besides the `side_to_side` behavior, it creates other net stapling according to the reference nets. The `side_to_side_full_nets` value can be generated only over the existing reference stripe

(with `-stapling {length refer_layer}`). Refer to the following example:

Example 4

`-stapling_nets_style side_to_side_full_nets` #Besides `side_to_side` behavior, creating other net stapling according to the reference nets

`-stapling {length refer_layer}` #Stapling stripe is generated over existing reference stripe



This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

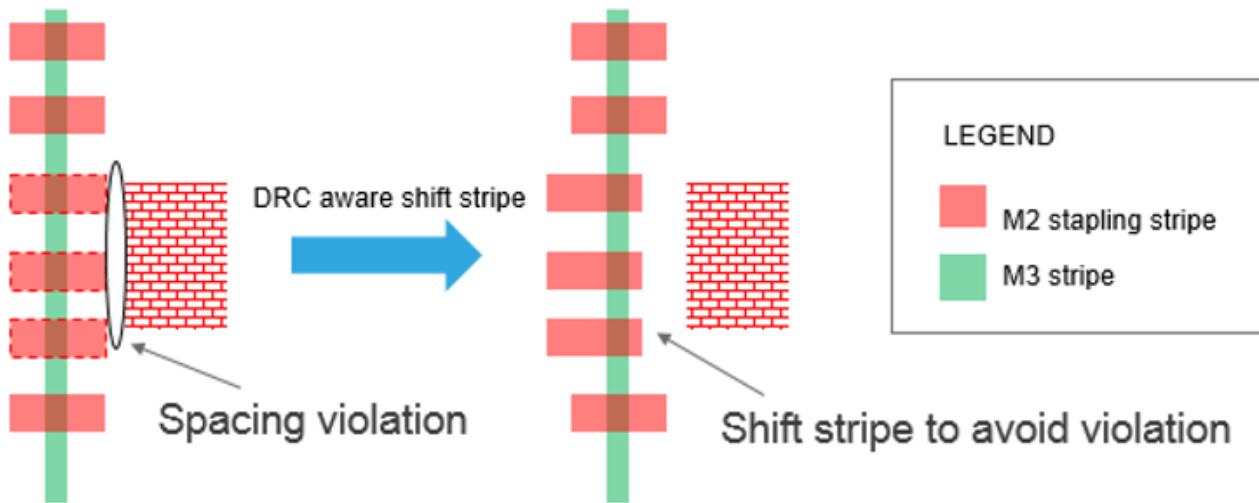
add_stripes_stapling_shift

`add_stripes_stapling_shift {true | false}`

Default: true

Data_type: bool, read/write

Shifts the stapling stripes automatically to avoid the DRC. See the example below:



This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_stop_at_closest_target

```
add_stripes_stop_at_closest_target { none | block_ring | core_ring | stripe }
```

Default: none

Data_type: enum* {none block_ring core_ring stripe}, read/write

Stops the stripe at closest target while drawing the stripe on the master cell.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_stop_at_last_wire_for_area

```
add_stripes_stop_at_last_wire_for_area {true | false}
```

Default: `false`

Data_type: `bool`, read/write

If set to `false`, the stripe antennas are trimmed back to the closest wire or pin of the same net. If set to `true`, stripe antennas are not trimmed.

Note: This option is only available if you specify the `-area` parameter of the `add_stripes` command.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_stripe_min_length

```
add_stripes_stripe_min_length {floating_point_value | stripe_width}
```

Default: `stripe_width`

Data_type: `string`, read/write

Specifies the minimum stripe length. `stripe_width` is a fixed string. The fixed string here refers to the width of present wire.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_stripe_min_width

```
add_stripes_stripe_min_width floating_point_value
```

Default: `0.0`

Data_type: `double`, read/write

Specifies the minimum width of the remaining stripe after being trimmed.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_switch_cell_name

```
add_stripes_switch_cell_name { cellA cellB...}
```

Default: none

Data_type: string, read/write

Treats the specified switch cell as a ring. Pins in the switch cell can then be used as targets for stripes. If the switch cell is in the core area, the software treats it as a block ring. If the switch cell is outside of the floorplan row, the cell is treated as a core ring, and used as the stripe boundary.

You can specify one or more switch cell names.

To turn off this functionality, specify one of the following:

- `-switch_cellname ""`
- `-switch_cellname none`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_switch_layer_overlap_length

```
add_stripes_switch_layer_overlap_length floating_point_value
```

Default: 0.0

If you omit this attribute, or set its value to 0, the power planning software creates a square via.

Data_type: double, read/write

Controls the overlap distance used for the via between a stripe layer and the layer being switched to.

Note: The overlap applies *only* to vias created for straight (non-jogging) connections.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_trim_antenna_back_to_shape

```
add_stripes_trim_antenna_back_to_shape {none | block_ring | core_ring | pad_ring | stripe}
```

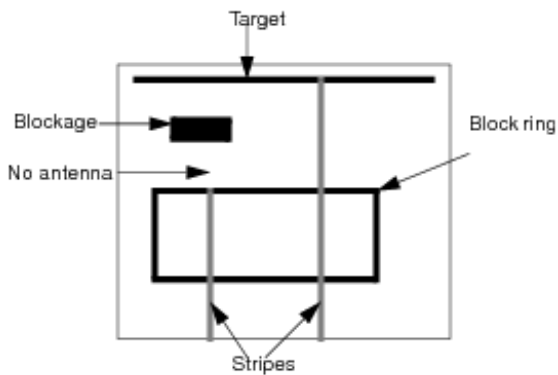
Default: none

Data_type: enum* {none block_ring core_ring pad_ring stripe}, read/write

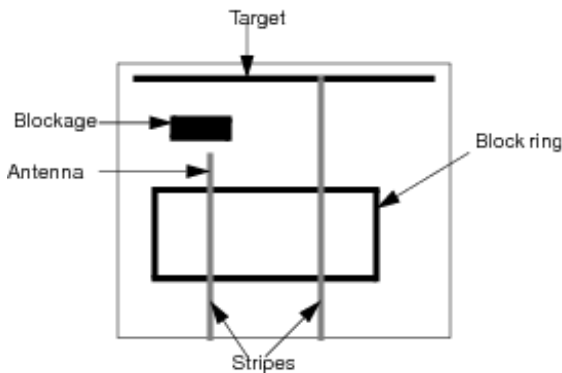
Prevents stripes from creating antennas when a wire of the specified shape is the last target before a blockage that prevents connection to another target. Specify one of the following shapes:

- block_ring
- core_ring
- pad_ring
- stripe

The following figure illustrates the software behavior if you specify this attribute with a value of `block_ring`.



Stripes extend to the blockage, leaving an antenna between the last connection and the blockage. The following figure illustrates the software behavior if you do not specify this attribute.



This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```
- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_strikes_trim_antenna_max_distance

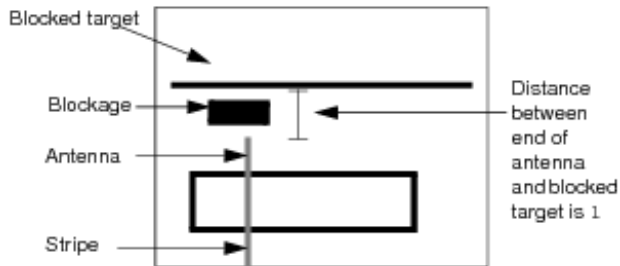
`add_strikes_trim_antenna_max_distance distance`

Default: 0.0

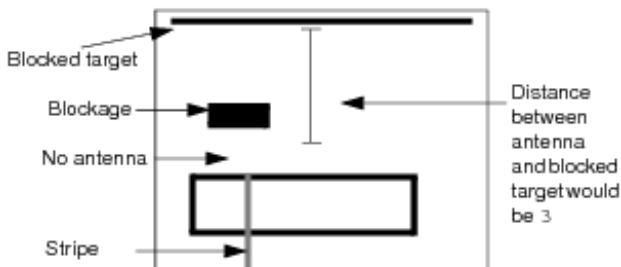
Data_type: double, read/write

Use this option to specify a distance, in micrometers, between the end of the antenna and a blocked target. This value controls whether an antenna is kept or trimmed for the selected shape. Cadence recommends that you specify a value equivalent to the distance between the core area and the pad ring or core ring.

- If the blocked target is within the specified distance, the antenna is kept, and you can use `route_special` to complete the connection. For example, if you specify a distance of 2 micrometers and the distance between the end of the antenna and the blocked target is 1 micrometer, the antenna is not trimmed.



- If the blocked target is beyond the specified distance, the antenna is trimmed back to the last connection. For example, if you specify a distance of 2 micrometers and the distance between the antenna and the blocked target would be 3 micrometers, the antenna is trimmed back.



This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
- To get the current value of any root attribute, use the `get_db` command:

```
set_db attribute_name value
```

```
get_db attribute_name
```

add_stripes_trim_stripe

```
add_stripes_trim_stripe { none | design_boundary | core_boundary }
```

Default: none

Data_type: enum {none design_boundary core_boundary}, read/write

Specifies if shape is trimmed by the specified shape.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_use_exact_spacing

```
add_stripes_use_exact_spacing {true | false}
```

Default: false

Data_type: bool, read/write

If set to `true`, the power stripe breaks using the exact spacing rule from the obstructing blockage, pin, or wire to avoid a design rule violation. If set to `0`, the power stripe breaks at 1.5 times the maximum range rule spacing of all layers.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_use_point_to_point_route

```
add_stripes_use_point_to_point_route {true | false}
```

Default: false

Data_type: bool, read/write

If set to `false`, the point-to-point router engine is turned off. If set to `true`, the `add_stripes` command automatically calls the internal point-to-point router engine to route the difficult power stripe connections (such as connecting power stripe antennas that were not connected by the `add_stripes` command).

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_use_stripe_width

```
add_stripes_use_stripe_width {true | false}
```

Default: true

Data_type: bool, read/write

If set to `true`, stripes that jog to connect to a ring always use the stripe width for the jog. If set to `false`, the jog is either the width of the stripe or the width of the ring segment, whichever is smaller.

Note: This attribute only works in conjunction with the `-jog_to_connect 0` parameter of the `add_stripes` command.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_via_using_exact_crossover_size

```
add_stripes_via_using_exact_crossover_size {true | false}
```

Default: true

Data_type: bool, read/write

Specifies whether to generate partial vias of the exact crossover size. A value of `1` allows partial vias to be generated. A value of `0` prevents partial vias, and the software always generates full-size vias.

Note: This option does not apply when both of the objects at the crossover are rings or stripes, or at the crossover of a ring and stripe. In these cases, the software always generates a full-size via array.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_stripes_width_file

```
add_stripes_width_file file_name
```

Default: ""

Data_type: string, read/write

The `file_name` specifies a list of recommended width values for the stripes.

Note: This option must be specified along with `-color_balance`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_target_pg

The root attributes in the `add_target_pg` category are:

- `add_target_pg_allow_weak_connect`
- `add_target_pg_max_extension_distance`
- `add_target_pg_pins`
- `add_target_pg_pins_group_distance`
- `add_target_pg_respect_routes`

- [add_target_pg_share_resource](#)

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `add_target_pg` category, use the following command:

```
get_db -category add_target_pg
```

add_target_pg_allow_weak_connect

```
add_target_pg_respect_clock_routes {true | false}
```

Default: false

Data_type: bool, read/write

When set to `false`, the number of wires created on every stack layer need to satisfy the defined count with DRC clean. Else, the PG pin is left open. When set to `true`, the count lesser than defined can also be accepted while creating the total count causing DRC.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_target_pg_max_extension_distance

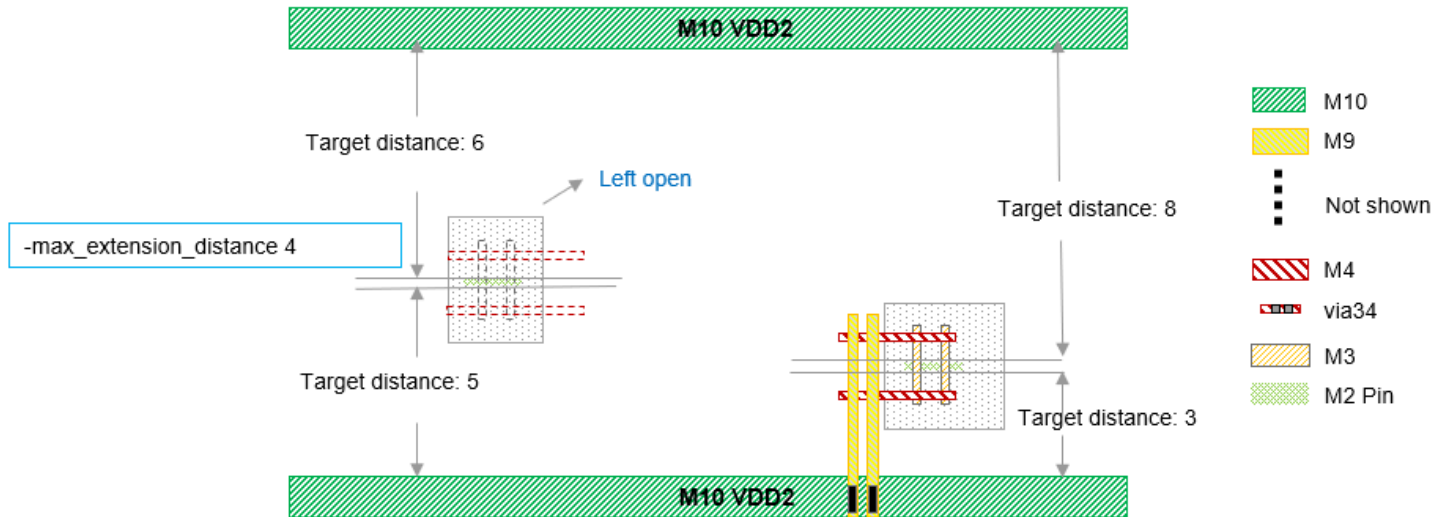
```
add_target_pg_max_extension_distance value
```

Default: 2.14748e+09

Data_type: double, read/write

Specifies the maximum distance that the target PG pin can extend to the target layer. If the distance between the target PG pin and the candidate target layer is lesser than or equal to the specified distance, the candidate target is treated as the real target.

If the target PG wire cannot be found within `add_target_pg_max_extension_distance`, the instance is left open. If this attribute is not used, the software connects the stripe layer to the closest target PG wire with DRC clean, by default. See the figure below for understanding:



This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

add_target_pg_pins

`add_target_pg_pins {cell1:pin1 cell1:pin2 cell2:pin1 ...}`

Default: ""

Data_type: string, read/write

Specifies the cell pins to be connected.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

add_target_pg_pins_group_distance

add_target_pg_pins_group_distance value

Default: auto

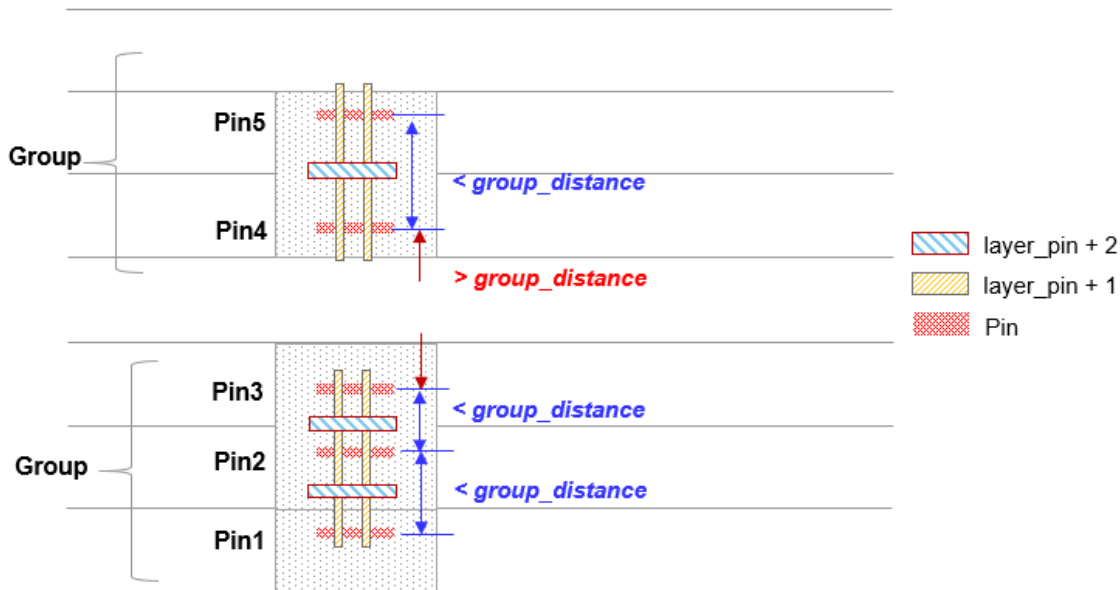
Data_type: string, read/write

Specifies the side-to-side distance of the pins to be treated as a group while connecting the target PG pins. The pins belonging to the same group share the same target PG stripes connection.

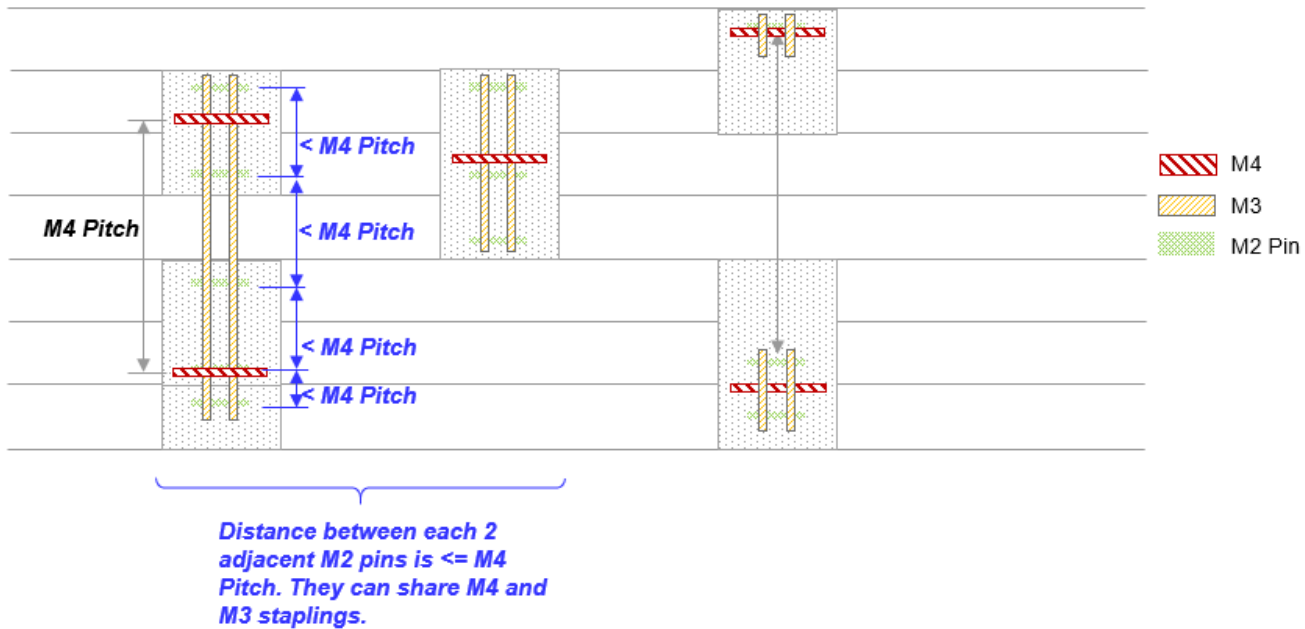
When this attribute is set to its default value, *auto*, the `add_target_pg` command uses the default behavior of target PG generation to group the pins. If this attribute is not specified, the software uses the pitch of metal layer above the pins layer with same direction (*layer_pin+2*) as the group distance.

If the pitch of metal layer above the pins layer with the same direction (*layer_pin+2*) is not defined, the pins will be connected separately.

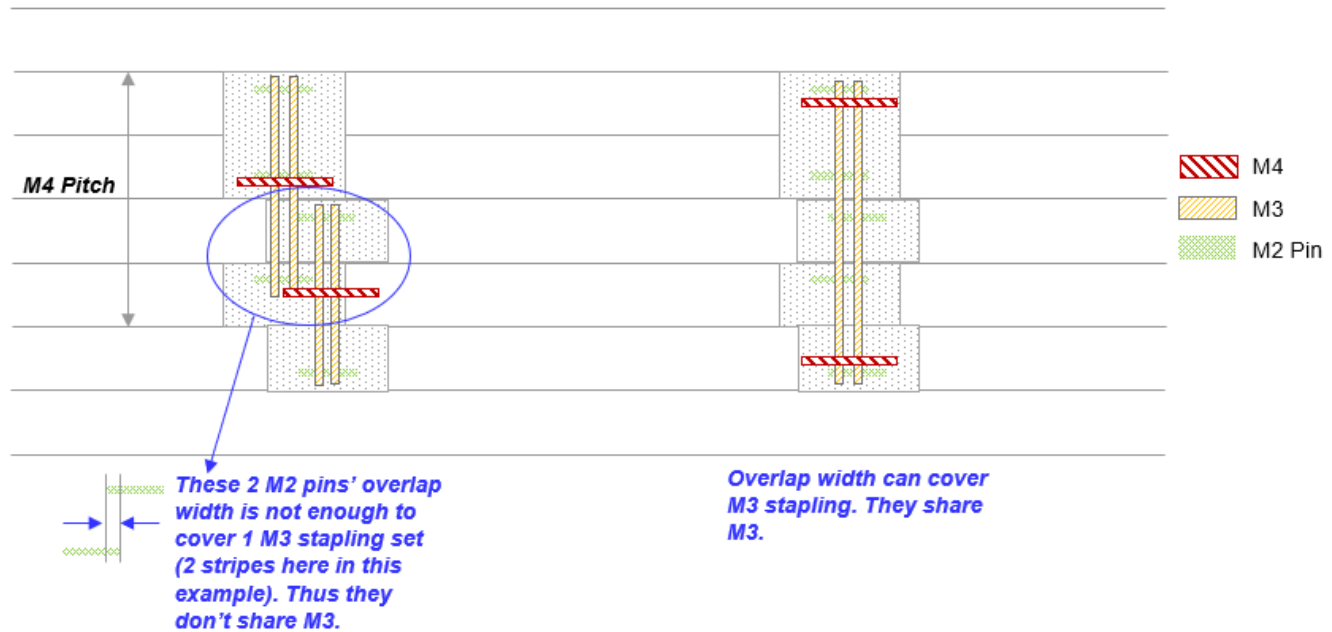
If the distance between two pins is lesser than *group_distance*, they will be treated as one group. See the figure below for understanding:



When *pins_group_distance* is not defined, the software uses the pitch of the metal layer above the pins layer with the same direction (*layer_pin+2*) as the group distance. See the figure below for understanding:



The following figure presents how the target PG pin's horizontal offset affects the stapling sharing:



This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

add_target_pg_respect_routes

`add_target_pg_respect_routes {none | fixed | fixed_and_clock | all}`

Default: `fixed_and_clock`

Data_type: enum {none | fixed fixed_and_clock all}, read/write

Specifies the route types to respect for DRC check during target PG generation. This attribute always honor all covers and PGs, but does not honor the trail routes, Following are the attribute values:

- `none`: Specifies that no route types are respected.
- `fixed`: Respects fixed regular wires.
- `fixed_and_clock`: Respects fixed and routed clocks.
- `all`: Respects fixed and routed clocks along with the routed signals.

The following table presents the support of the `add_target_pg_respect_routes` attribute values with various statuses of net:

Target PG Honoring a Wire		add_target_pg_respect_routes Values			
Net Use	Status	none	fixed	fixed_and_clock	all
P/G		Yes	Yes	Yes	Yes
Non-P/G	Cover	Yes	Yes	Yes	Yes
	Fixed	No	Yes	Yes	Yes
	Routed	Clock	No	Yes	Yes
	Signal	No	No	No	Yes
Unknown		No	No	No	No

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_target_pg_share_resource

```
add_target_pg_share_resource {true | false}
```

Default: true

Data_type: bool, read/write

When set to `true`, the newly added PGs of the intermediate stacked layer can connect to the existing PG wire of the same layers and are preferred to be shifted to the track of the existing PG to save the track resource. If there are DRCs between the added PG and the existing PG after shifting, the added PG will follow the behavior explained below:

- If the added wire is on different net from the existing PG, it will be shifted again to avoid DRC.
- If the added wire is on the same net of the existing PG, it will be merged with the existing wire.

When set to `false`, the stacked layer cannot connect to the existing intermediate layers between the cell PG pin and target PG layer. Additionally, the newly added PG will be shifted if its original location causes DRC.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

add_tieoffs

The root attributes in the `add_tieoffs` category are:

- [add_tieoffs_cells](#)
- [add_tieoffs_create_hports](#)
- [add_tieoffs_honor_dont_touch](#)
- [add_tieoffs_honor_dont_use](#)
- [add_tieoffs_max_distance](#)

- [add_tieoffs_max_fanout](#)
- [add_tieoffs_module_prevention](#)
- [add_tieoffs_prefix](#)
- [add_tieoffs_report_hports](#)

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `add_tieoffs` category, use the following command:

```
get_db -category add_tieoffs
```

add_tieoffs_cells

```
add_tieoffs_cells {{tieHi1 tieLo1} {tieHi2 tieLo2} ...}
```

Default: ""

Data_type: string

Specifies the pairs of tie cells to be used by the `add_tieoffs` command. You can specify a maximum of two tie-cells in a cell list, where one cell must be a tie-high driver, and the other a tie-low driver.

A single cell with both tie connections, high and low, can be specified as below.

```
add_tieoffs_cells {{TieHiLo1} {TieHiLo2} {TieHiLo3} ....}
```

The software selects the nearest cell pair to make new tie connections during the `post_mask` flow.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Command

- `add_tieoffs`

add_tieoffs_create_hports

`add_tieoffs_create_hports {true | false}`

Default: `false`

Data_type: bool, read/write

Allows the added tie cells to connect to tie pins across hierarchical boundaries if other constraints, such as maximum fanout and maximum distance, allow it.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Command

- `add_tieoffs`

add_tieoffs_honor_dont_touch

`add_tieoffs_honor_dont_touch {true | false}`

Default: `false`

Data_type: bool, read/write

Allows both `add_tieoffs` and `add_tieoffs` commands to honor the `dont_touch` attribute set on instances, nets, cells, and modules.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `add_tieoffs`
- `delete_tieoffs`

add_tieoffs_honor_dont_use

```
add_tieoffs_honor_dont_use {true | false}
```

Default: false

Data_type: bool, read/write

Honors `dont_use` attribute on cells.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```
- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `add_tieoffs`

add_tieoffs_max_distance

```
add_tieoffs_max_distance distanceValue
```

Default: 0.0

Data_type: double, read/write

Specifies the maximum Manhattan distance, in microns, between the tie-cell driver and the tie-pins. A value of 0 implies no distance constraint.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `add_tieoffs`
- `delete_tieoffs`

add_tieoffs_max_fanout

```
add_tieoffs_max_fanout fanOutValue
```

Default: 0

Data_type: int, read/write

Specifies the number of tie-pins a tie-net can drive. A value of 0 implies no fanout constraint.

Example

The following set of commands set `TIEHI` and `TIELO` cells a maximum distance of 20 microns between the tie-cell pin and the tie-off input pins and a tie-net can drive up to 10 tie-pins:

```
set_db add_tieoffs_max_fanout 10
set_db add_tieoffs_max_distance 20
set_db add_tieoffs_cells "TIEHI TIELO"
```

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `add_tieoffs`
- `delete_tieoffs`

add_tieoffs_module_prevention

`add_tieoffs_module_prevention {true | false}`

Default: false

Data_type: bool, read/write

When this attribute is set to `true`, the `add_tie_offs` command adds tie-cells for module instantiation port. Tie-cells may also be added inside the module to tie the output port of the module so that module function can be prevented.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `add_tieoffs`
- `delete_tieoffs`

add_tieoffs_prefix

`add_tieoffs_prefix prefixName`

Default: ""

Data_type: string, read/write

Specifies a prefix for the added tie-cell instances.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `add_tieoffs`
- `delete_tieoffs`

add_tieoffs_report_hports

```
add_tieoffs_report_hports {true | false}
```

Default: `false`

Data_type: `bool`, read/write

Report created ports when `add_tieoffs_create_hports` is set. Report is saved to a file named `tiehilo.rpt`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `add_tieoffs`
- `delete_tieoffs`

add_well_taps

The root attributes in the add_well_taps category are:

- [add_well_taps_abut_boundary_tap_distance](#)
- [add_well_taps_antenna_tap_break_cell](#)
- [add_well_taps_antenna_tap_left_cell](#)
- [add_well_taps_antenna_tap_right_cell](#)
- [add_well_taps_antenna_tap_cell](#)
- [add_well_taps_antenna_tap_pitch](#)
- [add_well_taps_antenna_tap_right_top_edge_cell](#)
- [add_well_taps_avoid_abutment](#)
- [add_well_taps_avoid_vertical_well_abutment](#)
- [add_well_taps_block_boundary_only](#)
- [add_well_taps_bottom_tap_cell](#)
- [add_well_taps_bottom_termination_cell](#)
- [add_well_taps_cell](#)
- [add_well_taps_channel_offset](#)
- [add_well_taps_check_channel](#)
- [add_well_taps_column_cells](#)
- [add_well_taps_create_rows](#)
- [add_well_taps_disable_check_zone_at_boundary](#)
- [add_well_taps_in_row_offset](#)
- [add_well_taps_insert_cells](#)
- [add_well_taps_replace_intersection_cell](#)
- [add_well_taps_rule](#)
- [add_well_taps_site_offset](#)
- [add_well_taps_tap_function_cells](#)

- [add_well_taps_tap_termination_alignment](#)
- [add_well_taps_termination_align](#)
- [add_well_taps_termination_cells](#)
- [add_well_taps_top_tap_cell](#)
- [add_well_taps_vertical_boundary_spacing](#)
- [add_well_taps_top_termination_cell](#)
- [add_well_taps_well_cut_cells](#)

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `add_well_taps` category, use the following command:

```
get_db -category add_well_taps
```

add_well_taps_abut_boundary_tap_distance

```
add_well_taps_abut_boundary_tap_distance {{cell11 cell12 distance1} {cell11 cell13  
distance1} {cell12 cell13 distance3} ... }
```

Default: ""

Data_type: string, read/write

Specifies the boundary cells and their tap distance when they are vertically abutted. If you do not set this attribute, the abut cell still uses the single cell distance. If you specify it, the abut cell tap cover distance uses the defined new value. You need to add additional tap cell at the area when the tap cell checkboard cannot cover it.

This attribute works for the N4 process.

You can control the tap distance of a single cell using the `set_db`
`add_well_taps_tap_function_cells {cellnames rule value}` attribute.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`

- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

add_well_taps_antenna_tap_break_cell

`add_well_taps_antenna_tap_break_cell cellList`

Default: ""

Data_type: string, read/write

Specifies the cells that would break the tap ground connection.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

add_well_taps_antenna_tap_left_cell

`add_well_taps_antenna_tap_left_cell cellList`

Default: ""

Data_type: string, read/write

Specifies the antenna tap cell names of the left boundary edge.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

add_well_taps_antenna_tap_right_cell

`add_well_taps_antenna_tap_right_cell cellList`

Default: ""

Data_type: string, read/write

Specifies the antenna tap cell names of the right boundary edge.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

add_well_taps_antenna_tap_cell

`add_well_taps_antenna_tap_cell cellList`

Default: ""

Data_type: string, read/write

Specifies the normal antenna tap cell.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

add_well_taps_antenna_tap_pitch

`add_well_taps_antenna_tap_pitch microns`

Default: 0

Data_type: float, read/write

Specifies the pitch of antenna tap cell. It should be twice of the TAP protect distance.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`

- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

add_well_taps_antenna_tap_right_top_edge_cell

`add_well_taps_antenna_tap_right_top_edge_cell cellList`

Default: ""

Data_type: string, read/write

Specifies the antenna tap cell names of the right incorner.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

add_well_taps_avoid_abutment

`add_well_taps_avoid_abutment {true|false}`

Default: false

Data_type: bool, read/write

Specifies whether the inserted taps should avoid horizontal and vertical abutment.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

add_well_taps_avoid_vertical_well_abutment

`add_well_taps_avoid_vertical_well_abutment {none ntap ptap all_tap}`

Default: none

Data_type: enum {none ntap ptap all_tap}

Specifies the type of well-tap cells to be forbidden from the vertical abutment.

This is a root attribute.

- To set the value of any root attribute, use the set_db command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the get_db command:
`get_db attribute_name`

add_well_taps_block_boundary_only

`add_well_taps_block_boundary_only {true|false}`

Default: false

Data_type: bool, read/write

Specifies whether to insert termination tap cells only for well tap columns on the block boundary. Default is false, termination cells are inserted on both core boundary row and block boundary row.

Note: This attribute works only when inserting termination cells.

This is a root attribute.

- To set the value of any root attribute, use the set_db command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the get_db command:
`get_db attribute_name`

add_well_taps_bottom_tap_cell

`add_well_taps_bottom_tap_cell cellName`

Default: ""

Data_type: string, read/write

Specifies the cell to be used as the bottom tap for the add_endcaps boundary cell insertion flow.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

add_well_taps_bottom_termination_cell

`add_well_taps_bottom_termination_cell` *cellList*

Default: ""

Data_type: string, read/write

Specifies the list of bottom termination cells to be used.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

add_well_taps_cell

`add_well_taps_cell` *cellName*

Default: ""

Data_type: string, read/write

Specifies the name of the cell in the technology library to use as a well-tap. The cell should be of class type CORE.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

add_well_taps_channel_offset

`add_well_taps_channel_offset offset_value`

Default: 0.0

Data_type: double, read/write

Specifies the offset value between taps and the blockage in channel.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

add_well_taps_check_channel

`add_well_taps_check_channel {true|false}`

Default: false

Data_type: bool, read/write

Specifies whether the blockage channel will be checked.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

add_well_taps_column_cells

`add_well_taps_column_cells celllist`

Default: ""

Data_type: string, read/write

Specifies the maximum distance, in microns, between the tie-cell driver and the tie-pins. A value of 0 implies no distance constraint.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

add_well_taps_create_rows

`add_well_taps_create_rows {true|false}`

Default: `false`

Data_type: `bool`, read/write

Creates rows for welltap cell techSites which do not have rows in the floorplan.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

add_well_taps_disable_check_zone_at_boundary

`add_well_taps_disable_check_zone_at_boundary {none vdd vss both}`

Default: `none`

Data_type: `enum {none vdd vss both}`, read/write

Creates rows for welltap cell techSites which do not have rows in the floorplan.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

add_well_taps_in_row_offset

`add_well_taps_in_row_offset offset_value`

Default: 0.0

Data_type: double, read/write

Specifies the distance between the first welltap in a row and the start of the row.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

add_well_taps_insert_cells

`add_well_taps_insert_cells {cellName rule dense_layer dense__rule boundary_layer boundary_rule}`

Default: ""

Data_type: string, read/write

Inserts well-taps based on cell, layer, and interval.

- *cellName*: Specifies the cell name.
- *rule*: Cell with this rule would be added as a general tap cell for all placeable area. The center to center spacing of tap cells would be $\leq (2 * \$rule)$ without `-checker_board` and $\leq (4 * \$rule)$ with `-checker_board`.
Note: Center of the tap cell is used to check the spacing.
- *dense_layername*: Specifies the dense layer name.
- *dense_rule*: Cell with dense layer would be added to the given layer regions only. Cell with dense layer must follow the dense rule. The center to center spacing of dense tap cells would be $\leq (2 * \$dense_rule)$ without `-checker_board` and $\leq (4 * \$dense_rule)$ with `-checker_board`.
- *boundary_layername*: Specifies the boundary layer name.
- *boundary_rule*: A cell would be added for boundaries inside the given layer regions. The

spacing between the boundary to the center of boundary tap cells would be $\leq$ \$ boundary rule. Boundary layer regions which cover partial sites should be extended. Boundary taps can be located in or out of boundary layer regions.

Note:

- You can define multiple cells for -insert_cells by separating each cell by { }.
- There should be at least one rule, dense layer or boundary layer, defined for each cell.
- The order of tap cell insertion is:
 - a. Dense tap
 - b. Boundary tap
 - c. General tap
- If the area is covered by both dense layer region and boundary layer region, both layer rules must be satisfied.
- If -insert_cells is defined multiple times, the last setting is accepted by the tool.

This is a root attribute.

- To set the value of any root attribute, use the set_db command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the get_db command:
`get_db attribute_name`

add_well_taps_replace_intersection_cell

`add_well_taps_replace_intersection_cell <cellName>`

Default: ""

Data_type: string, read/write

Specifies the cell to be used for replacing the intersection break cell.

This is a root attribute.

- To set the value of any root attribute, use the set_db command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the get_db command:

`get_db attribute_name`

add_well_taps_rule

`add_well_taps_rule RuleValue`

Default: 0.0

Data_type: double, read/write

Specifies the rule to be honored when adding well taps.

To define multiple TAP rules in a design , `add_well_taps` parses TAPDISTANCE defined in the cell LEF and Tech LEF. It works on the following conditions:

- If you do not define `set_db add_well_taps_rule`, the tool will use the LEF rule.
- If both `set_db add_well_taps_rule` and LEF rule are not specified, the tool will error out.
- If there are multiple tap distance values defined in the Tech LEF, `add_well_taps` will take the smallest value.
- For existing tap cells, you need to use `add_well_taps -incremental`. If you specify a tap cell with tap distance bigger than the incremental cell, the tool will add the cell and print a warning message.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

add_well_taps_site_offset

`add_well_taps_site_offset number_of_sites`

Default: 1

Data_type: int, read/write

Specifies an offset value, in number of sites, so that the vertical taps are aware of.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

add_well_taps_tap_function_cells

`add_well_taps_tap_function_cells cell_list`

Default: ""

Data_type: string, read/write

Defines the endcap cells which have tap functions. Use of tap function cells reduces the number of tap cells to be inserted. The defined endcap cells are treated as tap function cells during the tap cell insertion. The tool will consider the tap distance between welltap cell and the defined endcap cell with tap function.

To insert tap cells considering the tap function not only on the checkboard but also on the termination tap cell insertion flow, `addWellTap` treats the endcap cell as a tap function cell when `set_db add_well_taps_tap_function_cells` is set.

The `check_well_taps` command honors the `add_well_taps_tap_function_cells` settings to check the tap distance.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

add_well_taps_tap_termination_alignment

`add_well_taps_tap_termination_alignment [true | false]`

Default: false

Data_type: bool, read/write

Specifies whether to align well-taps and termination cells.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

add_well_taps_termination_align

`add_well_taps_termination_align {left right center}`

Default: left

Data_type: enum {left right center}, read/write

Specifies the termination cell alignment.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

add_well_taps_termination_cells

`add_well_taps_termination_cells celllist`

Default: ""

Data_type: string, read/write

Specifies the list of cells that can be used as termination taps. The command `add_well_taps` selects the best fit termination cells from this list and inserts them at the top and bottom of well tap column segment. The existing non-termination well taps are deleted as needed. If the well tap columns have already been terminated with a valid termination cell, the existing termination cell are kept in the incremental run and no changes are made.

Note: It is recommended that well tap columns are added before termination cell insertion.

Note: This attribute must be specified for termination tap cell insertion.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

set_db attribute_name value

- To get the current value of any root attribute, use the *get_db* command:

get_db attribute_name

Related Commands

- *add_well_taps*

add_well_taps_top_tap_cell

add_well_taps_top_tap_cell cellName

Default: ""

Data_type: string, read/write

Specifies well taps inserted into top rows during enclosed end cap insertion.

Note: This attribute must be specified when *add_endcaps_boundary_tap* is set to *true*.

This is a root attribute.

- To set the value of any root attribute, use the *set_db* command:
set_db attribute_name value
- To get the current value of any root attribute, use the *get_db* command:

get_db attribute_name

add_well_taps_vertical_boundary_spacing

add_well_taps_vertical_boundary_spacing spacing_value

Default: -1.0

Data_type: double, read/write

Specifies the spacing between the center of tap cell and vertical boundary.

This is a root attribute.

- To set the value of any root attribute, use the *set_db* command:
set_db attribute_name value

- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

add_well_taps_top_termination_cell

`add_well_taps_top_termination_cell cellList`

Default: ""

Data_type: string, read/write

Specifies the list of top termination cells to be used.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

add_well_taps_well_cut_cells

`add_well_taps_well_cut_cells celllist`

Default: ""

Data_type: string, read/write

Specifies the list of cells with the well cuts. `addWellTap` considers the specified cells cutting the row.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

bscan

The following root attributes in the `bscan` category are supported through the `dft_design` command:

- `dft_boundary_cell_module_prefix`
- `dft_jtag_instance_name`
- `dft_jtag_module_name`

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `bscan` category, use the following command:

```
get_db -category bscan
```

dft_boundary_cell_module_prefix

```
dft_boundary_cell_module_prefix string
```

Description

Default:

Data_type: string, read/write

Specifies the prefix for the module names of boundary-scan cells added during the insertion of the boundary-scan logic.

Related Commands

- `dft_design`
- `synthesize_design`

dft_jtag_instance_name

dft_jtag_instance_name *string*

Description

Default: JTAG_MODULE

Data_type: string, read/write

Specifies the instance name of the JTAG macro.

Related Commands

- `dft_design`
- `synthesize_design`

dft_jtag_module_name

dft_jtag_module_name *string*

Description

Default: JTAG_MACRO

Data_type: string, read/write

Specifies the module name of the JTAG macro.

Related Commands

- `dft_design`
- `synthesize_design`

check_ac_limit

The root attributes in the `check_ac_limit` category are:

- `check_ac_limit_additional_individual_violation_report`
- `check_ac_limit_avg_recovery`
- `check_ac_limit_check_thermal_aware_em`
- `check_ac_limit_current_file`
- `check_ac_limit_current_scale_factor`
- `check_ac_limit_current_scale_table`
- `check_ac_limit_default_freq_for_unconstrained_nets`
- `check_ac_limit_delta_temperature`
- `check_ac_limit_delta_temperature_layer_list`
- `check_ac_limit_detailed`
- `check_ac_limit_effort_level`
- `check_ac_limit_em_cdf_percentage`
- `check_ac_limit_em_limit_scale_factor`
- `check_ac_limit_em_limit_scale_table`
- `check_ac_limit_em_res_width`
- `check_ac_limit_em_temperature`
- `check_ac_limit_em_temperature_layer_list`
- `check_ac_limit_em_threshold`
- `check_ac_limit_enable_seb`
- `check_ac_limit_env_temperature`
- `check_ac_limit_extraction_tech_file`
- `check_ac_limit_force_hold_view`
- `check_ac_limit_handle_pin_obs_via`
- `check_ac_limit_ict_em_models`

- [check_ac_limit_ircx_models](#)
- [check_ac_limit_lifetime](#)
- [check_ac_limit_lifetime_rating_factor_file](#)
- [check_ac_limit_max_error](#)
- [check_ac_limit_method](#)
- [check_ac_limit_min_peak_duty_ratio](#)
- [check_ac_limit_min_peak_freq](#)
- [check_ac_limit_net_file](#)
- [check_ac_limit_nets](#)
- [check_ac_limit_out_file](#)
- [check_ac_limit_peak_td_method](#)
- [check_ac_limit_read_detail_delta_temperature_file](#)
- [check_ac_limit_seb_lifetime](#)
- [check_ac_limit_seb_table](#)
- [check_ac_limit_seb_temperature](#)
- [check_ac_limit_report_db](#)
- [check_ac_limit_report_unannotated_shapes](#)
- [check_ac_limit_selected](#)
- [check_ac_limit_simulation_net](#)
- [check_ac_limit_simulation_net_file](#)
- [check_ac_limit_skip_category_mode](#)
- [check_ac_limit_skip_net](#)
- [check_ac_limit_skip_net_file](#)
- [check_ac_limit_toggle](#)
- [check_ac_limit_top_scope_ignore_block_internal_nets_on_boundary_path](#)
- [check_ac_limit_use_db_freq](#)
- [check_ac_limit_use_qrc_tech](#)

- [check_ac_limit_use_rms_delta_t](#)
- [check_ac_limit_view](#)

To set the value of any root attribute, use the [set_db](#) command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the [get_db](#) command:

```
get_db attribute_name
```

To get the current value of all attributes in the `check_ac_limit` category, use the following command:

```
get_db -category check_ac_limit
```

check_ac_limit_additional_individual_violation_report

```
check_ac_limit_additional_individual_violation_report {true | false}
```

Default: false

Data_type: Boolean

Specifies to generate separate violation reports for each current waveform calculation method (`rms`, `peak`, or `avg`). If this attribute is not specified, a single violation report file is generated using the `check_ac_limit_method`

`{RMS AVG peak}` attribute. This attribute gives you the flexibility of generating either a single collated report or individual reports for each analysis method.

Example:

```
set_db check_ac_limit_method {rms peak avg} \  
check_ac_limit_out_file context1.rpt \  
...  
check_ac_limit_additional_individual_violation_report true
```

This command will generate 3 different reports, namely `context1.rpt_avg`, `context1.rpt_peak`, and `context1.rpt_rms`. If there are no EM violations for a method, the tool will not generate the corresponding separate report. In this example, if there are violations for `avg` and `rms` but none for `peak`, then it will generate only two reports, `context1.rpt_avg` and `context1.rpt_rms`.

check_ac_limit_avg_recovery

```
check_ac_limit_avg_recovery em_recover
```


Default: 1.0

Data_type: Double

Specifies the recovery factor used for calculating I_{avg} current density with recovery. By default, the recovery factor (per layer) is read from the QRC techfile. If you specify `em_recover`, it overrides the QRC tech `em_recover` factor for all layers used in I_{avg} limits. The minimum value of this attribute is 1 and the maximum is 1.

check_ac_limit_check_thermal_aware_em

```
check_ac_limit_check_thermal_aware_em {true | false}
```

Default: false

Specifies to use the thermal map for EM checks. The thermal map file is specified using the `set_rail_analysis_config -read_thermal_map` parameter. The Rail Analysis engine uses the temperature specified in the thermal map file for EM analysis. EM limit has a strong temperature dependency. For a chip that has substantial temperature variation across the chip, the Rail Analysis engine can use the temperature specified in the thermal map file to perform accurate EM analysis.

check_ac_limit_current_file

```
check_ac_limit_current_file filename
```

Default: ""

Data_type: String

Specifies the current of a macro's output pins for EM calculation, in case the macro does not have the respective timing library. When the macro output pin current is set by the current file, the value calculated (if any) from AAE is overwritten.

check_ac_limit_current_scale_factor

```
check_ac_limit_current_scale_factor {{avg value} {rms value} {peak value}}
```

Default: ""

Data_type: String

Scales the signal EM current by the respective factor for avg, rms, or peak current specified by `{avg value} {rms value} {peak value}`.

The value specified for this attribute must be > 0 and <= 10.

check_ac_limit_current_scale_table

`check_ac_limit_current_scale_table current_scale_table_file`

Default: ""

Data_type: String

Specifies the name of the scale table file used for layer-based scaling of the signal EM current. The scale table contains the scale factor for avg, rms, or peak current for each layer.

check_ac_limit_default_freq_for_unconstrained_nets

`check_ac_limit_default_freq_for_unconstrained_nets freq_in_Hz`

Default: 1e+06

Data_type: Double

Specifies the frequency for EM calculation when a net has no defined frequency or has a defined frequency of 0 Hz in the design.

check_ac_limit_delta_temperature

`check_ac_limit_delta_temperature temp`

Default: 5.0

Data_type: Double

Specifies the maximum temperature rise permitted in units of Celsius. This value is normally used in the QRC tech file for the RMS current limit equation. For example:

```
em_jmax_ac_rms EQU sqrt(12.66 * delta_T * (w - 0.003)^2 * (w - 0.003 + 0.264) / (w - 0.003 + 0.0443))
```

check_ac_limit_delta_temperature_layer_list

`check_ac_limit_delta_temperature_layer_list {{<LEF_layer_name> <delta_T_in_C>}+}`

Default: ""

Data_type: String

Specifies the delta temperature for specific LEF layers during signal EM RMS current limit analysis.

check_ac_limit_detailed

check_ac_limit_detailed

Default: false

Data_type: Boolean

Generates a detailed report containing information for all signal nets, including those without violations.

check_ac_limit_effort_level

check_ac_limit_effort_level {low | medium | high}

Default: low

Data_type: Enum

Specifies the method used for current calculation of nets in the Signal ElectroMigration analysis flow. The two methods are:

- **non-simulation mode** - use the capacitance ratio to distribute the current down the stream from the driver to the receiver.
- **simulation mode** - simulates resistance and capacitance to get the true current on each segment. This mode is more accurate but requires a longer run time.

This attribute has the following three arguments:

- **low:** enables the non-simulation mode for all nets where the current distribution is based on the capacitance ratio.
- **medium:** enables the simulation mode for the nets with physical loops, and the non-simulation mode for the non-loop nets.
- **high:** enables the simulation mode for all nets.

The **medium** and **high** levels require a VTS-AA license.

Note: For the simulation mode with effort level **medium** or **high**:

- If you use `extract_rc`, run `set_db extract_rc_qrc_cmd_type`

partial extract_rc_qrc_cmd_file *qrc.user.cmd*, where *qrc.user.cmd* must include the following CCL commands:

```
extraction_setup -analysis em -cluster_vias_in_em true
output_db -type spef -subtype "EXTENDED"
```

Refer to the example below:

```
output_db \
  -type spef -busbit_delimiter "[]" -escape_special_character false \
  -hierarchy_delimiter "/" -output_incomplete_nets true \
  -output_unrouted_nets true -pin_delimiter ":" -subtype "EXTENDED" \
  -disable_subnodes false -add_explicit_vias true \
  -include_parasitic_res_width true -include_parasitic_res_width_drawn true
```

- If you run standalone Quantus to generate the required SPEF file in the extended format separately, then instead of `extract_rc`, use the following command for signal EM analysis:

```
read_spef -keep_star_node_locations -extended spef_file
```

check_ac_limit_em_cdf_percentage

```
check_ac_limit_em_cdf_percentage value
```

Default: 1e-09

Data_type: bool, read/write

Calculates the Cumulative Distribution Factor (CDF) derating factor based on the EM rules specified in the ICT EM file, given by the `check_ac_limit_ict_em_models` attribute. This attribute is used to calculate the fail rate and reliability of a design. The value of the CDF percentage should be between 0 and 100.

When the CDF related rule exists in the ICT EM file, the software uses the default value 1e-09 for `check_ac_limit_em_cdf_percentage` if it is not specified.

check_ac_limit_em_limit_scale_factor

```
check_ac_limit_em_limit_scale_factor {{avg value} {rms value} {peak value}}
```

Default: ""

Data_type: String

Scales the calculated signal EM limits by the respective factor for avg, rms or peak current limit specified by {avg value} {rms value} {peak value}. The software compares the current through the net against the scaled EM limit to report violations in signal EM.

The value specified for this attribute must be > 0 and <= 10.

check_ac_limit_em_limit_scale_table

```
check_ac_limit_em_limit_scale_table em_limit_scale_table_file
```

Default: ""

Data_type: String

Specifies the name of the scale table file used for layer-based scaling of the signal EM limit. The scale table contains the scale factor for avg, rms, or peak current for each layer.

check_ac_limit_em_res_width

```
check_ac_limit_em_res_width {drawn silicon}
```

Default: drawn

Data_type: Enum

Specifies width used to get EM limit.

check_ac_limit_em_temperature

```
check_ac_limit_em_temperature value
```

Default: -1.0

Data_type: Double

Specifies the temperature expected for the device and is used to look up the temperature scaling factor. By default, no scaling is done. The *value* is a float, in units of degrees Celsius.

check_ac_limit_em_temperature_layer_list

```
check_ac_limit_em_temperature_layer_list {{<LEF_layer_name> <em_temperature_in_C>}}+
```

Default: ""

Data_type: String

Sets a different temperature for specific LEF layers during signal EM AVG current limit analysis.

check_ac_limit_em_threshold

`check_ac_limit_em_threshold value`

Default: 1.0

Data_type: Double

Specifies to report violations in signal EM above the threshold value. The value is the ratio of signal current (avg/peak/rms) to the respective EM limit.

check_ac_limit_enable_seb

`check_ac_limit_enable_seb {true | false}`

Default: false

Data_type: Boolean

Specifies to enable the Statistical ElectroMigration Budgeting (SEB) analysis.

check_ac_limit_env_temperature

`check_ac_limit_env_temperature value`

Default: -1.0

Data_type: Double

Specifies the ambient temperature used to calculate S_{DC} for SEB analysis.

check_ac_limit_extraction_tech_file

`check_ac_limit_extraction_tech_file filename`

Default: ""

Data_type: String

Specifies an extraction technology file for the Signal ElectroMigration analysis flow. If this attribute is not specified, then by default the extraction technology file will be taken from the analysis view

(current RC corner).

check_ac_limit_force_hold_view

`check_ac_limit_force_hold_view`

Default: `false`

Data_type: **Boolean**

Forces `set_signal_em_analysis_mode` to use the hold view for current calculation even during setup fixing.

check_ac_limit_handle_pin_obs_via

`check_ac_limit_handle_pin_obs_via`

Default: `false`

Data_type: **Boolean**

Specifies to consider the vias defined in the `OBS` (obstructions/blockages) section of LEF in the Signal ElectroMigration flow. By default, the software does not consider via in obstruction.

This attribute does not honor the via obstructions with "`SPACING 0`".

check_ac_limit_ict_em_models

`check_ac_limit_ict_em_models` *file*

Default: `" "`

Data_type: **String**

Specifies the ICT-EM file which only contains the EM rules and excludes the RC extraction data in ICT. The software will use the EM rules in the specified ICT-EM file to calculate EM limits for signal net EM analysis even if there are embedded EM rules in the Quantus QRC tech file.

check_ac_limit_ircx_models

`check_ac_limit_ircx_models` `{RC.ircx EMIR.ircx}|{RC.ircx}`

Default: `" "`

Specifies an encrypted or unencrypted RC/EM technology (.ircx) file. The foundry-specific IRCX file, which contains the process information for extraction and EM modeling, can be used when the ICT-EM file is not available for electromigration analysis.

You can either specify two IRCX files, one file containing the RC process data (RC.ircx) and the other file containing EM rules (EM.ircx), or a single IRCX file that includes both the RC and EM sections (RC.ircx). A single IRCX file is used for the mature nodes, and separate RC and EM IRCX files are used for the advanced process nodes.

check_ac_limit_lifetime

check_ac_limit_lifetime *value*

Default: 87600.0

Data_type: Double

Specifies the total hours of operation expected for the device, and is used to look up the lifetime scaling factor. *value* is used to find the scaling factor by interpolating from a piece-wise-linear table in the LEF or the QRC tech file (em_jmax_ac model with jmax_lifetime table). If there is no scaling factor table, no scaling is done. *value* is a float, in units of hours.

check_ac_limit_lifetime_rating_factor_file

check_ac_limit_lifetime_rating_factor_file *filename*

Default: ""

Data_type: String

Specifies a lifetime rating factor file that contains jmax_life values for each layer. These values are defined based on temperature and lifetime pairs. When you provide this file, EM analysis will automatically select the appropriate jmax_life factor for each layer according to the specific temperature-lifetime combinations defined within the file. The lifetime and EM temperature can be specified using the check_ac_limit_lifetime and check_ac_limit_em_temperature attributes.

Following is a snippet of the lifetime rating factor file:

*Rating factor of M0/V0/M1/V1/M2/V3/M3...

TEMPERATURE	90	95	100	105	110 ...
-------------	----	----	-----	-----	---------

LifeTime					
----------	--	--	--	--	--

0.5	2.578	2.242	2.024	1.774	1.547
-----	-------	-------	-------	-------	-------

1	2.520	2.191	1.979	1.734	1.512
---	-------	-------	-------	-------	-------

2	2.430	2.113	1.908	1.672	1.458
3	2.382	2.072	1.871	1.639	1.430
...					

check_ac_limit_max_error

check_ac_limit_max_error value

Default: 10000

Data_type: Integer

Specifies the maximum number of errors to report. The command stops when the maximum value is reached.

check_ac_limit_method

check_ac_limit_method {rms peak avg}

Default: ""

Data_type: String

Specifies the current waveform calculation method. You can specify one or more of the following options:

- **avg:** Checks I_{avg} limits and reports any average current limit violations in the design.
- **peak:** Checks I_{peak} limits and reports any peak current limit violations in the design.
- **rms:** Checks I_{rms} limits and reports any RMS current limit violations in the design.

check_ac_limit_min_peak_duty_ratio

check_ac_limit_min_peak_duty_ratio duty_ratio

Default: -1.0

Data_type: Double

Sets the minimum duty ratio value for I_{peak} calculation. If the calculated duty ratio of a net is lower than the specified value, the specified value will be used to calculate I_{peak} . The minimum value for *duty_ratio* is 0.0 and the maximum is 1.0.

check_ac_limit_min_peak_freq

`check_ac_limit_min_peak_freq freq_in_Hz`

Default: 1e+06

Data_type: Double

Ignores I_{peak} current limit calculation for signal nets that have an effective frequency below the specified value (in Hertz).

check_ac_limit_net_file

`check_ac_limit_net_file list_file`

Default: ""

Data_type: String

Specifies the net from file to check the signalEM analysis. The format is one net name per line, with comments using # as the first character. For example:

```
#list of nets to skip
net_a
net_b
net_c
```

check_ac_limit_nets

`check_ac_limit_nets {netNames}`

Default: ""

Data_type: String

Specifies whether to check named or selected nets. If you name only one net, you do not need the braces around the net name. If you specify more than one net, you must separate the net names with a space.

check_ac_limit_out_file

`check_ac_limit_out_file filename`

Default: ""

Data_type: String

Specifies the report file for the violation data. The report file includes name of the net, I_{rms} , interpolated `ACCURRENTDENSITY` limit (I_{limit}), layer, width, X and Y location, rise and fall time, effective frequency for the net (F_{eff}), Cnet and Vdd for each net.

check_ac_limit_peak_td_method

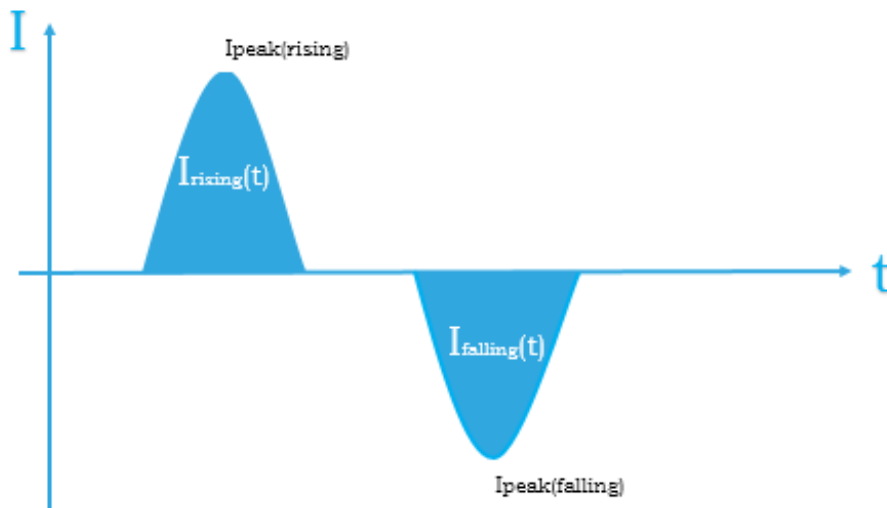
```
check_ac_limit_peak_td_method {effective_width_from_integration  
effective_half_peak_width max_equivalent_dc_peak sum_half_peak_width}
```

Default: effective_width_from_integration

Data_type: Enum

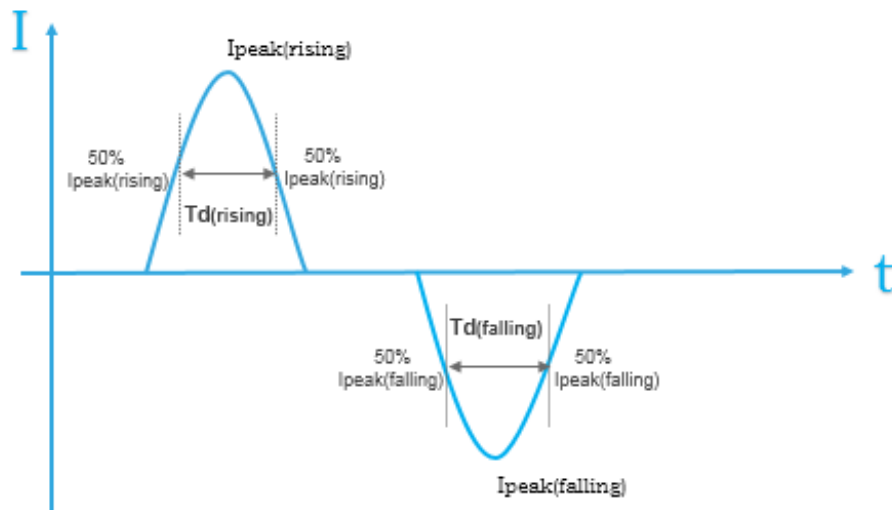
Specifies the Td calculation method for Ipeak check. The legal values are depicted below.

effective_width_from_integration



- $$Td = \frac{\int_0^{T_{SW}} |I(t)| dt}{I_{peak_measured}}$$
 - Tsw = Period of one switching cycle (rising + falling)
 - Ipeak_measured = $\text{abs.max}(I_{peak}(\text{rising}), I_{peak}(\text{falling}))$

effective_half_peak_width

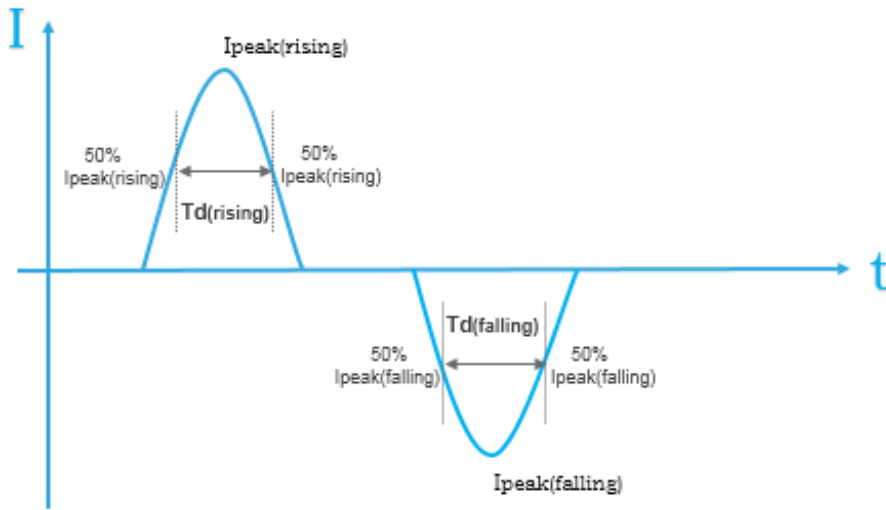


- When $I_{\text{peak(rising)}} \geq I_{\text{peak(falling)}}$

$$T_d = T_d(\text{rising}) + \left(\frac{I_{\text{peak(falling)}}}{I_{\text{peak(rising)}}} \right)^2 * T_d(\text{falling})$$
- When $I_{\text{peak(rising)}} < I_{\text{peak(falling)}}$

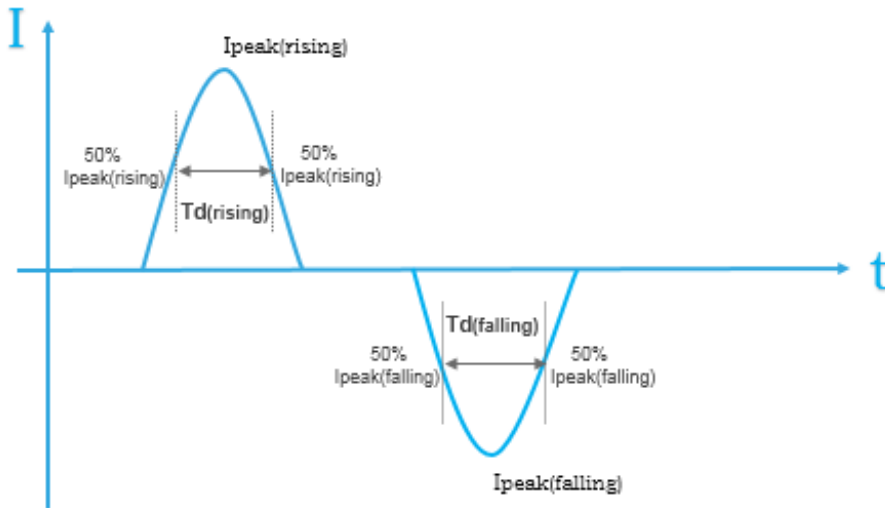
$$T_d = T_d(\text{falling}) + \left(\frac{I_{\text{peak(rising)}}}{I_{\text{peak(falling)}}} \right)^2 * T_d(\text{rising})$$

max_equivalent_dc_peak



- $R(\text{rising}) = Td(\text{rising}) / (T/2)$
- $R(\text{falling}) = Td(\text{falling}) / (T/2)$
- Choose $\max(I_{\text{peak}}(\text{rising}) * \sqrt{R(\text{rising})}, I_{\text{peak}}(\text{falling}) * \sqrt{R(\text{falling})})$ to compare against $I_{\text{peak_dc_limit}}$
- $I_{\text{peak_ac_limit}} = I_{\text{peak_dc_limit}} / \sqrt{R}$
- When $I_{\text{peak}}(\text{rising}) * \sqrt{R(\text{rising})} \geq I_{\text{peak}}(\text{falling}) * \sqrt{R(\text{falling})}$
 $Td = 2 * Td(\text{rising})$
- When $I_{\text{peak}}(\text{rising}) * \sqrt{R(\text{rising})} < I_{\text{peak}}(\text{falling}) * \sqrt{R(\text{falling})}$
 $Td = 2 * Td(\text{falling})$

sum_half_peak_width



- $Td = Td(\text{rising}) + Td(\text{falling})$

check_ac_limit_read_detail_delta_temperature_file

`check_ac_limit_read_detail_delta_temperature_file filename`

Default: ""

Data_type: String

Specifies to read the detailed delta temperature file generated by the `report_self_heat` command (self-heat analysis).

check_ac_limit_seb_lifetime

`check_ac_limit_seb_lifetime value`

Default: 43800.0

Data_type: Double

Specifies the life time of product in hours.

check_ac_limit_seb_table

`check_ac_limit_seb_table table_filename`

Default: ""

Data_type: String

Specifies the file name of the Failures in Time (FIT) table. This file is provided by the foundry. The file has the following columns:

- Layer
- MTF
- Sigma
- Activate energy
- Current density exponent

check_ac_limit_seb_temperature

`check_ac_limit_seb_temperature temperature`

Default: -273.0

Data_type: Double

Specifies the temperature used to update the median time to failure (MTF) with respect to the ambient temperature.

- If this temperature is specified, it would be used globally for all the resistors.
- If this temperature is not specified, the `seb_temperature` may take the following possible values depending on the specific circumstances:
 - If detailed delta temperature file is not used for SEB, the `seb_temperature` always takes the same value as that used for EM analysis.
 - If detailed delta temperature file is used for SEB, the `seb_temperature` is calculated by the following equation:

`seb_temperature = env_temperature + delta_T`

Where `delta_T` is the maximum temperature rise, including joule heat and coupling heat from FEOL, in the resistor's bounding box with zero extension.

check_ac_limit_report_db

`check_ac_limit_report_db`

Default: `false`

Data_type: Boolean

Specifies to write the signal EM data into a database file (*filename.db*) after performing signal EM analysis. This file contains the information required for loading and displaying signal EM data in GUI.

check_ac_limit_report_unannotated_shapes

```
check_ac_limit_report_unannotated_shapes {true | false}
```

Default: `false`

Data_type: Boolean

Provides visibility into partial net coverage within SPEF files. When this attribute is set to `true`, the `<report_name>.rpt.unannotated_shapes` is generated. This report identifies and lists nets that contain shapes without resistance annotations, or unanalyzed/ partially analyzed segments. The feature helps to detect incomplete parasitic extraction coverage and improve accuracy during validation and signoff workflows. The reporting of partial net coverage works only when `check_ac_limit_effort_level` is set to `high`.

Use Model:

```
set_db check_ac_limit_effort_level high check_ac_limit_report_unannotated_shapes true
```

Report Format:

```
NET: <net_name>          <- Specifies the name of the analyzed simulation nets with  
unannotated shapes  
LAYER (x1 y1 x2 y2)      <- Layer name and shape coordinates
```

Sample Report:

```
NET: Net_A  
LAYER (x1          y1          x2          y2)  
M2     (30.2850    12.0800    37.6150    12.2200)  
M2     (30.2000    12.0650    30.2850    12.2350)
```

 This attribute is applicable only when running in the DP mode; it is not supported in the XP mode.

check_ac_limit_selected

`check_ac_limit_selected filename`

Default: false

Data_type: Boolean

Analyzes the net specified by selecting a net in the main GUI window.

check_ac_limit_simulation_net

`check_ac_limit_simulation_net list_of_nets`

Default: ""

Forces the specified nets to use the simulation mode.

check_ac_limit_simulation_net_file

`check_ac_limit_simulation_net_file filename`

Default: ""

Forces the nets in the specified file to use the simulation mode.

check_ac_limit_skip_category_mode

`check_ac_limit_skip_category_mode {0 | 1 | 2}`

Default: 1

Data_type: Integer

Specifies to skip nets that belong to categories mapping to the specified mode. This parameter allows you to specify three modes, namely 0, 1 (default), and 2.

The following table lists the common categories for all modes, and the categories specific to each mode:

Common Categories	0 Skip Category Mode	1 Skip Category Mode	2 Skip Category Mode
• Non top scope nets • Internal nets on boundary path • P/G nets • Constant nets • User-defined skipped nets • Nets below effective frequency threshold • Physically broken nets • Nets without loading capacitance annotation • Nets with SPEF back-annotation failures during simulation • Nets failed in analysis	• Slave nets by Verilog Assign • Nets (non-constant) without frequency annotation • Nets without physical shape for output pin • Nets without wire shape connection to output pin • Nets with zero pin current	• Equivalent nets by Verilog Assign • Nets without driver • Nets without receiver	• Nets without pins • Equivalent nets by Verilog Assign • Nets without driver • Nets without receiver

For example, if you set `set_signal_em_analysis_mode -skip_category_mode` to 1, the report file lists all categories listed in the *Common Categories* and *1 Skip Category Mode* columns.

Following is a snippet of the report file for mode 1:

```

Total Nets in Design:          33451
Total Selected Nets in Design:    4
Total Analyzed Nets:           0
Total Skipped Nets:            4
  Non top scope nets:          0
  Internal nets on boundary path: 0
  Equivalent nets by Verilog Assign: 0
  P/G nets:                    0
  Constant nets:                2
  User-defined skipped nets:    0
  Nets below effective frequency threshold: 0
  Nets without driver:          1
  Nets without receiver:        1
  Physically broken nets:       0
  Nets without loading capacitance annotation: 0
  Nets with SPEF back-annotation failures during simulation: 0
  Nets failed in analysis:      0

```

check_ac_limit_skip_net

`check_ac_limit_skip_net net_name_list`

Default: ""

Data_type: String

Specifies the net to be skipped in the signalEM analysis.

Refer to the example below:

```
{net_a net_b net_c}
```

check_ac_limit_skip_net_file

`check_ac_limit_skip_net_file list_file`

Default: ""

Data_type: String

Specifies the net file to be skipped in the signalEM analysis.

The format is one net name per line, with comments using # as the first char. For example:

```
#list of nets to skip
net_a
net_b
net_c
```

check_ac_limit_toggle

`check_ac_limit_toggle value`

Default: 1.0

Data_type: Double

Specifies the switching factor for signal nets. A value of 1.0 means that if a flip-flop is clocked by a 20 Mhz clock, it changes state on 100% of the clock cycles. Therefore, the signal has 20 million transitions, which is half the number of transitions of the input clock. As a result, the final switching frequency of the signal net is 10 Mhz.

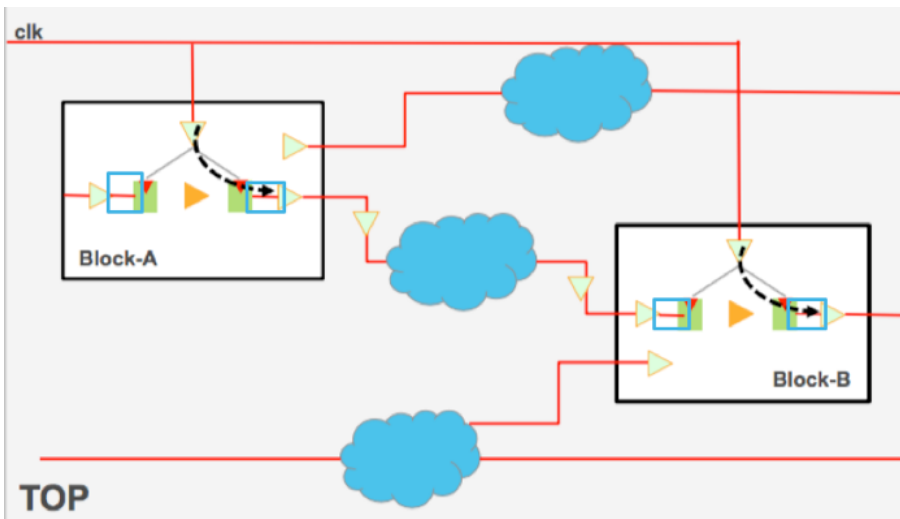
Value Range: 0.0 to 1.0

check_ac_limit_top_scope_ignore_block_internal_nets_on_boundary_path

`check_ac_limit_top_scope_ignore_block_internal_nets_on_boundary_path {true | false}`

Default: false

Ignores the internal nets from the first instance to the first flop on the boundary path for the signal EM checking. The following figure represents that when this parameter is used, the internal nets (highlighted in the blue rectangles) are ignored. However, all the nets highlighted through the red lines are reported in the TOP scope (`create_top_scope`) flow:



check_ac_limit_use_db_freq

check_ac_limit_use_db_freq

Default: false

Data_type: Boolean

Uses the database frequency value as the effective frequency per net.

check_ac_limit_use_qrc_tech

check_ac_limit_use_qrc_tech

Default: true

Data_type: Boolean

Forces I_{rms} checks to use the QRC tech file instead of the technology LEF file. If either I_{peak} or I_{avg} is also checked, all checks will use the QRC tech file, including I_{rms} . This attribute is enabled by default.

Note: The `-method avg` and `-method peak` analysis will only support AC current density rules from QRC tech file. If either `avg` or `peak` is also specified, then all checks will use the QRC tech file equations, including RMS.

check_ac_limit_use_rms_delta_t

```
check_ac_limit_use_rms_delta_t {true | false}
```

Default: false

Data_type: Boolean

When this parameter is set to `true`, the `delta_T` used to calculate `seb_temperature` would be based on the RMS delta T (joule heat only).

If this parameter is set to `false`, the `delta_T` used to calculate `seb_temperature` would be based on the sum of self-heat delta T and RMS delta T.

check_ac_limit_view

```
check_ac_limit_view viewName
```

Default: ""

Data_type: String

Specifies the name of the view.

check_design

The root attributes in the `check_design` category are:

- [check_design_across_hierarchy](#)
- [check_design_report_async_pins](#)
- [check_design_report_scan_pins](#)
- [check_design_threshold_fanout](#)

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `read_physical` category, use the following command:

```
get_db -category check_design
```

check_design_across_hierarchy

```
check_design_across_hierarchy {true | false}
```

Default: true

Data_type: boolean, read/write

Checks for undriven/multi-driven/unloaded/constant pins/ports by checking across hierarchical boundaries. By default checks are across hierarchy.

check_design_report_async_pins

```
check_design_report_async_pins {true | false}
```

Default: true

Data_type: boolean, read/write

Checks async pins in a netlist design.

check_design_report_scan_pins

```
check_design_report_scan_pins {true | false}
```

Default: true

Data_type: boolean, read/write

Checks scan pins in a netlist design.

check_design_threshold_fanout

```
check_design_threshold_fanout threshhold
```

Default: 10000

Data_type: int, read/write

Checks for upper threshold in a netlist design.

check_drc

The root attributes in the `check_drc` category are:

- `check_drc_area`
- `check_drc_check_illegal_trim_shapes`
- `check_drc_check_only`
- `check_drc_check_reverse`
- `check_drc_check_routing_halo`
- `check_drc_check_routing_halo_corner`
- `check_drc_disable_rules`
- `check_drc_enable_post_passive_fill`
- `check_drc_exclude_pg_net`
- `check_drc_ignore_cell_blockage`
- `check_drc_ignore_fill_wire`
- `check_drc_ignore_non_rectangle_shapes`
- `check_drc_ignore_trial_route`
- `check_drc_inside_via_def`
- `check_drc_layer_range`
- `check_drc_limit`
- `check_drc_max_wrong_way_halo`
- `check_drc_ndr_spacing`
- `check_drc_quiet`
- `check_drc_report`
- `check_drc_short_only`
- `check_drc_trim_length`
- `check_drc_uncolored`
- `check_drc_use_min_spacing_on_block_obs`

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `check_drc` category, use the following command:

```
get_db -category check_drc
```

check_drc_area

```
check_drc_area {coordinates}
```

Default: {0.0 0.0 0.0 0.0}

Data_type: rect, read/write

Specifies the coordinates of the area to verify.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

`check_drc`

check_drc_check_illegal_trim_shapes

```
check_drc_check_illegal_trim_shapes {true | false}
```

Default: false

Data_type: bool, read/write

Checks the missing trim shapes for a fulfilled design.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

`check_drc`

check_drc_check_only

```
check_drc_check_only {all | regular | special | selected_net | selected | cell |  
default}
```

Default: all

Data_type: enum {all regular special selected_net selected cell default}, read/write

Specifies which types of shapes are to be checked.

- `all`: This is the default value, and reports violations between all routing shapes (regular and special) and all other shapes (including routing blockages, and cell pin/obs shapes). This release has a limitation that it will not check for violations between cell pin/obs shapes (either inside one cell, or between two cells). You can check for cell overlap with `check_place` which normally finds any cell to cell violations.
- `regular`: Only reports violations between regular routing shapes (normally created by NanoRoute) and all other shapes. It does not report violations between special routes, routing blockages and cell shapes. This is useful to find violations caused only by NanoRoute.
- `special`: Only reports violations between special routes, and all other shapes. It does not report violations between regular routes, routing blockages, and cell shapes. This is useful when checking power routing or pre-routed special routes, and isolating these errors from other violations.
- `selected_net`: Reports only the violations between the selected list of nets. Use this attribute value when the DRC fixes are done on nets and complete/area based DRC check is time consuming or it flags multiple other violations.
- `selected`: Reports only the violations on special wire objects/vias of the secondary PG nets with the specified user class.

- `cell`: Reports only the cell.
- `default`: Reports violations between the default shapes.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

Related Commands

`check_drc`

check_drc_check_reverse

`check_drc_check_reverse {true | false}`

Default: `false`

Data_type: `bool`, read/write

Reports the `CONVEXCORNER REVERSE` violation in the flow.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

Related Commands

`check_drc`

check_drc_check_routing_halo

`check_drc_check_routing_halo {true | false}`

Default: false

Data_type: bool, read/write

Checks for routing halo violations.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

`check_drc`

check_drc_check_routing_halo_corner

```
check_drc_check_routing_halo_corner {true | false}
```

Default: false

Data_type: bool, read/write

Checks for routing halo violations in the corner.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

`check_drc`

check_drc_disable_rules

```
check_drc_disable_rules rule_name
```

Default: ""

Data_type: enum* {jog2jog_spacing eol_spacing cut_spacing min_cut enclosure color min_step protrusion min_area out_of_die off_manufacturing_grid off_routing_track}, read/write

Disables the specified rule.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

`check_drc`

check_drc_enable_post_passive_fill

```
check_drc_enable_post_passive_fill {true | false}
```

Default: false

Data_type: bool, read/write

Checks if the track is fully filled for the `full_fill` trim layer.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

`check_drc`

check_drc_exclude_pg_net

`check_drc_exclude_pg_net {true | false}`

Default: false

Data_type: bool, read/write

Excludes the checking of power and ground nets.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

`set_db attribute_name value`

- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

Related Commands

`check_drc`

check_drc_ignore_cell_blockage

`check_drc_ignore_cell_blockage {true | false}`

Default: false

Data_type: bool, read/write

Ignores geometries of cell blockages.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

`set_db attribute_name value`

- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

Related Commands

`check_drc`

check_drc_ignore_fill_wire

```
check_drc_ignore_fill_wire {true | false}
```

Default: false

Data_type: bool, read/write

Ignores metal fill wires while checking DRCs.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

[check_drc](#)

check_drc_ignore_non_rectangle_shapes

```
check_drc_ignore_non_rectangle_shapes {true | false}
```

Default: true

Data_type: bool, read/write

When set to `true`, this attribute ignores checking DRC markers for the objects with the non-rectangular shapes. When set to `false`, it checks DRC markers for the objects of all shapes.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

[check_drc](#)

check_drc_ignore_trial_route

```
check_drc_ignore_trial_route {true | false}
```

Default: false

Data_type: bool, read/write

Ignores trial route when checking DRC.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

`check_drc`

check_drc_inside_via_def

```
check_drc_inside_via_def {true | false}
```

Default: false

Data_type: bool, read/write

Checks the cut spacing from the same via.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

`check_drc`

check_drc_layer_range

```
check_drc_layer_range {layer1 [layer2]}
```

Default: ""

Data_type: string, read/write

Checks between the specified range of metal layers, including the cut layers in between. You can use the full layer name, or abbreviations like `metal1` or `1` for the first routing layer, etc. If `layer2` is not specified, it only checks DRC on `layer1`. For example, the following command checks all the layers between `metal1` and `metal3`:

```
set_db check_drc_layer_range {metal1 metal3}
```

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

[check_drc](#)

check_drc_limit

```
check_drc_limit value
```

Default: 1000

Data_type: int, read/write

Specifies the maximum number of errors to report.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```


Related Commands

[check_drc](#)

check_drc_max_wrong_way_halo

`check_drc_max_wrong_way_halo halo_length`

Default: ""

Data_type: double, read/write

Specifies the length up to which short jog shapes in halo area can be ignored. Routing halos are used to avoid cross-coupling between top-level and inside-the-block routes, and short jogs do not affect cross-coupling much. Normally, it is better to have some short jogs rather than more vias. When this option is specified, any segment with length less than `halo_length` is ignored for wrong way check, even if it overlaps with the halo region. `verify_drc` reports violations only for jog shapes with lengths greater than `halo_length`.

Default: If you do not set the `check_drc_max_wrong_way_halo` attribute, `check_drc` uses $4 \times \text{pitch}$ as the default value, where `pitch` is the METAL2 routing pitch.

This is a root attribute.

- To set the value of any root attribute, use the [set_db](#) command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the [get_db](#) command:

```
get_db attribute_name
```

Related Commands

[check_drc](#)

check_drc_ndr_spacing

`check_drc_ndr_spacing {true | false | auto}`

Default: auto

Data_type: enum {true false auto}, read/write

Checks non-default rule spacing as a hard rule. Following are the attribute values:

- `true`: Checks all non-default rules.
- `false`: Skips checking any non-default rule.
- `auto`: Checks only the non-default rules with `hard_spacing`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

`check_drc`

check_drc_quiet

`check_drc_quiet {true | false}`

Default: `false`

Data_type: `bool`, read/write

Prevents logging sub-area details in the logv file used for debugging.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

`check_drc`

check_drc_report

`check_drc_report filename`

Default: ""

Data_type: string, read/write

Specifies the report file that contains the violation information.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

`check_drc`

check_drc_short_only

```
check_drc_short_only {true | false}
```

Default: false

Data_type: bool, read/write

Excludes checking all other DRCs except the short ones.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

`check_drc`

check_drc_trim_length

```
check_drc_trim_length {true | false}
```

Default: false

Data_type: bool, read/write

Checks the maximum length of the LEF58_TRIMSHAPE value for detecting the DRC violations in the trim shape layer.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

`check_drc`

check_drc_uncolored

```
check_drc_uncolored {true | false}
```

Default: false

Data_type: bool, read/write

Reports color change violation when the DPT layer shape is 'free color'.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

`check_drc`

check_drc_use_min_spacing_on_block_obs

```
check_drc_use_min_spacing_on_block_obs {0 | 1 | auto}
```

Default: auto

Data_type: enum {true false auto}, read/write

Checks all the Macro OBS with minSpacing.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

`check_drc`

cpi

The root attribute in the `cpi` category is:

- `power_intent_insert_auto_ls`

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `cpi` category, use the following command:

```
get_db -category cpi
```

power_intent_insert_auto_ls

```
power_intent_insert_auto_ls {none combo_ls ls_only}
```

Default: none

Data_type: enum, read/write

Enables the capability to infer Level Shifter (LS) and to merge isolation (ISO) cells into the Enabled Level Shifter (ELS), depending on shifting requirements.

Possible values are:

- `combo_ls`: Inserts LS and merges existing ISO cells to ELS, if required.
- `ls_only`: Inserts only LS.
- `none`: Does not insert auto-ls.

cts

- [clock_trees](#)
- [clock_tree_source_groups](#)
- [cts_add_port_driver_max_distance_from_port](#)
- [cts_add_port_driver_max_distance_from_port_rows](#)
- [cts_adjacent_rows_legal](#)
- [cts_allow_non_std_clock_gate_inputs](#)
- [cts_allow_post_route_sizing_stack_via_rule_update](#)
- [cts_balance_mode](#)
- [cts_blackbox_default_driver_base_pin](#)
- [cts_blackbox_default_load_base_pin](#)
- [cts_buffer_cells](#)
- [cts_buffer_cells_leaf](#)
- [cts_buffer_cells_top](#)
- [cts_cell_density](#)
- [cts_cell_halo_mode](#)
- [cts_cell_halo_rows](#)
- [cts_cell_halo_sites](#)
- [cts_cell_halo_x](#)
- [cts_cell_halo_y](#)
- [cts_clock_gating_cells](#)
- [cts_clock_gate_movement_limit](#)
- [cts_clock_source_cells](#)
- [cts_clock_tree_source_group_clock_trees](#)
- [cts_clock_tree_source_input_max_transition_time](#)
- [cts_cloning_effort](#)

- [cts_cloning_inst_name_suffix](#)
- [cts_cloning_inst_name_suffix_source_group_assignment](#)
- [cts_declone_clock_gates](#)
- [cts_declone_clock_logic](#)
- [cts_delay_cells](#)
- [cts_detailed_cell_warnings](#)
- [cts_exit_if_no_placeable_area](#)
- [cts_exit_if_skew_target_over_constrained](#)
- [cts_exit_if_stage_delay_sigma_target_over_constrained](#)
- [cts_fix_clock_sinks](#)
- [cts_flexible_htree_allow_mixed_sink_polarity](#)
- [cts_flexible_htree_enable_metrics_reporting](#)
- [cts_flexible_htree_placement_legalization_effort](#)
- [cts_force_target_max_transition_time](#)
- [cts_ignore_problematic_skew_as_result_of_dont_touch_nets](#)
- [cts_inst_name_prefix](#)
- [cts_inverter_cells](#)
- [cts_inverter_cells_leaf](#)
- [cts_inverter_cells_top](#)
- [cts_load_capacitance_cells](#)
- [cts_logic_cells](#)
- [cts_manage_power_intent_violations](#)
- [cts_max_fanout](#)
- [cts_max_instance_height](#)
- [cts_max_source_to_sink_net_length](#)
- [cts_max_source_to_sink_net_length_leaf](#)
- [cts_max_source_to_sink_net_length_top](#)

- `cts_max_source_to_sink_net_length_trunk`
- `cts_mixed_fanout_net_type`
- `cts_move_clock_gates`
- `cts_move_logic`
- `cts_net_name_prefix`
- `cts_pin_insertion_delay_histogram_bin_size`
- `cts_post_route_enable`
- `cts_post_route_repair_drv`
- `cts_post_route_enable_routing_eco`
- `cts_post_route_repair_skew_by_buffering`
- `cts_primary_delay_corner`
- `cts_primary_reporting_skew_groups`
- `cts_primary_reporting_skew_groups_log_min_max_sinks`
- `cts_repair_congestion`
- `cts_repair_drv_by_buffering`
- `cts_report_skew_groups_only_with_targets`
- `cts_route_clock_tree_nets`
- `cts_route_type_flexible_htree`
- `cts_route_type_flexible_htree_backside`
- `cts_route_type_leaf`
- `cts_route_type_top`
- `cts_route_type_trunk`
- `cts_routing_preferred_layer_effort`
- `cts_safety_tmr_isolation_route_type`
- `cts_size_clock_gates`
- `cts_size_clock_sources`
- `cts_size_logic`

- `cts_skew_group_report_columns`
- `cts_skew_group_report_histogram_bin_size`
- `cts_spec_config_base_pin_trace_through_to`
- `cts_spec_config_create_clock_tree_source_groups`
- `cts_spec_config_create_generator_skew_groups`
- `cts_spec_config_create_generator_skew_groups_name_prefix`
- `cts_spec_config_create_reporting_only_skew_groups`
- `cts_spec_config_trace_multi_output_cells_with_single_clock_output`
- `cts_target_max_capacitance`
- `cts_target_max_capacitance_leaf`
- `cts_target_max_capacitance_top`
- `cts_target_max_capacitance_trunk`
- `cts_target_max_stage_delay_sigma`
- `cts_target_max_transition_time`
- `cts_target_max_transition_time_leaf`
- `cts_target_max_transition_time_top`
- `cts_target_max_transition_time_trunk`
- `cts_target_skew`
- `cts_timing_connectivity_based_skew_groups_balance_master_clocks`
- `cts_timing_connectivity_based_skew_groups`
- `cts_timing_connectivity_info`
- `cts_top_fanout_threshold`
- `cts_update_clock_latency`
- `cts_use_inverters`
- `cts_use_receiver_model_capacitance_for_drv`
- `flexible_htrees`
- `skew_groups`

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_cts_library_trace_through_toname value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `cts` category, use the following command:

```
get_db -category cts
```

Note: For attributes that are keyed, ensure that the design is loaded before using the `get_db attribute_name` command.

For more information about the CTS object attributes, see [Stylus Common UI Database Object Information](#). The following categories of CTS objects are available:

- [clock_tree](#)
- [clock_tree_source_group](#)
- [flexible_htree](#)
- [skew_group](#)

clock_trees

```
clock_trees {list | clock_tree}
```

Default: ""

Data_type: clock_tree*, read/write

Specifies a list of clock trees the pin is contained within. This includes parents of generated clock trees and all relevant parents when clock trees overlap.

Valid values: `list clock_tree`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```
- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `create_clock_tree`
- `create_clock_tree_spec`

clock_tree_source_groups

`clock_tree_source_groups clock_tree_source_group_list`

Default: ""

Data_type: `clock_tree_source_group*`, read-only

Specifies the list of clock tree source groups to which this clock tree belongs.

Valid values: `clock_tree_source_group_list`

Read-only: This attribute cannot be modified by `set_db`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `create_clock_tree`
- `create_clock_tree_spec`
- `create_clock_tree_source_group`

cts_add_port_driver_max_distance_from_port

`cts_add_port_driver_max_distance_from_port allowable_distance`

Default: `auto`

Data_type: string, read/write

This attribute specifies the allowable distance, in `um` if no units suffix is specified, that a port driver is

allowed to be placed from the port being buffered. If the driver cannot be added close enough to the port, it will not be added at all. This attribute, if set, overrides any value set using the attribute, `cts_add_port_driver_max_distance_from_port_rows`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
- To get the current value of any root attribute, use the `get_db` command:

```
set_db attribute_name value
```

```
get_db attribute_name
```

Related Commands

- `add_clock_tree_exclusion_drivers`
- `ccopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`

cts_add_port_driver_max_distance_from_port_rows

```
cts_add_port_driver_max_distance_from_port_rows allowable_cell_row_height_distance
```

Default: 50

Data_type: int, read/write

This attribute specifies the allowable distance, in standard cell row heights, that a port driver is allowed to be placed from the port being buffered. If the driver cannot be added close enough to the port it will not be added at all. The attribute `cts_add_port_driver_max_distance_from_port`, if set, overrides this value.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
- To get the current value of any root attribute, use the `get_db` command:

```
set_db attribute_name value
```

```
get_db attribute_name
```

Related Commands

- `add_clock_tree_exclusion_drivers`
- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`

cts_adjacent_rows_legal

`cts_adjacent_rows_legal {true | false}`

Default: `false`

Data_type: string, read/write

Defines the clock halo in the y direction. If set to true, the y direction clock halo is zero and other clock instances are allowed in adjacent rows. The following attributes can be used to assign y direction clock halos within CTS:

- `cts_cell_halo_y`
- `cts_adjacent_rows_legal`
- `cts_cell_halo_rows`

Only one of these attributes is used to determine the clock halo in the y direction. The following rules determine which:

- If `cts_cell_halo_y` is set to a non-auto value, then this defines the y direction clock halo. The attributes `cts_adjacent_rows_legal` and `cts_cell_halo_rows` have no effect.
- If `cts_cell_halo_y` is set to auto and `cts_adjacent_rows_legal` is set to a non-auto value then `cts_adjacent_rows_legal` defines the clock halo in the y direction. The attribute `cts_cell_halo_rows` has no effect.
- If both `cts_cell_halo_y` and `cts_adjacent_rows_legal` are set to auto then `cts_cell_halo_rows` defines the clock halo in the y direction.

Valid values: `true false`

See also:

```
. cts_cell_halo_y  
. cts_cell_halo_rows
```

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree_spec`
- `report_clock_tree_halo_violations`

cts_allow_non_std_clock_gate_inputs

```
cts_allow_non_std_clock_gate_inputs {true | false}
```

Default: false

Data_type: bool, read/write

If this attribute is set, CTS will allow the use of clock gates with non-standard pins. CTS considers the following pin types to be standard: clock pins, enable pins, test enable pins, retention pins, and power gating pins. Before starting clock tree synthesis, CTS will emit a warning, indicating which pin(s) it considers as non-standard.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree_spec`

cts_allow_post_route_sizing_stack_via_rule_update

```
cts_allow_post_route_sizing_stack_via_rule_update {resize_allow_svr_change |  
resize_dont_allow_svr_change}
```

Default: `resize_dont_allow_svr_change`

Data_type: string, read/write

Specifies whether to allow post-route resizing to make cell swaps, which require stack via rules (SVRs) to change, for a `clock_tree` pin. Enabling this feature may cause disruption to routed nets, which means ECO routing will have more to repair. By default, this feature is disabled.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```
- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree_spec`

cts_balance_mode

```
cts_balance_mode {full | cluster | trial}
```

Default: `full`

Data_type: string, read/write

Specifies the CTS mode setting to be used. If the default setting of `full` is not used, it causes CTS to halt before the final completion of the clock tree to facilitate clock tree inspection.

The possible values for this attribute are as follows:

- `full` - default value, a full CTS is performed.
- `cluster` - a cluster-only CTS is performed. The clock tree has no balancing delay applied.
- `trial` - The clock has only virtual (numeric annotation) balancing delays applied.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree_spec`

cts_blackbox_default_driver_base_pin

```
cts_blackbox_default_driver_base_pin pin_name
```

Default: ""

Data_type: string

Specifies the base pin that will be used for all timing and capacitance modeling for clock roots with a source pin at a blackbox output.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree_spec`

cts_blackbox_default_load_base_pin

`cts_blackbox_default_load_base_pin pin_name`

Default: ""

Data_type: string

Specifies the base pin that will be used for all timing and capacitance modeling for clock sinks at blackbox inputs. Any `cts_capacitance_override` attribute will be used in preference to this.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree_spec`

cts_buffer_cells

`cts_buffer_cells buffer_cell_list`

Default: ""

Data_type: string, read/write

Indices: power_domain

Specifies the buffer cells for CTS. If none are specified CTS will choose buffers from the libraries. Cell names may be specified as a Tcl list of names, or as a Tcl list of patterns to be expanded to match names. If set explicitly, CTS will ignore any don't use settings for the cells specified. Different buffer cells may be specified for any combination of clock tree and power domain.

To use different buffers for each net type set the `cts_buffer_cells_top` and `cts_buffer_cells_leaf` attributes.

Some examples are provided below:

To specify buffer cells for all clock trees and all power domains:

```
set_db cts_buffer_cells {bufAX* bufBX*}
```

To specify buffer cells for a particular clock tree and all power domains:

```
set_db clock_tree:<clk> .cts_buffer_cells {bufX20 bufX18}
```

To specify buffer cells for a particular clock tree and power domain:

```
set_db clock_tree:<clk> .cts_buffer_cells -index {power_domain <pd>} {bufX12 bufX8}
```

Valid values: a list of library cell names, or a list of patterns to expand to library cell names

Note: This attribute is applicable to object types, `clock_tree` and `power_domain`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree_spec`
- `report_cts_cell_filtering_reasons`

cts_buffer_cells_leaf

`cts_buffer_cells_leaf` *buffer_cell_list*

Default: ""

Data_type: string, read/write

Indices: power_domain

Specifies the buffer cells available for CTS to use on leaf nets. If none are specified, CTS will use the same buffers as on trunk nets - as specified in the `cts_buffer_cells` attribute. Cell names may be specified as a Tcl list of names, or as a Tcl list of patterns to be expanded to match names. If set explicitly, CTS will ignore any don't use settings for the cells specified.

Different leaf buffer cells may be specified for any combination of clock tree and power domain.

Valid values: library cell names, or a list of patterns to expand to library cell names

Note: This attribute is applicable to object types, `clock_tree` and `power_domain`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree_spec`
- `report_cts_cell_filtering_reasons`

cts_buffer_cells_top

`cts_buffer_cells_top` *buffer_cell_list*

Default: ""

Data_type: string, read/write

Indices: power_domain

Specifies the buffers cells available for CTS to use on top nets. If none are specified, CTS will use the same buffers as on trunk nets - as specified in the `cts_buffer_cells` attribute. Cell names may be specified as a Tcl list of names, or as a Tcl list of patterns to be expanded to match names. If set explicitly, CTS will ignore any don't use settings for the specified cells.

Different top buffer cells may be specified for any combination of clock tree and power domain.

Valid values: a list of library cell names, or a list of patterns to expand to library cell names

Note: This attribute is applicable to object types, `clock_tree` and `power_domain`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree_spec`
- `report_cts_cell_filtering_reasons`

cts_cell_density

```
cts_cell_density <value>
```

Default: 0.75

Valid Values: 0.01 to 1

Data_type: string, read/write

Specifies the clock halo in the x direction. Any x direction clock halo defined by this attribute is proportional to the cell width. The constant of proportionality is defined by the attribute value. For example:

- If `cts_cell_density = 0.25` then the x direction clock halo equals 3 * cell width.

- If `cts_cell_density = 0.5` then the x direction clock halo equals cell width.
- If `cts_cell_density = 0.75` then the x direction clock halo equals cell width / 3.
- If `cts_cell_density = 1` then the x direction clock halo equals zero.

This attribute can specify the x direction clock halo for all clock trees via:

```
set_db cts_cell_density 0.8
```

This attribute can specify the x direction clock halo per-clock tree via:

```
set_db clock_tree:<clk> .cts_cell_density 0.9
```

The following attributes can be used to assign x direction clock halos within CTS:

- `cts_cell_halo_x`
- `cts_cell_density`
- `cts_cell_halo_sites`

Only one of these attributes is used to determine the clock halo in the x direction. The following rules determine which:

- If `cts_cell_halo_x` is set to a non-auto value, then this defines the x direction clock halo. The attributes `cts_cell_density` and `cts_cell_halo_sites` have no effect.
- If `cts_cell_halo_x` is set to auto and `cts_cell_density` is set to a non-auto value then `cts_cell_density` defines the clock halo in the x direction. The attribute `cts_cell_halo_sites` has no effect.
- If both `cts_cell_halo_x` and `cts_cell_density` are set to auto then `cts_cell_halo_sites` defines the clock halo in the x direction.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```
- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree_spec`
- `report_clock_tree_halo_violations`

cts_cell_halo_mode

`cts_cell_halo_mode {max | sum}`

Default: `max`

Data_type: string, read/write

Specifies how clock halos are used to determine the minimum legal separation between a pair of clock instances. There are two possible modes 'max' and 'sum'. When set to 'max' the minimum legal separation is the larger of the two clock halos. When set to 'sum' the minimum legal separation is the sum of the two clock halos.

Valid values: `max` `sum`

See also:

- . `cts_cell_halo_x`
- . `cts_cell_halo_y`
- . `cts_cell_density`
- . `cts_adjacent_rows_legal`
- . `cts_cell_halo_sites`
- . `cts_cell_halo_rows`
- . `cts_effective_clock_halo_x`
- . `cts_effective_clock_halo_y`
- . `cts_effective_clock_halo_x_source`
- . `cts_effective_clock_halo_y_source`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree_spec`
- `report_clock_tree_halo_violations`

cts_cell_halo_rows

`cts_cell_halo_rows` value

Default: 1

Data_type: int, read/write

Specifies the clock halo in the y direction in rows for all clock cells.

The following attributes can be used to assign y direction clock halos within CCOpt:

- `cts_cell_halo_y`
- `cts_adjacent_rows_legal`
- `cts_cell_halo_rows`

Only one of these attributes is used to determine the clock halo in the y direction. The following rules determine which:

- If `cts_cell_halo_y` is set to a non-auto value, then this defines the y direction clock halo. The attributes `cts_adjacent_rows_legal` and `cts_cell_halo_rows` have no effect.
- If `cts_cell_halo_y` is set to auto and `cts_adjacent_rows_legal` is set to a non-auto value then `cts_adjacent_rows_legal` defines the clock halo in the y direction. The attribute `cts_cell_halo_rows` has no effect.
- If both `cts_cell_halo_y` and `cts_adjacent_rows_legal` are set to auto then `cts_cell_halo_rows` defines the clock halo in the y direction.

See also:

```
.cts_cell_halo_y
. cts_adjacent_rows_legal
```

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:


```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cconst_design`
- `clock_opt_design`
- `create_clock_tree_spec`
- `report_clock_tree_halo_violations`

cts_cell_halo_sites

```
cts_cell_halo_sites value
```

Default: 4

Data_type: int, read/write

Specifies the clock halo in the x direction in sites for all clock cells.

The following attributes can be used to assign x direction clock halos within CCOpt:

- `cts_cell_halo_x`
- `cts_cell_density`
- `cts_cell_halo_sites`

Only one of these attributes is used to determine the clock halo in the x direction. The following rules determine which:

- If `cts_cell_halo_x` is set to a non-auto value, then this defines the x direction clock halo. The attributes `cts_cell_density` and `cts_cell_halo_sites` have no effect.
- If `cts_cell_halo_x` is set to auto and `cts_cell_density` is set to a non-auto value then `cts_cell_density` defines the clock halo in the x direction. The attribute `cts_cell_halo_sites` has no effect.
- If both `cts_cell_halo_x` and `cts_cell_density` are set to auto then `cts_cell_halo_sites` defines the clock halo in the x direction.

See also:

. cts_cell_halo_x
. cts_cell_density

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree_spec`
- `report_clock_tree_halo_violations`

cts_cell_halo_x

`cts_cell_halo_x value`

Default: `auto`

Data_type: string, read/write

Indices: power_domain

Specifies the clock halo distance in the x direction. The default value of this attribute is `auto`. The following attributes can be used to assign x direction clock halos within CTS:

- `cts_cell_halo_x`
- `cts_cell_density`
- `cts_cell_halo_sites`

Only one of these attributes is used to determine the clock halo in the x direction. The following rules determine which:

- If `cts_cell_halo_x` is set to a non-`auto` value, then this defines the x direction clock halo. The attributes `cts_cell_density` and `cts_cell_halo_sites` have no effect.

- If `cts_cell_halo_x` is set to auto and `cts_cell_density` is set to a non-auto value then `cts_cell_density` defines the clock halo in the x direction. The attribute `cts_cell_halo_sites` has no effect.
- If both `cts_cell_halo_x` and `cts_cell_density` are set to auto then `cts_cell_halo_sites` defines the clock halo in the x direction.

See also:

`.cts_cell_density`
`.cts_cell_halo_sites`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree_spec`
- `report_clock_tree_halo_violations`

cts_cell_halo_y

`cts_cell_halo_y value`

Default: `auto`

Data_type: string, read/write

Indices: power_domain

Specifies the clock halo distance in the y direction. The default value of this attribute is auto. The following attributes can be used to assign y direction clock halos within CTS:

- `cts_cell_halo_y`
- `cts_adjacent_rows_legal`

- `cts_cell_halo_rows`

Only one of these attributes is used to determine the clock halo in the y direction. The following rules determine which:

- If `cts_cell_halo_y` is set to a non-auto value, then this defines the y direction clock halo. The attributes `cts_adjacent_rows_legal` and `cts_cell_halo_rows` have no effect.
- If `cts_cell_halo_y` is set to auto and `cts_adjacent_rows_legal` is set to a non-auto value then `cts_adjacent_rows_legal` defines the clock halo in the y direction. The attribute `cts_cell_halo_rows` has no effect.
- If both `cts_cell_halo_y` and `cts_adjacent_rows_legal` are set to auto then `cts_cell_halo_rows` defines the clock halo in the y direction.

See also:

. `cts_adjacent_rows_legal`
. `cts_cell_halo_rows`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `cconst_design`
- `clock_opt_design`
- `create_clock_tree_spec`
- `report_clock_tree_halo_violations`

cts_clock_gating_cells

`cts_clock_gating_cells cell_list`

Default: ""

Data_type: string, read/write

Indices: power_domain

Specifies the clock gates for CTS. If none are specified CTS will choose clock gates from the libraries. Cell names may be specified as a Tcl list of names, or as a Tcl list of patterns to be expanded to match names. If set explicitly, CTS will ignore any do not use settings for the cells specified.

A list of lists can be used to specify separate clock gate sub-lists. All CTS transforms will resize clock gates within the clock gate sub-list they are initially part of. If an incoming clock gate is not part of any clock gate sub-list, it will be resized to the first clock gate sub-list.

Different clock gates may be specified for any combination of clock tree and power domain.

Some examples are provided below:

To specify clock gates for all clock trees and all power domains:

```
set_db cts_clock_gating_cells {cgAX* cgBX*}
```

To specify clock gates for a particular clock tree and all power domains:

```
set_db clock_tree:<clk> .cts_clock_gating_cells {cgX20 cgX18}
```

To specify clock gates for a particular clock tree and power domain:

```
set_db clock_tree:<clk> .cts_clock_gating_cells -index {power_domain <pd>} {cgX12 cgX8}
```

To specify separate clock gate sub-lists:

```
set_db .cts_clock_gating_cells {{cgX12A1 cgX8A1} {cgX12A2 cgX8A2}}
```

Valid values: a list of library cell names, or a list of patterns to expand to library cell names, a list of lists to specify separate clock gate sub-lists

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cconst_design`
- `clock_opt_design`
- `create_clock_tree_spec`
- `merge_clock_tree_instances`

cts_clock_gate_movement_limit

`cts_clock_gate_movement_limit` *value*

Default: 10

Data_type: string, read/write

Each clock gate is restricted to a Manhattan ball centered on its original location with the CTS flow. The radius of the ball is a multiple of the clock gate height. This attribute controls the default value of that multiple.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `cconst_design`
- `clock_opt_design`
- `create_clock_tree_spec`
- `merge_clock_tree_instances`

cts_clock_source_cells

`cts_clock_source_cells` *list_of_cells*

Default: ""

Data_type: string, read/write

Indices: power_domain

Specifies the list of cells eligible to resize the instance whose driver pin is the clock tree root pin if the attribute `cts_size_clock_sources` is set to `true`. Sizing of clock tree roots is not enabled by default in CTS since those instances are not part of the clock tree network.

If set explicitly, CTS will ignore any don't use settings for the specified cells.

Different cells may be specified for clock trees or power domains. Only clock sources that are buffers, inverters, logic, and clock gating cells with a single output can be resized.

Valid values: a list of library cell names, or a list of patterns to expand to library cell names

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `add_clock_tree_source_group_roots`
- `assign_clock_tree_source_groups`
- `ccopt_design`
- `clock_opt_design`
- `create_clock_tree_spec`

cts_clock_tree_source_group_clock_trees

`cts_clock_tree_source_group_clock_trees clock_tree_list`

Default: ""

Data_type: string, read/write

A list of the clock trees relevant to this source group.

Valid values: `list clock_trees`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `add_clock_tree_source_group_roots`
- `assign_clock_tree_source_groups`
- `ccopt_design`
- `clock_opt_design`
- `create_clock_tree_spec`

cts_clock_tree_source_input_max_transition_time

```
cts_clock_tree_source_input_max_transition_time {double | 0}
```

Default: 0

Data_type: double, read/write

Indices: delay_corner

Specifies the slew that will be assumed at the input of the root driver.

Valid values: `double | 0`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```


Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree_spec`
- `get_clock_tree_transition_time`

cts_cloning_effort

`cts_cloning_effort {standard | high}`

Default: `high`

Data_type: string

Specifies the amount of cloning during clustering. By default, the cloning effort is `high`, which uses clones wherever possible. When `standard` is specified, clock gate cells and logic cells that drive leaf nets will not be cloned.

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree_spec`

cts_cloning_inst_name_suffix

`cts_cloning_inst_name_suffix suffix_name`

Default: `clone`

Data_type: string

If set, the suffix will be used generally for all insts cloned by CTS, else the suffix will default to "clone".

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cconst_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`

cts_cloning_inst_name_suffix_source_group_assignment

```
cts_cloning_inst_name_suffix_source_group_assignment suffix_name
```

Default: ""

Data_type: string, read/write

Specifically controls the suffix used for multi tap cloning. If this is set to a non-empty string, that string will be used as a suffix when naming cloned instances created by multi-tap cloning. Otherwise, the name suffix will be controlled by the `cts_cloning_inst_name_suffix` attribute.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cconst_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`

cts_declone_clock_gates

`cts_declone_clock_gates {true | false | cts_only | ccopt_only}`

Default: `ccopt_only`

Data_type: string, read/write

This attribute controls clock gate decloning (merging) in both CTS-only and full `clock_opt_design` (or ECF) flows.

Valid values:

- `false` - decloning is disabled in all flows
- `cts_only` - decloning is enabled for CTS-only flows
- `ccopt_only` - decloning is enabled for full flows
- `true` - decloning is enabled for all flows

See also attribute `cts_declone_clock_logic`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `ccopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`

cts_declone_clock_logic

`cts_declone_clock_logic {true | false | cts_only | ccopt_only}`

Default: `ccopt_only`

Data_type: string, read/write

This attribute controls clock logic decloning (merging) in both CTS-only and full clock_opt_design (or ECF) flows.

Valid values:

- `false` - decloning is disabled in all flows
- `cts_only` - decloning is enabled for CTS-only flows
- `cchopt_only` - decloning is enabled for full flows
- `true` - decloning is enabled for all flows

See also attribute `cts_declone_clock_gates`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`

cts_delay_cells

`cts_delay_cells cell_list`

Default: ""

Data_type: string, read/write

Indices: power_domain

Specifies the delay cells available for CTS. If none are specified CTS will not use delay cells. Setting this attribute to the string 'auto' means that CTS will choose delay cells from the

libraries to use. Cell names may be specified as a Tcl list of names, or as a Tcl list of patterns to be expanded to match names.

If set explicitly, CTS will ignore any don't use settings for the cells specified. Different delay cells may be specified for any combination of clock tree and power domain, or by omitting those arguments a global setting can be applied.

Some examples are provided below:

To specify delay cells for all clock trees and power domains:

```
set_db cts_delay_cells {delayAX* delayBX*}
```

To specify delay cells for a particular clock tree and all power domains:

```
set_db clock_tree:<clk> .cts_delay_cells {delayX1 delayX2}
```

To specify delay cells for a particular clock tree and power domain:

```
set_db clock_tree:<clk> .cts_delay_cells -index {power_domain <pd>} {delayX2 delayX3}
```

Valid values: a list of library cell names, or a list of patterns to expand to library cell names, or the string 'auto'

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`

cts_detailed_cell_warnings

```
cts_detailed_cell_warnings {true | false}
```

Default: false

Data_type: bool, read/write

If set to `true`, CTS outputs detailed cell warning diagnostics when it encounters issues with library cell selection, power domains and/or signal levels.

Valid values: `true`, `false`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`

cts_exit_if_no_placeable_area

```
cts_exit_if_no_placeable_area {true | false}
```

Default: `true`

Data_type: bool, read/write

End CTS during initialization if there is insufficient placeable area.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`

cts_exit_if_skew_target_over_constrained

`cts_exit_if_skew_target_over_constrained {true | false}`

Default: `true`

Data_type: `bool`, read/write

End CTS during initialization if the skew target is too low.

Valid values: `true`, `false`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`

cts_exit_if_stage_delay_sigma_target_over_constrained

`cts_exit_if_stage_delay_sigma_target_over_constrained {true | false}`

Default: `true`

Data_type: bool, read/write

If set, allows any max stage delay sigma target to be set. If the target is too low, it is likely to lead to runtime problems.

Valid values: `true`, `false`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`

cts_fix_clock_sinks

```
cts_fix_clock_sinks {true | false | soft}
```

Default: `false`

Data_type: string, read/write

If set to `true`, we will DEF lock clock tree sinks after routing in addition to any clock node locking (fixed).

If set to `soft`, we will DEF lock clock tree sinks after routing in addition to any clock node `soft_locking` (softFixed).

Valid values: `true` `false` `soft`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```


- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `check_clock_tree_convergence`
- `create_clock_tree`
- `create_generated_clock_tree`

cts_flexible_htree_allow_mixed_sink_polarity

`cts_flexible_htree_allow_mixed_sink_polarity {true | false}`

Default: `false`

Data_type: `bool`, read/write

When this attribute is set to `true`, the software is allowed to create flexible H-trees with mixed sink polarities for sink pins and final cells, where the polarity is honored for the final cell.

Valid values: `true false`

cts_flexible_htree_enable_metrics_reporting

`cts_flexible_htree_enable_metrics_reporting {true | false | post_route}`

Default: `post_route`

Data_type: `string`, read/write

Controls the reporting of metrics for flexible H-trees. The following values are allowed:

- `true`: Enables all reporting.
- `false`: Disables all reporting.
- `post_route`: Enables only the report at the end of flexible H-Tree synthesis.

Valid values: `true false post_route`

cts_flexible_htree_placement_legalization_effort

`cts_flexible_htree_placement_legalization_effort {high | low}`

Default: low

Data_type: string, read/write

The legalization effort for finding placement unblocked points on the synthesis grid for flexible H-trees. High placement legalization effort can avoid having to relax placement constraints when implementing H-trees but may lead to increased runtime.

Valid values: high low

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `convert_lib_clock_tree_latencies`
- `create_clock_tree`
- `get_clock_tree_effective_max_capacitance`

cts_force_target_max_transition_time

`cts_force_target_max_transition_time {true | false}`

Default: false

Data_type: bool, read/write

Specifies that CTS should allow any transition target to be used, even if it is likely to lead to runtime problems. By default, CTS computes a minimum transition target which should not lead to runtime problems, and does not allow any transition target to be set lower than this minimum. This attribute overrides that check, and allows any transition target to be used.

Valid values: `true`, `false`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `convert_lib_clock_tree_latencies`
- `create_clock_tree`
- `get_clock_tree_effective_max_capacitance`

cts_ignore_problematic_skew_as_result_of_dont_touch_nets

```
cts_ignore_problematic_skew_as_result_of_dont_touch_nets {true | false}
```

Default: `false`

Data_type: `bool`, read/write

If set to `true`, sinks directly connected to nets that are causing unfixable skew problems will be ignored for skew balancing.

Valid values: `true`, `false`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_skew_group`
- `get_skew_group_delay`
- `get_skew_group_path`
- `report_skew_groups`
- `update_skew_group`

cts_inst_name_prefix

`cts_inst_name_prefix` *prefix_name*

Default: CTS

Data_type: string, read/write

The name prefix of instances/nets created by CTS. The default value is "CTS". The default names of nets are CTS, CTS_1, CTS_2... ..The default names of instances are CTS_*.

Valid values: `string`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `cconst_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`

cts_inverter_cells

`cts_inverter_cells inverter_cell_list`

Default: ""

Data_type: string, read/write

Indices: power_domain

Specifies the inverter cells available for CTS. If none are specified CTS will choose inverters from the libraries. Cell names may be specified as a Tcl list of names, or as a Tcl list of patterns to be expanded to match names. If set explicitly, CTS will ignore any don't use settings for the cells specified. Different inverter cells may be specified for any combination of clock tree and power domain.

To use different inverters for each net type set the `cts_inverter_cells_top` and `cts_inverter_cells_leaf` attributes.

Some examples are provided below:

To specify inverter cells for all clock trees and all power domains:

```
set_db cts_inverter_cells {invAX* invBX*}
```

To specify inverter cells for a particular clock tree and all power domains:

```
set_db clock_tree:<clk> .cts_inverter_cells {invX20 invX18}
```

To specify inverter cells for a particular clock tree and power domain:

```
set_db clock_tree:<clk> .cts_inverter_cells -index {power_domain <pd>} {invX12 invX8}
```

Valid values: a list of library cell names, or a list of patterns to expand to library cell names

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `add_clock_tree_exclusion_drivers`
- `add_clock_tree_source_group_roots`
- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`

cts_inverter_cells_leaf

```
cts_inverter_cells_leaf inverter_cell_list
```

Default: ""

Data_type: string, read/write

Indices: power_domain

Specifies the inverter cells available for CTS to use on leaf nets. If none are specified, CTS will use the same inverters as on trunk nets - as specified in the `cts_inverter_cells` attribute. Cell names may be specified as a Tcl list of names, or as a Tcl list of patterns to be expanded to match names. If set explicitly, CTS will ignore any don't use settings for the cells specified.

Different leaf inverter cells may be specified for any combination of clock tree and power domain.

Valid values: a list of library cell names, or a list of patterns to expand to library cell names

Note: This attribute is applicable to object types, `clock_tree` and `power_domain`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `add_clock_tree_exclusion_drivers`
- `add_clock_tree_source_group_roots`
- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`

cts_inverter_cells_top

`cts_inverter_cells_top cell_list`

Default: ""

Data_type: string, read/write

Indices: power_domain

Specifies the inverter cells available for CTS to use on top nets. If none are specified, CTS will use the same inverters as on trunk nets - as specified in the `cts_inverter_cells` attribute. Cell names may be specified as a Tcl list of names, or as a Tcl list of patterns to be expanded to match names. If set explicitly, CTS will ignore any don't use settings for the cells specified.

Different top inverter cells may be specified for any combination of clock tree and power domain.

Valid values: a list of library cell names, or a list of patterns to expand to library cell names

Note: This attribute is applicable to object types, `clock_tree` and `power_domain`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `add_clock_tree_exclusion_drivers`
- `add_clock_tree_source_group_roots`
- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`

cts_load_capacitance_cells

`cts_load_capacitance_cells` *list_of_cellnames*

Default: ""

Data_type: string, read/write

Indices: power_domain

Specifies the list of cells that are available for CTS to add load to a net to optimize capacitance. Cell names may be specified as a Tcl list of names, or as a Tcl list of patterns to be expanded to match names.

Note: This attribute is applicable to object types, `clock_tree` and `power_domain`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```


Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`

cts_logic_cells

`cts_logic_cells clock_logic_cell_list`

Default: ""

Data_type: string, read/write

Indices: power_domain

Specifies the clock logics for CTS. If none are specified CTS will choose clock logics from the libraries. Cell names may be specified as a Tcl list of names, or as a Tcl list of patterns to be expanded to match names. If set explicitly, CTS will ignore any don't use settings for the cells specified. Different logic cells may be specified for any combination of clock tree and power domain.

Some examples are provided below:

To specify logic cells for all clock trees and all power domains:

```
set_db cts_logic_cells {and* mux*}
```

To specify logic cells for a particular clock tree and all power domains:

```
set_db clock_tree:<clk> .cts_logic_cells {andX20 andX18}
```

To specify logic cells for a particular clock tree and power domain:

```
set_db clock_tree:<clk> .cts_logic_cells -index {power_domain <pd>} {andX12 andX8}
```

Valid values: a list of library cell names, or a list of patterns to expand to library cell names

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```
- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`
- `create_clock_tree_source_group`
- `merge_clock_tree_instances`

cts_manage_power_intent_violations

```
cts_manage_power_intent_violations {true | false}
```

Default: `true`

Data_type: `bool`, read/write

If this attribute is set, the CTS algorithm will work around power management illegalities in the clock tree, as opposed to failing with an error when it encounters them. This allows the clock tree to be synthesized, but any power management illegalities will remain in the exported design.

Valid values: `true`, `false`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```
- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`

cts_max_fanout

`cts_max_fanout` *number*

Default: 100

Data_type: int, read/write

The maximum fanout at any point in the clock tree.

Valid values: integer ranged between 2 and 1000 inclusive

Note: This attribute is applicable to object type, `base_pin`. For detailed help of the attribute in the tool, type `help <obj_type> <attribute_name>`. For example:

```
help base_pin cts_max_fanout
```

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```
- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`

cts_max_instance_height

`cts_max_instance_height` *value*

Default: 3

Data_type: double, read/write

When selecting cells for use, CTS will filter the selection to remove all buffers, inverters and clock gates taller than this many row heights. This helps quickly eliminate very large cells that are unlikely to be useful.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `assign_clock_tree_source_groups`
- `ccopt_design`
- `clock_opt_design`
- `create_clock_tree`

cts_max_source_to_sink_net_length

```
cts_max_source_to_sink_net_length {auto | double}
```

Default: top auto trunk auto leaf auto

Data_type: string, read/write

The maximum routing length in microns between driving source pin and driven sink pin on each net that clock tree synthesis should observe. This constraint can be applied to either a pin, a clock tree, or a net type. By default (if this attribute is not set) no explicit clock tree net length constraint is enforced. However, other clock tree constraints such as maximum slew (transition) and maximum capacitance will indirectly limit the maximum net length.

Valid values: double

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `assign_clock_tree_source_groups`
- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_source_group`

cts_max_source_to_sink_net_length_leaf

```
cts_max_source_to_sink_net_length_leaf {auto | double}
```

Default: auto

Data_type: string, read/write

The maximum routing length in microns between driving source pin and driven sink pin on each net that clock tree synthesis should observe. This constraint can be applied to either a pin, a clock tree, or a net type. By default (if this attribute is not set) no explicit clock tree net length constraint is enforced. However, other clock tree constraints such as maximum slew (transition) and maximum capacitance will indirectly limit the maximum net length. This attribute only applies to leaf net type.

Valid values: auto | double

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `assign_clock_tree_source_groups`
- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_source_group`

cts_max_source_to_sink_net_length_top

`cts_max_source_to_sink_net_length_top {auto | double}`

Default: `auto`

Data_type: string, read/write

The maximum routing length in microns between driving source pin and driven sink pin on each net that clock tree synthesis should observe. This constraint can be applied to either a pin, a clock tree, or a net type. By default (if this attribute is not set) no explicit clock tree net length constraint is enforced. However, other clock tree constraints such as maximum slew (transition) and maximum capacitance will indirectly limit the maximum net length. This attribute only applies to top net type.

Valid values: `auto | double`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `assign_clock_tree_source_groups`
- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_source_group`

cts_max_source_to_sink_net_length_trunk

`cts_max_source_to_sink_net_length_trunk {auto | double}`

Default: `auto`

Data_type: string, read/write

The maximum routing length in microns between driving source pin and driven sink pin on each net that clock tree synthesis should observe. This constraint can be applied to either a pin, a clock tree, or a net type. By default (if this attribute is not set) no explicit clock tree net length constraint is enforced. However, other clock tree constraints such as maximum slew (transition) and maximum capacitance will indirectly limit the maximum net length. This attribute only applies to trunk net type.

Valid values: `auto | double`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

Related Commands

- `assign_clock_tree_source_groups`
- `cconst_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_source_group`

cts_mixed_fanout_net_type

`cts_mixed_fanout_net_type` {leaf | trunk}

Default: leaf

Data_type: string, read/write

Controls how CTS considers nets that have fanout consisting partially but not entirely of sinks. By default, having any sinks, for example DFFs, will make the net be considered leaf, but when set to trunk, a net has to drive only sinks to be considered as leaf. For example, a net driving a clock gate and a DFF would no longer count as a leaf net.

Valid values: leaf trunk

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `cconst_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`

cts_move_clock_gates

`cts_move_clock_gates {true | false}`

Default: `true`

Data_type: `bool`, read/write

If this attribute is set, the CTS algorithm will move clock gates that appear in the clock tree. Setting this attribute may cause the clock tree to have a lower insertion delay, but might break datapath timing. During optimization, this is not a problem, because the timing will be automatically recovered during the optimization process. If this attribute is false, CTS permits small movements of ICGs for legalization.

Note: The `cts_move_clock_gates` attribute indirectly controls multi-tap CTS because the logic is that if no integrated clock gates (ICGs) can be moved then multi-tap cloning is also disabled. So, it is recommend to not use `cts_move_clock_gates false` for multi-tap CTS.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `merge_clock_tree_instances`

cts_move_logic

`cts_move_logic {true | false}`

Default: `true`

Data_type: `bool`, read/write

If this attribute is set, the CTS algorithm will move logic that appears in the clock tree. "Logic" does

not include clock gates, buffers, and inverters in a clock tree, which are always moved unless they are locked, or clock generators that are above the root of the clock tree. Usually, this will affect multiplexers used for selecting one of a number of clocks, or for switching between a test clock and the main clock. Setting this attribute may cause the clock tree to have a lower insertion delay, but might break datapath timing. During optimization, this isn't a problem, because the timing will be automatically recovered during the optimization process.

Valid values: `true`, `false`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `merge_clock_tree_instances`

cts_net_name_prefix

`cts_net_name_prefix` *string*

Default: `CTS`

Data_type: string, read/write

The name prefix of instances/nets created by CTS. The default value is "CTS". The default names of nets are CTS, CTS_1, CTS_2... ..The default names of instances are CTS_*.

Valid values: `string`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`

cts_pin_insertion_delay_histogram_bin_size

```
pin_insertion_delay_histogram_bin_size {auto | time value in library units}
```

Default: auto

Data_type: string, read/write

This attribute specifies the preferred size of the histogram bins that should be used when reporting the distribution of pin insertion delays. The bin size is specified in time units and must be greater than zero. A value of auto (the default) leaves it to the tool to calculate a bin size.

This attribute affects pin insertion delay histograms that are logged during various flow commands. It also determines the preferred histogram bin size that is used by the `report_pin_insertion_delays` command when an explicit `-bin_size` argument has not been specified.

Valid values: auto, time value in library units

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```
- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `get_skew_group_delay`
- `reset_cts_config`
- `report_pin_insertion_delays`

cts_post_route_enable

`cts_post_route_enable {true | false}`

Default: `true`

Data_type: `bool`, read/write

Enable post-conditioning optimization after clock nets are routed.

Valid values: `true false`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`

cts_post_route_repair_drv

`cts_post_route_repair_drv {true | false}`

Default: `true`

Data_type: `bool`, read/write

If set to `false`, post-conditioning will skip its DRV fixing step.

Valid values: `true false`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

`set_db attribute_name value`

- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `clock_post_route_repair`
- `create_clock_tree`
- `create_clock_tree_spec`

cts_post_route_enable_routing_eco

`cts_post_route_enable_routing_eco {true | false}`

Default: `false`

Data_type: `bool`, read/write

If set to `false`, post-conditioning will skip its ECO-routing step.

Valid values: `true false`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cconst_design`
- `clock_opt_design`
- `clock_post_route_repair`
- `create_clock_tree`
- `create_clock_tree_spec`

cts_post_route_repair_skew_by_buffering

```
cts_post_route_repair_skew_by_buffering {true | false}
```

Default: `false`

Data_type: `bool`, read/write

If set to `true`, post-conditioning will attempt skew-fixing using rebuffering.

Valid values: `true false`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `clock_post_route_repair`
- `create_clock_tree`
- `create_clock_tree_spec`

cts_primary_delay_corner

`cts_primary_delay_corner corner_name`

Default: ""

Data_type: string, read/write

This specifies the delay corner in which clock tree balancing applies the slew and insertion delay targets. If more than one timing corner is defined, this must be set before running CTS. By default, this is set to the first defined delay corner.

Valid values: `corner name`, or `empty`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`

cts_primary_reporting_skew_groups

`cts_primary_reporting_skew_groups` *list of skew groups* | auto | none

Default: auto

Data_type: string, read/write

Specifies the primary skew groups used for reporting. By default, the value is specified as auto that automatically takes the skew group with maximum number of sinks as the primary reporting skew group. For invalid values, the default (auto) will be considered.

Valid values: a list of existing skew group names, auto or none

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- [ccopt_design](#)
- [clock_opt_design](#)
- [create_clock_tree](#)
- [create_clock_tree_spec](#)
- [create_skew_group](#)
- [get_skew_group_delay](#)
- [report_pin_insertion_delays](#)
- [report_skew_groups](#)
- [update_skew_group](#)

cts_primary_reporting_skew_groups_log_min_max_sinks

`cts_primary_reporting_skew_groups_log_min_max_sinks` on | off | logv

Default: `logv`

Data_type: string, read/write

If set to `on`, the sinks with the shortest and longest paths in each primary reporting skew group will be logged. If set to `logv`, they will be logged only to the `logv` file.

Valid values: `on`, `off`, `logv`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`
- `create_skew_group`
- `get_skew_group_delay`
- `report_pin_insertion_delays`
- `report_skew_groups`
- `update_skew_group`

cts_repair_congestion

`cts_repair_congestion {true | false}`

Default: `true`

Data_type: bool, read/write

If set, `place_fix_congestion` is called during CTS. This can reduce congestion and ease later

routing issues.

Valid values: `true false`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`

cts_repair_drv_by_buffering

`cts_repair_drv_by_buffering {true | false}`

Default: `false`

Data_type: `bool`, read/write

If set to `true`, post-conditioning will fix DRVs by adding buffers.

Valid values: `true false`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `clock_post_route_repair`
- `create_clock_tree`
- `create_clock_tree_spec`

cts_report_skew_groups_only_with_targets

`cts_report_skew_groups_only_with_targets {true | false}`

Default: false

Data_type: bool, read/write

The skew groups report (run using the `report_skew_groups` command) displays insertion delay, skew, and min/max path information for different combinations of skew group, timing corner, and early/late path. Set the `cts_report_skew_groups_only_with_targets` attribute to false (the default) to report on all skew group/timing corner/path combinations regardless of whether a skew target has been set. Set the `cts_report_skew_groups_only_with_targets` attribute to true to report only on skew group/timing corner/path combinations where a skew target has been set (either explicitly or using the 'auto' setting).

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `cconst_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`
- `create_skew_group`
- `get_skew_group_delay`
- `report_pin_insertion_delays`
- `report_skew_groups`
- `update_skew_group`

cts_route_clock_tree_nets

`cts_route_clock_tree_nets {true | false}`

Default: `true`

Data_type: `bool`, read/write

Specifies that detailed routing be performed during the final implementation clock routing phase.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`
- `create_route_type`

cts_route_type_flexible_htree

`cts_route_type_flexible_htree route_type`

Default: default

Data_type: string, read/write

Specifies the `route_type` to be used when routing flexible H-trees. If this attribute is not set, the value of 'route_type_top' is used.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`
- `create_route_type`

cts_route_type_flexible_htree_backside

`cts_route_type_flexible_htree_backside route_type`

Default: default

Data_type: string, read/write

Specifies the `route_type` to be used for routing flexible H-trees, when H-trees are routed on backside layers. This attribute must be set to enable H-Tree routing on backside layers.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`
- `create_route_type`

cts_route_type_leaf

`cts_route_type_leaf {route_type | default}`

Default: default

Data_type: string, read/write

Specifies the route type. Setting this attribute binds an existing user-defined `route_type` to one or more types of clock tree nets. Binding a `route_type` to a type of clock tree nets means that all nets of that type (including the nets created by CTS) will be routed according to the specification of that `route_type`. In the most common usage, the `route_type` is bound to one of the three types of clock tree nets (top, trunk, or leaf) with the optional `-net_type` argument. Omitting the `-net_type` argument causes the `route_type` to be bound to all three types of clock tree nets. The optional -

`clock_tree <pattern>` argument limits the binding to the clock trees whose name matches `<pattern>`. Omitting the `-clock_tree` argument causes the binding to apply to all clock trees.

For a `route_type` to be used in CTS, it must be bound to at least one net type. If net type is not bound to any `route_type`, a default `route_type` will be created for that net type at the start of CTS.

Valid values: names of `route_types` created with `create_route_type`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`
- `create_route_type`

cts_route_type_top

```
cts_route_type_top {route_type | default}
```

Default: default

Data_type: string, read/write

Specifies the route type. Setting this attribute binds an existing user-defined `route_type` to one or more types of clock tree nets. Binding a `route_type` to a type of clock tree nets means that all nets of that type (including the nets created by CTS) will be routed according to the specification of that `route_type`. In the most common usage, the `route_type` is bound to one of the three types of clock tree nets (top, trunk, or leaf) with the optional `-net_type` argument. Omitting the `-net_type` argument causes the `route_type` to be bound to all three types of clock tree nets. The optional `-clock_tree <pattern>` argument limits the binding to the clock trees whose name matches `<pattern>`. Omitting the `-clock_tree` argument causes the binding to apply to all clock trees.

For a `route_type` to be used in CTS, it must be bound to at least one net type. If net type is not bound to any `route_type`, a default `route_type` will be created for that net type at the start of CTS.

Valid values: names of `route_types` created with `create_route_type`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`
- `create_route_type`

cts_route_type_trunk

```
cts_route_type_trunk {route_type | default}
```

Default: default

Data_type: string, read/write

Specifies the route type. Setting this attribute binds an existing user-defined `route_type` to one or more types of clock tree nets. Binding a `route_type` to a type of clock tree nets means that all nets of that type (including the nets created by CTS) will be routed according to the specification of that `route_type`. In the most common usage, the `route_type` is bound to one of the three types of clock tree nets (top, trunk, or leaf) with the optional `-net_type` argument. Omitting the `-net_type` argument causes the `route_type` to be bound to all three types of clock tree nets. The optional `-clock_tree <pattern>` argument limits the binding to the clock trees whose name matches `<pattern>`. Omitting the `-clock_tree` argument causes the binding to apply to all clock trees.

For a `route_type` to be used in CTS, it must be bound to at least one net type. If net type is not bound to any `route_type`, a default `route_type` will be created for that net type at the start of CTS.

Valid values: names of route_types created with `create_route_type`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cconst_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`
- `create_route_type`

cts_routing_preferred_layer_effort

```
cts_routing_preferred_layer_effort {standard | high | auto}
```

Default: standard

Data_type: string, read/write

Controls preferred layer effort for clock routing. The following settings may be specified:

- `standard` (default) - No change made to current early global route settings.
- `high` - enables a package of early global route settings to increase layer adherence, possibly at the expense of wire length and via count.
- `auto` - maps to `high` effort only if the effective routing effort level for the vast majority of nets of `net_type trunk`, for all clock trees resolves to `high`. Otherwise, the `auto` setting maps to `standard`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cconst_design`
- `clock_opt_design`
- `commit_clock_tree_route_attributes`
- `create_clock_tree`
- `create_clock_tree_spec`
- `create_route_type`

cts_safety_tmr_isolation_route_type

```
cts_safety_tmr_isolation_route_type {route_type | default}
```

Default: default

Data_type: string, read/write

Sets a route type for nets driven by triple modular redundancy (TMR) buffers or inverters. These should be short single fanout nets and this route type is expected to be easily routable, for example, with minimal shielding and spacing requirements. If the route type is not specified, the `route_type` specified for the leaf `net_type` will be used.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_route_type`

cts_size_clock_gates

`cts_size_clock_gates {true | false}`

Default: `true`

Data_type: `bool`, read/write

When set to `true` (the default), the CTS algorithm sizes clock gates that appear in the clock tree. Setting this attribute may cause the clock tree to have a lower insertion delay, but might change the cell types of logic gates in the clock tree, which in turn may require them to be moved slightly to find a legal location for the new cell.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `assign_clock_tree_source_groups`
- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`
- `merge_clock_tree_instances`
- `report_cts_cell_filtering_reasons`

cts_size_clock_sources

`cts_size_clock_sources {true | false}`

Default: `false`

Data_type: bool, read/write

When set to true, CTS will try to size the clock source. Only clock sources that are buffers, inverters, logic and clock gating cells with a single output will be sized. The cells available for CTS to size clock sources can be set specified using the attribute, `cts_clock_source_cells`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `add_clock_tree_source_group_roots`
- `assign_clock_tree_source_groups`
- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`

cts_size_logic

`cts_size_logic {true | false}`

Default: `true`

Data_type: bool, read/write

When set to `true` (the default), the CTS algorithm sizes logic that appears in the clock tree. "Logic" does not include clock gates, buffers, and inverters in a clock tree, which are always sized unless they are locked, or clock generators that are above the root of the clock tree. Usually, this

affects multiplexers used for selecting one of a number of clocks, or for switching between a test clock and the main clock. Setting this attribute may cause the clock tree to have a lower insertion delay, but might change the cell types of logic gates in the clock tree, which in turn may require them to be moved slightly to find a legal location for the new cell.

Valid values: `true`, `false`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `assign_clock_tree_source_groups`
- `cconst_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`
- `merge_clock_tree_instances`
- `report_cts_cell_filtering_reasons`

cts_skew_group_report_columns

```
cts_skew_group_report_columns list_of_columns
```

Default: `name lib_cell event increment time slew capacitance location distance fanout status voltage`

Data_type: `string`, `read/write`

A Tcl list of columns to include in skew group reports produced by the skew group report. You can use this attribute to specify the columns you would like to include in the skew group report, and the order in which the columns should appear. Most of the legal values are straightforward. However, the set of legal values of the following form deserve further explanation:

```
summaryType_summaryLocation[_event]
```

These values let you report the delay value for other paths that go through the pin.

`summaryType` is one of: max (show the longest delay), min (show the shortest delay), or skew (show the skew, that is the difference between the longest and the shortest delay).

`summaryLocation` is one of: above (show the delay/skew above this pin), below (show the delay/skew below this pin), or through (show the delay/skew for paths through this pin).

`event` is one of: rise (show the delay/skew for the rise event at this pin), fall (show the delay/skew for the fall event at this pin), or both (show the delay/skew for both events at this pin).

Valid values:

capacitance
distance
event
fanout
increment
lib_cell
location
name
slew
status
time

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`
- `report_skew_groups`

cts_skew_group_report_histogram_bin_size

`cts_skew_group_report_histogram_bin_size {auto | string}`

Default: auto

Data_type: string, read/write

When set to a numeric value, that numeric value will be used as the histogram range size (in library units). For example, if the library time units are set to 1 nanosecond, a value of 0.010 for

`cts_skew_group_report_histogram_bin_size` will result in histogram ranges of 10 picoseconds.

When set to auto, the size of the histogram ranges are dependent on the skew targets that are set. If a skew target is set for a given half corner and skew group combination, then the histogram range size will be 10% of the skew target for that half corner and skew target combination. If no skew target is set for a half corner and skew group combination but a skew target is set for the primary half corner and skew group combination, then the histogram range size will be 10% of the skew target for the primary half corner and skew group combination.

In the event that no skew targets are set and `cts_skew_group_report_histogram_bin_size` is set to auto, a default value of 10 picoseconds will be used for the histogram range size.

Valid values: auto | string

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`
- `report_skew_groups`

cts_spec_config_base_pin_trace_through_to

cts_spec_config_base_pin_trace_through_to base_pin

Default: ""

Data_type: string, read/write

Clock tree definition will, by default, not continue through certain types of cell arc (for instance, the clock to Q arc in a DFF). This attribute allows you to override the default behavior by specifying the output base_pin to which the clock tree should propagate, when it arrives at an instance of the given input library pin. The specified output base_pin must be another pin on the same library cell as the given input pin. The configured value is applicable to all instances of the library cell in the netlist.

The output pin may be specified either by its fully qualified name (i.e. inclusive of the base_cell name), or else simply by its local (cell-relative) name.

Note: if both trace_through_to and library_trace_through_to are applicable at a given netlist instance pin, the trace_through_to value takes precedence.

Valid values: base_pin

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `ccopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`
- `reset_cts_config`

cts_spec_config_create_clock_tree_source_groups

`cts_spec_config_create_clock_tree_source_groups {true | false}`

Default: `false`

Data_type: string, read/write

Causes `create_clock_tree_spec` to set up a clock tree source group for SDC clocks with multiple source pins. If this attribute is set to active, `create_clock_tree_spec` defines one clock tree for each source pin and then uses the `create_clock_tree_source_group` command to collect those clock trees together, so that CTS can distribute sinks between the clock trees. If this attribute is set to `inactive` or `true`, the source groups are also created during `create_clock_tree_spec`, but they will be defined as inactive.

Only active source groups will take part in sink allocation.

By default, it is set to `false`, which means that no source groups are extracted from SDC multiple pin clocks.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

`set_db attribute_name value`

- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`
- `create_clock_tree_source_group`
- `reset_cts_config`

cts_spec_config_create_generator_skew_groups

cts_spec_config_create_generator_skew_group {true | false}

Default: true

Data_type: bool, read/write

This attribute will cause the `create_clock_tree_spec` command to create skew groups for sequential generators and their adjacent registers. Such skew groups will be specified with the same highest rank so that they can be balanced from the other normal skew groups that share some sinks of them. The adjacent registers of a generator are registers that have a datapath timing path to talk with the generator directly. When this attribute is set to 1, one skew group will be created per sequential generator instance, master clock and generated clock tree triple. The resulting skew groups will by default be named in the pattern:

```
_clock_gen_<master_clock_name>_<generator_local_name>  
<_optional_number>/<constraint_mode_name>.
```

For example, for a pair of generators, with the same local name "reg_clkgen", CTS creates generated clock trees from the same master clock named "fclk" in a constraint mode named "func" the skew groups emitted into the clock tree specification file would be named:

```
_clock_gen_fclk_reg_clkgen_1/func  
_clock_gen_fclk_reg_clkgen_2/func
```

The prefix for the names of such skew groups is controlled by the `cts_spec_config_create_generator_skew_group_name_prefix` CTS attribute and defaults to "_clock_gen" (the start underscore is used to group such skew groups at the end of any skew group listing ordered by name).

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cconst_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`

cts_spec_config_create_generator_skew_groups_name_prefix

`cts_spec_config_create_generator_skew_groups_name_prefix prefix_name`

Default: `_clock_gen`

Data_type: string, read/write

This attribute controls the skew group name prefix used for skew groups generated to balance generator flops with their adjacent flops. Default is "`_clock_gen`". The default has a start underscore at the beginning to cause listings of skew groups ordered by name to collect such skew groups together at the end of a list.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `cconst_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`

cts_spec_config_create_reporting_only_skew_groups

`cts_spec_config_create_reporting_only_skew_groups {true | false}`

Default: `true`

Data_type: `bool`, read/write

Controls whether spec creation will synthesize reporting-only skew groups.

A reporting-only skew group is a skew group whose `cts_skew_group_constrains` attribute is set to `none`. Such a skew group imposes no clock balancing constraint and will not be considered by CTS.

The typical example is a generated clock that is synchronous to its master clock. The relationship between the two clocks implies that the sinks of both clocks should be balanced together. To model this, a clock spec created for such a design will have the skew group corresponding to the master clock span the domain of both the master clock and also the generated clock. The skew group of the generated clock is then a subset of the master clock skew group: as such it is a redundant constraint, and may be safely omitted. If it is synthesized, such a skew group will be created as a reporting-only skew group.

When this attribute is set to `false`, reporting-only skew groups are completely omitted from the generated spec. When set to `true`, reporting-only skew groups are included in the generated spec, but are marked with the `cts_skew_group_constrains` attribute set to `none`. Either way, they impose no balancing constraint.

Valid values: `true false`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `cconst_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`
- `report_skew_groups`

cts_spec_config_trace_multi_output_cells_with_single_clock_output

`cts_spec_config_trace_multi_output_cells_with_single_clock_output {true | false | combinational}`

Default: `combinational`

Data_type: string, read/write

This attribute controls whether and how `create_clock_tree_spec` will extract through cells that have multiple outputs but where only one of those outputs has a clock emanating from it; call these types of instance multi-output single-clock instances.

If the feature is disabled, spec creation will not trace through multi-output single-clock instances. The resulting clock spec will specify a generated clock tree on the clock-carrying outputs of every multi-output instance. Clock-carrying inputs that are connected to each clock output by way of timing arcs will be generator inputs to the corresponding generated clock tree. The presence of these generated clock trees may impact CTS operations on the instance such as sizing, moving, and cloning.

If the feature is enabled, the resulting clock spec will include `cts_spec_config_trace_through_to` settings for multi-output single-clock instances. These settings will allow the instance to be treated as a normal clock instance, most commonly as a clock logic. In this case there will be no need for the generated clock tree on the clock-carrying output. The treatment of instances with multiple clock-carrying outputs is unchanged: on such an instance each clock-carrying output will take a generated clock trees, as before.

The feature may be enabled conditionally, such that it is only applied to multi-output single-clock instance which are combinational. In this case multi-output single-clock instances which are sequential will continue to take a generated clock tree on the clock output.

The possible values for this attribute are as follows:

- `false` - The feature is completely disabled.
- `true` - The feature is enabled for all cell types; both combinational and sequential.
- `combinational` - The feature is enabled for combinational cells, but disabled for sequential cells.

Valid values: `false true combinational`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`

cts_target_max_capacitance

```
cts_target_max_capacitance {auto | double}
```

Default: top auto trunk auto leaf auto

Data_type: string, read/write

Indices: delay_corner

Specifies the target maximum capacitive load to allow during clock tree synthesis. This attribute specifies a maximum (combined pin and wire) capacitance that the clock tree synthesis algorithm will allow any given library pin to drive in a given clock tree when driving a given net_type. It is specified in library units. It currently only constrains the primary delay corner capacitance values - other delay corners can be specified but will not be constrained. This attribute is applied in addition to the `max_capacitance` constraints read from the liberty library data - the tightest (lowest) of the constraint specified by this attribute and the constraint present in the liberty data will be used. It also does not apply at the root pins of clock trees - to constrain those nets the

`cts_source_max_capacitance` CTS attribute should be used instead.

Valid values: auto | double

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`

cts_target_max_capacitance_leaf

`cts_target_max_capacitance_leaf {auto | double}`

Default: `auto`

Data_type: `string`, `read/write`

Indices: `delay_corner`

The target maximum capacitive load to allow during clock tree synthesis. This attribute specifies a maximum (combined pin and wire) capacitance that the clock tree synthesis algorithm will allow any given library pin to drive in a given clock tree when driving a given `net_type`. It is specified in library units. It currently only constrains the primary delay corner capacitance values - other delay corners can be specified but will not be constrained. This attribute is applied in addition to the `max_capacitance` constraints read from the liberty library data - the tightest (lowest) of the constraint specified by this attribute and the constraint present in the liberty data will be used. It also does not apply at the root pins of clock trees - to constrain those nets the `cts_source_max_capacitance` CTS attribute should be used instead. This attribute only applies to the leaf net type.

Valid values: `auto | double`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`

cts_target_max_capacitance_top

`cts_target_max_capacitance_top {auto | double}`

Default: `auto`

Data_type: `string`, `read/write`

Indices: `delay_corner`

The target maximum capacitive load to allow during clock tree synthesis. This attribute specifies a maximum (combined pin and wire) capacitance that the clock tree synthesis algorithm will allow any given library pin to drive in a given clock tree when driving a given `net_type`. It is specified in library units. It currently only constrains the primary delay corner capacitance values - other delay corners can be specified but will not be constrained. This attribute is applied in addition to the `max_capacitance` constraints read from the liberty library data - the tightest (lowest) of the constraint specified by this attribute and the constraint present in the liberty data will be used. It also does not apply at the root pins of clock trees - to constrain those nets the `cts_source_max_capacitance` CTS attribute should be used instead. This attribute only applies to the top net type.

Valid values: `auto` | `double`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`

cts_target_max_capacitance_trunk

`cts_target_max_capacitance_trunk {auto | double}`

Default: `auto`

Data_type: string, read/write

Indices: `delay_corner`

The target maximum capacitive load to allow during clock tree synthesis. This attribute specifies a maximum (combined pin and wire) capacitance that the clock tree synthesis algorithm will allow any given library pin to drive in a given clock tree when driving a given `net_type`. It is specified in library units. It currently only constrains the primary delay corner capacitance values - other delay corners can be specified but will not be constrained. This attribute is applied in addition to the `max_capacitance` constraints read from the liberty library data - the tightest (lowest) of the constraint specified by this attribute and the constraint present in the liberty data will be used. It also does not apply at the root pins of clock trees - to constrain those nets the `cts_source_max_capacitance` CTS attribute should be used instead. This attribute only applies to the trunk net type.

Valid values: `auto | double`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`

cts_target_max_stage_delay_sigma

`cts_target_max_stage_delay_sigma { double | ignore }`

Default: ignore

Data_type: string, read/write

Indices: delay_corner

The max per-stage statistical on-chip variation (SOCV) delay sigma target to use for CTS.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`

cts_target_max_transition_time

`cts_target_max_transition_time { default | auto | ignore | double }`

Default: top default trunk default leaf default

Data_type: string, read/write

Indices: delay_corner, power_domain

The target slew used for clock tree synthesis. This attribute specifies a maximum slew time that the clock tree synthesis algorithm will allow in this clock tree, in library units. 'default' means 'auto' in primary half corner and 'ignore' in other half corners. If set to 'auto', CTS picks an appropriate value based on the collection of allowed buffer sizes and library parameters, although this may not give optimal quality of results. If set to 'ignore', CTS does not constrain the corner. The tightest (lowest) of the constraint specified by this attribute and the constraint present in the liberty data (`max_transition`) will be used.

For details, see Transition Target in [CTS Configuration and Method](#).

Valid values: default | auto | ignore | double

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `convert_lib_clock_tree_latencies`
- `create_clock_tree`
- `create_clock_tree_spec`

cts_target_max_transition_time_leaf

`cts_target_max_transition_time_leaf {default | auto | ignore | double}`

Default: default

Data_type: string, read/write

Indices: delay_corner, power_domain

The target slew used for clock tree synthesis. This attribute specifies a maximum slew time that the clock tree synthesis algorithm will allow in this clock tree, in library units. 'default' means 'auto' in primary half corner and 'ignore' in other half corners. If set to 'auto', CTS picks an appropriate value based on the collection of allowed buffer sizes and library parameters, although this may not give optimal quality of results. If set to 'ignore', CTS does not constrain the corner. This attribute only applies to the leaf net type.

For details, see Transition Target in [CTS Configuration and Method](#).

Valid values: `default` | `auto` | `ignore` | `double`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `convert_lib_clock_tree_latencies`
- `create_clock_tree`
- `create_clock_tree_spec`

cts_target_max_transition_time_top

`cts_target_max_transition_time_top` {`default` | `auto` | `ignore` | `double`}

Default: `default`

Data_type: string, read/write

Indices: `delay_corner`, `power_domain`

The target slew used for clock tree synthesis. This attribute specifies a maximum slew time that the clock tree synthesis algorithm will allow in this clock tree, in library units. 'default' means 'auto' in primary half corner and 'ignore' in other half corners. If set to 'auto', CTS picks an appropriate value based on the collection of allowed buffer sizes and library parameters, although this may not

give optimal quality of results. If set to 'ignore', CTS does not constrain the corner. This attribute only applies to the top net type.

Valid values: `default` | `auto` | `ignore` | `double`

For details, see Transition Target in [CTS Configuration and Method](#).

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `convert_lib_clock_tree_latencies`
- `create_clock_tree`
- `create_clock_tree_spec`

cts_target_max_transition_time_trunk

`cts_target_max_transition_time_trunk` {`default` | `auto` | `ignore` | `double`}

Default: `default`

Data_type: string, read/write

Indices: `delay_corner`, `power_domain`

The target slew used for clock tree synthesis. This attribute specifies a maximum slew time that the clock tree synthesis algorithm will allow in this clock tree, in library units. 'default' means 'auto' in primary half corner and 'ignore' in other half corners. If set to 'auto', CTS picks an appropriate value based on the collection of allowed buffer sizes and library parameters, although this may not give optimal quality of results. If set to 'ignore', CTS does not constrain the corner. This attribute only applies to the trunk net type.

Valid values: `default` | `auto` | `ignore` | `double`

For details, see Transition Target in [CTS Configuration and Method](#).

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `convert_lib_clock_tree_latencies`
- `create_clock_tree`
- `create_clock_tree_spec`

cts_target_skew

```
cts_target_skew { default | auto | ignore | double }
```

Default: default

Data_type: string, read/write

Indices: delay_corner

This specifies the target skew for clock tree balancing. This may be set to a numeric value, or one of 'auto', 'ignore' or 'default'.

If set to 'auto' this indicates that an appropriate skew target should be computed.

If set to 'ignore' this indicates that skew should not be balanced for this corner/path combination.

If unspecified then the value of this attribute is 'default'.

If the value of the attribute is 'default' the target skew for late delays in the primary delay corner is interpreted as 'auto' and as 'ignore' otherwise.

Valid values: default | auto | ignore | double

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`
- `create_skew_group`

cts_timing_connectivity_based_skew_groups_balance_master_clocks

```
cts_timing_connectivity_based_skew_groups_balance_master_clocks {true | false}
```

Default: false

Data_type: bool, read/write

This attribute has no effect if `cts_timing_connectivity_based_skew_groups` is set to 'off'.

With timing connectivity based skew groups enabled, this attribute controls how CTS will address the synchrony of disjoint (i.e. non-overlapping) master (i.e. non-generated) SDC clocks.

When this attribute is set to `false`, all disjoint master clocks are assumed to be mutually asynchronous. The skew groups generated by `create_clock_tree_spec` will not constrain the sinks of two such clocks to be balanced together.

When this attribute is set to `true`, the synchrony of disjoint master clocks is determined by consulting the SDC assertions in accordance with the value of attribute

```
cts_timing_connectivity_based_skew_groups.
```

Additional skew groups will be generated by `create_clock_tree_spec`, such that the sinks of each pair of synchronous disjoint master clocks will be balanced together by CTS.

Valid values `true false`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`
- `create_skew_group`

cts_timing_connectivity_based_skew_groups

```
cts_timing_connectivity_based_skew_groups string
```

Default: `off`

Data_type: string, read/write

SDC false path assertions may render a generated SDC clock asynchronous to its master clock. Under such a condition, the sinks of the generated clock need not be balanced against the sinks of the master clock: there are no timing paths between the two clocks that can be affected by inter-clock skew. This balancing 'relaxation' is realized by adjusting the skew groups that are generated for the two clocks.

This attribute specifies which SDC assertions are considered when deciding whether a generated clock is asynchronous to its master. Valid values for this attribute are as follows:

- `off`: In this mode, every generated SDC clock is treated as being synchronous to its master SDC clock.
- `clock_false_path`: In this mode, only clock/clock `set_false_path` and `set_clock_group` assertions are considered.

Valid values: `off` `clock_false_path`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`
- `create_skew_group`

cts_timing_connectivity_info

`cts_timing_connectivity_info` *string*

Default: ""

Data_type: string, read/write

This attribute is populated by clock spec creation when timing connectivity based skew groups are enabled. Configuration of this attribute will be written to the generated spec. It documents the clock/clock balancing relationships that were determined for the production of timing connectivity based skew groups. This attribute is documenting only.

If timing connectivity based skew groups are not enabled, this attribute is not populated by spec creation.

The value of the attribute is a set of nested Tcl dictionaries, four levels in total. The top-level dictionary indicates the information category. Currently there is only one such category:

'clock_relationships'. The

second level is keyed on constrained mode name; the lowest level dictionaries are both keyed on SDC clock name. These keys taken together identify two SDC clocks in a given constraint mode. The leaf value specifies the resolved balancing relationships between the two SDC clocks. It is an enum with the following members:

- `direct` - The two clocks are determined to belong to the same clock group and must be balanced together.

- indirect - The two clocks do not belong to the same clock group; however by transitive closure they must balance together.

Clocks in a direct or indirect balancing relationship will share one or more skew groups so as to ensure that their sinks are balanced together by CTS. Missing entries should be taken as 'need not balance'; i.e. the pair of SDC clocks have neither a direct nor indirect balancing relationship.

The indirect balancing relationship can be explored in more detail using this same Tcl dictionary to probe out the transitive closure over the direct balancing relationships that link two indirect balance clocks together. For example, if clock A directly balances with clock B only, and clock B directly balances with clock C only; then clock A indirectly balances with clock C.

Below is an example dict value, corresponding to clock groups {clkA clkB} {clkB clkC} in constraint mode cm1, and clock groups {clkP clkQ} {clkR clkS} in constraint mode cm2.

```
{clock_relationships \
  {cm1 {clkA {clkB direct clkC indirect} \
        clkB {clkA direct clkC direct} \
        clkC {clkA indirect clkB direct}}} \
  cm2 {clkP {clkQ direct} \
        clkQ {clkP direct} \
        clkR {clkS direct} \
        clkS {clkR direct}}}}
```

Notice that although they are in separate clock groups, clkA and clkC nevertheless indirectly balance. This is due to the fact that clkB is present in both clock groups. Note also that we omit information about the relationship between clkP and clkS: they are 'need not balance'.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cconst_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`
- `create_skew_group`

cts_top_fanout_threshold

`cts_top_fanout_threshold` *number_of_sinks*

Default: `unset`

Data_type: string, read/write

Minimum number of transitive fanout in the clock tree for a net to be routed as a top net. Nets with at least this many sinks in their transitive fanout in the clock tree will have the special routing rules applied to them.

Valid values: `integer`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `cconst_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`

cts_update_clock_latency

`cts_update_clock_latency {true | false}`

Default: `true`

Data_type: `bool`, read/write

Determine whether to update IO latencies within `cconst_design`.

Valid values: `true`, `false`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cconst_design`
- `clock_opt_design`
- `convert_lib_clock_tree_latencies`
- `create_clock_tree`
- `create_clock_tree_spec`

cts_use_inverters

`cts_use_inverters {auto | true | false}`

Default: `auto`

Data_type: `string`, read/write

Specifies whether clock tree synthesis should prefer to use inverters rather than buffers when balancing the clock tree. If set to `true`, CTS will use inverters for the clock tree balancing process. If set to `false`, CTS will use the minimum number of levels of inverters required to maintain logical correctness. If set to `auto` (the default) CTS will build a tree that is built from buffers. A very small number of inverters may be inserted for surgical balancing optimizations.

Valid values: `auto` | `true` | `false`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `add_clock_tree_exclusion_drivers`
- `cchopt_design`
- `clock_opt_design`
- `convert_lib_clock_tree_latencies`
- `create_clock_tree`
- `create_clock_tree_spec`
- `report_cts_cell_filtering_reasons`

cts_use_receiver_model_capacitance_for_drv

```
cts_use_receiver_model_capacitance_for_drv {auto | true | false}
```

Default: `false`

Data_type: string, read/write

When this attribute is set to `auto`, CTS will use receiver model capacitance for drv if the attribute, `timing_report_use_receiver_model_capacitance` is `true`. When set to `true`, CTS will use receiver model capacitance for drv even if `timing_report_use_receiver_model_capacitance` is `false`. By default, this attribute is set to `false`, which means that CTS will not use receiver model capacitance for drv.

Valid values: `auto` `true` `false`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cconst_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`

flexible_htrees

```
flexible_htrees list of flexible_htree
```

Default: ""

Data_type: flexible_htree*, read-only

Returns the list of flexible h-trees in the design.

Valid values: list of flexible_htree

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `cchopt_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`
- `create_flexible_hrtree`
- `route_flexible_hrtrees`
- `synthesize_flexible_hrtrees`

skew_groups

`skew_groups` *list of skew_groups*

Default: ""

Data_type: skew_group*, read-only

Returns the list of skew groups in the design.

Valid values: list of skew_group

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `cconst_design`
- `clock_opt_design`
- `create_clock_tree`
- `create_clock_tree_spec`
- `create_skew_group`

cwd

The root attributes in the `cwd` category are:

- `cwd_setup_file`

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `cwd` category, use the following command:

```
get_db -category cwd
```

cwd_setup_file

`cwd_setup_file` *string*

Default: ""

Data_type: string, read/write

Specifies the setup file that contains the commands for the ChipWare Developer flow.

ChipWare Developer enables you to add user-defined ChipWare components and user-defined implementations of tool-defined ChipWare components. The commands to set up the ChipWare Developer flow must be specified in the setup file. The tool automatically sources this file to set up the necessary infrastructure. This is required to correctly pass the information to the super-thread servers.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

delaycal

The root attributes in the "delaycal" category are:

- [delaycal_accuracy_level](#)
- [delaycal_accurate_receiver_out_load](#)
- [delaycal_advanced_pin_cap_mode](#)
- [delaycal_combine_mmmc](#)
- [delaycal_disable_randomcell_in_socv](#)
- [delaycal_enable_accurate_lle_analysis](#)
- [delaycal_degrade_slew_on_early_nets](#)
- [delaycal_default_net_delay](#)
- [delaycal_default_net_load](#)
- [delaycal_disable_positive_slack_shift](#)
- [delaycal_enable_corner_based_beol](#)
- [delaycal_enable_high_fanout](#)
- [delaycal_enable_input_slew_sensitivity_on_constraint](#)
- [delaycal_enable_quiet_receivers_for_hold](#)
- [delaycal_enable_si](#)
- [delaycal_enable_separate_wire_var_sigma_multiplier](#)
- [delaycal_enable_wire_load_model](#)
- [delaycal_equivalent_waveform_model](#)
- [delaycal_equivalent_waveform_type](#)
- [delaycal_equivalent_waveform_model_for_timing_check](#)
- [delaycal_library_interpolation_mode](#)
- [delaycal_honor_slew_propagate_constraint](#)
- [delaycal_input_transition_delay](#)
- [delaycal_irdrop_data_type](#)

- delaycal_early_irdrop_data_type
- delaycal_late_irdrop_data_type
- delaycal_irdrop_window_based
- delaycal_report_out_bound
- delaycal_separate_params_for_via
- delaycal_slew_out_bound_limit_high
- delaycal_slew_out_bound_limit_low
- delaycal_skip_slew_merge_from_disabled_path
- delaycal_socv_accuracy_mode
- delaycal_socv_lvf_mode
- delaycal_socv_machine_learning_level
- delaycal_socv_use_lvf_tables
- delaycal_support_output_pin_cap
- delaycal_timing_create_clock_use_ideal_slew
- delaycal_use_default_delay_limit
- delaycal_via_corner_sigma_multiplier
- delaycal_via_correlation_type
- delaycal_via_reset
- delaycal_wire_corner_sigma_multiplier
- delaycal_wire_correlation_type
- delaycal_wire_layer_avg_rpt
- delaycal_wire_rc_correlation_factor
- delaycal_wire_reset
- delaycal_wire_use_nsigma_coupling_capacitance
- delaycal_wire_use_nsigma_slew_for_timing_check
- delaycal_wire_var_reporting_sigma_multiplier
- delaycal_wire_variation_correlation_mode

- [delaycal_wire_variation_rc_correlation](#)
- [delaycal_wire_wire_cell_variation_correlation](#)

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `power` category, use the following command:

```
get_db -category delaycal
```

For delay calculation reports, use the following command:

- [report_delay_calculation](#)

delaycal_accuracy_level

```
delaycal_accuracy_level level
```

Default: 3

Data_type: Integer

Sets the accuracy level for delay calculations.

You can use this attribute to enable new enhancements for different process nodes, without using the `design_process_node` attribute.

This setting replaces earlier use models where the `set_design_mode -process` parameter was used to enable delay calculation accuracy settings.

The following commands show mapping of old and new options to use for specific nodes:

```
set_design_mode -process 16 == delaycal_accuracy_level 0  
set_design_mode -process 10 == delaycal_accuracy_level 1  
set_design_mode -process 7 == delaycal_accuracy_level 2  
set_design_mode -process 5 == delaycal_accuracy_level 3
```

delaycal_accurate_receiver_out_load

```
delaycal_accurate_receiver_out_load {true | false}
```

Default: `true`

Data_type: Boolean

When set to `true`, Tempus uses distributed interconnect model as receiver output load to compute accurate miller effect.

In this mode, Tempus uses a reduced interconnect model for receivers instead of lumped capacitance as the receiver load. This improves computation of the back miller effect and the delay accuracy.

When set to `false`, the software uses lumped capacitance as the receiver load.

delaycal_advanced_pin_cap_mode

`delaycal_advanced_pin_cap_mode {0 | 1 | 2}`

Default: 2

Data_type: Enum

Simulates the current stage and receiver stages together, such that the back miller effect is modeled dynamically.

- 0: Receiver current sources are computed using ECSM/CCS capacitance tables with the estimated input slew.
- 1: Receivers are no longer modeled as simple non-linear current source models. This results in increasing the delay calculation accuracy significantly as the modeling back miller effect improves.
- 2: Uses advanced simulation-based approach to model the receiver back miller effect.

delaycal_combine_mmmc

`delaycal_combine_mmmc {none | early_late | early_late_corner}`

Default: `early_late`

Data_type: Enum

Specifies whether delay calculation runs are to be combined for delay calculation simulations. This attribute provides user control for improving the runtime for all the analysis types by combining runs for early and late simulations into a single simulation. The views with common delay corners may also be combined into a single delay calculation simulation for added runtime improvement. You can use the following options:

- `none`: Does not combine any delay calculation runs. Individual delay calculation simulations are performed for early and late for example, which will result in significant CPU performance degradation due to the increased number of simulation runs performed.
- `early_late`: Combines early and late simulations of a single corner into one delay calculation simulation and will improve CPU performance by avoiding unnecessary delay calculation simulations.
- `early_late_corner`: Combines early / late simulations of a single corner, like with the `early_late` setting, but also combines views with common delay corners into one single delay calculation simulation. For example, the software will combine views having different `set_case_analysis` statements in one single simulation. This can result in significant improvement in CPU performance by avoiding unnecessary delay calculation simulations.

delaycal_disable_randomcell_in_socv

```
delaycal_disable_randomcell_in_socv {true | false}
```

Default: false

Data_type: Boolean

Disables random cell sensitivity in SOCV mode.

delaycal_enable_accurate_lle_analysis

```
delaycal_enable_accurate_lle_analysis {true | false}
```

Default: false

Data_type: Boolean

Enables accurate LLE analysis.

delaycal_degrade_slew_on_early_nets

```
delaycal_degrade_slew_on_early_nets {true | false}
```

Default: true

Data_type: Boolean

Controls whether slews in early path delay calculation are degraded as they pass across the

interconnect from driver to receiver. You can specify a value of false to turn off interconnect slew degradation for early paths and have the driver slews copied to the receivers. Early path delay calculation affects all Hold paths in best-case/worst-case analysis mode, capture clock paths in on-chip variation Setup analysis, and clock launch and data paths in on-chip variation Hold analysis.

delaycal_default_net_delay

`delaycal_default_net_delay delay_value`

Default: 1000ps

Data_type: String

Sets the default net delay for delay calculation. The software uses this default value for nets that exceed 1,000 terminals. Delay calculation is performed for nets with fewer than 1,000 terminals.

delaycal_default_net_load

`delaycal_default_net_load load_value`

Default: 0.5pF

Data_type: String

Sets the default net load for delay calculation. You can use this attribute to specify the default net load value that is used for nets that exceed 1,000 terminals.

delaycal_disable_positive_slack_shift

`delaycal_disable_positive_slack_shift {true | false}`

Default: false

Data_type: Boolean

Disables positive slack shift caused by back end-of-line (BEOL) variation.

delaycal_enable_corner_based_beol

`delaycal_enable_corner_based_beol {true | false}`

Default: false

Data_type: Boolean

Enables anchor corner-based BEOL (back end-of-line) analysis.

delaycal_enable_high_fanout

```
delaycal_enable_high_fanout {true | false}
```

Default: false

Data_type: Boolean

When this attribute is set to `true`, the default net delay will be annotated on high fanout nets. A net will be treated as a high fanout net when the fanout is more than the limit specified using the `delaycal_use_default_delay_limit` attribute. The default value of this attribute is 1000 fanouts. The default net delay is also specified using this attribute. The default net delay is 1ns. The cell delay will use the lumped capacitance that is specified using the `delaycal_default_net_load` attribute for table lookup. The default net load is 0.5pF. The driver slew will use the default value of 0ps, which can be changed by using the `delaycal_input_transition_delay` attributes and there will be no slew degradation along the interconnect.

delaycal_enable_input_slew_sensitivity_on_constraint

```
delaycal_enable_input_slew_sensitivity_on_constraint {true | false}
```

Default: true

Data_type: Boolean

Enables the input slew sensitivity on constraints.

When set to `true`, the input slew sensitivities at constrained and related pins will be considered by timing check sigma calculation.

When set to `false`, the timing check arc is derived based on the nominal or mean slews only.

delaycal_enable_quiet_receivers_for_hold

```
delaycal_enable_quiet_receivers_for_hold {true | false}
```

Default: false

Data_type: Boolean

Forces some receivers to be static ("quiet").

When set to `true`, the software assumes that the receivers are not switching - to compute the worst-

case delays for early analysis. This yields more conservative early delays.

When set to `false`, the software assumes that the receivers may switch.

delaycal_enable_si

```
delaycal_enable_si {true | false}
```

Default: `false`

Data_type: Boolean

Enables SI-Aware delay calculation that also includes cross-talk induced delays.

Note: The SI-Aware delay calculation requires the design to be in Multi-Mode, Multi-Corner (MMMC) Mode and the analysis type to be set to on-chip variations.

delaycal_enable_separate_wire_var_sigma_multiplier

```
delaycal_enable_separate_wire_var_sigma_multiplier {true | false}
```

Default: `false`

Data_type: Boolean

Allows separate reporting sigma multiplier for wire variations.

delaycal_enable_wire_load_model

```
delaycal_enable_wire_load_model {0 | 1}
```

Default: `1`

Data_type: Integer

Enables support of Liberty wire-load models and related SDC commands.

delaycal_equivalent_waveform_model

```
delaycal_equivalent_waveform_model {none | no_propagation | propagation}
```

Default: `none`

Data_type: Enum

Specifies the equivalent waveform model to be used.

delaycal_equivalent_waveform_type

delaycal_equivalent_waveform_type {moments | simulation}

Default: simulation

Data_type: Enum

Specifies the equivalent waveform model (EWM) to be used:

The options are described below:

- **moments:** Enables moment-based equivalent waveform model.
- **simulation:** Enables simulation-based equivalent waveform model. The simulation-based EWM is required for predicting delay in noisy waveforms.

It is recommended to use simulation-based EWM as it is more accurate than moment-based EWM.

delaycal_equivalent_waveform_model_for_timing_check

delaycal_equivalent_waveform_model_for_timing_check {true | false}

Default: false

Data_type: Boolean

Models the waveform distortion effect for timing check value look ups at both the data and clock pins.

The model uses simulations to convert the distorted waveforms into equivalent waveforms (pre-driver waveforms in the libraries) by matching the pin CCB output waveforms. This feature requires the CCSN library.

delaycal_library_interpolation_mode

delaycal_library_interpolation_mode {linear | non_linear}

Default: linear

Data_type: Enum

Controls library scaling mode - `linear` or `non_linear` - for the interpolation flow.

delaycal_honor_slew_propagate_constraint

```
delaycal_honor_slew_propagate_constraint {true | false}
```

Default: true

Data_type: Boolean

Determines whether to propagate the slew from the disabled timing arcs to the output pin.

When set to `false`, the slew from the disabled timing arc affects the delay of the next stage. Setting this attribute to false results in more accurate slew computation for the disabled timing arcs but can take longer run time.

When set to `true`, the timing constraints defined using commands such as `set_disable_timing` and `set_case_analysis` impact the slew propagation, and the slew from the disabled timing arcs is not propagated.

delaycal_input_transition_delay

```
delaycal_input_transition_delay " "
```

Default: 4ps

Data_type: String

Sets the default input transition delays and the ideal clock transition time for delay calculation. The software uses this default transition time for nets that exceed 1,000 terminals.

It is mandatory to specify time units for this command. Valid time units are ps, ns, etc.

In addition, the default input transition value is applied to those pins to which the slew was not propagated. For example:

- Tiehi or tielo pins
- Macro output pins, which do not have a related input pin

A macro output pin, which has a related input pin will most likely have slew propagation from the related input pin to the output pin.

- Macro output pins for which all the timing arcs are disabled
- Input ports without a user-specified transition value

delaycal_irdrop_data_type

```
delaycal_irdrop_data_type {best_average | worst | best | average | worst_average}
```

Default: worst

Data_type: Enum

Allows you to set early/late IR drop EIV values for different EIV voltages.

delaycal_early_irdrop_data_type

```
delaycal_early_irdrop_data_type {best_average | worst | best | average | worst_average}
```

Default: worst

Data_type: Enum

Allows you to set early IR drop EIV values for different EIV voltages.

delaycal_late_irdrop_data_type

```
delaycal_late_irdrop_data_type {best_average | worst | best | average | worst_average}
```

Default: worst

Data_type: Enum

Allows you to set late IR drop EIV values for different EIV voltages.

delaycal_irdrop_window_based

```
delaycal_irdrop_window_based {none | late | early | both}
```

Default: both

Data_type: Enum

Searches window-based EIV values from the EIV database.

delaycal_report_out_bound

```
delaycal_report_out_bound {true | false}
```

Default: false

Data_type: Boolean

Generates a report that contains a list of index values (input transition) in the delay tables that are

beyond the index range. The index values detected by this attribute can cause inaccurate delay values to be calculated. The report generated by this attribute can be used for debugging.

When this attribute is set to `true`, the extrapolation information will be dumped into the following two files:

- `<design-name>.dc.outbound.slew.early.<view-name>.txt`
- `<design-name>.dc.outbound.slew.late.<view-name>.txt`

The software writes a warning message in the log files noting that there are input transition values that are out of the index range of the timing table. The warning message also provides the file name for viewing the detail report.

delaycal_separate_params_for_via

```
delaycal_separate_params_for_via {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, the software creates separate parameters for via variation.

delaycal_slew_out_bound_limit_high

```
delaycal_slew_out_bound_limit_high double
```

Default: `3.40282e+38`

Data_type: Double

Limits the maximum slew used in the analysis to the specified value times max characterized slew in a library.

delaycal_slew_out_bound_limit_low

```
delaycal_slew_out_bound_limit_low double
```

Default: `0.5`

Data_type: Double

Limits the minimum slew used in the analysis to the specified value times min characterized slew in a library.

delaycal_skip_slew_merge_from_disabled_path

delaycal_skip_slew_merge_from_disabled_path {true | false}

Default: false

Data_type: Boolean

Avoids merging slews from unconstrained, segmented paths and black-box output pin.

When set to `true`, the software does not allow merging transitions from disabled paths.

delaycal_socv_accuracy_mode

delaycal_socv_accuracy_mode {low | medium | high | ultra}

Default: low

Data_type: Enum

Allows the ability to control levels of accuracy.

This attribute specifies level-based accuracy effort for delays and slews. You can use one of the following options:

- `low`: Ignores correlation between slews and delays.
- `medium`: Models correlation between immediate slew and delay.
- `high`: Considers skewness of non-gaussian distribution into account for computing quantiles. The skewness is local to one stage only.
- `ultra`: Considers skewness of non-gaussian distribution, and also propagates skewness to the next stage in PBA analysis.

delaycal_socv_lvf_mode

delaycal_socv_lvf_mode *string*

Default: moments

Data_type: Enum

Controls SOCV LVF analysis modes. You can specify `moments` or `early_late` option.

delaycal_socv_machine_learning_level

delaycal_socv_machine_learning_level {0 | 1 | 3}

Default: 0

Data_type: Integer

Enables the SOCV machine learning mode.

For accurate SOCV analysis, the Tempus software creates several statistical models, which are not available with standard LVF library, such as correlation between cells delay and output slew, the conversion of moments from late/early sigma tables, and effects of waveform distortion on sigma and mean shift.

You can specify one of the following options:

- 1: Enables several machine learning algorithms to generate such models internally.
- 3: Enabling this mode uses improved modeling of slew-delay correlation using advanced machine learning techniques. This option enables accurate quantile modeling, which involves quantile computation from mean, stddev and skewness values.

Note: The default setting of 0 is recommended for nodes 110, 40, 28, and 22nm. For other technology nodes, option 3 can be used. For this setting you must enable the ultra mode using – socv_accuracy_mode ultra option.

delaycal_socv_use_lvf_tables

delaycal_socv_use_lvf_tables string

Default: all

Data_type: String

Controls which type of variations – all, delay, slew, or constraint - are considered during analysis.

delaycal_support_output_pin_cap

delaycal_support_output_pin_cap {true | false}

Default: true

Data_type: Boolean

Includes the output pin capacitance when calculating delay values. When you specify false, the

software excludes only the pin capacitance of the driver for which you are calculating the delay.

delaycal_timing_create_clock_use_ideal_slew

```
delaycal_timing_create_clock_use_ideal_slew {true | false}
```

Default: false

Data_type: Boolean

Enables the use of ideal slew at the inputs of the clock driver cells.

When set to `true`, the software detects created clocks, and uses zero input slew. Other slew values, including those set using `set_annotated_transition` command, will be ignored.

When set to `false`, the software honors `set_annotated_transition` command settings.

delaycal_use_default_delay_limit

```
delaycal_use_default_delay_limit value
```

Default: 1000

Data_type: Integer

Sets the pin number threshold for nets for delay calculation. Nets that have more than the specified number of pins are excluded from delay calculation, and use the default delay values instead.

Net default delay values are:

- Net delay: 1 ns
- Transition time: 0 ps
- Net load: 0.5 pF

Note: In post route stage, default delay calculation does not honor this attribute. Slews and delays are computed based on actual parasitics.

To change the default delay values, set the following attributes:

- `delaycal_default_net_delay`
- `delaycal_default_net_load`
- `delaycal_input_transition_delay`

delaycal_via_corner_sigma_multiplier

delaycal_via_corner_sigma_multiplier *String*

Default: " "

Data_type: string

Provides the scaling multiplier for corner scaling. This attribute allows you to specify per via sigma multiplier and handles the advanced via variation scaling.

delaycal_via_correlation_type

delaycal_via_correlation_type *String*

Default: corner

Data_type: Enum

Specifies the inter-layer correlation values.

delaycal_via_reset

delaycal_via_reset {true | false}

Default: false

Data_type: Boolean

When set to `true`, the software resets the via variation modes to their default value.

delaycal_wire_corner_sigma_multiplier

delaycal_wire_corner_sigma_multiplier *string*

Default: " "

Data_type: string

Provides scaling multiplier for corner scaling.

delaycal_wire_correlation_type

delaycal_wire_correlation_type {statistical | binomial | corner | correlated}

Default: `statistical`

Data_type: Enum

Provides control to handle inter-layer correlation. The accepted values are - statistical, binomial, corner, and correlated.

delaycal_wire_layer_avg_rpt

```
delaycal_wire_layer_avg_rpt {true | false}
```

Default: `false`

Data_type: Boolean

Enables layer averaging reporting.

delaycal_wire_rc_correlation_factor

```
delaycal_wire_rc_correlation_factor int
```

Default: `1`

Data_type: Integer

Allows same layer R and C correlation. The accepted values are 1 and -1.

delaycal_wire_reset

```
delaycal_wire_reset {true | false}
```

Default: `false`

Data_type: Boolean

Resets wire variation modes to the default value.

delaycal_wire_use_nsigma_coupling_capacitance

```
delaycal_wire_use_nsigma_coupling_capacitance {true | false}
```

Default: `false`

Data_type: Boolean

Allows to use mean and nSigma coupling capacitance.

delaycal_wire_use_nsigma_slew_for_timing_check

delaycal_wire_use_nsigma_slew_for_timing_check {true | false}

Default: false

Data_type: Boolean

Enables use of wire variation-aware slews for timing check lookup.

delaycal_wire_var_reporting_sigma_multiplier

delaycal_wire_var_reporting_sigma_multiplier double

Default: 0.0

Data_type: Double

Specifies deterministic sigma multiplier for reporting BEOL variations.

delaycal_wire_variation_correlation_mode

delaycal_wire_variation_correlation_mode {statistical | binomial}

Default: statistical

Data_type: Enum

Specifies wire variation inter-layer correlation. When set to `statistical`, all the layers are considered independent of each other.

delaycal_wire_variation_rc_correlation

delaycal_wire_variation_rc_correlation double

Default: 1.0

Data_type: Double

Specifies same layer R and C correlation for wire variation. When set to 1, the R and C sensitivities are considered as fully correlated.

delaycal_wire_wire_cell_variation_correlation

delaycal_wire_wire_cell_variation_correlation {true | false}

Default: `false`

Data_type: Boolean

Controls whether to RSS or add random cell variation and wire variation for arrivals and slack.

design

The root attributes in the design category are:

- [design_backside_bottom_routing_layer](#)
- [design_backside_top_routing_layer](#)
- [design_bottom_routing_layer](#)
- [design_compressed_pg_db](#)
- [design_cong_effort](#)
- [design_early_clock_flow](#)
- [design_ideal_hold_fixing](#)
- [design_merge_trim_shapes](#)
- [design_optimization_density_screen_margin](#)
- [design_pessimistic_mode](#)
- [design_tech_node](#)
- [design_top_routing_layer](#)
- [design_trim_grid_group](#)

To set the value of any root attribute, use the [set_db](#) command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the [get_db](#) command:

```
get_db attribute_name
```

To get the current value of all attributes in the `design` category, use the following command:

```
get_db -category design
```

design_backside_bottom_routing_layer

`design_backside_bottom_routing_layer` *layerName*

Default: ""

Data_type: string, read/write

Specifies the highest backside LEF layer name for global and detail routing.

design_backside_top_routing_layer

`design_backside_top_routing_layer` *layerName*

Default: ""

Data_type: string, read/write

Specifies the lowest backside LEF layer name for global and detail routing.

design_bottom_routing_layer

`design_bottom_routing_layer` *layerName*

Default: ""

Data_type: bool

Specifies the lowest LEF layer name for global and detail routing.

design_compressed_pg_db

`design_compressed_pg_db` {true | false}

Default: true

Data_type: bool

Specifies whether the compressed PG feature is enabled. When it is set to `true`, `read_db` will automatically compress the PG data while loading the design. You can use `get_db current_design .is_pg_compressed` to check if PG in the design is compressed.

design_cong_effort

`design_cong_effort {low medium high auto}`

Specifies the effort level for relieving congestion.

Default: `auto`

Data_type: `enum {low medium high auto}`

Specify one of the following values:

- `auto`: Automatically determines whether the design is congested and performs extra congestion driven effort for highly congested designs.
- `high`: Runs more iterations of placement in an effort to achieve better congestion results. This parameter increases the run time.
- `low`: Runs fewer iterations of placement to arrive quickly at a legal placement. This parameter might decrease placement quality.
- `medium`: Runs placement on designs at a normal effort level.

design_early_clock_flow

`design_early_clock_flow {true | false}`

Default: `false`

Data_type: `bool`

Enables early clock flow.

design_ideal_hold_fixing

`design_ideal_hold_fixing {true | false}`

Default: `false`

Data_type: `bool, read/write`

Enables the ideal hold fixing flow.

design_merge_trim_shapes

```
design_merge_trim_shapes {{layers layer_name_list gap gap_value max_merge_num number_of_max_merge masks {mask_num ...}} ...}
```

Default: ""

Data_type: string, read/write

Specifies the setting for trim shapes merge.

- **layers:** You use this to specify the trim layer list which needs to merge trim shapes.
- **Gap:** You use this setting when the end to end gap is less the value.
- **max_merge_num:** You use this to specify the max trim shapes number that are merged.
- **masks:** You use this to specify the trim shapes color that needs to be merged.

design_optimization_density_screen_margin

```
design_optimization_density_screen_margin {{layers layer_name_list gap gap_value max_merge_num number_of_max_merge masks {mask_num ...}} ...}
```

Default: 0.1

Min: 0

Max: 1

Data_type: double, read/write

Increases margin on the top of original density screen for optimization.

design_pessimistic_mode

```
design_pessimistic_mode {true false}
```

Default: false

Data_type: enum {true false}

Specifies that the delay calculation compatibility mode should be used for correlating to external signoff analysis tools. This mode adjusts delay calculation settings to ensure the best match to the third party engine while maintaining a degree of pessimism to help limit optimistic outliers in the external signoff analysis tool.

design_tech_node

```
design_tech_node {N12 N10 N7 N7Plus N6 N5 N3 S11 S10 S8 S7 S5 S4 S3 G7 G5 ICF I7  
unspecified}
```

Default: unspecified

Data_type: enum {N22 N12 N10 N7 N7Plus N6 N5 N5PlusPlus N4 N3 N3E N2 A16 A14 A12 S11 S10
SF8 SF7 SF5 SF4 SF3 SF2 SF1.4 G7 G5 ICF I7 I5 P1222 P1276 P1277 P1278 P1278x P1280 20A
18A C12 C7 unspecified}

Sets the design technology node.

The node value is normally automatically set by `read_physical` while loading the LEF technology and the tool prints a message mentioning the auto-detected node. However, if the auto-detection does not work, you will see a message like this:

```
## Check design process and node:  
## Both design process and tech node are not set.
```

In such cases, you will need to set the value directly using `design_process_node` and `design_tech_node`.

design_top_routing_layer

```
design_top_routing_layer layerName
```

Default: ""

Data_type: string

Specifies the highest LEF layer name for global and detail routing.

design_trim_grid_group

```
design_trim_grid_group groupName
```

Default: ""

Data_type: string

Specifies the GROUP name of the set from which the trim metal grid will be used for placement and routing.

If the specified group name does not exist in LEF, the tool will return an ERROR.

dp

The root attributes in the `dp` category are:

- `apply_booth_encoding`
- `control_logic_optimization`
- `dp_analytical_opt`
- `dp_area_mode`
- `dp_csa`
- `dp_csa_factorize`
- `dp_opt_power_opt`
- `dp_opt_power_opt_scaling`
- `dp_opt_toggle_skew_threshold_for_booth_encoding`
- `dp_rename`
- `dp_sharing`
- `dp_speculation`
- `dp_ungroup_during_syn_map`

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name valu
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `dp` category, use the following command:

```
get_db -category dp
```

apply_booth_encoding

`apply_booth_encoding {nonbooth | auto_bitwidth | auto_togglerate | manual | inherited}`

Description

Default: `inherited` (for *hinst/inst*), `auto_bitwidth` (for *root*)

Data_type: `enum`, `read/write`

Applies booth encoding by using user-specified method (for all instances or a specified instance).

Applies to:

- `hinst`
- `inst`
- `root`

control_logic_optimization

`control_logic_optimization {advanced | basic | inherited | none}`

Description

Default: `inherited` (for *design/module*), `basic` (for *root*)

Data_type: `enum`, `read/write`

Controls the optimization of control logic (described in the RTL through conditional constructs like case statements, if-then-else statements, conditional selects, and so on) during generic synthesis.

You can specify any of the following values:

<code>basic</code>	Applies basic optimization.
<code>advanced</code>	Applies advanced level optimization.
<code>inherited</code>	Inherits the value from the <code>control_logic_optimization</code> root attribute. Note: Applies only to <code>design</code> and <code>module</code> .
<code>none</code>	Turns off optimization.

All control optimization transformations are verifiable. The advanced transformations might result in better QoR but can also increase the runtime. For best results, set this attribute after you have elaborated the design, but before generic synthesis.

Applies to:

- design
- module
- root

Related Information

- [synthesize_design](#)

dp_analytical_opt

dp_analytical_opt {standard | extreme | off}

Description

Default: standard

Data_type: enum, read/write

Specifies the effort level for analytical optimization of datapath logic. You can specify any of the following values:

standard	Enables analytical optimizations that reduce area without degrading timing.
extreme	Enables the most aggressive area-mode analytical optimizations, possibly degrading timing.
off	Disables analytical optimization.

For best results, set the attribute prior to the [elaborate_design](#) command.

Analytical optimization is not affected by the value the `dp_area_mode` attribute.

Related Information

- `elaborate_design`
- `synthesize_design`

dp_area_mode

`dp_area_mode {true | false}`

Description

Default: `false`

Data_type: `bool`, read/write

Setting the attribute to true enables datapath optimizations that focus on improving area results.

Related Information

- `elaborate_design`
- `synthesize_design`

dp_csa

`dp_csa {basic | inherited | none}`

Description

Default: `inherited` (for *design/module*), `basic` (for *root*)

Data_type: `enum`, read/write

Controls the carry-save adder (CSA) transformations in datapath synthesis. By default, CSA transformations are performed when synthesis is performed with the `-effort` option set to `high`.

You can specify any of the following values:

<code>basic</code>	Applies basic transformation.
<code>inherited</code>	Inherits the value from the <code>dp_csa</code> root attribute. Note: Applies only to <code>design</code> and <code>module</code> .
<code>none</code>	Turns off transformation.

Applies to:

- `design`
- `module`
- `root`

Related Information

- [`synthesize\_design`](#)

dp_csa_factorize

`dp_csa_factorize {basic | none}`

Default: `basic`

Data_type: enum {basic none}, read/write

Controls the CSA factorize optimization in datapath synthesis.

You can specify any of the following values:

<code>basic</code>	Enables factorization optimization.
--------------------	-------------------------------------

none	Turns off optimization.
------	-------------------------

Related Information

- [synthesize_design](#)

dp_opt_power_opt

`dp_opt_power_opt {false | true | inherited}`

Description

Default: `false` (root), `inherited` (design and module)

Data_type: `bool` (root), `enum` (design and module), read/write

Controls the datapath power optimization flow. Some runtime hit is expected.

Related Information

- [dp_opt_power_opt_scaling](#)
- [synthesize_design](#)

dp_opt_power_opt_scaling

`dp_opt_power_opt_scaling double`

Description

Default: `1.0`

Data_type: `double`, read/write

Adjusts the tradeoff between area and power within the datapath power optimization flow. For example:

- Smaller (< 1.0) values give less weight to predicted power and area change has a bigger impact on datapath decisions.
- Bigger (> 1.0) values accept larger area degradations if the power is expected to improve.

The recommended range is '0.1' to '10.0'.

Related Information

- `dp_opt_power_opt`
- `synthesize_design`

dp_opt_toggle_skew_threshold_for_booth_encoding

`dp_opt_toggle_skew_threshold_for_booth_encoding` *integer*

Description

Default: 0

Data_type: int, read/write

Sets the integer value for the skew threshold when in the `auto_togglerate` mode for `apply_booth_encoding`.

Related Information

- `apply_booth_encoding`

dp_rename

`dp_rename` {advanced | basic | inherited | none}

Description

Default: inherited (for *design/module*), basic (for *root*)

Data_type: enum, read/write

Controls how the datapath rewriting optimization is applied during synthesis and generic optimization with 'high' effort.

You can specify any one of the following values:

advanced	Applies advanced optimization
basic	Applies basic optimization.
inherited	Inherits the value from the root attribute. Note: Applies only to <code>design</code> and <code>module</code> .
none	Turns off optimization.

All basic transformations are verifiable. The advanced transformations might result in better QoR but not all of them are verifiable.

Applies to:

- `design`
- `module`
- `root`

Related Information

- `synthesize_design`

dp_sharing

`dp_sharing {inherited | none | advanced}`

Description

Default: `inherited` (for *design/hinst/module*) and `advanced` (for *root*)

Data_type: enum, read/write

Controls resource sharing in datapath on the design during `syn_generic` with high effort. If the value is set to `inherited`, the effort level of the optimization on the design will be identical to ("inherited from") the effort specified at the root.

<code>inherited</code>	Inherits the value from the <code>dp_sharing</code> root attribute. Note: Only applies to <code>design</code> , <code>hinst</code> , and <code>module</code> .
<code>advanced</code>	Applies advanced level optimization.
<code>none</code>	Turns off optimization.

Example

The following command turns off RTL sharing transformations:

```
set_db dp_sharing none
```

Applies to:

- `design`
- `hinst`
- `module`
- `root`

Related Information

- `synthesize_design`

dp_speculation

```
dp_speculation {basic | inherited | none}
```

Description

Default: `inherited` (for *design/module*), `basic` (for *root*)

Data_type: `enum`, `read/write`

Controls RTL speculation (unsharing) transformations in datapath synthesis.

You can specify any of the following values:

<code>basic</code>	When set to <code>basic</code> and the <code>dp_analytical_opt</code> attribute is set to <code>off</code>, speculation transformations are performed when synthesis is performed with the <code>syn_generic_effort</code> attribute set to <code>high</code>. In this case, speculation can only be applied to operators driven by a single MUX. When set to <code>basic</code> and the <code>dp_analytical_opt</code> attribute is set to <code>standard</code> or <code>extreme</code>, speculation are applied to operators driven by a chain of MUXES. In this case, speculation is independent of the <code>syn_generic_effort</code> setting.
<code>inherited</code>	Inherits the value from the <code>dp_speculation</code> root attribute. Note: Only applies to <code>design</code> and <code>module</code>.
<code>none</code>	Turns off RTL speculation (unsharing) transformations.

Applies to:

- `design`
- `module`
- `root`

Related Information

- `dp_analytical_opt`
- `synthesize_design`

`dp_ungroup_during_syn_map`

`dp_ungroup_during_syn_map {true | false}`

Description

Default: `true`

Data_type: `bool`, read/write

Controls whether the inferred datapath hierarchies and instantiated ChipWare hierarchies should be ungrouped during mapping (`syn_map`). When set to `false`, ungrouping occurs during optimization (`syn_opt`).

Related Information

- `synthesize_design`

eco

The root attributes in the eco category are:

- [eco_add_ports](#)
- [eco_batch_mode](#)
- [eco_check_logical_equivalence](#)
- [eco_client_tempus_executable](#)
- [eco_def_file_list](#)
- [eco_delay_calculation_effort](#)
- [eco_dynamic_view](#)
- [eco_drv_view_list](#)
- [eco_hold_view_list](#)
- [eco_honor_dont_touch](#)
- [eco_honor_dont_use](#)
- [eco_honor_fixed_status](#)
- [eco_honor_fixed_wires](#)
- [eco_honor_power_intent](#)
- [eco_leakage_view](#)
- [eco_lef_file_list](#)
- [eco_prefix](#)
- [eco_preserve_hpin_function](#)
- [eco_refine_place](#)
- [eco_select_partition_list](#)
- [eco_setup_view_list](#)
- [eco_si_effort](#)
- [eco_update_only_layers](#)
- [eco_update_timing](#)

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `eco` category, use the following command:

```
get_db -category eco
```

eco_add_ports

```
eco_add_ports {true|false}
```

Default: true

Data_type: Boolean

Determines whether to add hierarchical module ports during buffering.

Note: This attribute works only for the Manual ECO commands.

eco_batch_mode

```
eco_batch_mode {true|false}
```

Default: false

Data_type: Boolean

Speeds up the runtime in case the user performs lots of interactive ECO operations. When the user specifies `eco_batch_mode true`, various ECO operations are performed in a batch and initialization steps are not performed for every ECO call. In addition, updating timing is performed only once for all the ECO operations. Tool will only perform timing initialization at the first ECO command in the batch mode and timing update at the time of exiting the batchMode. User needs to specify `batchMode false` to exit the batchMode.

The batch mode is activated by `eco_batch_mode true` and exited by `eco_batch_mode false`, and only `eco_add_repeater`, `eco_delete_repeater` and `eco_update_cell` could be given in between `eco_batch_mode true` and `eco_batch_mode false`:

```
eco_batch_mode true
eco commands (eco_add_repeater, eco_delete_repeater, eco_update_cell)
eco_batch_mode false
```

If a user activates the batch mode, then the user needs to remember to exit it. If the user does not exit the batch mode, the user might see some unexpected results. For example, if the tool automatically added cell padding for blocks when `place_detail` is called by ECO commands for some reason, the cell padding will not be removed and the block size will change unless the user exits the batch mode.

All other settings should be made prior to activating the batch mode. If you would like to change settings, you need to exit the batch mode first.

Note: This option supports all commit ECO commands. Evaluation commands are not supported.

Related Commands

- `place_detail`

eco_check_logical_equivalence

`eco_check_logical_equivalence {true|false}`

Default: `true`

Data_type: Boolean

Enables functionality checking when swapping cells. The software can swap cells that are logically equivalent.

When `-eco_check_logical_equivalence` is `true`:

- `eco_add_repeater` adds inverter pair if the specified cell is inverter.
- `eco_delete_repeater` deletes inverter pair if the specified instance is inverter.
- `eco_update_cell` changes cell to another cell type with the same function as long as they have same number of pins.

When `-eco_check_logical_equivalence` is set to `false`:

- `eco_add_repeater` can add only one single inverter if the specified cell is inverter.
- `eco_delete_repeater` can delete only one single inverter if the specified instance is inverter.
- `eco_update_cell` can perform nonequivalent cell change as long as they have same number of pins.

Related Commands

- `eco_add_repeater`
- `eco_delete_repeater`
- `eco_update_cell`

`eco_client_tempus_executable`

`eco_client_tempus_executable` *string*

Default: " "

Data_type: [String](#)

Sets the executable name for Tempus clients in ECO mode.

`eco_def_file_list`

`eco_def_file_list` *string*

Default: " "

Data_type: [String](#)

Sets the DEF files for using with the Tempus logical flow.

`eco_delay_calculation_effort`

`eco_delay_calculation_effort` {`low` | `medium` | `high`}

Default: `high`

Data_type: [String](#)

Determines the delay calculation effort level as `low`, `medium`, or `high`.

`eco_dynamic_view`

`eco_dynamic_view` *string*

Default: " "

Data_type: [String](#)

Sets the dynamic power view name for the ECO master session.

Note: This global is used only in the Paradime ECO flow.

eco_drv_view_list

`eco_drv_view_list` *string*

Default: " "

Data_type: String

Specifies the list of DRV views for Tempus ECO run.

Note: This global is used only in the Paradime ECO flow. The views specified in `eco_drv_view_list` along with `eco_setup_view_list` will be considered as DRV views.

eco_hold_view_list

`eco_hold_view_list` *string*

Default: " "

Data_type: String

Sets the view names for Tempus ECO master.

eco_honor_dont_touch

`eco_honor_dont_touch` *cellName*

Default: true

Data_type: Boolean

Checks for the don't touch attribute on nets and instances. If you specify a terminal list, this parameter checks each terminal for the don't touch attribute. The `ecoAddRepeater` command honors the don't touch nets. The `eco_delete_repeater` and `eco_update_cell` commands honor don't touch instances.

Related Commands

- `eco_delete_repeater`
- `eco_update_cell`

eco_honor_dont_use

`eco_honor_dont_use {true|false}`

Default: `true`

Data_type: [Boolean](#)

Checks for the don't use attribute on cells. The `eco_add_repeater`, `eco_delete_repeater`, and `eco_update_cell` commands honor don't use cells.

Related Commands

- `eco_add_repeater`
- `eco_delete_repeater`
- `eco_update_cell`

eco_honor_fixed_status

`eco_honor_fixed_status {true|false}`

Default: `true`

Data_type: [Boolean](#)

Checks for fixed net wires.

It restricts the addition or deletion on fixed net so that the fixed status of the wires is not altered.

eco_honor_fixed_wires

`eco_honor_fixed_wires {true|false}`

Default: `true`

Data_type: [Boolean](#)

Checks for fixed net wires.

It restricts the addition or deletion on fixed net so that the fixed status of the wires is not altered.

When `-eco_honor_fixed_wires` is set to `false`, `eco_add_repeater` can insert a cell on the wires with the FIXED/COVER status and change the status of these wires from FIXED/COVER to ROUTED.

eco_honor_power_intent

`eco_honor_power_intent {true|false}`

Default: `true`

Data_type: Boolean

Performs Multi Supply Voltage Design (MSV) checks during ECO.

When this parameter is set to `true`:

- `eco_update_cell` does not allow:
 - Resizing a regular cell with an always-on cell (and vice-versa).
 - Resizing with cells belonging to different power domains.
- `eco_add_repeater` does not allow:
 - Adding buffer to a power domain it doesn't belong to.
 - Buffering a cross power domain net if it can cause MSV violations.
 - Adding a regular buffer to a net that requires an always-on buffer.
 - Adding an always-on buffer to a net that does not require an always-on buffer.
- `eco_delete_repeater` does not allow deletion of a buffer or an inverter-pair that will cause:
 - A redundant isolation cell.
 - A CPF-constrained net to be deleted.

eco_leakage_view

`eco_leakage_view string`

Default: " "

Data_type: String

Sets the leakage power view name for the ECO master session.

Note: This global variable is used only in the Paradime ECO flow.

eco_lef_file_list

`eco_lef_file_list` *string*

Default: " "

Data_type: [String](#)

Sets the LEF files for using with Tempus logical flow.

eco_prefix

`eco_prefix` *prefix*

Default: ECO

Data_type: [String](#)

Adds a prefix to the inserted cells.

eco_preserve_hpin_function

`eco_preserve_hpin_function` *prefix*

Default: false

Data_type: [Boolean](#)

Determines whether to preserve logical functions at the hierarchical module ports.

Note: This option works only for manual ECO commands.

eco_refine_place

`eco_refine_place` {true|false}

Default: true

Data_type: [Boolean](#)

Legalizes placement whenever `eco_add_repeater` or `eco_update_cell` are used. If false, you must

`run_place_detail` after ECO if you want to legalize placement.

Note: Specifying `eco_refine_place true` can affect performance when large numbers of cells are added or modified.

eco_select_partition_list

`eco_select_partition_list string`

Default: " "

Data_type: [String](#)

Lists the subset of partitions for which the physical data has to be loaded while merging the DEF files although user might point to all available DEF files using the `eco_def_file_list` global variable.

Note: This global variable is used only in the Paradime ECO flow.

eco_setup_view_list

`eco_setup_view_list string`

Default: " "

Data_type: [String](#)

Sets the view names for Tempus ECO master.

eco_si_effort

`eco_si_effort {low|medium}`

Default: `low`

Data_type: [String](#)

Sets the SI effort level as `low` or `medium` used in the timing update.

eco_update_only_layers

`eco_update_only_layers string`

Default: " "

Data_type: String

Specifies the layers on which the wires can be cut during buffering.

eco_update_timing

`eco_update_timing {true|false}`

Default: true

Data_type: Boolean

Controls whether to `eco*` commands recompute timing. If `true`, the `eco*` commands recompute timing. If `false`, the commands do not recompute timing, and us can later use `report_timing` to recompute timing.

Related Commands

- [eco_add_repeater](#)
- [eco_delete_repeater](#)
- [eco_update_cell](#)

extract_rc

The root attributes in the `extract_rc` category are:

- [extract_rc_cap_filter_mode](#)
- [extract_rc_cerebrus_license_only](#)
- [extract_rc_compress_rcdb](#)
- [extract_rc_coupled](#)
- [extract_rc_coupling_cap_threshold](#)
- [extract_rc_lef_tech_file_map](#)
- [extract_rc_local_cpu](#)
- [extract_rc_pvs_fill](#)
- [extract_rc_qrc_cmd_file](#)
- [extract_rc_qrc_cmd_type](#)

- [extract_rc_qrc_stream_map_file](#)
- [extract_rc_qrc_run_mode](#)
- [extract_rc_relative_cap_threshold](#)
- [extract_rc_shrink_factor](#)
- [extract_rc_signoff_stream_layer_map](#)
- [extract_rc_total_cap_threshold](#)
- [extract_rc_tsv_subckt_file](#)
- [extract_rc_turbo_reduce](#)
- [extract_rc_write_def_options_for_signoff_extract](#)

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `extract_rc` category, use the following command:

```
get_db -category extract_rc
```

extract_rc_cap_filter_mode

```
extract_rc_cap_filter_mode { relative_only | relative_and_coupling |  
relative_or_coupling }
```

Default: relative_and_coupling

Note: For standalone Quantus extraction, if the process node is 130nm or below, the default value is `relative_and_coupling`. If the process node is above 130nm, the default value is `relative_only`.

Data_type: Enum

Specifies the capacitance filtering mode. The filtering modes are conditions which define how to use the `extract_rc_coupled` and `extract_rc_coupling_cap_threshold` attribute values, or their combination. Based on the filtering mode specified, the software determines the nets in the design, which will have their coupling capacitance lumped to the ground.

Note: The behavior of the `extract_rc_total_cap_threshold` attribute is independent of

the `extract_rc_cap_filter_mode` attribute setting. Nets that have a total capacitance value lower than the value specified with the `extract_rc_total_cap_threshold` attribute will have their coupling capacitances grounded regardless of the `extract_rc_cap_filter_mode` attribute setting.

You can use one of the following modes:

- `relative_only`: Grounds the coupling capacitance of nets based on the threshold value set by the `extract_rc_relative_cap_threshold` attribute.
- `relative_and_coupling`: Grounds the coupling capacitance of nets only when it is lower than the threshold value specified with the `extract_rc_relative_cap_threshold` attribute as well as the `extract_rc_coupling_cap_threshold` attribute. In this mode, you enforce maximum restriction on grounding the coupling capacitances.
- `relative_or_coupling`: Grounds the coupling capacitance of nets if it is lower than the threshold value specified with one of the `extract_rc_relative_cap_threshold` or `extract_rc_coupling_cap_threshold` attributes.

Related Command

- `extract_parasitics`

`extract_rc_cerebrus_license_only`

```
extract_rc_cerebrus_license_only {true | false}
```

Default: false

Data_type: Boolean

This attribute is applicable to the Cerebrus-Quantus flow.

When set to `true`, only the Cerebrus license will be considered and Innovus will not fall back to use Quantus (QTS) license when Cerebrus license(s) is/are not available. By default, the attribute is set to `false`, which means the regular QTS license will be considered when Cerebrus license(s) is/are not available.

`extract_rc_compress_rcdb`

```
extract_rc_compress_rcdb {true | false}
```

Default: false

Data_type: Boolean

Generates compressed RCDB data.

Note: This functionality reduces the disk usage but leads to marginal increase in run time.

Related Command

- `extract_parasitics`

extract_rc_coupled

```
extract_rc_coupled {true | false}
```

Default: true

Data_type: Boolean

Separately outputs grounded and coupling capacitance. The 0 value of this attribute is typically used for static timing analysis. Signal Integrity analysis requires the coupled mode.

extract_rc_coupling_cap_threshold

```
extract_rc_coupling_cap_threshold file_name
```

Default: 0.1

Data_type: Double

Specifies the threshold value (in femtoFarads) that determines when the extractor lumps a net's coupling capacitance to ground. This attribute is applicable only when the `extract_rc_coupled` attribute is set to 1. The software decouples the coupling capacitance of nets when the total coupling capacitance between the pair of nets is lower than the threshold specified with this attribute. The actual decoupling also depends on the setting of the `extract_rc_cap_filter_mode` attribute.

Related Command

- `extract_parasitics`

extract_rc_lef_tech_file_map

`extract_rc_lef_tech_file_map filename`

Default: ""

Data_type: String

Specifies the location of the optional layer map file for the Standalone Quantus QRC engine.

Note: If used, the layer map file must be in the Common Command Language (CCL) format. The following is an example of the layer map file format:

```
extraction_setup \  
-technology_layer_map \  
# <LEF/DEF Name> #<Tech File Name>  
M1 METAL_1 \  
VIA1 VIA_1 \  
M2 METAL_2 \
```

Note: For cell-level Quantus layermap, the DEF/LEF layer names are case sensitive, but ICT layer names are case-insensitive.

Related Command

- `extract_parasitics`

extract_rc_local_cpu

`extract_rc_local_cpu number_of_cpus`

Default: 0

(As specified in the `set_multi_cpu_usage -local_cpu` setting.)

Data_type: Integer

Specifies the number of CPUs to be used for IQuantus extraction. This attribute is only applicable for the IQuantus extraction engine. When specified, it overrides the `set_multi_cpu_usage -local_cpu` option for IQuantus.

Related Command

- `extract_parasitics`

extract_rc_pvs_fill

```
extract_rc_pvs_fill {true | false}
```

Default: false

Data_type: Boolean

When this attribute is set to true, the PVS fill data attached to the Innovus database is sent to Quantus so that the effects of the fill data are extracted. This attribute only affects signoff Quantus extraction. It has no effect on tQuantus or iQuantus extraction done inside Stylus. This attribute requires that either the `extract_rc_qrc_stream_map_file` attribute or the `extract_rc_signoff_stream_layer_map` attribute is specified.

extract_rc_qrc_cmd_file

```
extract_rc_qrc_cmd_file filename
```

Default: ""

Data_type: String

Specifies the user-created Quantus command file to be used for running Standalone Quantus QRC. This command file should be in the new CCL syntax.

Note: This attribute is only valid if `extract_rc_qrc_cmd_type` `partial` or `custom` is selected.

Related Command

- `extract_parasitics`

extract_rc_qrc_cmd_type

```
extract_rc_qrc_cmd_type {auto | partial | custom}
```

Default: auto

Data_type: Enum

Specifies whether to derive the settings for running Quantus from the software settings, or from user-created command file using the `extract_rc_qrc_cmd_file` attribute.

This attribute supports the following values:

- `auto`: Derives the Quantus settings automatically from the software parameter settings.

Note: In this mode, the `extract_rc_lef_tech_file_map` attribute is required to be specified.

- **partial:** Appends user commands from the `qrcCmdFile` to the settings derived from the software parameter settings. You must provide the partial mode command file in CCL syntax. Use the partial command file only when you add extraction directives for those options that cannot be set in software. Following are some important restrictions:
 - Do not specify the following options in the partial mode command file:
 - Threshold setting
 - Coupled/decoupled mode
 - Quantus Techfile (s)
 - Operating Temperature
 - Do not specify "`-directory_name`" and "`-temporary_directory_name`" and "`-file_name`" in `output_setup` section of the command file
 - Do not specify output file using `output_setup -file_name` command in partial command file

There is no checking for syntax or content of this file, and it will be appended as is to the automatically created Quantus command file.

- **custom:** Gets the complete Quantus settings from the user-created command file. Except for input and output, no settings from software parameters are used. You must provide the complete Quantus command file in CCL syntax.

Following are some important restrictions:

- Do not specify an input DEF/OA DB in the custom mode command file
- Do not specify "`-directory_name`" and "`-temporary_directory_name`" in `output_setup` section of the command file.
- Specify output SPEF with the following convention for the name: `<design.spef>`.
In MMC, the output SPEF files will be named: `<design>_<corner>_<Temperature>.spef`. Here "design" refers to the top cell name.

Note: Do not prefix or postfix the output file name.

- Specify the Quantus Techfile(s) to be used. In MMC, the corner names should match the

names used in the `create_rc_corner` command

- Provide Quantus Techfile layer mapping information in custom command file
- If GDS is provided for cell contents, provide a Quantus Techfile to GDS layer number matching

Note: There is no checking for syntax or content of the command file prior to running the extraction. The command file will be used as is. If the output file is not specified correctly, the resulting parasitic file will not be read into the Tempus RC database.

Related Command

- `extract_parasitics`

extract_rc_qrc_stream_map_file

`extract_rc_qrc_stream_map_file filename`

Default: ""

Data_type: String

This attribute is specified only when `-pvs_fill` parameter is set to true. When the `extract_rc_pvs_fill` attribute is set to true, a Quantus stream/oasis map Tcl command is required to map the PVS layer/dtype data to the corresponding Quantus layers.

If the `pvs_fill.data` is generated based on (GDS), the map file should be in the following format:

```
#layer gds_layer gds_dtype
<layer_name> <layer_number> <data_type>
...
```

If the `pvs_fill.data` is generated based on OASIS, the map file should be in the following format:

```
#layer oasis_layer oasis_dtype
<layer_name> <layer_number> <data_type>
...
```

Where,

- `layer_name` is the layername in the design

- `layer_number` is the layer ID in the GDS/OASIS file
- `data_type` is the datatype specified in the GDS/OASIS file

For example,

```
M1  31  0
```

```
M1  31  1
```

```
M2  32  0
```

```
M2  32  1
```

Related Command

- [extract_parasitics](#)

extract_rc_qrc_run_mode

```
extract_rc_qrc_run_mode {concurrent | sequential}
```

Default: `concurrent`

Data_type: [Enum](#)

Specifies the Quantus run mode in MMMC designs.

Note: For MMMC designs, in Quantus partial command type, do not specify the output SPEF file name and RC corner name in the command file to run Quantus in concurrent or sequential mode. For MMMC designs, in Quantus custom command type, do not specify the output SPEF file name and RC corner name together in the command file. Quantus sequential mode is not supported in custom command type.

This attribute supports the following values:

- `concurrent` : Runs Quantus for all active corners concurrently.
- `sequential` : Runs Quantus for all active corners sequentially.

Related Command

- [extract_parasitics](#)

extract_rc_relative_cap_threshold

`extract_rc_relative_cap_threshold value`

Default: 1.0

Data_type: Double

Sets a ratio threshold value that determines when the extractor lumps a net's coupling capacitance to ground. This attribute is applicable only when the `extract_rc_coupled` attribute is set to 1. If the total coupling capacitance between a pair of nets is less than the percentage (specified with this attribute) of the total capacitance of the net with the smaller total capacitance in the pair, the coupling capacitance between these two nets will be considered for grounding. The actual decoupling also depends on the setting of the `extract_rc_cap_filter_mode` attribute.

The range of this value is 0 to 1.

Related Command

- `extract_parasitics`

extract_rc_shrink_factor

`extract_rc_shrink_factor shrink_factor_value`

Default: 1.0

Data_type: Double

Sets the value by which the software shrinks the design before extraction. The shrink factor is used only for extraction calculations. It does not change the actual design. This attribute must only be used when restoring design.

Note: This attribute is not recommended for newer technologies where shrink factor is defined in the Quantus QRC techfiles or capacitance tables.

Related Command

- `extract_parasitics`

extract_rc_signoff_stream_layer_map

`extract_rc_signoff_stream_layer_map file_name`

Default: ""

Data_type: String

This attribute is specified only when the `extract_rc_pvs_fill` attribute is set to `true`. When the `extract_rc_pvs_fill` attribute is set to `true`, a Quantus stream/oasis map Tcl command is required to map the PVS layer/dtype data to the corresponding Quantus layers.

If the `pvs_fill.data` is generated based on stream (GDS/OASIS), the map file should be in the following format:

```
#layer gds_layer gds_dtype use-type color-ID  
  
<layer_name> <layer_number> <data_type> <use-type> <color-ID>  
  
...
```

Where,

- `layer_name` is the LEF layername in the design
- `layer_number` is the layer ID in the GDS/OASIS file
- `data_type` is the datatype of the layer in the GDS/OASIS file
- `use_type` is the keyword for identifying the use of the data. The valid values are:
 - `gray`: library data in the GDS/OASIS file
 - `fill`: regular metal fill data in the GDS/OASIS file
 - `active`: active metal fill data in the GDS/OASIS file
- `color-ID` is the color ID. Use “1” or “2” if the layer is colored, or use hyphen “-” if the layer is not colored

For example,

```
M1  31  0 gray -  
M1  31  1 fill -  
M2  32  0 gray -  
M2  32  1 fill -
```

Related Command

- `extract_parasitics`

extract_rc_total_cap_threshold

`extract_rc_total_cap_threshold value_in_fF`

Default: 0.0

Data_type: Double

Specifies the threshold value (femtoFarads) that determines when the extractor lumps a net's coupling capacitance to ground. This attribute is applicable only when the `extract_rc_coupled` attribute is set to 1. The software grounds the coupling capacitances for the nets which have a total capacitance value less than the value specified with this attribute.

Related Command

- `extract_parasitics`

extract_rc_tsv_subckt_file

`extract_rc_tsv_subckt_file filename`

Default: ""

Data_type: String

Specifies the name of the Through Silicon Via (TSV) subckt file to be used. The file contains the .subckt model information for TSV. The file has a Spice format. A sample file is shown below.

```
.subckt TSV top bottom
R1 top n2 0.03075
R2 n2 bottom 0.03075
C1 n2 0 28f
.ends
```

Where,

`top` and `bottom` refer to the terminal nodes on the top and bottom layers of the TSV

`n2` is the internal node between the top and bottom layers

`R1 top n2 0.03075` means that a resistor with a resistance of `0.03075 Ohms` is added between the `top` and `n2` nodes

`R2 n2 bottom 0.03075` means that a resistor with a resistance of `0.03075 Ohms` is added between the `bottom` and `n2` nodes

C1 n2 0 28f means that a ground capacitance of 28 fF is added on the n2 node

Note: The IQuantus engine might not keep the subckt connectivity as it is. The optimization of RES network will remove node n2 (if there is no other connections) and keep a single resistor between the top and bottom nodes with an accumulated resistance. For example, in the above example, the accumulated resistance will be 0.0615 Ohms. Capacitance will also be moved to a “higher layered node”. In the above example, this layer is top.

Related Command

- `extract_parasitics`

extract_rc_turbo_reduce

```
extract_rc_turbo_reduce {0 | 1 | auto}
```

Default: auto

Data_type: Enum

Controls the turbo reduction feature for Standalone Quantus QRC extraction. You can set the following options:

- **auto:** When set to `auto` , the software automatically turns on turbo reduction and proceeds when the required Quantus QRC license is available. However, if the required Quantus QRC license is not installed in the license server, the software automatically turns off Turbo reduction and proceeds with a warning message. If the license is installed in the license server but temporarily not available, then two scenarios are possible. First, if the license wait mode is not turned on, then Quantus will look for the license right away and error out because the license is not available. Second, if the license wait mode is turned on, then Quantus will wait and check for the license periodically until timeout. If the license is available before the timeout, turbo reduction will be performed but if the license is not available before timeout, QRC will error out.
- **1:** When set to `true` , the turbo reduction engine is on . However, if the required Quantus QRC license is not installed in the license server, then standalone Quantus will error out with a message and the run will stop. It is because setting the option to true means the user really wants to perform turbo reduction.
- **0:** The turbo reduction engine is switched off .

For details, see the table below.

Behavior of Turbo Reduction

	Availability of Quantus QRC AA License		
Option Selected – extract_rc_turbo_reduction {1 0 auto}	<u>Quantus QRC AA Installed and Available</u> (IQRC gets the license right away even in the wait mode because sufficient licenses are available in the license server)	<u>Quantus QRC AA Installed but not Available</u> (If license wait mode is not turned on, then Quantus looks for the license. If the wait mode is turned on and if the license is temporarily not available, then IQRC waits and checks for the license periodically until timeout.)	<u>Quantus QRC AA Not Installed on License Server</u>
auto	perform turbo reduction	1. error out if wait mode is not on or if the specified wait time is exceeded 2. perform turbo reduction if wait mode is on and the license is available within the specified time.	print warning and do not perform turbo reduction
1	perform turbo reduction	error out	error out because the user explicitly wants to use turbo reduction
0	do not perform turbo reduction	do not perform turbo reduction	do not perform turbo reduction

extract_rc_write_def_options_for_signoff_extract

extract_rc_write_def_options_for_signoff_extract *list_of_def_options*

Default: ""

Data_type: String

Specifies the list of `write_def` options for signoff extraction.

For signoff extraction, `set_db extract_rc_engine post_route set_db extract_rc_effort_level signoff`, the software internally calls the `write_def` command. However, in some specific scenarios, you need to provide additional options for the `write_def` command. Use this attribute to provide the list of `write_def` options to be included.

When this attribute is set, `extract_parasitics` will include these options while calling the `write_def` command.

Related Command

- `extract_parasitics`

flip_chip

The root attributes in the `flip_chip` category are:

- `flip_chip_allow_layer_change`
- `flip_chip_allow_routed_bump_edit`
- `flip_chip_auto_pairing_file`
- `flip_chip_bottom_layer`
- `flip_chip_bump_use_octagon_shape`
- `flip_chip_cell_connection_type`
- `flip_chip_check_bump_access_directions`
- `flip_chip_compaction`
- `flip_chip_connect_bump_to_block`
- `flip_chip_connect_bump_to_bump`
- `flip_chip_connect_power_cell_to_bump`
- `flip_chip_constraint_file`
- `flip_chip_drop_via_on_all_shapes`
- `flip_chip_drop_via_on_power_mesh`
- `flip_chip_extra_config`
- `flip_chip_finger_direction`
- `flip_chip_finger_max_width`
- `flip_chip_finger_min_width`
- `flip_chip_finger_target_mesh_layer_range`
- `flip_chip_honor_bump_connect_target_constraint`
- `flip_chip_ignore_pad_type_check`
- `flip_chip_lower_layer_route_width`
- `flip_chip_lower_layer_prevent_diagonal_routing`
- `flip_chip_merge_nearby_pin`

- [flip_chip_multi_pad_routing_style](#)
- [flip_chip_multiple_connection](#)
- [flip_chip_pg_mesh_direction](#)
- [flip_chip_pg_mesh_main_width](#)
- [flip_chip_pg_mesh_max_width](#)
- [flip_chip_pg_mesh_width](#)
- [flip_chip_prevent_diagonal_wire_access_bumps](#)
- [flip_chip_prevent_via_under_bump](#)
- [flip_chip_prevent_via_under_bump_extension](#)
- [flip_chip_route_bump_cluster](#)
- [flip_chip_route_pg_style](#)
- [flip_chip_route_style](#)
- [flip_chip_route_width](#)
- [flip_chip_top_layer](#)
- [flip_chip_via_abut_bump](#)
- [flip_chip_wire_to_pad_honor_min_spacing](#)

To set the value of any root attribute, use the [set_db](#) command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the [get_db](#) command:

```
get_db attribute_name
```

To get the current value of all attributes in the `flip_chip` category, use the following command:

```
get_db -category flip_chip
```

flip_chip_allow_layer_change

```
flip_chip_allow_layer_change {true | false}
```

Default: `false`

Data_type: `bool`, read/write

Turns on the layer change feature, when set to `true`. Only the PIO mode supports this feature. Both Manhattan and 45-degree routing support this constraint.

Different setting for routing layers will have the different behaviors as shown in the following examples. Assume the top RDL is `TOP_RDL` and the second top RDL is `2nd_RDL`:

- Use `TOP_RDL` as much as possible and `2nd_RDL` is used only when a single layer cannot finish routing in case of cross-over:

```
flip_chip_bottom_layer TOP_RDL flip_chip_top_layer TOP_RDL
```

- Freely change layers so that the tool will use the layer resources by its intelligence

```
flip_chip_bottom_layer 2nd_RDL flip_chip_top_layer TOP_RDL
```

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_flip_chip`

flip_chip_allow_routed_bump_edit

```
flip_chip_allow_routed_bump_edit {true | false}
```

Default: `true`

Data_type: `bool`, read/write

Specifies whether Innovus should check routing status of bumps before allowing manipulation. By default, the tool ignores routing status and allows manipulation of routed bumps. Manipulation actions include moving, deleting, re-assigning, and swapping. Set this attribute to `false` if you want the tool to check routing status of bumps and disallow manipulation of routed bumps.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `assign_bumps`
- `assign_signal_to_bump`
- `copy_bumps`
- `delete_bumps`
- `swap_bump_assignment`
- `unassign_bumps`
- `align_selected`
- `flip_or_rotate_object`
- `move_obj`
- `space_selected`

flip_chip_auto_pairing_file

```
flip_chip_auto_pairing_file filename
```

Default: ""

Data_type: string, read/write

Specifies the output file name to dump out `route_flip_chip` auto pairing information for bump to I/O connection. Depending on the design, the format of the output file can be as follows:

- If the connection between the bump and the instance is known, the format is:

```
create_bump_connect_target_constraint -bumps bump_name -io_inst inst_name
```
- If the connection between the bump and the instance pin is known, the format is:

```
create_bump_connect_target_constraint -bumps bump_name -io_inst inst_name -  
pin_name pin_name
```
- If the connection between the bump and the pin port of the instance is known, the format is:

```
create_bump_connect_target_constraint -bumps bump_name -io_inst inst_name -  
pin_name pin_name -port_num port_number
```

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_flip_chip`
- `create_bump_connect_target_constraint`

flip_chip_bottom_layer

```
flip_chip_bottom_layer layer_name
```

Default: ""

Data_type: string, read/write

Specifies the bottom-most metal layer by layer number or layer name that the software can use when routing bumps.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `place_peripheral_ios`
- `route_flip_chip`

flip_chip_bump_use_octagon_shape

```
flip_chip_bump_use_octagon_shape {true | false}
```

Default: `false`

Data_type: `bool`, read/write

Controls how `route_flip_chip` handles special bumps with multiple shapes:

- When this attribute is set to `true`, the tool uses an octagonal shape to mimic the special bump.
- When this attribute is set to `false`, the tool:
 - Uses the bump's exact LEF description if the bump is simple enough.
 - May replace the bump shape with an octagonal shape if the bump is not simple according to the tool's internal criterion.

Note: When the special bump is replaced with an octagonal shape, the routing quality may be improved, but there may be DRC violations.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `route_flip_chip`

flip_chip_cell_connection_type

`flip_chip_cell_connection_type {route_width | bump_edge | bump_center | bump_end}`

Default: `bump_center`

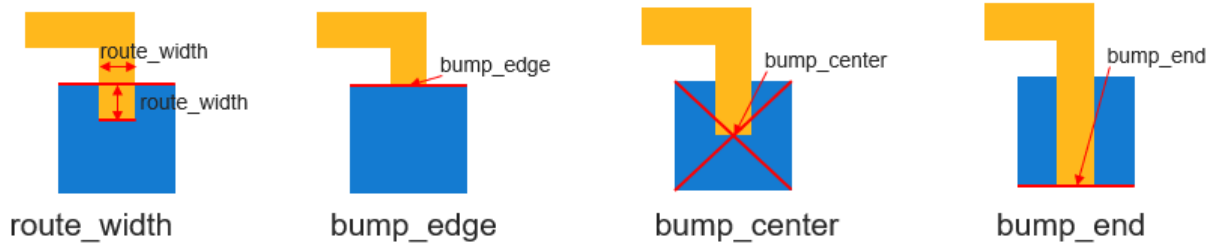
Data_type: `enum {route_width bump_edge bump_center bump_end}`, read/write

Specifies the type of bump cell connections. Each type represents a distinct connection method:

- `route_width`: Connects the cell by extending the routing path by a distance equal to one route width.
- `bump_edge`: Connects the cell to the outer edge of the bump. In this configuration, the connection terminates precisely at the bump's perimeter and does not extend further into the

bump structure.

- **bump_center**: Connects the cell directly to the geometric center of the bump.
- **bump_end**: Connects the cell to the bump edge and then continues the connection path until it reaches the far end of the bump structure.



This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `route_flip_chip`

flip_chip_check_bump_access_directions

`flip_chip_check_bump_access_directions {true | false}`

Default: false

Data_type: bool, read/write

Specifies whether `route_flip_chip` should check the access direction of bump cells. Use this attribute if you have an octagonal bump pad structure with a pre-drawing wire for outside access, where the pre-drawing wire is a pin in the bump LEF.

- When this attribute is set to `true`, `route_flip_chip` checks the bump LEF to detect the bump access direction and then connects the pad to the bump pre-routed wire based on the detected direction. If the pre-routed wire is a 45-degree wire, the `manhattan` routing style is not supported.

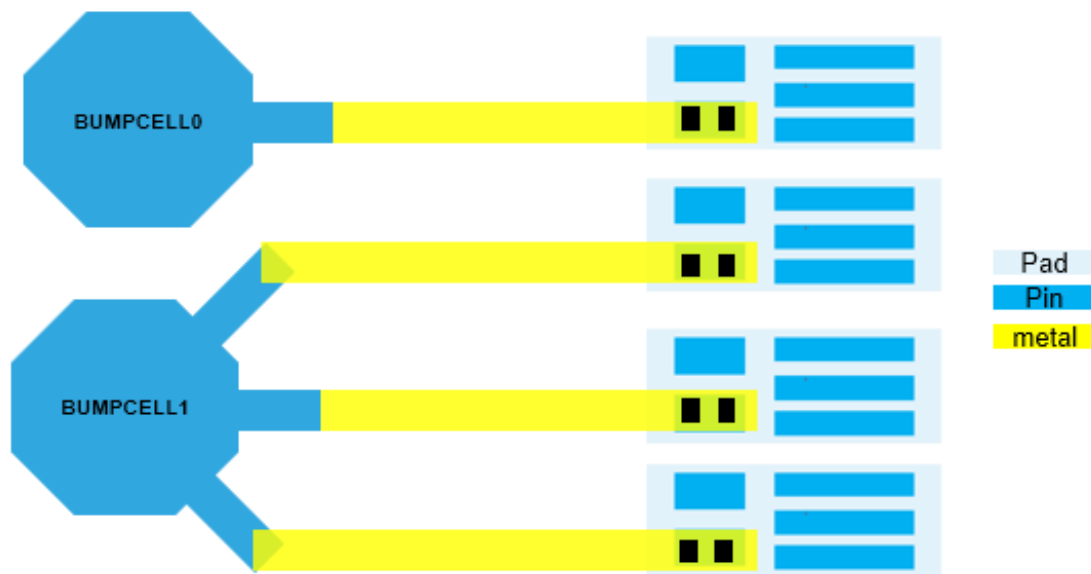
Note that one bump cell can have multiple access directions.

- When this attribute is set to `false`, `route_flip_chip` skips checking bump access direction.

Note: This attribute is applicable only for `route_flip_chip -target connect_bump_to_pad`. If this option is used for `route_flip_chip -target connect_bump_to_ring_stripe`, `route_flip_chip` will ignore the setting and display a warning.

In the example below, `route_flip_chip` checks the bump LEF to detect the bump access directions and then connects the pads to the bump pre-routed wires based on the detected directions:

```
> set_db flip_chip_check_bump_access_directions true  
> route_flip_chip -target connect_bump_to_pad -route_engine global_detail
```



This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `route_flip_chip`

flip_chip_compaction

`flip_chip_compaction {true | false}`

Default: `false`

Data_type: `bool`, read/write

Enables compact routing, when set to `true`.

In the flip chip post-route stage, the compaction function pushes the routing as close as possible to bumps in order to leave more routing resource for subsequent routing steps, such as P/G bump connections or general power routing.

Note: This feature cannot be used with resistance driven and diffpair routings because the compaction action may change the wire length and routing pattern. This feature does not support two-layer RDL routing.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
- To get the current value of any root attribute, use the `get_db` command:

```
set_db attribute_name value
```

```
get_db attribute_name
```

Related Commands

- `place_peripheral_ios`
- `route_flip_chip`

flip_chip_connect_bump_to_block

`flip_chip_connect_bump_to_block {true | false}`

Default: `false`

Data_type: `bool`, read/write

Connects bumps directly to block pins. When this option is enabled, `route_flip_chip` supports the routing of bumps to macro pins, even if the `CLASS BUMP` keyword is absent from the macro LEF.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_flip_chip`

flip_chip_connect_bump_to_bump

```
flip_chip_connect_bump_to_bump {true | false}
```

Default: `false`

Data_type: `bool`, read/write

Enables bump-to-bump routing. When this option is set to `true`, the tool can connect all the bumps with the same net in the area-based routing mode.

This option works only in the area-based routing mode.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_flip_chip`

flip_chip_connect_power_cell_to_bump

```
flip_chip_connect_power_cell_to_bump {true | false}
```

Default: `false`

Data_type: `bool`, read/write

Connects all power bumps to the I/O cell pin. It does not connect to a power or ground stripe or ring.

When used with the `route_flip_chip -target signal` parameter, you can connect power bumps to I/O power pads. To make this type of connection, you need the following information:

- The Verilog netlist must contain the I/O power pad, for example:

```
VDDCELL VDD_INST (.PAD());
```

- The LEF macro pin must contain the USE POWER statement, for example:

```
MACRO VDDCELL  
PIN PAD  
  
USE POWER;
```

To improve the quality of routing in situations where there may be many power bumps in the design, you can pair power bumps to power cells by using this attribute in conjunction with the `flip_chip_constraint_file` attribute. For example:

```
set_db flip_chip_connect_power_cell_to_bump true
```

```
set_db flip_chip_constraint_file pair.const
```

where `pair.const` specifies the following information:

```
### Syntax: net pad [pad2] bump [bump2] ###
```

```
PAIR
```

```
VDD IOPADS_INST/esd Bump_90_9_8
```

```
END PAIR
```

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `place_peripheral_ios`
- `route_flip_chip`

flip_chip_constraint_file

`flip_chip_constraint_file file_name`

Default: ""

Data_type: string, read/write

Specifies the file that contains constraints for flip chip routing. For more information, see [Useful Constraints for Flip Chip Routing](#) in the *User Guide*.

To set the constraint file, you can use this attribute as follows:

```
set_db flip_chip_constraint_file share_pair.const
```

where `share_pair.const` specifies the following information:

```
### Syntax: net pad [pad2] bump [bump2] ###
```

```
SHARE_PAIR
```

```
VDD IOPADS_INST/esd Bump_90_9_8
```

```
END SHARE_PAIR
```

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `place_peripheral_ios`
- `route_flip_chip`

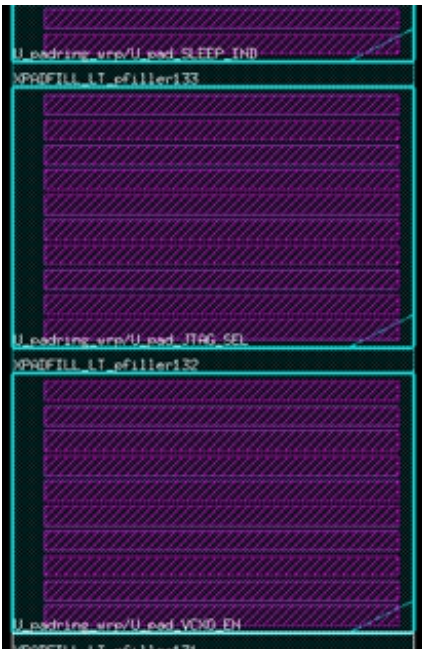
flip_chip_drop_via_on_all_shapes

`flip_chip_drop_via_on_all_shapes {true | false}`

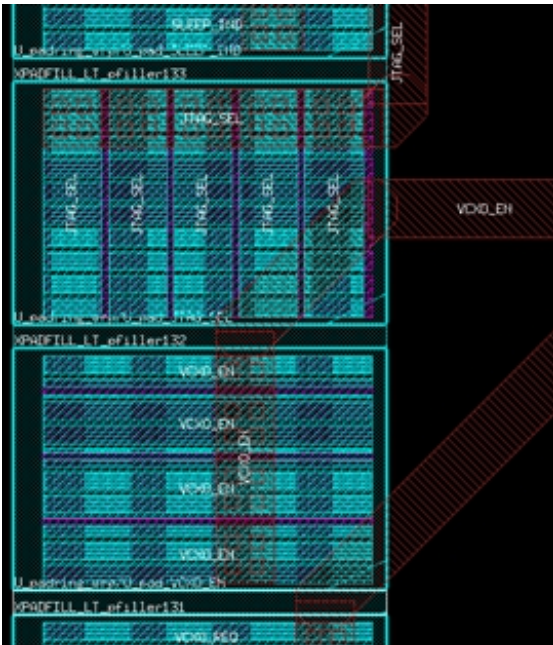
Default: false

Data_type: bool, read/write

Drops vias on all defined geometries. Consider the pin shape below:



For the given shape, the routing result will be as follows when `flip_chip_drop_via_on_all_shapes` is set to `true`:



This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `place_peripheral_ios`
- `route_flip_chip`

flip_chip_drop_via_on_power_mesh

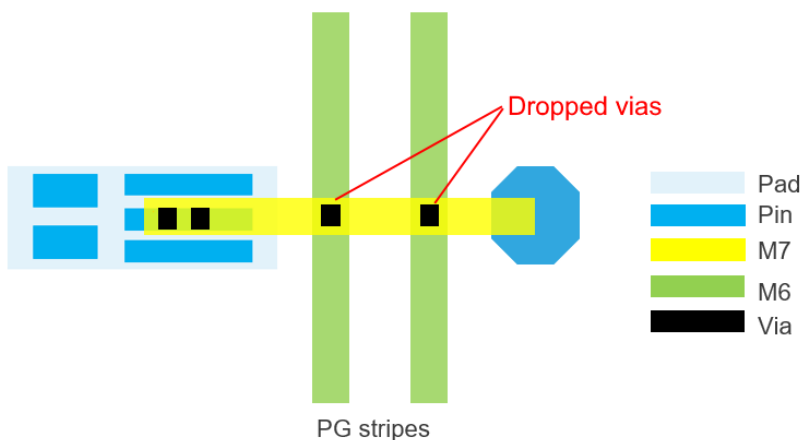
```
flip_chip_drop_via_on_power_mesh layer_name
```

Default: ""

Data_type: string, read/write

Specifies the number or name of the metal layer that can be used by the tool for dropping a via on a PG mesh. `route_flip_chip` automatically drops a via on the PG mesh on the specified metal layer if the PG RDL route crosses the PG mesh. For example:

```
set_db flip_chip_drop_via_on_power_mesh M6
set_db flip_chip_connect_power_cell_to_bump true
route_flip_chip -target connect_bump_to_pad -route_engine global_detail
```



The via is dropped only if there is no DRC violation due to the via drop.

The `flip_chip_drop_via_on_power_mesh` attribute does not affect the routing results from PG bumps to pad.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:


```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `place_peripheral_ios`
- `route_flip_chip`

flip_chip_extra_config

```
flip_chip_extra_config file_name
```

Default: ""

Data_type: string, read/write

Specifies the name of an extra configuration file.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `place_peripheral_ios`
- `route_flip_chip`

flip_chip_finger_direction

```
flip_chip_finger_direction {n|e|s|w}
```

Default: n

Data_type: enum {n e s w}, read/write

Specifies the main expansion direction for PG finger routing. The default direction is North.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `place_peripheral_ios`
- `route_flip_chip`

flip_chip_finger_max_width

```
flip_chip_finger_max_width value
```

Default: 0.0

Data_type: double, read/write

Specifies the maximum routing width for one wire segment of the PG finger. By default, there is no constraint on the max width; `route_flip_chip` uses `MAXWIDTH` in LEF. If this constraint is not specified in LEF, `route_flip_chip` uses the maximum width possible.

If there is a conflict between the user-specified constraint and the LEF rule, the LEF rule is used as the maximum width.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `place_peripheral_ios`
- `route_flip_chip`

flip_chip_finger_min_width

`flip_chip_finger_min_width value`

Default: 0.0

Data_type: double, read/write

Specifies the minimum routing width for one wire segment of the PG finger. The default is the RDL minimum width defined in LEF. If there is a conflict between the user-specified constraint and the LEF rule, the LEF rule is used as the minimum width.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `place_peripheral_ios`
- `route_flip_chip`

flip_chip_finger_target_mesh_layer_range

`flip_chip_finger_target_mesh_layer_range layer_range`

Default: ""

Data_type: string, read/write

Specifies the target mesh layer range of the power mesh that will be connected for finger routing. If not specified, no layer of the power mesh will be connected.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `place_peripheral_ios`
- `route_flip_chip`

flip_chip_honor_bump_connect_target_constraint

`flip_chip_honor_bump_connect_target_constraint {true | false}`

Default: `false`

Data_type: `bool`, read/write

Enables port numbering routing, when set to `true`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `place_peripheral_ios`
- `route_flip_chip`

flip_chip_ignore_pad_type_check

`flip_chip_ignore_pad_type_check {true | false}`

Default: `true`

Data_type: `bool`, read/write

Specifies whether or not CLASS PAD cells are supported in the flip chip flow.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `place_peripheral_ios`
- `route_flip_chip`

flip_chip_lower_layer_route_width

```
flip_chip_lower_layer_route_width value
```

Default = {}

Data_type: string, read/write

Specifies the value for the lower routing layers. You can specify either:

- Layer-width pairs as shown below:

```
set_db flip_chip_lower_layer_route_width {layer1 width1 layer2 width2 ...}
```

Use this method if you want to specify different routing widths for different layers. For example, you can specify the main routing width in the M5 and M3 layers with the

`flip_chip_route_width` attribute and different routing widths for the lower M1, M2, M4 layers:

```
set_db flip_chip_route_width 15
```

```
set_db flip_chip_lower_layer_route_width {M1 5 M2 8 M4 10}
```

- A single value in microns as shown below:

```
set_db flip_chip_lower_layer_route_width value
```

Use this method if you want to specify a single value for all lower layers. The minimum possible value is 0.000000 while the maximum is 10000000000.000000. If the routing layer is not specified with `flip_chip_lower_layer_route_width`, the tool will follow the main routing width specified with `flip_chip_route_width` or the Tech LEF default width. For example, you can specify the main routing width in the M5 layer with the `flip_chip_route_width` attribute and a single different routing width for the lower M1, M2, M3, and M4 layers:

```
set_db flip_chip_route_width 15
```

```
set_db flip_chip_lower_layer_route_width 10
```

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `place_peripheral_ios`
- `route_flip_chip`

flip_chip_lower_layer_prevent_diagonal_routing

```
flip_chip_lower_layer_prevent_diagonal_routing {true | false}
```

Default: `false`

Data_type: `bool`, read/write

Prevents diagonal routing for lower layer, when set to `true`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
- To get the current value of any root attribute, use the `get_db` command:

```
set_db attribute_name value
```

```
get_db attribute_name
```

Related Commands

- `place_peripheral_ios`
- `route_flip_chip`

flip_chip_merge_nearby_pin

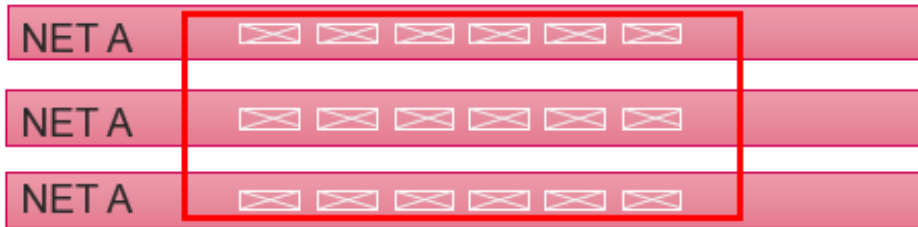
```
flip_chip_merge_nearby_pin {true | false}
```

Default: `false`

Data_type: `bool`, read/write

Merges pins that are close but not touching. If this attribute is set to `true` and there are multiple non-

overlapping pin shapes sharing the same net, the tool will merge these pins and drop a via automatically on these pin shapes at the same time while connecting the bumps to pins. A single shape in the specified layer that contains an array of vias is transferred to flip chip route by VIAGEN. The array of vias is the largest via that fits on each of the multiple physical pin shapes.



- The merged pins must have the same net on the same layer.
- The distance between these merged pins should be small and there should be no other objects between the pins.
- The pin shapes should be aligned with each other.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_flip_chip`

flip_chip_multi_pad_routing_style

```
flip_chip_multi_pad_routing_style {default serial star}
```

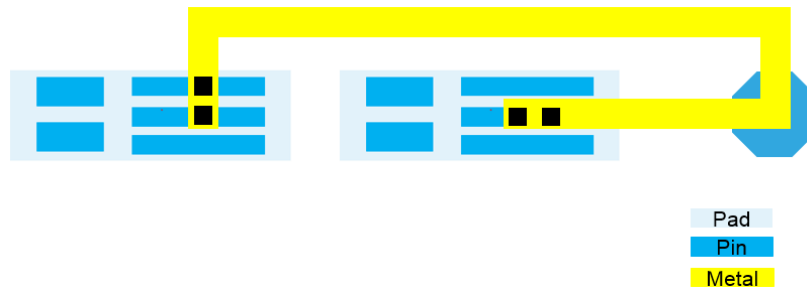
Default: default

Data_type: enum {default serial star}, read/write

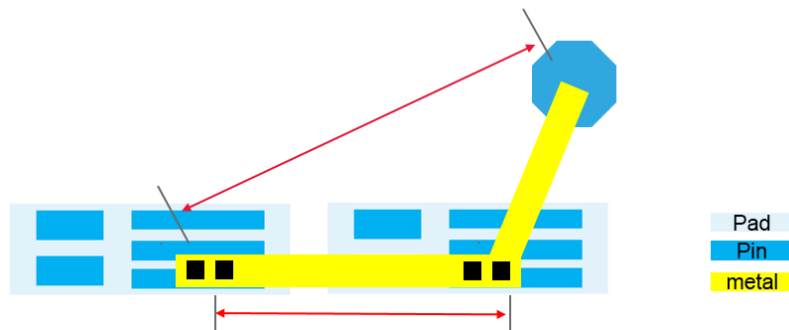
Specifies the routing style for multi-pad routing:

- **default:** The flip chip router (`route_flip_chip`) connects a bump to multiple pads with

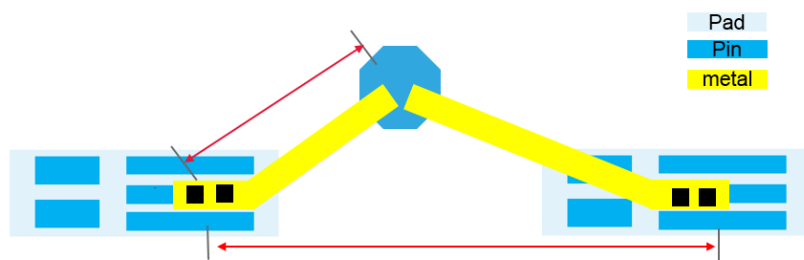
multiple different wires as shown below:



- `serial:route_flip_chip` connects a bump first to one pad and then to the other pad, as shown below, provided that the distance from the bump to either pad is more than the distance between the two pads:

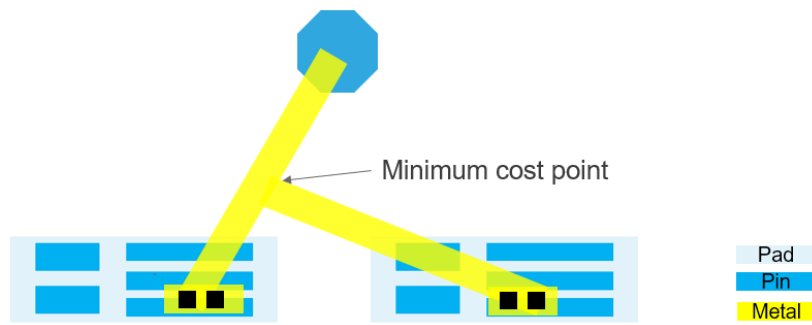


If the distance from the bump to any pad is shorter than the distance between the two pads, `route_flip_chip` reverts to the default behavior of multiple wire connection even if `flip_chip_multi_pad_routing_style` is set to `serial`.



This mode supports only two-pad routing.

- `star:route_flip_chip` connects a bump first to the closest pad. It then finds a minimum cost point on its routing path and connects this point to another pad as shown below:



The minimum cost point could be anywhere on the first routing path, including its beginning and end.

Both the `serial` and `star` modes are useful for saving routing resources in complex designs as they share the routing between pads. However, the `serial` mode supports only two-pad routing. Use the `star` mode if you want to route more than two pads.

Note: The `flip_chip_multi_pad_routing_style` works only if the `flip_chip_multiple_connection_multiple_pads_to_bump` is specified. For example, to use the `star` mode, you should specify the following commands before running `route_flip_chip`:

```
set_db flip_chip_multiple_connection multiple_pads_to_bump
set_db multi_pad_routing_style star
```

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `place_peripheral_ios`
- `route_flip_chip`

flip_chip_multiple_connection

```
flip_chip_multiple_connection {multiple_pads_to_bump | multiple_bumps_to_pad |
multiple_bumps_to_multiple_pads | default}
```

Default: default

Data_type: enum {multiple_pads_to_bump multiple_bumps_to_pad
multiple_bumps_to_multiple_pads default}, read/write

Specifies routing connections between multiple pads and bumps. You can specify one of the following options:

- `multiple_pads_to_bump`
Enables routing from multiple pads to one bump in parallel. This option is supported by both AIO and PIO routing styles.
- `multiple_bumps_to_pad`
Enables routing from multiple bumps to one pad in parallel. This option is supported by both AIO and PIO routing styles.
- `multiple_bumps_to_multiple_pads`
Enables routing from multiple bumps to multiple pads. When this option is set, `route_flip_chip` routes all bumps and pads with the same net in the `serial` routing style. If you want to route the same-net bumps and pads routing in different groups, use the `SHARE_PAIR` constraint to configure the pairing between the bumps and pads.

Multiple bumps to multiple pads connections are only supported when `flip_chip_multi_pad_routing_style` is set to `serial`.

- `default`
Enables routing from one bump to one pad in parallel. This means there is no multiple connection.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `place_peripheral_ios`
- `route_flip_chip`

flip_chip_pg_mesh_direction

`flip_chip_pg_mesh_direction {default | horizontal | vertical}`

Default: default

Data_type: enum {default horizontal vertical}, read/write

Specifies the routing direction of the PG mesh. You can specify one of the following values for this option:

- `default` - Uses the preferred routing direction defined in the Tech LEF.
- `horizontal` - Uses the horizontal routing direction.
- `vertical` - Uses the vertical routing direction.

The `flip_chip_pg_mesh_direction` attribute can be used only if `flip_chip_route_pg_style` is set to `mesh`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `route_flip_chip`

flip_chip_pg_mesh_main_width

`flip_chip_pg_mesh_main_width value`

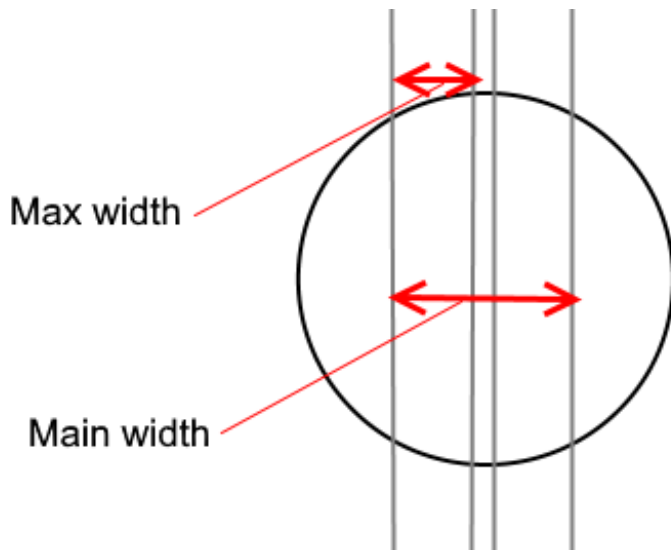
Default: 0.0

Min: 0

Max: 100000000000

Data_type: double, read/write

Specifies the main width of the PG mesh. This attribute can be used only if `flip_chip_route_pg_style` is set to `mesh`.



This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `route_flip_chip`

flip_chip_pg_mesh_max_width

`flip_chip_pg_mesh_max_width value`

Default: 0.0

Min: 0

Max: 10000000000

Data_type: double, read/write

Specifies the maximum width of the routing wire in a PG mesh. This attribute can be used only if `flip_chip_route_pg_style` is set to `mesh`.

By default there is no constraint on the maximum width. `fcroute` uses `MAXWIDTH` in the Tech LEF. If there is a conflict between the value you specify and the LEF rule, the tool uses the LEF rule as the

maximum width.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_flip_chip`

flip_chip_pg_mesh_width

```
flip_chip_pg_mesh_width value
```

Default: ""

Data_type: string, read/write

Specifies the exact routing width of the PG mesh. If only one value is specified with `pg_mesh_width`, the value is applicable for the width of both horizontal and vertical wires. If two values are specified, the first value is applicable for the width of horizontal wires and the second value is applicable for the width of vertical wires.

This attribute can be used only if `flip_chip_route_pg_style` is set to `mesh`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_flip_chip`

flip_chip_prevent_diagonal_wire_access_bumps

`flip_chip_prevent_diagonal_wire_access_bumps {true | false}`

Default: false

Data_type: bool, read/write

Specifies whether or not diagonal wires can be used to access bumps.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_flip_chip`

flip_chip_prevent_via_under_bump

`flip_chip_prevent_via_under_bump {true | false}`

Default: false

Data_type: bool, read/write

Specifies whether or not vias are generated directly under the bump. If you set this parameter to `true`, the vias generated are prevented from connecting directly to the bump.

This option is available for both signal and power bumps.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `place_peripheral_ios`
- `route_flip_chip`

flip_chip_prevent_via_under_bump_extension

`flip_chip_prevent_via_under_bump_extension value`

Default: default

Data_type: string, read/write

Specifies a spacing value to define an extension area around the bump in which vias should not be dropped. By default (or when `value` is set to `default`), the tool automatically chooses the appropriate spacing around bumps to avoid dropping vias.

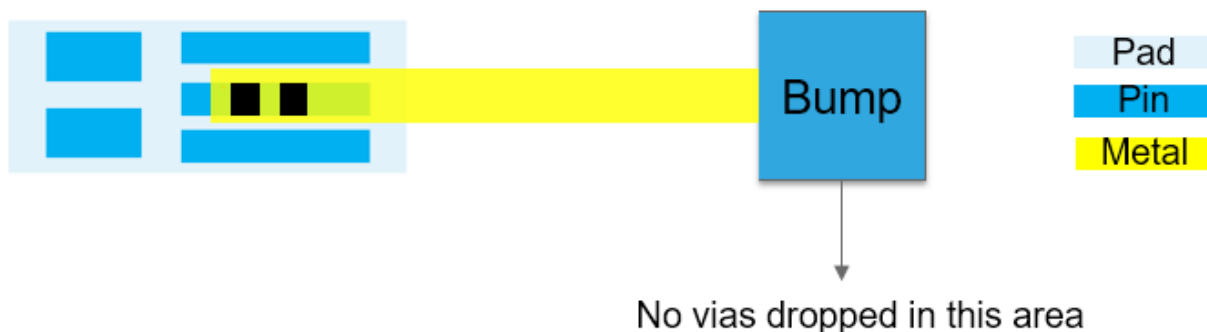
The following examples illustrate how the

`flip_chip_prevent_via_under_bump` and `flip_chip_prevent_via_under_bump_extension` attributes can be used to control the spacing around bumps when vias are dropped:

- In the example below, no vias are dropped directly under the bump
as `flip_chip_prevent_via_under_bump` is set to `true`:

```
> set_db flip_chip_prevent_via_under_bump true
```

```
> route_flip_chip -target connect_bump_to_pad -route_engine global_detail
```

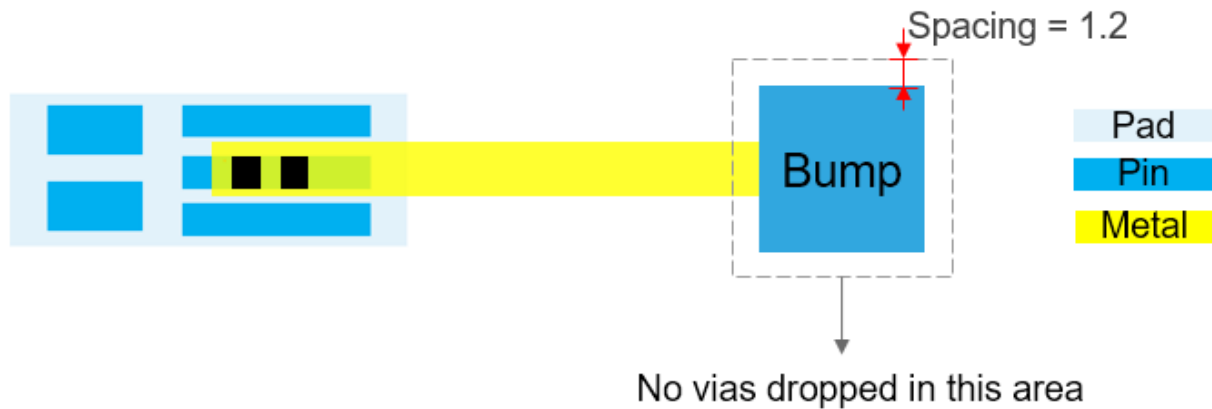


If, additionally, `flip_chip_prevent_via_under_bump_extension` is set to 0, vias can be dropped in the space immediately around the bump.

By default, however, the tool will automatically choose the appropriate spacing around bumps in which to avoid dropping vias.

- In the example below, no vias are dropped in the extension area defined around the bump:

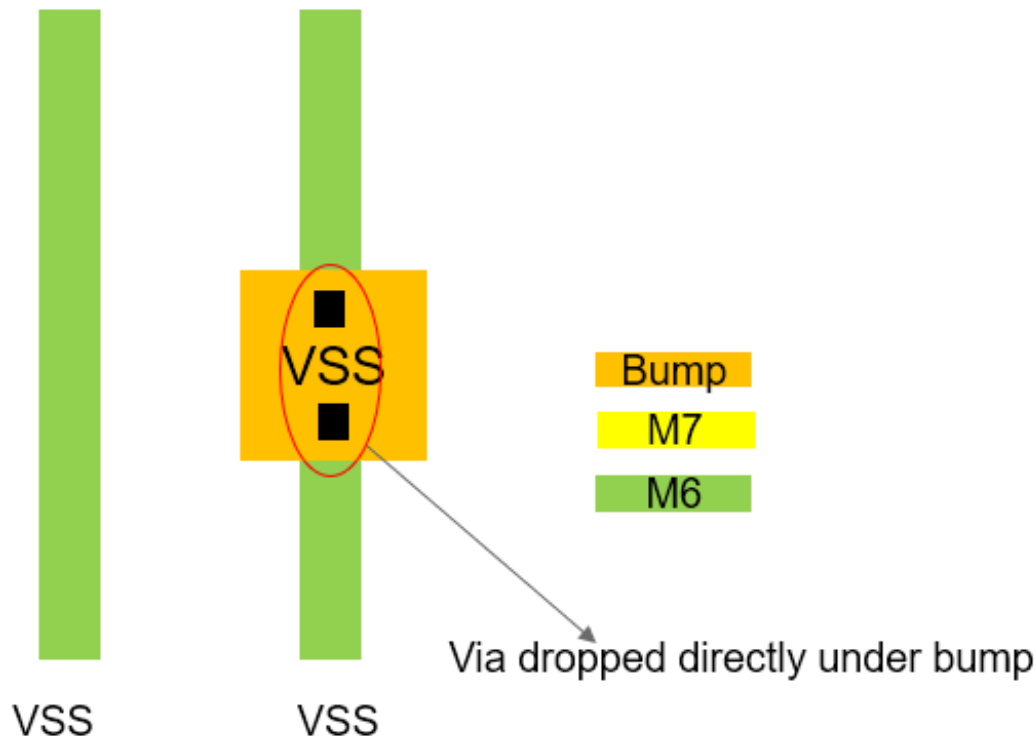
```
> set_db flip_chip_prevent_via_under_bump_extension 1.2  
> route_flip_chip -target connect_bump_to_pad -route_engine global_detail
```



- In the example below, vias are dropped directly under the power bumps

as `flip_chip_prevent_via_under_bump` is set to `false`:

```
> set_db flip_chip_prevent_via_under_bump false  
> route_flip_chip -target connect_bump_to_ring_stripe
```

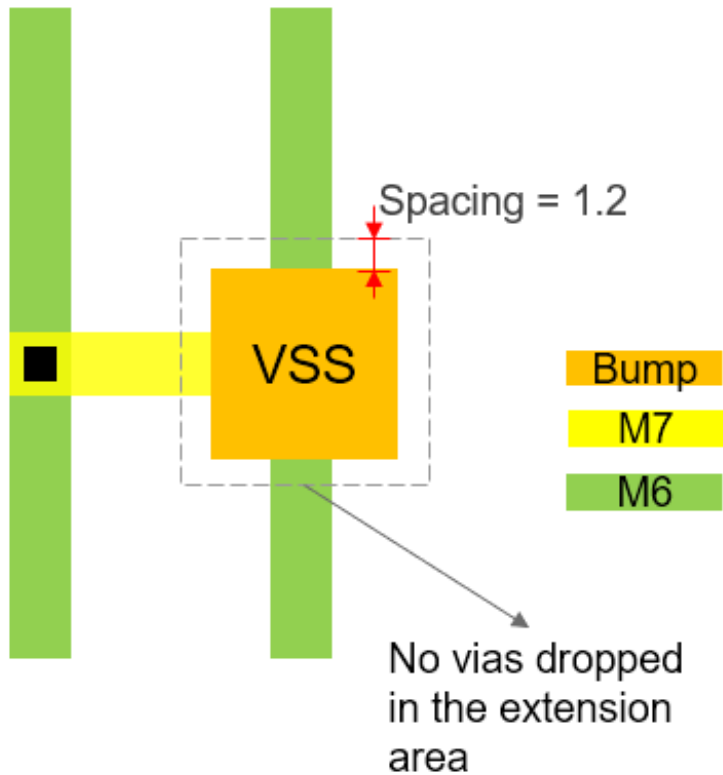


- In the example below, no vias are dropped in the extension area defined around the power bumps:

```
> set_db flip_chip_prevent_via_under_bump true
```



```
> set_db flip_chip_prevent_via_under_bump_extension 1.2  
> route_flip_chip -target connect_bump_to_ring_stripe
```



This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `route_flip_chip`

flip_chip_route_bump_cluster

```
flip_chip_route_bump_cluster {true | false}
```

Default: false

Data_type: bool, read/write

Routes bump clusters to IO and probing pads.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `create_bump_cluster`
- `route_flip_chip`

flip_chip_route_pg_style

`flip_chip_route_pg_style {none finger mesh}`

Default: none

Data_type: enum {none finger mesh}, read/write

Specifies the routing pattern for flip chip route power routing:

- `none`: For normal PG routing
- `finger`: For PG finger routing style, in which you need to specify the routing direction. This option must be used with `route_flip_chip -region` option.
- `mesh`: For mesh-style PG routing on the redistribution layer (RDL). This option is used to increase PG density. If you specify PG mesh routing, you must specify the main width of the PG mesh by using the `flip_chip_pg_mesh_main_width` parameter.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `place_peripheral_ios`
- `route_flip_chip`

flip_chip_route_style

```
flip_chip_route_style {manhattan | diagonal}
```

Default: diagonal

Data_type: enum {manhattan diagonal}, read/write

Determines the type of routing, either `manhattan` or `diagonal`.

The `diagonal` routing style does not work in conjunction with the following `route_flip_chip` parameter:

- `-double_bend_route`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```
- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `place_peripheral_ios`
- `route_flip_chip`

flip_chip_route_width

```
flip_chip_route_width value
```

Default: 0.0

Data_type: double, read/write

Connects wires to bumps using the specified route width not wider than the I/O pad pin.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `place_peripheral_ios`
- `route_flip_chip`

flip_chip_top_layer

```
flip_chip_top_layer layer_name
```

Default: ""

Data_type: string, read/write

Specifies the top-most metal layer by layer number or layer name that the software can use when routing bumps.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `place_peripheral_ios`
- `route_flip_chip`

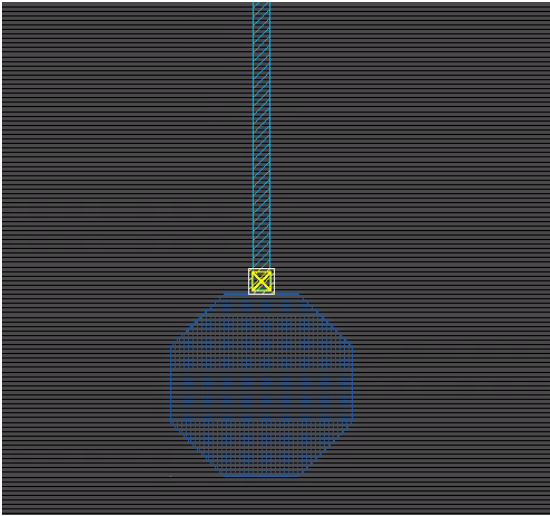
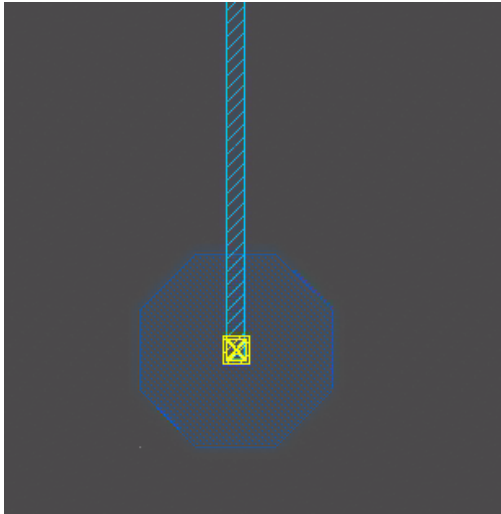
flip_chip_via_abut_bump

`flip_chip_via_abut_bump {true | false}`

Default: false

Data_type: bool, read/write

Specifies that via should abut the bump only if routing can be done successfully.

With <code>flip_chip_via_abut_bump</code> set to true	Without <code>flip_chip_via_abut_bump</code> set to true
	 In this case, via is dropped inside bump as shown above. This is the default setting.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_flip_chip`

flip_chip_wire_to_pad_honor_min_spacing

`flip_chip_wire_to_pad_honor_min_spacing value`

Default: 0.0

Data_type: double, read/write

Specifies the minimum spacing between the bump/pad and a wire.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

`set_db attribute_name value`

- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

Related Commands

- `route_flip_chip`

flow

The root attributes in the `flow` category are:

- `flow_branch`
- `flow_caller_data`
- `flow_db_directory`
- `flow_error_errorinfo`
- `flow_error_message`
- `flow_error_write_db`
- `flow_exclude_time_for_init_flow`
- `flow_exit_when_done`
- `flow_features`
- `flow_feature_values`
- `flow_footer_tcl`
- `flow_header_tcl`
- `flow_hier_path`
- `flow_history`
- `flow_log_directory`
- `flow_log_prefix_generator`
- `flow_mail_on_error`
- `flow_mail_to`
- `flow_metrics_file`
- `flow_metrics_snapshot_parent_uuid`
- `flow_metrics_snapshot_uuid`
- `flow_overwrite_db`
- `flow_plugin_canonical_current`
- `flow_plugin_names`

- [flow_plugin_steps](#)
- [flow_post_db_overwrite](#)
- [flow_remark](#)
- [flow_report_directory](#)
- [flow_run_tag](#)
- [flow_schedule](#)
- [flow_starting_db](#)
- [flow_startup_directory](#)
- [flow_status_file](#)
- [flow_step_canonical_current](#)
- [flow_step_check_tcl](#)
- [flow_step_current](#)
- [flow_step_last](#)
- [flow_step_last_msg](#)
- [flow_step_last_status](#)
- [flow_step_next](#)
- [flow_summary_tcl](#)
- [flow_template_feature_definition](#)
- [flow_template_tools](#)
- [flow_template_type](#)
- [flow_template_version](#)
- [flow_user_templates](#)
- [flow_top](#)
- [flow_verbose](#)
- [flow_working_directory](#)
- [flow_write_db_snapshot](#)
- [flow_write_db_snapshot_exclude_time](#)

- [flow_yamllint_exec](#)

To set the value of any root attribute, use the [set_db](#) command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the [get_db](#) command:

```
get_db attribute_name
```

To get the current value of all attributes in the `flow` category, use the following command:

```
get_db -category flow
```

flow_branch

```
flow_branch string
```

Default: ""

Specifies the branch being run for a hierarchical flow.

flow_caller_data

```
flow_caller_data string
```

Default: ""

Specifies the data used by the caller of the tool for identifying the flow.

flow_db_directory

```
flow_db_directory string
```

Default: dbs

Specifies the flow directory where results databases are stored.

flow_error_errorinfo

```
flow_error_errorinfo string
```

Default: ""

Specifies the TCL error stack in the error database.

flow_error_message

`flow_error_message` *string*

Default: ""

Specifies the TCL error message in the error database.

flow_error_write_db

`flow_error_write_db` {true | false}

Default: true

Runs the database when an error occurs in a flow step.

flow_exclude_time_for_init_flow

`flow_exclude_time_for_init_flow` {true | false}

Default: false

Marks all those snapshots as `exclude_time_metric`, which were created after running the `init_flow` command. This marking adds user steps to the `init_flow` internal plugin locations.

flow_exit_when_done

`flow_exit_when_done` {true | false}

Default: false

Exits the flow after running the final step.

flow_features

`flow_features` *string*

Default: ""

Specifies the global feature definitions.

flow_feature_values

`flow_feature_values` *string*

Default: ""

Specifies the global feature settings.

flow_footer_tcl

`flow_footer_tcl` *string*

Default: ""

Specifies the TCL script to run when a flow ends.

flow_header_tcl

`flow_header_tcl` *string*

Default: ""

Specifies the TCL script to run when a flow starts.

flow_hier_path

`flow_hier_path` *string*

Default: ""

Displays the path of the current flow within the flow hierarchy.

flow_history

`flow_history` *string*

Default: ""

Displays the complete flow run history.

flow_log_directory

`flow_log_directory` *string*

Default: logs

Specifies the flow directory where log files are stored.

flow_log_prefix_generator

`flow_log_prefix_generator string`

Default: ""

Specifies the TCL script to create log filenames in flowtool.

flow_mail_on_error

`flow_mail_on_error {true | false}`

Default: false

Sends an email to 'flow_mail_to', if an error is detected.

flow_mail_to

`flow_mail_to string`

Default: ""

Specifies the email address where results are sent.

flow_metrics_file

`flow_metrics_file string`

Default: ""

Specifies the flow metrics file for reporting results.

flow_metrics_snapshot_parent_uuid

`flow_metrics_snapshot_parent_uuid string`

Default: ""

Specifies the snapshot uuid to which the results from this flow will be appended.

flow_metrics_snapshot_uuid

flow_metrics_snapshot_uuid *string*

Default: ""

Specifies the snapshot uuid of the most recent flow step executed.

flow_overwrite_db

flow_overwrite_db {true | false}

Default: false

Enables overwriting databases while saving.

flow_plugin_canonical_current

flow_plugin_canonical_current *string*

Default: ""

Identifies the canonical plugin step currently being run, enabling accurate detection of plugin within the Flowtool-based flows. When a plugin is not run, this attribute returns an empty value. Following is an example value of this attribute:

```
tool flowtool flow Cadence.plugin.flowkit.read_db.pre canonical_path {.end_steps  
flow_step:test} step flow_step:test features {} str test
```

flow_plugin_names

flow_plugin_names *string*

Default: ""

Displays the names of all the internal plugin points.

flow_plugin_steps

flow_plugin_steps *string*

Default: ""

Displays the list of plugin steps that have been added through `edit_flow`.

flow_post_db_overwrite

`flow_post_db_overwrite string`

Default: ""

When this attribute is set within `skip_db flow_step`, you can identify the name and type of database from the current to future flows as `flow_starting_db`.

flow_remark

`flow_remark string`

Default: ""

Displays the remarks from the last loaded `flow.yaml` file.

flow_report_directory

`flow_report_directory string`

Default: `reports`

Specifies the flow directory where reports are written.

flow_run_tag

`flow_run_tag string`

Default: ""

Specifies the tags for the particular flow run.

flow_schedule

`flow_schedule string`

Default: ""

Specifies the set of flows to execute after the current flow completes.

flow_starting_db

`flow_starting_db string`

Default: ""

Specifies the starting database for the flow run.

flow_startup_directory

`flow_startup_directory string`

Default: /home/user_id

Specifies the directory from where the software was run.

flow_status_file

`flow_status_file string`

Default: ""

Specifies the file for flow status.

flow_step_canonical_current

`flow_step_canonical_current string`

Default: ""

Specifies the currently running canonical flow step.

flow_step_check_tcl

`flow_step_check_tcl string`

Default: ""

Specifies the procedure to check the steps.

flow_step_current

`flow_step_current string`

Default: ""

Specifies the currently running flow step.

flow_step_last

`flow_step_last` *string*

Default: ""

Specifies the last flow step to be completed.

flow_step_last_msg

`flow_step_last_msg` *string*

Default: ""

Specifies the message provided for the last step that was run.

flow_step_last_status

`flow_step_last_status` *string*

Default: `not_run`

Specifies the status for the last flow step run.

flow_step_next

`flow_step_next` *string*

Default: ""

Specifies the next step to run in the flow.

flow_summary_tcl

`flow_summary_tcl` *string*

Default: ""

Specifies the TCL script to run during `run_flow` summary.

flow_template_feature_definition

`flow_template_feature_definition` *string*

Default: ""

Features list and status for the current template.

flow_template_tools

`flow_template_tools` *string*

Default: ""

Specifies the list of tools included in the flow templates being run.

flow_template_type

`flow_template_type` *string*

Default: ""

Specifies the type of template being run.

flow_template_version

`flow_template_version` *string*

Default: ""

Specifies the version of the template being run.

flow_user_templates

`flow_user_templates` *string*

Default: ""

Specifies the flow user template definitions.

flow_top

`flow_top` *string*

Default: ""

Specifies the flow to be executed by default.

flow_verbose

`flow_verbose {true | false}`

Default: true

Enables printing the run information in the log file.

flow_working_directory

`flow_working_directory string`

Default: "."

Specifies the flow directory where the flow is being run.

flow_write_db_snapshot

`flow_write_db_snapshot {true | false}`

Default: true

Data_type: bool, read/write

Creates a snapshot for `write_db` on flow steps, which saves a database.

flow_write_db_snapshot_exclude_time

`flow_write_db_snapshot_exclude_time {true | false}`

Default: true

Data_type: bool, read/write

Marks all snapshots created for the `write_db` command on the flow steps, which save a database as `exclude_time_metric`.

flow_yamllint_exec

`flow_yamllint_exec` *string*

Default: `yamllint`

Specifies where `yamllint` can be found to check the `yaml` syntax.

flowtool

The root attributes in the `flowtool` category are:

- `flowtool_exit_timeout`
- `flowtool_extra_arguments`
- `flowtool_metrics_qor_html`
- `flowtool_metrics_qor_excel`
- `flowtool_metrics_qor_text`
- `flowtool_metrics_qor_vivid`
- `flowtool_predict_full_names`
- `flowtool_schedule_flow_immediate`
- `flowtool_summary_tcl`

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `flowtool` category, use the following command:

```
get_db -category flowtool
```

flowtool_exit_timeout

`flowtool_exit_timeout` *int*

Default: `30`

Data_type: Integer

Specifies the maximum time (in seconds) for which flowtool will wait to complete post exiting from the tool.

flowtool_extra_arguments

`flowtool_extra_arguments` *string*

Default: ""

Data_type: String

Specifies the extra arguments passed to flowtool when running in the tool mode..

flowtool_metrics_qor_html

`flowtool_metrics_qor_html` *string*

Default: ""

Data_type: String

Specifies the metrics file in the HTML format to write after tool's exit.

flowtool_metrics_qor_excel

`flowtool_metrics_qor_excel` *string*

Default: ""

Data_type: String

Specifies the metrics file in the Excel format to write after tool's exit.

flowtool_metrics_qor_text

`flowtool_metrics_qor_text` *string*

Default: ""

Data_type: String

Specifies the metrics file in the txt format to write after tool's exit.

flowtool_metrics_qor_vivid

`flowtool_metrics_qor_vivid` *string*

Default: ""

Data_type: String

Specifies the metrics file to write after the tool's exit.

flowtool_predict_full_names

`flowtool_predict_full_names` {true | false}

Default: false

Data_type: Boolean

Shows the `skip_metric` flow name in prediction.

flowtool_schedule_flow_immediate

`flowtool_schedule_flow_immediate` {true | false}

Default: false

Data_type: Boolean

When set to `true`, flowtool will start the scheduled flows immediately. When set to `false`, it waits for the software to exit before starting the scheduled flows.

flowtool_summary_tcl

`flowtool_summary_tcl` *string*

Default: `puts [report_metric -format text]`

Data_type: String

Specifies the TCL script to run during `run_flow` summary.

fv

The root attributes in the `fv` category are:

- [design_verification_effort](#)
- [verification_directory_naming_style](#)
- [write_verification_files](#)

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `fv` category, use the following command:

```
get_db -category fv
```

design_verification_effort

`design_verification_effort` *string*

Default: `high`

Data_type: `string`, `read/write`

Specifies the default design verification effort level for Logical Equivalence Checking (LEC). This attribute also disables problematic optimizations in bulk. You can use it for any module where the module-level attribute value is set to `inherited`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

verification_directory_naming_style

`verification_directory_naming_style` *string*

Default: `fv/%s`

Data_type: `string`, `read/write`

Specifies the format to be used to name the directory to which the verification files are written when

the `write_verification_files` attribute is set to `true`.

The attribute value can contain an arbitrary string, zero or one `%s` representing the design name, and zero or one `%d` representing the number of the session.

- If the value contains neither `%s` nor `%d`, each elaboration run writes the files in the same directory, overwriting the existing ones.
- If the value contains `%d`, each elaboration run creates a new directory.
- If the value contains `%s` but no `%d`, a new directory is created only if elaboration is given a different top design.

By default, the verification directory will be overwritten each time you run a new session of the tool.

The following files will be written to this directory:

- `fv_map.v.gz`: Intermediate netlist generated during `syn_map`
- `rtl_to_fv_map.do`: A do file for comparison between RTL and the intermediate netlist generated during `syn_map`
- `fv_map.fv.json`: Auxiliary information

Note that the `syn_map` command specified above is only valid within the script being passed to `synthesize_design`. You cannot use it directly at the Innovus prompt.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- [elaborate_design](#)
- [synthesize_design](#)
- [write_verification_files](#)

write_verification_files

`write_verification_files {true | false}`

Default: `true`

Data_type: `bool`, `read/write`

Controls whether or not to write intermediate files to the verification directory, specified by the `verification_directory_naming_style` attribute.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- `elaborate_design`
- `synthesize_design`
- `verification_directory_naming_style`

generate_special_via

The root attributes in the generate_special_via category are:

- [generate_special_via_accuracy_effort](#)
- [generate_special_via_add_pin_to_pin_vias](#)
- [generate_special_via_align_merged_stack_via_metal](#)
- [generate_special_via_allow_via_expand](#)
- [generate_special_via_allow_wire_shape_change](#)
- [generate_special_via_area_only](#)
- [generate_special_via_bot_enclosure](#)
- [generate_special_via_check_signal_routes](#)
- [generate_special_via_create_double_row_cut_via](#)
- [generate_special_via_create_max_row_cut_via](#)
- [generate_special_via_cut_class_preference](#)
- [generate_special_via_cut_spacing](#)
- [generate_special_via_disable_via_merge](#)
- [generate_special_via_enable_check_drc](#)
- [generate_special_via_extend_out_wire_end](#)
- [generate_special_via_full_cut_via_only](#)
- [generate_special_via_hookup_auto_set_viacell](#)
- [generate_special_via_hookup_contact_max](#)
- [generate_special_via_hookup_contact_pg_track](#)
- [generate_special_via_hookup_fixed_grid](#)
- [generate_special_via_hookup_min_distance](#)
- [generate_special_via_hookup_preplace_fix](#)
- [generate_special_via_hookup_rail_pair](#)
- [generate_special_via_hookup_via_distance](#)

- generate_special_via_hookup_via_style
- generate_special_via_hookup_via_rule
- generate_special_via_hookup_virtual_trim_grid
- generate_special_via_ignore_design_boundary
- generate_special_via_ignore_drc
- generate_special_via_ignore_rule_enclosure
- generate_special_via_inherit_wire_status
- generate_special_via_keep_existing_via
- generate_special_via_keep_fixed_via
- generate_special_via_mincut_preference
- generate_special_via_opt_cross_via
- generate_special_via_opt_via_on_routing_track
- generate_special_via_parameterized_via_only
- generate_special_via_partial_overlap_threshold
- generate_special_via_preferred_vias_only
- generate_special_via_prefix
- generate_special_via_reference_boundary
- generate_special_via_respect_stdcell_geometry
- generate_special_via_respect_wire_ndr
- generate_special_via_rule_preference
- generate_special_via_set_via_expand_dir
- generate_special_via_snap_via_center_to_grid
- generate_special_via_split_long_via_global_grid
- generate_special_via_symmetrical_via_only
- generate_special_via_top_enclosure
- generate_special_via_use_track_offset
- generate_special_via_use_trim_metal_enclosure

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `generate_special_via` category, use the following command:

```
get_db -category generate_special_via
```

The following commands use the `generate_special_via` category attributes for various functionalities:

- `update_power_vias`
- `add_stripes`
- `add_rings`
- `add_target_pg`
- `add_reinforce_pg`
- `route_special`

generate_special_via_accuracy_effort

```
generate_special_via_accuracy_effort {low | medium | high}
```

Default: `high`

Data_type: `enum {low medium high}`, read/write

Controls the balance between accuracy and runtime during via generation.

- When set to `high`, viagen checks full DRC rules and creates DRC clean vias.
- When set to `medium`, viagen runs DRC check in the moderate mode.
- When set to `low`, viagen runs in the quick DRC check mode. With this value specified, some available vias may be missed, since the conservative rules are checked.

The vias generated through this parameter are DRC clean. This parameter generates vias with reduced runtime in the special scenarios where complex DRC rules are triggered. These complex rules include (but are not limited to) `same metal aligned cuts spacing`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

generate_special_via_add_pin_to_pin_vias

```
generate_special_via_add_pin_to_pin_vias {true | false}
```

Default: false

Data_type: bool, read/write

Inserts vias to connect pins between different cover macros.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

generate_special_via_align_merged_stack_via_metal

```
generate_special_via_align_merged_stack_via_metal {true | false}
```

Default: false

Data_type: bool, read/write

Aligns merged stack vias.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

generate_special_via_allow_via_expand

```
generate_special_via_allow_via_expand {true | false}
```

Default: true

Data_type: bool, read/write

Allows via to expand out of intersection area to meet LEF rules.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

generate_special_via_allow_wire_shape_change

```
generate_special_via_allow_wire_shape_change {true | false}
```

Default: true

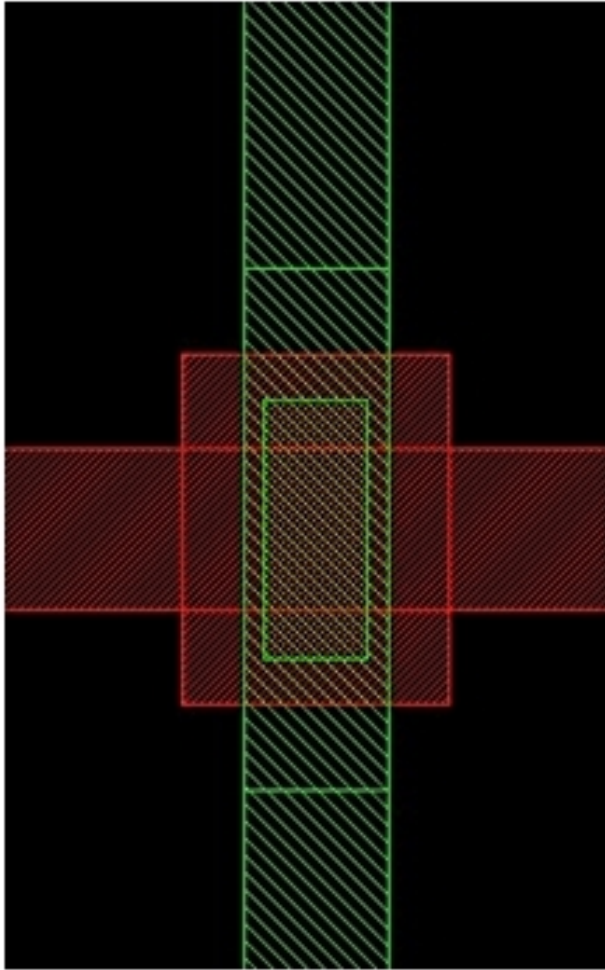
Data_type: bool, read/write

Allows the via to change the metal shape of existing wires.

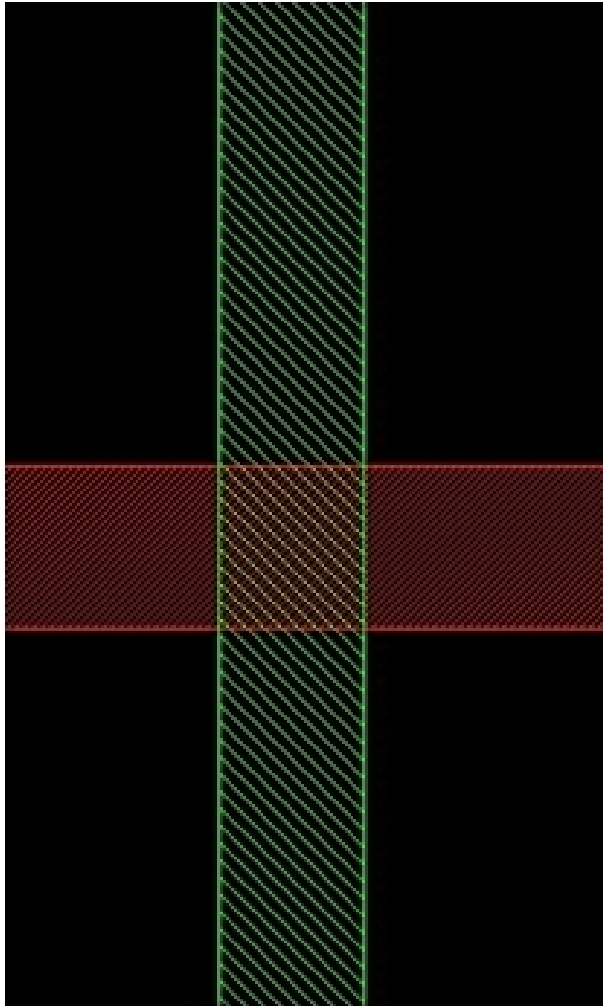
Note: For stack vias, the option is only valid for top and bottom metal layers.

Example:

- `generate_special_via_allow_wire_shape_change true`



- generate_special_via_allow_wire_shape_change false



This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

generate_special_via_area_only

`generate_special_via_area_only {true | false}`

Default: false

Data_type: bool, read/write

Adds vias only within the specified areas.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

generate_special_via_bot_enclosure

```
generate_special_via_bot_enclosure {overhang1 overhang2}
```

Default: ""

Data_type: string, read/write

Generates vias with the specified bottom enclosure values, without modifying the LEF file. Set the `generate_special_via_ignore_rule_enclosure` attribute to `false` while using this attribute, so that the engine can compare the specified `overhang1` and `overhang2` values with the Via Rule Generate values.

The `overhang1` and `overhang2` values must be above the Via Rule Generate values.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

generate_special_via_check_signal_routes

```
generate_special_via_check_signal_routes {0 | 1 | 2}
```

Default: 0

Data_type: enum {0 1 2}, read/write

Controls whether the via generation software honors pre-existing signal routes.

If set to 1, instructs the via generation software to read pre-existing signal routes and avoid creating DRC violations for the routes.

If set to 2, instructs the via generation software to read pre-existing signal routes and avoid creating DRC violations for the routes. And improve the runtime of VIA12 process.

Note: Using this setting can affect software performance significantly.

If set to 0, instructs the via generation software to ignore any pre-existing signal routes.

Note: DRC violations can occur if this setting is used.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

generate_special_via_create_double_row_cut_via

```
generate_special_via_create_double_row_cut_via {0 | 1 | 2}
```

Default: 0

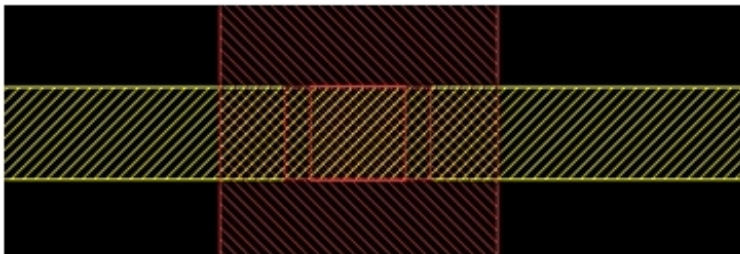
Data_type: enum {no_extra_cut add_extra_cut add_extra_cut_and_metal}, read/write

Try to expand vias to have one additional cut than original when set the option to 1 or 2. That is, expand original vias of $1 \times n$ or $n \times 1$ cuts to vias of $1 \times (n+1)$ or $(n+1) \times 1$ cuts.

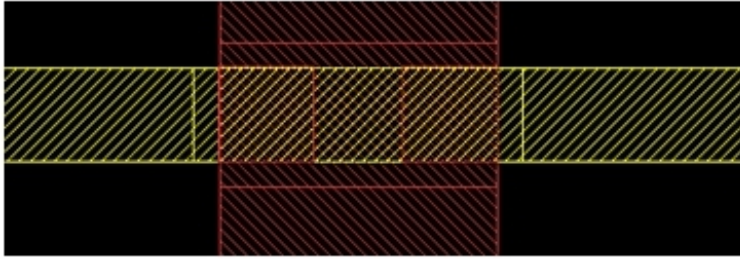
- 1: create vias with one additional cut where wires on both top and down layers are enough and need no shape change.
- 2: create vias with one additional cut where wires on one of the layers are enough.

Example:

- `generate_special_via_create_double_row_cut_via false`



- `generate_special_via_create_double_row_cut_via true`



This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

generate_special_via_create_max_row_cut_via

`generate_special_via_create_max_row_cut_via {true | false}`

Default: false

Data_type: bool, read/write

Try to expand vias to have one additional row or column of cuts than original when the option is set to true. It works similar to `generate_special_via_create_double_row_cut_via` except that there is no restriction on one row or one column via.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

generate_special_via_cut_class_preference

`generate_special_via_cut_class_preference {default | {[square] [bar] [large]}} | cutclass_name_list | file_name}`

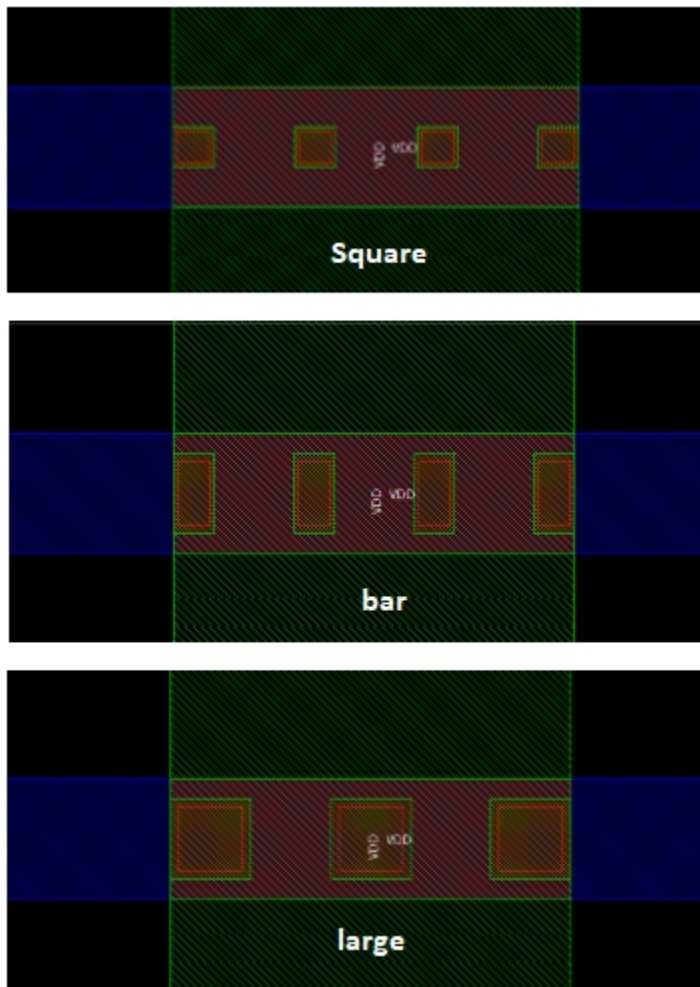
Default: default

Data_type: string, read/write

Specifies the preference for the via cut class while inserting special vias. You can specify one of the following values:

- **default**: Prefers to use the first viacell in tech LEF without drc violation or the first generate viacell to generate maximum cut area. For example, $\text{cutWidth} * \text{cutHeight} * \text{cutNumber}$.
- **square**: Prefers to use the SINGLE cut vias. In the LEF definition, the cut width and length are usually the same and the smallest in that layer. For example, 0.032 by 0.032.
- **bar**: Prefers to use the DOUBLE cut vias. In the LEF definition, the cut width is the usually the smallest. For example, 0.032 by 0.08. It mostly equals to 2 SINGLE cut.
- **large**: Prefers to use the LARGE cut vias. In the LEF definition, the neither the cut width not the length is the smallest. For example, 0.08 by 0.08, equals to 4 SINGLE cut.

The following figure shows the difference between the square, bar, and large values:



Note:

1. The attribute setting does not support incremental specification. The last `cutClass` preference setting is used.
2. When you specify `{[square] [bar] [large]}`, there should be 2 or 3 `cutClass` defined in the LEF file. If Innovus detects that there are more than 3 or less than 2 `cutClasses`, the setting is ignored.
3. In the specified file, different arguments can be separate by space or line break.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```
- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

generate_special_via_cut_spacing

```
generate_special_via_cut_spacing {x_spacing y_spacing}
```

Default: ""

Data_type: string, read/write

Generates vias with the specified cut spacing values, without modifying the LEF file.

The `x_spacing` and `y_spacing` values must be above the Via Rule Generate values.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```
- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

generate_special_via_disable_via_merge

```
generate_special_via_disable_via_merge {true | false}
```

Default:

- `false` in normal scenarios
- `true` for advance nodes (with designs of 12 nm or below)

Data_type: bool, read/write

Disable via merging in the layers next to the followpin layer within the range of two when set the option to true and there is followpin layer in the design. Trimming will be the only way to solve drc issues.

For example, if the followpin layer is in metal1, disable via merging in metal1/metal2/metal3; if the followpin layer is in metal2, disable via merging in metal1/metal2/metal3/metal4.

This function helps if the merging of via consumes additional wire track unnecessarily.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```
- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

generate_special_via_enable_check_drc

```
generate_special_via_enable_check_drc {true | false}
```

Default: false

Data_type: bool, read/write

Enables or disables `check_drc` called by the via generation software.

- When set to `true`, the via generation software calls `check_drc` to detect any `min_step` violations, then calls the `fix_via -min_step` command to fix them. This may impact the run-time performance.
- When set to `false`, the via generation software will not call `check_drc`. This may leave `min_step` violations in the design.

Note: When both the

`generate_special_via_enable_check_drc` and `generate_special_via_opt_cross_via` attributes are set to `true`, and the process node is above 10N, DRC violations can be reported.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

generate_special_via_extend_out_wire_end

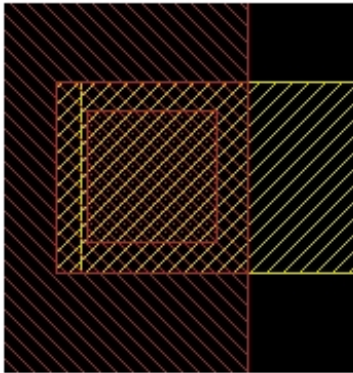
```
generate_special_via_extend_out_wire_end {true | false}
```

Default: true

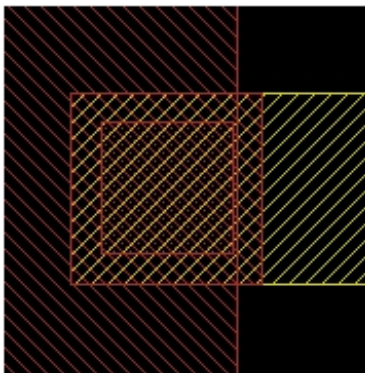
Data_type: bool, read/write

Allows via extending outside of existing wire ends at T-type of intersection. When set to false, via extending outside of the existing wire end are not allowed and may shift inside.

- `generate_special_via_extend_out_wire_end true`



- `generate_special_via_extend_out_wire_end false`



This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

generate_special_via_full_cut_via_only

```
generate_special_via_full_cut_via_only {true | false}
```

Default: false

Data_type: bool, read/write

Allows the user to create only full cut vias.

Example

The following via is generated because of the second left cut will have drc violations when the option is set to false.

- `generate_special_via_full_cut_via_only false`



When set the option to true, only full cut vias will be generated:

- `generate_special_via_full_cut_via_only true`



This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

generate_special_via_hookup_auto_set_viacell

```
generate_special_via_hookup_auto_set_viacell {true | false}
```

Default: `false`

Data_type: `bool`, read/write

When set to `true`, the via generation engine selects via cells automatically in the Voltus insight flow for every cut layer, when hookup via is generated.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

generate_special_via_hookup_contact_max

```
generate_special_via_hookup_contact_max {distance1_max distance2_max}
```

Default: `{0, 0}`

Data_type: `string`, read/write

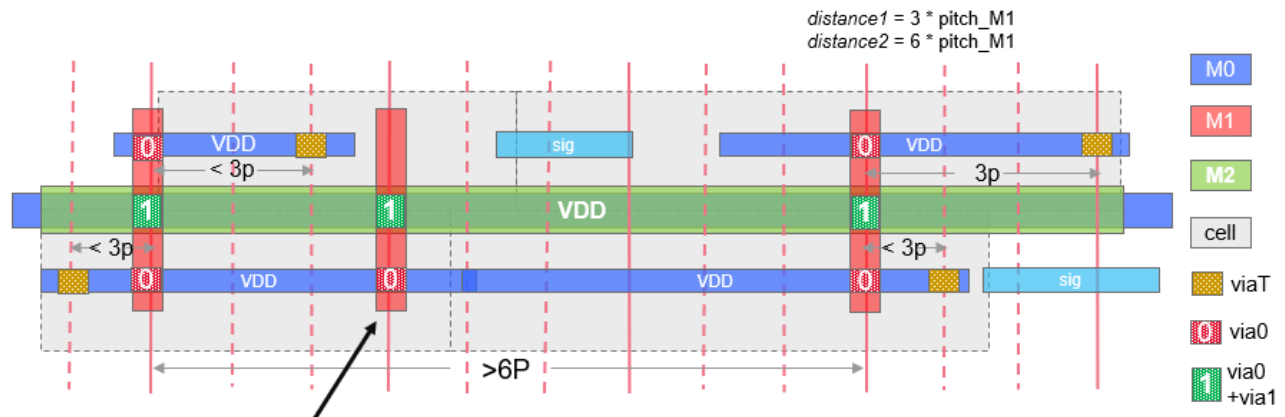
Sets the maximum distance to create hookup via from contact, and the maximum distance between two hookup via on a continuous `M0` PG pin. Following are the attribute values:

- `distance1_max`: Specifies the distance required by every viaT for at least one hookup via drop. For deciding the location of hookup via for `distance1_max`, every viaT needs at least one hookup via drop within `distance1`.
- `distance2_max`: Specifies the maximum distance between any two PG hookup vias on a continuous `M0` PG pin. An additional hookup via must be dropped from the previous hookup via on a continuous `M0` PG segment with `distance2_max`. Also, an additional hookup via must be dropped within the maximum `distance2` from the endpoint of a continuous `M0` PG pin.

Default: `0 0`

The following scenarios depict the usage of this attribute:

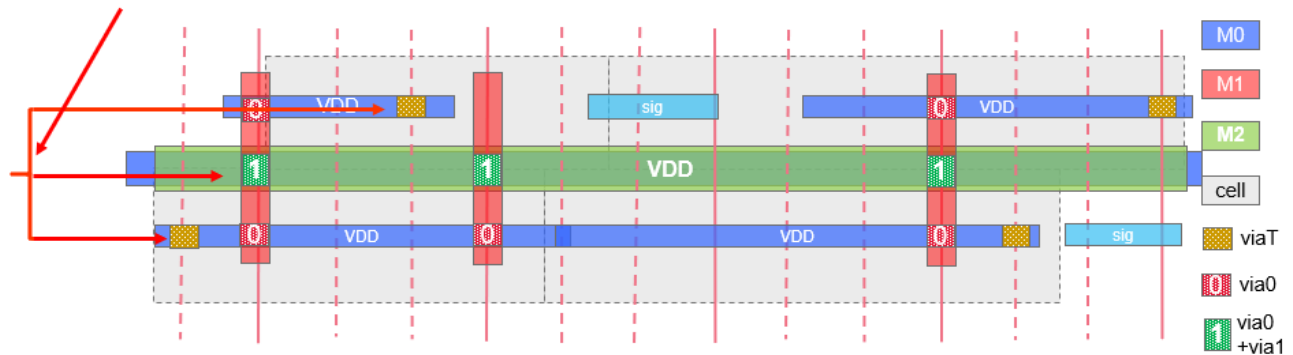
- Deciding the location of hookup via by viaT:



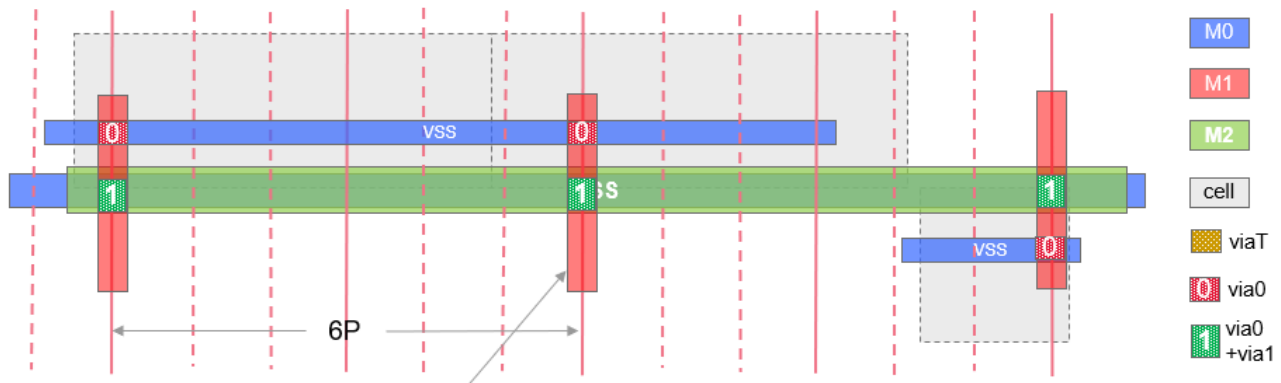
Additional PG hookup be added to ensure that the space between two PG V0s (on a continuous M0 PG track segment) is lesser than or equal to *distance2*.

- Referencing of the location of viaT by the tool to add hookup vias:

The location of hookup via is decided by viaT. The tool references only the viaT, which aligns with the cuts defined by via cells.



- At least one hookup via needs to be available for each continuous M0 PG segment, including PG segment at both sides of the row edge and the PG pin. The continuous M0 PG segment without viaT also needs to follow the decision rule of *distance2*:



Add an additional hookup via to meet 6P max distance

- Four individual via cells are defined in the LEF file and VIAMAP. Of these, one is 3-cut, two are 2-cut, and the remaining one is 1-cut. The further breakup of these via cells is mentioned below:
 - 3-cut via cells: Include three `v0` cuts, three `m0` metals, and one `m1` metals
 - 2-cut via cells: Include two `v0` cuts, two `m0` metals, and one `m1` metals
 - 1-cut via cell: Include one `v0` cuts, one `m0` metals, and one `m1` metals

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

generate_special_via_hookup_contact_pg_track

`generate_special_via_hookup_contact_pg_track {start_value pitch_value}`

Default: {0, 0}

Data_type: string, read/write

Sets start point and the pitch of PG track used by hookup contact. Following are the attribute values:

- `start_value`: Specifies the location of the first PG track.
- `pitch_value`: Specifies the distance (steps) between two adjacent PG tracks. It needs to be an integer multiple of the `m1` pitch.

Note: The PG track needs to be aligned with the `m1` track.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

generate_special_via_hookup_fixed_grid

`generate_special_via_hookup_fixed_grid {true | false}`

Default: false

Data_type: bool, read/write

When set to `false`, the added hookup via can be shifted to avoid DRC. When set as `true`, the hookup via can only be added at the fixed grid.

The fixed grid works with `generate_special_via_hookup_via_style`. For the `loose` and `moderate` styles of `generate_special_via_hookup_via_style`, the hookup via will be removed, if it has DRC while getting created. For the `compact` style of `generate_special_via_hookup_via_style`, the hookup via will be kept, whether or not it has DRC.

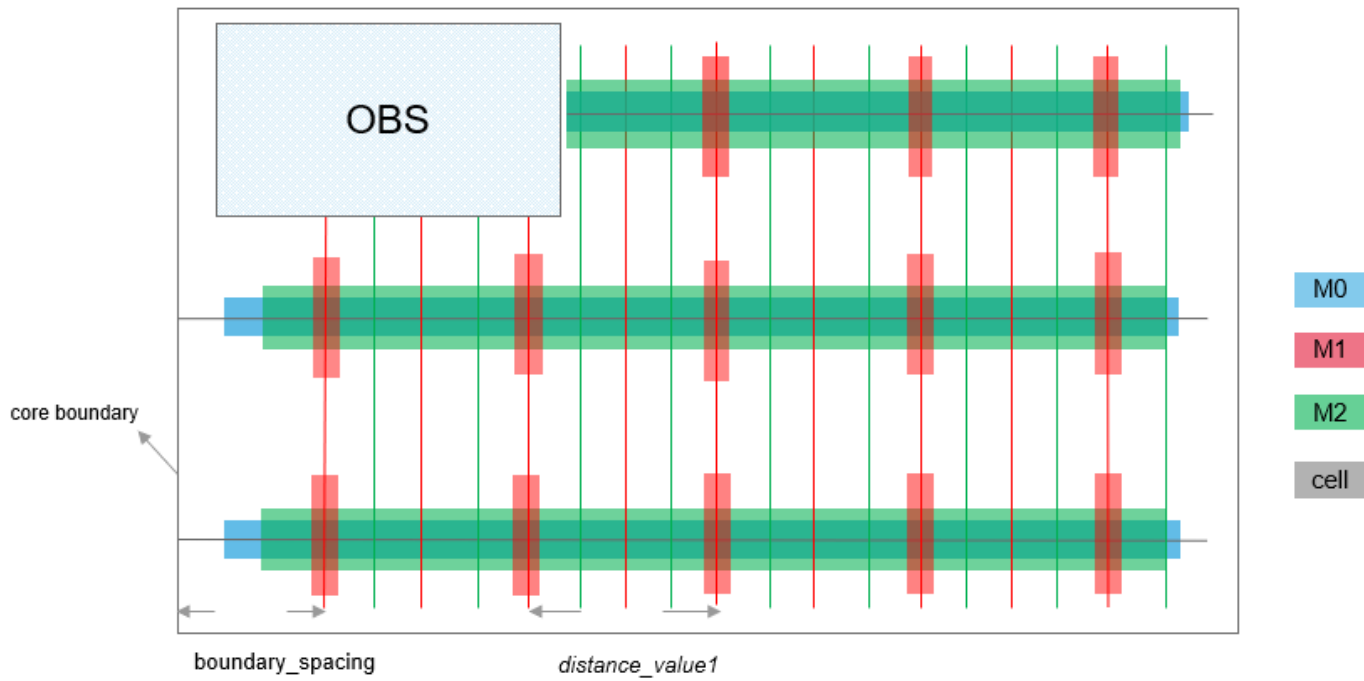
The fixed grid is controlled by the following attribute:

```
set_db generate_special_via_hookup_via_distance {distance_value1 [distance_value2  
[boundary_spacing]]}
```

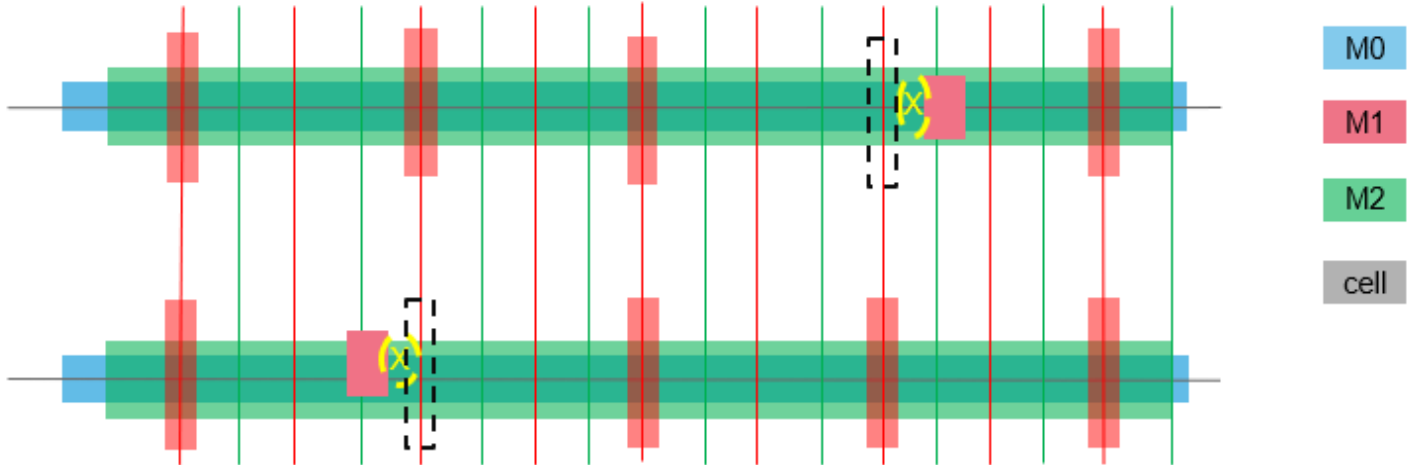
Here:

- `distance_value1` is the pitch of grid.
- `boundary_spacing` is the distance between the first grid and core boundary.
- The fixed grid does not get influenced by the shape of the design or macros.
- `distance_value2` needs to be set same as `distance_value1`.

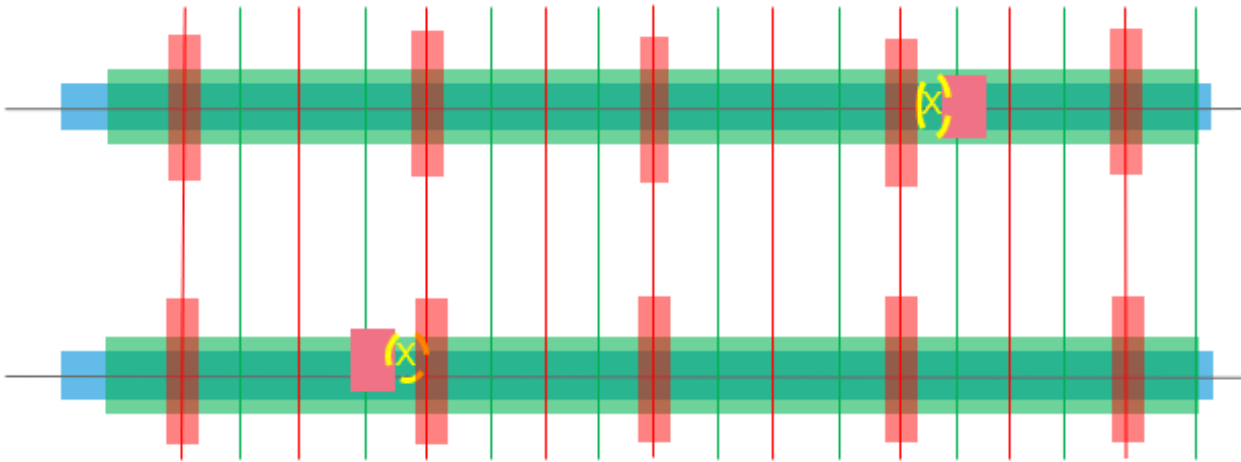
Below is an example of formation fixed grid using `boundary_spacing` and `distance_value1`. Note that all the vias of different lines need to be aligned.



Below is an example of DRC control for the `loose` and `moderate` types of hookup via styles. Here, the hookup vias are removed, if DRC exists.



Below is an example of DRC control for the `compact` type of hookup via style. Here, the hookup via will be kept, irrespective of the existence of DRC.



This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

generate_special_via_hookup_min_distance

`generate_special_via_hookup_min_distance` *string*

Default: `min`

Data_type: string, read/write

Specifies the minimum distance between two PG vias. This attribute works with the distance between:

1. Two newly added PG vias
2. Existing PG vias and newly added PG vias on the followpin

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

generate_special_via_hookup_preplace_fix

`generate_special_via_hookup_preplace_fix {true | false}`

Default: false

Data_type: int, read/write

Enhances the hookup via generation runtime during the DRC check.

This attribute works only in the multiple-CPU processing mode.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

`set_db attribute_name value`

- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

generate_special_via_hookup_rail_pair

`generate_special_via_hookup_rail_pair {true | false}`

Default: false

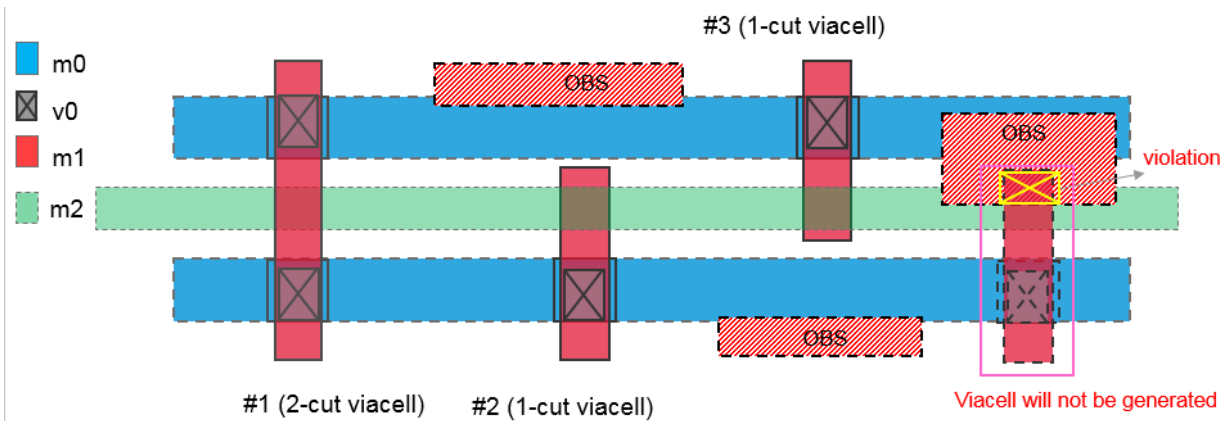
Data_type: bool, read/write

Adds hookup via over the pairing M_0 followpins that are distributed on both sides of a row with specially defined viacells at LEF file. Note that while using this attribute, the PG pins need to be on track, and the width of the M_0 followpin needs to be uniform.

This attribute works only with setting the `loose` value of `generate_special_via_hookup_via_style`. Three viacells are defined in the LEF file. One of those is 2-cut, and the remaining two are 1-cut. Refer to their details below:

- 2-cut viacell = Two v_0 cuts + Two m_0 metals + One m_1 metals
- 1-cut viacell = One v_0 cuts + One m_0 metals + One m_1 metals

For the 1-cut viacell, m_1 needs to overlap completely with M_2 . If this is not possible, the viacell will not be generated. Refer to the example below:



This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

generate_special_via_hookup_via_distance

```
generate_special_via_hookup_via_distance {distance_value1 [distance_value2 [boundary_spacing]]}
```

Default: {4pitch 4pitch 0}

Data_type: string

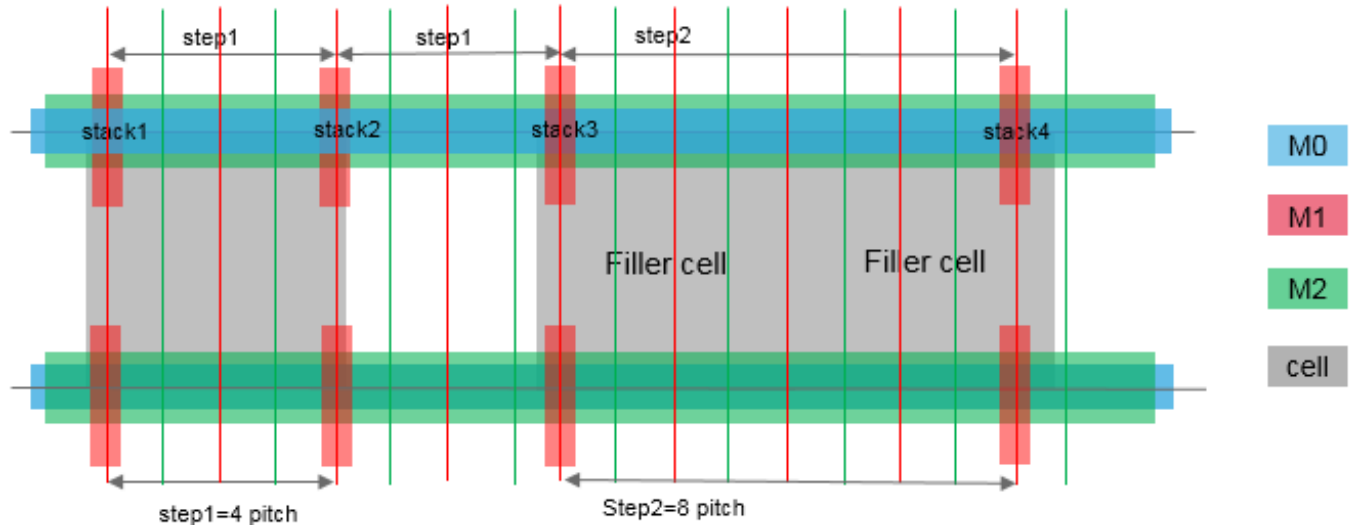
Enables control to put the first pg via for each followpin. You can use this attribute in the following ways:

```
set_db generate_special_via_hookup_via_distance {distance_value1}
set_db generate_special_via_hookup_via_distance {distance_value1 distance_value2}
set_db generate_special_via_hookup_via_distance {distance_value1 distance_value2 boundary_spacing}
```

Following are the possible attribute values:

- `distance_value1`: Specifies the distance (step) between via stacks and is rounded down to `M1 pitch*n`. Default value is `M1 pitch*4`.
- `distance_value2`: Specifies the distance between via stacks on filler cell. This value can work

with all hookup styles. Default value is $M1 \text{ pitch} * 4$. It indicates the via distance when two neighboring via stacks are over filler cells. Refer to the figure below:



- *boundary_spacing*: Specifies the distance between the first via and core boundary. It also refers the minimum distance between the last via and core boundary.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

generate_special_via_hookup_via_style

```
generate_special_via_hookup_via_style {none loose compact moderate}
```

Default: none

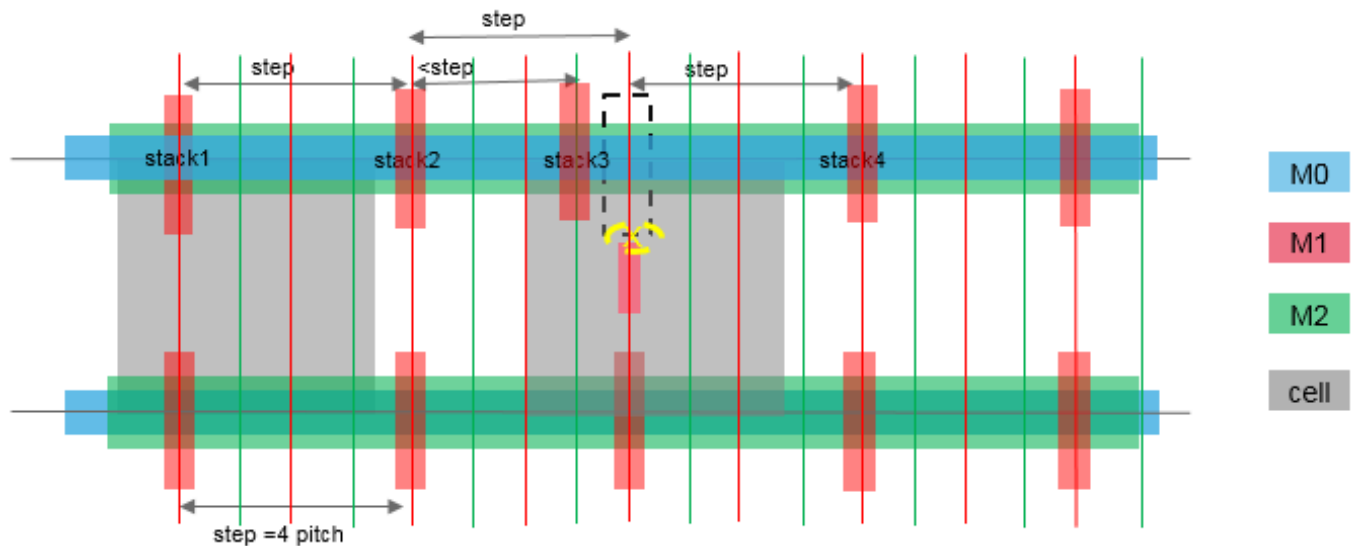
Data_type: enum {none loose compact moderate}, read/write

Sets hookup via style. The hookup via works between two parallel metal layers (like `M0` and `M2`, or `M1` and `M3`). Assuming it for `M0` and `M2`, viagen adds stack vias between `M0` and `M2` according to the hookup via settings.

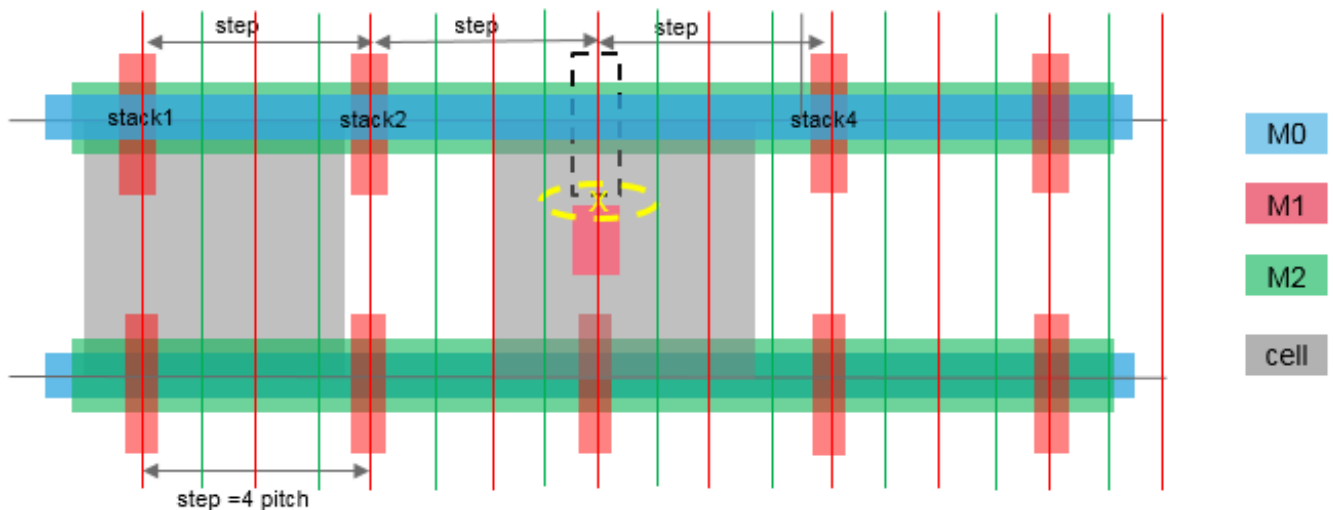
The associated functionality of various options is provided below:

- `none`: No via hookup style is set. This is the default value.

- `loose`: The tool inserts vias between paralleled wires with the specified step in `generate_special_via_hookup_via_distance`. The center of the left-most via needs to be on the track, which is inside the intersection. The via also needs to be inside the intersection. If a via stack (like `stack3`) cannot be dropped at a certain step position (dot line area in the figure below) because of the DRC violation, it is moved to a closer or farther position with the previous stack (`stack2`) to ensure DRC clean. Additionally, the next via (`stack4`) needs to keep the step value with the original position of moved stack (dot line area in the figure below).

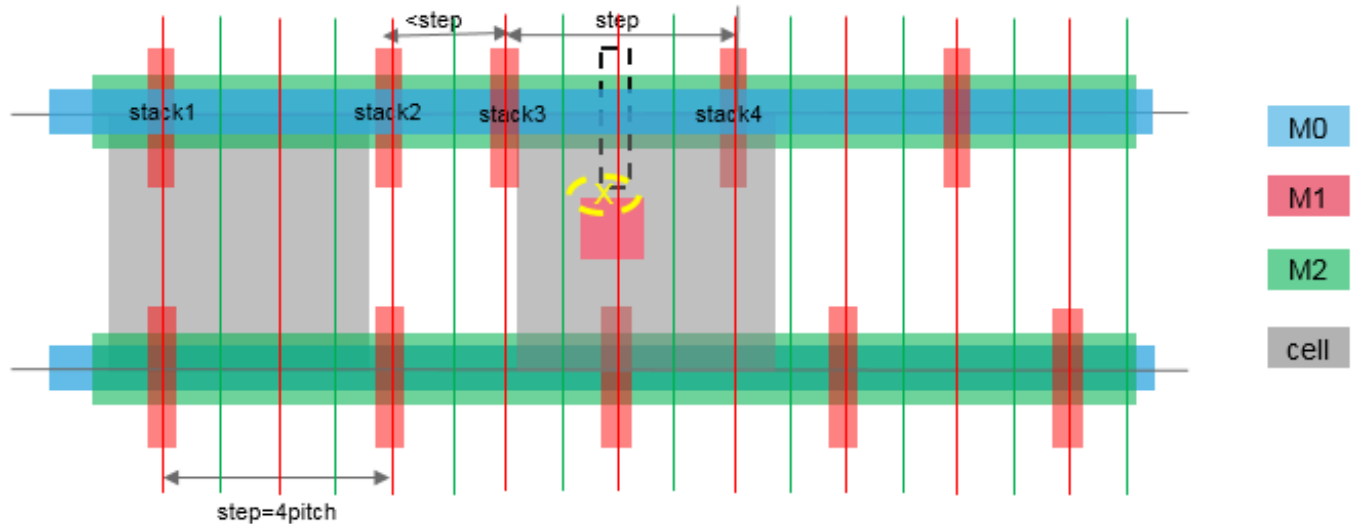


If DRC clean does not occurs, the via stack is removed, refer to the figure below:

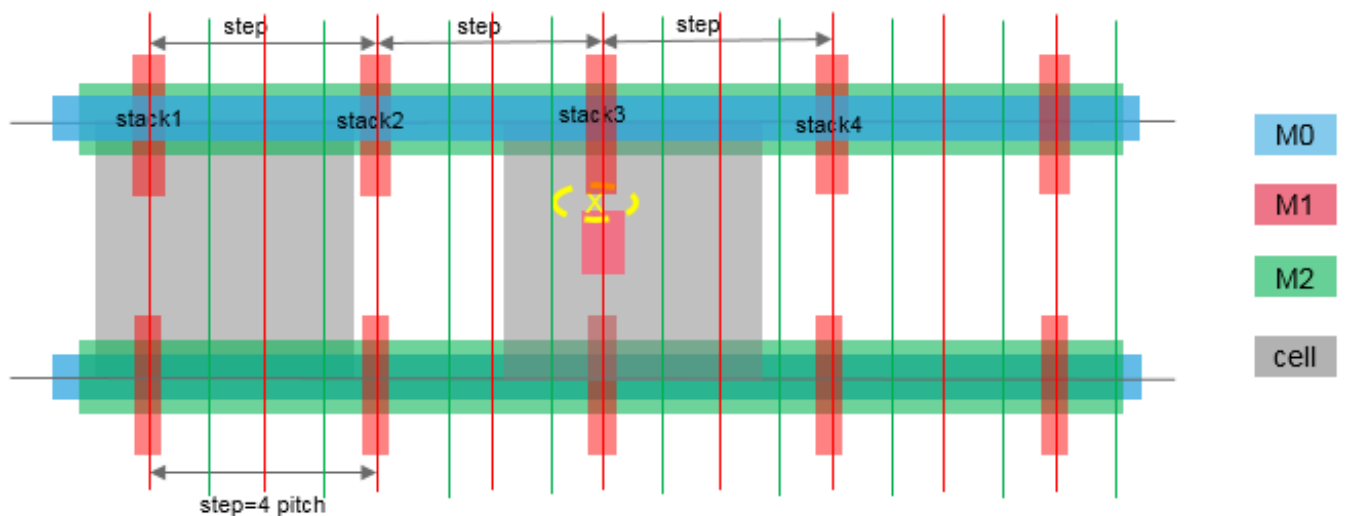


Note: The hookup style is `loose` by default, if the via map table does not exist.

- **compact**: If a via stack (like *stack3*) cannot be dropped at a certain step position (dot line area in the figure below) because of DRC violation, it is moved to a closer position with the previous stack (*stack2*) to ensure DRC clean. And the next via (*stack4*) needs to keep the step value with the said via (*stack3*). Refer to the figure below:



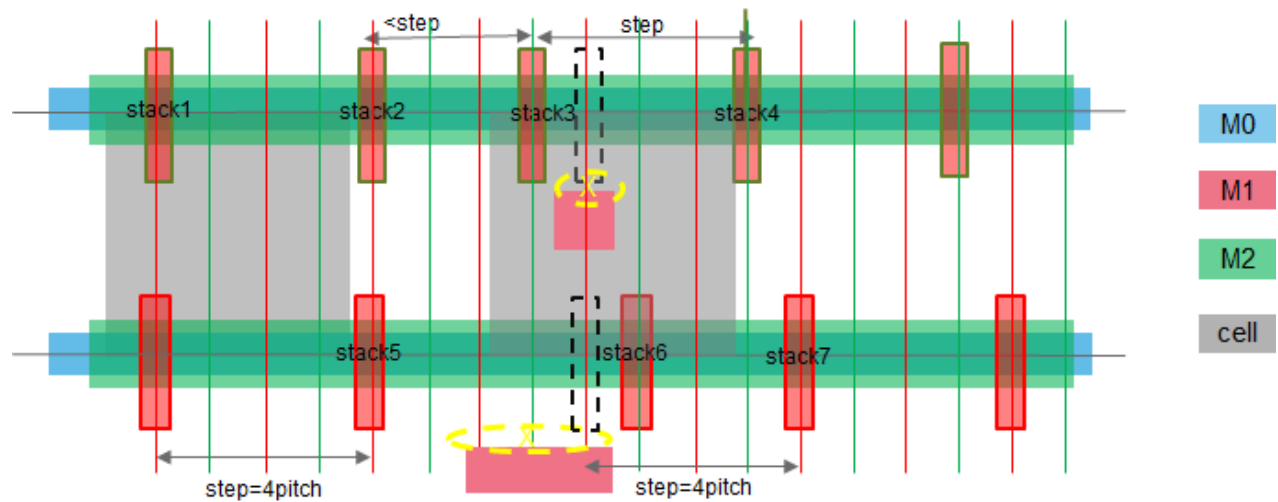
If DRC clean does not occurs by moving the stack, insert the via stack at the original position with DRC violation marker. Refer to the figure below:



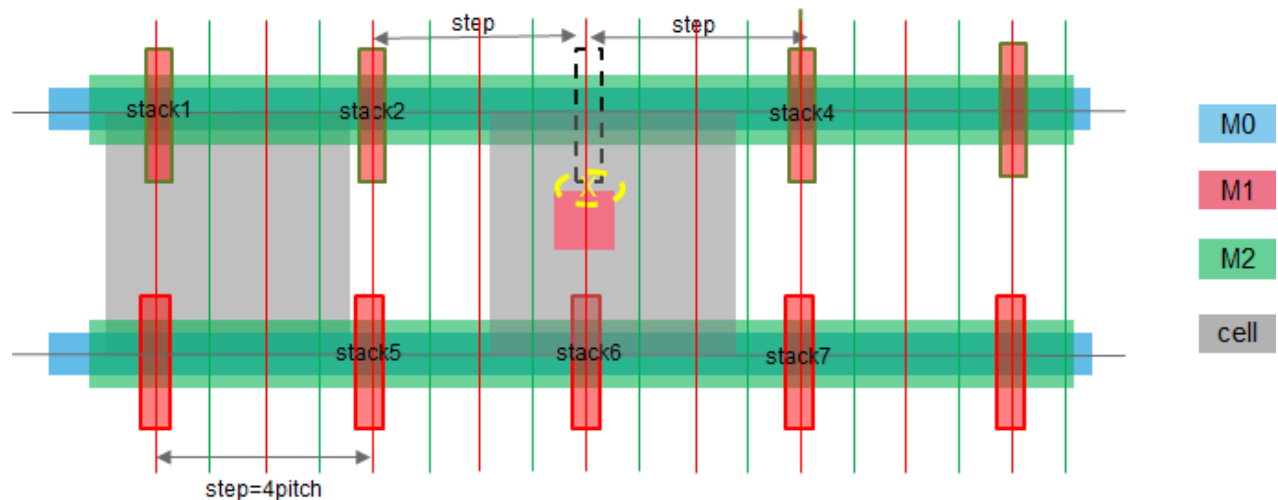
Note: The hookup style is **compact** by default, if the via map table exists.

- **moderate**: If a via stack (like *stack3*) cannot be dropped at a certain step position (dotted line area in the figure below) due to the DRC violation, it is moved to a closer or farther position with the previous stack (*stack2*) to ensure DRC clean. If the via stack (*stack3*) is dropped to a closer position with the previous stack (*stack2*), the next via (*stack4*) needs to keep the step

value with the said stack (*stack3*). If the via stack (*stack6*) is dropped to a farther position with the previous stack (*stack5*), the next via (*stack7*) needs to keep the step value with the original position of the moved stack.



If DRC clean is not occurred, *viaGen* checks the next tracks followed to insert the via stack at the nearest DRC clean track.



This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

generate_special_via_hookup_via_rule

```
generate_special_via_hookup_via_rule string
```

Default: ""

Data_type: string, read/write

Specifies the preferred viarule for the designated cell area during the hookup via generation. This attribute can also work with the `generate_special_via_preferred_vias_only` attribute to use different types of viarule orders for filler cell to be DRC clean.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

generate_special_via_hookup_virtual_trim_grid

```
generate_special_via_hookup_virtual_trim_grid {-layer layerNumber -mask1 {-pitch value  
-core_offset value} -mask1 {-pitch value ...} -mask2 {-pitch value ...} -mask2 {-pitch  
value ...}}
```

Default: ""

Data_type: string, read/write

Creates a virtual trim grid for PG hookup via generation. The created virtual trim grid should align with the via metal of the newly created hookup via. When this attribute is specified, the existing trim grid is ignored.

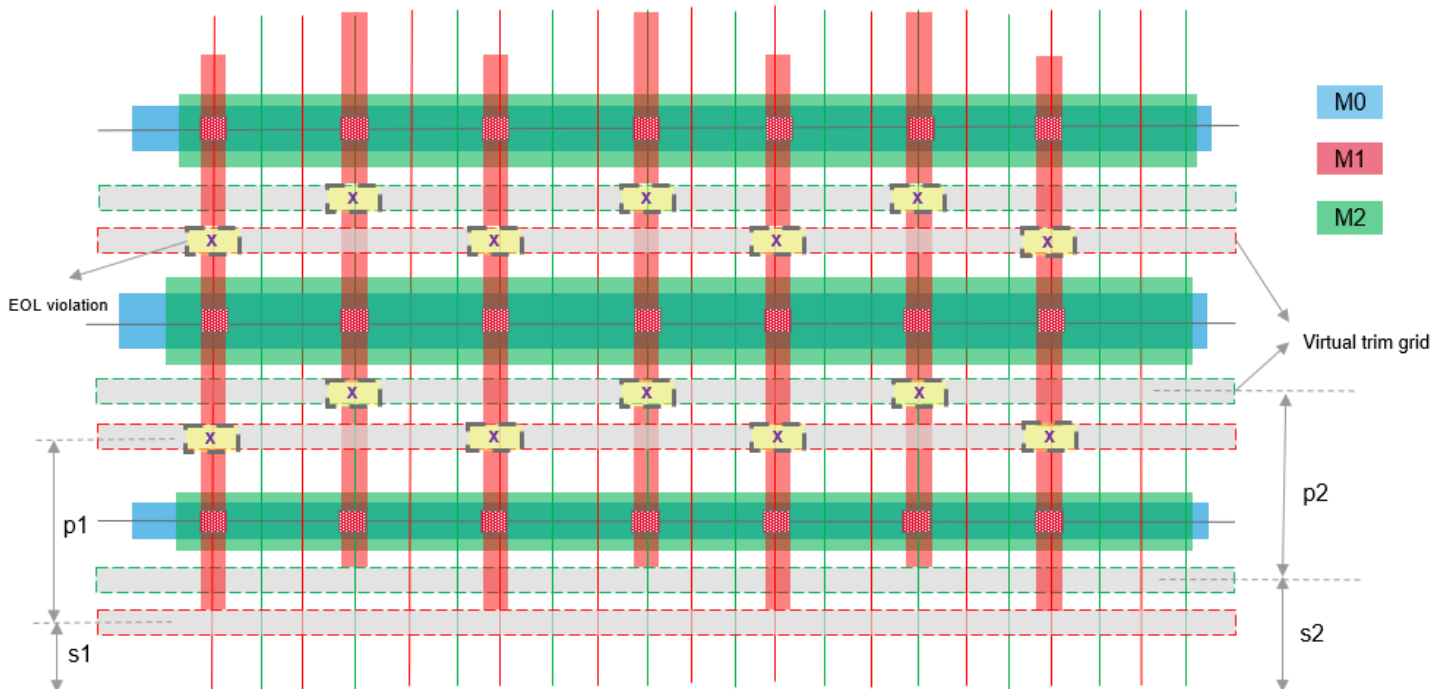
To honor the existing trim grid, either remove this attribute or define the existing trim grid within this parameter. `-core_offset` is calculated from the bottom core boundary to the center of the first grid.

Note: The width value for this attribute is taken from the LEF file.

Following is an example of creating a virtual trim grid for PG hookup via generation:

```
generate_special_via_hookup_virtual_trim_grid -hookup_virtual_trim_grid {-layer M1 -  
mask1 {-pitch p1 -core_offset s1 } -mask2 { -pitch p2 -core_offset s2 }}
```

Following is the output of the above command:



This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
- To get the current value of any root attribute, use the `get_db` command:

```
set_db attribute_name value
```

```
get_db attribute_name
```

generate_special_via_ignore_design_boundary

```
generate_special_via_ignore_design_boundary {true | false}
```

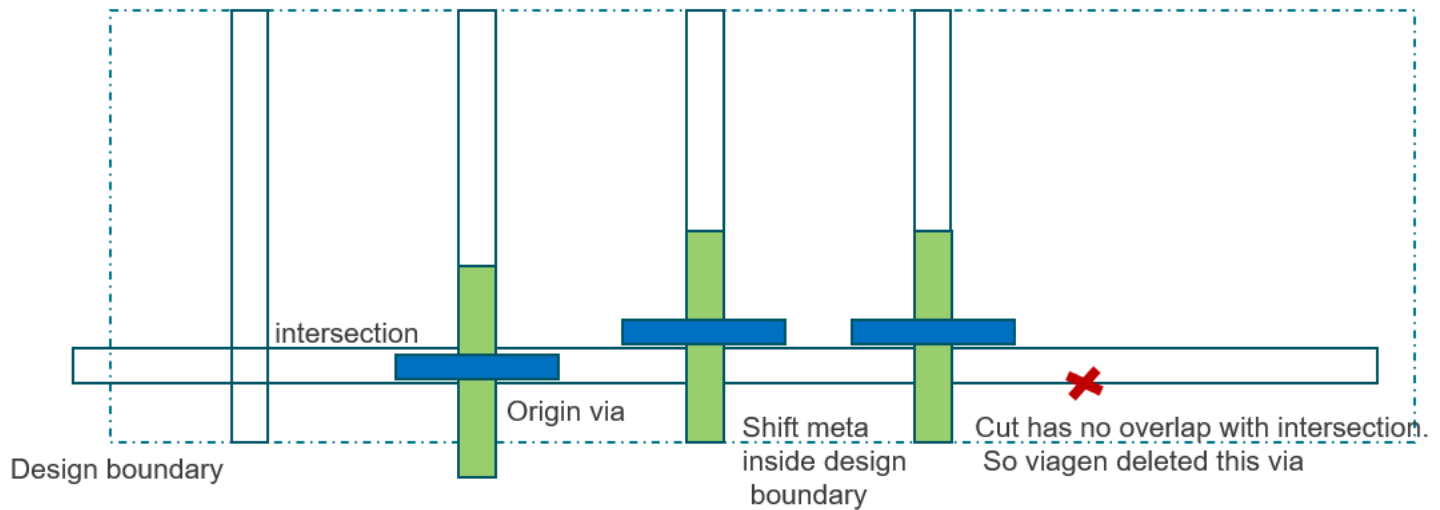
Default: false

Data_type: bool, read/write

Specifies whether a via can be extended out of the design boundary (floorplan area).

- When set to `true`, viagen generates vias and ignores the design boundary.
- When set to `false`, viagen tries to shift vias first inside the boundary area. If it does not overlap with the intersection point after shifting, viagen deletes it. If it overlaps with the intersection point after shifting with DRC clean, viagen keeps it.

See the figure below for understanding the usage of this attribute:



This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
- To get the current value of any root attribute, use the `get_db` command:

```
set_db attribute_name value
```

```
get_db attribute_name
```

generate_special_via_ignore_drc

```
generate_special_via_ignore_drc {true | false}
```

Default: false

Data_type: bool, read/write

Setting this option ignores DRC but honors user-defined values when creating vias. This speeds up via creation during power-planning but normally causes DRC violations.

It should only be used during early prototyping to experiment with different power-planning choices that will be updated later to be DRC clean.

Note: This attribute ignores only the DRC violations from third party objects, and the software will still check the violation inside via.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

generate_special_via_ignore_rule_enclosure

```
generate_special_via_ignore_rule_enclosure {true | false}
```

Default: true

Data_type: bool, read/write

When set to `true`, the software ignores enclosure value in VIARULE GENERATE and uses the optimized enclosure from cut layer rules, to get the maximum number of cuts, minimum via metal shape. When set to `false`, the enclosure value in VIARULE GENERATE must be satisfied. When this VIARULE fails DRC check, the software takes next VIARULE or viacell from LEF file.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

generate_special_via_inherit_wire_status

```
generate_special_via_inherit_wire_status {true | false}
```

Default: false

Data_type: bool, read/write

Setting this attribute to `true` creates via and makes the status of the via inherit the target wire's status. However, if the status of the top or bottom wire is SHIELD, this attribute is ignored and the generated vias will always inherit the SHIELD status. When this attribute is set to `false`, via status is set to ROUTED.

For `update_power_vias`, the target wire is the bottom wire. For other commands calling the VIAGEN engine, either the bottom or the top wire is the target wire, depends on order of wire read-in. So if this attribute is designed to use, `update_power_vias` is the recommended command.

`update_power_vias` does not delete or modify vias having status as FIXED/COVER.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

generate_special_via_keep_existing_via

```
generate_special_via_keep_existing_via {remove | keep_with_new_via |  
keep_with_no_new_via}
```

Default: remove

Data_type: enum {remove keep_with_new_via keep_with_no_new_via}, read/write

If a via exists in an intersection and does not cover the whole intersection, a new via is tried to be dropped in the same intersection. Following are the attribute values:

- `remove`: Prevents keeping an existing via, while dropping a new via.
- `keep_with_new_via`: Keeps an existing via, while dropping a new via.
- `keep_with_no_new_via`: Keeps an existing via and does not drop a new via at the same intersection, even when the existing via does not cover the whole intersection area.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

generate_special_via_keep_fixed_via

```
generate_special_via_keep_fixed_via {true | false}
```

Default: true

Data_type: bool, read/write

Controls whether to preserve the fixed vias or not when `viaGen` is called for via generation.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

generate_special_via_mincut_preference

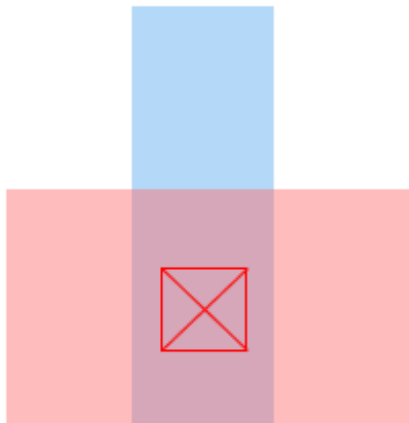
```
generate_special_via_mincut_preference int_value
```

Default: 1

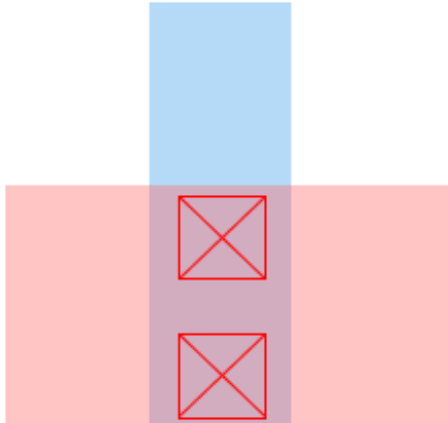
Data_type: int, read/write

Specifies the minimum number of cuts during via generation at an intersection. For example:

- If this attribute is set to 1, viagen generates at least one cut at an intersection, as shown below:



- If this attribute is set to 2, viagen generates at least two cuts at an intersection, as shown below:



This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```
- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

generate_special_via_opt_cross_via

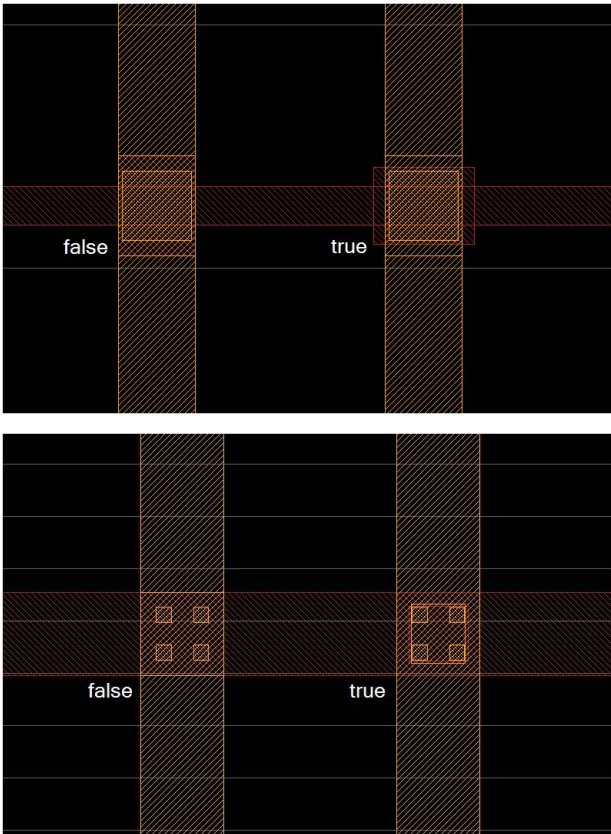
`generate_special_via_opt_cross_via {true | false}`

Default:

- `true` for advanced node (10 NM and 7 NM) designs.
- `false` for other designs.

Data_type: bool, read/write

Generate vias with the minimum enclosure value on each layer. The different result with false or true are shown below:



This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```
- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

generate_special_via_opt_via_on_routing_track

```
generate_special_via_opt_via_on_routing_track {true | false}
```

Default: false

Data_type: bool, read/write

Free up routing tracks by shift or modify the vias minimally in intermediate layers and not applied to layers where the wires or pins exist. This option only supports generated vias and not predefined vias.

Note: Free up routing tracks against forbidden spacing rule for cross via and save routing tracks

against endofline spacing rule for non cross via. `generate_special_via`
`generate_special_via_opt_cross_via 1` will try to generate cross via.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

generate_special_via_parameterized_via_only

`generate_special_via_parameterized_via_only {auto | true | false}`

Default:

- `false`, when the `oa_write_cds_fixed_vias` attribute is enabled in the OpenAccess design.
- `auto`, for all other cases.

Data_type: enum {auto true false}, read/write

When the value of this attribute is set to:

- `true`, ViaGen generates parameterized vias only, which means the DEF syntax of generated vias is with VIARULE. It helps in OpenAcceess designs.
- `false`, ViaGen generates both parameterized and fixed vias.
- `auto`, it is `true` for OpenAccess design, `false` for non-OpenAccess design.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

generate_special_via_partial_overlap_threshold

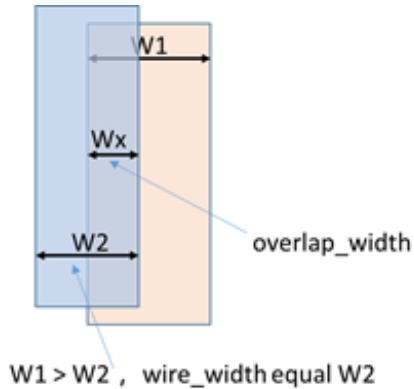
`generate_special_via_partial_overlap_threshold float`

Default: 0.0

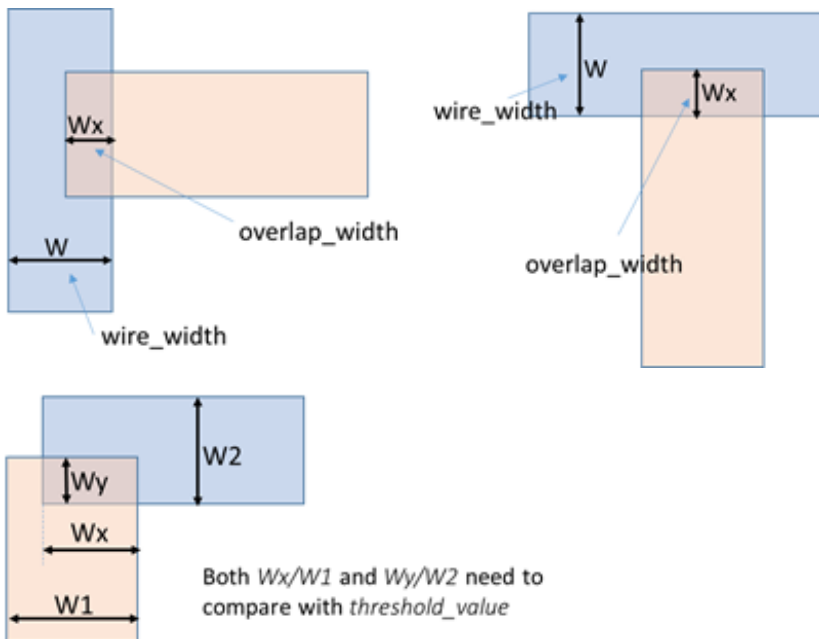
Data_type: double, read/write

Specifies ratio threshold of via creation on partial overlap wires. The parameter value may range from 0 to 1. The parameter value 1 specifies that no via is dropped on partial overlap intersection. The tool drops via only when $\text{overlap_width}/\text{wire_width}$ is greater than or equal to the *threshold_value*.

The following figure depicts parallel overlapping wires:



The following figures depict vertical cross wires:



This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```
- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

generate_special_via_preferred_vias_only

```
generate_special_via_preferred_vias_only {use_lef | open | keep}
```

Default: use_lef

Data_type: enum {use_lef open keep}, read/write

Generates only the vias set in `generate_special_via_rule_preference`. This attribute must be used with `generate_special_via_rule_preference`.

When the value of this parameter is set to:

- **use_lef:** viarule or via cell specified in `generate_special_via_rule_preference` is firstly considered. If none of them fits, viagen chooses the first DRC clean predefined viarule in the LEF file or auto viarule that can bring maximum cut area. Note that:
 - Cut area = single cut area * cut number
 - Auto viarule includes generate viarule and virtual viarule. Virtual viarule means viarule generated by viagen based on the cut layer rule.
- **open:** Only predefined viarule or via cell specified in `generate_special_via_rule_preference` is considered. The first one with DRC clean is kept. If all of them have DRC issues, the whole stack is left open.
- **keep:** Only predefined viarule or via cell specified in `generate_special_via_rule_preference` is considered. The first one with DRC clean is kept. If all of them have DRC issues, keep the first one which is DRC clean with surrounding. If all of them have DRC issues with surrounding, the whole stack is left open.

For more information on predefined viarule and generate viarule, see the *Via Rule* and *Generate Via Rule* sections respectively of the [LEF Syntax](#) chapter of the *LEF/DEF Language Reference* document.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

generate_special_via_prefix

`generate_special_via_prefix viaNamePrefix`

Default: none

Data_type: string, read/write

Sets the specified prefix name as the prefix for all generated vias created by Innovus. This helps user to select and edit the vias that need further modification.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

generate_special_via_reference_boundary

`generate_special_via_reference_boundary {design | core}`

Default: design

Data_type: enum {design core}, read/write

Specifies the offset reference when specifying offset values in the `generate_special_via_split_long_via_global_grid` attribute. Following are the attribute values:

- `design`: ViaGen uses design boundary as reference.
- `core`: ViaGen uses core boundary as reference.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

generate_special_via_respect_stdcell_geometry

`generate_special_via_respect_stdcell_geometry {true | false}`

Default: `false`

Data_type: bool, read/write

Respects geometry inside standard cell patterns.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

`set_db attribute_name value`

- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

generate_special_via_respect_wire_ndr

`generate_special_via_respect_wire_ndr {true | false}`

Default: `false`

Data_type: bool, read/write

Controls spacing between hookup via and signal routes. This allows viagen to support all NDR rules.

- When set to `false`, viagen honors a set of NDR rules, which include PG wire NDR, PG via NDR, Pin wire NDR, pin via NDR, and signal via NDR.
- When set to `true`, the signal wire NDR is also honored, including those listed above with the `false` value selection of this attribute. This means that setting this attribute to `true` honors all the NDR rules.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

`set_db attribute_name value`

- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

generate_special_via_rule_preference

`generate_special_via_rule_preference {default | predefined | generated | <list of via rule/cell names> | <file name>}`

Default: default

Data_type: string, read/write

Sets the preferred via-rule usage. Use this parameter before running the `fcroute`, `sroute`, or power planning commands.

- `default`
Uses the first via rule defined in the LEF technology file as the preferred usage.
- `predefined`
Uses the predefined via rule defined in the LEF technology file as the preferred usage.
- `generated`
Uses the generated via rule defined in the LEF technology file as the preferred usage.
- `list of via rule/cell names`
Uses the list of via rules/cells defined in the LEF technology file as the preferred usage.
- `file name`
Uses via rules/cells which are contained in the specific file.

This attribute works in the incremental mode. See the example below:

```
set_db generate_special_via_rule_preference {V1 V2}  
set_db generate_special_via_rule_preference {V3}  
V1 V2 V3
```

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

generate_special_via_set_via_expand_dir

`generate_special_via_set_via_expand_dir {auto | horizontal | vertical}`

Default: auto

Data_type: enum {auto horizontal vertical}, read/write

Sets via expansion directions.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

generate_special_via_snap_via_center_to_grid

```
generate_special_via_snap_via_center_to_grid { layer1 {none | grid | half_grid | either  
} layer2 {none | grid | half_grid | either } ... }
```

Default: none

Data_type: string, read/write

Specifies the via center alignment style per routing layer.

none

Keeps previous behavior, only snaps to wire center line.

grid

Snaps via to the full grid layer track.

half_grid

Ignores the full grid. Only snaps to the `half_grid` layer track.

either

Can snap to both full and half grid layer track.

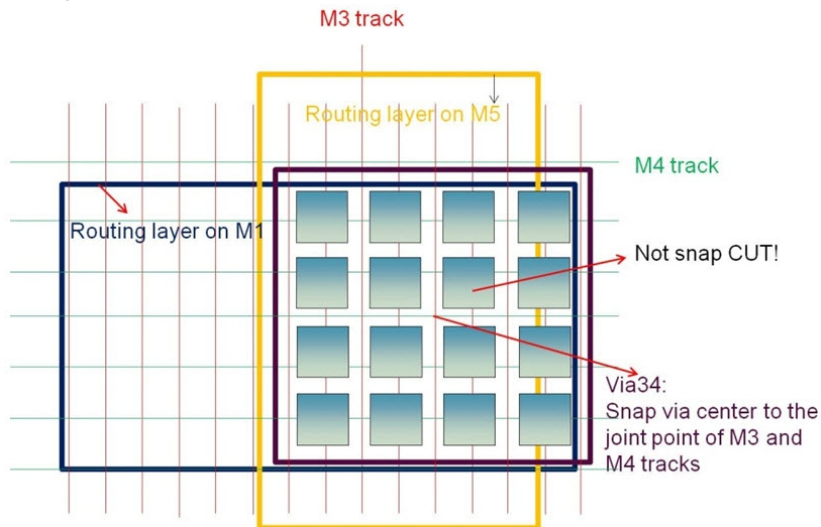
Note: Metal1 and Metal2 cannot be defined in option `snap_via_center_to_grid`. Innovus will not shift the via even when Metal1 and Metal2 are defined in the option.

- V12 is not shifted in both vertical and horizontal directions
- V23 is not shifted in vertical direction

Examples

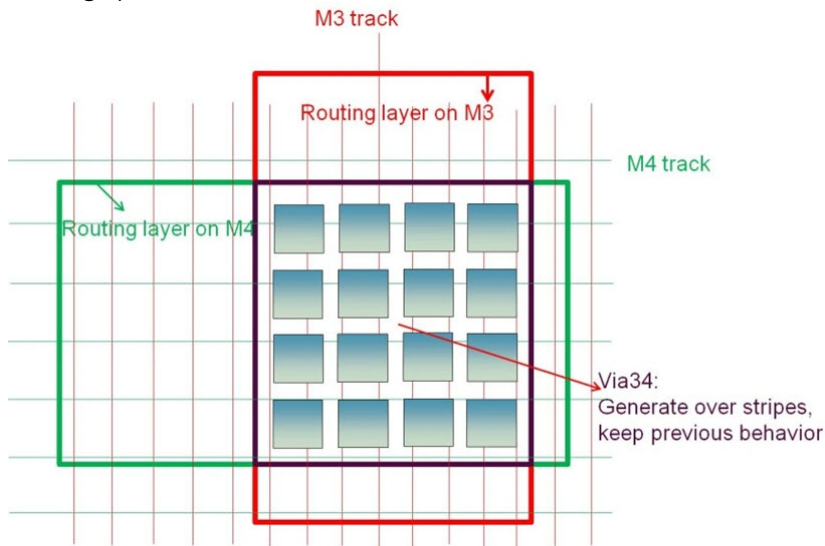
- **Snap the vias to each adjacent up and down layers, except Metal1 and Metal2**

- NO stripes on up and down layers
- Snap the center of via (the whole via, not CUT) to the closest track intersection



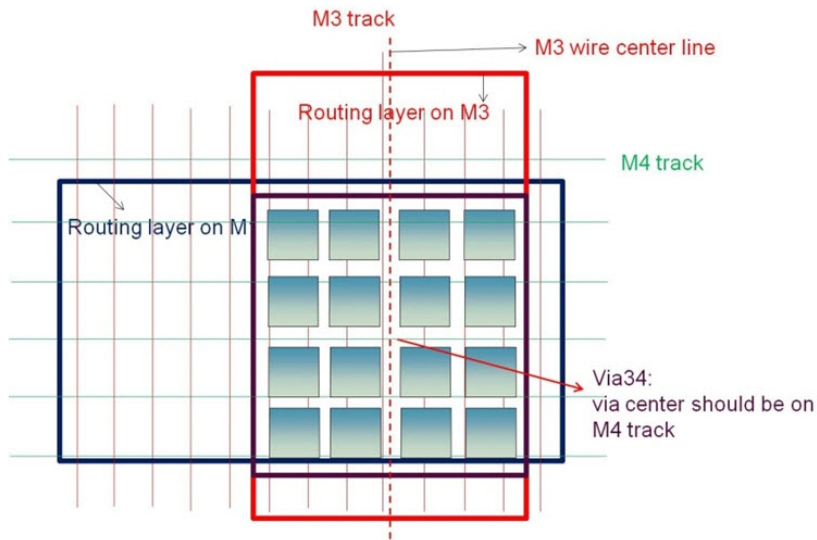
- **Stripes on both up and down layers**

- Generate via on the stripes intersection area (the same as previous behavior, NO change)



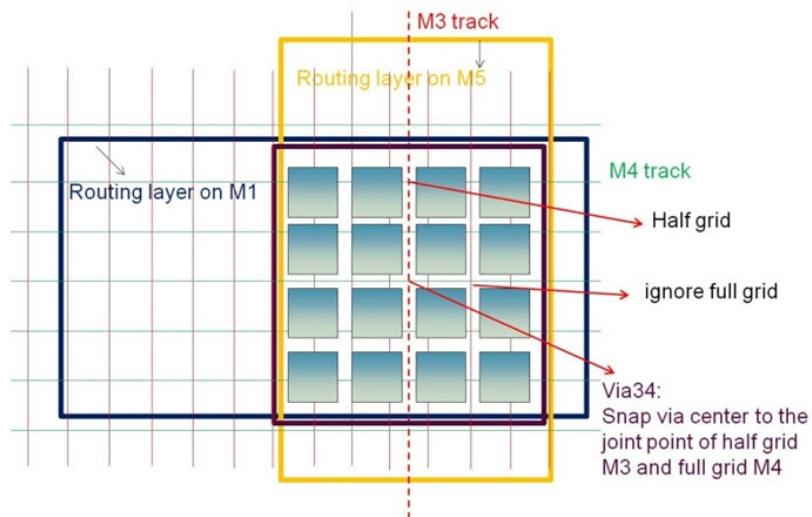
- **Stripe on adjacent layer metal3 and NO stripe on the other neighboring routing layer**

- Find the joint point between stripe center and the closest track
- Snap the center of via (the whole via, not CUT) to this joint point



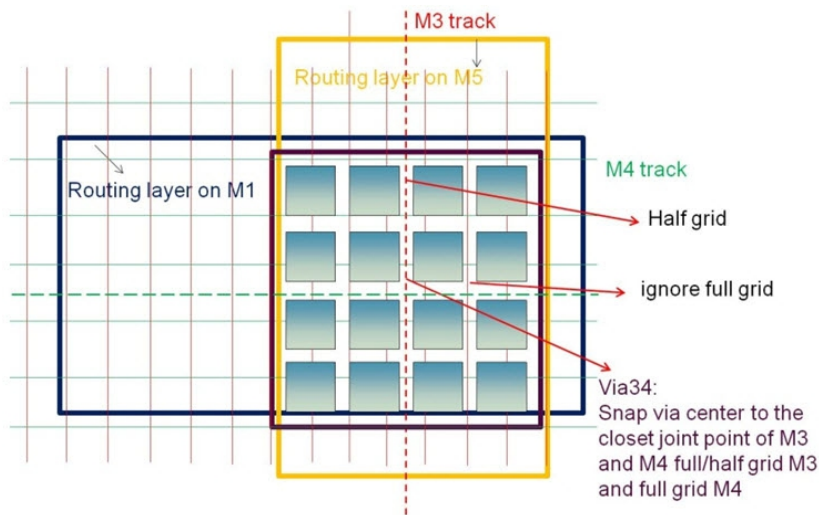
- **Snap via to the full grid of metal4) and half grid of metal3**

- `generate_special_via -snap_via_to_grid {metal3 half_grid metal4 grid}`
- Snap via to metal3 half grid and metal4 full grid



- **Snap via to full or half grid of metal3 and metal4**

- `generate_special_via -snap_via_to_grid {metal3 either metal4 either}`
- Tool could snap to closest full or half grid automatically



This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

generate_special_via_split_long_via_global_grid

```
generate_special_via_split_long_via_global_grid { x_offset x_pitch  
x_length_threshold x_length_multiplier y_offset y_pitch  
y_length_threshold y_length_multiplier }
```

Default: default

Data_type: string, read/write

Provides options to control global grids so that vias on the grids are in line on x and/or y directions.

- `x_offset, y_offset` : specifies in microns the offset from design boundry
- `x_pitch, y_pitch` : specifies in microns the centre to centre distance in split vias
- `x_length_threshold, y_length_threshold` : specifies vias longer or equal to threshold are being split
- `x_length_multiplier, y_length_multiplier` : specifies unit via length multiplier

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

generate_special_via_symmetrical_via_only

```
generate_special_via_symmetrical_via_only {true | false}
```

Default: false

Data_type: enum {auto true false}, read/write

By default the metal-width of a generated-via matches the width of the intersection of the metal above and below the via. If the width is an odd number of manufacturing grids (mgrids) wide, the resulting via cannot be represented by a parameterized via (LEF/DEF VIA with VIARULE and parameters or OpenAccess standardVia) because they require a symmetrical enclosure values on both sides of the cuts. In that case, a fixed-name via (LEF/DEF VIA with RECT statements or OpenAccess customVia) must be created to have the same width as the intersection area.

If this option is true, the via metal is enlarged one mgrid in the odd-mgrid directions order to make the via symmetrical so it can be represented as a parameterized via. This does not cause any DRC violations in general because the via metal should still be covered by the metal above/below the via and avoid any min-step violations.

Fixed-name vias can cause problems in hierarchical flows as the fixed-names from different blocks may collide. Parameterized via-names can be freely changed by applications and avoid this problem. If the design has odd-mgrid width wires or pins, this option is normally a good choice.

If this attribute is selected as `auto`, it is `true` for OpenAccess design, `false` for non-OpenAccess design.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

generate_special_via_top_enclosure

`generate_special_via_top_enclosure {overhang1 overhang2}`

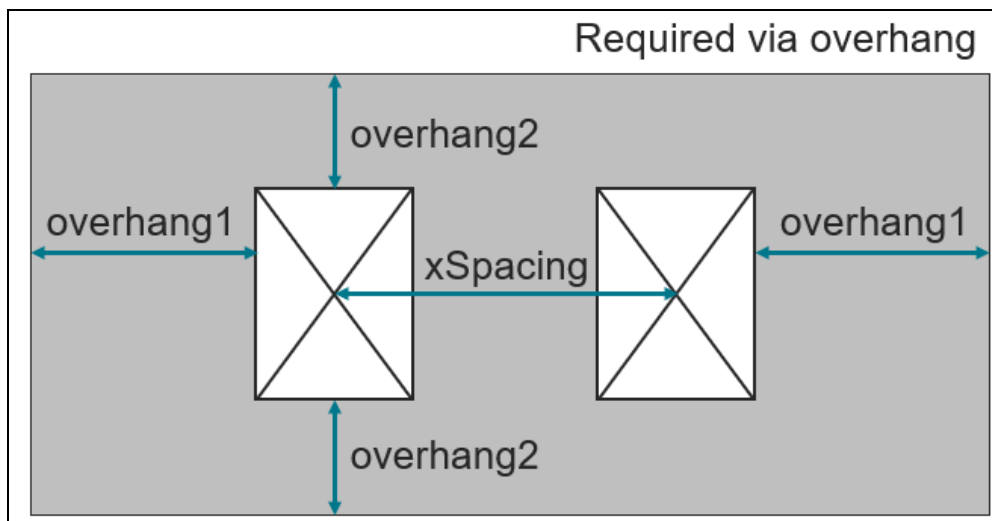
Default: ""

Data_type: string, read/write

Generates vias with the specified top enclosure values, without modifying the LEF file. Set the `generate_special_via_ignore_rule_enclosure` attribute to `false` while using this attribute, so that the engine can compare the specified `overhang1` and `overhang2` values with the Via Rule Generate values.

The `overhang1` and `overhang2` values must be above the Via Rule Generate values.

When the `generate_special_via_top_enclosure` attribute is used, the via is covered by metal layers on two opposite sides by at least `overhang1` distance, and on the other two sides by at least `overhang2` distance. The via generation code then chooses the direction of overhang that best maximizes the number of cuts to fit in the generated via. See the example below:



This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

generate_special_via_use_track_offset

`generate_special_via_use_track_offset {true | false}`

Default: `false`

Data_type: bool, read/write

Uses track offset.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

`set_db attribute_name value`

- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

generate_special_via_use_trim_metal_enclosure

`generate_special_via_use_trim_metal_enclosure {true | false}`

Default: `false`

Data_type: bool, read/write

Forces the tool to use enclosures with the trimmed layer.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

`set_db attribute_name value`

- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

get_db Category Attributes

The root attributes in the `get_db` category are:

- `get_db_display_limit`

To set the value of any root attribute, use the `set_db` command:


```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `get_db` category, use the following command:

```
get_db -category get_db
```

get_db_display_limit

```
get_db_display_limit {integer}
```

Default: 10

Controls the length of lists and number of objects displayed on the screen using queries that contain wildcards.

Note: This variable does not change the return value of `get_db` command. To control the number of characters in the return value of `get_db` use `tcl_return_display_length_limit`.

Example

The following example sets the number of objects and length of the lists to 1 and then calls `get_db` to display instance attributes. `get_db` command outputs one instance object and every attribute list is truncated to one element.

```
set_db get_db_display_limit 1
get_db insts .*
```

```
Object: inst:TOP/I0
```

```
arcs: NC
```

```
area: 0.7056
```

```
base_cell: base_cell:BUFFD0BWP
```

```
bbox: {0.0 0.0 0.0 0.0}
```

```
.....
```

```
pg_base_pins: base_pin:BUFFD0BWP/VDD ... (total length 2) #truncated as length
of the list is set to 1
```

```
pg_pin_nets: net:TOP/VDD ... (total length 2) #truncated as length of the
list is set to 1
```

```
pins: pin:I0/Z ... (total length 2) #truncated as length of the list
is set to 1
```

```
...
```

```
route_halo_top_layer: {}  
test_reset_inst:      1
```

Note: Number of objects exceeds limit. Use `set_db get_db_display_limit <value>` to control the number of objects evaluated by `get_db <object> .* query`.

Related Topics

- [extract_rc_cap_filter_mode](#)
- [extract_rc_coupling_cap_threshold](#)
- [extract_rc_relative_cap_threshold](#)
- [extract_rc_total_cap_threshold](#)

hdl

The root attributes in the `hdl` category are:

- [dp_ungroup_separator](#)
- [hdl_allow_inout_constant_port_connect](#)
- [hdl_allow_instance_name_conflict](#)
- [hdl_allow_positional_connections_for_pg_inst](#)
- [hdl_append_generic_ports](#)
- [hdl_array_naming_style](#)
- [hdl_async_set_reset](#)
- [hdl_auto_async_set_reset](#)
- [hdl_auto_sync_set_reset](#)
- [hdl_bus_wire_naming_style](#)
- [hdl_case_mux_threshold](#)
- [hdl_case_sensitive_instances](#)
- [hdl_convert_one_bit_vector_to_scalar](#)
- [hdl_convert_onebit_vector_wire_to_scalar](#)

- `hdl_create_label_for_unlabeled_generate`
- `hdl_decimal_parameter_name`
- `hdl_delete_transparent_latch`
- `hdl_early_redundant_logic_removal`
- `hdl_enable_proc_name`
- `hdl_enable_real_support`
- `hdl_error_on_blackbox`
- `hdl_error_on_latch`
- `hdl_error_on_logic_abstract`
- `hdl_error_on_negedge`
- `hdl_exclude_parameter_in_cell_search`
- `hdl_ff_keep_explicit_feedback`
- `hdl_ff_keep_feedback`
- `hdl_file_row_column_consider_modules_as_leaf_insts`
- `hdl_flatten_complex_port`
- `hdl_flatten_complex_port_in_bottom_up_flow`
- `hdl_generate_index_style`
- `hdl_generate_separator`
- `hdl_ignore_pragma_names`
- `hdl_index_mux_threshold`
- `hdl_inout_assign`
- `hdl_inout_wand_wor_assign`
- `hdl_instance_array_naming_style`
- `hdl_interface_separator`
- `hdl_keep_first_module_definition`
- `hdl_language`
- `hdl_latch_keep_feedback`

- `hdl_link_from_any_lib`
- `hdl_max_loop_limit`
- `hdl_max_map_to_mux_control_width`
- `hdl_max_memory_address_range`
- `hdl_max_recursion_limit`
- `hdl_module_definition_resolution`
- `hdl_nc_compatible_module_link`
- `hdl_new_inout_assign`
- `hdl_overwrite_command_line_macros`
- `hdl_parameter_module_name`
- `hdl_parameter_naming_style`
- `hdl_preserve_async_set_reset_priority_logic`
- `hdl_preserve_dangling_output_nets`
- `hdl_preserve_supply_nets`
- `hdl_preserve_sync_control_logic`
- `hdl_preserve_sync_set_reset`
- `hdl_preserve_unused_flop`
- `hdl_preserve_unused_latch`
- `hdl_preserve_unused_registers`
- `hdl_primitive_input_multibit`
- `hdl_record_naming_style`
- `hdl_register_array_naming_style`
- `hdl_register_naming_style`
- `hdl_register_record_naming_style`
- `hdl_rename_cdn_flop_pins`
- `hdl_rename_cdn_latch_pins`
- `hdl_report_case_info`

- `hdl_resolve_instance_with_lib_cell`
- `hdl_resolve_parameterized_instance_with_structural_module`
- `hdl_sv_module_wrapper`
- `hdl_sync_set_reset`
- `hdl_track_filename_row_col`
- `hdl_track_module_elab_memory_and_runtime`
- `hdl_unconnected_value`
- `hdl_use_block_prefix`
- `hdl_use_case_generate_prefix`
- `hdl_use_current_dir_before_hdl_search_path`
- `hdl_use_cw_first`
- `hdl_use_default_parameter_values_in_name`
- `hdl_use_for_generate_prefix`
- `hdl_use_if_generate_prefix`
- `hdl_use_port_default_value`
- `hdl_verilog_defines`
- `hdl_vhdl_assign_width_mismatch`
- `hdl_vhdl_case`
- `hdl_vhdl_environment`
- `hdl_vhdl_preferred_architecture`
- `hdl_vhdl_range_opt`
- `hdl_vhdl_read_version`
- `hdl_vhdl_strict_compliance`
- `init_blackbox_for_undefined`
- `init_hdl_search_path`
- `input_assert_one_cold_pragma`
- `input_assert_one_hot_pragma`

- [input_asynchro_reset_blk_pragma](#)
- [input_asynchro_reset_pragma](#)
- [input_case_cover_pragma](#)
- [input_case_decode_pragma](#)
- [input_map_to_mux_pragma](#)
- [input_pragma_keyword](#)
- [input_synchro_enable_blk_pragma](#)
- [input_synchro_enable_pragma](#)
- [input_synchro_reset_blk_pragma](#)
- [input_synchro_reset_pragma](#)
- [proto_hdl](#)
- [script_begin](#)
- [script_end](#)
- [synthesis_off_command](#)
- [synthesis_on_command](#)

To set the value of any root attribute, use the [set_db](#) command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the [get_db](#) command:

```
get_db attribute_name
```

To get the current value of all attributes in the `hdl` category, use the following command:

```
get_db -category hdl
```

dp_ungroup_separator

```
dp_ungroup_separator string
```

Description

Default: :

Data_type: string, read/write

Specifies the user-defined booth encoded setting to input port.

Related Information

- [elaborate_design](#)

hdl_allow_inout_constant_port_connect

```
hdl_allow_inout_constant_port_connect {false | true}
```

Description

Default: false

Data_type: bool, read/write

When set to `false`, issues an error message if an output or inout pin of an instantiated submodule is connected to a constant value.

This attribute is supported only in the RTL flow.

Related Information

- [HDL-Related Attributes](#) section of the *Innovus Synthesis HDL Modeling Guide*

hdl_allow_instance_name_conflict

```
hdl_allow_instance_name_conflict {false | true}
```

Description

Default: false

Data_type: bool, read/write

Controls whether the input Verilog file can have a variable (reg or wire) and an instance with the

same name. By default, this results in an error. Set this attribute to `true` to allow this situation.

Related Information

- [elaborate_design](#)

hdl_allow_positional_connections_for_pg_inst

```
hdl_allow_positional_connections_for_pg_inst {false | true}
```

Description

Default: `false`

Data_type: `bool`, read/write

Specifies whether to allow positional association for instantiation of cell with power and ground pins.

Related Information

- [elaborate_design](#)

hdl_append_generic_ports

```
hdl_append_generic_ports {true | false}
```

Description

Default: `true`

Data_type: `bool`, read/write

When set to `false`, does not append generic port information to the module name. By default, the tool changes the name of the module by appending the interface name and modport name to the original module name.

Example

Consider the following RTL:

```
interface nand_intf (
    input wire [1:0] a,
    input wire [1:0] b,
    output wire [1:0] y );
modport FOO (
    input a, b,
    output y);
endinterface

// Top module
module top;
nand_intf intf_inst ();
test1 t1(
    intf_inst.FOO
);
endmodule

// Sub Module
module test1(nand_intf.FOO intf);
assign intf.y = intf.a & intf.b;
endmodule
```

By default, the generated netlist after elaboration looks like:

```
module test1_intf_nand_intf_FOO(intf_a, intf_b, intf_y);
    input [1:0] intf_a, intf_b;
    output [1:0] intf_y;
    wire [1:0] intf_a, intf_b;
    wire [1:0] intf_y;
    and g1 (intf_y[0], intf_a[0], intf_b[0]);
    and g2 (intf_y[1], intf_a[1], intf_b[1]);
endmodule

module top;
    wire [1:0] intf_inst_a;
    wire [1:0] intf_inst_b;
    wire [1:0] intf_inst_y;
    test1_intf_nand_intf_FOO t1(intf_inst_a, intf_inst_b, intf_inst_y);
endmodule
```

When you set the `hdl_append_generic_ports` to `false` before reading in the RTL file,

the generated netlist after elaboration looks like:

```
module test1(intf_a, intf_b, intf_y);
    input [1:0] intf_a, intf_b;
    output [1:0] intf_y;    wire [1:0] intf_a, intf_b;
    wire [1:0] intf_y;
    and g1 (intf_y[0], intf_a[0], intf_b[0]);
    and g2 (intf_y[1], intf_a[1], intf_b[1]);
endmodule

module top;
    wire [1:0] intf_inst_a;
    wire [1:0] intf_inst_b;
    wire [1:0] intf_inst_y;
    test1 t1(intf_inst_a, intf_inst_b, intf_inst_y);
endmodule
```

Related Information

- [elaborate_design](#)

hdl_array_naming_style

hdl_array_naming_style *string*

Description

Default: %s[%d]

Data_type: string, read/write

Specifies the format used to name individual bits of array variables in the RTL. %s is the variable name and %d is the individual bit. Set this attribute before using the `elaborate_design` command.

This attribute is supported only in the RTL flow.

Related Information

- [elaborate_design](#)
- [Naming Individual Bits of Array and Record Ports and Registers](#)

hdl_async_set_reset

```
hdl_async_set_reset Tcl_list
```

Description

Default: ""

Data_type: string, read/write

Specifies that the tool should implement the listed signals using an asynchronous set and reset pins on a latch if that logic controls an asynchronous assignment.

This attribute is supported only in the RTL flow and does not affect the `synthesize_design` command, which may implement the set and reset logic using latch data pins.

Example

```
set_db hdl_async_set_reset {r1 r2} /
```

The above command has the same effect as using the `async_set_reset` pragma in the RTL. For example:

```
// cadence async_set_reset "r1 r2"
```

Related Information

- [elaborate_design](#)
- [synthesize_design](#)
- RTL example and the corresponding schematic for `hdl_async_set_reset` in the [HDL-Related Attributes](#) section of the *Innovus Synthesis HDL Modeling Guide*

hdl_auto_async_set_reset

```
hdl_auto_async_set_reset {false | true}
```

Description

Default: `false`

Data_type: `bool`, read/write

When set to `true`, specifies that the tool should implement logic using asynchronous set and reset pins on a latch if that logic controls an asynchronous assignment of a constant `0` or a constant `1`.

This attribute is supported only in the RTL flow and does not affect the `synthesize_design` command, which may implement the set and reset logic using latch data pins.

Example

The following command implements the `reset` signal in the RTL using a latch asynchronous reset pin:

```
set_db hdl_auto_async_set_reset true
```

Related Information

- [elaborate_design](#)
- [synthesize_design](#)
- RTL example and the corresponding schematic in the [HDL-Related Attributes](#) section of the *Innovus Synthesis HDL Modeling Guide*

hdl_auto_sync_set_reset

```
hdl_auto_sync_set_reset {false | true}
```

Description

Default: `true`

Data_type: `bool`, `read/write`

When set to `true`, specifies that the tool should implement logic using a synchronous set and reset pins on a flip-flop if that logic controls a synchronous assignment of a constant 0 or a constant 1.

This attribute does not affect the `synthesize_design` command, which may implement the set and reset logic using flip-flop data pins.

This attribute is supported only in the RTL flow.

Example

The following command implements the `reset` signal shown in the RTL using a flip-flop synchronous reset pin:

```
set_db hdl_auto_sync_set_reset true
```

Related Information

- [elaborate_design](#)
- [HDL-Related Attributes](#) section of the *Innovus Synthesis HDL Modeling Guide*

hdl_bus_wire_naming_style

`hdl_bus_wire_naming_style` *string*

Description

Default: `%s[%d]`

Data_type: `string`, `read/write`

Specifies the format used to name individual bits of bus wires. `%s` is the variable name and `%d` is the individual bit. Set this attribute before using the `elaborate_design` command.

Related Information

- [elaborate_design](#)

hdl_case_mux_threshold

hdl_case_mux_threshold *integer*

Description

Default: 2

Data_type: int, read/write

Determines whether a binary multiplexer (encoded select inputs) or a one-hot multiplexer (decoded select inputs) is generated to implement signals assigned within case, if-then-else, and choice statements. If the number of distinct values assigned to the signal is greater than or equal to the specified value, a binary multiplexer is implemented; otherwise, a one-hot multiplexer is implemented.

Related Information

- [elaborate_design](#)

hdl_case_sensitive_instances

hdl_case_sensitive_instances {none | false | true}

Description

Default: none

Data_type: string, read/write

Controls how Verilog instances must be linked to modules. Following the language rules, instances in Verilog files will by default be linked case sensitively to modules in Verilog/ VHDL files, while instances in VHDL files will be linked case-insensitively to modules in Verilog/VHDL files.

This attribute can have the following values:

false	Allows case-insensitive instance name matching for Verilog instances to modules in Verilog/VHDL files.
none	Indicates that the parent language rules govern.
true	Allows only case-sensitive instance name matching for Verilog instances to modules in Verilog/VHDL files during linking.

Related Information

- [elaborate_design](#)

hdl_convert_one_bit_vector_to_scalar

hdl_convert_one_bit_vector_to_scalar {true | false}

Description

Default: false

Data_type: bool, read/write

Specifies whether to convert one-bit vectors present in RTL to scalars when elaborating the design.

Example

Consider the following RTL code:

```
module x(X, A1, A2);
  input [0:0] A1 ;
  input [0:0] A2 ;
  output X;
  and U$1(X, A1, A2);
endmodule
```

If you use the default value (false), the netlist after elaboration will look like:

```
module x(X, A1, A2);  
  input [0:0] A1, A2;  
  output X;  
  wire [0:0] A1, A2;  
  wire X;  
  and U$1 (X, A1, A2);  
endmodule
```

If you set this attribute to `true`, the netlist after elaboration will look like:

```
module x(X, A1, A2);  
  input A1, A2;  
  output X;  
  wire A1, A2;  
  wire X;  
  and U$1 (X, A1, A2);  
endmodule
```

Related Information

- [elaborate_design](#)

hdl_convert_onebit_vector_wire_to_scalar

`hdl_convert_onebit_vector_wire_to_scalar {true | false}`

Description

Default: `false`

Data_type: `bool`, read/write

Converts a one-bit vector declaration into a scalar.

Example

In the following example, `hdl_convert_onebit_vector_wire_to_scalar` affects vector wire `y` in module `mid1`:


```
module mid1(a, b);
  input a;
  output b;
  wire [0:0] y;

  mid2 m3(a, y[0]);
  mid2 m4(y[0], b);
endmodule
```

The following netlist is created post elaboration, by using the default value (`false`) for `hdl_convert_onebit_vector_wire_to_scalar` attribute:

```
module mid1(a, b);
  input a;
  output b;
  wire a;
  wire b;
  wire [0:0] y;
  mid2 m3(a, y);
  mid2 m4(y, b);
endmodule
```

The following netlist is created post elaboration, by setting the value `true` for `hdl_convert_onebit_vector_wire_to_scalar` attribute:

```
module mid1(a, b);
  input a;
  output b;
  wire a;
  wire b;
  wire y;
  mid2 m3(a, y);
  mid2 m4(y, b);
endmodule
```

The single bit vector wire `y` was converted to a scalar.

Related Information

- [elaborate_design](#)

hdl_create_label_for_unlabeled_generate

```
hdl_create_label_for_unlabeled_generate {false | true}
```

Description

Default: `true`

Data_type: `bool`, read/write

Controls whether or not to create a label for an unnamed block. The created label will be of the form `genblkx`.

Related Information

- [elaborate_design](#)

hdl_decimal_parameter_name

```
hdl_decimal_parameter_name {false | true}
```

Description

Default: `false`

Data_type: `bool`, read/write

Controls the format for parameters that are appended to the names of instantiated parameterized modules in the netlist after elaboration. When set to `true`, the tool uses a decimal format for parameters with values less than or equal to the size of a 32-bit integer, prepending the value with width and sign information (if any). Otherwise, the tool uses binary or hex format, as applicable.

Example

Consider the following RTL code:

```
// RTL file test.v
module TOP(q, d);
    output q;
    input  d;
    SUB #(32'b1) u1(q, d);
endmodule
```

```
module SUB(q, d);
    parameter p = 123;
    output q;
    input d;
    assign q = d > p;
endmodule
```

Using the default (`false`) setting of the `hdl_decimal_parameter_name` attribute, the following netlist is printed after elaboration:

```
module TOP(q, d);
    input d;
    output q;
    wire d;
    wire q;
    SUB_p32h00000001 u1(q, d);
endmodule
.....
.....
```

If the `hdl_decimal_parameter_name` attribute is set to `true` before elaboration, the following netlist is printed:

```
module TOP(q, d);
    input d;
    output q;
    wire d;
    wire q;
    SUB_p32d1 u1(q, d);
endmodule
.....
.....
```

Related Information

- [elaborate_design](#)
- [hdl_parameter_naming_style](#)

hdl_delete_transparent_latch

```
hdl_delete_transparent_latch {false | true}
```

Description

Default: `true`

Data_type: `bool`, `read/write`

Controls whether transparent latches are preserved or deleted during elaboration. When set to `true`, deletes latches that are always enabled.

This attribute is supported only in the RTL flow.

Related Information

- [elaborate_design](#)

hdl_early_redundant_logic_removal

`hdl_early_redundant_logic_removal {true| false}`

Description

Default: `true`

Data_type: `bool`, `read/write`

Specifies whether to do an early cleanup of redundant steering logic nodes. An early cleanup of redundant steering logic nodes is performed, by default.

Related Information

- [elaborate_design](#)

hdl_enable_proc_name

`hdl_enable_proc_name {false | true}`

Description

Default: `false`

Data_type: `bool`, read/write

When set to `true`, updates the value of the `hdl_proc_name` instance attribute for sequential elements during elaboration.

Related Information

- [elaborate_design](#)
- `hdl_proc_name` in `inst` in *Stylus Common UI Database Object Information Guide*

hdl_enable_real_support

```
hdl_enable_real_support {true | false}
```

Description

Default: `false`

Data_type: `bool`, read/write

When set to `true`, enables elaboration of constant expressions of real type.

⚠ Real support is disabled by default to ensure that users are extra conscious about any real usage in their designs. Cadence recommends using gate-level simulation to verify the designs when reals are used.

Related Information

- [elaborate_design](#)

hdl_error_on_blackbox

```
hdl_error_on_blackbox {true | false}
```

Description

Default: false

Data_type: bool, read/write

When set to `true`, issues an error message if there is an unresolved reference (black-box) during elaboration.

This attribute is supported only in the RTL flow.

Related Information

- [elaborate_design](#)
- [Global Versus User Control on Elaboration](#) in the *Innovus Synthesis HDL Modeling Guide*

hdl_error_on_latch

```
hdl_error_on_latch {true | false}
```

Description

Default: false

Data_type: bool, read/write

When set to `true`, issues an error message if a latch is inferred for a design.

- This attribute is supported only in the RTL flow.

Related Information

- [elaborate_design](#)
- [Global Versus User Control on Elaboration](#) in the *Innovus Synthesis HDL Modeling Guide*

hdl_error_on_logic_abstract

```
hdl_error_on_logic_abstract {true | false}
```

Description

Default: `false`

Data_type: `bool`, read/write

When set to `true`, issues an error message if a logic abstract is inferred for a design.

- This attribute is supported only in the RTL flow.

Related Information

- [elaborate_design](#)
- [Global Versus User Control on Elaboration](#) in the *Innovus Synthesis HDL Modeling Guide*

hdl_error_on_negedge

```
hdl_error_on_negedge {true | false}
```

Description

Default: `false`

Data_type: `bool`, read/write

When set to `true`, issues an error message if a design infers a flip-flop that is triggered by a falling clock edge.

This attribute is supported only in the RTL flow.

Related Information

- [elaborate_design](#)
- [Global Versus User Control on Elaboration](#) in the *Innovus Synthesis HDL Modeling Guide*
- [HDL-Related Attributes](#) section of the *Innovus Synthesis HDL Modeling Guide*

hdl_exclude_parameter_in_cell_search

`hdl_exclude_parameter_in_cell_search string`

Description

Default:

Data_type: string, read/write

Controls how a parameterized module instantiation is resolved with a liberty complex cell as opposed to an HDL module. Lists the names of the cells for which the search must be done with the name of the module but without the parameter information.

Related Information

- [elaborate_design](#)

hdl_ff_keep_explicit_feedback

`hdl_ff_keep_explicit_feedback {true | false}`

Description

Default: true

Data_type: bool, read/write

Controls how flip-flop stable states are implemented for feedback assignments that are explicitly specified in the RTL.

- This attribute is supported only in the RTL flow.
- This attribute is ignored, and all flip-flop feedback is removed when the `insert_clock_gating` attribute is set to `true`.

Related Information

- [elaborate_design](#)
- `insert_clock_gating` attribute in [power Category Attributes](#)
- [Global Versus User Control on Elaboration](#) in the *Innovus Synthesis HDL Modeling Guide*
- [HDL-Related Attributes](#) section of the *Innovus Synthesis HDL Modeling Guide*

hdl_ff_keep_feedback

```
hdl_ff_keep_feedback {true | false}
```

Description

Default: `false`

Data_type: `bool`, read/write

Controls how flip-flop stable states are implemented. When set to `true`, implements a feedback path from the Q output to the D input. When set to `false`, uses a synchronous flip-flop enable signal to implement the stable states.

- This attribute is supported only in the RTL flow.
- This attribute is ignored, and all flip-flop feedback is removed when the `insert_clock_gating` attribute is set to `true`.

Examples

- The following command implements a feedback path from the Q output to the D input for the RTL:

```
set_attribute hdl_ff_keep_feedback true /
```
- The following command implements a synchronous enable signal from the Q output to the D input for the RTL:

```
set_attribute hdl_ff_keep_feedback false /
```

Related Information

- [elaborate_design](#)
- `insert_clock_gating` attribute in [power Category Attributes](#)
- [Global Versus User Control on Elaboration](#) in the *Innovus Synthesis HDL Modeling Guide*
- [HDL-Related Attributes](#) section of the *Innovus Synthesis HDL Modeling Guide*

hdl_file_row_column_consider_modules_as_leaf_insts

`hdl_file_row_column_consider_modules_as_leaf_insts` *string*

Description

Default:

Data_type: string, read/write

Specifies a list of modules that should be treated as leaf level instances. All the instances within those modules get the same file, row, and column tracking information as the module, assuming it was a leaf level instance.

Related Information

- [elaborate_design](#)

hdl_flatten_complex_port

```
hdl_flatten_complex_port {true | false}
```

Description

Default: `false`

Data_type: `bool`, `read/write`

Controls how complex ports in the RTL are represented in the generated netlist. A port is considered complex, when it is not declared as a simple vector of bits, or as a single bit scalar signal. For example, a port declared using a structure `typedef` is a complex port.

When you set this attribute to `true`, elaboration creates a single lumped (packed) vector port for any complex port. The bits in the single vector port (created with the `true` setting) are set in the order in which the fields/dimensions are ordered in the definition of the complex port. By default, elaboration generates (and names) multiple bit-vector ports, which correspond to various disjoint fields/pieces of the complex port.

Limitation

This attribute has no impact on any complex port that is defined using interface `modport`. The tool will continue to generate (and name) multiple ports for such a complex port.

Related Information

- [elaborate_design](#)

hdl_flatten_complex_port_in_bottom_up_flow

```
hdl_flatten_complex_port_in_bottom_up_flow {true | false}
```

Description

Default: `true`

Data_type: `bool`, `read/write`

Creates one-dimensional packed version of complex ports in the bottom-up flow.

Related Information

- [elaborate_design](#)

hdl_generate_index_style

hdl_generate_index_style *string*

Description

Default: %s[%d]

Data_type: string, read/write

Specifies the format to be used to compose the instance name of instances instantiated inside a `for-generate` statement.

The format string contains the `for-generate` block label (%s) and optionally the individual `for-generate` bit index (%d). The default value of %s[%d] complies with the naming rules defined in Section 12.1.3.1 in Verilog-2001 LRM and in Section 12.4.1 in Verilog-2005 LRM.

In VHDL, a `generate` statement must have a label. In Verilog, the block of a `generate` statement can be unnamed. If a `generate` block is unlabeled, it is given a name based on the rules defined in the Verilog-2005 LRM (in Section 12.4.3 of IEEE Std 1364-2005).

You must specify this attribute before you elaborate the design.

This attribute affects instances in a `for-generate` statement. It does not affect instances in a `if-generate` or `case-generate` statement.

Related Information

- [elaborate_design](#)
- [hdl_generate_separator](#)
- [hdl_use_block_prefix](#)

hdl_generate_separator

hdl_generate_separator *string*

Description

Default: .

Data_type: string, read/write

Specifies the separator string that appears between block labels in the instance name of an instance instantiated inside a `generate` statement. A generated instance name contains multiple block labels if it is inside of nested generate statements.

In Verilog, each layer of the nested `generate` can be a `for-generate`, an `if-generate`, or a `case-generate`. In VHDL, each layer can be either a `for-generate` or an `if-generate`.

You must specify this attribute before you elaborate the design.

Related Information

- [elaborate_design](#)
- [hdl_generate_index_style](#)
- [hdl_use_block_prefix](#)
- [hdl_use_if_generate_prefix](#)

hdl_ignore_pragma_names

hdl_ignore_pragma_names *string*

Description

Default: coverage

Data_type: string, read/write

Specifies a Tcl list of one or more pragma names that must be ignored when reading in Verilog or VHDL.

Related Information

- [elaborate_design](#)
- [Synthesis Pragmas](#) section of the *Innovus Synthesis HDL Modeling Guide*

hdl_index_mux_threshold

hdl_index_mux_threshold *integer*

Description

Default: 0

Data_type: int, read/write

Specifies the minimum number of data inputs the tool requires to implement binary multiplexers instead of AND/OR logic for variable index array references, such as `y = x[i]`.

Related Information

- [elaborate_design](#)

hdl_inout_assign

hdl_inout_assign {false | true}

Description

Default: false

Data_type: bool, read/write

When set to `true`, specifies that the tool must interpret Verilog continuous assign statements where the right-hand-side is undriven as being bidirectional. In this case, the tool matches the behavior of the Conformal[®] Logical Equivalence Checker. To prevent bidirectional assignments, set this attribute to `false`. When set to `false`, the tool matches the behavior of the Verilog Language Reference and the Incisive[®] Simulator.

This attribute does not affect VHDL designs.

Related Information

- [elaborate_design](#)

hdl_inout_wand_wor_assign

```
hdl_inout_wand_wor_assign {false | true}
```

Description

Default: `false`

Data_type: `bool`, `read/write`

Controls the handling of assignments involving signal names declared as `wand` or `wor` net types in System Verilog (SV) designs.

The tool can treat assignments as bidirectional, if the LHS and RHS of the assignment only consists of simple signal references or their constant-indexed part-selects or concatenations. In such assignments, the driver pins or load pins can be on either side of the assignment, since the assignment is treated like a simple connection between the pins.

If connections involve multiple driver pins then the type of the net (as per declaration type `wand/wor/wire`) determines the final value of the net and this value is used to drive all the load pins on the net.

The attribute addresses situations, when two conditions are present:

1. A connection (net) is determined by one/more bidirectional assignments.
2. One or more of the signals involved in the assignments are declared as `wand/wor`.

When the attribute is set to `false` (default), assignments to `wand/wor` declared signals are treated as unidirectional and the driver/loads on the LHS side of the assignment are not considered part of the same net as the driver-loads of the RHS side of the assignment.

When the attribute is set to `true`, assignments to `wand/wor` declared signals are treated as bidirectional and the driver/loads on the LHS side of the assignment are considered part of the same net as the driver-loads of the RHS side of the assignment. As a result, all drivers (from both

sides) are resolved as per wand/wor resolution to determine the value of the net.

Example

Consider the following RTL:

```
module t (input in1, in2, in4, input in3, output out1, out2 );
    wire temp1,temp2,temp3;
    wor  temp1_wor;

    assign temp1 = (in1 == in2);
    assign temp2 = (in3 == in4);
    assign out1 = temp1;
    assign temp1_wor = temp1;  //// assignment to a wor type
    assign temp1_wor = temp2;  //// assignment to a wor type
    assign out2 = ~temp3;
    assign temp3 = temp1_wor & in3;

endmodule
```

By default, the generated netlist after elaboration looks as follows:

```
module t(in1, in2, in4, in3, out1, out2);
    input in1, in2, in4, in3;
    output out1, out2;
    wire in1, in2, in4, in3;
    wire out1, out2;
    wire n_2, temp1_wor, temp3;
    not g1 (out2, temp3);
    and g2 (temp3, temp1_wor, in3);
    or g3 (temp1_wor, out1, n_2);
    xnor g4 (out1, in1, in2);
    xnor g5 (n_2, in3, in4);
endmodule
```

Note: out1 is xnor of in1 and in2. out1 does not depend on in3, in4.

When you set this attribute to `true`, the generated netlist after elaboration looks as shown below:

```
module t(in1, in2, in4, in3, out1, out2);
    input in1, in2, in4, in3;
    output out1, out2;
    wire in1, in2, in4, in3;
    wire out1, out2;
    wire n_1, n_2, temp3;
    not g1 (out2, temp3);
```



```
and g2 (temp3, out1, in3);  
or g3 (out1, n_1, n_2);  
xnor g4 (n_1, in1, in2);  
xnor g5 (n_2, in3, in4);  
endmodule
```

Note: out1 depends on all four inputs: in1, in2, in3 and in4.

Related Information

- [elaborate_design](#)

hdl_instance_array_naming_style

hdl_instance_array_naming_style *string*

Description

Default: %s[%d]

Data_type: [string](#), read/write

Specifies the format used to name individual instance names of an array in the RTL. %s is the array name and %d is the individual bit. Set this attribute before using the [elaborate_design](#) command.

This attribute is supported only in the RTL flow.

Related Information

- [elaborate_design](#)

hdl_interface_separator

hdl_interface_separator *string*

Description

Default: `_`

Data_type: `string`, `read/write`

Specifies the string used to separate the interface elements.

Related Information

- [elaborate_design](#)

hdl_keep_first_module_definition

```
hdl_keep_first_module_definition {false | true}
```

Description

Default: `false`

Data_type: `bool`, `read/write`

Controls whether the first seen or last seen definition for a given module name is used. When encountering multiple definitions for the same module name, the Synthesis engine keeps the last seen definition by default while ignoring other definitions for the same module name. To force the Synthesis engine to retain the first seen (instead of the last seen) definition for a given module name, set the `hdl_keep_first_module_definition` attribute to `true`.

Related Information

- [elaborate_design](#)

hdl_language

```
hdl_language {v2001 | v1995 | vhdl | sv}
```

Description

Default: `sv`

Data_type: `string`, `read/write`

Specifies the default HDL language mode assumed when you run the `read_hdl` command without specifying the language mode.

This attribute is supported only in the RTL flow.

Related Information

- [elaborate_design](#)

hdl_latch_keep_feedback

`hdl_latch_keep_feedback {true | false}`

Description

Default: `false`

Data_type: `bool`, `read/write`

Controls how explicitly-specified latch stable states (for example, `q <= q`) are implemented. When set to `true`, implements a feedback path from the Q output to the D input, resulting in a combinational loop. When set to `false`, implements a latch with an enable signal.

- This attribute is supported only in the RTL flow.
- This attribute is ignored, and all flip-flop feedback is removed when the `insert_clock_gating` attribute is set to `true`.

Related Information

- [elaborate_design](#)
- [Global Versus User Control on Elaboration](#) in the *Innovus Synthesis HDL Modeling Guide*
- [HDL-Related Attributes](#) section of the *Innovus Synthesis HDL Modeling Guide*

hdl_link_from_any_lib

```
hdl_link_from_any_lib {true | false}
```

Description

Default: `true`

Data_type: `bool`, `read/write`

Links to a unique definition from any library. This is useful when a module is instantiated and there is no specification in the instantiation as to the definition to which it should be linked. If there are multiple definitions, the instantiated module is not linked.

Related Information

- [elaborate_design](#)

hdl_max_loop_limit

```
hdl_max_loop_limit integer
```

Description

Default: `4096`

Data_type: `int`, `read/write`

Determines the maximum number of iterations for unfolding a loop construct of any type. The tool stops and produces an error message when it needs to unroll a loop that has more iterations than the specified threshold.

This is a safety measure to avoid infinite unrolling, as well as to avoid loops with an unexpectedly

large number of iterations, which can cause errors in the RTL code. Usually the overflow detection mechanism catches overflow conditions that cause infinite unrolling.

For example, unrolling the following loop causes an infinite loop and is therefore disallowed by the overflow detection mechanism:

```
reg [3:0] i;  
for (i=0; i<16; i=i+1)  
...
```

Being 4-bit wide, the loop index goes back to 0 after reaching 15.

The overflow detection mechanism also catches infinite unrolling that is *not* caused by overflow conditions. For example:

```
parameter incr = 0;  
reg [4:0] i;  
for (i=0; i<16; i=i+incr)  
...
```

This attribute is supported only in the RTL flow.

Related Information

- [elaborate_design](#)

hdl_max_map_to_mux_control_width

hdl_max_map_to_mux_control_width *integer*

Description

Default: 10

Data_type: int, read/write

Specifies the maximum size of multiplexers that can be generated by the tool for HDL `case` statements that are marked with the `map_to_mux` pragma. If the width of the `case` condition, which is implemented as the multiplexer control input, exceeds the value set by this attribute, the tool will

ignore the pragma and generate more efficient logic using AND and OR gates.

Related Information

- [elaborate_design](#)

hdl_max_memory_address_range

`hdl_max_memory_address_range integer`

Description

Default: 4194304

Data_type: int, read/write

Specifies the maximum range of indexed memory access (read or write) that can be elaborated and synthesized.

Related Information

- [elaborate_design](#)

hdl_max_recursion_limit

`hdl_max_recursion_limit integer`

Description

Default: 1024

Data_type: int, read/write

Sets the maximum number of elaborations for recursive instantiations to prevent possible infinite recursions.

This attribute is supported only in the RTL flow.

Related Information

- [elaborate_design](#)

hdl_module_definition_resolution

hdl_module_definition_resolution {last | first | first_dir_netlist | first_dir_rtl}

Description

Default: last

Data_type: string, read/write

Controls which definition of a given module name is used when using the `read_hdl` command with or without the `-netlist` option. When encountering multiple definitions for the same module name, the tool keeps the last seen definition by default while ignoring other definitions for the same module name.

This attribute can have the following values:

<code>first</code>	Uses the first seen definition in case of multiple definitions of the same module.
<code>first_dir_netlist</code>	Uses the first read module definition in case different files exist in different directories containing the same module definition and uses the last read module in case the same directory contains multiple files with the same module definition when using <code>read_hdl -netlist</code> . In case of a conflict, modules read with <code>read_hdl -netlist</code> are given priority over the same modules read with the <code>read_hdl</code> and <code>read_hdl -vhdl</code> commands.
<code>first_dir_rtl</code>	Uses the first read module definition in case different files exist in different directories containing the same module definition and uses the last read module in case the same directory contains multiple files with the same module definition when using <code>read_hdl</code> without the <code>-netlist</code> option.

last	Uses the last seen definition in case of multiple definitions of the same module.
------	---

Related Information

- [elaborate_design](#)

hdl_nc_compatible_module_link

hdl_nc_compatible_module_link {true | false}

Description

Default: true

Data_type: bool, read/write

By default, implements Native Compiler (NC) compatible binding rules for linking module or entity instantiations with corresponding definitions.

To link a module instantiation and choose a definition that should be linked, the following rules describe the order in which a search is applied to find a module to link.

- For a Verilog instance, searches:
 - The entity in the work default library.
 - All libraries if the hdl_nc_compatible_module_link attribute is set to true.
- For a VHDL instance, searches:
 - The specified library to find a module definition if the library name is specified as part of the instantiation, for example:

```
instance_name: entity lib_name.entity_name
```

If the library is not found, the search is not attempted across any other library and a message is issued stating that the specified binding was not found.
 - The library in which the component was compiled.
 - A design unit made visible with a USE clause given to the architecture instantiating the component.

- A design unit made visible with a `USE` clause given to the entity of the architecture instantiating the component.
- A design unit in Genus's default library.
- A design unit made visible with a `LIBRARY` clause given to the architecture instantiating the component.
- A design unit made visible with a `LIBRARY` clause given to the entity of the architecture instantiating the component.

With these rules in place, if more than one match is found for a particular search rule, the search order is as follows:

- For a Verilog instance, searches only for a case-sensitive match.
- For a VHDL instance, searches:
 - A VHDL-specific binding. A matching VHDL module that was read in first will be picked up for linking.
 - A Verilog module whose name and case matches that used in the component declaration.
 - A Verilog module with a matching name that is all lowercase.
 - A Verilog module with a matching name in any case.

When multiple module definitions exist for an instantiation, setting the attribute to `false` allows the tool to search in all libraries for a module definition. The module resolution for the instance may or may not be successful depending on finding the exact match. If there are multiple matches, linking resolution may not happen. When set to `false`, the tool does not follow any particular set of rules when performing linking resolution.

✓ It is recommended to set the attribute to `true`, especially when multiple definitions exist for the same module.

Related Information

- [elaborate_design](#)

hdl_new_inout_assign

```
hdl_new_inout_assign {true| false}
```

Description

Default: true

Data_type: bool, read/write

Controls whether to interpret Verilog continuous assign statements as being bidirectional.

Related Information

- [elaborate_design](#)

hdl_overwrite_command_line_macros

```
hdl_overwrite_command_line_macros {false | true}
```

Description

Default: true

Data_type: bool, read/write

Enables overwriting of the values of macros passed with the `read_hdl -define` command with the values specified in the RTL files. If you set this attribute to `false`, the command-line values are retained throughout unless a particular macro is undefined in some file. You need to set this attribute before you read the RTL files.

Example

Consider the following RTL file:

```
// test1.v
`define MACRO1 4
module test1 (out1);
    output[7:0] out1;
    assign      out1 = `MACRO1;
endmodule // test1
```

Consider the following script:

```
read_hdl -define MACRO1=5 test1.v
```

- When you use the default setting (`true`), the value used for `MACRO1` will be 4. That is, the value in the RTL file (4) overwrites the value of the command line (5).
- When you set the `hdl_overwrite_command_line_macros` to `false`, the value of the command line will be used, and the value of `MACRO1` will be 5.

Related Information

- [elaborate_design](#)

hdl_parameter_module_name

```
hdl_parameter_module_name {true | false}
```

Description

Default: `true`

Data_type: `bool`, read/write

Controls whether to add the parameter values in the parameterized module name for the top-level design or the design specified with the `elaborate` command. By default, the parameter value is added to the design name.

This attribute is supported only in the RTL flow.

Note that the `elaborate` command specified above is only valid within the script being passed to `elaborate_design`. You cannot use it directly at the Innovus prompt.

Related Information

- [elaborate_design](#)

hdl_parameter_naming_style

hdl_parameter_naming_style *string*

Description

Default: `_%s%d`

Data_type: `string`, `read/write`

Specifies the format of the suffix added to the original module name for each parameter overwrite.

Each uniquified parameterized module is named by concatenating the original module name and a number of suffix strings, one string per parameter.

For the top-level module, the attribute applies to the parameters whose values are specified with the `-parameters` option of the `elaborate` command.

For an instantiated module, the attribute is effective if both of the following conditions apply:

- The module name of the child module is not parameterized.
- The parameter value is given by the instantiation statement.

This attribute is useful when you have a design where Verilog parameters or generic ports in the VHDL are causing long module names that exceed another tool's internal limit.

This attribute is supported only in the RTL flow.

Note that the `elaborate` command specified above is only valid within the script being passed to `elaborate_design`. You cannot use it directly at the Innovus prompt.

Related Information

- `elaborate_design`

hdl_preserve_async_set_reset_priority_logic

hdl_preserve_async_set_reset_priority_logic {true | false}

Description

Default: `false`

Data_type: `bool`, read/write

When set to `true`, the tool is prevented from generating redundant logic when, during mapping, a flip-flop cell is selected that has the same priority for the asynchronous set and reset operations as specified in the input HDL.

Related Information

- [elaborate_design](#)

hdl_preserve_dangling_output_nets

```
hdl_preserve_dangling_output_nets {true | false}
```

Description

Default: `true`

Data_type: `bool`, read/write

When set to `true`, the tool preserves the names of dangling nets connected to instance output ports.

This attribute is supported only in the structural flow.

Related Information

- [elaborate_design](#)

hdl_preserve_supply_nets

```
hdl_preserve_supply_nets {true | false}
```

Description

Default: `true`

Data_type: `bool`, `read/write`

When set to `true`, the tool preserves the supply nets in the design that is read in.

Example

The following module has two supply nets in the design.

```
module test_64(Y, Z, A);
    input A;
    output Y, Z;

    supply0 s0;
    supply1 s1;

    assign Z = s1;
    and u1 (Y, s0, A);
endmodule
```

Using the default value for the `hdl_preserve_supply_nets` attribute produces the following post-elaboration netlist:

```
module test_64(Y, Z, A);
    input A;
    output Y, Z;
    wire A;
    wire Y, Z;
    supply0 s0;
    supply1 s1;
    assign Z = s1;
    and u1 (Y, s0, A);
endmodule
```

Setting the value for the `hdl_preserve_supply_nets` attribute to `false` produces the following post-elaboration netlist:

```
module test_64(Y, Z, A);
    input A;
    output Y, Z;
    wire A;
    wire Y, Z;
    assign Z = 1'b1;
```

```
    and u1 (Y, 1'b0, A);  
endmodule
```

Related Information

- [elaborate_design](#)

hdl_preserve_sync_control_logic

```
hdl_preserve_sync_control_logic {true | false}
```

Description

Default: false

Data_type: bool, read/write

When set to `true`, the tool will try to keep the synchronous control logic close to the flip-flops and prevent merging it with surrounding logic in the fanin cone.

You must set this attribute before reading the design so the tool can automatically preserve the synchronous control pins of the flip-flops. Enabling this attribute can impact QoR.

Related Information

- [elaborate_design](#)

hdl_preserve_sync_set_reset

```
hdl_preserve_sync_set_reset {true | false}
```

Description

Default: false

Data_type: bool, read/write

When set to `true`, the tool will try to keep the synchronous set (reset) control logic close to the flip-flops and prevent merging it with surrounding logic in the fanin cone.

You must set this attribute before reading in the design so the tool can automatically preserve the synchronous control pins of the flip-flops. Enabling this attribute can impact QoR.

Related Information

- [elaborate_design](#)

hdl_preserve_unused_flop

```
hdl_preserve_unused_flop {true | false}
```

Description

Default: false

Data_type: bool, read/write

When set to `true`, the tool does not remove flip-flops that do not, directly or indirectly, affect any outputs. This can be used, for example, to keep flops that are only used to observe internal nets through scan chains in test mode.

This attribute only affects flops that are inferred by elaboration. It does not affect flops that are explicitly instantiated in the HDL code.

This attribute is supported only in the RTL flow and ignored in the structural flow.

Example

In the following example, the `hdl_preserve_unused_flop` attribute affects the `x1_3_reg` and `yf_3_reg` instances that are inferred during elaboration. It does not affect the `u3` and `u4` instances that are explicitly instantiated in the HDL code.

```
module test (y, d, clk);
    input clk, d;
    output y;
    wire x1_2, yf_2;
    reg x1_3, yf_3;
    latch_in_lib u3 (.D(d), .G(clk), .Q(x1_2));
    flop_in_lib u4 (.CP(clk), .D(d), .Q(yf_2));
```



```

always @(clk or d)
    begin if (clk)
        xl_3 <= d;
    end
always @(posedge clk)
    begin
        yf_3 <= d;
    end
assign y = ~d;
endmodule

```

Using the default setting (`false`) of the `hdl_preserve_unused_flop` attribute creates the following post-elaboration netlist:

```

module test (y, d, clk);
    input d, clk;
    output y;
    wire d, clk;
    wire y;
    wire xl_2, yf_2;
    latch_in_lib u3(.D (d), .G (clk), .Q (xl_2));
    flop_in_lib u4(.CP (clk), .D (d), .Q (yf_2));
    not g1 (y, d);
endmodule

```

Setting the `hdl_preserve_unused_flop` attribute to `true` creates the following post-elaboration netlist:

```

module test (y, d, clk);
    input d, clk;
    output y;
    wire d, clk;
    wire y;
    wire UNCONNECTED, xl_2, yf_2;
    latch_in_lib u3(.D (d), .G (clk), .Q (xl_2));
    flop_in_lib u4(.CP (clk), .D (d), .Q (yf_2));
    not g2 (y, d);
    CDN_flop yf_3_reg(.clk (clk), .d (d), .sena (1'b1), .aclr (1'b0),
        .apre (1'b0), .srl (1'b0), .srd (1'b0), .q (UNCONNECTED));
endmodule
`ifdef RC_CDN_GENERIC_GATE
`else
module CDN_flop(clk, d, sena, aclr, apre, srl, srd, q);
    input clk, d, sena, aclr, apre, srl, srd;

```

```
output q;
wire clk, d, sena, aclr, apre, srl, srd;
wire q;
reg qi;
assign #1 q = qi;
always
    @(posedge clk or posedge apre or posedge aclr)
        if (aclr)
            qi <= 0;
        else if (apre)
            qi <= 1;
        else if (srl)
            qi <= srd;
        else begin
            if (sena)
                qi <= d;
        end

initial
    qi <= 1'b0;
endmodule
`endif
```

Related Information

- [elaborate_design](#)
- [hdl_preserve_unused_registers](#)

hdl_preserve_unused_latch

```
hdl_preserve_unused_latch {true | false}
```

Description

Default: false

Data_type: bool, read/write

When set to `true`, the tool does not remove latches that do not, directly or indirectly, affect any output. This can be used, for example, to keep registers that are only used to observe internal nets

through scan chains in test mode.

This attribute affects only latches that are inferred by elaboration. It does not affect latches that are explicitly instantiated in the HDL code.

This attribute is supported only in the RTL flow and ignored in the structural flow.

Example

In the following example, the `hdl_preserve_unused_latch` attribute affects the `x1_3_reg` and `yf_3_reg` instances that are inferred during elaboration. It does not affect the `u3` and `u4` instances that are explicitly instantiated in the HDL code.

```
module test (y, d, clk);
    input clk, d;
    output y;
    wire x1_2, yf_2;
    reg x1_3, yf_3;
    latch_in_lib u3 (.D(d), .G(clk), .Q(x1_2));
    flop_in_lib u4 (.CP(clk), .D(d), .Q(yf_2));
    always @(clk or d)
        begin if (clk)
            x1_3 <= d;
        end
    always @(posedge clk)
        begin
            yf_3 <= d;
        end
    assign y = ~d;
endmodule
```

Using the default setting (`false`) of the `hdl_preserve_unused_latch` attribute creates the following post-elaboration netlist:

```
module test (y, d, clk);
    input d, clk;
    output y;
    wire d, clk;
    wire y;
    wire x1_2, yf_2;
    latch_in_lib u3(.D (d), .G (clk), .Q (x1_2));
    flop_in_lib u4(.CP (clk), .D (d), .Q (yf_2));
    not g1 (y, d);
```

```
endmodule
```

Setting the `hdl_preserve_unused_latch` **attribute to** `true`, **creates the following post-elaboration netlist:**

```
module test (y, d, clk);
    input d, clk;
    output y;
    wire d, clk;
    wire y;
    wire UNCONNECTED, xl_2, yf_2;
    latch_in_lib u3(.D (d), .G (clk), .Q (xl_2));
    flop_in_lib u4(.CP (clk), .D (d), .Q (yf_2));
    not g2 (y, d);
    CDN_latch xl_3_reg(.d (d), .ena (clk), .aclr (1'b0), .apre (1'b0),
        .q (UNCONNECTED));
endmodule

`ifdef RC_CDN_GENERIC_GATE
`else
module CDN_latch(ena, d, aclr, apre, q);
    input ena, d, aclr, apre;
    output q;
    wire ena, d, aclr, apre;
    wire q;
    reg qi;
    assign #1 q = qi;
    always
        @(d or ena or apre or aclr)
            if (aclr)
                qi <= 0;
            else if (apre)
                qi <= 1;
            else begin
                if (ena)
                    qi <= d;
            end

    initial
        qi <= 1'b0;
endmodule
`endif
```

Related Information

- [elaborate_design](#)
- [hdl_preserve_unused_registers](#)

hdl_preserve_unused_registers

hdl_preserve_unused_registers {true | false}

Description

Default: false

Data_type: bool, read/write

When set to `true`, Genus does not remove registers (latches and flip-flops) that do not, directly or indirectly, affect any outputs. This can be used, for example, to keep registers that are only used to observe internal nets through scan chains in test mode.

This attribute only affects registers that are inferred by elaboration. It does not affect registers that are explicitly instantiated in the HDL code.

This attribute is supported only in the RTL flow and ignored in the structural flow.

Example

In the following example, the `hdl_preserve_unused_registers` attribute affects the `x1_3_reg` and `yf_3_reg` instances that are inferred during elaboration. It does not affect the `u3` and `u4` instances that are explicitly instantiated in the HDL code.

```
module test (y, d, clk);
    input clk, d;
    output y;
    wire x1_2, yf_2;
    reg x1_3, yf_3;
    latch_in_lib u3 (.D(d), .G(clk), .Q(x1_2));
    flop_in_lib u4 (.CP(clk), .D(d), .Q(yf_2));
    always @(clk or d)
        begin if (clk)
            x1_3 <= d;
```

```

        end
    always @(posedge clk)
        begin
            yf_3 <= d;
        end
    assign y = ~d;
endmodule

```

Using the default setting (`false`) of the `hdl_preserve_unused_registers` attribute creates the following post-elaboration netlist:

```

module test (y, d, clk);
    input d, clk;
    output y;
    wire d, clk;
    wire y;
    wire x1_2, yf_2;
    latch_in_lib u3(.D (d), .G (clk), .Q (x1_2));
    flop_in_lib u4(.CP (clk), .D (d), .Q (yf_2));
    not g1 (y, d);
endmodule

```

Setting the `hdl_preserve_unused_registers` attribute to `true` creates the following post-elaboration netlist:

```

module test (y, d, clk);
    input d, clk;
    output y;
    module test (y, d, clk);
        input d, clk;
        output y;
        wire d, clk;
        wire y;
        wire UNCONNECTED, UNCONNECTED0, x1_2, yf_2;
        latch_in_lib u3(.D (d), .G (clk), .Q (x1_2));
        flop_in_lib u4(.CP (clk), .D (d), .Q (yf_2));
        not g3 (y, d);
        CDN_latch x1_3_reg(.d (d), .ena (clk), .aclr (1'b0), .apre (1'b0),
            .q (UNCONNECTED));
        CDN_flop yf_3_reg(.clk (clk), .d (d), .sena (1'b1), .aclr (1'b0),
            .apre (1'b0), .srl (1'b0), .srd (1'b0), .q (UNCONNECTED0));
    endmodule
`ifdef RC_CDN_GENERIC_GATE
`else

```

```
module CDN_latch(ena, d, aclr, apre, q);
    input ena, d, aclr, apre;
    output q;
    wire ena, d, aclr, apre;
    wire q;
    reg qi;
    assign #1 q = qi;
    always
        @(d or ena or apre or aclr)
            if (aclr)
                qi <= 0;
            else if (apre)
                qi <= 1;
            else begin
                if (ena)
                    qi <= d;
                end

    initial
        qi <= 1'b0;
endmodule

`endif

`ifndef RC_CDN_GENERIC_GATE
`else
module CDN_flop(clk, d, sena, aclr, apre, srl, srd, q);
    input clk, d, sena, aclr, apre, srl, srd;
    output q;
    wire clk, d, sena, aclr, apre, srl, srd;
    wire q;
    reg qi;
    assign #1 q = qi;
    always
        @(posedge clk or posedge apre or posedge aclr)
            if (aclr)
                qi <= 0;
            else if (apre)
                qi <= 1;
            else if (srl)
                qi <= srd;
            else begin
                if (sena)
                    qi <= d;
                end

    initial
```

```
    qi <= 1'b0;  
endmodule  
`endif
```

Related Information

- [elaborate_design](#)

hdl_primitive_input_multibit

```
hdl_primitive_input_multibit {false | true}
```

Description

Default: `false`

Data_type: `bool`, `read/write`

Controls whether a Verilog primitive gate instantiated in RTL can have a multibit output pin connected to the primitive. For example, gates like `NOT`, `bufif0`, `bufif1`, `notif0`, `notif1`, and `tran` expect a single bit output. However, sometimes multibit outputs are connected in the RTL to these primitives.

Set this attribute to `true` to allow the LSB bit of a multibit output to be connected to the input of these primitives. By default, RTL with multibit signals connected to the input would not be allowed.

Related Information

- [elaborate_design](#)

hdl_record_naming_style

```
hdl_record_naming_style string
```

Description

Default: `%s[%s]`

Data_type: string, read/write

Specifies the format used to name individual bits of record variables in the RTL. The first %s the variable name and the second %s is the field name. Set this attribute before using the `elaborate_design` command.

This attribute is supported only in the RTL flow.

Related Information

- [elaborate_design](#)
- [Naming Individual Bits of Array and Record Ports and Registers](#)

hdl_register_array_naming_style

`hdl_register_array_naming_style` *string*

Description

Default:

Data_type: string, read/write

Specifies the format for naming array type registers.

Related Information

- [elaborate_design](#)

hdl_register_naming_style

`hdl_register_naming_style` *string*

Description

Default: %s_reg%s

Data_type: string, read/write

Specifies the format used to name flip-flop and latch instances inferred from scalar and vector variables in the RTL. The first %s is the variable name. If the variable is a vector, the second %s is the individual bit of the vector as specified by the `hdl_array_naming_style` attribute.

This attribute is supported only in the RTL flow.

Related Information

- [elaborate_design](#)
- [hdl_array_naming_style](#)
- [Naming Individual Bits of Array and Record Ports and Registers](#)

hdl_register_record_naming_style

`hdl_register_record_naming_style` *string*

Description

Default:

Data_type: string, read/write

Specifies the format for naming record type registers.

Related Information

- [elaborate_design](#)

hdl_rename_cdn_flop_pins

`hdl_rename_cdn_flop_pins` *list_of_pin_mappings*

Description

Default: ""

Data_type: string, read/write

Renames the specified pins of inferred CDN_flop components. Specify a list of pin mappings, where each mapping renames the given pin.

After elaboration and prior to mapping, the tool uses CDN_flop components to represent the flip-flops. The default CDN_flop pin names are:

Pin Name	Pin Function
aclr	asynchronous clear
apre	asynchronous preset
clk	clock
d	data input
q	output
sena	synchronous data enable
srd	synchronous set/reset data
srl	synchronous set/reset load

When different CDN_flop pin names are required (for example, an SDC constraints file may use different pin names), use this attribute to rename the CDN_flop pins.

Related Information

- [elaborate_design](#)
- [Performing Elaboration](#)

hdl_rename_cdn_latch_pins

`hdl_rename_cdn_latch_pins list_of_pin_mappings`

Description

Default: ""

Data_type: string, read/write

Renames the specified pins of inferred CDN_latch components. Specify a list of pin mappings, where each mapping renames the given pin.

After elaboration and prior to mapping, Genus uses CDN_latch components to represent the latches.

When different CDN_latch pin names are required (for example, an SDC constraints file may use different pin names), use this attribute to rename the CDN_latch pins.

Related Information

- [elaborate_design](#)
- [Performing Elaboration](#)

hdl_report_case_info

```
hdl_report_case_info {true | false}
```

Description

Default: false

Data_type: bool, read/write

When set to `true`, reports information about how the `elaborate_design` command infers `full_case` and `parallel_case` for every "multi-way decision statement" (like a case statement). The reporting is module-by-module, with one report for each module. Since `full_case` and `parallel_case` pragmas are not yet supported in the tool for VHDL case statements, no information is printed for VHDL modules (entities/architectures).

The reports can help users understand whether the tool:

- Honored the `full_case` or `parallel_case` pragma
- Automatically inferred a full or parallel case

Related Information

- [elaborate_design](#)

hdl_resolve_instance_with_lib_cell

```
hdl_resolve_instance_with_lib_cell {true | false}
```

Description

Default: false

Data_type: bool, read/write

When set to `true`, the tool binds the instances of design `M` to component `M` from the technology library. If there is no `M` component in the technology library, the tool binds the instances of `M` to a user-defined module `M`, provided this module exists. If there is no `M` user-defined module, the instances of `M` are assumed to be unresolved references (blackboxes).

This attribute is supported in the RTL flow and the structural flow.

If you want to bind an instance of design `M` to component `M` from the technology library even when a parameterized user module with name `M` is present, and you are using the RTL flow, you must also set the root attribute `hdl_resolve_parameterized_instance_with_structural_module` to `true`.

Related Information

- [elaborate_design](#)
- `hdl_resolve_parameterized_instance_with_structural_module` in [hdl Category Attributes](#)
- [Logic Abstract Modeling](#) in the *Innovus Synthesis HDL Modeling Guide*

hdl_resolve_parameterized_instance_with_structural_module

```
hdl_resolve_parameterized_instance_with_structural_module {true | false}
```

Description

Default: false

Data_type: bool, read/write attribute

Controls whether module linking uses the parameterized module name. During elaboration, the tool

tries to identify a child module for every instantiation statement in the RTL code.

By default, the tool does not look for a parameterized module name to resolve the instantiation, but instead looks for a module with the exact same name as given in the instantiation statement.

If you set this attribute to `true`, the tool looks for the parameterized module names during module linking.

- If the instantiation statement provides the parameter name(s), the child module name is parameterized using the `_%s%d` style as illustrated in the first example.
- If the instantiation statement does not provide a parameter name, the child module name is parameterized using the `_%d` style as illustrated in the second example.
- Child modules with a parameter in their body are not considered during linking as illustrated in the third example.

At the module-linking step during elaboration, the `hdl_parameter_naming_style` attribute setting does not affect how the instantiated module name is parameterized.

This attribute is supported only in the RTL flow.

Examples

- The following RTL code of the top-level module has an instantiation statement that contains the parameter name `p` and references the parameter value `16`:

```
module top (y, a, b);  
    parameter w = 16;  
    input [w-1:0] a, b;  
    output [w-1:0] y;  
    btm #(.p(w)) ul (y, a, b);  
endmodule
```

If the attribute is set to `false`, the tool links this instance to a Verilog module (or a VHDL entity) whose name is `btm`. If set to `true`, the tool links the instance to a module or entity whose name is `btm_p16`. The parameterized module name includes both the parameter name `p` and the parameter value `16`.

The linked child module might be similar to:

```
module btm_p16 (z, c, d);  
    input [15:0] c, d;
```

```
    output [15:0] z;
    ...
endmodule
```

- The following RTL code of the top-level module has an instantiation statement that references a parameter value (16) but has no parameter name:

```
module top (y, a, b);
    parameter w = 16;
    input [w-1:0] a, b;
    output [w-1:0] y;
    btm #(w) u1 (y, a, b);
endmodule
```

If the attribute is set to `false`, the tool links this instance of instantiation to a Verilog module (or a VHDL entity) whose name is `btm`. If set to `true`, the tool links the instance to a module or entity whose name is `btm_16`. This parameterized module name does not include the parameter name, because the tool cannot derive that from the instantiation statement. The linked child module might be similar to:

```
module btm_16 (z, c, d);
    input [15:0] c, d;
    output [15:0] z;
    ...
endmodule
```

- The following RTL code of the top-level module has an instantiation statement that references a parameter value (16) and has no parameter name. Even when the attribute is set to `true`, the tool will not link instance `u0` to module `btm_16` because module `btm_16` has a parameter statement:

```
module top (...);
    btm #(16) u0 (...);
    ....
endmodule

module btm_16 (...);
    parameter p = 32;
    ...
endmodule
```

Related Information

- [elaborate_design](#)

hdl_sv_module_wrapper

hdl_sv_module_wrapper {true | false}

Description

Default: false

Data_type: bool, read/write

Enables the generation of a System Verilog module wrapper that connects the original I/O interfaces to the port names in the Verilog netlist.

Set this attribute to `true` before you elaborate the design to generate the module wrapper with the definitions of the complex types used.

This attribute applies only to RTL using System Verilog interfaces. When interfaces are used, the I/O ports written in the Verilog netlist will by default not match the test bench.

Related Information

- [elaborate_design](#)
- [Performing Elaboration](#)

hdl_sync_set_reset

hdl_sync_set_reset Tcl_list

Description

Default:

Data_type: string, read/write

Specifies that the tool should implement the listed signals using a synchronous set and reset pins on a flip-flop if that logic controls a synchronous assignment.

- This attribute is supported only in the RTL flow.
- This attribute does not affect the `syn_map` command, which may implement the set and reset logic using flip-flop data pins.

Example

```
set_db / .hdl_sync_set_reset {r1 r2}
```

This command has the same effect as using the `sync_set_reset` pragma in the RTL:

```
// cadence sync_set_reset "r1 r2"
```

Note that the `syn_map` command specified above is only valid within the script being passed to `synthesize_design`. You cannot use it directly at the Innovus prompt.

Related Information

- [elaborate_design](#)
- [HDL-Related Attributes](#) section of the *Innovus Synthesis HDL Modeling Guide*

hdl_track_filename_row_col

```
hdl_track_filename_row_col {true | false}
```

Description

Default: false

Data_type: bool, read/write

Enables or disables file, row, and column information tracking. By default, all the file, row, and column information is deleted. Set this attribute to `true` before using the `elaborate_design` command to enable file, row, column information tracking.

This attribute enables the tool to keep track of filenames, line numbers, and column numbers for all instances before optimization and use this information in selected reports and messages.

Enabling this attribute can have an impact on the runtime.

This attribute is supported in the RTL flow and the structural flow.

Related Information

- `elaborate_design`
- `hdl_file_row_column_consider_modules_as_leaf_insts`

hdl_track_module_elab_memory_and_runtime

```
hdl_track_module_elab_memory_and_runtime {true | false}
```

Description

Default: false

Data_type: bool, read/write

Controls whether the tool tracks the memory used (in MegaBytes) and run time taken (cpu-time in seconds) during elaboration of each module.

If you set this attribute to `true`, the run time and memory usage numbers are printed (to standard output) for each module during elaboration. The numbers printed for each module indicate the time and memory needed to elaborate that module, but the numbers do not include the time and memory taken to elaborate other modules instantiated by the module.

Set this attribute to `true` for designs that have a long elaboration time to identify the modules that are impacting performance.

This attribute is supported only in the RTL flow. The attribute will only print out the numbers for modules, which are read and elaborated by the `read_hdl` command (the high level RTL flow) and will have no effect for modules which are read by the `read_netlist` command (the structural elaboration flow).

Note that the `read_hdl` and `read_netlist` commands specified above are only valid within the script being passed to `elaborate_design`. You cannot use them directly at the Innovus prompt.

Related Information

- `elaborate_design`

hdl_unconnected_value

`hdl_unconnected_value {0 | 1 | X | none}`

Description

Default: 0

Data_type: string, read/write

Connects any undriven signal (undriven input in module or cell instantiation, undriven output or undriven signal) in a module to the specified value during generic synthesis.

If you set the attribute value to `none`, then an undriven signal remains undriven. If you specify the `x` value, then an undriven signal is driven by an `x` dont-care value. The specified value is case-insensitive. You can specify `x`, `X`, `none`, `None`, or `NONE`.

Examples

Example 1: In the following example, the `wc` signal is undriven:

```
module test (y, a, b);
    input [3:0] a, b;
    output [3:0] y;
    wire [3:0] wa, wb, wc;
    assign wa = a;
    assign wb = b;
    assign y = wa & wb & wc;
endmodule
```

- If you set the attribute to 0, the netlist after generic synthesis will look as follows:

```
module test(y, a, b);
    input [3:0] a, b;
    output [3:0] y;
    wire [3:0] a, b;
    wire [3:0] y;
    assign y[0] = 1'b0;
    assign y[1] = 1'b0;
    assign y[2] = 1'b0;
    assign y[3] = 1'b0;
endmodule
```

- If you set the attribute to 1, the netlist after generic synthesis will look as follows:

```
module test(y, a, b);
    input [3:0] a, b;
    output [3:0] y;
    wire [3:0] a, b;
    wire [3:0] y;
    and g18 (y[1], a[1], b[1]);
    and g19 (y[0], a[0], b[0]);
    and g20 (y[2], a[2], b[2]);
    and g21 (y[3], a[3], b[3]);
endmodule
```

- If you set the attribute to `none`, the netlist after generic synthesis will look like:

```
module test(y, a, b);
```

```
input [3:0] a, b;
output [3:0] y;
wire [3:0] a, b;
wire [3:0] y;
wire [3:0] wc;
wire n_19, n_22, n_25, n_28;
nand g29 (n_19, a[1], b[1]);
nand g30 (n_22, a[0], b[0]);
nand g31 (n_28, a[3], b[3]);
nand g32 (n_25, a[2], b[2]);
and g33 (y[1], wc0, wc[1]);
not gc (wc0, n_19);
and g34 (y[2], wc1, wc[2]);
not gc0 (wc1, n_25);
and g35 (y[0], wc2, wc[0]);
not gc1 (wc2, n_22);
and g36 (y[3], wc3, wc[3]);
not gc2 (wc3, n_28);
endmodule
```

- If you set the attribute to `x`, the netlist after generic synthesis will look like:

```
module test(y, a, b);
    input [3:0] a, b;
    output [3:0] y;
    wire [3:0] a, b;
    wire [3:0] y;
    assign y[0] = 1'b0;
    assign y[1] = 1'b0;
    assign y[2] = 1'b0;
    assign y[3] = 1'b0;
endmodule
```

Example 2: In the following example, the `c` input port of the `and3` sub-module is unconnected in all three instantiation statements:

```
module and3 (y, a, b, c);
    input [7:0] a, b, c;
```

```
    output [7:0] y;
    assign y = a & b & c;
endmodule
module test (x, y, z, a, b);
    input [7:0] a, b;
    output [7:0] x, y, z;
    and3 u1 (.y(x), .a(a), .b(b), .c());
    and3 u2 (.y(y), .a(a), .b(b));
    and3 u3 (z, a, b);
endmodule
```

- If you specify 0, the netlist after generic synthesis will look like:

```
module test(x, y, z, a, b);
    input [7:0] a, b;
    output [7:0] x, y, z;
    wire [7:0] a, b;
    wire [7:0] x, y, z;
    assign z[0] = 1'b0;
    assign z[1] = 1'b0;
    assign z[2] = 1'b0;
    assign z[3] = 1'b0;
    assign z[4] = 1'b0;
    assign z[5] = 1'b0;
    assign z[6] = 1'b0;
    assign z[7] = 1'b0;
    assign y[0] = 1'b0;
    assign y[1] = 1'b0;
    assign y[2] = 1'b0;
    assign y[3] = 1'b0;
    assign y[4] = 1'b0;
    assign y[5] = 1'b0;
    assign y[6] = 1'b0;
    assign y[7] = 1'b0;
    assign x[0] = 1'b0;
    assign x[1] = 1'b0;
    assign x[2] = 1'b0;
```

```
    assign x[3] = 1'b0;
    assign x[4] = 1'b0;
    assign x[5] = 1'b0;
    assign x[6] = 1'b0;
    assign x[7] = 1'b0;
endmodule
```

- If you specify 1, the netlist after generic synthesis will look like:

```
module and_op(A, B, Z);
    input [7:0] A, B;
    output [7:0] Z;
    wire [7:0] A, B;
    wire [7:0] Z;
    and g25 (Z[7], A[7], B[7]);
    and g26 (Z[6], A[6], B[6]);
    and g27 (Z[5], A[5], B[5]);
    and g28 (Z[0], A[0], B[0]);
    and g29 (Z[3], A[3], B[3]);
    and g30 (Z[2], A[2], B[2]);
    and g31 (Z[1], A[1], B[1]);
    and g32 (Z[4], A[4], B[4]);
endmodule

module and3(y, a, b, c);
    input [7:0] a, b, c;
    output [7:0] y;
    wire [7:0] a, b, c;
    wire [7:0] y;
    and_op g1(.A (a), .B (b), .Z (y));
endmodule

module test(x, y, z, a, b);
    input [7:0] a, b;
    output [7:0] x, y, z;
    wire [7:0] a, b;
    wire [7:0] x, y, z;
```

```
    assign z[0] = x[0];
    assign z[1] = x[1];
    assign z[2] = x[2];
    assign z[3] = x[3];
    assign z[4] = x[4];
    assign z[5] = x[5];
    assign z[6] = x[6];
    assign z[7] = x[7];
    assign y[0] = x[0];
    assign y[1] = x[1];
    assign y[2] = x[2];
    assign y[3] = x[3];
    assign y[4] = x[4];
    assign y[5] = x[5];
    assign y[6] = x[6];
    assign y[7] = x[7];

    and3 ul(.y (x), .a (a), .b (b), .c (8'b11111111));
endmodule
```

- If you specify `none`, the netlist after generic synthesis will look like:

```
module ....
    .....
endmodule

module test(x, y, z, a, b);
    input [7:0] a, b;
    output [7:0] x, y, z;
    wire [7:0] a, b;
    wire [7:0] x, y, z;
    wire UNCONNECTED, UNCONNECTED0, UNCONNECTED1, UNCONNECTED2, UNCONNECTED3,
        UNCONNECTED4, UNCONNECTED5, UNCONNECTED6;
    wire UNCONNECTED7, UNCONNECTED8, UNCONNECTED9, UNCONNECTED10,
        UNCONNECTED11, UNCONNECTED12, UNCONNECTED13, UNCONNECTED
    wire UNCONNECTED15, UNCONNECTED16, UNCONNECTED17, UNCONNECTED18,
        UNCONNECTED19, UNCONNECTED20, UNCONNECTED21, UNCONNECTED22;
```



```
and3 u1(.y (x), .a (a), .b (b), .c ({UNCONNECTED6, UNCONNECTED5,
    UNCONNECTED4, UNCONNECTED3, UNCONNECTED2, UNCONNECTED1, UNCONNECTED0,
    UNCONNECTED}));
and3_1 u2(.y (y), .a (a), .b (b), .c ({UNCONNECTED14, UNCONNECTED13,
    UNCONNECTED12, UNCONNECTED11, UNCONNECTED10, UNCONNECTED9, UNCONNECTED8,
    UNCONNECTED7}));
and3_2 u3(z, a, b, {UNCONNECTED22, UNCONNECTED21, UNCONNECTED20,
    UNCONNECTED19, UNCONNECTED18, UNCONNECTED17, UNCONNECTED16, UNCONNECTED15});
endmodule
```

- If you specify `x`, the netlist after generic synthesis will look like:

```
module test(x, y, z, a, b);
    input [7:0] a, b;
    output [7:0] x, y, z;
    wire [7:0] a, b;
    wire [7:0] x, y, z;
    assign z[0] = 1'b0;
    assign z[1] = 1'b0;
    assign z[2] = 1'b0;
    assign z[3] = 1'b0;
    assign z[4] = 1'b0;
    assign z[5] = 1'b0;
    assign z[6] = 1'b0;
    assign z[7] = 1'b0;
    assign y[0] = 1'b0;
    assign y[1] = 1'b0;
    assign y[2] = 1'b0;
    assign y[3] = 1'b0;
    assign y[4] = 1'b0;
    assign y[5] = 1'b0;
    assign y[6] = 1'b0;
    assign y[7] = 1'b0;
    assign x[0] = 1'b0;
    assign x[1] = 1'b0;
```

```
    assign x[2] = 1'b0;
    assign x[3] = 1'b0;
    assign x[4] = 1'b0;
    assign x[5] = 1'b0;
    assign x[6] = 1'b0;
    assign x[7] = 1'b0;
endmodule
```

Example 3: In the following example, the x output port is undriven:

```
module test (x, y, a, b);
    input [3:0] a, b;
    output [3:0] x, y;
    assign y = a & b;
endmodule
```

- If you specify `none`, the netlist after generic synthesis will look as follows:

```
module and_op(A, B, Z);
    input [3:0] A, B;
    output [3:0] Z;
    wire [3:0] A, B;
    wire [3:0] Z;
    and g13 (Z[3], A[3], B[3]);
    and g14 (Z[0], A[0], B[0]);
    and g15 (Z[1], A[1], B[1]);
    and g16 (Z[2], A[2], B[2]);
endmodule

module test(x, y, a, b);
    input [3:0] a, b;
    output [3:0] x, y;
    wire [3:0] a, b;
    wire [3:0] x, y;
    and_op g1(.A (a), .B (b), .Z (y));
endmodule
```

- If you specify `0`, the netlist after generic synthesis will look like:

```
module and_op(A, B, Z);
    input [3:0] A, B;
```

```
    output [3:0] Z;
    wire [3:0] A, B;
    wire [3:0] Z;
    and g13 (Z[3], A[3], B[3]);
    and g14 (Z[0], A[0], B[0]);
    and g15 (Z[1], A[1], B[1]);
    and g16 (Z[2], A[2], B[2]);
endmodule

module test(x, y, a, b);
    input [3:0] a, b;
    output [3:0] x, y;
    wire [3:0] a, b;
    wire [3:0] x, y;
    assign x[0] = 1'b0;
    assign x[1] = 1'b0;
    assign x[2] = 1'b0;
    assign x[3] = 1'b0;
    and_op g1(.A (a), .B (b), .Z (y));
endmodule
```

- If you specify 1, the netlist after generic synthesis will look like:

```
module and_op(A, B, Z);
    input [3:0] A, B;
    output [3:0] Z;
    wire [3:0] A, B;
    wire [3:0] Z;
    and g13 (Z[3], A[3], B[3]);
    and g14 (Z[0], A[0], B[0]);
    and g15 (Z[1], A[1], B[1]);
    and g16 (Z[2], A[2], B[2]);
endmodule

module test(x, y, a, b);
    input [3:0] a, b;
    output [3:0] x, y;
```

```
wire [3:0] a, b;
wire [3:0] x, y;
assign x[0] = 1'b1;
assign x[1] = 1'b1;
assign x[2] = 1'b1;
assign x[3] = 1'b1;
and_op g1(.A (a), .B (b), .Z (y));
endmodule
```

- If you specify x, the netlist after generic synthesis will look like:

```
module and_op(A, B, Z);
    input [3:0] A, B;
    output [3:0] Z;
    wire [3:0] A, B;
    wire [3:0] Z;
    and g13 (Z[3], A[3], B[3]);
    and g14 (Z[0], A[0], B[0]);
    and g15 (Z[1], A[1], B[1]);
    and g16 (Z[2], A[2], B[2]);
endmodule

module test(x, y, a, b);
    input [3:0] a, b;
    output [3:0] x, y;
    wire [3:0] a, b;
    wire [3:0] x, y;
    assign x[0] = 1'b0;
    assign x[1] = 1'b0;
    assign x[2] = 1'b0;
    assign x[3] = 1'b0;
    and_op g1(.A (a), .B (b), .Z (y));
endmodule
```

Related Information

- `elaborate_design`
- `synthesize_design`

hdl_use_block_prefix

`hdl_use_block_prefix {true | false}`

Description

Default: `true`

Data_type: `bool`, read/write

Controls whether block statement labels should be used as a prefix for the instance names specified in the input RTL description.

Related Information

- `elaborate_design`
- `hdl_generate_index_style`
- `hdl_generate_separator`
- `hdl_use_if_generate_prefix`

hdl_use_case_generate_prefix

`hdl_use_case_generate_prefix {true | false}`

Description

Default: `true`

Data_type: `bool`, read/write

Specifies whether 'case' generate labels should be used to prefix instances.

Related Information

- [elaborate_design](#)

hdl_use_current_dir_before_hdl_search_path

```
hdl_use_current_dir_before_hdl_search_path {true | false}
```

Description

Default: `false`

Data_type: `bool`, read/write

Controls the order of the directories from where the include files must be read. By default, the tool uses the directories specified with the `init_hdl_search_path` attribute to search for include files. When the `hdl_use_current_dir_before_hdl_search_path` attribute is set to `true`, the tool uses the directory of the source file (containing the include file) before the HDL search path.

Related Information

- [elaborate_design](#)

hdl_use_cw_first

```
hdl_use_cw_first {true | false}
```

Description

Default: `false`

Data_type: `bool`, read/write

When set to `true`, uses ChipWare components to instantiate the functionality in the design, instead of user-defined modules (Verilog) or entities (VHDL). This attribute is applicable in the cases in which the RTL code has a user-defined module with the identical name of a ChipWare component.

Related Information

- [elaborate_design](#)

hdl_use_default_parameter_values_in_name

hdl_use_default_parameter_values_in_name {true | false}

Description

Default: false

Data_type: bool, read/write

Shortens the name of the parametrized module by using only the parameter values specified at instantiation, by default. On setting this attribute to `true`, the tool uses all the available parameters in the parametrized module name.

This attribute is supported only in the RTL flow.

Related Information

- [elaborate_design](#)

hdl_use_for_generate_prefix

hdl_use_for_generate_prefix {true | false}

Description

Default: true

Data_type: bool, read/write

Controls whether the block label of a `for-generate` statement will be used with the `for-generate` index values as prefix of the name of an instance instantiated within the `generate` construct.

- If this attribute is set to `false`, the `for-generate` index values are appended to the name of

instances instantiated within the `generate` construct.

- If this attribute is set to `true`, instance naming follows the naming style defined in the Verilog-2005 LRM (in Section 12.4.2 and 12.4.3 of IEEE Std 1364-2005).

You must specify this attribute before you elaborate the design.

Example

For the following examples consider the following VHDL code:

```
L1: FOR i IN 2 TO 4 generate u1 : sub port map(q => q(i), d1 => d1(i), d2 => d2(i));end generate;
```

- When you set the `hdl_use_for_generate_prefix` attribute to `true`, the tool generates the following:

```
sub \L1[2].u1 (.q (q[2]), .d1 (d1[2]), .d2 (d2[2]));  
sub \L1[3].u1 (.q (q[3]), .d1 (d1[3]), .d2 (d2[3]));  
sub \L1[4].u1 (.q (q[4]), .d1 (d1[4]), .d2 (d2[4]));
```

- When you set the `hdl_use_for_generate_prefix` attribute to `false`, the tool generates the following:

```
sub \u1.[2] (.q (q[2]), .d1 (d1[2]), .d2 (d2[2]));  
sub \u1.[3] (.q (q[3]), .d1 (d1[3]), .d2 (d2[3]));  
sub \u1.[4] (.q (q[4]), .d1 (d1[4]), .d2 (d2[4]));
```

Related Information

- [elaborate_design](#)
- [hdl_generate_index_style](#)
- [hdl_generate_separator](#)

hdl_use_if_generate_prefix

```
hdl_use_if_generate_prefix {true | false}
```


Description

Default: `true`

Data_type: `bool`, `read/write`

Controls whether the block label of an `if-generate` statement will be used as prefix of the instance name of an instance instantiated within the `generate` construct.

- If this attribute is set to `false`, this part of instance naming is backward-compatible with prior releases.
- If this attribute is set to `true`, this part of instance naming follows the naming style defined in the Verilog-2005 LRM (in Section 12.4.2 of IEEE Std 1364-2005).

In VHDL, a `generate` statement must have a label. In Verilog, the block of a `generate` statement can be unnamed. If without a label, it is given a name based on the rules defined in the Verilog-2005 LRM (in Section 12.4.3 of IEEE Std 1364-2005).

This attribute does not affect a Verilog `case-generate` statement, with which the block label is always used when composing instance names.

You must specify this attribute before you elaborate the design.

Related Information

- [elaborate_design](#)
- [hdl_generate_separator](#)
- [hdl_use_block_prefix](#)

hdl_use_port_default_value

`hdl_use_port_default_value {true | false}`

Description

Default: `true`

Data_type: `bool`, `read/write`

When set to `true`, the tool honors the default initial values of the input ports in a VHDL component

declaration or entity declaration.

This attribute is supported only in the RTL flow.

Related Information

- [elaborate_design](#)

hdl_verilog_defines

`hdl_verilog_defines macro_definition_list`

Description

Default: `SYNTHESIS`

Data_type: `string`, `read/write`

Specifies a list of global macro definitions that must be applied to each subsequent `read_hdl` (except with `-vhdl`) and `read_netlist` command. This includes calls to `read_hdl` made during a call to `elaborate -libpath`.

Each macro definition has the form `id` or `id=expr`.

Note that the `read_hdl` and `elaborate` commands specified above are only valid within the script being passed to `elaborate_design`. You cannot use them directly at the Innovus prompt.

Related Information

- [elaborate_design](#)
- [read_netlist](#)

hdl_vhdl_assign_width_mismatch

```
hdl_vhdl_assign_width_mismatch {true | false}
```

Description

Default: false

Data_type: bool, read/write

Controls whether to allow an assignment in VHDL when the left-hand side (LHS) and right-hand side (RHS) have the same array type but mismatching width. When set to `true`, the RHS gets truncated or extended to the width of the LHS. By default, VHDL does not allow such assignments.

Example

Consider the following RTL where the ports `Mult_A` and `Z` have the same array type but have incompatible bit widths:

```
library ieee; use ieee.std_logic_1164.all; use ieee.numeric_std.all;
entity mmnumeric1 is
    generic (wmA : integer := 2;
             wZ  : integer := 5);
    port ( Mult_A : in  signed (wmA-1 downto 0);
          Z      : out signed (wZ-1  downto 0) );
end;

architecture rtl of mmnumeric1 is
begin
    Z <= Mult_A;
end;
```

With the default setting of `hdl_vhdl_assign_width_mismatch (false)`, the tool issues the error message CDFG-283 during elaboration.

When you set `hdl_vhdl_assign_width_mismatch` to `true`, the RTL will successfully elaborate, but the tool issues the warning message CDFG-239.

Related Information

- [elaborate_design](#)
- [VHDL-Specific Attributes](#) section of the *Innovus Synthesis HDL Modeling Guide*

hdl_vhdl_case

```
hdl_vhdl_case {original | lower | upper}
```

Description

Default: original

Data_type: string, read/write

Stores VHDL identifiers and operators in lowercase, uppercase, or the case given in the source file.

Related Information

- [elaborate_design](#)
- [VHDL-Specific Attributes](#) section of the *Innovus Synthesis HDL Modeling Guide*

hdl_vhdl_environment

```
hdl_vhdl_environment {common | synergy}
```

Description

Default: common

Data_type: string, read/write

Specifies the selection of the predefined arithmetic libraries.

Related Information

- [elaborate_design](#)
- [VHDL-Specific Attributes](#) section of the *Innovus Synthesis HDL Modeling Guide*

hdl_vhdl_preferred_architecture

```
hdl_vhdl_preferred_architecture string
```

Description

Default:

Data_type: string, read/write

Sets the name of preferred architecture to use with an entity when there are multiple architectures.

Related Information

- [elaborate_design](#)
- [VHDL-Specific Attributes](#) section of the *Innovus Synthesis HDL Modeling Guide*

hdl_vhdl_range_opt

```
hdl_vhdl_range_opt {true | false}
```

Description

Default: false

Data_type: bool, read/write

Controls the optimization of the widths of arithmetic operators (+, *, -) that have ranged integer arguments in VHDL.

Example

Consider the following RTL:

```
entity rngopt is port(  
    in1 : in integer range 0 to 12;  
    in2 : in integer range 0 to 3;  
    output : out integer);  
end;  
  
architecture rtl of rngopt is  
begin  
    output <= in1 + in2;  
end;
```

- When you set `hdl_vhdl_range_opt` to `true`, the tool infers an unsigned adder with an output width of 4 bits during elaboration, because the result will be in the range of 0 to 15 (12+3).
- When you use the default setting (`false`), the tool infers an unsigned adder with an output width of 5 bits during elaboration.

Related Information

- [elaborate_design](#)
- [VHDL-Specific Attributes](#) section of the *Innovus Synthesis HDL Modeling Guide*

hdl_vhdl_read_version

`hdl_vhdl_read_version {1993 | 1987 | 2008}`

Description

Default: 1993

Data_type: string, read/write

Specifies the VHDL version to be used when reading VHDL designs. If you change the `hdl_vhdl_read_version` read version at any time, the tool automatically re-maps the IEEE and STD VHDL libraries to the correct directory in the software release. In this way, the correct versions of the standard packages in these libraries are picked up.

Related Information

- [elaborate_design](#)
- [VHDL-Specific Attributes](#) section of the *Innovus Synthesis HDL Modeling Guide*

hdl_vhdl_strict_compliance

`hdl_vhdl_strict_compliance {true | false}`

Description

Default: `false`

Data_type: `bool`, `read/write`

When set to `true`, the `read_hdl` command enforces a strict interpretation of the VHDL LRM. This variable lets you verify that your VHDL code is compliant with the LRM, such that it is likely to work on other VHDL tools.

When set to `false` the following features are allowed:

- Treats a concatenation as a locally static expression, which allows constructs such as the following:

```
case expr is
    when "001" & '1' => ...
```

- Initializes an interface object with a function call.
- Lets you use name `x` in the definition of a different `x`:

```
constant c : integer := 3;
function f (c : integer := c) is ...
```

- In VHDL-1987, treats a function call with globally static arguments as a globally static expression. For instance:

```
function f(x : integer) return integer;
generic (y : integer := f(3));
```

Note that the `read_hdl` command specified above is only valid within the script being passed to `elaborate_design`. You cannot use it directly at the Innovus prompt.

Related Information

- [elaborate_design](#)
- [VHDL-Specific Attributes](#) section of the *Innovus Synthesis HDL Modeling Guide*

init_blackbox_for_undefined

```
init_blackbox_for_undefined {true | false}
```

Description

Default: `true`

Data_type: `bool`, `read/write`

Controls how a logic abstract that is inferred from an empty module, an entity without an architecture, or an entity whose architecture is empty, is interpreted.

If set to `true`, the logic abstract becomes an unresolved reference in the design. The tool tries to find a library cell with a name corresponding to that of the logic abstract. If it finds such a cell, the logic abstract is represented as an instance of that library cell. Otherwise, the logic abstract becomes an unresolved reference.

❗ If set to `false`, the logic abstract remains as a module in the design hierarchy. In this case, an empty module is created, which is not considered unresolved. This is an unrealistic situation. The tool would treat the output ports of the module as undriven and would assign them values based on the setting of the `hdl_unconnected_value` attribute. This can potentially create false non-equivalence points during verification with LEC. Therefore, this setting should be avoided.

This attribute is supported in the RTL flow and the structural flow.

Related Information

- [elaborate_design](#)
- [read_netlist](#)

init_hdl_search_path

`init_hdl_search_path` *string*

Description

Default: `.`

Data_type: `string`, `read/write`

Specifies the list of UNIX directories in which the tool should search for files associated with the `read_hdl` command. The behavior is similar to the search path in UNIX.

In Verilog, this attribute directs the search of Verilog files specified with the `read_hdl` command and ``include` files specified in the Verilog code.

In VHDL, this attribute directs the search of VHDL files specified with the `read_hdl` command.

- This attribute is supported in the RTL flow and the structural flow.
- The "~" character is supported.

Note that the `read_hdl` command specified above is only valid within the script being passed to `elaborate_design`. You cannot use it directly at the Innovus prompt.

Related Information

- `elaborate_design`
- `read_netlist`

input_assert_one_cold_pragma

`input_assert_one_cold_pragma` *string*

Description

Default: `assert_one_cold one_cold`

Data_type: `string`, `read/write`

Carries a Tcl list of one or more names, each being treated as a `one_cold` pragma when reading in Verilog or VHDL.

Related Information

- [elaborate_design](#)
- [Synthesis Pragmas](#) section of the *Innovus Synthesis HDL Modeling Guide*

input_assert_one_hot_pragma

`input_assert_one_hot_pragma string`

Description

Default: `assert_one_hot one_hot`

Data_type: `string, read/write`

Carries a Tcl list of one or more names, each being treated as a `one_hot` pragma when reading in Verilog or VHDL.

Related Information

- [elaborate_design](#)
- [Synthesis Pragmas](#) section of the *Innovus Synthesis HDL Modeling Guide*

input_asynchro_reset_blk_pragma

`input_asynchro_reset_blk_pragma string`

Description

Default: `async_set_reset_local asynchro_reset_blk`

Data_type: `string, read/write`

Carries a Tcl list of one or more names, each being treated as an `asynchro_reset_blk` pragma when reading in Verilog or VHDL.

Specifies the aliases for the `asynchro_reset_blk` pragma:

```
//cadence asynchro_reset_blk <namedBlock> [{rst1 rst2}]
```

Any `if` conditions in the specified block containing `rst1` and `rst2` are considered as asynchronous reset condition inside the asynchronous `if-then-else` decoding code.

This attribute is supported only in the RTL flow.

Related Information

- [elaborate_design](#)
- [Synthesis Pragmas](#) section of the *Innovus Synthesis HDL Modeling Guide*

input_asynchro_reset_pragma

`input_asynchro_reset_pragma` *string*

Description

Default: `async_set_reset asynchro_reset`

Data_type: `string`, `read/write`

Carries a Tcl list of one or more names, each being treated as an `asynchro_reset` pragma when reading in Verilog or VHDL.

Specifies the aliases for the `asynchro_reset` pragma:

```
//cadence asynchro_reset [{rst1 rst2}]
```

Any `if` conditions containing `rst1` and `rst2` are considered as asynchronous reset conditions inside the asynchronous `if-then-else` decoding code.

This attribute is supported only in the RTL flow.

Related Information

- [elaborate_design](#)
- [Synthesis Pragmas](#) section of the *Innovus Synthesis HDL Modeling Guide*

input_case_cover_pragma

`input_case_cover_pragma string`

Description

Default: `full_case`

Data_type: `string`, `read/write`

Carries a Tcl list of one or more names, each being treated as a `full_case` pragma when reading in Verilog or VHDL.

Specifies the equivalent used in input pragmas for the `case_logic cover` pragma.

This attribute is supported only in the RTL flow.

Related Information

- [elaborate_design](#)
- [Synthesis Pragmas](#) section of the *Innovus Synthesis HDL Modeling Guide*

input_case_decode_pragma

`input_case_decode_pragma string`

Description

Default: `parallel_case`

Data_type: `string`, `read/write`

Carries a Tcl list of one or more names, each being treated as a `parallel_case` pragma when reading in Verilog or VHDL.

Specifies the equivalent used in input pragmas for the `case_logic no_priority` pragma.

This attribute is supported only in the RTL flow.

Related Information

- [elaborate_design](#)
- [Synthesis Pragmas](#) section of the *Innovus Synthesis HDL Modeling Guide*

input_map_to_mux_pragma

`input_map_to_mux_pragma string`

Description

Default: `map_to_mux infer_mux_override`

Data_type: `string, read/write`

Carries a Tcl list of one or more names, each being treated as a `map_to_mux` pragma when reading in Verilog or VHDL.

Specifies the equivalent used in input pragmas for the `map_to_mux` pragma.

This attribute is supported only in the RTL flow.

Related Information

- [elaborate_design](#)
- [Synthesis Pragmas](#) section of the *Innovus Synthesis HDL Modeling Guide*

input_pragma_keyword

input_pragma_keyword *string*

Description

Default: cadence synopsys ambit pragma synthesis

Data_type: string, read/write

Carries a Tcl list of one or more names, each being treated as the keyword indicating a pragma when reading in Verilog or VHDL.

A pragma is a comment whose first word is a pragma keyword specified in this attribute.

You cannot remove the `cadence` keyword from the list.

This attribute is supported in the RTL flow and the structural flow.

Related Information

- [elaborate_design](#)
- [read_netlist](#)
- [Synthesis Pragas](#) section of the *Innovus Synthesis HDL Modeling Guide*

input_synchro_enable_blk_pragma

input_synchro_enable_blk_pragma *string*

Description

Default: synchro_enable_blk

Data_type: string, read/write

Carries a Tcl list of one or more names, each being treated as a `synchro_enable_blk` pragma when reading in Verilog or VHDL.

Specifies the alias names for the `synchro_reset_blk` pragma.

Related Information

- [elaborate_design](#)
- [Synthesis Pragmas](#) section of the *Innovus Synthesis HDL Modeling Guide*

input_synchro_enable_pragma

`input_synchro_enable_pragma string`

Description

Default: `synchro_enable`

Data_type: `string`, read/write

Carries a Tcl list of one or more names, each being treated as a `synchro_enable` pragma when reading in Verilog or VHDL.

Specifies the alias names for the `synchro_enable` pragma.

Related Information

- [elaborate_design](#)
- [Synthesis Pragmas](#) section of the *Innovus Synthesis HDL Modeling Guide*

input_synchro_reset_blk_pragma

`input_synchro_reset_blk_pragma string`

Description

Default: `sync_set_reset_local synchro_reset_blk`

Data_type: `string`, read/write

Carries a Tcl list of one or more names, each being treated as a `synchro_reset_blk` pragma when

reading in Verilog or VHDL.

Specifies the aliases for the `sync_set_reset_local` pragma.

```
//cadence sync_set_reset_local namedBlock "comma-separated_list_of_signals"
```

This attribute is supported only in the RTL flow.

Related Information

- [elaborate_design](#)
- [Synthesis Pragas](#) section of the *Innovus Synthesis HDL Modeling Guide*

input_synchro_reset_pragma

```
input_synchro_reset_pragma string
```

Description

Default: `sync_set_reset synchro_reset`

Data_type: `string`, `read/write`

Carries a Tcl list of one or more names, each being treated as a `synchro_reset` pragma when reading in Verilog or VHDL.

Specifies the aliases for the `synchro_reset` pragma:

```
//cadence sync_set_reset [{rst1 rst2}]
```

Any `if` conditions containing `rst1` and `rst2` are considered as synchronous reset conditions inside the synchronous `if-then-else` decoding code.

This attribute is supported only in the RTL flow.

Related Information

- [elaborate_design](#)
- [Synthesis Pragmas](#) section of the *Innovus Synthesis HDL Modeling Guide*

proto_hdl

`proto_hdl string`

Description

Default:

Data_type: string, read/write

Enables support for incomplete HDL.

Related Information

- [elaborate_design](#)

script_begin

`script_begin string`

Description

Default: dc_script_begin dc_tcl_script_begin script_begin

Data_type: string, read/write

Specifies the keyword in the RTL netlist that indicates the beginning of a script included in the netlist.

This attribute is supported only in the RTL flow.

Related Information

- [elaborate_design](#)
- [script_end](#)
- [Supported Pragmas](#) section of the *Innovus Synthesis HDL Modeling Guide*

script_end

`script_end string`

Description

Default: `dc_script_end dc_tcl_script_end script_end`

Data_type: `string, read/write`

Specifies the keyword in the RTL netlist that indicates the end of a script included in the netlist.

Related Information

- [elaborate_design](#)
- [script_begin](#)
- [Supported Pragmas](#) section of the *Innovus Synthesis HDL Modeling Guide*

synthesis_off_command

`synthesis_off_command string`

Description

Default: `translate_off synthesis_off`

Data_type: `string, read/write`

Specifies the pragma that is used to indicate the beginning of non-synthesizable constructs in the RTL source code or in the generated generic netlist.

This attribute is supported in the RTL flow and the structural flow.

Related Information

- `elaborate_design`
- `input_pragma_keyword`
- `read_netlist`
- `synthesis_on_command`
- [Synthesis Pragmas](#) section of the *Innovus Synthesis HDL Modeling Guide*

synthesis_on_command

`synthesis_on_command` *string*

Description

Default: `translate_on synthesis_on`

Data_type: `string`, `read/write`

Specifies the pragma that is used to indicate the end of non-synthesizable constructs in the RTL source code or in the generated generic netlist.

This attribute is supported in the RTL flow and the structural flow.

Related Information

- `elaborate_design`
- `input_pragma_keyword`
- `read_netlist`
- `synthesis_off_command`
- [Synthesis Pragmas](#) section of the *Innovus Synthesis HDL Modeling Guide*

hier

The root attribute in the `hier` category is:

- `assemble_design_generic_hier`

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `hier` category, use the following command:

```
get_db -category hier
```

assemble_design_generic_hier

```
assemble_design_generic_hier {true | false}
```

Default: `false`

Data_type: `bool`, `read/write`

Allows to read in hierarchical DEF in `assemble_design`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
- To get the current value of any root attribute, use the `get_db` command:

```
set_db attribute_name value
```

```
get_db attribute_name
```

Related Commands

- `assemble_design`

ilm

The root attributes in the ilm category are:

- [ilm_auto_use_sdc_files](#)
- [ilm_filter_internal_path](#)
- [ilm_keep_async](#)
- [ilm_keep_flatten](#)
- [ilm_keep_high_fanout_ports](#)
- [ilm_keep_inst_in_sdc](#)
- [ilm_keep_loopback](#)
- [ilm_max_num_insts](#)
- [ilm_max_num_registers](#)
- [ilm_slack_driven](#)

To set the value of any root attribute, use the [set_db](#) command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the [get_db](#) command:

```
get_db attribute_name
```

To get the current value of all attributes in the `ilm` category, use the following command:

```
get_db -category ilm
```

ilm_auto_use_sdc_files

```
ilm_auto_use_sdc_files {true|false}
```

Default: `false`

Data_type: `bool`, read/write

Reloads the `sdc_files` of top-level design.

Related Command

- `write_ilm`

ilm_filter_internal_path

`ilm_filter_internal_path {true|false}`

Default: `false`

Data_type: `bool`

When set to true, group paths are created for internal paths with slack adjustment of 9999. Therefore, all internal paths are filtered out from the timing report.

Related Command

- `write_ilm`

ilm_keep_async

`ilm_keep_async {true|false}`

Default: `false`

Data_type: `bool`

Keeps paths from asynchronous input ports. For example, paths from an input port to the reset pin of interface sequential instances, but not to the internal sequential instances.

ilm_keep_flatten

`ilm_keep_flatten {true|false}`

Default: `true`

Data_type: `bool`

Makes flow work under the flattenIlm mode.

ilm_keep_high_fanout_ports

`ilm_keep_high_fanout_ports {true|false}`

Default: `true`

Data_type: `bool`

Keeps high-fanout input port paths.

ilm_keep_inst_in_sdc

`ilm_keep_inst_in_sdc {true|false}`

Default: `false`

Data_type: `bool`

Keeps instances which have constraints on it.

ilm_keep_loopback

`ilm_keep_loopback {true|false}`

Default: `false`

Data_type: `bool`

Keeps loop back paths.

When set to `true`, then any path touching a net used for the ILM model is also included. This is done for better model accuracy, so that the actual capacitance on an output port could affect the delay on a net which feeds an internal path that is normally not included in the ILM model. This also degrades the reduction ratio ($\frac{\#ILM_insts}{\#all_inst}$) for the block.

ilm_max_num_insts

`ilm_max_num_insts maxInstances`

Default: `1000`

Data_type: `int, read/write`

Specifies the maximum number of instances.

ilm_max_num_registers

`ilm_max_num_registers` *maxRegisters*

Default: 40

Data_type: int, read/write

Specifies the maximum number of registers.

ilm_slack_driven

`ilm_slack_driven` {true|false}

Default: false

Data_type: bool, read/write

Specifies slack driven ILM generation.

init

The root attributes in the `init` category are:

- `init_check_netlist`
- `init_delete_floating_hnets`
- `init_keep_empty_modules`
- `init_read_netlist_allow_port_mismatch`
- `init_read_netlist_allow_undefined_cells`
- `init_oa_tech_lib`
- `init_power_intent_files`
- `init_sync_relative_path`
- `init_timing_enabled`
- `init_tiers`

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `init` category, use the following command:

```
get_db -category init
```

init_check_netlist

```
init_check_netlist {true | false}
```

Default: `false`

Data_type: `bool`

After reading in the netlist, it performs the same checks as `check_design -netlist`.

init_delete_floating_hnets

```
init_delete_floating_hnets {true | false}
```

Default: true

Data_type: bool

If true, an hnet (Verilog wire statement) that has nothing connected to it, is deleted from the netlist. The default value is true in Tempus to delete non-timed nets and reduce memory.

init_keep_empty_modules

```
init_keep_empty_modules {true | false}
```

Default: true

Data_type: bool

Keeps empty modules. If set to `false`, the software converts the empty modules into special leaf cells that do not have a physical library associated with them.

If the module is a hard macro, using a value of `true` or `false` produces the identical behavior: the software keeps the module.

Related Commands

- [read_db](#)

init_read_netlist_allow_port_mismatch

```
init_read_netlist_allow_port_mismatch {true | false}
```

Default: false

Data_type: bool

Specifies whether new Instance ports can be created during Verilog netlist parsing.

init_read_netlist_allow_undefined_cells

```
init_read_netlist_allow_undefined_cells {true | false}
```

Default: false

Data_type: bool

Creates a dummy .lib file and a user-defined noise model (UDN) for cells that are not present in the loaded .lib or cdB.

init_oa_tech_lib

```
init_oa_tech_lib lib_name
```

Default: ""

Data_type: string, read/write

Specifies the OpenAccess library to use as a source of technology information. If not specified, the first library in the `init_oa_ref_lib` list will be used. Applies only when `read_netlist` is used to process a Verilog netlist. It is set by the `read_physical -oa_tech_lib` command.

init_power_intent_files

```
init_power_intent_files string
```

Default: ""

Data_type: string

Specifies the power intent files read by the previous run of the `read_power_intent` command.

init_sync_relative_path

```
init_sync_relative_path {true | false}
```

Default: false

Note: You must change your working directory to the directory saved in the previous design session before you restore the design.

Data_type: bool

Synchronizes all relative paths in the top.globals file to the current working directory. Use this parameter if you want to restore the design without first changing your current working directory to the directory saved in the previous design session.

Example

- The following command synchronizes all relative paths to the current working directory:

```
set_db init_sync_relative_path true
```

Related Commands

- `read_db`

init_timing_enabled

```
init_timing_enabled {true | false}
```

Default: true

Data_type: bool

If `read_physical` is issued before `read_mmmc` or `read_libs` in Genus and Tempus, an error will be issued. In Innovus, doing this will result in `init_timing_enabled` being set to `false` by the tool. Innovus is now in physical only mode, and any attempt to set timing data will result in an error.

Related Commands

- `read_physical`
- `read_mmmc`
- `read_libs`

init_tiers

```
init_tiers string
```

Default: ""

Data_type: string, read/write

Specifies the orientation of each tier, either 'up' or 'down', starting with the bottommost, for a multi-tier design in `init_design`. You can specify the orientation of each tier using the keywords `up` or `down`, starting from the bottommost tier. At least two tiers must be defined; specifying a single tier is not allowed and results in an error.

Examples (two-tier designs):

- { up down } – Face-to-Face (F2F)
- { down up } – Back-to-Back (B2B)
- { up up } – Face-to-Back (F2B)
- { down down } – Face-to-Back (F2B)

For designs with more than two tiers, you need to add additional entries for Tier2, Tier3, and so on. Only the up and down keywords are accepted.

lec

The root attribute in the `lec` category is:

- [clp_enable_1801_hierarchical_bbox](#)
- [wcdc_clock_dom_comb_propagation](#)
- [write_clp_lib_statetable](#)
- [write_do_lec_add_no_black_box_retime_subdesign](#)
- [write_do_lec_analyze_abort](#)
- [write_do_lec_analyze_setup](#)
- [write_do_lec_auto_analyze](#)
- [write_do_lec_black_box_ilm_modules](#)
- [write_do_lec_compare_threads](#)
- [write_do_lec_composite_compare](#)
- [write_do_lec_dft_constraint_file](#)
- [write_do_lec_gzip_lec_json](#)
- [write_do_lec_hier_append_string](#)
- [write_do_lec_hier_comp_threshold](#)
- [write_do_lec_hier_compare_string](#)
- [write_do_lec_hier_prepend_string](#)
- [write_do_lec_lib_statetable](#)
- [write_do_lec_low_power_analysis](#)

- [write_do_lec_multithread_license_list](#)
- [write_do_lec_no_dft_constraints](#)
- [write_do_lec_parallel_threads](#)
- [write_do_lec_post_add_no_translate_modules](#)
- [write_do_lec_run_hier_check_non_equivalent](#)
- [write_do_lec_set_cdn_syn_root](#)
- [write_do_lec_uniquify](#)
- [write_do_lec_use_lec_model](#)
- [write_do_lec_use_smart_lec](#)
- [write_do_lec_verbose](#)

To set the value of any root attribute, use the following command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the following command:

```
get_db attribute_name
```

To get the current value of all attributes in the `lec` category, use the following command:

```
get_db -category lec
```

clp_enable_1801_hierarchical_bbox

```
clp_enable_1801_hierarchical_bbox {false | true}
```

Description

Default: `false`

Data_type: `bool`, read/write

Determines whether to enable the 1801 hierarchical black-boxed block flow in Conformal Low Power (CLP).

Related Information

- root attributes in the [Database Objects](#) chapter in *Stylus Common UI Database Object Information* manual

wcdc_clock_dom_comb_propagation

```
wcdc_clock_dom_comb_propagation {logic_nophase | exact_phase | either_phase | wire |  
logic_phase}
```

Description

Default: `logic_nophase`

Data_type: `string`, read/write

Controls clock domain propagation through combinational elements in the Conformal® Extended Checks software.

Related Information

- root attributes in the [Database Objects](#) chapter in *Stylus Common UI Database Object Information* manual

write_clp_lib_statetable

```
write_clp_lib_statetable {true | false}
```

Description

Default: `true`

Data_type: `bool`, read/write

Affects the `read library -liberty` command in the generated dofile. If set to `true`, that command is equipped with a `-statetable` option. If set to `false`, that command will not have `-statetable`.

Note: The Conformal LEC commands mentioned above are for informational purpose only. These

commands are not executed by Innovus.

Related Information

- root attributes in the [Database Objects](#) chapter in *Stylus Common UI Database Object Information* manual

write_do_lec_add_no_black_box_retime_subdesign

```
write_do_lec_add_no_black_box_retime_subdesign {true | false}
```

Description

Default: true

Data_type: bool, read/write

Controls whether to add the Conformal LEC command that excludes the specified retimed modules from the hierarchical dofile script generation. Specify this attribute before writing out the intermediate netlist.

The default value is the recommended value for retiming verification.

Related Information

- root attributes in the [Database Objects](#) chapter in *Stylus Common UI Database Object Information* manual

write_do_lec_analyze_abort

```
write_do_lec_analyze_abort {true | false}
```

Description

Default: false

Data_type: bool, read/write

Adds the Conformal LEC command that analyzes the abort points to the dofile. The command

works on the abort points in the design and tries to resolve them by adding partition points, increasing the compare effort, and more. Specify this attribute before writing out the intermediate netlist.

If the attribute is set to `true` in a hierarchical compare, the following commands are added in the dofile:

```
write hier_compare dofile <dofile name> -noexact_pin_match -constraint -usage
    -replace -run_hier -prepend_string
"remodel -seq_constant -repeat"
run hier_compare <dofile name> -analyze_abort
```

If the attribute is set to `true` in a *flat* compare, the following commands are added in the dofile:

```
set system mode lec
remodel -seq_constant -repeat
map key points
report mapped points
report unmapped points -summary
report unmapped points -extra -unreachable -notmapped
add compared points -all
compare
usage
// if seeing abort, please try 'analyze abort -compare'
report compare data
```

Note: The Conformal LEC commands mentioned above are for informational purpose only. These commands are not executed by Innovus.

Related Information

- root attributes in the [Database Objects](#) chapter in *Stylus Common UI Database Object Information* manual

write_do_lec_analyze_setup

```
write_do_lec_analyze_setup {true | false}
```

Description

Default: `false`

Data_type: bool, read/write

Adds the Conformal LEC command `analyze setup -verbose` to the dofile. The command analyzes setup related issues and tries to correct any key point mapping related issues before starting the comparison. Specify this attribute before writing out the intermediate netlist.

If the attribute is set to `true` in a *hierarchical* compare, the following commands will be added in the dofile:

```
write hier_compare dofile <dofile name> -noexact_pin_match -constraint \  
    -usage -replace -run_hier -prepend_string  
  
"remodel -seq_constant -repeat;analyze setup -verbose; map key points"
```

If the attribute is set to `true` in a *flat* compare, the following commands are added:

```
set system mode lec  
remodel -seq_constant -repeat  
analyze setup -verbose  
map key points
```

Note: The Conformal LEC commands mentioned above are for informational purpose only. These commands are not executed by Innovus.

Related Information

- root attributes in the [Database Objects](#) chapter in *Stylus Common UI Database Object Information* manual
- `write_do_lec`

write_do_lec_auto_analyze

```
write_do_lec_auto_analyze {true | false}
```

Description

Default: `true`

Data_type: bool, read/write

Adds the Conformal LEC command `set analyze option -auto` to the dofile. This command handles both setup related issues as well as abort points (if any). This attribute is a superset of the

`write_do_lec_analyze_abort` and `write_do_lec_analyze_setup` attributes. Hence, if you keep this attribute at its default value of `true`, you do not need to specify neither `write_do_lec_analyze_setup` nor `write_do_lec_analyze_abort`.

By default the *hierarchical* compare will have the following commands in the dofile:

```
set flatten model -seq_constant -seq_constant_x_to 0
set flatten model -nodff_to_dlat_zero -nodff_to_dlat_feedback
set analyze option -auto

write hier_compare dofile <dofile name> -noexact_pin_match -constraint -usage \
    -replace -run_hier -prepend_string "remodel -seq_constant -repeat"
run hier_compare <dofile name>
```

By default, the *flat* compare will have the following commands in the dofile:

```
set flatten model -seq_constant -seq_constant_x_to 0
set flatten model -nodff_to_dlat_zero -nodff_to_dlat_feedback
set analyze option -auto
set system mode lecremodel -seq_constant -repeat
```

If all of the `write_do_lec_auto_analyze`, `write_do_lec_analyze_setup`, and `write_do_lec_analyze_abort` attributes are set to `true`, the `write_do_lec_auto_analyze` will take precedence.

Note: The Conformal LEC commands mentioned above are for informational purpose only. These commands are not executed by Innovus.

Related Information

- root attributes in the [Database Objects](#) chapter in *Stylus Common UI Database Object Information* manual
- `write_do_lec_analyze_abort` attribute
- `write_do_lec_analyze_setup` attribute

write_do_lec_black_box_ilm_modules

```
write_do_lec_black_box_ilm_modules {true | false}
```

Description

Default: `false`

Data_type: bool, read/write

When true, Conformal LEC treats ILM modules as black boxes.

Related Information

- root attributes in the [Database Objects](#) chapter in *Stylus Common UI Database Object Information* manual
- `write_do_lec`

write_do_lec_compare_threads

`write_do_lec_compare_threads <integer>`

Description

Default: 4

Data_type: int, read/write

Adds the Conformal[®] LEC command `set compare options` with the `-threads` option to the dofile. The attribute value specifies the number of compare threads.

Note: The Conformal LEC commands mentioned above are for informational purpose only. These commands are not executed by Innovus.

Related Information

- root attributes in the [Database Objects](#) chapter in *Stylus Common UI Database Object Information* manual

write_do_lec_composite_compare

`write_do_lec_composite_compare {true | false}`

Description

Default: `true`

Data_type: `bool`, `read/write`

Specifies that the generated .do file include two compare operations: `rtl-to-fv_map` and `fv_map-to-revised`. The 'fv_map' netlist is automatically written in the verification directory during mapping (`syn_map`). When disabled, the generated .do file contains only one compare operation: `golden-to-revised`.

- Only applicable when you execute the `write_do_lec` command after the `syn_map` command with the `-golden_design` defaulting to the RTL.
- This attribute has no effect when root attribute `one_pass_formal_verification` is `true`.

Related Information

- root attributes in the [Database Objects](#) chapter in *Stylus Common UI Database Object Information* manual
- `write_do_lec`

write_do_lec_dft_constraint_file

`write_do_lec_dft_constraint_file <file>`

Description

Default:

Data_type: `string`, `read/write`

Specifies a TCL script to be sourced from the generated .do dofile. This provides a method to apply additional DFT constraints during formal verification when a third-party DFT tool is used. This file must exist when verification is performed.

Related Information

- root attributes in the [Database Objects](#) chapter in *Stylus Common UI Database Object Information* manual
- `write_do_lec`

write_do_lec_gzip_lec_json

`write_do_lec_gzip_lec_json {true | false}`

Description

Default: `false`

Data_type: `bool`, `read/write`

When set to `true`, compresses the verification information files, `<name>.fv.json.gz`, written by `write_do_lec` to the verification directory.

Conformal 19.10-s300 or higher version is required to support compressed `.fv.json.gz` files.

Related Information

- root attributes in the [Database Objects](#) chapter in *Stylus Common UI Database Object Information* manual
- `write_do_lec`

write_do_lec_hier_append_string

`write_do_lec_hier_append_string <file>`

Description

Default:

Data_type: string, read/write

When specified, the value is used as the `-append_string` argument to the Conformal LEC `write_hier_compare_dofile` command written to the dofile by `write_do_lec` command.

Note: The Conformal LEC commands mentioned above are for informational purpose only. These commands are not executed by Innovus.

Related Information

- root attributes in the [Database Objects](#) chapter in *Stylus Common UI Database Object Information* manual
- `write_do_lec`

write_do_lec_hier_comp_threshold

`write_do_lec_hier_comp_threshold <integer>`

Description

Default: 50

Data_type: int, read/write

Specifies the minimum number of instances required in a module to perform a hierarchical comparison on that module with the Conformal[®] Logical Equivalence Checker.

Related Information

- root attributes in the [Database Objects](#) chapter in *Stylus Common UI Database Object Information* manual
- `write_do_lec`

write_do_lec_hier_compare_string

`write_do_lec_hier_compare_string <file>`

Description

Default:

Data_type: string, read/write

When specified, the value is used as the `-compare_string` argument to the Conformal LEC `write_hier_compare_dofile` command written to the dofile by `write_do_lec` command.

Note: The Conformal LEC commands mentioned above are for informational purpose only. These commands are not executed by Innovus.

Related Information

- root attributes in the [Database Objects](#) chapter in *Stylus Common UI Database Object Information* manual
- `write_do_lec`

write_do_lec_hier_prepend_string

`write_do_lec_hier_prepend_string <file>`

Description

Default:

Data_type: string, read/write

When specified, the value is used as the `-prepend_string` argument to the Conformal LEC `write_hier_compare_dofile` command written to the dofile by `write_do_lec` command.

Note: The Conformal LEC commands mentioned above are for informational purpose only. These commands are not executed by Innovus.

Related Information

- root attributes in the [Database Objects](#) chapter in *Stylus Common UI Database Object Information* manual
- `write_do_lec`

write_do_lec_lib_statetable

```
write_do_lec_lib_statetable {true | false}
```

Description

Default: true

Data_type: bool, read/write

Affects the `read library -liberty` command in the generated dofile. If set to `true`, that command is equipped with a `-statetable` option. If set to `false`, that command will not have `-statetable`.

Note: The Conformal LEC commands mentioned above are for informational purpose only. These commands are not executed by Innovus.

Related Information

- root attributes in the [Database Objects](#) chapter in *Stylus Common UI Database Object Information* manual
- [write_do_lec](#)

write_do_lec_low_power_analysis

```
write_do_lec_low_power_analysis {true | false}
```

Description

Default: false

Data_type: bool, read/write

When enabled, adds the Conformal Low Power commands to compare the power domains between the golden and revised designs. The design needs to have power intent information.

This attribute controls the following the Conformal Low Power commands:
`check_lowpower_cells` (CPF) and `compare_power_consistency` (1801 flow), and
`compare_power_intent`.

Note: The Conformal LEC commands mentioned above are for informational purpose only. These commands are not executed by Innovus.

Related Information

- root attributes in the [Database Objects](#) chapter in *Stylus Common UI Database Object Information* manual
- `compare_power_intent`
- `write_do_lec`

write_do_lec_multithread_license_list

`write_do_lec_multithread_license_list <string>`

Description

Default:

Data_type: string, read/write

Specifies the Conformal license or list of Conformal licenses to use for multi-threaded processing. If the list is empty, the Conformal default list will be used.

Related Information

- root attributes in the [Database Objects](#) chapter in *Stylus Common UI Database Object Information* manual
- `write_do_lec`

write_do_lec_no_dft_constraints

`write_do_lec_no_dft_constraints {true | false}`

Description

Default: false

Data_type: bool, read/write

When set to true, specifies that automatically generated DFT constraints not be included in the Conformal LEC dofile written by `write_do_lec` command.

Related Information

- root attributes in the [Database Objects](#) chapter in *Stylus Common UI Database Object Information* manual
- `write_do_lec`

write_do_lec_parallel_threads

`write_do_lec_parallel_threads` <integer>

Description

Default: 4

Data_type: int, read/write

Adds the Conformal LEC command `set parallel option` with the `-threads` option to the dofile. The attribute value specifies the number of parallel threads.

Note: The Conformal LEC commands mentioned above are for informational purpose only. These commands are not executed by Innovus.

Related Information

- root attributes in the [Database Objects](#) chapter in *Stylus Common UI Database Object Information* manual
- `write_do_lec`

write_do_lec_post_add_no_translate_modules

`write_do_lec_post_add_no_translate_modules` <string>

Description

Default:

Data_type: string, read/write

Specifies a Tcl script to be sourced after any `add_no_translate_modules` command execution and before `read_library` command in the .do script generated by `write_do_lec`. Use `add_no_translate_modules` commands in the `write_do_lec_post_add_no_translate_modules` script to overwrite any automatically-generated `add_no_translate_modules` commands.

Note: The Conformal LEC commands mentioned above are for informational purpose only. These commands are not executed by Innovus.

Related Information

- root attributes in the [Database Objects](#) chapter in *Stylus Common UI Database Object Information* manual
- `write_do_lec`

write_do_lec_run_hier_check_non_equivalent

```
write_do_lec_run_hier_check_non_equivalent {true | false}
```

Description

Default: false

Data_type: bool, read/write

Adds `-check_noneq` option to the `run_hier_compare` command in the dofile, which results in Conformal LEC to skip the `analyze_datapath` command if a quick compare operation first determines that the submodule is not equivalent. This option reduces Conformal LEC runtime for designs that require dynamic flattening.

Note: The Conformal LEC commands mentioned above are for informational purpose only. These commands are not executed by Innovus.

Related Information

- root attributes in the [Database Objects](#) chapter in *Stylus Common UI Database Object Information* manual
- `write_do_lec`

write_do_lec_set_cdn_syn_root

```
write_do_lec_set_cdn_syn_root {true | false}
```

Description

Default: `false`

Data_type: `bool`, read/write

This attribute only applies if the design contains ChipWare or third party libraries. By default, the `CDN_SYNTH_ROOT` environment variable will not be set in the generated dofile. This allows you the flexibility to manually set `CDN_SYNTH_ROOT`.

If this attribute is set to `true`, the `CDN_SYNTH_ROOT` is set to the directory in which the tool is installed. This ensures that the ChipWare/third party library components are picked from the same tool session that was used during synthesis.

Related Information

- root attributes in the [Database Objects](#) chapter in *Stylus Common UI Database Object Information* manual
- `write_do_lec`

write_do_lec_uniquify

```
write_do_lec_uniquify {true | false}
```

Description

Default: `true`

Data_type: bool, read/write

Controls uniquification of all modules in the design. If this attribute is set to `true`, the `write_do_lec` command adds the following LEC command to the generated dofile:

```
uniquify -all -nolib
```

This attribute is only effective if the generated dofile performs a hierarchical Conformal comparison.

If the RTL has instantiated ChipWare components, the `uniquify` command will be generated in the dofile regardless of the value of the `write_do_lec_uniquify` attribute. This is to prevent these modules from getting skipped during hierarchical compare.

Note: The Conformal LEC commands mentioned above are for informational purpose only. These commands are not executed by Innovus.

Related Information

- root attributes in the [Database Objects](#) chapter in *Stylus Common UI Database Object Information* manual
- `write_do_lec`

write_do_lec_use_lec_model

```
write_do_lec_use_lec_model {true | false}
```

Description

Default: `true`

Data_type: bool, read/write

Controls whether all ChipWare simulation models should be included.

If this attribute is set to `false`, the simulation for all ChipWare components will be loaded.

If set to `true`, the simulation model for all ChipWare components will be loaded except the following two:

- `CW_mult`

- DW02_mult

The generated dofile uses the Conformal built-in models instead. For these two components, the `write_do_lec` command

- Assumes the existence of LEC built-in models if

```
set_db write_do_lec_use_lec_model true
```

- Adds the tool provided models into the dofile if

```
set_db write_do_lec_use_lec_model false
```

This attribute is effective only if the golden design is the RTL code that was loaded into the tool.

Related Information

- root attributes in the [Database Objects](#) chapter in *Stylus Common UI Database Object Information* manual
- `write_do_lec`

write_do_lec_use_smart_lec

```
write_do_lec_use_smart_lec {true | false}
```

Description

Default: `false`

Data_type: `bool`, `read/write`

Controls whether to add adaptive-proof commands and information on massively parallel hierarchical compare to the dofile.

A Conformal Smart LEC license is required.

Related Information

- root attributes in the [Database Objects](#) chapter in *Stylus Common UI Database Object Information* manual
- `write_do_lec`

write_do_lec_verbose

`write_do_lec_verbose {true | false}`

Description

Default: `false`

Data_type: `bool`, read/write

Generates a dofile with verbose reporting. Alternatively, use the `-verbose` option of the `write_do_lec` command.

Related Information

- root attributes in the [Database Objects](#) chapter in *Stylus Common UI Database Object Information* manual
- `write_do_lec`

mesg

The root attributes in the `mesg` category are:

- [mesg_severity_downgrade](#)
- [report_library_message_summary](#)

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `mesg` category, use the following command:

```
get_db -category mesg
```

mesg_severity_downgrade

```
mesg_severity_downgrade {true| false}
```

Description

Default: `false`

Data_type: `bool`, read/write

Allows to downgrade the severity of a message from error to warning.

Related Information

- `severity` in [message](#) and in *Stylus Common UI Database Object Information Guide*

report_library_message_summary

```
report_library_message_summary {true| false}
```

Description

Default: `true`

Data_type: `bool`, read/write

Enables the printing of a summary of the messages issued during library parsing. Messages issued during the processing of the library are not included.

Example

```
Loading library /.../lib/mylib.lib
Info      : An unsupported construct was detected in this library. [LBR-40]
           : /.../lib/mylib.lib:102:16: Construct 'output_voltage' is not supported.
           : Check to see if this construct is really needed for synthesis.  Many
liberty constructs are not actually required.
Info      : An unsupported construct was detected in this library. [LBR-40]
           : /.../lib/mylib.lib:108:15: Construct 'input_voltage' is not supported.
Info      : Created nominal operating condition. [LBR-412]
           : Operating condition '_nominal_' was created for the PVT values (1.000000,
2.250000, 125.000000) in library '/.../lib/mylib.lib'.
           :The nominal operating condition represents either the nominal PVT values
if specified in the library source, or the default PVT values (1.0, 1.0, 1.0).
```

```
Message Summary for Library /.../lib/mylib.lib:
```

```
*****
```

```
An unsupported construct was detected in this library. [LBR-40]: 2
```

```
Created nominal operating condition. [LBR-412]: 1
```

```
*****
```

metric

The root attributes in the metric category are:

- [metric_advanced_url_endpoint](#)
- [metric_capture_3d_hotspots](#)
- [metric_capture_depth](#)
- [metric_capture_design_image](#)
- [metric_capture_design_image_blockages](#)
- [metric_capture_design_image_blockages_exclude_layers](#)
- [metric_capture_design_image_blockages_threshold](#)
- [metric_capture_design_image_power_intent](#)
- [metric_capture_design_image_route_drc](#)
- [metric_capture_max_drc_markers](#)
- [metric_capture_min_count](#)
- [metric_capture_overwrite](#)
- [metric_capture_pba_tns_histogram](#)
- [metric_capture_reg2reg_metrics](#)
- [metric_capture_timing_analysis_mode](#)
- [metric_capture_tns_histogram](#)
- [metric_capture_tns_histogram_buckets](#)
- [metric_capture_tns_histogram_max_slack](#)
- [metric_capture_tns_histogram_paths](#)
- [metric_capture_timing_paths](#)
- [metric_capture_timing_path_groups](#)
- [metric_capture_per_view](#)
- [metric_capture_vth_metrics](#)
- [metric_capture_vth_per_power_domain](#)

- [metric_category_default](#)
- [metric_capture_wns_average_paths](#)
- [metric_current_run_id](#)
- [metric_enable](#)
- [metric_summary_metrics](#)

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `metric` category, use the following command:

```
get_db -category metric
```

metric_advanced_url_endpoint

```
metric_advanced_url_endpoint base_url
```

Default: ""

Specifies the base URL for the advanced metric server.

metric_capture_3d_hotspots

```
metric_capture_3d_hotspots {true | false}
```

Default: true

Data_type: boolean, read/write

Specifies the depth for capturing hinst design and power metrics.

metric_capture_depth

```
metric_capture_depth integer
```

Default: 1

Specifies the depth for capturing hinst design and power metrics.

metric_capture_design_image

`metric_capture_design_image {true | false}`

Default: `true`

Captures design image.

metric_capture_design_image_blockages

`metric_capture_design_image_blockages {true | false}`

Default: `true`

Data_type: string, read/write

Captures blockages in the design image.

metric_capture_design_image_blockages_exclude_layers

`metric_capture_design_image_blockages_exclude_layers layer_names`

Default: `" "`

Data_type: string, read/write

Specifies a list of layers to exclude when capturing the blockage image metric (design.blockage.image). For example, it can be useful for skipping blockages related to metal fill.

metric_capture_design_image_blockages_threshold

`metric_capture_design_image_blockages_threshold thresholdValue`

Default: `100`

Data_type: integer, read/write

Threshold for capturing blockages in metric image.

metric_capture_design_image_power_intent

`metric_capture_design_image_power_intent {true | false}`

Default: `true`

Data_type: string, read/write

Captures power intent objects in the design image.

metric_capture_design_image_route_drc

```
metric_capture_design_image_route_drc {true | false}
```

Default: true

Data_type: string, read/write

Captures route DRC markers in the design image.

metric_capture_max_drc_markers

```
metric_capture_max_drc_markers max_number
```

Default: 1000

Specifies the maximum number of DRC markers to include in metric image.

metric_capture_min_count

```
metric_capture_min_count integer
```

Default: 1000

Specifies the minimum instance count for capturing hinst design and power metrics.

metric_capture_overwrite

```
metric_capture_overwrite {true | false}
```

Default: false

Overwrite pending metrics during `create_snapshot` category capture.

metric_capture_pba_tns_histogram

```
metric_capture_pba_tns_histogram {true | false}
```

Default: false

Captures the PBA histogram data for TNS.

metric_capture_reg2reg_metrics

```
metric_capture_reg2reg_metrics {true | false}
```

Default: false

Data_type: bool, read/write

Forces the capture of reg2reg metrics regardless of path groups.

metric_capture_timing_analysis_mode

```
metric_capture_timing_analysis_mode {gba | ipba}
```

Default: gba

Data_type: enum, read only

Controls the timing analysis mode for metric capture during create_snapshot categories setup and hold; gba, ipba.

- gba: graph based analysis
- ipba: path based analysis, retime_method == path_slew_propagation, retime_mode == exhaustive, timing_path_based_exhaustive_pba_bounded_mode == true

metric_capture_tns_histogram

```
metric_capture_tns_histogram {true | false}
```

Default: true

Captures the histogram data for TNS.

metric_capture_tns_histogram_buckets

```
metric_capture_tns_histogram_buckets integer
```

Default: 50

Specifies the number of buckets for the TNS histogram.

metric_capture_tns_histogram_max_slack

`metric_capture_tns_histogram_max_slack` *maximum_slack_value*

Default: 0

Specifies the maximum slack value for the TNS histogram. You can change its value if you need to shift the y axis.

metric_capture_tns_histogram_paths

`metric_capture_tns_histogram_paths` *integer*

Default: 10000

Specifies the number of paths to capture for the TNS histogram.

metric_capture_timing_paths

`metric_capture_timing_paths` *integer*

Default: 10

Specifies the number of paths to capture for detailed display.

metric_capture_timing_path_groups

`metric_capture_timing_path_groups` *path_group*

Default: ""

Data_type: string

Controls the path groups for which detailed timing path information (full timing path report and display) is to be captured . By default, it is empty and the tool captures all and reg2reg. If the global is set to a list of groups, the tool will use the listed groups. The related metric name is `timing.setup.paths.path_group:<group>`.

metric_capture_per_view

`metric_capture_per_view` {true | false}

Default: true

Captures timing metrics per analysis_view.

metric_capture_vth_metrics

```
metric_capture_vth_metrics {true | false}
```

Default: true

Data_type: bool, read/write

Captures vth metrics.

metric_capture_vth_per_power_domain

```
metric_capture_vth_per_power_domain {true | false}
```

Default: false

Captures vth metrics per power domain.

metric_category_default

```
metric_category_default category
```

Default: design

Specifies to capture the default metric categories if none are provided.

metric_capture_wns_average_paths

```
metric_capture_wns_average_paths paths
```

Default: ""

Data_type: string, read/write

Specifies a list of integers defining the number of paths to use for average wns metric capture.

Example

The following command:

```
set_db metric_capture_wns_average_paths "100 300 500"
```

Captures the following metrics (in part):

```
timing.setup.wns.avgpath:100.path_group:reg2reg  
timing.setup.wns.avgpath:300.path_group:reg2reg  
timing.setup.wns.avgpath:300.path_group:reg2reg
```

metric_current_run_id

```
metric_current_run_id run_id
```

Default: ""

Specifies the current unique run ID returned by the advanced metric server.

metric_enable

```
metric_enable {true | false}
```

Default: false

Specifies the current unique run ID returned by the advanced metric server.

metric_summary_metrics

```
metric_summary_metrics {flow.cputime flow.realtime timing.setup.tns timing.setup.wns}
```

Default: flow.cputime flow.realtime timing.setup.tns timing.setup.wns

Specifies summary metrics to be inherited when viewing snapshots and runs.

oa_read_write

The root attributes in the `oa_read_write` category are:

- [oa_allow_analysis_only](#)
- [oa_allow_bit_connection](#)
- [oa_allow_tech_update](#)
- [oa_bindkey_file](#)
- [oa_cell_view_dir](#)
- [oa_convert_diagonal_path_to](#)

- oa_cut_rows
- oa_display_resource_file
- oa_display_resource_file_in_library
- oa_drc_fill_purpose
- oa_enclose_quick_abstract_pins
- oa_full_layer_list
- oa_full_path
- oa_hybrid_rows
- oa_inst_placed_if_none
- oa_lib_create_mode
- oa_lock
- oa_logic_only_import
- oa_merge_trim
- oa_new_lib_compress_level
- oa_non_rectilinear_shapes
- oa_pin_purpose
- oa_push_pin_constraint
- oa_quick_abstract_for_custom_cells
- oa_read_system_reserved_layers
- oa_silently_ignore_unsupported_vias
- oa_text_purpose
- oa_tie_net
- oa_update_mode
- oa_use_virtuoso_bindkey
- oa_use_virtuoso_color
- oa_view_sub_type
- oa_write_cds_fixed_vias

- [oa_write_mask_data_locked](#)
- [oa_write_net_voltage](#)
- [oa_write_relative_path](#)

To set the value of any root attribute, use the [set_db](#) command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the [get_db](#) command:

```
get_db attribute_name
```

To get the current value of all attributes in the `oa_read_write` category, use the following command:

```
get_db -category oa_read_write
```

oa_allow_analysis_only

```
oa_allow_analysis_only {true | false}
```

Default: false

Data_type: Boolean

Allows analysis only flow.

assemble_design is not allowed when oa_allow_analysis_only is set to false and oa_update_mode is set to true, thereby locking the OpenAccess database. To allow assemble_design to proceed, you must change oa_allow_analysis_only to true. If you set oa_allow_analysis_only to true and the Tempus session is in oa_update_mode = true mode (whether set explicitly or internally on the fly), the oa_update_mode setting is ignored.

Note that there may be loss of some semantic information, such as MPPs, while saving the design when oa_allow_analysis_only is set to true. However, the saved design would be geometrically correct.

oa_allow_bit_connection

```
oa_allow_bit_connection {true | false}
```

Default: false

Data_type: Boolean

Creates databases for instances that only have bit-based terminals, without requiring the

corresponding bus terminals.

When set to `true`, it saves the connectivity information by bits if the bus connections are not available.

When set to `false`, an error will be issued if the abstract view is missing expected bus terminal information.

oa_allow_tech_update

```
oa_allow_tech_update {true | false}
```

Default: `true`

Data_type: `Boolean`

Allows the technology database to be updated. The common reason that a technology database would need to be updated is that new vias have been added to the design that are not in a parameterized form. Via masters that are a simple list of rectangles require that a "custom via" be created in the technology database. If the design library had been created with an attached technology, the value should be set to `false` allowing a reference library technology that is shared by multiple designs to have design specific vias to be added to it. For design libraries that have been created in reference mode, allowing the technology database to be updated is typically fine but normally it is best to always use parameterized vias for OpenAccess-based flows.

Using `set_db generate_special_via_symmetrical_via_only true` before any power planning or routing will typically prevent the creation of non-parameterized vias in the flow.

When `oa_allow_tech_update` is set to `false`, the creation of the OpenAccess database will fail if any custom vias would need to be created within the design library.

Related Commands and Attributes

```
generate_special_via_symmetrical_via_only
```

oa_bindkey_file

```
oa_bindkey_file fileName
```

Default: `" "`

Data_type: `String`

Maps into Tempus equivalent bindkeys. If the bindkey file is not specified, the Virtuoso default bindkey information is used. This attribute must be used with

the `oa_use_virtuoso_bindkey` attribute.

Related Commands and Attributes

`oa_use_virtuoso_bindkey`

`oa_cell_view_dir`

`oa_cell_view_dir` *string*

Default: ""

Data_type: String

Specifies the path name to the physical location of the lib/cell/view that was restored using `read_db -oa_lib_cell_view`. `oa_cell_view_dir` is a system-defined, transient (non-persistent) attribute used in the OpenAccess flow. This should be treated as a *read-only* attribute.

Related Commands

`read_db`

`oa_convert_diagonal_path_to`

`oa_convert_diagonal_path_to` {polygon | default_style}

Default: polygon

Data_type: Enum

Enables interoperability of the end style of geometries of type path segment between Virtuoso and Tempus.

This attribute applies to the diagonal path segments only.

If the end-style of a diagonal path segment is `custom`, then the end style is retained as is.

If the end style of a diagonal path segment is not `custom`, then if you set the value to `polygon`, Tempus converts the path segment into a geometrically equivalent polygon.

If the value specified is `default_style`, then the end style of the path segment is set to `default_style`. In this case, Tempus retains the semantics of the object to be a path segment.

Important: To ensure that there is no loss of semantic or geometric information, the path segments

in Virtuoso -XL and Virtuoso-GXL should use the `custom` end-style on diagonal path segments which can be created using *Create-Shape-Geometric Wire*.

oa_cut_rows

```
oa_cut_rows {true | false}
```

Default: `false`

Data_type: `Boolean`

Cuts rows against placement obstructions, block halos, or power domain fence minimum gaps.

The following command cuts rows against placement obstructions, block halos, or power domain fence minimum gaps:

```
set_db oa_cut_rows true
```

oa_display_resource_file

```
oa_display_resource_file pathName
```

Default: `""`

Data_type: `String`

Allows you to specify a specific `display.drf` file that is to be used. It overrides all the searching options and the specified file is used.

The following command searches for the `mydisplay.drf` file irrespective of the value of the `oa_display_resource_file_in_library` attribute:

```
set_db oa_display_resource_file /servers/oastore/mydisplay.drf
```

oa_display_resource_file_in_library

```
oa_display_resource_file_in_library {true | false}
```

Default: `false`

Data_type: `Boolean`

Allows the tool to search for the `display.drf` file in the technology library (or complete technology graph in case of ITDB) when set to `true`. If not found, it looks for the file in the current working directory followed by the home directory.

When set to `false`, the tool searches for the `display.drf` file first in the current working directory followed by the home directory.

oa_drc_fill_purpose

```
oa_drc_fill_purpose {gap_fill | drawing}
```

Default: `gap_fill`

Data_type: `Enum`

Specifies whether to save DRCFILL shapes to the `gap_fill` or `drawing` purpose. By default, DEF routes with + SHAPE DRCFILL are mapped to the `gapFill` purpose in OpenAccess. However, some OpenAccess tech files do not have this purpose defined so any shapes mapped to the `gapFill` purpose would not be visible in Virtuoso and would not be written out in GDS or OASIS. In such cases, set the `oa_drc_fill_purpose` attribute to `drawing` to save these shapes to the `drawing` purpose. When you do so, these shapes get an additional internal attribute so they can round-trip back to DRCFILL shapes properly.

oa_enclose_quick_abstract_pins

```
oa_enclose_quick_abstract_pins {true | false}
```

Default: `true`

Data_type: `Boolean`

Includes shapes that enclose layout pin shapes.

oa_full_layer_list

```
oa_full_layer_list {true | false}
```

Default: `true`

Data_type: `Boolean`

Imports the complete list of OpenAccess layers, when importing an OpenAccess design.

If you specify `false`, only OpenAccess layers defined with `oacMaterial` and type are imported.

oa_full_path

```
oa_full_path {true | false}
```

Default: false

Data_type: Boolean

Creates the `DEFINE` statements in the `lib.def` files using full paths. If you specify `-oa_full_path false`, the software creates the statements using relative paths.

The following command instructs the Tempus software to create the `DEFINE` statements using full paths:

```
set_db oa_full_path true
```

oa_hybrid_rows

```
oa_hybrid_rows {true | false}
```

Default: false

Data_type: Boolean

Reads hybrid rows from the Process Design Kit (PDK). When this attribute is set to `true`, the tool checks whether the sites are abutted horizontally or vertically and calculates the complex site width and height accordingly.

oa_inst_placed_if_none

```
oa_inst_placed_if_none {true | false}
```

Default: true

Data_type: Boolean

Maps the placement status of instances in the OpenAccess database to the placement status in the Tempus software. When set to `true`, the instances in the OpenAccess database that have an unknown placement status are treated as "placed" in the Tempus software. When set to `false`, instances with an unknown placement status are treated as "unplaced."

oa_lib_create_mode

```
oa_lib_create_mode {none | attach | copy | reference}
```

Default: `reference`

Data_type: `Enum`

Specifies the mode in which `write_db -oa_lib_cell_view` creates a library.

- `none`: Turns off the ability to have `write_db` create a library if it does not already exist.
- `attach`: Attaches the technology database specified by the technology library to the library that is to be created.
- `copy`: Copies the technology database specified by `techLib` to the library specified by `libName`, and saves the library to the current working directory.
- `reference`: Creates a local technology database in the design library that can be modified incrementally. This is the recommended setting to handle the case where the design may have "custom vias" added at some point by power planning/routing.

Related Commands

- `write_db`

oa_lock

```
oa_lock {true | false}
```

Default: `false`

Data_type: `Boolean`

Locks the OpenAccess database when restoring the design. When set to `true`, other tools can still access the OpenAccess database in read-only mode, but cannot edit the database as long as it is locked for the Tempus session. To remove the lock, specify the `reset_design` command, or exit the Tempus software.

Notes

- If you set the `oa_update_mode` attribute to `true`, the software automatically locks the database, even if the `oa_lock` attribute is set to `false`.
- The value of `oa_lock` cannot be changed while a design is in memory.

Related Commands

- `unlock_oa_design`

oa_logic_only_import

`oa_logic_only_import {true | false}`

Default: `false`

Data_type: `Boolean`

Controls what information the software reads when importing an OpenAccess maskLayout database.

When set to `true`, the software reads only the logical (Verilog netlist equivalent) connectivity during the `init_design` process. When set to `false` (the default value), the software automatically reads physical information, such as floorplan, placement, and routing, from the database, and updates physical net connectivity for power and ground connections.

Note: The `oa_logic_only_import` attribute applies only to OpenAccess maskLayout databases. For other types of OpenAccess databases, the software always reads only the logical connectivity.

Related Commands

- `init_design`

oa_merge_trim

`oa_merge_trim {true | false}`

Default: `true`

Data_type: `Boolean`

Merges the trim metal shapes during save when the trim metal gap is less than or equal to the `max_gap` in microns.

oa_new_lib_compress_level

`oa_new_lib_compress_level value`

Default: `1`

Data_type: Integer

Sets the compression level of a library during the library creation process. Cellviews written to a library will follow the compression level of the library and not the current `oa_new_lib_compress_level` setting.

Valid values range between 0 to 9. A value of 0 indicates that compression is turned off.

Related Commands

- `create_oa_lib`

oa_non_rectilinear_shapes

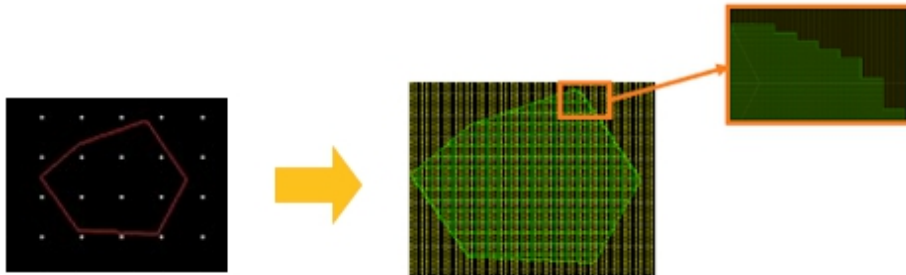
`oa_non_rectilinear_shapes {bbox | approximate}`

Default: `bbox`

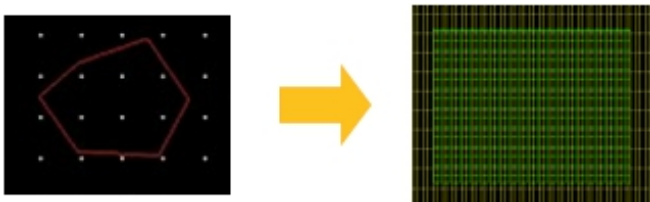
Data_type: Enum

Translates non-rectilinear shapes to rectilinear shapes while reading in an OpenAccess design:

- `approximate` - Converts non-rectilinear shapes into approximate stepped rectilinear shapes.



- `bbox` - Converts non-rectilinear shapes into bounding box shapes.



Related Commands

- `read_oa`

oa_pin_purpose

```
oa_pin_purpose {true | false}
```

Default: false

Data_type: Boolean

Writes out pin shapes with the purpose `pin`. When set to `false`, writes out pin shapes with the purpose `drawing`.

oa_push_pin_constraint

```
oa_push_pin_constraint {true | false}
```

Default: false

Data_type: Boolean

Specifies whether or not `write_db` should push an interoperable pin constraint into its corresponding blackbox abstract cellview.

Note: Interoperable pin constraints refer to pin constraints that are understood by Tempus, Virtuoso, and PVS.

Related Commands

`write_db`

oa_quick_abstract_for_custom_cells

```
oa_quick_abstract_for_custom_cells {pcells stdcells all off}
```

Default: all

Data_type: Enum

Trims design shapes of custom cells. By default, this capability is turned on for all types of custom cells. The following settings are allowed for this attribute:

- `pcells` - Trims design shapes of only custom parameterized cells (PCells).
- `stdcells` - Trims design shapes of only custom standard cells.
- `all` - Trims design shapes of all custom cells.

- `off` - Does not trim design shapes of any custom cell.

This feature needs the INVS30 (Innovus Mixed Signal Option) license. If this license is not available, the abstract that gets generated for custom cells will not work properly in Innovus.

oa_read_system_reserved_layers

`oa_read_system_reserved_layers {all | text | off}`

Default: `all`

Data_type: Enum

Reads the system-reserved layers from the PDK. The following settings are supported:

- `all` - All system-reserved layers are read.
- `text` - Only text layer is read.
- `off` - No system-reserved layer is read.

oa_silently_ignore_unsupported_vias

`oa_silently_ignore_unsupported_vias {true | false}`

Default: `false`

Data_type: Boolean

Specifies whether `read_db` should issue messages about vias that are not supported in Tempus when a Virtuoso design is read in.

oa_text_purpose

`oa_text_purpose purpose_name`

Default: `drawing`

Data_type: String

Specifies the purpose name that will be used when text labels are written.

By default, any text label created in Tempus (typically via the `create_text` command) is automatically mapped to the `drawing` purpose. Use the `oa_text_purpose` attribute to map the text

labels to a user-defined purpose.

Note: `oa_text_purpose purpose_name` overrides the default drawing purpose on output.

oa_tie_net

```
oa_tie_net {tieHighNetName tieLowNetName}
```

Default: ""

Data_type: String

If these names exist in the design locally in any module, then the global names should be changed.

Specifies the global tie high and tie low net names to be created in the OpenAccess database. The nets are used to represent the Verilog `1'b1/1'b0` connections to the instance ports.

oa_update_mode

```
oa_update_mode {true | false | auto}
```

Default: false

Data_type: Enum

Ensures that `read_db -oa_lib_cell_view` processes parameterized cells (PCells) and other custom objects correctly in the Tempus database. For information about generating/regenerating the PCell cache, refer to the "Preparing the IP Library" section in the "Technology Data Preparation" chapter of the *Mixed Signal (MS) Interoperability Guide*.

The `oa_update_mode` attribute also ensures that `write_db -oa_lib_cell_view` updates the Tempus-modified portion of an OpenAccess database without making changes to PCells or other custom objects.

In the `oa_update_mode true` mode, the tool locks the source cellview for the duration of the session.

This attribute provides the ability to import OpenAccess database created from Virtuoso that contains objects which are not understood by Tempus. Complete the digital part in Tempus and update only the objects modified by Tempus in the original OpenAccess database. If your design has any of the following objects, they would only be visible (not editable) in Tempus:

- Pcells
- Modgen cells

- Fig-groups
- Pcell vias
- RODs (Example, Multi-Part Paths, and so on)
- User defined properties on design objects such as nets, shapes (`lxStickyNet`), instances, abstract views, intermediate hierarchical instances, and so on.

You won't be allowed to edit or modify these objects.

On using the `oa_update_mode` attribute, the property on shapes (for example, `lxStickyNet`) and nets created in VLS-XL / GXL would be kept as-is.

In case of designs where parameterized vias are used, Tempus understands these attributes and evaluates them on-the-fly to create an exact shape and orientation of the cuts.

If the `oa_update_mode` attribute is set to `auto` and the cellview contains objects that are not supported in Tempus, the software internally enables update mode with a message such as follows:

```
INFO: Objects have been found in the cellview requiring updateMode to be enabled so  
that there is no loss of semantic information when the cellview is saved.
```

Note that enabling update mode can affect `write_db` performance.

Notes

- The value of the `oa_update_mode` attribute cannot be changed while a design is in memory. After this attribute has been set to `true` before importing an OpenAccess design into Tempus, you cannot convert the `oa_update_mode` attribute to `false` in any new Tempus session.
- The on-disk OpenAccess cellview is locked between the restore and save steps when the `oa_update_mode` attribute is set to `true`.
- In `oa_update_mode auto`, the lock is applied at the time that `read_oa` or `read_db - oa_lib_cell_view` starts and released at the end of the command if no objects were encountered that required `updateMode` to be enabled.

oa_use_virtuoso_bindkey

```
oa_use_virtuoso_bindkey {true | false}
```

Default: `false`

Data_type: Boolean

Allows you to use Virtuoso bindkeys.

oa_use_virtuoso_color

`oa_use_virtuoso_color {true | false}`

Default: `false`

Data_type: Boolean

Allows you to display colors and stipples for the design layers (including full custom layers). Using this attribute, Tempus utilizes the same colors and stipples that you would see if you open the design in Virtuoso.

oa_view_sub_type

`oa_view_sub_type {vce | vx1 | none}`

Default: `vx1`

Data_type: Enum

Indicates which Virtuoso application should be used for viewing the layout cellviews in the saved OpenAccess database.

When set to `vce`, cellviews are opened in Virtuoso Chip Editor.

When set to `vx1`, cellviews are opened in Virtuoso XL.

When set to `none`, it indicates that the OpenAccess database can be edited by any editor within Virtuoso platform based on its license availability.

oa_write_cds_fixed_vias

`oa_write_cds_fixed_vias {new_vias_only | off}`

Default: `new_vias_only`

Data_type: enum, read/write

Controls how fixed vias are created when saving a design with custom vias from Innovus to OpenAccess:

- `new_vias_only` - Any newly created custom via is saved as a fixed via while any pre-existing custom via in the Incremental Technology Database (ITDB) is reused when saving the design.
- `off` - No `cdsFixedVias` are created. All fixed/custom vias in the design are saved as

oaCustomVia.

This feature needs the Mixed Signal Option license. If this license is not available, the `off` value will be used and no fixed vias will be created.

oa_write_mask_data_locked

```
oa_write_mask_data_locked {true | false}
```

Default: `false`

Data_type: Boolean

Sets the shapes with mask coloring (mask values other than 0) to locked in the saved cellview, when set to `true`.

By default, `write_db` write out shapes with their mask coloring information in an unlocked state. Some tools treat the unlocked color information as the recommended color. However, for some foundry processes, shapes with unlocked mask coloring information are written to GDSII/Stream as gray (uncolored). In such cases, set the `oa_write_mask_data_locked` parameter to `true` to lock the mask coloring of shapes explicitly.

Note: This option affects only the saving of shapes with non-zero mask coloring values. Review the map file that will be used to create GDSII/Stream output from the OpenAccess cellview to determine if the map file requires shapes to be locked for them to be written out as colored.

Related Commands

- `write_db`

oa_write_net_voltage

```
oa_write_net_voltage {true | false}
```

Default: `false`

Data_type: Boolean

Saves the minimum and maximum net voltage across all loaded timing views to every net. This is useful for Virtuoso, and other tools that may read the cellview, to understand the minimum and maximum net voltage without needing to interpret the CPF or .lib file to get that information. The round-trip of the information into Tempus is not dependent on this setting.

oa_write_relative_path

oa_write_relative_path {true | false}

Default: true

Data_type: Boolean

Specifies whether or not to save the relative path in the configuration file. Defines the current working directory (\$cwd) in the configuration file as ".". When set to `true`, a design can be restored, even if the directory path of the design file is changed.

opt

The root attributes in the `opt` category are:

- [auto_partition](#)
- [auto_ungroup](#)
- [auto_ungroup_max_threshold](#)
- [avoid_tied_inputs](#)
- [boundary_optimize_constant_hpins](#)
- [boundary_optimize_equal_opposite_hpins](#)
- [boundary_optimize_feedthrough_hpins](#)
- [boundary_optimize_invert_hpins](#)
- [boundary_optimize_invert_hpins_rename_nets](#)
- [combinational_sequential_merge_message_threshold](#)
- [delete_flops_on_preserved_net](#)
- [delete_hier_insts_on_preserved_net](#)
- [delete_unloaded_insts](#)
- [delete_unloaded_seqs](#)
- [double_cell_search_pattern](#)
- [drc_first](#)
- [drc_max_cap_first](#)

- drc_max_fanout_first
- drc_max_trans_first
- enable_strict_percent_control
- fix_min_drcs
- map_clock_tree
- map_drc_first
- map_latch_allow_async_decomp
- map_prefer_non_inverted_clock_line
- map_respect_rtl_clk_phase
- merge_combinational_hier_instances
- merge_non_scan_to_scan_flops
- minimize_uniquify
- multibit_allow_unused_bits
- multibit_area_power_scoring
- multibit_aware_seq_mapping
- multibit_aware_seq_mapping_higher_priority
- multibit_debug
- multibit_invert_clock_phase
- multibit_mapping_effort_level
- multibit_predefined_allow_unused_bits
- multibit_unused_input_value
- opt_activity_refresh_args
- opt_activity_refresh_flow
- opt_activity_refresh_stages
- opt_adaptive_subd1_flow
- opt_add_always_on_feed_through_buffers
- opt_add_insts

- `opt_add_ports`
- `opt_add_repeater_report_failure_reason`
- `opt_all_end_points`
- `opt_allow_floating_outputs`
- `opt_allow_only_cell_swapping`
- `opt_allow_multi_bit_on_flop_with_sdc`
- `opt_area_recovery`
- `opt_area_recovery_setup_target_slack`
- `opt_boundary_optimize_invert_hpins_rename_extension`
- `opt_clock_traverse_by_level`
- `opt_clone_insts_list`
- `opt_concatenate_default_and_user_prefixes`
- `opt_consider_routing_congestion`
- `opt_constant_inputs`
- `opt_constant_nets`
- `opt_delete_insts`
- `opt_detail_drv_failure_reason`
- `opt_detail_drv_failure_reason_max_num_nets`
- `opt_dont_use_qn_sequential_pins`
- `opt_down_size_insts`
- `opt_drv`
- `opt_drv_margin`
- `opt_drv_with_miller_cap`
- `opt_duplicate_cte_constrained_hport`
- `opt_early_hold_fixing`
- `opt_enable_always_on_type_in_remove_assign`
- `opt_enable_clock_pulse_width_checks`

- [opt_enable_data_to_data_checks](#)
- [opt_enable_drv_failure_reason_warn](#)
- [opt_enable_podv2_clock_opt_flow](#)
- [opt_enable_restructure](#)
- [opt_enable_targeted_synthesis](#)
- [opt_fix_fanout_load](#)
- [opt_flop_pins_report](#)
- [opt_flops_report](#)
- [opt_hier_add_antenna_cell](#)
- [opt_hier_opt_stage](#)
- [opt_high_effort_cells](#)
- [opt_hold_allow_overlap](#)
- [opt_hold_allow_resize](#)
- [opt_hold_allow_setup_tns_degradation](#)
- [opt_hold_ignore_path_groups](#)
- [opt_hold_cells](#)
- [opt_hold_on_excluded_clock_nets](#)
- [opt_hold_slack_threshold](#)
- [opt_hold_target_slack](#)
- [opt_honor_density_screen](#)
- [opt_honor_fences](#)
- [opt_icg_enable_pin_rebuffering](#)
- [opt_leakage_to_dynamic_ratio](#)
- [opt_max_density](#)
- [opt_max_length](#)
- [opt_move_insts](#)
- [opt_multi_bit_combinational_mode](#)

- `opt_multi_bit_combinational_opt`
- `opt_multi_bit_combinational_merge_timing_effort`
- `opt_multi_bit_combinational_split_timing_effort`
- `opt_multi_bit_flop_merge_bank_label_inference`
- `opt_multi_bit_flop_merge_timing_effort`
- `opt_multi_bit_flop_opt`
- `opt_multi_bit_flop_name_prefix`
- `opt_multi_bit_flop_name_separator`
- `opt_multi_bit_flop_name_suffix`
- `opt_multi_bit_flop_reorder_bits`
- `opt_multi_bit_flop_split_post_cts`
- `opt_multi_bit_flop_split_post_route`
- `opt_multi_bit_flop_split_report_failure_reason`
- `opt_multi_bit_flop_split_timing_effort`
- `opt_multi_bit_merge_scan_head`
- `opt_multi_bit_split_scan_head`
- `opt_multi_bit_unused_bits`
- `opt_multi_bit_unused_bit_count`
- `opt_multibit_allow_shift_register_head_flop_merge`
- `opt_new_inst_prefix`
- `opt_new_net_prefix`
- `opt_one_pass_lec`
- `opt_pin_swapping`
- `opt_podv2_flow_effort`
- `opt_post_route_allow_overlap`
- `opt_post_route_area_reclaim`
- `opt_post_route_art_flow`

- `opt_post_route_check_antenna_rules`
- `opt_post_route_drv_recovery`
- `opt_post_route_fix_clock_drv`
- `opt_post_route_fix_glitch`
- `opt_post_route_fix_si_transitions`
- `opt_post_route_hold_recovery`
- `opt_post_route_report_si_transitions`
- `opt_post_route_setup_recovery`
- `opt_power_effort`
- `opt_pre_route_ndr_aware`
- `opt_preserve_all_sequential`
- `opt_preserve_hpin_function`
- `opt_print_ports_nets_preserved_for_callback`
- `opt_remove_redundant_insts`
- `opt_report_multi_bit_unmerged_reasons`
- `opt_resize_flip_flops`
- `opt_resize_level_shifter_and_iso_insts`
- `opt_resize_power_switch_insts`
- `opt_retime_move_mux_loop_with_register`
- `opt_retime_register_naming_suffix`
- `opt_route_opt_recovery`
- `opt_sequential_genus_restructure_report_failure_reason`
- `opt_setup_target_slack`
- `opt_skew`
- `opt_skew_apply_delay_limits_to_full_flow`
- `opt_skew_ccopt`
- `opt_skew_post_route`

- `opt_skew_pre_cts`
- `opt_skew_delay_pre_cts`
- `opt_skew_macro_only`
- `opt_skew_max_allowed_delay`
- `opt_skew_min_allowed_delay`
- `opt_skew_no_boundary`
- `opt_target_based_opt_file`
- `opt_target_based_opt_hold_file`
- `opt_target_based_opt_file_only`
- `opt_tied_inputs`
- `opt_time_design_compress_reports`
- `opt_time_design_expanded_view`
- `opt_time_design_num_paths`
- `opt_time_design_report_net`
- `opt_time_design_report_path_type`
- `opt_time_design_vertical_timing_summary`
- `opt_tns`
- `opt_unfix_clock_insts`
- `opt_use_multi_output_gates`
- `opt_verbose`
- `optimize_constant_0_flops`
- `optimize_constant_1_flops`
- `optimize_constant_across_preserved`
- `optimize_constant_feedback_seqs`
- `optimize_constant_latches`
- `optimize_merge_flops`
- `optimize_merge_latches`

- `optimize_seq_x_to`
- `optimize_yield`
- `percent_control_tolerance_for_map`
- `phys_early_skew_flow`
- `preserve_combinational_loop_ports_nets`
- `propagate_constant_from_timing_model`
- `proto_feasible_target`
- `proto_feasible_target_adjust_slack_pct`
- `proto_feasible_target_threshold`
- `proto_feasible_target_threshold_clock_pct`
- `retime_async_reset`
- `retime_effort_level`
- `retime_optimize_reset`
- `retime_verification_flow`
- `syn_exact_match_sequential_async_control`
- `st_launch_wait_time`
- `timing_library_ui_respect_preserve`
- `timing_library_use_max_cap_look_up_table`
- `tns_critical_range`

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `opt` category, use the following command:

```
get_db -category opt
```

auto_partition

```
auto_partition {true | false}
```

Description

Default: true

Data_type: bool, read/write

Activates automatic internal partitioning of large designs for efficient synthesis. This attribute must be set before synthesis.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [synthesize_design](#)

auto_ungroup

```
auto_ungroup {none | both | area | timing}
```

Description

Default: both

Data_type: enum, read/write

Activates automatic ungrouping to improve area and timing optimization during synthesis. This attribute must be specified before synthesis.

none	Ungrouping will not be performed.
both	Ungrouping will be performed with an emphasis on optimizing both timing and area.

Aggressive ungrouping of user hierarchies happens during:

- RTL optimization—`synthesize_design` with high effort
- Technology mapping—`synthesize_design` with high effort

Some ungrouping can occur with low or medium effort as well.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- `ungroup_ok` in [module](#) in *Stylus Common UI Database Object Information Guide*
- [synthesize_design](#)
- "Automatic Ungrouping" in [Grouping and Ungrouping Objects](#)

auto_ungroup_max_threshold

```
auto_ungroup_max_threshold <integer>
```

Description

Default: 0

Data_type: integer, read/write

Specifies that any module with more instances than the specified max threshold value must not be ungrouped by the auto-ungroup feature. But allows ungrouping for the instances within the specified max threshold value.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

avoid_tied_inputs

```
avoid_tied_inputs {true | false}
```

Description

Default: `false`

Data_type: `bool`, `read/write`

When enabled, the mapper will try to avoid mapping to cells with tied or connected inputs, when possible. This restriction impacts QoR.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [synthesize_design](#)

boundary_optimize_constant_hpins

```
boundary_optimize_constant_hpins {true | false | inherited}
```

Applies to:

- `root`
- `module`
- `hpin`

Description

Default: `true` (root), `inherited` (module and hpin)

Data_type: `bool` (root), `enum` (module and hpin), `read/write`

Controls constant propagation through the hierarchical boundary pins.

You can specify the following values:

<code>true</code>	Allows constant propagation.
<code>false</code>	Prevents constant propagation.
<code>inherited</code>	Inherits the value of the <code>boundary_optimize_constant_hpins</code> root attribute.

If the `boundary_opto` attribute on a module is set and disables boundary optimization on the pins, the setting of this attribute is ignored for that module. If the `boundary_optimize_constant_hpins` attribute on an instance pin or module is set to `false` or `true` (does not have the `inherited` value), the setting of this root attribute is ignored for that pin or module.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- The following attributes in [hpin](#) and [module](#) in *Stylus Common UI Database Object Information Guide*:
 - `boundary_opto`
 - `boundary_optimize_equal_opposite_hpins`
 - `boundary_optimize_feedthrough_hpins`
 - `boundary_optimize_invert_hpins`
- [synthesize_design](#)
- [Setting Boundary Optimization](#) in *Innovus Stylus Common UI User Guide*

boundary_optimize_equal_opposite_hpins

`boundary_optimize_equal_opposite_hpins {true | false | inherited}`

Applies to:

- `root`
- `module`
- `hpin`

Description

Default: `true` (root), `inherited` (module and hpin)

Data_type: `bool` (root), `enum` (module and hpin), read/write

Controls collapsing of equal and opposite hierarchical boundary pins. Two hierarchical boundary pins are considered equal (opposite), if the tool determines that they always have the same (opposite or inverse) logic value.

You can specify the following values:

<code>true</code>	Allows collapsing of the hierarchical boundary pins.
<code>false</code>	Prevents collapsing of the hierarchical boundary pins.

inherited

Inherits the value of the `boundary_optimize_equal_opposite_hpins` root attribute.

If the `boundary_opto` attribute on a module is set and disables boundary optimization on the pins, the setting of this attribute is ignored for that module. If the `boundary_optimize_equal_opposite_hpins` attribute on an instance pin or module is set to false or true (does not have the inherited value), the setting of this root attribute is ignored for that pin or module.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- The following attributes in [hpin](#) and [module](#) in *Stylus Common UI Database Object Information Guide*:
 - `boundary_opto`
 - `boundary_optimize_constant_hpins`
 - `boundary_optimize_feedthrough_hpins`
 - `boundary_optimize_invert_hpins`
- [synthesize_design](#)
- [Setting Boundary Optimization](#) in *Innovus Stylus Common UI User Guide*

boundary_optimize_feedthrough_hpins

```
boundary_optimize_feedthrough_hpins {true | false | inherited}
```

Applies to:

- root
- module
- hpin

Description

Default: `true` (root), `inherited` (module and hpin)

Data_type: `bool` (root), `enum` (module and hpin), `read/write`

Controls optimization of feedthrough pins. Hierarchical boundary pins are feedthrough pins, if output pins have always the same (or inverted) logic value as an input pin. Such feedthrough pins can be routed around the module and no connections or logic is needed inside the module for these pins. To disable this type of boundary optimization, set the attribute to `false`.

You can specify the following values:

<code>true</code>	Allows optimization of feedthrough pins.
<code>false</code>	Prevents optimization of feedthrough pins.
<code>inherited</code>	Inherits the value of the <code>boundary_optimize_feedthrough_hpins</code> module attribute.

If the `boundary_opto` attribute on a module is set and disables boundary optimization on the pins, the setting of this attribute is ignored for that module. If the `boundary_optimize_feedthrough_hpins` attribute on an instance pin or module is set to `false` or `true` (does not have the `inherited` value), the setting of this root attribute is ignored for that pin or module.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- The following attribute in [module](#) in *Stylus Common UI Database Object Information Guide*:
 - `boundary_opto`
- `boundary_optimize_constant_hpins`
- `boundary_optimize_equal_opposite_hpins`
- `boundary_optimize_invert_hpins`
- `synthesize_design`
- [Setting Boundary Optimization](#) in *Innovus Stylus Common UI User Guide*

boundary_optimize_invert_hpins

`boundary_optimize_invert_hpins {true | false | inherited}`

Applies to:

- `root`
- `module`
- `hpin`

Description

Default: `false`(`root`), `inherited` (`module` and `hpin`)

Data_type: `bool` (`root`), `enum` (`module` and `hpin`), `read/write`

Controls hierarchical boundary pin inversion. By default, hierarchical boundary pin inversion is disabled. If set to `true`, the boundary pin inversion is controlled by the `boundary_opto` module attributes.

You can specify the following values:

<code>true</code>	Allows boundary pin inversion.
<code>false</code>	Prevents boundary pin inversion.

inherited	Inherits the value of the <code>boundary_optimize_invert_hpins</code> module attribute.
-----------	---

- If the `boundary_opto` attribute on a module is set and disables boundary optimization on the pins, the setting of this attribute is ignored for that module. If the `boundary_optimize_invert_hpins` attribute on an instance pin or module is set to false or true (does not have the inherited value), the setting of this root attribute is ignored for that pin or module.
- If this attribute is set to true, the `write_do_lec` command adds the following LEC command to the dofile:

```
SET Naming Rule _BAR -Inverted_pin_extension
```

This LEC command specifies the string that is appended to pin names and net names by the tool when hierarchical boundary pins are inverted during synthesis. To modify this string use the `boundary_optimize_invert_hpins_rename_extension` root attribute. In the command above, the `_BAR` value corresponds to the default value of this attribute.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- The following attribute in [module](#) in *Stylus Common UI Database Object Information Guide*:
 - `boundary_opto`
- `boundary_optimize_constant_hpins`
- `boundary_optimize_equal_opposite_hpins`
- `boundary_optimize_feedthrough_hpins`
- `synthesize_design`
- [Setting Boundary Optimization](#) in *Innovus Stylus Common UI User Guide*
- [Interfacing with Conformal Logical Equivalence Checker](#) in *Innovus Synthesis Conformal Interface Guide*

boundary_optimize_invert_hpins_rename_nets

`boundary_optimize_invert_hpins_rename_nets {true | false}`

Description

Default: true

Data_type: bool, read/write

Controls renaming of nets driven by the inverted hierarchical boundary pins. By default, the nets will be renamed.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- [boundary_optimize_invert_hpins](#)
- [opt_boundary_optimize_invert_hpins_rename_extension](#)
- [synthesize_design](#)

combinational_sequential_merge_message_threshold

`combinational_sequential_merge_message_threshold` <integer>

Description

Default: 10

Data_type: integer, read/write

Enables the printing of detailed messages when hierarchical instances with more cells than the specified threshold value are merged.

Example

```
set_db combinational_sequential_merge_message_threshold 2
2
```

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- [synthesize_design](#)
- [merge_combinational_hier_instances](#)

delete_flops_on_preserved_net

```
delete_flops_on_preserved_net {true | false}
```

Description

Default: `true`

Data_type: `bool`, `read/write`

Controls the deletion of load flip-flops on a preserved net. By default, the flops can be deleted.

Set this attribute to `false` to keep load flip-flops even if they do not drive any primary output, can be replaced with a constant, or can be deleted in some other optimization.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [synthesize_design](#)

delete_hier_insts_on_preserved_net

```
delete_hier_insts_on_preserved_net {true | false}
```

Description

Default: `true`

Data_type: `bool`, `read/write`

Controls the deletion of empty hierarchical instances driven by a preserved net.

Set this attribute to `false` to keep the empty modules on a preserved net, even if they do not drive any primary output.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [synthesize_design](#)

delete_unloaded_insts

```
delete_unloaded_insts {true | false | inherited}
```

Applies to:

- module
- root

Description

Default: `true` (root), `inherited` (module)

Data_type: `bool` (root), `enum` (module), `read/write`

Controls the deletion of unloaded hierarchical instances. You can specify one of the following values:

<code>true</code>	Removes logic if none of the outputs are connected.
-------------------	---

false	Preserves any pre-existing hierarchical instances. If you just want to maintain mapped and unmapped sequential instances, set the <code>delete_unloaded_seqs</code> attribute to <code>false</code>. Unmapped (boolean) non-hierarchical instances cannot be rescued if they are unloaded.
inherited	Inherits the value from the <code>delete_unloaded_insts</code> root attribute.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- `synthesize_design`
- `delete_unloaded_seqs`

delete_unloaded_seqs

`delete_unloaded_seqs {true | false | inherited}`

Applies to:

- `design`
- `module`
- `root`

Description

Default: `true` (root), `inherited` (design and module)

Data_type: bool (root), enum (design and module), read/write

Controls the deletion of unloaded sequential instances.

You can specify one of the following values:

true	Removes flip-flops and logic if they are not connected (transitively fanning out to output ports).
false	Prevents the deletion of unloaded sequential instances.
inherited	Inherits the value from the <code>delete_unloaded_seqs</code> root attribute.

This attribute only affects the `synthesis_design` command, while the `hdl_preserve_unused_registers` attribute only affects the `elaborate_design` command.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- `boundary_opto` in [module](#) in *Stylus Common UI Database Object Information Guide*
- `hdl_preserve_unused_registers` in [hdl Category Attributes](#)
- [synthesize_design](#)

double_cell_search_pattern

```
double_cell_search_pattern <string>
```

Description

Default:

Data_type: string, read/write

In case LEF is not provided, a search pattern is needed to identify which cells are double cells.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

drc_first

```
drc_first {true | false}
```

Description

Default: `false`

Data_type: `bool`, `read/write`

Specifies whether to give *all* design rule constraints higher priority than the timing constraints.

The design rule constraints are optimized in the following order:

1. `max_transition`
2. `max_capacitance`
3. `max_fanout`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- `report_constraint`
- `report_timing`
- `synthesize_design`
- The following attributes in `design` in *Stylus Common UI Database Object Information Guide*
 - `ignore_library_drc`
 - `max_capacitance`
 - `max_fanout`
 - `max_transition`
- `drc_max_cap_first`
- `drc_max_fanout_first`
- `drc_max_trans_first`
- `map_drc_first`
- `timing_disable_internal_inout_net_arcs`

drc_max_cap_first

`drc_max_cap_first {true | false}`

Description

Default: `false`

Data_type: `bool`, read/write

Specifies whether to give the maximum capacitance cost higher priority than the timing constraints. To optimize the maximum capacitance cost before the timing constraints, set the `drc_first` attribute to `false` and this attribute to `true`. In this case, the two other design rule constraints are optimized after the timing constraints have been taken into account.

If you set the `drc_max_cap_first`, `drc_max_fanout_first`, and `drc_max_trans_first` attributes to `true`, all design rule constraints are optimized before the timing constraints and they are optimized in the following order:

1. `max_transition`
2. `max_capacitance`
3. `max_fanout`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- `report_constraint`
- `report_timing`
- `synthesize_design`
- `ignore_library_drc` and `max_capacitance` in [design](#) in *Stylus Common UI Database Object Information Guide*
- `drc_first`
- `drc_max_fanout_first`
- `drc_max_trans_first`
- `timing_disable_internal_inout_net_arcs`

drc_max_fanout_first

```
drc_max_fanout_first {true | false}
```

Description

Default: `false`

Data_type: `bool`, read/write

Specifies whether to give the maximum fanout cost higher priority than the timing constraints.

To optimize the maximum fanout cost before the timing constraints, set the `drc_first` attribute to `false` and this attribute to `true`. In this case, the two other design rule constraints are optimized after the timing constraints have been taken into account.

If you set the `drc_max_cap_first`, `drc_max_fanout_first`, and `drc_max_trans_first` attributes to `true`, all design rule constraints are optimized before the timing constraints and in the following order:

1. `max_transition`
2. `max_capacitance`
3. `max_fanout`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- `report_constraint`
- `report_timing`
- `synthesize_design`
- `ignore_library_drc` and `max_fanout` in `design` in *Stylus Common UI Database Object Information Guide*
- `drc_first`
- `drc_max_cap_first`
- `drc_max_trans_first`
- `timing_disable_internal_inout_net_arcs`

drc_max_trans_first

`drc_max_trans_first {true | false}`

Description

Default: `false`

Data_type: `bool`, read/write

Specifies whether to consider the maximum transition cost before the timing constraints.

To optimize the maximum transition cost before the timing constraints, set the `drc_first` attribute to `false` and this attribute to `true`. In this case, the two other design rule constraints are optimized after the timing constraints have been taken into account.

If you set the `drc_max_cap_first`, `drc_max_fanout_first`, and `drc_max_trans_first` attributes to `true`, the design rule constraints are optimized in the following order:

1. `max_transition`
2. `max_capacitance`
3. `max_fanout`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- `report_constraint`
- `report_timing`
- `synthesize_design`
- `ignore_library_drc` in [design](#) in *Stylus Common UI Database Object Information Guide*
- `drc_first`
- `drc_max_cap_first`
- `drc_max_fanout_first`
- `timing_disable_internal_inout_net_arcs`

enable_strict_percent_control

```
enable_strict_percent_control {true | false}
```

Description

Default: `false`

Data_type: `bool`, read/write

Threshold value to be used as hard limit.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```


- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

fix_min_dracs

```
fix_min_dracs {true | false}
```

Description

Default: `false`

Data_type: `bool`, read/write

When set to `true`, fixes the minimum design rule costs based on calculations in the library. Specifically, the tool fixes the minimum capacitance, minimum transition, and minimum fanout design rule violations by resizing the driver and its loads.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [synthesize_design](#)

map_clock_tree

```
map_clock_tree {true | false}
```

Description

Default: `false`

Data_type: `bool`, read/write

Enables mapping of clock logic to a specified set of library cells. You should specify allowed clock tree cells by using the following attributes: `cts_buffer_cells`, `cts_inverter_cells`, `cts_clock_gating_cells`, and `cts_logic_cells`. Specification of clock tree cells and enabling mapping of clock logic should be done before calling `synthesize_design`. At the end of `synthesize_design` all clock tree logic will then be mapped to the specified set of cells.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- `synthesize_design`
- `cts_buffer_cells`
- `cts_clock_gating_cells`
- `cts_inverter_cells`
- `cts_logic_cells`

map_drc_first

```
map_drc_first {true | false}
```

Description

Default: `false`

Data_type: `bool`, read/write

Specifies whether to use the design rule constraints as a preferred costing factor for selecting cells from the library during mapping. These constraints are considered with the other costs, such as area, timing and power, but by default they are not given the highest weight.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [synthesize_design](#)
- [drc_first](#)

map_latch_allow_async_decomp

```
map_latch_allow_async_decomp {true | false}
```

Description

Default: `false`

Data_type: `bool`, read/write

If set to `true`, the tool will implement asynchronous control logic outside the latch using combinational gates if the library does not have latch libcells with the required asynchronous functionality. For example, a reset latch could be implemented using a simple latch libcell with combinational logic driving the `d` and `ena` inputs.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [synthesize_design](#)

map_prefer_non_inverted_clock_line

map_prefer_non_inverted_clock_line {true | false}

Description

Default: true

Data_type: bool, read/write

Controls the insertion of inverters on the clock line. By default, the tool tries to avoid inserting inverters on the clock line. If you set this attribute to `false`, the tool chooses the best solution for QoR, which might include inserting inverters.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [synthesize_design](#)

map_respect_rtl_clk_phase

map_respect_rtl_clk_phase {true | false}

Description

Default: false

Data_type: bool, read/write

If set to `true`, the tool will use sequential elements with the same sensitivity, as specified in the RTL:

- In case of high-level RTL using `@posedge` or `@negedge`
This applies to flip-flops only. The desired clock edge of the target flip-flop is derived from the `@xxxedge`.

- In case of netlist RTL
If the tool finds already instantiated library cells when reading an RTL description it remembers the clock edge for flip-flops or the enable active level for latches and tries to keep that in subsequent remappings.

If the library has no libcells with the required sensitivity, the tool will use any available libcells.

By default, the tool will use flip-flop libcells with a certain clock phase that avoid adding an inverter for the clock.

Example

Consider the following module in RTL:

```
module mix_edge ( clk, d0, q0, q1, q0_inv, q1_inv, d1, d2,d3);
input clk, d0, d1, d2, d3;
output q0, q1, q0_inv, q1_inv;
reg q0, q1, q0_inv, q1_inv;
wire clk_n;

assign clk_n = ~clk;
always @(negedge clk)
    q0 = d0;
always @(posedge clk)
    q1 = d1;
always @(negedge clk_n)
    q0_inv = d2;
always @(posedge clk_n)
    q1_inv = d3;
endmodule
```

- If you use the default setting of false for `map_respect_rtl_clk_phase`, the tool will use the following mappings:
 - `q0_reg` will be mapped to `FFRNQ1`, a negative-edge triggered flip-flop
 - `q1_reg` will be mapped to `FFPQ1`, a positive-edge triggered flip-flop
 - `q0_inv_reg` will be mapped to `FFPQ1`, a positive-edge triggered flip-flop
 - `q1_inv_reg` will be mapped to `FFRNQ1`, a negative-edge triggered flip-flop
- If you set `map_respect_rtl_clk_phase` to true, the tool uses the following mappings:
 - `q0_reg` will be mapped to `FFRNQ1`, a negative-edge triggered flip-flop

- `q1_reg` will be mapped to `FFPQ1`, a positive-edge triggered flip-flop
- `q0_inv_reg` will be mapped to `FFRNQ1`, a negative-edge triggered flip-flop + inverter
- `q1_inv_reg` will be mapped to `FFPQ1`, a positive-edge triggered flip-flop + inverter

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [synthesize_design](#)
- [map_prefer_non_inverted_clock_line](#)

merge_combinational_hier_instances

```
merge_combinational_hier_instances {true | false}
```

Description

Default: `true`

Data_type: `bool`, read/write

Allow merging of combinational hierarchical instances.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [synthesize_design](#)

merge_non_scan_to_scan_flops

`merge_non_scan_to_scan_flops {true | false}`

Description

Default: `false`

Data_type: `bool`, read/write

If set to `true`, it enables merging of non scan flops to scan flops during multibit merging.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

`set_db attribute_name value`

- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

minimize_uniquify

`minimize_uniquify {true | false | inherited}`

Applies to:

- `module`
- `root`

Description

Default: `false` (root), `inherited` (module)

Data_type: `bool` (root), `enum` (module), read/write

Controls uniquification of all modules in the design. By default, the tool uniquifies multiple instantiations with different context (timing, constants, and so on) to deliver the best QoR.

Set this attribute to `true` to limit the scenarios of uniquification of multiply-instantiated modules and potentially improve runtime with a trade-off against QoR. This will not necessarily prevent uniquification from algorithmic considerations of the design topology.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [synthesize_design](#)

multibit_allow_unused_bits

```
multibit_allow_unused_bits {true | false}
```

Description

Default: `true`

Data_type: `bool`, read/write

Controls whether a mapped multibit instance can have one or more unused bits. By default, the tool will try to merge the single-bit registers to multibit instances without unused bits. If during merging, the tool cannot find a suitable multibit lib_cell with a width that matches the left-over bits of a bus, it will choose the closest matching width which leaves the minimum unused bits. If the attribute is set to `false`, any merging which results in unused (undriven/unloaded) bits will be prevented and thus reduce the multibit merging coverage.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```


- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [synthesize_design](#)
- `use_multibit_cells` in [root](#) in *Stylus Common UI Database Object Information Guide*
- `bit_width` in [lib_cell](#) in *Stylus Common UI Database Object Information Guide*
- [Using the Multi-Bit Flip-Flop Flow](#) in *Innovus Stylus Common UI User Guide*

multibit_area_power_scoring

```
multibit_area_power_scoring {none | area_only | area_power}
```

Description

Default: none

Data_type: enum, read/write

Controls whether area and power are to be scored in merging.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

multibit_aware_seq_mapping

```
multibit_aware_seq_mapping {auto | true | false}
```

Description

Default: auto

Data_type: enum, read/write

The mapper prefers sequential cells which have corresponding multibit variants. If the library does not have suitable cells with multibit equivalents, all sequential cells are used. The tool does not perform this optimization for the instances where multibit merging is disabled with 'dont_merge_multibit'.

This attribute can have the following values:

auto	Enables the multibit-aware mapping only if <code>use_multibit_cells</code> is true.
true	Enables the multibit-aware mapping independent from <code>use_multibit_cells</code> .
false	Disables the multibit-aware mapping.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- [synthesize_design](#)
- `use_multibit_cells` in [root](#) in *Stylus Common UI Database Object Information Guide*
- `dont_merge_multibit` and `merge_multibit` in [inst](#) in *Stylus Common UI Database Object Information Guide*

multibit_aware_seq_mapping_higher_priority

`multibit_aware_seq_mapping_higher_priority {true | false}`

Description

Default: false

Data_type: bool, read/write

Prioritize `multibit_aware_seq_mapping` over other constraints such as `avoid_tied_inputs`, `map_prefer_non_inverted_clock_line`, `syn_exact_match_sequential_async_control`, or `syn_exact_match_sequential_sync_control`. When enabled, `multibit_aware_seq_mapping` still has lower priority than `map_to_register`, and `map_to_ser_register` constraints. Additionally, scan and state retention requirements will also be honored.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- [synthesize_design](#)
- `multibit_aware_seq_mapping`
- [Using the Multi-Bit Flip-Flop Flow](#) in *Innovus Stylus Common UI User Guide*

multibit_debug

```
multibit_debug {true | false}
```

Description

Default: `false`

Data_type: `bool`, read/write

Controls whether to write debug information related to multibit mapping to the logfile.

Example

Consider the following netlist after mapping and before incremental synthesis:

```
module test(resetn, clk, in, out);  
    input resetn, clk;
```

```
input [1:0] in;
output [1:0] out;

DFCF cnt_0_reg[0] (.CDN (resetn), .CP (clk), .D(in[0]), .Q (out[0] ));
DFCF cnt_0_reg[1] (.CDN (resetn), .CP (clk), .D(in[1]), .Q (out[1] ));
endmodule

module top(resetn, clk, in, out);
input resetn, clk;
input [1:0] in;
output [1:0] out;
test t(resetn, clk, in, out);
endmodule
```

When `multibit_debug` is set to true, the following debug information is dumped in logfile during multibit mapping:

- If multibit instance is created:

```
Trying to make multibit bank CDN_MBIT_cnt_0_reg[0]_MB_cnt_0_reg[1] out of
cnt_0_reg[0]          (DFCF)
cnt_0_reg[1]          (DFCF)
CDN_MBIT_cnt_0_reg[0]_MB_cnt_0_reg[1] multibit bank of type DUALDFSF accepted
in sandbox.
```

- If multibit instance is not created:

```
Trying to make multibit bank CDN_MBIT_cnt_0_reg[0]_MB_cnt_0_reg[1] out of
cnt_0_reg[0]          (DFCF)
cnt_0_reg[1]          (DFCF)
CDN_MBIT_cnt_0_reg[0]_MB_cnt_0_reg[1] multibit bank rejected (worse QoR).
```

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [synthesize_design](#)
- `use_multibit_cells` in [root](#) in *Stylus Common UI Database Object Information Guide*
- `bit_width` in [lib_cell](#) in *Stylus Common UI Database Object Information Guide*
- [Using the Multi-Bit Flip-Flop Flow](#) in *Innovus Stylus Common UI User Guide*

multibit_invert_clock_phase

```
multibit_invert_clock_phase {false | true}
```

Description

Default: `false`

Data_type: `bool`, read/write

If set to `'true'`, enables merging of cells with inverted clock phase.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```
- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

multibit_mapping_effort_level

```
multibit_mapping_effort_level {auto | high | low}
```

Description

Default: `high`

Data_type: `enum`, read/write

Controls the effort level for physical-aware multibit mapping. That is, controls the tradeoff between

multibit coverage and QoR.

This attribute can have the following values:

auto	Balances multibit coverage and QoR to get the most optimal results.
high	Increases multibit coverage but might impact QoR.
low	Gives the best QoR.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- [synthesize_design](#)
- `use_multibit_cells` in [root](#) in *Stylus Common UI Database Object Information Guide*
- [Using the Multi-Bit Flip-Flop Flow](#) in *Innovus Stylus Common UI User Guide*

multibit_predefined_allow_unused_bits

`multibit_predefined_allow_unused_bits {true | false}`

Description

Default: `false`

Data_type: `bool`, read/write

Controls whether instances of predefined multibit cells can have unused bits. By default, the tool does not allow unused bits in predefined multibit instances.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [synthesize_design](#)
- `bit_width` in [lib_cell](#) in *Stylus Common UI Database Object Information Guide*
- `use_multibit_cells` in [root](#) in *Stylus Common UI Database Object Information Guide*
- [Using the Multi-Bit Flip-Flop Flow](#) in *Innovus Stylus Common UI User Guide*

multibit_unused_input_value

```
multibit_unused_input_value {0 | 1 | none}
```

Description

Default: 0

Data_type: string, read/write

Specifies the value to which all unconnected input pins in the multibit cells must be connected. By default, the unused input pins of multibit cells are connected to constant 0. If you set the attribute to `none`, the input pins will be left floating.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [synthesize_design](#)
- `use_multibit_cells` in [root](#) in *Stylus Common UI Database Object Information Guide*

opt_activity_refresh_args

`opt_activity_refresh_args` "list_of_arguments"

Default: ""

Data_type: string

Specifies the list of simulation options to be passed for xreplay/flash replay in Innovus. The following arguments can be specified using this attribute:

- `[-xrun_path]` Specify xrun binary path to run Xcelium for replay simulation.
- `[-rtl_stim]` Specify the input RTL stimulus file.
- `[-stim_format]` Suboption of `-rtl_stim`, provides format of rtl stimulus.
- `[-dut_instance]` Suboption of `-rtl_stim`, provides dut hierarchy in rtl stimulus.
- `[-start]` Specify start time to run replay for particular stimulation time duration, for example 24ns. Default is simulation start time.
- `[-end]` Specify end time to run replay for stimulation time duration, for example 100ns. Default is simulation end time.
- `[-verilog_model_files]` Specify list of library verilog models
- `[-stim_annotation]` Honor stim annotation of only the specified objects.
- `[-pre_script]` Specify the Tcl file to be sourced before xreplay.
- `[-post_script_options_file]` Specify the user settings file and source it before generating the post_script in the glitch optimization flow.
- `[-out]` Specify the output replayed gate-level stimulus file. Default is `joulesWorkDir/replayed_stim.shm`
- `[-reset_activity_before_readback]` Will reset the previous activity before reading the xreplayed activity file.
- `[-read_stimulus_options {-slew_type max_slew}]` Ignores small glitch power.

Note: With this option, glitches in power report may be less than those with the default flow, that is, without `-slew_type max_slew`.

- `[-delay_mode none | zero | unit | punit | delay | pdelay]` (default: none)
- `[-gen_sim_models lec | liberate]` Generates the simulation model files. It has two options, `lec` and `liberate`. When specifying this command, one of these two options is mandatory.
 - `-lec_path path_to_lec` Specify the complete path to `lec` executable. Generates zero delay model files.
 - `-liberate_path path_to_liberate` Specify the complete path to `liberate` executable. Generates delay based model files.

For example, `set_db opt_activity_refresh_args "...in addition to other collaterals...-gen_sim_models [lec | liberate] [-lec_path path_to_lec or -liberate_path path_to_liberate]"`

For setting the arguments: `set_db opt_activity_refresh_args "<args>"`

For querying the arguments: `get_db opt_activity_refresh_args`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- [opt_design](#)
- [opt_activity_refresh_flow](#)

opt_activity_refresh_flow

`opt_activity_refresh_flow {xreplay | flash_replay}`

Default: `xreplay`

Data_type: `enum {xreplay flash_replay}`

Provides an option to switch between xreplay and flash replay stimulus flows. The simulation options for xreplay and flash replay are specified using the `opt_activity_refresh_args` attribute.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- `opt_activity_refresh_args`

opt_activity_refresh_stages

```
opt_activity_refresh_stages {elab pre_map post_map}
```

Default: ""

Data_type: string, read/write

Specifies the list of stages at which the activity refresh options are to be passed for xreplay or flash replay in Innovus. Valid values are one or more of the following stages:

- `elab`
- `pre_map`
- `post_map`

This attribute requires the `opt_activity_refresh_args` attribute to be set.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- `opt_activity_refresh_args`
- `opt_activity_refresh_flow`
- `synthesize_design`

opt_adaptive_subd1_flow

```
opt_adaptive_subd1_flow {true | false}
```

Default: `false`

Data_type: `bool`

When this attribute is set to `true`, the software adaptively uses weak cells with driving strength smaller or equal to D1 in optimization flow.

By default, this attribute is set to `false`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- `opt_design`
- `add_repeater_by_rule`

opt_add_always_on_feed_through_buffers

```
opt_add_always_on_feed_through_buffers {true | false}
```

Default: `false`

Data_type: `bool`

Used by `opt_design` and PD-aware `add_repeater_by_rule`. When this attribute is set to `true`,

`opt_design` will use always-on buffers in both pre-Route and post-Route for all the optimization steps; and `add_repeater_by_rule` will use always-on buffer for repeater insertion. It should be used for aggressive always-on buffering during optimization.

The default `opt_design` behavior is to use always-on buffers only during pre-Route DRV fixing.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)
- [add_repeater_by_rule](#)

opt_add_insts

```
opt_add_insts {true | false}
```

Default: true

Data_type: bool

Specifies whether or not timing optimization can add instances.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_add_ports

`opt_add_ports {true | false}`

Default: `true`

Data_type: `bool`

Controls whether or not timing optimization adds ports to the lower-level hierarchies of the design. Prohibiting the software from adding ports (when this attribute is set to false) might not be optimal because buffers are not allowed to be added exactly where needed; however, this preserves hierarchical boundaries of the design so that generated test vectors are still valid, and module-by-module verification is possible.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_add_repeater_report_failure_reason

`opt_add_repeater_report_failure_reason {true | false | verbose}`

Default: `false`

Data_type: `enum {true false verbose}`

When set to `true`, enables the printing of detailed failure reasons for each net for the [add_repeater_by_rule](#) command. By default, the report will be generated in the `timingReports` directory. If the directory is not available, it will be created. For example,

```
<run_dir>/timingReports/ARRFailureReason/ARRRpt.preRoute.1
```

Use Model

```
set_db opt_add_repeater_report_failure_reason true
```

```
add_repeater_by_rule -include_nets_file ./net.list -rule $scr/rule -pre_route -  
report_ignored_nets ignore_nets
```

Note the following:

- The ignored nets report is appended at the beginning of the failure reason report file. If “`-report_ignored_nets IGNORE_NET_REPORT_NET_FILE_NAME`” is specified, then the ignored nets report is also saved to the file “`IGNORE_NET_REPORT_NET_FILE_NAME`”.
- For multiple calls of ARR with different set of nets and constraints, equal number of failure reports will be generated under the `ARRFailureReason` directory. A suffix, “`_<n>`” will be added, with `n` being the `nth` call of ARR. For example, three calls of ARR will lead to the generation of the following three files:

```
<run_dir>/timingReports/ARRFailureReason/ ARRRpt.preRoute.1
<run_dir>/timingReports/ARRFailureReason/ ARRRpt.preRoute.2
<run_dir>/timingReports/ARRFailureReason/ ARRRpt.preRoute.3
```

- The default mode of ARR reporting is `pre_route`. If you specify `-post_route` or `-along_route`, the following will be appended in the report

```
names:<run_dir>/timingReports/ARRFailureReason/ ARRRpt.postRoute.1
<run_dir>/timingReports/ARRFailureReason/ ARRRpt.alongRoute.1
```

Note: The behavior of the `add_repeater_by_rule -report_ignored_nets` parameter remains unchanged. It writes the reports with user-specified file name under the run directory.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- [opt_design](#)

opt_all_end_points

```
opt_all_end_points {true | false}
```

Default: `false`

Data_type: `bool`

Runs total negative slack optimization in addition to the worst negative slack optimization and does so on all end points to ensure a high quality timing result.

This feature is automatically activated while running either of the `set_db design_flow_effort standard | extreme` timing-closure flows. So, despite the default being `false` it is overwritten by the `design_flow_effort` attribute setting.

Note: When this attribute is set to `false`, there may still be some fast TNS optimization done during `pre_route` optimization but there is no effect on `post_route` setup optimization. It also has no effect on hold optimization, which means that regardless of this setting, all hold violating paths will always be optimized.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- [opt_design](#)
- [place_design](#)

opt_allow_floating_outputs

`opt_allow_floating_outputs {false | true}`

Description

Default: `false`

Data_type: `bool`, read/write

If set to `'true'`, mapping and incremental optimization are allowed to use multi-output gates and leave some outputs dangling.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

opt_allow_only_cell_swapping

```
opt_allow_only_cell_swapping {true | false}
```

Default: false

Data_type: bool, read/write

Forces the post-route optimization to perform only same-size Vth swapping for timing improvements. This attribute works best on multi-Vth library designs.

Note: Use this attribute for reoptimizing the timing in specific cases where routing should remain unchanged.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_allow_multi_bit_on_flop_with_sdc

```
opt_allow_multi_bit_on_flop_with_sdc {true | false | merge_only | split_only}
```

Default: true

Data_type: enum {true false merge_only split_only}, read/write

When this attribute is set to `true`, or by default, the software will not honor the SDC constraints while merging or splitting of flops in the multi-bit flip flop flow. Set this attribute to `false` to honor SDC constraints while merging or splitting of flops.

Use `merge_only` to ignore the SDC constraints while merging but honor for splitting. Use `split_only`

to ignore the SDC constraints while splitting but honor for merging.

Note: When this attribute is set to `true`, the software will do merge/split even if there is SDC on D/Q pins and the SDC will be copied to the corresponding D/Q pins in the new flop.

But if there are SDC constraints on SE/SI pin, the merging becomes more complicated. For example, a single flop A and flop B have 1 SE pin and D/Q pin. When merging is done, the SDC on D/Q pin will be copied to the new D0/Q0 and D1/Q1, and the SDC information will not be lost. But in the new 2-bit flop, there will be only one SE pin. This raises the issue of which SDC should be copied into the new flop. In this case, the software will check the SE pin SDC in flop A and flop B. If they are compatible, then it will do merging. Otherwise, the merging will not be done, even if the attribute is set to `true`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_area_recovery

```
opt_area_recovery {false | true | default}
```

Default: `default`

Data_type: enum {`true` `false` `default`}, read/write

Controls whether timing optimization creates additional space by downsizing gates or deleting buffers, while maintaining worst slack and total negative slack.

This attribute can be used in three modes: `default`, `true`, and `false`.

- `default`: By default, area reduction is always called after global optimization and after the second TNS optimization. During the rest of `opt_design`, area reduction is called depending on the density level of the current design.
- `true`: When this attribute is set to `true`, area reduction is always called before the global optimization stage, after the global optimization stage, and after the second TNS optimization.

During the rest of `opt_design`, area reduction is called depending on the density level of the current design.

- `false`: No area reduction is performed during `opt_design`.

Note: This attribute is used to reclaim area during pre-Route optimization. For reclaiming area during post-Route optimization, use the `opt_post_route_area_reclaim` attribute.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- `opt_design`

opt_area_recovery_setup_target_slack

```
opt_area_recovery_setup_target_slack slack
```

Default: 0.0

Data_type: double, read/write

Specifies a user-defined target slack for area reclaim vs. timing. When a slack value is specified, the software prevents area reclaim optimization from creating or working on near critical paths. Typically, you can set this attribute to 0.1 with `opt_setup_target_slack` attribute at the default value of 0.0 seconds. This implies that the area reclaim will not work on any paths less than 0.1ns.

Note that the value for `opt_setup_target_slack` attribute takes higher priority. For example, if `opt_area_recovery_setup_target_slack` is set to -0.1 and `opt_setup_target_slack` is at 0.0 then setup timing will not be allowed to degrade past 0ps and a warning will be issued.

Similarly, if say `opt_area_recovery_setup_target_slack` is at +0.1 and `opt_setup_target_slack` is set to +0.2 then timing will target +0.2 and a warning will be issued.

You need to set this attribute before any area reclaim is done.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_boundary_optimize_invert_hpins_rename_extension

```
opt_boundary_optimize_invert_hpins_rename_extension <string>
```

Description

Default: `_BAR`

Data_type: `string`, `read/write`

Specifies the string to be appended to pin names and net names when hierarchical boundary pins are inverted during synthesis.

- If you specify an empty string, you implicitly disable renaming for pins and nets. However this is not recommended, because the formal verification tools need the naming extension to identify inverted boundary pins.
- If the `boundary_optimize_invert_hpins` root attribute is set to true, the `write_do_lec` command adds the following LEC command to the dofile:

```
SET Naming Rule _BAR -Inverted_pin_extension
```

This LEC command specifies the string that is appended to pin names and net names by the tool when hierarchical boundary pins are inverted during synthesis. To modify this string use the `boundary_optimize_invert_hpins_rename_extension` root attribute. In the command above, the `_BAR` value corresponds to the default value of this attribute.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [synthesize_design](#)
- [boundary_optimize_invert_hpins](#)
- [boundary_optimize_invert_hpins_rename_nets](#)

opt_clock_traverse_by_level

```
opt_clock_traverse_by_level {true | false}
```

Description

Default: false

Data_type: bool, read/write

Traverses instances in the clock path, 1 level at a time.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

opt_clone_insts_list

```
opt_clone_insts_list { {...} ... }
```

Default: ""

Data_type: string, read/write

Specifies the list of instances to be cloned. The instances are cloned by the command, [eco_clone_inst_group](#).

```
set_db opt_clone_insts_list { {...} ... }
```

```
eco_clone_inst_group -max_fanout max_fanout
```

Example:

```
set_db opt_clone_insts_list { {subinst/registergate subinst/il subinst/nandgate} }  
  
eco_clone_inst_group -max_fanout 16
```

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_concatenate_default_and_user_prefixes

```
opt_concatenate_default_and_user_prefixes {true | false}
```

Default: `true`

Data_type: `bool`, read/write

Controls whether to combine the default prefix for cells and nets added by timing optimization with the prefixes specified by `opt_new_net_prefix` and `opt_new_inst_prefix` attributes. By default, the software combines the specified prefixes with the default prefix. Specify `false` for this attribute to ignore the default prefix and use only the prefix specified by `opt_new_net_prefix` and `opt_new_inst_prefix` attributes.

For example, if the default prefix is `PSN_000234` and you set the following attributes:

```
opt_concatenate_default_and_user_prefixes true
```

```
opt_new_net_prefix POSTROUTE_N
```

```
opt_new_inst_prefix POSTROUTE_C
```

The software uses the following prefixes:

- For cells added by optimization: `POSTROUTE_C_PSN_000234`
- For nets added by optimization: `POSTROUTE_N_PSN_000234`

If the default prefix is `PSN_000234` and you set the following attribute:

```
opt_concatenate_default_and_user_prefixes false
```

```
opt_new_net_prefix POSTROUTE_N
```

```
opt_new_inst_prefix POSTROUTE_C
```

The software uses the following prefixes:

- For cells added by optimization: `POSTROUTE_C`
- For nets added by optimization: `POSTROUTE_N`

Note: If you specify `false` for this attribute, you must also specify `opt_new_inst_prefix` and `opt_new_net_prefix` attributes. In addition, you must specify a different prefix for instances and nets.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
- To get the current value of any root attribute, use the `get_db` command:

```
set_db attribute_name value
```

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_consider_routing_congestion

```
opt_consider_routing_congestion {auto | false | true}
```

Default: `auto`

Data_type: enum {auto false true}, read/write

Checks the routing congestion at the post-Route stage during optimization of setup, hold, drv, or glitch. This attribute also impacts post-CTS hold fixing. By default, this attribute is set to `auto` for both `opt_design -post_route` and `opt_design -post_route -hold` options. This attribute helps improve NR DRC's as well as Timing jumps caused by routing detours on congested designs.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_constant_inputs

```
opt_constant_inputs {true | false}
```

Default: false

Data_type: bool, read/write

Forces optimization of instances with constant inputs, even if it worsens the objective gain - in terms of area, power, and delay.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_constant_nets

```
opt_constant_nets {true | false}
```

Default: true

Data_type: bool, read/write

Specifies whether or not timing optimization optimizes constant nets.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_delete_insts

```
opt_delete_insts {true | false}
```

Default: true

Data_type: bool, read/write

Specifies whether or not timing optimization can delete instances.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_detail_drv_failure_reason

```
opt_detail_drv_failure_reason {true | false}
```

Default: false

Data_type: bool, read/write

When set to `true`, enables the printing of detailed DRV failure reason report file that contains the failure summary with the net name. The log file contains a simple version of failure summary without the net name. In addition, the top number(default 50) of violating nets are printed in separate files

along with the violation reasons.

By default, this attribute is set to `false`, which means that only a simple version of failure summary without the net name is printed in the log file and no other files are created.

Note: If there is no DRV failure during the optimization flow, the failure reason report file will not be generated even when this attribute is set to `true`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_detail_drv_failure_reason_max_num_nets

```
opt_detail_drv_failure_reason_max_num_nets value
```

Default: 50

Data_type: int, read/write

Specifies the number of violating nets for which the DRV failure reasons are to be printed in the report files. The default value is 50, which means failure reasons for 50 violating nets are printed by default. The minimum value that can be specified is 1. The maximum value that can be specified is 9999. If this value is specified, failure reasons for all the violating nets are printed.

For example, if you set this attribute to 100, then the top 100 max tran violating nets, if present, will be printed in file, “xx.maxTran” and the top 100 max cap violating nets, if present, will be printed in file, “xx.maxCap”. Also, a file, “xx.maxFanout” may also be generated.

For each file that is generated, the first printed net has the worst violation value. The file is sorted based on violation values while printing.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_dont_use_qn_sequential_pins

```
opt_dont_use_qn_sequential_pins {true | false}
```

Description

Default: `false`

Data_type: `bool`, read/write

Controls the use of the Qbar output pins of sequential libcells during optimization if other possibilities exist. By default, these pins can be used.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- `dont_use_qbar_pin` in [inst](#) in *Stylus Common UI Database Object Information Guide*
- [synthesize_design](#)

opt_down_size_insts

```
opt_down_size_insts {true | false}
```

Default: `true`

Data_type: bool, read/write

Specifies whether or not timing optimization can downsize instances.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_drv

```
opt_drv {true | false}
```

Default: true

Data_type: bool, read/write

Repairs all nets that are constrained for maximum capacitance violations and maximum transition violations. The capacitance and transition constraint values are specified in the timing constraint file or in the library. If you specify false for this attribute, the software repairs only the paths with timing violations.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_drv_margin

`opt_drv_margin margin`

Default: 0.0

Data_type: double, read/write

Scales the maxCap and maxTran constraints according to the *margin* decimal value specified. For example, specifying a value of 0.2 for this attribute tightens the margin (makes it more pessimistic) by 20 percent during optimization and specifying a value of -0.2 relaxes the margin (makes it more optimistic) by 20 percent. Introducing a pessimistic margin improves correlation between the number of DRC violations before and after routing.

Value range: -0.7 ~ 0.7

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_drv_with_miller_cap

`opt_drv_with_miller_cap {true | false}`

Default: false

Data_type: bool, read/write

When set to `true`, enables ECSM/CCS miller capacitance DRV reporting and fixing for `time_design` and `opt_design`. However, the `timing_report_use_receiver_model_capacitance` attribute must also be set to `true`.

By default, this attribute is set to `false`, which means that miller cap DRV reporting and fixing is disabled.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)
- [time_design](#)

opt_duplicate_cte_constrained_hport

```
opt_duplicate_cte_constrained_hport {true | false}
```

Default: `true`

Data_type: `bool`, read/write

When set to `true`, allows buffering to duplicate hports with CTE constraints to fix DRVs and setup violations.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_early_hold_fixing

```
opt_early_hold_fixing {none | ideal | propagated}
```

Default: `none`

Data_type: `enum {none ideal propagated}`, read/write

Sets the ideal or propagated clock mode for hold fixing in pre-CTS optimization. The following settings are available:

With Early Clock Flow (ECF)

- `none`: turns off the hold fixing in `place_opt_design` (POD). This is the default setting.
- `ideal`: turns on the hold fixing in `place_opt_design` (with ideal clock mode).
- `propagated`: turns on the hold fixing in `place_opt_design` (with propagated clock mode).

Without ECF

- `none`: turns off the hold fixing in `place_opt_design` (POD).
- `ideal`: turns on the hold fixing in `place_opt_design` (with ideal clock mode).
- `propagated`: same as `ideal`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- [opt_design](#)
- [place_opt_design](#)
- [design Category Attributes](#)

opt_enable_always_on_type_in_remove_assign

`opt_enable_always_on_type_in_remove_assign {true | false}`

Description

Default: `false`

Data type: `bool`, read/write

If set to true, it enables checking of aon type compatibility during remove assigns.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

opt_enable_clock_pulse_width_checks

```
opt_enable_clock_pulse_width_checks {true | false}
```

Default: false

Data_type: bool, read/write

When set to `true`, checks and fixes minimum clock pulse width violations.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_enable_data_to_data_checks

```
opt_enable_data_to_data_checks {true | false}
```

Default: false

Data_type: enum {true false}, read/write

Enables data to data checks during optimization. During optimization the data to data timing checks are set to off by default to ensure that the engine focuses on other paths. To allow checks to be visible during optimization you can set this option to true.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_enable_drv_failure_reason_warn

```
opt_enable_drv_failure_reason_warn {true | false}
```

Default: false

Data_type: bool, read/write

When set to `true`, enables the printing of the following warning messages for the DRV failure reasons in the log file:

```
**WARN:(IMPOPT-7398): Total %d net(s) have violations which can't be fixed by DRV optimization.
```

```
**WARN:(IMPOPT-7399): Total %d net(s) were new nets created by previous iteration of DRV buffering. Further DRV fixing might remove some violation.
```

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_enable_podv2_clock_opt_flow

```
opt_enable_podv2_clock_opt_flow {true | false}
```

Default: true

Data_type: bool, read/write

When set to false, enables the [place_opt_design](#) and [ccopt_design](#) flow. By default it is set to true, which means the [place_opt_design V2](#) (PODV2 or pod_turbo) and [clock_opt_design](#) flow is enabled.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [clock_opt_design](#)
- [opt_design](#)

opt_enable_restructure

```
opt_enable_restructure {true | false}
```

Default: true

Data_type: bool, read/write

Specifies whether or not timing optimization swaps pins and restructures the netlist.

Note: This attribute is not enabled when the `design_flow_effort` attribute is set to `express`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_enable_targeted_synthesis

```
opt_enable_targeted_synthesis {true | false}
```

Default: false

Data_type: bool, read/write

When set to `true`, enables the targeted synthesis mode for a group of instances that are added during ECO. Setting this attribute to `true` enables the optimizer to do DRV, setup, and reclaim hold for the instances. An object attribute, `targeted_synthesis`, is defined on the new instances using the following command:

```
set_db [get_cells $new_insts] .targeted_synthesis true
```

Where,

`new_insts` is the collection of the new instances added using ECO. Optimization will work on these instances when this root attribute is set to `true` and the object attribute, `targeted_synthesis`, is defined on the instances.

The `opt_design` command can be run to do DRV, setup, and hold optimization.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_fix_fanout_load

```
opt_fix_fanout_load {true | false}
```

Default: false

Data_type: bool, read/write

Forces timing optimization to correct fanout load violations, which can either be maximum load or maximum fanout count violations, depending on the libraries.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_flop_pins_report

```
opt_flop_pins_report {true | false}
```

Default: false

Data_type: enum {false true}, read/write

Set this attribute to `true` for printing out full hierarchical pin names used in the incoming netlist for flops deleted during simplifyNetlist. The output file name format is `<userPrefix>_flops_pins.rpt`.

You can choose to write both files with pins or without pins (`opt_flops_report`) in same session.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_flops_report

`opt_flops_report {true | false}`

Default: false

Data_type: enum {false true}, read/write

Set this attribute to `true` for printing full hierarchical flop names used in the incoming netlist for flops deleted during simplifyNetlist. The output file name format is `<userPrefix>_flops.rpt`.

You can choose to write both files with pins (`opt_flop_pins_report`) or without pins in same session.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_hier_add_antenna_cell

`opt_hier_add_antenna_cell {true | false}`

Default: false

Data_type: bool, read/write

Adds antenna cells to the interface nets crossing partition boundaries. The antenna violations could not be created due to the partition boundary nets. These violations are seen only when the full design is assembled. Therefore, when this attribute is set to `true`, it will add antenna cells to fix these violations.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

opt_hier_opt_stage

```
opt_hier_opt_stage {type {enum pre_cts post_cts unset}}
```

Default: unset

Data_type: enum {pre_cts post_cts unset}, read/write

Ensures proper settings are turned on for corresponding `opt_design` stages. `opt_hier_opt_stage` should be set before `opt_design`, and in cases where module view types are defined, `opt_hier_opt_stage` should be defined after module view types for all partitions (including FlexILMs) are defined.

For example, `opt_hier_opt_stage pre_cts` ensures that `opt_design -pre_cts` honors fence, keep partition boundary port, etc.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_high_effort_cells

```
opt_high_effort_cells list_of_cells
```

Default: ""

Data_type: string, read/write

Specifies the list of cells to be used for high-effort timing optimization. This attribute is applicable in both pre-Route and post-Route setup optimization.

Note: If the `dont_touch` constraint is set to `true` for the specified cells, then these cells will not be used for high-effort timing optimization.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_hold_allow_overlap

```
opt_hold_allow_overlap {auto | true | false}
```

Default: auto

Data_type: enum {auto false true}, read/write

Allows hold-fixing algorithm to insert buffers with overlap when large enough empty space locations are not available. The placement will then be legalized before ECO routing. Setting this attribute allows the tool to fix hold violations, but might have some negative impact on setup timing.

- `auto`: Hold fixing will run in the "allow overlapping" mode in post-CTS and not in post-Route stage.
- `true`: Hold fixing will run in the "allow overlapping" mode in both post-CTS and post-Route stage.
- `false`: Hold fixing will run in "allow overlapping" mode in both post-CTS and post-Route stage.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_hold_allow_resize

`opt_hold_allow_resize {true | false}`

Default: true

Data_type: enum {false true}, read/write

Controls the resizing capability inside hold optimization. When set to `true` or `false`, resizing is turned "on" or "off" in both post-CTS and post-Route hold optimization.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

Related Information

- [opt_design](#)

opt_hold_allow_setup_tns_degradation

`opt_hold_allow_setup_tns_degradation {true | false}`

Default: true

Data_type: bool, read/write

Controls whether the software allows total negative slack to degrade during hold fixing operations or not. By default, the software allows the slack to degrade in order to fix more hold violations. To override the default behavior and ensure that the setup total negative slack is maintained during hold fixing, set this attribute to `false`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

`set_db attribute_name value`

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_hold_ignore_path_groups

```
opt_hold_ignore_path_groups {groupAgroupB....}
```

Default: ""

Data_type: string, read/write

Specifies the clock domains or path groups that should be excluded from hold timing optimization. This attribute impacts only `opt_design -post_cts/-post_route -hold`.

Note: To reset this attribute, apply an empty list.

Note: If a violated path is in two path groups and only one of those paths is specified using this attribute, this path is still fixed during hold fixing.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_hold_cells

```
opt_hold_cells list_of_buffers
```

Default: ""

This will trigger the default flow.

Data_type: string, read/write

Allows you to force the usage of only a specific list of buffer and delay cells. The *list_of_buffers* contains regular buffers and delay buffer cells. When this list contains at least one cell, use only the specified cells.

Note: This attribute does not support wildcards. The *list_of_buffers* must contain only exact cell names.

Note: When this list of buffers is specified, the `opt_design` command disables automatic cell filtering, which could impact the runtime/QOR/leakage of `opt_design`.

Note: The `opt_design` command uses the specified cells without honoring the `dont_use` attribute.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- `opt_design`

opt_hold_on_excluded_clock_nets

`opt_hold_on_excluded_clock_nets {true | false}`

Default: false

Data_type: bool, read/write

Controls whether or not hold fixing will be performed on excluded clock nets. This attribute is applicable for both post-CTS and post-Route Hold optimization.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- [opt_design](#)

opt_hold_slack_threshold

`opt_hold_slack_threshold slack`

Default: -1000.0

Note: The unit is ns

Data_type: double, read/write

Allows you to provide a hold slack threshold beyond which the hold timing violations will not be fixed. This is used to avoid fixing out of bound violations at an early stage when the timing constraints and design environment might not be mature enough. The slack value is specified in nano seconds (ns).

Note: When the hold slack threshold value is specified using this attribute, the hold fixing timing table excludes the violations with slack worse than the specified value.

Note: The threshold value specified is for hold WNS.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_hold_target_slack

`opt_hold_target_slack holdTargetSlack`

Default: 0.0

Data_type: double, read/write

Specifies a target slack value in nanoseconds to use for hold analysis only. During setup violation repair, the setup target slack is defined by the attribute, `opt_setup_target_slack` and the hold target slack value is 0. Generally, you use both the `opt_setup_target_slack` and the `opt_hold_target_slack` attributes for hold analysis. If you specify only `opt_hold_target_slack` attribute, the setup target slack value is 0.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- [opt_design](#)

opt_honor_density_screen

`opt_honor_density_screen {true | false}`

Default: `false`

Data_type: bool, read/write

Controls whether or not density screens (partial placement blockages used to control congestion) are honored while optimizing timing. By default, this attribute is set to `false`, which means that the density screens are not honored.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- [opt_design](#)

opt_honor_fences

```
opt_honor_fences {true | false}
```

Default: false

Data_type: bool, read/write

Specifies that the timing optimization takes fences or region constraints into account. This attribute is honored by each `opt_design` step. If you optimize the design with this attribute set to true, the placement is legal.

This feature is useful for hierarchical designs. Because this attribute adds constraints to timing optimization, enable it only when needed.

Note: `opt_design` will not honor very small fences when this attribute is set to true if it would cause fence utilization to be greater than 100 percent.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_icg_enable_pin_rebuffering

```
opt_icg_enable_pin_rebuffering {true | auto | false}
```

Default: false

Data_type: enum {true auto false}, read/write

When set to `true`, rebuffers nets to enable pins of inverted clock gate (ICG) instances after clock tree insertion. When set to `auto`, rebuffers affected ICG enable. By default, this attribute is set to `false`, which means the feature is disabled.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_leakage_to_dynamic_ratio

```
opt_leakage_to_dynamic_ratio ratio
```

Default: 1.0

Data_type: double, read/write

Controls the priority of the power-driven optimization. You can set any value between 0 and 1. Use this attribute along with the `opt_power_effort` attribute.

Some examples of what the value of this attribute indicates are provided below:

- `opt_leakage_to_dynamic_ratio 1.0`
This is the default value and implies that timing optimization will be purely leakage-power driven optimization.
- `opt_leakage_to_dynamic_ratio 0`
This implies that timing optimization will be purely dynamic power-driven optimization.
- `opt_leakage_to_dynamic_ratio 0.5`
This implies that timing optimization will be both dynamic and leakage power-driven and trade-offs may need to be done to get a balance between the leakage and dynamic power optimization.
- `opt_leakage_to_dynamic_ratio 0.1`
This implies that timing optimization will be both dynamic and leakage power-driven but dynamic power optimization will be favored.
- `opt_leakage_to_dynamic_ratio 0.9`
This implies that timing optimization will be both dynamic and leakage power-driven but leakage power optimization will be favored.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_max_density

```
opt_max_density value
```

Default: 0.95

Data_type: double, read/write

Specifies the maximum value for density (area utilization). The `opt_design` command does not grow the netlist above this value. When reaching a density close to the value specified by this attribute, `opt_design` has limited optimization capability.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_max_length

```
opt_max_length length
```

Default: -1.0

min=-1, max=2147483647

Data_type: double, read/write

Specifies that `opt_design` honors the `maxLength` rule as any other DRV rule (`max_cap/max_tran/max_fanout`). The `opt_design` command will fix all the nets that are violating this rule and will ensure to not create any long nets that violate it. The value is specified in micrometer.

Note: This attribute implies that more buffers gets inserted in the netlist. This attribute is disabled by default.

Note: During the pre-Route stage, some `maxLength` violations might still exist after running `opt_design` due to routing topology changes taking place during the flow.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- [opt_design](#)

opt_move_insts

`opt_move_insts {true | false}`

Default: `true`

Data_type: bool, read/write

Allows cells in the critical path to move when the software repairs setup violations or optimizes timing along the critical path, if allowing them to move improves timing.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- [opt_design](#)

opt_multi_bit_combinational_mode

`opt_multi_bit_combinational_mode {auto | power | area}`

Default: `auto`

Data_type: enum {auto power area}, read/write

Enables the user-specific multi-bit combinational cell (MBCC) merging and splitting based on area and power.

`auto`: Enables merging or splitting of MBCCs based on area (*if power effort is none*), otherwise based on power.

`power`: Enables merging or splitting of MBCCs based on power.

`area`: Enables merging or splitting of MBCCs based on area.

Default: `auto`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)
- [report_multibit](#)

opt_multi_bit_combinational_opt

`opt_multi_bit_combinational_opt {false | true | mergeOnly | splitOnly}`

Default: `splitOnly`

Data_type: enum {false true mergeOnly splitOnly}, read/write

Enables and disables the multi-bit combinational flow. This flow is called as a part of the pre-CTS optimization stage. The following settings are available:

- **true**: enables both merging and splitting of combinational cells
- **false**: disables the flow
- **mergeOnly**: only merges the combinational cells
- **splitOnly**: only splits the combinational cells

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

Related Information

- [opt_design](#)
- [report_multibit](#)

opt_multi_bit_combinational_merge_timing_effort

`opt_multi_bit_combinational_merge_timing_effort {none | low | medium | high}`

Default: medium

Data_type: enum {low medium high}, read/write

Specifies the timing effort-level for multi-bit combinational merging.

- If the attribute is set to **high**, the software merges the combinational cell to maintain or improve the timing.
- If the attribute is set to **low**, the software merges the combinational cell with allowing timing degradation.
- If the attribute is set to **medium**, the software allows merging without/minimal degradation in timing.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)
- [report_multibit](#)

opt_multi_bit_combinational_split_timing_effort

```
opt_multi_bit_combinational_split_timing_effort {none | low | medium | high}
```

Default: low

Data_type: enum {low medium high}, read/write

Specifies the timing effort-level for multi-bit combinational splitting.

- If the attribute is set to `low`, the software allows splitting to critical paths only.
- If the attribute is set to `medium`, the splitting happen to critical paths to improve timing.
- If the attribute is set to `high`, the software performs aggressive splitting to improve timing.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)
- [report_multibit](#)

opt_multi_bit_flop_merge_bank_label_inference

`opt_multi_bit_flop_merge_bank_label_inference {true | false}`

Default: false

Data_type: bool, read/write

Enables soft merging of flops with the same bank label.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_multi_bit_flop_merge_timing_effort

`opt_multi_bit_flop_merge_timing_effort {low | medium | high}`

Default: medium

Data_type: enum {low medium high}, read/write

Specifies the timing effort level for MBFF merging.

- `low` effort setting allows degrading in timing to achieve merging.
- `medium` effort means that merging happens either without or with minimal degradation of timing after merging.
- `high` effort setting does not allow any degradation and does merging only if it either maintain or improves timing.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- `opt_design`
- `report_multibit`

opt_multi_bit_flop_opt

```
opt_multi_bit_flop_opt {true | false | mergeOnly | splitOnly}
```

Default: `splitOnly`

Data_type: enum {false true mergeOnly splitOnly}, read/write

Enables and disables the multi-bit flip flop flow. This flow is called as a part of the pre_CTS optimization stage. Set this attribute to `true` to enable both merging and splitting of flops. Set it to `false` to disable the flow. Set it to `mergeOnly` to only merge flops. By default, the attribute is set to `splitOnly`, which means the flow is enabled only for splitting MBFFs.

Note:

- This option is independent of `opt_enable_restructure`, `opt_resize_flip_flops`, and `opt_preserve_all_sequential` attributes.
- To ensure sufficient multi-bit conversion ratio, allow more usable multi-bit flop cells from the libraries. Use the command, `report_multibit -equiv` to write out the corresponding multi-bit flop cells to the single-bit flop cells.
- To reduce long runtime in flop merge steps, ensure that the scan enable signal nets connecting to the flops are buffered, which means without max-fanout violations.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```
- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- `opt_design`
- `report_multibit`

opt_multi_bit_flop_name_prefix

`opt_multi_bit_flop_name_prefix prefix_name`

Default: `CDN_MBIT_`

Data_type: string, read/write

Specifies the MBFF name prefix. When single-bit flip-flops are merged into a multi-bit flip-flop, you can use this attribute to specify the prefix for the new merged flip-flop. If no prefix is specified, the name of the merged flip-flop starts with the default name, `CDN_MBIT_`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- `opt_design`
- `report_multibit`

opt_multi_bit_flop_name_separator

`opt_multi_bit_flop_name_separator separator_name`

Default: `_MB_`

Data_type: string, read/write

Specifies the flip-flop merge and split name separator. The separator is used while splitting and merging flip-flops. The default separator name is `_MB_`. The use model is detailed below.

Scenario1:

When two or more SBFFs are merged, the separators are used between names of single bit flip-flops. The MBFF is named as; `prefix_bit1_separator_bit2_separator_bit3..._separator_bit#`.

Scenario2:

When an MBFF is split into smaller bits, then each smaller bit is named as;
`prefix_bit1_separator_bit2`.

Scenario3:

When an MBFF is split into the original number of single-bit flip-flops, then each SBFF reverts to its original name, which is without prefix and separator such as; `bit1, bit2...`

Consider the example below to understand the above scenarios:

There are four SBFFs: `bit1, bit2, bit3, and bit4`.

- With default settings, when they are merged, the following MBFF is generated:
`CDN_MBIT_bit1_MB_bit2_MB_bit3_MB_bit4`
This MBFF has the default prefix and the default separator.
- When the above MBFF is split into two-bit MBFFs, the following is generated:
`CDN_MBIT_bit1_MB_bit2`, and
`CND_MBIT_bit3_MB_bit4`
- When the MBFF is split into the original SBFFs, the names of the SBFFs revert to the original names:
`bit1, bit2, bit3, and bit4`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- [opt_design](#)
- [report_multibit](#)

opt_multi_bit_flop_name_suffix

`opt_multi_bit_flop_name_suffix` *_user_suffix*

Default: ""

Data_type: string, read/write

Specifies the MBFF name suffix. When this attribute is set, the software adds the user-specified suffix at the end of the multi-bit cell instance name along with the bit-width of the multi-bit cell.

For example, the below command is used to specify `_FTREG` as suffix:

```
set_db opt_multi_bit_flop_name_suffix _FTREG
```

```
CDN_MBIT_bit1_MB_bit2_FTREG2
```

 for a 2-bit multibit cell

```
CDN_MBIT_bit1_MB_bit2_MB_bit3_MB_bit4_FTREG4
```

 for a 4-bit multibit cell

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)
- [report_multibit](#)

opt_multi_bit_flop_reorder_bits

`opt_multi_bit_flop_reorder_bits` {false | true | timing | power}

Default: false

Data_type: enum {false true timing power}, read/write

Enables swap MBFF bits to take advantage of mixed-drive strength (MDS) multi-bit cells. The following settings are possible:

- `false`: the software does not identify the MDS cells and no bit-reordering is done

- `timing`: software identifies and uses the MDS cells for timing optimization
- `power`: software identifies and uses the MDS cells for power optimization
- `true`: software identifies and uses the MDS cells for both timing and power optimization

Note: When this attribute is set to `true`, `power`, or `timing`, a "Bit-strength (strong to weak)" column will be added in the report generated using the `report_multibit -cell` parameter.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- `opt_design`
- `report_multibit`

opt_multi_bit_flop_split_post_cts

`opt_multi_bit_flop_split_post_cts {false | true | wns | tns}`

Default: `false`

Data_type: enum {false true wns tns}, read/write

Enables MBFF splitting in post-CTS stage. The following settings are available:

- `false`: Disable MBFF split in post-CTS stage.
- `wns`: Enable MBFF split in WNS optimization.
- `tns`: Enable MBFF split in TNS optimization.
- `true`: Enable MBFF split in both, WNS and TNS optimization.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)
- [report_multibit](#)

opt_multi_bit_flop_split_post_route

```
opt_multi_bit_flop_split_post_route {false | true | wns | tns}
```

Default: false

Data_type: enum {false true wns tns}, read/write

Enables MBFF splitting in post-Route stage. The following settings are available:

- `false`: Disable MBFF split in post-Route stage.
- `wns`: Enable MBFF split in WNS optimization.
- `tns`: Enable MBFF split in TNS optimization.
- `true`: Enable MBFF split in both, WNS and TNS optimization.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)
- [report_multibit](#)

opt_multi_bit_flop_split_report_failure_reason

`opt_multi_bit_flop_split_report_failure_reason {true | false}`

Default: `true`

Data_type: `bool`, read/write

When set to `false`, disables the printing of MBFF split failure reasons. By default this attribute is set to `true`, which means the reasons for MBFF instances not splitting are printed. This is the default behavior.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)
- [report_multibit](#)

opt_multi_bit_flop_split_timing_effort

`opt_multi_bit_flop_split_timing_effort {low | medium | high}`

Default: `low`

Data_type: `enum {low medium high}`, read/write

Specifies the timing effort level for MBFF splitting.

- `low` effort setting means splitting will be done only when absolutely necessary for critical paths.
- `medium` effort setting means splitting happens for more critical paths to improve timing and remove critical timing bottlenecks .
- `high` effort setting does aggressive splitting and try to improve timing.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)
- [report_multibit](#)

opt_multi_bit_merge_scan_head

```
opt_multi_bit_merge_scan_head {true | false}
```

Default: false

Data_type: bool, read/write

When set to `true`, this attribute enables the merging of the scan chain head or tail. By default, optimization does not merge the scan chain head or tail.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [eco_split_flop](#)
- [opt_design](#)
- [report_multibit](#)

opt_multi_bit_split_scan_head

```
opt_multi_bit_split_scan_head {true | false}
```

Default: `false`

Data_type: `bool`, read/write

When set to `true`, this attribute enables the splitting of the scan chain head or tail. By default, optimization does not merge the scan chain head or tail.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [eco_split_flop](#)
- [opt_design](#)
- [report_multibit](#)

opt_multi_bit_unused_bits

```
opt_multi_bit_unused_bits {true | false | pre_place | pre_cts}
```

Default: `pre_place`

Data_type: `enum {false true pre_place pre_cts}`, read/write

Specifies whether a mapped multi-bit instance can have one or more unused bits. By default, at `pre_place` stage the software tries to merge the single-bit registers to multi-bit instances without unused bits. If during merging, the software is unable to find a suitable multi-bit `lib_cell` with a width that matches the left-over bits of a bus, it will choose the closest matching width, which leaves the minimum unused bits and, therefore, increases the multi-bit merging coverage.

When set to `false`, any merging that results in unused (undriven/unloaded) bits is prevented.

When set to `true`, the software tries to merge the single-bit register to multi-bit instances with unused bits at both, `pre_place` and `pre_cts` stages.

When set to `pre_cts`, the software merges the single-bit register to multi-bit instances with unused bits at the `pre_cts` stage.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)
- [report_multibit](#)

opt_multi_bit_unused_bit_count

```
opt_multi_bit_unused_bit_count integer
```

Default: 1

Min: 1

Max: 32

Data_type: int, read/write

Specifies the maximum number of unused bits to be allowed in each multi-bit cell.

Note: To use this attribute, set the `opt_multi_bit_unused_bits` attribute to `true`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)
- [report_multibit](#)

opt_multibit_allow_shift_register_head_flop_merge

opt_multibit_allow_shift_register_head_flop_merge {true | false}

Description

Default: false

Data_type: bool, read/write

If set to true, it enables merging of head flops of shift register during multibit merging.

opt_new_inst_prefix

opt_new_inst_prefix prefix

Default: ""

Data_type: string, read/write

Specifies a prefix string to add to instances created by timing optimization. For example, if a default prefix is OFC_000234, when you use this attribute to specify the prefix, POSTROUTE_, the combined prefix added to the instance name is POSTROUTE_OFC_000234.

Note: To ignore the default prefix, set the opt_concatenate_default_and_user_prefixes attribute to false.

This is a root attribute.

- To set the value of any root attribute, use the set_db command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the get_db command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_new_net_prefix

`opt_new_net_prefix prefix`

Default: ""

Data_type: string, read/write

Specifies a prefix string to add when timing optimization creates nets. For example, if a default prefix is `PSN_000234`, when you use this attribute to specify the prefix, `POSTROUTE_`, the combined prefix added to the net name is `POSTROUTE_PSN_00234`.

Note: To ignore the default prefix, set the `opt_concatenate_default_and_user_prefixes` attribute to `false`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

`set_db attribute_name value`

- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

Related Information

- [opt_design](#)

opt_one_pass_lec

`opt_one_pass_lec {true | false}`

Default: `false`

Data_type: bool, read/write

When set to `true`, enables the one pass Logical Equivalence Check (LEC) flow. In one pass LEC, you can do a Register Transfer Level (RTL) to netlist comparison between RTL and post-route optimization (PRO) netlist - any place and route (PnR) stage netlist - in a single step.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

`set_db attribute_name value`

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_pin_swapping

```
opt_pin_swapping {true | false}
```

Default: true

Data_type: bool, read/write

Controls whether timing optimization allows pin swapping.

Note: This attribute is not enabled when the `design_flow_effort` attribute is set to `express`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_podv2_flow_effort

```
opt_podv2_flow_effort {auto | standard | extreme}
```

Default: auto

Data_type: enum {auto standard extreme}, read/write

Specifies the effort level for the `place_opt_design V2` (PODV2) flow. See [clock_opt_design](#).

The following settings are available:

- `auto`: The software follows the `design_flow_effort` attribute setting to perform optimization.

- `standard`: The software performs a fast prects optimization and runtime is less. This option is recommended in the early design stage or in simple designs for PPA estimation.
- `extreme`: The software performs a deep optimization to chase the best PPA. Runtime is high.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- [clock_opt_design](#)
- [opt_design](#)

opt_post_route_allow_overlap

`opt_post_route_allow_overlap {true | false}`

Default: `true`

Data_type: `bool`, read/write

Controls whether timing optimization allows overlap during `opt_design -post_route`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- [opt_design](#)

opt_post_route_area_reclaim

`opt_post_route_area_reclaim {none | setup_aware | hold_and_setup_aware}`

Default: none

Data_type: enum {none setup_aware hold_and_setup_aware}, read/write

This attribute can be used after routing to reclaim area during `opt_design -post_route`. It safely reclaims area while not impacting the timing and DRV violations. You can use the following options:

- `none`: No area reclaim is done. This is the default setting.
- `setup_aware`: Reclaim is done but is not hold aware. This may degrade hold timing significantly but will maintain the setup timing while giving the best possible reclaim results.
- `hold_and_setup_aware`: Reclaim is done but is both hold and setup aware. It will not give as good reclaim results as `setup_aware` alone. But it will mean both setup and hold timing will be maintained.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- [opt_design](#)

opt_post_route_art_flow

`opt_post_route_art_flow {true | false}`

Default: false

Data_type: bool, read/write

Ensures that timing optimization applies to ART-based flow. In the ART-based flow, it partially activates the design at the top-level and at the interface path of partition blocks so that there is a reduction in memory usage and run time if the design is large. This attribute marks the core logic

inside the partition as `dont_touch`, so the logic is not optimized.

Note: This attribute requires partition definition information and applies to the post-Route mode only.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_post_route_check_antenna_rules

```
opt_post_route_check_antenna_rules {true | false}
```

Default: `true`

Data_type: `bool`, read/write

Allows optimization to check/ignore antenna rules for LEF-safe swaps. The default behavior is to check the antenna rules, in order to avoid antenna violations caused by the cell swappings. The LEF files have certain antenna rules defined in them. By default, when doing LEF-safe swaps for leakage power optimization and for LEF-safe timing recovery, it is ensured that the cells have the same antenna rules. If cells do not have the same antenna rules, they are not swapped.

However, if this attribute is set to `false`, in post-Route mode, antenna rules will not be considered when doing LEF-safe swaps. Therefore, setting this attribute to `false` improves timing and leakage but it can potentially lead to routing having antenna violations, which will require a call to the `route_eco` command for cleanup.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_post_route_drv_recovery

```
opt_post_route_drv_recovery {auto | false | true}
```

Default: auto

Data_type: enum {auto true false}, read/write

Controls the DRV/glitch recovery step performed when `opt_design -post_route -setup -hold` and `opt_design -post_route -hold` commands have been specified. You can specify the following options:

- `false`: Disables DRV/glitch recovery.
- `auto`: Allows the software to determine when to trigger DRV/glitch recovery. Here recovery will be triggered if any of the DRV's or glitches degrade beyond a certain margin.
- `true`: Always triggers DRV/glitch recovery if there are any left in the design.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_post_route_fix_clock_drv

```
opt_post_route_fix_clock_drv {true | false}
```

Default: true

Data_type: bool, read/write

Controls whether [clock_post_route_repair](#) is called or not at the start of post-Route optimization.

By default this attribute is set `true`, which means that `ccopt_post_route` is called to repair clock tree design rule violations (DRVs) that have occurred in the clock tree after routing. These DRVs are most commonly the user-set clock transTime violations. To disable the fixing of DRVs, set the attribute to `false`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- [clock_post_route_repair](#)
- [opt_design](#)

opt_post_route_fix_glitch

```
opt_post_route_fix_glitch {true | false}
```

Default: `true`

Data_type: `bool`, read/write

Determines whether to fix glitch violations during SI optimization or not. To disable fixing of glitch violations, set this attribute to `false`.

Note: For SI glitch fixing, you must set the [si](#) category attribute, `si_glitch_enable_report` to `true`. If the `opt_post_route_fix_glitch` attribute is set to `true` but the `si_glitch_enable_report` attribute is set to `false`, the software will issue a warning during glitch fixing.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- [opt_design](#)

opt_post_route_fix_si_transitions

```
opt_post_route_fix_si_transitions {true | false}
```

Default: false

Data_type: bool, read/write

Enables fixing of SI-induced slew violations in post-Route optimization.

Note: The transition values will be used for maximum transition violation fixing only and will not affect the timing calculation in the timing engine. This is because the transition effect is considered by the internal SI engine during delay pushout calculation.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_post_route_hold_recovery

```
opt_post_route_hold_recovery {auto | false | true}
```

Default: auto

Data_type: enum {auto true false}, read/write

Controls the hold recovery step performed when `opt_design -post_route -setup -hold` and `opt_design -post_route -hold` commands have been specified. You can specify the following options:

- `false`: Disables hold recovery.

- `auto`: Allows the software to determine when to trigger setup recovery. Here recovery will be triggered if WNS or TNS degrade beyond a certain margin.
- `true`: Always triggers hold recovery if timing is not met, irrespective of the gain/degradation in post-route timing.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- [opt_design](#)

opt_post_route_report_si_transitions

`opt_post_route_report_si_transitions {true | false}`

Default: `false`

Data_type: `bool`, read/write

Enables reporting of SI slews for DRV maximum transition reporting only. The delay calculation will continue to use base slews for timing analysis commands, such a

as `reportTranViolation`, `report_constraint`, `report_delay_calculation`, and `report_timing`.

The DRV violations will be reported by `time_design` under the `./timingReports/<design name>.tran` file only and the number of maximum transition violations due to SI slews will be listed in the `time_design` summary report.

Note : The `opt_post_route_report_si_transitions true` setting does not enable fixing of SI slews, this command only enables reporting of them for `time_design` DRV reporting. To enable SI slew fixing, use `opt_post_route_fix_si_transitions true`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)
- [time_design](#)

opt_post_route_setup_recovery

```
opt_post_route_setup_recovery {auto | false | true}
```

Default: auto

Data_type: enum {auto true false}, read/write

Controls the setup recovery step performed when `opt_design -post_route -setup -hold` and `opt_design -post_route -hold` commands have been specified. You can specify the following options:

- `false`: Disables setup recovery.
- `auto`: Allows the software to determine when to trigger setup recovery. Here recovery will be triggered if WNS or TNS degrade beyond a certain margin.
- `true`: Always triggers setup recovery if timing is not met, irrespective of the gain/degradation in post-route timing.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_power_effort

`opt_power_effort {none | low | high}`

Default: none

Data_type: enum {none low high}, read/write

Specifies the power-driven optimization functionality. This attribute provides the effort level for optimizing leakage power and dynamic power in the design. Use this attribute along with the `opt_leakage_to_dynamic_ratio` attribute to give a superior QOR and to trigger power-driven optimization. Power-driven optimization implies that the `opt_design` command now performs Power, Performance, and Area (PPA) trade-offs at the transform level at every step of timing optimization.

You can specify any one of the following options:

`none` : Specifies that no power optimization is to be performed.

`low`: Specify this option when power optimization is not the highest priority. Using the low effort level option indicates the following:

- Power optimization is performed only at certain stages of the `opt_design` flow, which is dependent on the value of the `opt_leakage_to_dynamic_ratio` attribute. When the `opt_leakage_to_dynamic_ratio` attribute is 1.0, `opt_design` will optimize leakage power after each timing optimization step and no high leakage cells are inserted during hold fixing. When `opt_leakage_to_dynamic_ratio` attribute is 0.0, `opt_design` will optimize dynamic power at the preCTS stage. For all values of `opt_leakage_to_dynamic_ratio` attribute between 0.0 and 1.0, both of the above are done.
- Timing optimization is power-aware but uses only low-effort techniques when choosing the best timing optimization fixes to implement.
- Timing closure remains the first priority and the worst negative slack achieved is the same as that in effort mode “`none`”.
- On an average, the CPU full flow runtime impact is +26% for a `opt_leakage_to_dynamic_ratio` of 0.5 as compared to that in effort mode “`none`”. This can vary depending on both the ratio being used and on the design.
- On an average, the MEM full flow impact is +5%.
- Utilization remains roughly at par with effort level “`none`” results.
- On an average, the overall standard cell dynamic power saving is -30% and can go up to -

69% as compared to that achieved in effort mode “none”. On an average, the overall standard cell leakage power saving is -41% and can go up to -90% on some designs. This is achieved with a `opt_leakage_to_dynamic_ratio` of 0.5 and will vary if this ratio is changed.

Note: The above numbers are estimates and are completely dependent on the type of design. Therefore, they may be very different from the averages given above.

high : Use the high effort level option when having low power is the highest priority. Using the high effort level option indicates the following:

- Power optimization is performed during all calls to `opt_design` , which is dependent on the value of the `opt_leakage_to_dynamic_ratio` attribute. Timing Optimization is power aware and uses high-effort techniques when choosing the best timing optimization fixes to implement.
- Timing closure should still be achievable, if it was previously, and in the majority of cases the same worst negative slack is reached as effort mode " none ".
- CPU full flow runtime impact is on an average +63% for a `opt_leakage_to_dynamic_ratio` of 0.5 compared to effort " none ". This can vary depending on both the ratio being used and on the design.
- MEM full flow impact is on an average +11%.
- Utilization is generally improved by -1.3% on average compared to effort level " none " results.
- On an average, the overall standard cell dynamic power saving is -32% and can go up to -70% as compared to that achieved in effort mode “none”. On an average, the overall standard cell leakage power saving is -54% and can go up to -98% on some designs. This is achieved with a `opt_leakage_to_dynamic_ratio` of 0.5 and will vary if this ratio is changed.

Note : The above numbers are estimates and are completely dependent on the type of design. Therefore, they may be very different from the averages given above.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- [opt_design](#)

opt_pre_route_ndr_aware

`opt_pre_route_ndr_aware list`

Default: ""

Data_type: string, read/write

Enables NDR-based optimization by providing a list of NDR names. Using NDR-based optimization, GigaOpt assigns NDR on timing critical nets in order to improve timing. GigaOpt will automatically assess the list of NDRs provided and choose the one that can help timing. It is not recommended to use many NDRs from turn-around-time (TAT) point of view.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_preserve_all_sequential

`opt_preserve_all_sequential {true | false}`

Default: false

Data_type: bool, read/write

Preserves sequential elements when netlists are optimized. When the `opt_remove_redundant_insts` attribute is set to `true`, unobservable or useless logic is removed. However, you can retain sequential elements by setting this attribute to `true`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_preserve_hpin_function

```
opt_preserve_hpin_function {true | false}
```

Default: false

Data_type: bool, read/write

Determines whether to preserve logical functions at hierarchical module ports. When set to false, only the hierarchical module ports that have a timing assertion applied on them will be preserved (not deleted, duplicated, or polarity changed). This is the recommended setting since it allows more freedom for timing optimization while not impacting the netlist functionality or the timing constraints.

When set the true, every port of every hierarchical mode will be preserved (not deleted, duplicated, or polarity changed). This is very constraining.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_print_ports_nets_preserved_for_callback

```
opt_print_ports_nets_preserved_for_callback {true | false}
```

Description

Default: `false`

Data_type: `bool`, read/write

Prints nets and hierarchical pins involved in combinational loops that were preserved for verification.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

opt_remove_redundant_insts

```
opt_remove_redundant_insts {true | false}
```

Default: `true`

Data_type: `bool`, read/write

Enables simplifying the netlist during timing optimization. When it is set to `true`, which is the default value, the software recovers area, decreases congestion, and improves runtime by simplifying the netlist in the following ways:

- Removing dangling output instances
- Propagating constants
- Removing unobservable logic
- Remapping useless logic

The software respects the `set_dont_touch` constraint, so it does not remove the constrained flip-flops, and does not touch non-uniquified modules.

Note: This attribute is not enabled when the `design_flow_effort` attribute is set to `express`.

Note: If a netlist contains spare cells, ensure that you define them using the `set_spare_insts` command else they might be removed by this attribute.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)
- [set_spare_insts](#)

opt_report_multi_bit_unmerged_reasons

```
opt_report_multi_bit_unmerged_reasons {true | false}
```

Default: false

Data_type: bool, read/write

When this attribute is set to `true`, the reasons for unmerged multibit flops/latches are printed in the MBFF summary. For details of the use model, see [Reporting Unmerged Reasons for MBFFs](#).

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_resize_flip_flops

```
opt_resize_flip_flops {true | false}
```

Default: true

Data_type: bool, read/write

Controls whether timing optimization upsizes or downsizes flip-flops.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_resize_level_shifter_and_iso_insts

```
opt_resize_level_shifter_and_iso_insts {true | false}
```

Default: false

Data_type: bool, read/write

Controls whether timing optimization resizes shifters and isolation instances. In default mode (false), shifters and isolations are considered "don't-touch" instances and are not changed during timing optimization. When this attribute is set to true, the shifters and isolation cells are resized for timing optimization by using `opt_design`.

Note: When this attribute is set to true, all the shifters and isolation cells can be resized even if they are marked as `dont_touch`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_resize_power_switch_insts

```
opt_resize_power_switch_insts {true | false}
```

Default: false

Data_type: bool, read/write

When set to `true`, enables resizing of power switch instances during optimization. By default, this attribute is set to `false`, which means the power switch instances are not resized during optimization.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_retime_move_mux_loop_with_register

```
opt_retime_move_mux_loop_with_register {true | false}
```

Description

Default: true

Data_type: bool, read/write

Directs retiming to maintain the mux-feedback loop for a flop with the flop itself during the retiming moves. This implies that retiming cannot separate the mux loop and move logic through the feedback loop.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```


- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- `dont_retime` in [hinst](#) and [inst](#) in *Stylus Common UI Database Object Information Guide*
- `retime` in [design](#) and [module](#) in *Stylus Common UI Database Object Information Guide*
- [synthesize_design](#)

opt_retime_register_naming_suffix

```
opt_retime_register_naming_suffix string
```

Description

Default: `_reg`

Data_type: `string`, `read/write`

Marks those registers that were moved due to retiming optimization with the specified suffix.

Example

The following example instructs the tool to add the `'_retimed_reg'` suffix to all registers that are moved during retiming optimization:

```
set_db opt_retime_register_naming_suffix _retimed_reg
```

```
Setting attribute of root '/': 'opt_retime_register_naming_suffix' = _retimed_reg
1 _retimed_reg
```

The affected registers could look like the following example:

```
(n_124)); D_F_LPH0002_H 16_retime_reg(.E (ck), .D (n_118), .L2N (n_159)); INVERT_J
g48(.A (n_167), .Z (n_119)); D_F_LPH0001_E 17_retime_reg(.E (ck), .D (n_118), .L2
(n_158)); D_F_LPH0002_E 18_retime_reg(.E (ck), .D (n_112), .L2N (n_165)); INVERT_H
g52(.A (n_116), .Z (n_117)); INVERT_H g55(.A (n_104), .Z (n_115));
```

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- `dont_retime` in [hinst](#) and [inst](#) in *Stylus Common UI Database Object Information Guide*
- `retime` in [design](#) and [module](#) in *Stylus Common UI Database Object Information Guide*
- `retime_async_reset`
- `retime_original_registers` in [hinst](#) and [inst](#) in *Stylus Common UI Database Object Information Guide*
- `synthesize_design`
- `trace_retime` in [hinst](#) and [inst](#) in *Stylus Common UI Database Object Information Guide*

opt_route_opt_recovery

```
opt_route_opt_recovery {true | false | auto}
```

Default: auto

Data_type: enum {auto true false}, read/write

Controls the timing recovery step in `route_opt_design` flow. It is used when `route_opt_design` with standard effort (`set_db design_flow_effort standard`) timing is not closed. The following settings are available:

- `false`: Disables timing recovery.
- `auto`: Allows the software to determine when to trigger hold/drv recovery. Here recovery will be triggered if hold/drv degrade beyond a certain margin.
- `true`: Always triggers timing recovery if timing is not met, irrespective of the gain/degradation in post-route timing.

For example, to enable setup timing recovery, use the following settings:

```
set_db opt_route_opt_recovery true
set_db opt_post_route_setup_recovery true
```

route_opt_design

Note: You can specify the following attributes with the `opt_route_opt_recovery true` setting to enable setup/hold/drv recovery, as required. However, it will result in increased runtime.

```
set_db opt_post_route_setup_recovery true
set_db opt_post_route_hold_recovery true
set_db opt_post_route_drv_recovery true
```

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- [opt_design](#)
- [route_opt_design](#)

opt_sequential_genus_restructure_report_failure_reason

```
opt_sequential_genus_restructure_report_failure_reason {true | false}
```

Default: false

Data_type: bool, read/write

When set to true, the sequential critical region resynthesis (CRR) failure reasons are reported in the `SeqCrrFailReasons.rpt` file. The report includes the instance name and the corresponding failure reason.

Note: When CRR is enabled, Innovus uses Genus to restructure portions of the design to improve timing/area/power during setup optimization.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- [opt_design](#)

opt_setup_target_slack

`opt_setup_target_slack` *setuptargetslack*

Default: 0.0

Data_type: double, read/write

Specifies a target slack value in nanoseconds for setup analysis. During setup violation repair, the setup target slack is `opt_setup_target_slack` and the hold target slack is set to 0. Generally, you use both `opt_setup_target_slack` and `opt_hold_target_slack` for hold analysis. If you specify `opt_hold_target_slack` only, the `opt_setup_target_slack` value is 0.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_skew

`opt_skew` {true | false}

Default: true

Data_type: bool, read/write

Controls whether timing optimization runs the `opt_clock_skew` command. This attribute produces scheduling and latency files before CTS runs, or adds buffers and inverters after CTS, just as `opt_clock_skew` does. This attribute sets the `timing_analysis_useful_skew` attribute to true, so timing analysis can read the latency file. Useful skew optimization is run iteratively within `opt_design`, so you do not need to set this attribute more than once.

Note: This attribute is not enabled when the `design_flow_effort` attribute is set to `express`.

Note: This attribute is not supported during `opt_design -post_route`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)
- [opt_clock_skew](#)

opt_skew_apply_delay_limits_to_full_flow

```
opt_skew_apply_delay_limits_to_full_flow {true | false}
```

Default: `false`

Data_type: `bool`, read/write

When set to `true`, specifies that the useful skew limit is to be applied to the whole flow. With this setting, the Early Clock Flow is modified so that pre-existing tool-generated useful skews count towards the limit imposed by the `opt_skew_max_allowed_delay` attribute. The tool-generated useful skew comprises any pin insertion delay (PID) of category `useful_skew` or `global_skew`. PIDs of other categories are not considered.

By default, this attribute is set to `false`, which means the pre-existing skews are not considered while calculating the limit imposed by using the `opt_skew_max_allowed_delay` attribute.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- `opt_design`
- `opt_clock_skew`

opt_skew_ccopt

`opt_skew_ccopt {none | standard | extreme}`

Default: `standard`

Data_type: enum {none standard extreme}, read/write

Specifies the effort for useful skew in both CTS and post-CTS flow. It replaces the attributes, `opt_skew_pre_cts`, `opt_skew_post_cts`.

This attribute setting is only respected by `ccopt_design` and `opt_design -post_cts`.

- `none`: low effort `ccopt_design` with no skewing
- `standard`: low effort `ccopt_design`. This is the default setting.
- `extreme`: high effort `ccopt_design`

Note: To achieve the best possible timing result, the `extreme` setting will significantly increase the execution time of the `ccopt_design` command. Also, for best results the extreme setting should be used with `set_db design_flow_effort extreme` attribute setting.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- `opt_design`
- `opt_clock_skew`

opt_skew_post_route

`opt_skew_post_route {true | false}`

Default: `true`

Data_type: bool, read/write

When set to `false`, it disables useful skew in post-Route optimization.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

`set_db attribute_name value`

- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

Related Information

- [opt_design](#)
- [opt_clock_skew](#)

opt_skew_pre_cts

`opt_skew_pre_cts {true | false}`

Default: `true`

Data_type: bool, read/write

When set to `false`, it disables useful skew in preCTS optimization.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

`set_db attribute_name value`

- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

Related Information

- [opt_design](#)
- [opt_clock_skew](#)

opt_skew_delay_pre_cts

`opt_skew_delay_pre_cts {true | false}`

Default: `true`

Data_type: bool, read/write

Specifies that the useful skew feature is allowed to delay sequential elements in preCTS stage, in addition to just advancing them. This enables a more aggressive algorithm which is recommended when timing closure is gated by optimal skewing.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)
- [opt_clock_skew](#)

opt_skew_macro_only

`opt_skew_macro_only {true | false}`

Default: `false`

Data_type: bool, read/write

Allows you to limit skewing only for macro cells. When set to true, regular sequential elements (such as flip flops) will not be skewed.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)
- [opt_clock_skew](#)

opt_skew_max_allowed_delay

```
opt_skew_max_allowed_delay delay
```

Default: 1.0

Note: The unit is `ns`

Data_type: double, read/write

Limits the amount of slack (in nanoseconds) that the software can borrow from neighboring flip-flops when performing useful skew operations. The delay calculation and RC extraction methods might differ from those of sign-off tools, so other setup violations might occur if the software borrows too much slack. By having control over slack borrowing, you can prevent these setup violations. Limiting borrowed skew also limits the clock tree skew to avoid large hold violations.

Value Range: `min=0 max=1000`

Note:

This attribute is not honored in the `post_cts` mode of [opt_clock_skew](#).

Note: In the non-early clock flow, when [opt_clock_skew](#) is triggered inside [opt_design](#), and this attribute is set to its default value, the default value of this attribute will change from 1 to become a certain percentage of clock cycle. This is determined by the software on the basis of internal calculations.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)
- [opt_clock_skew](#)

opt_skew_min_allowed_delay

```
opt_skew_min_allowed_delay delay
```

Default: 0.0

Data_type: double, read/write

Allows the tool to skip any skewing that would be smaller than the minimum delay that is set. The purpose is to avoid creating very small delay macro models which would not be correctly honored during CTS stage but also to reduce the amount of macro models generated.

Value Range: min=0 max=1000

Note: This attribute is supported in both standalone `opt_clock_skew` command and for `opt_useful_skew` command called within `opt_design`. This is called by setting the `opt_skew` attribute to true.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```
- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)
- [opt_clock_skew](#)

opt_skew_no_boundary

`opt_skew_no_boundary {true | false}`

Default: false

Data_type: bool, read/write

Excludes or includes boundary sequential cells in addition to ordinary sequential elements in useful skew calculations.

Note: This attribute is applicable to pre-CTS, post-CTS, and post-Route useful skew behaviors.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

`set_db attribute_name value`

- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

Related Information

- [opt_design](#)
- [opt_clock_skew](#)

opt_target_based_opt_file

`opt_target_based_opt_file filename`

Default: ""

Data_type: string, read/write

Specifies the file containing information that target-based optimization (TBOpt) will use to do fine grain target-based optimization using the `opt_design -post_route -targeted` command.

The format of the TBOpt file is as follows:

```
NETS : {net1} {net2} {netN}
FROMTOS : {fromTerm1 toTerm1} {fromTerm2 toTerm2} {fromTermN toTermN}
INSTS : {inst1} {inst2} {instN}
PATHS : {endpoint1} {endpoint2} {endpointN}
END_POINT_ADJ : {endpoint1} {slack_adj: value in ps}
```

```
AREA_RECLAIM : {ModuleName} {x1 y1 x2 y2}
PWR_RECLAIM : {ModuleName} {x1 y1 x2 y2}
DRVS : {netName1} {netName2} {max_tran: e.g. 100}
```

An example of a TBOpt file is provided below:

```
NETS : {mod_i/mod_sub_i/reset_l_sync_3} {mod_i/mod_sub_i/FE_OCPN210_reset_l_sync_3}
INSTS : {mod_i/mod_sub_i/reset_l_sync_reg_15}
{mod_i/mod_sub_i/FE_OCPC210_reset_l_sync_16}
PATHS : {mod_i/mod_sub_i/ser_en/data_d1_reg_32/D}
```

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_target_based_opt_hold_file

```
opt_target_based_opt_hold_file hold_file_name
```

Default: ""

Data_type: string, read/write

Specifies the file containing information that target-based optimization (TBOpt) will use to perform hold optimization.

```
set_db opt_target_based_opt_hold_file hold_file_name
```

```
opt_design -post_route -targeted -hold
```

Note: You can launch setup optimization followed by hold optimization using the following command:

```
opt_design -post_route -targeted -setup -hold
```

Hold optimization is only called in the `opt_target_based_opt_file_only` mode.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_target_based_opt_file_only

```
opt_target_based_opt_file_only {true | false}
```

Default: `true`

Data_type: enum {false true}, read/write

This attribute is used along with the `opt_target_based_opt_file` attribute. It is set to `true` by default. When set to `true`, TBOpt will only optimize the nodes contained within the file specified using the `opt_target_based_opt_file` attribute. When set to `false`, the TBOpt will optimize other nodes in addition to those specified in the file, if the software considers them to be critical.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)

opt_tied_inputs

```
opt_tied_inputs {true | false}
```

Default: `false`

Data_type: bool, read/write

When set to `true`, forces optimization of instances with inputs tied together, even if it worsens the objective gain - in terms of area, power, and delay.

Note: This attribute does not need to be set when the `eco_remove_tied_inputs` command is run. In that case this attribute is automatically set to `true` and instances with inputs tied together are optimized. After the command has been run, the attribute reverts to the value before executing the command.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- [eco_remove_tied_inputs](#)
- [opt_design](#)

opt_time_design_compress_reports

```
opt_time_design_compress_reports {true | false}
```

Default: `true`

Data_type: bool, read/write

When set to `true`, this attribute forces `time_design` and `opt_design` to output timing reports in a compressed format.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- `opt_design`
- `time_design`

opt_time_design_expanded_view

`opt_time_design_expanded_view {true | false}`

Default: `false`

Data_type: `bool`, read/write

When set to `true`, this attribute will force `time_design` or `opt_design` to print summary timing reports as per the view in the log file.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- `opt_design`
- `time_design`

opt_time_design_num_paths

`opt_time_design_num_paths number_of_paths`

Default: `50`

Data_type: `int`, read/write

Allows you to select the number of paths that should be reported per path group during `time_design` or `opt_design` final summary reports.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- `opt_design`
- `time_design`

opt_time_design_report_net

`opt_time_design_report_net {true | false}`

Default: `true`

Data_type: `bool`, read/write

When set to true, it forces `time_design` or `opt_design` to print timing reports with the net section.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- `opt_design`
- `time_design`

opt_time_design_report_path_type

`opt_time_design_report_path_type {end | summary | full | full_clock | end_slack_only | summary_slack_only}`

Default: `full_clock`

Data_type: end summary full full_clock end_slack_only summary_slack_only, read/write

Specifies the path type for timing path reports. The path type can be one of the following:

end, summary, full, full_clock, end_slack_only, **or** summary_slack_only

For details of these path types, see [report_timing](#) -path_type.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)
- [time_design](#)

opt_time_design_vertical_timing_summary

```
opt_time_design_vertical_timing_summary {true | false}
```

Default: false

Data_type: bool, read/write

When set to `true`, prints vertical timing summary after `time_design` and final summary report.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [opt_design](#)
- [time_design](#)

opt_tns

```
opt_tns {true | false}
```

Description

Default: true

Data_type: bool, read/write

Controls whether timing slack is optimized only on the most critical path or also on other paths with negative slack.

By default, the `synthesize_design` command focuses on the critical path to produce a smaller design. When the critical path has negative slack, other near critical paths can also be relaxed to recover area.

When the `opt_tns` attribute is set to `true`, it forces the tool to consider all the endpoints for the optimization. The appropriate weight is given to the slack of each endpoint during the optimization. More drastic area recovery is not performed on violating paths to prevent worsening the total negative slack (TNS).

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [synthesize_design](#)

opt_unfix_clock_insts

```
opt_unfix_clock_insts {true | false}
```

Default: true

Data_type: bool, read/write

Controls whether the software is allowed to resize fixed sequential elements and to move them

during placement legalization. `opt_design` temporarily removes the fixed attribute of each sequential element at the beginning of this step and sets it back to its original value at the end. This attribute does not remove the routed wires of the clock tree. It minimizes the movement of sequential elements as much as possible. This attribute impacts only `opt_design -post_cts` and `opt_design -post_route`. You can set the following options:

- When set to `true`, the software is allowed to resize and move fixed registers.
- When set to `false`, the software is not allowed to resize or move fixed registers.

Note: If you specify `true` for this attribute, Cadence recommends you use the `route_design` command for routing your design.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- [opt_design](#)
- [route_design](#)

opt_use_multi_output_gates

`opt_use_multi_output_gates {true | false | auto}`

Description

Default: `auto`

Data_type: `enum`, read/write

When set, the tool can use more complex multi-output cells beyond half and full adders during mapping and incremental optimizations. The valid values for this attribute are:

<code>auto</code>	The mapper does not use multi-output cells but incremental optimizations do.
-------------------	--

false	None of the optimization steps (mapper, incremental optimizations) use multi-output cells.
true	Both the mapper and incremental optimizations use multi-output cells.

Related Information

- [synthesize_design](#)

opt_verbose

`opt_verbose {true | false}`

Default: false

Data_type: bool, read/write

Controls the amount of information timing optimization outputs to the log file. Specify true to output additional information to use to debug design issues. When you set this attribute to true, the various reasons for not fixing hold violations are listed in the log file.

For more information, see the section below.

Reasons for Hold Violations to Remain Unfixed

The following table lists the various reasons for hold violations to remain unfixed:

Reason	Description
No legal location	There are no valid locations to insert buffers on those violated nets. Limit the local placement congestion and use instance/cell padding during initial placement. In parallel, to allow the tool to insert buffers at non-legal location, set the following attribute to true: <pre>set_db opt_hold_allow_overlap true</pre>

Degradation in setup register-to-register WNS	The nets are violated in hold but they are also a part of the register-to-register setup timing-critical paths. So inserting a buffer would degrade the setup register-to-register WNS. To fix these nets at the cost of register-to-register setup degradation, set a negative setup target slack using the <code>opt_setup_target_slack</code> attribute.
Degradation in setup TNS	The nets are violated in hold but they are also timing critical during setup. So, inserting a buffer would increase the setup TNS. To fix these nets at the cost of setup TNS degradation, set the following attribute to true: <pre>set_db opt_hold_allow_setup_tns_degradation true</pre>
Degradation in setup WNS	The nets are violated in hold but they are also setup timing critical. So inserting a buffer would degrade the setup WNS. To fix these nets at the cost of setup degradation, set a negative setup target slack using the <code>opt_setup_target_slack</code> attribute.
Degradation in hold	The nets are violated in hold but inserting a buffer would further degrade hold timing. Check the topology of those nets and probably enlarge the list of available buffer/delay cells that are allowed to be used.
Degradation in maximum capacitance violation	The nets are violated in hold but inserting a buffer would create a new setup maximum capacitance violation. To fix these nets at the cost of DRV degradation, set the following attribute: <pre>set_db opt_drv false</pre>
Degradation in maximum transition violation	The nets are violated in hold but inserting a buffer would create a new setup maximum transition violation. To fix these nets at the cost of DRV degradation, set the following attribute: <pre>set_db opt_drv false</pre>
DRV violations	The nets are violated in hold but also in setup DRV, so they are skipped. To fix these nets at the cost of DRV degradation, set the following attribute: <pre>set_db opt_drv false</pre>

Nets are set as <code>dont_touch</code>	To allow hold fixing on these nets, remove the <code>dont_touch</code> attribute.
Nets are routed incorrectly	Verify the routing wires for those nets.
Nets are clock nets	No buffers can be inserted on clock nets. You should check your timing constraint file.
None of the always-on-buffers are usable	Verify the environment to allow the use of always-on-buffers.
None of the valid mode is found on a net	The tool is not able to insert a buffer on such a net. This could be related to specific design cases.
Any internal failure	The tool was not able to find a solution for fixing the violation of those nets. This could be related to libraries or specific design cases.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- [opt_design](#)

optimize_constant_0_flops

```
optimize_constant_0_flops {true | false}
```

Description

Default: `true`

Data_type: `bool`, read/write

Allows constant 0 propagation through flip-flops.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- `optimize_constant_0_seq` in [inst](#) in *Stylus Common UI Database Object Information Guide*
- [elaborate_design](#)
- [synthesize_design](#)

optimize_constant_1_flops

```
optimize_constant_1_flops {true | false}
```

Description

Default: `true`

Data_type: `bool`, `read/write`

Allows constant 1 propagation through flip-flops.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- `optimize_constant_0_seq` in [inst](#) in *Stylus Common UI Database Object Information Guide*
- [elaborate_design](#)
- [synthesize_design](#)

optimize_constant_across_preserved

`optimize_constant_across_preserved {true | false}`

Description

Default: `true`

Data_type: `bool`, read/write

Allows constant to propagate across preserved instances.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- [synthesize_design](#)

optimize_constant_feedback_seqs

`optimize_constant_feedback_seqs {true | false}`

Description

Default: `true`

Data_type: bool, read/write

Controls constant propagation through a FF with feedback. By default, these flip-flops can be optimized, that is, replaced with a logic constant.

Example

Consider the following RTL:

```
module test(clk, a, z, z2);
    input clk, a;
    output z, z2;
    wire    clk, d;
    reg     z, z2, q;

    assign d2 = 1'b1;
    assign d = q & a;

    always @(posedge clk) begin
        z2 <= d2;
        q <= d;
        z <= q;
    end
endmodule // test
```

Assume the following script:

```
set_db library tutorial.lib
set_db information_level 9
elaborate_design -script
test.tcl
#set_db optimize_constant_feedback_seqs false
synthesize_design
write_netlist
```

- When you use the default value of the `optimize_constant_feedback_seqs` attribute, the following netlist is written after mapping:

```
module test(clk, a, z, z2);
    input clk, a;
    output z, z2;
    wire clk, a;
    wire z, z2;
    assign z2 = 1'b1;
    assign z = 1'b0;
```

```
endmodule
```

Register `q` is an uninitialized flip-flop with feedback. Because the input `d` is the output of an AND logic, the stable state will be constant 0. When both the

`optimize_constant_feedback_seqs` root attribute and the `optimize_constant_feedback_seq` instance attribute for `q_reg` are set to `true`, `q_reg` is replaced with constant 0.

- When you set the `optimize_constant_feedback_seqs` attribute to `false`, the following netlist is written after mapping:

```
module test(clk, a, z, z2);
    input clk, a;
    output z, z2;
    wire clk, a;
    wire z, z2;
    wire n_0, n_1, q;
    assign z2 = 1'b1;
    fflopdp q_reg(.CK (clk), .D (n_1), .Q (q));
    inv1 g27(.A (n_0), .Y (n_1));
    nand2 g28__7837(.A (q), .B (a), .Y (n_0));
    fflopdp z_reg(.CK (clk), .D (q), .Q (z));
endmodule
```

In this case, the `q_reg` (with feedback loop) is not replaced by a constant.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- `optimize_constant_0_seq` in [inst](#) in *Stylus Common UI Database Object Information Guide*
- [elaborate_design](#)
- [synthesize_design](#)

optimize_constant_latches

```
optimize_constant_latches {true | false}
```

Description

Default: `true`

Data_type: `bool`, read/write

When set to `true`, a latch whose output never changes is replaced by the corresponding constant value.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- The following attributes in [inst](#) in *Stylus Common UI Database Object Information Guide*:

- `optimize_constant_0_seq`

- `optimize_constant_1_seq`

- [elaborate_design](#)

- [synthesize_design](#)

optimize_merge_flops

```
optimize_merge_flops {true | false}
```

Description

Default: `true`

Data_type: `bool`, read/write

Controls merging of equivalent flops. Set this attribute to `false` to prevent merging.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- `optimize_merge_seq` in [inst](#) in *Stylus Common UI Database Object Information Guide*
- [synthesize_design](#)
- `optimize_merge_latches`

optimize_merge_latches

```
optimize_merge_latches {true | false}
```

Description

Default: `true`

Data_type: `bool`, `read/write`

Controls merging of equivalent latches. Set this attribute to `false` to prevent merging.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- `optimize_merge_seq` in [inst](#) in *Stylus Common UI Database Object Information Guide*
- [synthesize_design](#)
- `optimize_merge_flops`

optimize_seq_x_to

optimize_seq_x_to {0 | 1 | x}

Description

Default: 0

Data_type: enum, read/write

Allows the propagation of the specified constant value (0, 1, or dont_care(x)) through flops/latches when they are in dont_care(x) state.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

optimize_yield

optimize_yield {false | true}

Description

Default: false

Data_type: bool, read/write

Sets the tool into yield optimization mode. You must set this attribute to `true` to use the design for manufacturing (DFM) flow.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

percent_control_tolerance_for_map

percent_control_tolerance_for_map <double>

Description

Default: 10.0

Data_type: double, read/write

Specifies the tolerance to add for tall and short limits for mapping (does not apply to doubles).

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

phys_early_skew_flow

phys_early_skew_flow {false | true}

Description

Default: false

Data_type: bool, read/write

Enables early clock flow based early skewing. The attribute works in the physical flow and needs to be set before `synthesize_design`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [synthesize_design](#)

preserve_combinational_loop_ports_nets

`preserve_combinational_loop_ports_nets {true | false}`

Description

Default: `true`

Data_type: `bool`, read/write

Preserves nets and hierarchical pins involved in combinational loops after elaboration to keep reference points for further verification.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

Related Information

- [opt_print_ports_nets_preserved_for_callback](#)

propagate_constant_from_timing_model

`propagate_constant_from_timing_model {true | false | inherited}`

Applies to:

- `hinst`
- `inst`
- `root`

Description

Default: `true` (root), `inherited` (hinst and inst)

Data_type: `bool` (root), `enum` (hinst and inst), `read/write`

Controls constant propagation from timing model instances. If your design contains instances of timing models whose output function is defined as constant 0 or 1, you must indicate whether the tool can propagate the constant. If you set the attribute to `false`, constants will not be propagated. The available values are:

<code>true</code>	Allows constant propagation from this timing model instance.
<code>false</code>	Prevents constant propagation from this timing model instance.
<code>inherited</code>	Inherits the value of the <code>propagate_constant_from_timing_model</code> root attribute.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [synthesize_design](#)

proto_feasible_target

```
proto_feasible_target {true | false}
```

Description

Default: `false`

Data_type: `bool`, `read/write`

Enables support for incomplete SDC constraints. When enabled, the mapper ignores huge negative slack endpoints in the target computation and optimization.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [synthesize_design](#)

proto_feasible_target_adjust_slack_pct

```
proto_feasible_target_adjust_slack_pct <integer>
```

Description

Default: 100

Data_type: integer, read/write

Specifies the percentage of slack value to be used as positive adjust value in `path_adjust` exceptions.

This attribute has only effect if the `proto_feasible_target` attribute was enabled.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [synthesize_design](#)
- [proto_feasible_target](#)

proto_feasible_target_threshold

proto_feasible_target_threshold <float>

Description

Default:

Data_type: delay, read/write

Specifies the minimum threshold delay for which `path_adjust` should be applied.

This attribute has only effect if the `proto_feasible_target` attribute is enabled.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- [synthesize_design](#)
- [proto_feasible_target](#)

proto_feasible_target_threshold_clock_pct

Default: 75

Data_type: int, read/write

Specifies the slack to clock period percentage for which `path_adjust` should be applied.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- `synthesize_design`
- `proto_feasible_target`
- `proto_feasible_target_threshold`

retime_async_reset

```
retime_async_reset {true | false}
```

Description

Default: `true`

Data_type: bool, read/write

Specifies whether registers with asynchronous set or reset signals should be retimed. Registers with both asynchronous set and reset signals will not be retimed regardless of this attribute's value.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- `dont_retime` in [hinst](#) and [inst](#) in *Stylus Common UI Database Object Information Guide*
- `retime` in [design](#) and [module](#) in *Stylus Common UI Database Object Information Guide*
- [retime_optimize_reset](#)
- [synthesize_design](#)

retime_effort_level

`retime_effort_level {low | medium | high}`

Description

Default: `medium`

Data_type: `enum, read/write`

Controls the optimizations used during retiming.

Using the default `medium` effort ensures that the design can be verified with the Conformal[®] Logical Equivalence Checker using the default flow. Using `high` effort causes the tool to use optimization techniques that could trade off better QoR against ease of formal verification.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- `dont_retime` in [hinst](#) and [inst](#) in *Stylus Common UI Database Object Information Guide*
- `retime` in [design](#) and [module](#) in *Stylus Common UI Database Object Information Guide*
- [synthesize_design](#)
- [write_do_lec](#)

retime_optimize_reset

```
retime_optimize_reset {true | false}
```

Description

Default: `false`

Data_type: `bool`, `read/write`

Specifies whether the asynchronous reset signals of registers should be optimized during retiming optimization. If set to `true`, any asynchronous reset signal that can be eliminated while preserving the functionality of the logic will be removed. That is, the reset will be dropped if the computation results in a `dont_care` value.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- `dont_retime` in [hinst](#) and [inst](#) in *Stylus Common UI Database Object Information Guide*
- `retime` in [design](#) and [module](#) in *Stylus Common UI Database Object Information Guide*
- [retime_async_reset](#)
- [synthesize_design](#)

retime_verification_flow

```
retime_verification_flow {true | false}
```

Description

Default: `true`

Data_type: bool, read/write

Enables retiming verification with the Conformal[®] Logical Equivalence Checker. Set this attribute to `true` before you use the `synthesize_design` command.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- `dont_retime` in [hinst](#) and [inst](#) in *Stylus Common UI Database Object Information Guide*
- `retime` in [design](#) and [module](#) in *Stylus Common UI Database Object Information Guide*
- `retime_async_reset`
- `retime_original_registers` in [hinst](#) and [inst](#) in *Stylus Common UI Database Object Information Guide*
- `synthesize_design`
- `trace_retime` in [hinst](#) and [inst](#) in *Stylus Common UI Database Object Information Guide*

syn_exact_match_sequential_async_control

`syn_exact_match_sequential_async_control {true | false}`

Description

Default: false

Data_type: bool, read/write

If set to `true`, the tool tries to avoid tying off the asynchronous control inputs of a flop to constants during technology mapping.

You must set this attribute to `true` before you load your library.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- [synthesize_design](#)
- [syn_exact_match_sequential_sync_control](#)
- [Set and Reset Synthesis Pragmas](#) in *Innovus Synthesis HDL Modeling Guide*

st_launch_wait_time

`st_launch_wait_time <integer>`

Description

Default: 10

Data_type: int, read/write

Specifies the maximum wait time (in minutes) before launching super-thread processes.

If all required resources (specified by the `max_cpus_per_server` and `super_thread_servers` attributes) can be obtained before the specified wait time is finished, the tool will start the synthesis job. If not, the tool will start with the resources that are available at the end of the wait time.

The longer the waiting time, the better the chance you have to obtain all required resources. You are encouraged to change this value according to your resource conditions.

If all required resources can be obtained before the specified wait time is finished, the tool will start

the synthesis job. If not, the tool will start with the resources that are available at the end of the wait time.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

timing_library_ui_respect_preserve

```
timing_library_ui_respect_preserve {true | false}
```

Description

Default: `true`

Data_type: `bool`, `read/write`

Specifies whether netlist editing commands should fail if they attempt to modify preserved logic. Set this attribute to `false` to allow netlist editing commands to modify preserved instances.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- `add_tieoffs`
- `delete_obj`

timing_library_use_max_cap_look_up_table

```
timing_library_use_max_cap_look_up_table {true | false}
```


Description

Default: `true`

Data_type: `bool`, `read/write`

Controls the use of frequency-based `max_cap` lookup table values from the library for design rule verification. By default, the use of frequency-based `max_cap` lookup table values is enabled.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Information

- [report_constraint](#)
- [timing_library_look_up_drv_per_frequency](#) in [timing_library Category Attributes](#)

tns_critical_range

`tns_critical_range <delay>`

Description

Default: `0.0`

Data_type: `delay`, `read/write`

When set to a positive value (in ps), logic structuring treats paths with a slack within the specified range from the worst negative slack (WNS) as critical paths to help optimizing the total negative slack (TNS).

 You cannot set the attribute to a negative value.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Information

- [synthesize_design](#)

opt_signoff

The root attributes in the `opt_signoff` category are:

- `opt_signoff_add_inst`
- `opt_signoff_add_load`
- `opt_signoff_allow_multiple_incremental`
- `opt_signoff_allow_skewing`
- `opt_signoff_along_route_buffering`
- `opt_signoff_buffer_cell_list`
- `opt_signoff_check_type`
- `opt_signoff_clock_cell_list`
- `opt_signoff_clock_max_level`
- `opt_signoff_create_smart_mmmc_db`
- `opt_signoff_delete_inst`
- `opt_signoff_disable_geometry_checks`
- `opt_signoff_dont_touch_partition_list`
- `opt_signoff_drv_margin`
- `opt_signoff_eco_file_prefix`
- `opt_signoff_em_type`
- `opt_signoff_enable_high_capacity_eco`
- `opt_signoff_fix_cell_em`
- `opt_signoff_fix_clock_drv`
- `opt_signoff_fix_data_drv`
- `opt_signoff_fix_glitch`
- `opt_signoff_fix_hold_allow_setup_degrade`
- `opt_signoff_fix_hold_allow_setup_optimization`

- `opt_signoff_fix_hold_allow_setup_tns_degrade`
- `opt_signoff_fix_hold_with_margin`
- `opt_signoff_fix_hold_variation_aware`
- `opt_signoff_fix_ir_drop`
- `opt_signoff_fix_max_cap`
- `opt_signoff_fix_max_transition`
- `opt_signoff_fix_osus_glitch`
- `opt_signoff_fix_si_slew`
- `opt_signoff_fix_slack_in_range_lower_limit`
- `opt_signoff_fix_slack_in_range_upper_limit`
- `opt_signoff_fix_xtalk`
- `opt_signoff_fix_xtalk_preserve_hold`
- `opt_signoff_fix_xtalk_preserve_setup`
- `opt_signoff_high_effort_cell_list`
- `opt_signoff_hold_target_slack`
- `opt_signoff_hold_target_slack_per_view`
- `opt_signoff_hold_xtalk_delta_threshold`
- `opt_signoff_hold_xtalk_slack_threshold`
- `opt_signoff_keep_tmp_files`
- `opt_signoff_ignore_drv_checks`
- `opt_signoff_include_hold_path_groups`
- `opt_signoff_include_setup_path_groups`
- `opt_signoff_inter_partition`
- `opt_signoff_inter_partition_exclusive`
- `opt_signoff_intra_partition`
- `opt_signoff_legal_only`
- `opt_signoff_load_cell_list`

- `opt_signoff_max_cap_margin`
- `opt_signoff_max_local_density`
- `opt_signoff_max_opt_runtime`
- `opt_signoff_max_paths`
- `opt_signoff_max_runtime`
- `opt_signoff_max_slack`
- `opt_signoff_max_transition_margin`
- `opt_signoff_nworst`
- `opt_signoff_optimize_core_only`
- `opt_signoff_optimize_replicated_modules`
- `opt_signoff_optimize_sequential_cells`
- `opt_signoff_partition_list_file`
- `opt_signoff_path_group_file_name`
- `opt_signoff_pba_effort`
- `opt_signoff_post_mask`
- `opt_signoff_power_opt_focus`
- `opt_signoff_post_sta_tcl`
- `opt_signoff_power_aware`
- `opt_signoff_pre_sta_tcl`
- `opt_signoff_prefix`
- `opt_signoff_read_eco_opt_db`
- `opt_signoff_read_irdrop_db`
- `opt_signoff_read_smart_mmmc_db`
- `opt_signoff_resize_inst`
- `opt_signoff_retime`
- `opt_signoff_retime_depth`
- `opt_signoff_retime_depth_early`

- `opt_signoff_retime_mode`
- `opt_signoff_route_congestion_threshold`
- `opt_signoff_routing_congestion_aware`
- `opt_signoff_select_drv_net_file`
- `opt_signoff_select_drv_pin_file`
- `opt_signoff_select_hold_endpoints`
- `opt_signoff_select_setup_endpoints`
- `opt_signoff_setup_recovery`
- `opt_signoff_setup_target_slack`
- `opt_signoff_setup_target_slack_per_view`
- `opt_signoff_setup_xtalk_delta_threshold`
- `opt_signoff_setup_xtalk_slack_threshold`
- `opt_signoff_set_hold_endpoints_margin`
- `opt_signoff_set_setup_endpoints_margin`
- `opt_signoff_skew_core_only`
- `opt_signoff_skip_drv_net_file`
- `opt_signoff_smart_mmmc_effort`
- `opt_signoff_smart_mmmc_force_keep_hold_view_list`
- `opt_signoff_smart_mmmc_force_keep_setup_view_list`
- `opt_signoff_smart_mmmc_max_view_count`
- `opt_signoff_smart_mmmc_reduce_clients`
- `opt_signoff_swap_inst`
- `opt_signoff_post_mask_gate_array_filler_cells`
- `opt_signoff_post_mask_decap_cells`
- `opt_signoff_post_mask_filler_cells`
- `opt_signoff_post_mask_logic_cells`
- `opt_signoff_touch_partition_list`

- `opt_signoff_upsize_in_power_opt`
- `opt_signoff_verbose`
- `opt_signoff_write_eco_opt_db`

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `opt_signoff` category, use the following command:

```
get_db -category opt_signoff
```

opt_signoff_add_inst

```
opt_signoff_add_inst {true | false}
```

Default: `true`

Data_type: Boolean

Specifies whether or not Setup/Hold/DRV optimizer is allowed to add buffer/delay/swap cell instances.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_add_load

```
opt_signoff_add_load {true | false}
```

Default: `false`

Data_type: Boolean

Specifies whether Hold optimizer is allowed to add dummy cell instances.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_allow_multiple_incremental

```
opt_signoff_allow_multiple_incremental {true | false}
```

Default: false

Data_type: Boolean

Allows to run multiple types of optimization in incremental mode. Note that there should not be any `report_*` commands between successive `opt_signoff` commands. The recommended order to fix violations is DRV, Hold, and Setup. Hold and Setup can be interchanged but DRV optimization must be performed first. You must specify a unique ECO file prefix before each optimization step. You can then source all the ECO files in the same order as these were generated to run the What-If analysis later.

Note:

If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_allow_skewing

```
opt_signoff_allow_skewing {true | false}
```

Default: false

Data_type: Boolean

Allows clock skewing to be done during Setup timing optimization to further improve Setup slack. When set to `true`, `opt_signoff -setup` works on any level of the clock tree to modify the clock skew in order to improve Setup timing closure. Modification on the clock tree can be either resizing of the last level of the clock tree or buffering to pad endpoint clock pin.

Note:

- If set, this parameter must be applied before `opt_signoff`.
- When `opt_signoff_allow_skewing` is active, `opt_signoff` will automatically unfix all clock tree

instances/nets. This is also performed when implementing the ECOs.

- As for data path optimization, skewing is not allowed to create new Hold violated paths.

Related Commands

- `opt_signoff`

opt_signoff_along_route_buffering

```
opt_signoff_along_route_buffering {true | false}
```

Default: `true`

Data_type: `Boolean`

When set to 1, the software uses node location provided through the parasitic data in order to insert buffers along the routing topology not only at the start or end of the net. This helps to better balance the slew or the load for long nets, which is typically needed for setup timing optimization or DRV fixing.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_buffer_cell_list

```
opt_signoff_buffer_cell_list cell_list
```

Default: `""`

Data_type: `String`

Specifies a list of buffer/delay cells that can be used for the timing optimizer, regardless of whether those cells have a `dontUse` attribute. When the list is empty, the software will automatically identify the list of usable buffer/delay cells.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_check_type

`opt_signoff_check_type {early | late | both}`

Default: both

Data_type: Enum

Specifies the check type for path based analysis.

Note: If set, this attribute must be applied before `write_eco_opt_db`.

Related Commands

- `opt_signoff`

opt_signoff_clock_cell_list

`opt_signoff_clock_cell_list cell_list`

Default: " "

Data_type: String

Restricts the list of buffers/inverters/gating or any other cells that are allowed to be used by ECO on the clock tree.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_clock_max_level

`opt_signoff_clock_max_level integer`

Default: 0

Data_type: Integer

Specifies the maximum levels in clock skewing thus limiting the amount of clock tree levels and avoiding the creation of an unbalanced clock tree. This attribute sets an upper boundary when adding a buffer or pair of inverters to the clock nets. The software counts the number of levels in the clock tree branch that is being modified to ensure that the maximum level will be equal to or less than the value set by this attribute.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_create_smart_mmmc_db

```
opt_signoff_create_smart_mmmc_db {true | false}
```

Default: false

Data_type: Boolean

Enables saving the additional compact timing information in the ECO DB directory during the STA session. When set to `true`, the attribute must be applied before the `write_eco_opt_db` command and has no effect on the `opt_signoff` command.

Related Commands

- `opt_signoff`

opt_signoff_delete_inst

```
opt_signoff_delete_inst {true | false}
```

Default: true

Data_type: Boolean

Enables buffer deletion when doing timing, power, or area reduction. This option is not honored by the `-leakage` parameter of `opt_signoff` since it performs only swapping.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_disable_geometry_checks

```
opt_signoff_disable_geometry_checks {true | false}
```

Default: false

Data_type: Boolean

Performs Vth swapping without checking the cell pins and geometry. Skipping the geometry checks during Vth swapping allows the software to only check the cell size while performing Vth swapping. This will lead to more Vth swaps. However, if you use this attribute, ecoRouting will need to be done once the ECOs are applied since the pin geometries between old and new cells might not match.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_dont_touch_partition_list

```
opt_signoff_dont_touch_partition_list string
```

Default: " "

Data_type: String

Selects a list of partition module names not to be touched during optimization(s). For paths that traverse across the specified hierarchies, optimization would only be applied to a portion of the paths that lie on the non-dont_touch side.

Notes:

- This parameter is exclusive to `opt_signoff_touch_partition_list`.
- This parameter must be applied before `opt_signoff` and can be changed between each call of `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_drv_margin

`opt_signoff_drv_margin value`

Default: 0.0

Data_type: Integer

Specifies a margin value by which ratio the DRV check will be relaxed or tightened.

Note:

- The software only accepts values from -0.7 to +0.7. A positive value indicates tighter constraints.
- If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_eco_file_prefix

`opt_signoff_eco_file_prefix prefix`

Default: " "

Data_type: String

Specifies the prefix name for the ECO files that are generated.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_em_type

```
opt_signoff_em_type {avg | peak | rms | all}
```

Default: all

Data_type: Enum

Fixes the electromigration (EM) type during DRV optimization in standalone Tempus ECO. You can specify any one of the following options:

- **avg:** Specifies the average current limit violations.
- **peak:** Specifies the peak current limit violations.
- **rms:** Specifies the Root-Mean-Square (RMS) current limit violations.
- **all:** Specifies the worst of the avg/peak/rms violations.

Related Commands

- `opt_signoff`

opt_signoff_enable_high_capacity_eco

```
opt_signoff_enable_high_capacity_eco {true | false}
```

Default: false

Data_type: Boolean

Enables database size reduction on the manager session for high capacity ECO timing/power closure, based on timing violations, list of partition name, or user-defined path_group. You must apply this parameter before the `write_eco_opt_db` command, whereas it is not required during the ECO fixing session. Note that in the Tempus Paradime flow or Certus, this parameter is replaced by `set_distribute_mmmc_mode -enable_high_capacity_eco`, and is set before `distribute_read_db`.

Related Commands

- `opt_signoff`

opt_signoff_fix_cell_em

`opt_signoff_fix_cell_em {true | false}`

Default: false

Data_type: Boolean

Fixes the cell level signal electromigration violations during DRV fixing using resizing and additional transforming techniques.

Note:

- If set, this parameter must be applied before `opt_signoff -drv`.
- If set during DRV fixing, the tool will fix only the cell EM violations. This implies that fixing of max_transition/max_capacitance/SI glitch/SI Xtalk/IR drop will be disabled.

Related Commands

- `opt_signoff`

opt_signoff_fix_clock_drv

`opt_signoff_fix_clock_drv`

Default: false

Data_type: Boolean

Fixes max_cap/max_tran violations on clock nets.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_fix_data_drv

`opt_signoff_fix_data_drv`

Default: true

Data_type: Boolean

Fixes max_cap/max_tran violations on the data networks. This is performed only when the `opt_signoff -drv` command is run.

Related Commands

- `opt_signoff`

opt_signoff_fix_glitch

```
opt_signoff_fix_glitch {true | false}
```

Default: false

Data_type: Boolean

Performs SI glitch fixing during DRV fixing using resizing and buffering techniques.

Notes:

- This attribute must be set during ECO DB generation.
- The glitch fixing is done before max_tran/max_cap fixing.
- If set, this attribute must be applied before `write_eco_opt_db`.

Related Commands

- `opt_signoff`

opt_signoff_fix_hold_allow_setup_degrade

```
opt_signoff_fix_hold_allow_setup_degrade {none | low | medium}
```

Default: none

Data_type: Enum

Guides the Hold fixing engine to determine the effort it has to put to resolve timing violations, and how much flexibility it has to impact the Setup timing.

- When set to `low`, the Hold optimization engine will buffer sink pins that have large Setup positive slack even though the net Setup slack is hardly meeting timing.

- When set to `medium`, the Hold optimization engine will behave the same as in the `low` mode but will also allow Setup TNS to degrade, if required.

Note:

- This attribute is honored by the Hold optimization engine.
- If set, this attribute must be applied before `opt_signoff`.
- When `opt_signoff_fix_hold_allow_setup_degrade` is set to `low` or `medium`, the `opt_signoff_fix_hold_allow_setup_tns_degrade` attribute is ignored.

Related Commands

- `opt_signoff`

opt_signoff_fix_hold_allow_setup_optimization

```
opt_signoff_fix_hold_allow_setup_optimization {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, incremental Setup fixing is performed during Hold fixing in order to create more Setup margins on Hold violated nodes that could not be fixed earlier due to Setup timing conflict.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_fix_hold_allow_setup_tns_degrade

```
opt_signoff_fix_hold_allow_setup_tns_degrade {true | false}
```

Default: `false`

Data_type: Boolean

When specified, the Setup Total Negative Slack can be degraded during Hold time violations fixing while still maintaining the Setup Worst Negative Slack in every Setup active views.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_fix_hold_with_margin

`opt_signoff_fix_hold_with_margin value`

Default: 0.0

Data_type: Double

Specifies a slack margin value in nanoseconds for Hold time violations fixing. The purpose is to overfix any Hold violations by the amount of the margin value. This margin is not applied on nets that are already meeting the Hold target slack.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_fix_hold_variation_aware

`opt_signoff_fix_ir_drop {true | false}`

Default: false

Data_type: Boolean

When set to `true`, more sizing and Vth swapping is performed during Hold fixing in order to reduce the cost of buffer insertion. This is particularly effective when the Hold timing paths have incremental delay due to SI or large SOCV impacted instances.

Note:

- This attribute is only honored by `opt_signoff -hold`.
- If set, this attribute must be applied before `opt_signoff`.

opt_signoff_fix_ir_drop

```
opt_signoff_fix_ir_drop {true | false}
```

Default: false

Data_type: Boolean

Fixes the IR drop voltages for all instances in the design.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_fix_max_cap

```
opt_signoff_fix_max_cap {true | false}
```

Default: true

Data_type: Boolean

Specifies that nets with max_cap violations should be fixed.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_fix_max_transition

```
opt_signoff_fix_max_transition {true | false}
```

Default: true

Data_type: Boolean

Specifies that nets with max_tran violations should be fixed during DRV optimization.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_fix_osus_glitch

```
opt_signoff_fix_osus_glitch {true | false}
```

Default: false

Data_type: Boolean

Fixes nets with overshoot and undershoot glitches during DRV optimization.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_fix_si_slew

```
opt_signoff_fix_si_slew {true | false}
```

Default: false

Data_type: Boolean

Reports signal integrity (SI) slews violations and fixed during DRV optimization. This is run as the first step of optimization before proceeding for any other incremental optimization.

This attribute must be set during ECO DB generation.

Note: If set, this attribute must be applied before `write_eco_opt_db` and `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_fix_slack_in_range_lower_limit

```
opt_signoff_fix_slack_in_range_lower_limit
```

Default: -1e+30

Data_type: Double

Sets the lower slack limit for a violated path to be considered during timing closure.

Related Commands

- `opt_signoff`

opt_signoff_fix_slack_in_range_upper_limit

`opt_signoff_fix_slack_in_range_upper_limit`

Default: 1e+30

Data_type: Double

Sets the upper slack limit for a violated path to be considered during timing closure.

Related Commands

- `opt_signoff`

opt_signoff_fix_xtalk

`opt_signoff_fix_talk {true | false}`

Default: false

Data_type: Boolean

Reduces the crosstalk on the net that violates the thresholds set by you.

Note: If set, this attribute must be applied before `write_eco_opt_db` and `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_fix_xtalk_preserve_hold

```
opt_signoff_fix_talk_preserve_hold {true | false}
```

Default: false

Data_type: Boolean

When this attribute is set to `true`, the Xtalk fixing engine is not allowed to degrade Hold timing.

Note:

- This attribute is only honored by `opt_signoff -drv` when Xtalk fixing is enabled.
- If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_fix_xtalk_preserve_setup

```
opt_signoff_fix_talk_preserve_setup {true | false}
```

Default: false

Data_type: Boolean

When this attribute is set to `true`, the Xtalk fixing engine is not allowed to degrade Setup timing.

Note:

- This attribute is only honored by `opt_signoff -drv` when Xtalk fixing is enabled.
- If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_high_effort_cell_list

```
opt_signoff_high_effort_cell_list list of cells
```

Default: " "

Data_type: String

Enables the controlled utilization of high-leakage or high-power cells during signoff optimization. These cells are typically classified as `dont_use` or `dont_touch` due to their power characteristics; however, they may be beneficial in specific scenarios. When this attribute is specified, ensure that the `dont_use` constraint is set to `false` for cells included in `opt_signoff_high_effort_cell_list`.

- In setup timing optimization, the cells are considered only if they are essential to meet timing requirements without causing a significant increase in power consumption.
- In power optimization, the cells may be inserted if their usage leads to an overall decrease in power usage, such as enhancements in fanin/fanout cone.

This attribute permits considering the specified cells as high-effort resources, using them only when necessary to enhance timing or power efficiency, while maintaining the broader design objective of reducing power impact.

Related Commands

- `opt_signoff`

opt_signoff_hold_target_slack

`opt_signoff_hold_target_slack value`

Default: 0.0

Data_type: Double

Specifies a target slack value in nanoseconds for hold timing optimize.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_hold_target_slack_per_view

`opt_signoff_hold_target_slack_per_view {viewName1 slack1 viewName2 slack2 ... }`

Default: " "

Data_type: String

When enabled, this attribute allows to force a specific hold target slack per view. If a given Hold active view is not listed in this attribute, the software will use the global Hold target slack set by the `opt_signoff_hold_target_slack` attribute for that view.

Note:

- This attribute is honored by Setup, Hold, Area, Power, and Leakage optimization engines.
- If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_hold_xtalk_delta_threshold

`opt_signoff_hold_xtalk_delta_threshold value_in_ns`

Default: 0.3 ns

Data_type: Double

Fixes the nets with the crosstalk delta delay value equal to or greater than a specified threshold value for hold views.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_hold_xtalk_slack_threshold

`opt_signoff_hold_xtalk_slack_threshold value_in_ns`

Default: 1000.0 ns

Data_type: Double

Fixes nets with the slack threshold value equal to or less than a specified threshold value for hold views.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_keep_tmp_files

```
opt_signoff_keep_tmp_files {true | false}
```

Default: false

Data_type: Boolean

Keeps all the temporary files generated by DMMMC and Tempus clients for debugging purposes.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_ignore_drv_checks

```
opt_signoff_ignore_drv_checks {true | false}
```

Default: false

Data_type: Boolean

Specifies that DRV checks should be ignored in setup/hold/area/leakage/dynamic optimization.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_include_hold_path_groups

```
opt_signoff_include_hold_path_groups
```

Default: " "

Data_type: String

Allows you to select which path group should be considered for Hold timing during `opt_signoff`. It is set before `write_eco_opt_db`, and also specified before `opt_signoff` in case of a different session.

Related Commands

- `opt_signoff`

opt_signoff_include_setup_path_groups

`opt_signoff_include_setup_path_groups`

Default: " "

Data_type: String

Allows you to select which path group should be considered for Setup timing during `opt_signoff`. It is set before `write_eco_opt_db`, and also specified before `opt_signoff` in case of a different session.

Related Commands

- `opt_signoff`

opt_signoff_inter_partition

`opt_signoff_inter_partition`

Default: " "

Data_type: String

Selects a list of partition module names to be optimized for the interface logic of the specified partition. During the High Capacity ECO flow, it will reduce the database having the interface logic of the listed partitions only. Each listed partition is treated separately, thus all from/to paths for each partition are included. This attribute must be applied before `write_eco_opt_db` and `opt_signoff`. In case of the Tempus Paradime flow or Certus, it must be applied before `distribute_read_db`.

Note:

It is mandatory to list the partition name inside the file specified with `opt_signoff_partition_list_file`.

- For High Capacity ECO flow, `opt_signoff_partition_list_file` should be specified before `distribute_read_db`.
- For non-High Capacity ECO flow, `opt_signoff_partition_list_file` should be specified before first optimization.

However, any partition name, which is a part of `opt_signoff_inter_partition` and missing in `opt_signoff_partition_list_file` will result in an error, as shown below:

```
**ERROR: (IMPESO-616): Partition 'ptn_wrapper' is missing in  
opt_signoff_partition_list_file file. Partition specified for inter/intra partition  
fixing should be present in opt_signoff_partition_list_file list as well.
```

In case of Certus, since `opt_signoff_partition_list_file` is not required, the software will use `opt_signoff_distributed_optimization_block_restore_file` to identify the partition names.

Related Commands

- `opt_signoff`

opt_signoff_inter_partition_exclusive

`opt_signoff_inter_partition_exclusive`

Default: " "

Data_type: String

Selects a list of partition module pair names to be optimized for the interface logic between the specified partitions. During the High Capacity ECO flow, the tool will reduce the database having the interface logic of the listed partitions pairs only. Each listed partition pair is treated separately, all from/to paths between partition pairs are included.

Notes:

- This parameter must be applied before `write_eco_opt_db` and `opt_signoff`. However, in case of the Tempus Paradime flow or Certus, it must be applied before `distribute_read_db`.
- It is mandatory to list the partition name(s) inside the file specified using the `opt_signoff_partition_list_file` parameter:
 - For High Capacity ECO flow, the `opt_signoff_partition_list_file` parameter must be specified before `distribute_read_db`.

- For non-High Capacity ECO flow, the `opt_signoff_partition_list_file` parameter must be specified before the first optimization.

However, an error is issued when any partition name, which is a part of

`opt_signoff_inter_partition_exclusive` is missing in `opt_signoff_partition_list_file`, as shown below:

```
**ERROR: (IMPESO-616): Partition 'tdsp_core' is missing in
opt_signoff_partition_list_file file. Partition specified for
inter/intra/inter_partition_exclusive partition fixing should be present in
opt_signoff_partition_list_file list as well.
```

In case of Certus, since `opt_signoff_partition_list_file` is not required, the software will use `opt_signoff_distributed_optimization_block_restore_file` to identify the partition names.

opt_signoff_intra_partition

`opt_signoff_intra_partition`

Default: " "

Data_type: String

Selects a list of partition module names to be optimized for the core logic of the specified partition. During the High Capacity ECO flow, it will reduce the database having the core logic of the listed partitions only. This attribute must be applied before `write_eco_opt_db` and `opt_signoff`. In case of the Tempus Paradime flow or Certus, it must be applied before `distribute_read_db`.

Note:

It is mandatory to list the partition name inside the file specified with

`opt_signoff_partition_list_file`.

- For High Capacity ECO flow, `opt_signoff_partition_list_file` should be specified before `distribute_read_db`.
- For non- High Capacity ECO flow, `opt_signoff_partition_list_file` should be specified before first optimization.

However, any partition name, which is a part of `opt_signoff_intra_partition` and missing in `opt_signoff_partition_list_file` will result in an error :

```
**ERROR: (IMPESO-616): Partition 'ptn_wrapper' is missing in
opt_signoff_partition_list_file file. Partition specified for inter/intra partition
fixing should be present in opt_signoff_partition_list_file list as well.
```

In case of Certus, since `opt_signoff_partition_list_file` is not required, the software will use `opt_signoff_distributed_optimization_block_restore_file` to identify the partition names.

Related Commands

- `opt_signoff`

opt_signoff_legal_only

```
opt_signoff_legal_only {true | false}
```

Default: true

Data_type: Boolean

Enables the tool to use the physical data to search for legal location while doing any buffer insertion or instance movement. This ensures that there is no instance overlap at the end of the ECO fixing stage, and after sourcing the ECO file in the implementation tool, you can directly run ECO routing.

When this parameter is set to `false`, `opt_signoff` will not check for legal location although physical data is provided. In this mode, `opt_signoff` behaves in a logical mode flow, which means that physical data will be ignored and this might create an instance overlap and will not check local density while inserting buffers. Also, the routing congestion checks are skipped when this parameter is set to `false`. So it is recommended that you first run ECO fixing in the legal-only mode (which is the default mode) and then, depending on the requirement, you may perform an incremental run by setting `opt_signoff_legal_only` to `false`.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_load_cell_list

```
opt_signoff_load_cell_list cell_list
```

Default: ""

Data_type: String

Specifies the list of load cells to be inserted during Hold fixing, regardless of whether those cells have a `dontUse` attribute.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_max_cap_margin

`opt_signoff_max_cap_margin cap_margin_value`

Default: 0.0

Data_type: Double

Minimum value: -10.000000

Maximum value: 0.990000

Specifies the ratio by which the max_cap check should be relaxed or tightened.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_max_local_density

`opt_signoff_max_local_density threshold_value`

Default: -1.0

Data_type: Double

Applies a local placement density threshold that the software will honor during timing optimization. When the local density for a given area has overpassed the threshold, the software stops adding or moving more instances in that local area.

Using the default value, the software will not perform any local placement density checks because by default it is running with `opt_signoff_legal_only true`.

This attribute [accepts values greater than 0](#). For example, to allow a maximum local placement density of 98%, set the value to 0.98.

Note:

- This attribute is honored by DRV, Setup, and Hold optimization engines.
- If set, this attribute must be applied before `opt_signoff`.
- The `opt_signoff_max_local_density` (greater than 1) and `opt_signoff_legal_only false` settings can cause some placement in later stages while doing `eco_route`.

Related Commands

- `opt_signoff`

opt_signoff_max_opt_runtime

`opt_signoff_max_opt_runtime integer`

Default: 0

Data_type: Integer

Specifies the timeout period for the timing or power optimization phase after which `opt_signoff` will exit the session. When the value of this attribute is set to a positive value, the software restricts the optimization runtime by that value, specified in minutes. The default value of this attribute is 0. This means that there is no limit for the timeout period. When the timeout period is reached, the software will exit the optimization phase, and displays the Tempus prompt with a stable DB.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_max_paths

`opt_signoff_max_paths integer`

Default: 5000000

Data_type: Integer

Specifies the max paths or points control for path-based analysis.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_max_runtime

`opt_signoff_max_runtime integer`

Default: 0

Data_type: Integer

Specifies the time out period (in minutes) after which `opt_signoff` will exit the session.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_max_slack

`opt_signoff_max_slack value`

Default: 0.0

Data_type: Double

Specifies the maximum slack value to be used during ECO Timing DB generation when using PBA. Setting a positive `opt_signoff_max_slack` value forces the tool to retime positive GBA slack paths so that ECO optimization engine can benefit from an extra PBA Setup slack credit when doing Hold fixing or Power recovery. A positive `opt_signoff_max_slack` value implies larger runtime during ECO Timing DB generation, but tool will retime only as many paths as what is specified through the `opt_signoff_max_paths` attribute.

Note: If set, this attribute must be applied before `write_eco_opt_db`.

Related Commands

- `opt_signoff`

opt_signoff_max_transition_margin

`opt_signoff_max_transition_margin tran_margin_value`

Default: 0.0

Data_type: Double

Minimum value: -10.000000

Maximum value: 0.990000

Specifies the ratio by which the max_tran check should be relaxed or tightened.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_nworst

`opt_signoff_nworst integer`

Default: 50

Data_type: Integer

Specifies the nworst control for path-based analysis.

Note: If set, this attribute must be applied before `write_eco_opt_db`.

Related Commands

- `opt_signoff`

opt_signoff_optimize_core_only

`opt_signoff_optimize_core_only {true | false}`

Default: false

Data_type: Boolean

Performs only reg2reg optimization, when set to `true`.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_optimize_replicated_modules

```
opt_signoff_optimize_replicated_modules {true | false}
```

Default: `false`

Data_type: Boolean

Allows the software to perform timing optimization inside replicated hierarchies while generating only one ECO file per sets of clones. The cell names of the replicated hierarchies have to be listed in a file pointed by the `opt_signoff_partition_list_file` attribute.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_optimize_sequential_cells

```
opt_signoff_optimize_sequential_cells {true | false}
```

Default: `true`

Data_type: Boolean

Allows the software to perform timing optimization for sequential cells.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_partition_list_file

`opt_signoff_partition_list_file filename`

Default: ""

Data_type: String

Enables the software to select the hierarchical instances that should be treated as partitions. The file specified with this parameter must contain a list of module names of those hierarchical instances. The `opt_signoff` command will generate an ECO file for the top module and the partitions (with relative placement locations for cells). The default ECO files are generated as mentioned below:

`top_eco_tempus.tcl` and `top_eco_innovus.tcl`; `blockA_eco_tempus.tcl` and `blockA_eco_innovus.tcl` and so on.

where the top ECO file contains all ECOs, which is a union of top and partition ECOs.

Notes:

- It is mandatory that the modules that are treated as partitions are of type FENCE.
- To avoid performing ECO in a given partition, you need to apply `dont_touch` attribute on that module.
- To perform top-level buffering only, you need to apply `dont_touch` on all the partitions.
- If set, this parameter must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_path_group_file_name

`opt_signoff_path_group_file_name filename`

Default: ""

Data_type: String

Allows you to pass a TCL file, which will be sourced on each Tempus client, after they loaded their respective saved session, which contains the path group definitions to be used by the tool when running in the High Capacity ECO mode. Those path groups can then be referred to `set_db`

`opt_signoff_include_setup_path_groups` and `set_db opt_signoff_include_hold_path_groups`

applied on the manager session.

Note: This parameter is only applicable for the Tempus Paradime flow or Certus, and must be specified before `distribute_read_db`.

Related Commands

- `opt_signoff`

opt_signoff_pba_effort

```
opt_signoff_pba_effort {medium | high}
```

Default: high

Data_type: Enum

When high effort mode is specified, all the optimization engines will perform more aggressive timing closure or area/power recovery with respect to path-based (PBA) timing.

Note: If set, this attribute must be applied before `write_eco_opt_db` and `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_post_mask

```
opt_signoff_post_mask {true | false}
```

Default: false

Data_type: Boolean

Enables the timing closure engine to perform the following netlist changes without touching the base layer mask:

- Inserts the Gate Array buffers or an inverter pair in place of existing Gate Array filler cells.
- Resizes regular cells to Gate Array cells. It also resize Gate Array cells to Gate Array cells.
- Deletes a regular cell.

The software automatically identifies the Gate Array cells by checking which library instances have the same SITE name as the Gate Array filler cells specified by the user. Here, empty spaces will be

filled up by the Gate Array filler cells to ensure 100% placement density.

Note:

- This is supported by DRV, Setup, and Hold fixing.
- When `set_db add_tieoffs_cells {{<tieLo_name> <tieHi_name>}}` setting is used, the Metal ECO flow will insert the TieLo cells to connect dangling input pins, in case the existing TieLo cells in the floorplan are extremely far or too overloaded.

Related Commands

- `opt_signoff`

opt_signoff_power_opt_focus

`opt_signoff_power_opt_focus {total | leakage | dynamic | leakageOnly | dynamicOnly}`

Default: `total`

Data_type: Enum

Specifies whether a given netlist should be optimized only for leakage or dynamic power reduction, or more for leakage or dynamic power while still reducing the overall total power during total power optimization.

The following options are available:

- `total`: The power optimization engine focuses on getting the optimal total power in the design.
- `leakage`: The power optimization engine optimizes total power but focuses more on leakage power reduction than on dynamic power reduction.
- `dynamic`: The power optimization engine optimizes total power but focuses more on dynamic power reduction than on leakage power reduction.
- `leakageOnly`: The power optimization engine focuses only on leakage power reduction.
- `dynamicOnly`: The power optimization engine focuses only on dynamic power reduction.

Note:

- This attribute is honored by `opt_signoff -power` and also by `opt_signoff -drv/-setup/-hold` when the `opt_signoff_power_aware` attribute is set to `true`.
- If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_post_sta_tcl

`opt_signoff_post_sta_tcl filename`

Default: ""

Data_type: String

Specifies the file containing the TCL code that will be executed immediately after signoff timing analysis per view.

Note: If set, this attribute must be applied before `write_eco_opt_db`.

Related Commands

- `opt_signoff`

opt_signoff_power_aware

`opt_signoff_power_aware {true | false}`

Default: false

Data_type: Boolean

Tempus ECO provides the ability to take power as part of the overall cost function when doing timing closure.

When this parameter is enabled before the start of an optimization session, the current optimization and any subsequent optimizations (if running in incremental mode) checks for power changes for all optimizers and transforms during any changes in the design.

In any of the above cases, the first priority remains getting best final timing but enabling this option will get there with possibly slighter ECO although resulting in lesser final power.

For example, while fixing a Setup timing path violation, by default, the tool chooses to size one instance to a high leakage cell, while with the `-opt_signoff_power_aware` mode, the tool tries to get to same timing by doing multiple sizing to non-high leakage cells.

Note :

- This feature requires power activity to be annotated on the netlist with the `report_power` command, either through external activity file (recommended) or using propagated activity based on some default toggling rate.
- This feature honors the `-opt_signoff_power_opt_focus` parameter, which means that you can decide to force the tool in reducing more the total or leakage or dynamic power increase.

Related Commands

- `opt_signoff`

opt_signoff_pre_sta_tcl

`opt_signoff_pre_sta_tcl filename`

Default: ""

Data_type: Boolean

Specifies the file containing the TCL code that will be executed immediately before signoff timing analysis per view.

Note: If set, this attribute must be applied before `write_eco_opt_db`.

Related Commands

- `opt_signoff`

opt_signoff_prefix

`opt_signoff_prefix prefix`

Default: ESO

Data_type: String

Specifies the prefix to be assigned to the new instances inserted.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_read_eco_opt_db

`opt_signoff_read_eco_opt_db dir_name`

Default: ""

Data_type: String

Specifies the path of directory where previously generated database files are stored.

Related Commands

- `opt_signoff`

opt_signoff_read_irdrop_db

Default: ""

Data_type: String

Specifies the path of the directory where command, `debug_irdrop`, generated IR drop DB files are kept.

Related Commands

- `opt_signoff`

opt_signoff_read_smart_mmmc_db

`opt_signoff_read_smart_mmmc_db <dirName>`

Default: ""

Data_type: String

Points Tempus ECO to the ECO DB directory containing the additional compact timing information during the ECO fixing session. This attribute must be set before the design is loaded and must point to an ECO DB directory, which was previously generated through `write_eco_opt_db` using the setting, `set_db opt_signoff_create_smart_mmmc_db true`.

Note: Before `opt_signoff`, you still need to point to the ECO DB directory using the setting, `set_db opt_signoff_read_smart_mmmc_db <dirName>`.

Related Commands

- `opt_signoff`

opt_signoff_resize_inst

```
opt_signoff_resize_inst {true | false}
```

Default: `true`

Data_type: Boolean

Enables or disables resizing transform.

Related Commands

- `opt_signoff`

opt_signoff_retime

```
opt_signoff_retime {none | aocv | path_slew_propagation | aocv_path_slew_propagation |  
waveform_propagation | aocv_waveform_propagation}
```

Default: `path_slew_propagation`

Data_type: Enum

Selects the retime mode for path-based analysis.

For more details related to the parameters, see the `report_timing` command.

Related Commands

- `opt_signoff`

opt_signoff_retime_depth

`-opt_signoff_retime_depth {default|infinite}`

Default: default

Data_type: Enum

Retimes depth for exhaustive path-based analysis.

Related Commands

- `opt_signoff`

opt_signoff_retime_depth_early

`-opt_signoff_retime_depth_early {default|infinite}`

Default: default

Data_type: Enum

Retimes depth for exhaustive path-based early analysis.

Related Commands

- `opt_signoff`

opt_signoff_retime_mode

`-opt_signoff_retime_mode {none|path|exhaustive}`

Default: path

Data_type: Enum

Retimes mode for path-based analysis.

Related Commands

- `opt_signoff`

opt_signoff_route_congestion_threshold

```
opt_signoff_route_congestion_threshold threshold_value
```

Default: 0.7

Data_type: Double

Allows to change the threshold value at which the software considers a local area for routing congestion buffering, to further allow any ECO change. This parameter accepts value between 0 (minimum) and 1 (maximum).

To tighten the routing congestion checks, apply a value between 0 and 0.7. To relax the routing congestion checks, apply a value between 0.7 and 1.

- *Min value:* 0
- *Max value:* 1

Note:

- This parameter is honored by DRV, Setup, Hold, Area, and Power optimization engines.
- If set, this parameter must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_routing_congestion_aware

```
opt_signoff_routing_congestion_aware {true | false}
```

Default: false

Data_type: Boolean

Checks routing congestion before adding buffers.

Related Commands

- `opt_signoff`

opt_signoff_select_drv_net_file

```
opt_signoff_select_drv_net_file {true | false}
```

Default: ""

Data_type: String

Specifies a file containing the list of nets to be optimized during DRV fixing. The fixing of max_transition, max_capacitance, SI glitch, SI Xtalk, and SI Slew violations occurs only on the specified nets and the other violating nets are ignored. The file specified includes one net per line.

Note:

- Wild cards and regular expressions are not supported in this file.
- If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_select_drv_pin_file

```
opt_signoff_select_drv_pin_file
```

Default: ""

Data_type: String

Allows you to provide a list of instance pin, which is optimized during `opt_signoff -drv`. It is set before `write_eco_opt_db` for the High Capacity ECO flow, and also specified before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_select_hold_endpoints

```
opt_ signoff_select_hold_endpoints    file_name
```

Default: ""

Data_type: String

Specifies a file containing the list of selected endpoints for hold fixing.

Note: When including endpoints, the `opt_signoff_optimize_core_only` attribute must be set to `false`.

File format:

```
<View> include/exclude <Endpoints>
```

`<View>` can be specified as either `viewName` or `V*` (if the endpoints are to be included/excluded for all views)

`<Endpoints>` can be specified in three ways:

- As space-separated list of endpoint names: for example, `E1 E2 E3`.
- Slack range in nanosecond: `<minSlack> <maxSlack>`, where all endpoints with the slack greater than or equal to `minSlack` and slack less than or equal to `maxSlack` will be included/excluded, for example, `-2.0 -1.0`.
- `E*`, if all endpoints for the view need to be included/excluded.

Related Topics

- ["Signoff ECO" chapter in the *Tempus Stylus User Guide*:](#)
 - [Optimization using Endpoint Control](#)

Related Command

- `opt_signoff`

opt_signoff_select_setup_endpoints

```
opt_ signoff_select_setup_endpoints  file_name
```

Default: ""

Data_type: String

Specifies a file containing the list of selected endpoints for setup fixing.

Note: When including endpoints, the `opt_signoff_optimize_core_only` attribute must be set to `false`.

File format:

<View> include/exclude <Endpoints>

<View> can be specified as either viewName or V* (if the endpoints are to be included/excluded for all views)

<Endpoints> can be specified in three ways:

- As space-separated list of endpoint names: for example, E1 E2 E3.
- Slack range in nanosecond: <minSlack> <maxSlack>, where all endpoints with the slack greater than or equal to minSlack and slack less than or equal to maxSlack will be included/excluded, for example, -2.0 -1.0.
- E*, if all endpoints for the view need to be included/excluded.

Related Topics

- ["Signoff ECO" chapter in the *Tempus Stylus User Guide*:](#)
 - [Optimization using Endpoint Control](#)

Related Command

- `opt_signoff`

opt_signoff_setup_recovery

```
opt_signoff_setup_recovery {true | false}
```

Default: false

Data_type: Boolean

Specifies power-aware setup recovery for setup fixing.

Related Commands

- `opt_signoff`

opt_signoff_setup_target_slack

```
opt_signoff_setup_target_slack slack
```

Default: 0.0

Data_type: Double

Specifies a target slack value in nanoseconds for Setup timing optimization.

Related Commands

- `opt_signoff`

opt_signoff_setup_target_slack_per_view

```
opt_signoff_setup_target_slack_per_view {viewName 1 slack1 viewName2 slack2 ... }
```

Default: ""

Data_type: String

When enabled, this attribute allows to force a specific setup target slack per view. If a given Setup active view is not listed in this attribute, the software will use the global Setup target slack set by the `opt_signoff_setup_target_slack` attribute for that view.

Note:

- This attribute is honored by Setup, Hold, Area, Power, and Leakage optimization engines.
- If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_setup_xtalk_delta_threshold

```
opt_signoff_setup_xtalk_delta_threshold value_in_ns
```

Default: 0.3 ns

Data_type: Double

Fixes the nets with the crosstalk delta delay value equal to or greater than a specified threshold value for setup views.

Related Commands

- `opt_signoff`

opt_signoff_setup_xtalk_slack_threshold

`opt_signoff_setup_xtalk_slack_threshold value_in_ns`

Default: 1000.0 ns

Data_type: Double

Fixes the nets with the slack threshold value equal to or less than a specified threshold value for setup views.

Related Commands

- `opt_signoff`

opt_signoff_set_hold_endpoints_margin

`opt_signoff_set_hold_endpoints_margin file_name`

Default: " "

Data_type: String

Specifies endpoint-specific margin for hold fixing.

Related Command

- `opt_signoff`

opt_signoff_set_setup_endpoints_margin

`opt_signoff_set_setup_endpoints_margin file_name`

Default: " "

Data_type: String

Specifies endpoint-specific margin for setup fixing.

Related Command

- `opt_signoff`

opt_signoff_skew_core_only

`opt_signoff_skew_core_only {true | false}`

Default: false

Data_type: Boolean

When set to `true`, it does not allow the skewing engine to skew sequential elements that are part of an IO boundary timing path.

Note:

- This attribute is only honored by `opt_signoff -setup`.
- If set, this attribute must be applied before `opt_signoff`.

Related Command

- `opt_signoff`

opt_signoff_skip_drv_net_file

`opt_signoff_skip_drv_net_file file_name`

Default: " "

Data_type: String

Specifies a file containing the list of nets to be excluded during DRV fixing. The fixing of `max_transition` and `max_capacitance` violations occurs on nets other than specified nets. The file specified includes one net per line.

Note:

- Wild cards and regular expressions are not supported in this file.
- If set, this attribute must be applied before `opt_signoff`.

Related Command

- `opt_signoff`

opt_signoff_smart_mmmc_effort

```
opt_signoff_smart_mmmc_effort {low | medium | high}
```

Default: low

Data_type: Enum

Controls the trade-off between memory or runtime and timing QoR for timing or power ECO closure flow. This means that higher the effort, the lesser memory or runtime but lesser timing QoR while running `opt_signoff`.

- `low`: Is the recommended (default) setting and ensures there is no timing QoR loss.
- `medium`: Reduces memory and runtime significantly with minimal timing QoR loss.
- `high`: Reduces memory and runtime further when compared to the medium mode, but with additional timing QoR loss.

In the `low` effort mode, which is the default and recommended setting, there is no compromise on timing QoR.

The `medium` and `high` effort modes can be used for initial rounds of ECO timing/power ECO closure flow.

Note: This parameter must be set before the design is loaded.

Related Commands

- `opt_signoff`

opt_signoff_smart_mmmc_force_keep_hold_view_list

```
opt_signoff_smart_mmmc_force_keep_hold_view_list <string>
```

Default: " "

Data_type: String

List of hold views, which have to be retained after SmartMMMC view compaction. These views represent the minimum hold views retained by the system, on top of which additional views can be

retained.

Related Commands

- `opt_signoff`

opt_signoff_smart_mmmc_force_keep_setup_view_list

```
opt_signoff_smart_mmmc_force_keep_setup_view_list <string>
```

Default: " "

Data_type: String

List of setup views, which have to be retained after SmartMMMC view compaction. These views represent the minimum setup views retained by the system, on top of which additional views can be retained.

Related Commands

- `opt_signoff`

opt_signoff_smart_mmmc_max_view_count

```
opt_signoff_smart_mmmc_max_view_count {{setup <maxViewCount>} {hold <maxViewCount>}}  
{drv <maxViewCount>}}
```

Default: " "

Data_type: String

Allows you to force the software to collect only a given number of views to cover Setup timing, Hold timing, and DRV. It acts as a limit to prevent smartMMMC from selecting more views than what is specified by the user. This attribute is especially useful in combination with `opt_signoff_smart_mmmc_reduce_clients` as it enables a user-defined limit on hardware resources for Tempus clients.

Note:

- This attribute is only honored by Tempus Paradime or Certus, and should be set before design loading.
- When `opt_signoff_smart_mmmc_max_view_count` is specified, `opt_signoff_smart_mmmc_effort_level` is ignored.

- When `opt_signoff_smart_mmmc_max_view_count` is specified, you need to provide `maxViewCount` for each Setup, Hold, and DRV field.
- `maxViewCount` should be equal or greater than 1 for setup and hold but can be 0 for drv only if you do not specify `set_analysis_view -drv` earlier.

Related Commands

- `opt_signoff`

opt_signoff_smart_mmmc_reduce_clients

`opt_signoff_smart_mmmc_reduce_clients {true | false}`

Default: `false`

Data_type: Boolean

When set to `true`, the SmartMMMC engine evaluates which views should be retained and available on Tempus clients. This helps to reduce the number of Tempus clients started by Tempus Paradime or Certus.

Default: `false`

Note:

- This attribute is only honored by Tempus Paradime or Certus, and must be set before design loading.
- The Tempus STA sessions must have been saved with `opt_signoff_create_smart_mmmc_db true`.
- SmartMMMC must be enabled through `set_distributed_mmmc_mode -enable_smart_mmmc`.

Related Commands

- `opt_signoff`

opt_signoff_swap_inst

`opt_signoff_swap_inst {true | false}`

Default: `true`

Data_type: Boolean

Enables/disables VT swap resizing. The setup/leakage optimizers do not honor this attribute and always perform VT-swapping.

Related Commands

- `opt_signoff`

opt_signoff_post_mask_gate_array_filler_cells

`opt_signoff_post_mask_gate_array_filler_cells {list_of_Gate_Array_filler_cells_name}`

Default: ""

Data_type: String

Enables the software to list all the Gate Array filler cells that can be deleted or inserted back.

Note: This attribute is applicable in the Metal ECO mode only.

Related Commands

- `opt_signoff`

opt_signoff_post_mask_decap_cells

`opt_signoff_post_mask_decap_cells list_of_decap_cells`

Default: ""

Data_type: String

Lists all the programmable decap cells that are allowed to be removed/reinserted in post-mask mode.

Note: This attribute is applicable in the post-mask mode only.

Related Commands

- `opt_signoff`

opt_signoff_post_mask_filler_cells

`opt_signoff_post_mask_filler_cells list_of_filler_cells`

Default: ""

Data_type: String

Lists all the programmable filler cells that are allowed to be removed in post-mask mode.

Note: This attribute is applicable in the post-mask mode only.

Related Commands

- `opt_signoff`

opt_signoff_post_mask_logic_cells

`opt_signoff_post_mask_logic_cells list_of_regular_cells`

Default: ""

Data_type: String

Lists all the programmable regular logic/ECO cells that are allowed to be inserted in place of filler/decap cells during optimization in post-mask mode.

Note: This attribute is applicable in the post-mask mode only.

Example:

The following set of commands allow Hold timing closure in post-mask mode using programmable filler cells:

```
set_db opt_signoff_post_mask true
set_db opt_signoff_post_mask_filler_cells {ln10_ecofiller40x ... ln10_ecofiller4x}
set_db opt_signoff_post_mask_logic_cells {ln10_ecofillsgcap40x ... ln10_ecofillsgcap4x}
set_db opt_signoff_post_mask_decap_cells {ln10_ecodlybt01 ln10_ecobuft04 ...
ln10_ecoand02 ... ln10_ecoinv04 ...}
opt_signoff -hold
```

Related Commands

- `opt_signoff`

opt_signoff_touch_partition_list

`opt_signoff_touch_partition_list` *string*

Default = " "

Data_type: String

Selects a list of partition module names to be considered as touch only during optimizations. Any partition that is not specified using this parameter will be considered as don't_touch during optimization.

Notes:

- This parameter is exclusive to `opt_signoff_dont_touch_partition_list`.
- This parameter must be applied before `opt_signoff` and can be changed between each call of `opt_signoff`.

opt_signoff_upsize_in_power_opt

`opt_signoff_upsize_in_power_opt` {true | false}

Default: false

Data_type: Boolean

Allows the power optimization engine to perform cell upsizing in order to recover more power.

Note :

- This attribute is only honored by `opt_signoff -power`.
- If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_verbose

`opt_signoff_verbose` {true | false}

Default: false

Data_type: Boolean

Specifies whether the command should output log file reporting with verbosity or not.

Related Commands

- `opt_signoff`

opt_signoff_write_eco_opt_db

```
opt_signoff_write_eco_opt_db dir_name
```

Default: ecoTimingDB

Data_type: String

Specifies a path of directory where generated database files are kept.

Related Commands

- `opt_signoff`

partition_floorplan

The root attributes in the `partition_floorplan` category are:

- `partition_floorplan_export`
- `partition_floorplan_import`

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `partition_floorplan` category, use the following command:

```
get_db -category partition_floorplan
```


partition_floorplan_export

`partition_floorplan_export {macro | special_route | pin_constraint}`

Default: `special_route`

Note: default data + special route is always saved

Data_type: `enum* {macro special_route pin_constraint}`, read/write

Specifies the objects to be exported. This is used by the `write_partitions - floorplan_only` command to save the data to the fp file.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `write_partitions`
- `assemble_design`

partition_floorplan_import

`partition_floorplan_import {macro | special_route | pin_constraint}`

Default:

By default, it brings only defaults objects

Data_type: `enum* {macro special_route pin_constraint}`, read/write

Specifies the objects to be imported. This is used by the `assemble_design - floorplan_only` command to load the data from the fp file.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

Related Commands

- `write_partitions`
- `assemble_design`

place

The following types of place category attributes are covered in this chapter:

- [place_3d_use_existing_bump_tsv](#)
- [place_cell_edge_spacing](#)
- [place_check_report_fixed_inst_on_blockage](#)
- [place_design_enable_3d](#)
- [place_design_integrity_ir_fix_effort](#)
- [place_design_refine_macro](#)
- [place_detail_allow_border_pin_abut](#)
- [place_detail_allow_single_height_row_symmetry_x](#)
- [place_detail_check_route](#)
- [place_detail_color_aware_legal](#)
- [place_detail_context_aware_legal](#)
- [place_detail_honor_inst_pad](#)
- [place_detail_ir_aware_max_drive_strength](#)
- [place_detail_pad_fixed_insts](#)
- [place_detail_swap_eeq_cells](#)
- [place_detail_m3_stripe_push_down](#)
- [place_detail_m3_stripe_shrink](#)
- [place_detail_use_diffusion_transition_fill](#)
- [place_detail_use_gate_array_filler_groups](#)
- [place_detail_pgftv_insertion_cell_list](#)
- [place_global_adaptive_subd1](#)
- [place_global_ignore_low_effort_path_groups](#)

To set the value of any root attribute, use the [set_db](#) command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `place` category, use the following command:

```
get_db -category place
```

place_3d_use_existing_bump_tsv

```
place_3d_use_existing_bump_tsv {true|false}
```

Default: false

Data_type: string, read-only

Enables 3D placement to use preplaced but unassigned bump/TSV in the design for signal net assignment. The preplaced bump/TSV must be the specified cells for 3D concurrent bump/TSV insertion and legalization.

Note: When you set this option to `true`, you need to ensure that enough bump/TSV is preplaced and the corresponding bump grid is created in each die before invoking the 3D placement.

place_cell_edge_spacing

```
place_cell_edge_spacing {cell_edge_type1 cell_edge_type2 spacing}
```

Default: ""

Data_type: string

Specifies the list of cell edge-type spacing values from LEF `CELLEDGESPACINGTABLE` or OA techfile that affects the placer. It specifies the min-spacing between different edge-types of std-cells, in a list format: {cell_edge_type1 cell_edge_type2 spacing}. The edge-type names match the base_cell's `right_edge_type` and `left_edge_type` attribute values.

place_check_report_fixed_inst_on_blockage

```
place_check_report_fixed_inst_on_blockage {true | false}
```

Default: false

Data_type: bool, read/write

When `true`, `check_place` reports placement blockage violations for fixed instances under

placement blockages.

place_design_enable_3d

```
place_design_enable_3d {true | false}
```

Default: false

Data_type: bool, read/write

Enables 3D placement in Integrity 3D-IC platform. You need to set this to true before invoking concurrent macro placement (`place_design -concurrent_macros`), standard cell placement (`place_opt_design -place`), and bump/TSV insertion (`place_bump_tsv_detail`) in a 3D design.

place_design_integrity_ir_fix_effort

```
place_design_integrity_ir_fix_effort {low|medium|high}
```

Default: medium

Data_type: enum, read/write

Sets the power integrity placement IR fixing effort.

place_design_refine_macro

```
place_design_refine_macro {true | false}
```

Default: false

Data_type: bool, read/write

Specifies whether to run `place_macro_detail` after `place_design -concurrent_macros`. When `place_design -concurrent_macros` is run, it sets the value of `place_opt_run_global_place` attribute to seed.

place_detail_allow_border_pin_abut

```
place_detail_allow_border_pin_abut {true | false}
```

Default: false

Data_type: bool

Some libraries have cells with pin geometry that is not set back by half-routing pitch from the normal PR-boundary of that cell. In cases where the pin geometries touch the cell PR_boundary, placement of an instance of this cell can cause a DRV violation with a normal pin geometry of the other cells. To avoid these DRVs, placer will add one Site padding to the cell-side during pre-processing, thus satisfying the min-spacing rules.

The `place_detail_allow_border_pin_abut` attribute disables this pre-emptive DRV avoidance behavior, and no auto-padding is done when the option is set to `true`.

place_detail_allow_single_height_row_symmetry_x

```
place_detail_allow_single_height_row_symmetry_x siteName
```

Default: ""

Data_type: bool, read/write

Specifies a single height row tech-site, which allows symmetry X during detail place.

place_detail_check_route

```
place_detail_check_route {true | false}
```

Default: false

Data_type: bool

Considers pre-routed FIXED signal wires and avoids creating DRC violations between FIXED signal wires and instance pins. When this attribute is not specified, `place_detail` ignores FIXED signal wires.

Note: Using this attribute might result in longer run time.

If you specify `place_detail -io_pin_blockage`, you do not need to specify `-check_route` for signal pins because `place_detail` will treat the signal pins similar to supply pins for blockages. However, signal wires still need this attribute.

place_detail_color_aware_legal

```
place_detail_color_aware_legal {true | false}
```

Default: false

Data_type: bool

Enables placer to honor and fix double pattern constraints between adjacent cells.

Note: This attribute does not affect cell to pre-route color DRC checks.

place_detail_context_aware_legal

`place_detail_context_aware_legal {none|all|optional|required|user|ignore_soft}`

Default: `all`

Data_type: `enum* {none all optional required user ignore_soft}`

Instructs placer to support cell edge to cell edge spacing constraints as defined in the LEF files.

The `optional`, `required`, `user` argument values can be used separately or together, but they are exclusive to `all` and `none`.

place_detail_honor_inst_pad

`place_detail_honor_inst_pad {true | false}`

Default: `false`

Data_type: `bool`

Honors padding from `set_inst_padding` in detail placement.

place_detail_ir_aware_max_drive_strength

`place_detail_ir_aware_max_drive_strength maxDriveStrength`

Default: `0.0`

Data_type: `double, read/write`

Specifies the current density value for the IR critical instances above which the instances gets padded during `place_detail`. This means that the IR critical instances with the current density value more than the specified value gets padded.

Note: There may be larger cell movement with this option, which can degrade the timing. So, it is preferred to be used at the pre-CTS stage in the flow. The current density values for the IR-critical instances are dumped in the Voltus Reports.

place_detail_pad_fixed_insts

```
place_detail_pad_fixed_insts {true | false}
```

Default: false

Data_type: bool

Specifies whether the `place_detail` command honors cell padding on `FIXED` instances.

place_detail_swap_eeq_cells

```
place_detail_swap_eeq_cells {true | false}
```

Default: false

Data_type: bool

Determines whether master cells can be replaced by EEQ cells during detailed placement to improve routability. After detailed placement, the software reports the number of replacements. EEQ cells are defined in the LEF file.

place_detail_m3_stripe_push_down

```
place_detail_m3_stripe_shrink site_units
```

Default: -1

Data_type: int

Specifies the width threshold in site units for virtual push down of M3 stripe to M2 for M1 pin-access.

If the default value is set to `-1`, it indicates that no M3 stripe will be pushed down. If the default value is set to `0`, it indicates that all M3 stripes will be pushed down. If the default value is larger than `0`, then all M3 stripes having width greater than the specified value will be pushed down.

place_detail_m3_stripe_shrink

```
place_detail_m3_stripe_shrink site_units
```

Default: 0

Data_type: int

Specifies the shrink effort of the virtual M2 stripe by given total site units. This global should work

with place_detail_m3_stripe_push_down.

If the default value is set to 0, it indicates that the entire pushed down M3 stripe is used to evaluate M1 pin access. If the default value is set to more than 0, it indicates that M3 stripe will shrink by the specified value before it is pushed down to M2. The left side will shrink first. For example, if the specified value is set to 1, the M3 stripe will shrink by 1 site on left, and if the specified value is set to 2, 1 site on left will shrink, along with 1 site on right.

place_detail_use_diffusion_transition_fill

```
place_detail_use_diffusion_transition_fill {true | false}
```

Default: false

Data_type: bool

Specifies that the design has fillers with jogged diffusion obs. Use this attribute for designs where fillers with jogged diffusion obs are to be inserted between cells with varying diffusion widths.

place_detail_use_gate_array_filler_groups

```
place_detail_use_gate_array_filler_groups {true | false}
```

Default: false

Data_type: bool

Specifies that the filler cell instances will only be replaced by logic instances that belong to same GA groups.

place_detail_pgftv_insertion_cell_list

```
place_detail_pgftv_insertion_cell_list listOfCells
```

Default: ""

Data_type: string, read/write

Specifies the list of cells as PG FTV cells.

place_global_adaptive_subd1

```
place_global_adaptive_subd1 {true|false}
```

Default: `false`

Data_type: bool, read/write

When set to true, placement will call Opt API to get a list of cells not usable for subD1 and disable the usage of cells in the placement engine.

`place_global_adaptive_subd1` can be used independently to get the list of subD1 cells.

place_global_ignore_low_effort_path_groups

`place_global_ignore_low_effort_path_groups {true | false}`

Default: `false`

Data_type: bool, read/write

When `place_global_enable_distributed_place` is set to `true`, for low effort path groups, the tool ignores it and does not work on the timing.

power

The root attributes in the `power` category are:

- `power_add_simulation`
- `power_adjust_input_activity_in_iterations`
- `power_adjust_macro_activity_in_iterations`
- `power_keep_clock_gate_ratio_in_iterations`
- `power_analysis_temperature`
- `power_annotation_detail_report`
- `power_auto_twf_delay_annotation`
- `power_average_rise_fall_cap`
- `power_block_independent_peakpower`
- `power_bulk_pins`
- `power_capacity`

- [power_case_insensitive_mapping](#)
- [power_clock_source_as_clock](#)
- [power_comprehensive_automapping](#)
- [power_compress_reports](#)
- [power_constant_override](#)
- [power_corner](#)
- [power_create_driver_db](#)
- [power_current_generation_method](#)
- [power_db_name](#)
- [power_decap_cell_list](#)
- [power_default_frequency](#)
- [power_default_slew](#)
- [power_default_supply_voltage](#)
- [power_detailed_view_current_only](#)
- [power_disable_clock_gate_clipping](#)
- [power_disable_leakage_scaling](#)
- [power_disable_static](#)
- [power_distribute_switching_power](#)
- [power_distributed_combine_report_format](#)
- [power_distributed_setup](#)
- [power_domain_based_clipping](#)
- [power_dynamic_glitch_filter](#)
- [power_enable_auto_queue](#)
- [power_enable_auto_mapping](#)
- [power_enable_disk_mapping](#)
- [power_enable_duty_propagation_with_global_activity](#)
- [power_enable_dynamic_current_slew_load_interpolation](#)

- `power_dynamic_scale_clock_by_frequency`
- `power_dynamic_scale_clock_by_name`
- `power_enable_interactive_reports`
- `power_enable_mt_state_propagation`
- `power_equivalent_annotation`
- `power_dynamic_vectorless_ranking_methods`
- `power_enable_dynamic_scaling`
- `power_enable_flop_state_propagation`
- `power_enable_force_propagation`
- `power_enable_generated_clock`
- `power_enable_input_net_power`
- `power_enable_multithread_reports`
- `power_enable_power_target_flow`
- `power_enable_pba_for_tempus_pi`
- `power_enable_rtl_dynamic_vector_based`
- `power_enable_scan_report`
- `power_enable_single_cycle_scheduling`
- `power_enable_slew_based_ccs_pin_cap`
- `power_enable_state_propagation`
- `power_enable_stable_clock_gating`
- `power_enable_stable_flop_scheduling`
- `power_enable_tempus_pi`
- `power_enable_xp`
- `power_enhanced_blackbox_avg`
- `power_enhanced_blackbox_max`
- `power_event_based_leakage_power`
- `power_external_load_config_file`

- [power_extraction_tech_file](#)
- [power_extractor_include](#)
- [power_fanout_limit](#)
- [power_force_library_merging](#)
- [power_from_x_transition_factor](#)
- [power_from_z_transition_factor](#)
- [power_generate_flop_ranking_data](#)
- [power_generate_activity_mapping_report](#)
- [power_generate_current_for_rail](#)
- [power_generate_static_report_from_state_propagation](#)
- [power_generate_leakage_power_map_based_on_calculated_leakage](#)
- [power_grid_libraries](#)
- [power_handle_glitch](#)
- [power_handle_transport_glitch](#)
- [power_handle_tristate](#)
- [power_hier_delimiter](#)
- [power_honor_combinational_logic_on_clock_net](#)
- [power_honor_default_switching_activity_in_scheduling](#)
- [power_honor_negative_energy](#)
- [power_honor_net_activity](#)
- [power_honor_non_mission_leakage](#)
- [power_hybrid_analysis](#)
- [power_ignore_control_signals](#)
- [power_ignore_data_phase_for_clock](#)
- [power_ignore_end_toggles_in_profile](#)
- [power_ignore_glitches_at_same_time_stamp](#)
- [power_ignore_inout_pin_cap](#)

- `power_ignore_macro_leakage_scale_for_temperature`
- `power_ignore_unconstraint_net_with_timing_data`
- `power_include_file`
- `power_include_initial_x_transitions`
- `power_include_sequential_clock_pin_power`
- `power_include_timing_in_current_file`
- `power_ir_derated_timing_view`
- `power_leakage_scale_factor_for_temperature`
- `power_lib_files`
- `power_library_preference`
- `power_macro_pgv_leakage`
- `power_macro_toggle_pin_percentage`
- `power_match_state_for_logic_x`
- `power_memory_current_scaling_method`
- `power_multibit_flop_toggle_behavior`
- `power_multibit_flop_toggle_pin_percentage`
- `power_multi_scenario_simulation`
- `power_off_pg_bulk_leakage`
- `power_merge_switched_net_currents`
- `power_method`
- `power_min_leaf_count`
- `power_off_pg_nets`
- `power_output_current_data_prefix`
- `power_output_dir`
- `power_partition_count`
- `power_partition_spf`
- `power_partition_twf`

- [power_pin_based_twf](#)
- [power_precision](#)
- [power_pre_simulation_empty_period](#)
- [power_pre_simulation_period](#)
- [power_pre_simulation_power_exclude_period](#)
- [power_quit_on_activity_coverage_threshold](#)
- [power_read_rcdb](#)
- [power_relax_arc_match](#)
- [power_report_library_usage](#)
- [power_report_missing_bulk_connectivity](#)
- [power_report_missing_input](#)
- [power_report_black_boxes](#)
- [power_report_clock_instance_switching_info](#)
- [power_report_idle_instances](#)
- [power_report_instance_switching_info](#)
- [power_report_instance_switching_list](#)
- [power_report_missing_nets](#)
- [power_report_scan_chain_statistics](#)
- [power_report_statistics](#)
- [power_report_time_display_fraction_digits](#)
- [power_report_twf_attributes](#)
- [power_reuse_flop_ranking_data](#)
- [power_reuse_flop_ranking_data_hier](#)
- [power_scale_to_sdc_clock_frequency](#)
- [power_scan_chain_activity](#)
- [power_scan_chain_name_pattern](#)
- [power_scan_control_file](#)

- `power_set_power_event_based`
- `power_settling_buffer`
- `power_show_cell_usage_stats`
- `power_smart_window`
- `power_scan_multi_bit_flop_chain_type`
- `power_split_bus_power`
- `power_start_time_alignment`
- `power_state_dependent_leakage`
- `power_stateprop_ignore_unannot_pins`
- `power_static_multi_mode_scenario_file`
- `power_static_netlist`
- `power_switching_power_on_rise_only`
- `power_thermal_input_file`
- `power_thermal_leakage_temperature_scale_table_file`
- `power_to_x_transition_factor`
- `power_to_z_transition_factor`
- `power_transition_factor_based_duty`
- `power_transition_time_method`
- `power_twf_delay_annotation`
- `power_twf_load_cap`
- `power_unified_power_switch_flow`
- `power_use_cell_leakage_power_density`
- `power_use_fastest_clock_for_dynamic_scheduling`
- `power_use_lef_for_missing_cells`
- `power_use_only_ddv_current`
- `power_worst_case_vector_activity`
- `power_worst_step_size`

- [power_worst_window_count](#)
- [power_worst_window_size](#)
- [power_worst_window_type](#)
- [power_worst_window_coverage](#)
- [power_worst_window_reports](#)
- [power_write_dynamic_currents](#)
- [power_use_physical_partition](#)
- [power_use_twf_stable_randomized_arrival_delay](#)
- [power_use_zero_delay_vector_file](#)
- [power_vector_based_multithread](#)
- [power_vector_profile_mode](#)
- [power_write_gui_db](#)
- [power_write_bbox](#)
- [power_write_db](#)
- [power_write_default_pti_files](#)
- [power_write_profiling_db](#)
- [power_write_simulation_db](#)
- [power_write_static_currents](#)
- [power_x_transition_factor](#)
- [power_x_count_transition_using_3_states](#)
- [power_z_transition_factor](#)
- [power_zero_delay_vector_toggle_shift](#)

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `power` category, use the following command:

```
get_db -category power
```

power_add_simulation

```
power_add_simulation {true | false}
```

Default: false

Data_type: Boolean

Specifies to add simulation if there are no simulation vectors or if all primary sources are not annotated by vectors. This parameter can be used to ensure complete design coverage. Event simulation is done for primary sources like primary input, flops, and macros.

This attribute is only applicable to the event-based power analysis flow.

Related Commands

- [set_power](#)

power_adjust_input_activity_in_iterations

```
power_adjust_input_activity_in_iterations {true | false}
```

Default: true

Data_type: Boolean

Specifies whether to keep the input activity fixed or automatically adjust the activity value for each iteration of state propagation. This attribute is only applicable to the power target flow (`power_enable_power_target_flow true`).

Related Commands

- [set_power](#)

power_adjust_macro_activity_in_iterations

```
power_adjust_macro_activity_in_iterations {true | false}
```

Default: true

Data_type: Boolean

Specifies whether to keep the macro activity fixed or automatically adjust the activity value for each iteration of state propagation. This attribute is only applicable to the power target flow (`power_enable_power_target_flow true`).

Related Commands

- `set_power`

power_keep_clock_gate_ratio_in_iterations

```
power_keep_clock_gate_ratio_in_iterations {true | false}
```

Default: `false`

Data_type: Boolean

Specifies whether to keep the clock gate output (CGO) fixed or automatically adjust the value for each iteration of state propagation. This attribute is only applicable to the power target flow (`power_enable_power_target_flow true`).

Related Commands

- `set_power`

power_analysis_temperature

```
power_analysis_temperature temperature
```

Default: `-274.000000`

Data_type: Double

Specifies the temperature for static power calculation.

Note: Static power calculation gets the temperature value of each instance from the following sources (in order of precedence):

- Instance temperature file
- `power_analysis_temperature` attribute
- The current operating condition temperature

Related Commands

- `report_power`

power_annotation_detail_report

`power_annotation_detail_report {true | false}`

Default: `false`

Data_type: Boolean

Specifies to generate a detailed annotation report per instance in the Vector Profiling flow (`power_method vector_profile`). When specified, it creates the `AnnotationDetail.rpt` file in the user-specified output directory.

Related Commands

- `report_power`

power_auto_twf_delay_annotation

`power_auto_twf_delay_annotation {true | false}`

Default: `false`

Data_type: Boolean

Specifies to automatically detect the zero-delay vectors and perform delay annotation. As a result, you do not need to specify additional parameters (`set_db power_use_zero_delay_vector_file` and `read_activity_file -zero_delay`) to indicate the vectors that have delay annotation. This attribute is applicable only to the [event-based power analysis flow](#).

Related Commands

- `report_power`

power_average_rise_fall_cap

`power_average_rise_fall_cap {true | false}`

Default: `false`

Data_type: Boolean

Specifies to use the average of rise and fall capacitance from the liberty file.

Related Commands

- `report_power`

power_block_independent_peakpower

```
power_block_independent_peakpower {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to identify the worst power interval of each block, and run power analysis of the full design using the worst interval of each block. This attribute allows you to perform power and current analysis for design blocks that have independent peak power intervals.

When set to `false`, power analysis is performed using the common worst interval found using simulation of all vectors together.

Related Commands

- `report_power`

power_bulk_pins

```
power_bulk_pins {bulk_pin_list}
```

Default: `""`

Data_type: String

Defines power and ground bulk LEF pins for the design.

Use this attribute when library cells have bulk pins defined in LEF but the liberty file does not contain power associated with these pins. If a liberty file does not have body bias definition, power analysis does not distribute power to the body bias domain.

Related Commands

- `report_power`

power_capacity

Default: `low`

Data_type: Enum

Enables segmented power processing. When this attribute is specified, the software performs concurrent processing of vector simulation annotation, power calculation, and report construction.

This attribute enables performance optimization with some trade-off on memory consumption and runtime. The arguments of `power_capacity` are:

- `low` - reduces memory for a flat run.
- `medium` - further reduces memory with moderate increase in runtime (1.2X).
- `high` - maximum memory reduction at about 2X runtime.

The `power_capacity` attribute is used for reducing memory when processing long simulation vectors. In case of the FSDB and SHM format activity file, the runtime can increase up to 2X.

Related Commands

- `report_power`

power_case_insensitive_mapping

`power_case_insensitive_mapping {true | false}`

Default: `false`

Data_type: Boolean

Specifies whether the instance name mapping between the RTL netlist and GATE level netlist in the mapping file is case sensitive or not. When set to `true`, the name mappings are not case sensitive.

power_clock_source_as_clock

```
power_clock_source_as_clock {true | false}
```

Default: false

Data_type: Boolean

Specifies to use the correct clock frequency for activity calculation. If you stop the arrival of the clock signal by setting `set_case_analysis` to false, the transition density at the output net of the flop is calculated using the correct clock frequency instead of the default frequency.

Related Commands

- `report_power`

power_comprehensive_automapping

```
power_comprehensive_automapping {true | false}
```

Default: false

Data_type: Boolean

Specifies to perform all-inclusive instance name mapping between the RTL netlist and GATE level netlist. When set to `true`, the software honors all possible naming rules (includes handling of characters like '[', ']', ':', '/', '\\', '_', ':') and handling of strings like "MBFF", "_reg", "_MB_" etc.) for better annotation coverage.

This feature is applicable to both file-based mapping (`read_activity_map_file -rtl_to_gate | -two_column_mapping_file`) and automatic mapping (`power_enable_auto_mapping`).

When `power_comprehensive_automapping` is specified with either `-rtl_to_gate` or `-two_column_mapping_file`, the RTL mapping entries are given priority over the automatic rule-based mapping entries and are used to annotate the design objects.

Related Commands

- `report_power`

power_compress_reports

```
power_compress_reports {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to compress the power output files in the GNU Zip format. The compressed files will have the suffix `.gz`.

The possible arguments are:

- `true`: Sets the compression option with compression level 6
- `false`: default, no compression
- 1-9: the compression levels vary from 1 to 9. 1 provides the fastest compression speed but at a lower ratio, so the file size will be larger. 9 provides the highest compression ratio but at a lower speed.

Related Commands

- `report_power`

power_constant_override

```
power_constant_override {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to give higher precedence to propagated constants defined using the `set_case_analysis` command. It allows propagated constants to override global activity.

Related Commands

- `report_power`

power_corner

```
power_corner {min|max}
```

Default: `{}`

Data_type: String

Defines the library corner (for non-MMMC setup).

- `min` corner means power calculator will use min timing libraries for the power calculation.
- `max` corner means power calculator will use max timing libraries for the power calculation.

Related Commands

- `report_power`

power_create_driver_db

```
power_create_driver_db {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to determine all the drivers/receivers on the ground net.

Related Commands

- `report_power`

power_current_generation_method

```
power_current_generation_method {avg | peak}
```

Default: `avg`

Data_type: String

Specifies the current waveform generation method during dynamic power analysis. The two possible arguments of this attribute are:

- `avg` - Specifies to use the average current value of each time step. The output current filename is `filename.ptiavg`. The default value is `avg`.
- `peak` - Specifies to use the peak current value of each time step. The output current filename is `filename.ptipeak`. It is recommended to use the `peak` method when using the Composite Current Source Power (CCSP) Liberty files.

Related Commands

- `report_power`

power_db_name

`power_db_name filename`

Default: ""

Data_type: String

Specifies the name of the binary power database file (`filename.db`).

You can specify this attribute in scenarios where more than one db is generated and you need to track which one was generated in which mode when the restoring the db, or if your design has several blocks and analysis is done on one block all the time and all the databases are named `power.db`.

Related Commands

- `report_power`

power_decap_cell_list

`power_decap_cell_list cell_list`

Default: ""

Data_type: String

Specifies the physical only cells, such as decap cells, to include in the power report.

Note: This is only supported for Static Power Analysis.

Related Commands

- `report_power`

power_default_frequency

`power_default_frequency value`

Default: -1.0

Data_type: Double

Specifies to set the default frequency in the MHz unit for net not annotated by TWF. You can specify a numerical value without unit.

Related Commands

- `report_power`

power_default_slew

`power_default_slew value`

Default: " "

Data_type: String

Specifies to set the default slew in the ps unit for net not annotated by TWF. You can specify a numerical value without unit.

Related Commands

- `report_power`

power_default_supply_voltage

`power_default_supply_voltage value`

Default: " "

Data_type: String

Specifies to set a default voltage for power nets in a scenario where the power engine cannot determine the voltage of these nets. The default value of this attribute is 1.0.

Related Commands

- `report_power`

power_detailed_view_current_only

```
power_detailed_view_current_only {true | false}
```

Default: true

Data_type: Boolean

Controls instance power reporting for cells that have power information available in PGV. When set to `true`, specifies to skip writing of the switching and leakage current and use only the internal current from DDV PGV. When set to `false`, specifies to report the switching and leakage current values in addition to the internal current from DDV PGV. If a cell is defined in both PGV and dotlib, leakage is computed from dotlib. It is recommended to set this attribute to `true` because that would prevent over-estimation of the instance current.

Related Commands

- `report_power`

power_disable_clock_gate_clipping

```
power_disable_clock_gate_clipping {true | false}
```

Default: true

Data_type: Boolean

Specifies to enable clock gate output clipping so that the output transition density of an ICG cell does not exceed the input clock pin transition density.

Related Commands

- `report_power`

power_disable_leakage_scaling

```
power_disable_leakage_scaling {true | false}
```

Default: false

Data_type: Boolean

Specifies to exclude leakage power during scaling factor computation when target power is

specified using the `set_power` command. When this option is set to `true`, the leakage power does not get scaled, and only the internal and switching components are scaled to meet the user-specified total power.

This attribute is relevant only for the `set_power` commands for which total power is specified, and has no effect for the `set_power` command specified with the `-internal`, `-switching`, or `-leakage` parameters.

Related Commands

- `report_power`

power_disable_static

```
power_disable_static {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true` in the dynamic vector-based or dynamic vectorless flows, the command performs only dynamic analysis and turns-off static power calculation.

Related Commands

- `report_power`

power_distribute_switching_power

```
power_distribute_switching_power {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to equally distribute the switching power among all drivers if a net has more than one driver. When set to `false`, stores all switching power on one of the net drivers.

Related Commands

- `report_power`

power_distributed_combine_report_format

power_distributed_combine_report_format {none | detailed | reduced}

Default: reduced

Data_type: String

Specifies to generate consolidated XP power reports. This parameter allows you to combine power reports that are stored across multiple partition directories to give a single unified view of power data. The possible arguments of this parameter are:

- none - generates distributed reports.
- reduced - generates consolidated reports for the following:
 - power.missingdata.* (static and dynamic flows)
 - power.stateprop.* (dynamic flow)
 - power.stats.setpower (dynamic flow)
- detailed - generates consolidated reports for the following:
 - power.rpt (static and dynamic flows)
 - power.instpwr.rpt (static flow)
 - power.lib.rpt (static flow)
 - power.pgpinpwr.rpt (static flow)
 - power.pgnet.detailed.rpt
 - power.missingdata.* (static and dynamic flows)
 - power.pgnet.detailed.rpt (static and dynamic flows)
 - power.stateprop.* (dynamic flow)
 - power.stats.setpower (dynamic flow)
 - pg_trigger.txt (dynamic flow)
 - trigger.txt (dynamic flow)
 - *.totalcurrent (dynamic flow)
 - voltus_power.cellstat (static and dynamic flows)
 - voltus_power.ptifiles (static and dynamic flows)

Related Commands

- `report_power`

power_distributed_setup

`power_distributed_setup file`

Default: ""

Data_type: String

Specifies a file containing the customized setup details for running power analysis in the distributed mode.

The specified file has 4 columns:

```
INST/CELL <name> hier/block <power.inc>
```

where,

- **INST/CELL:** Keyword indicating cell or instance to be run in the distributed mode. Cell can be useful for repeated DEF blocks.
- **name :** Name of the hierarchical block or instance.
- **hier/block:** Keyword indicating hierarchy or block. `hier` implies that the full hierarchy will be preserved by including the DEFs from top to the block, and `block` implies that only the lower level DEF will be included
- **power.inc :** name of the power include file. This column is optional.

The unspecified instance/cell will be run with the top level as a single group. Each distributed block/instance should have corresponding TWF and SPEF to improve runtime.

Note: Each hierarchy or block will be run independently from other hierarchies/blocks, thus the external load or cell seen at the pin of the hierarchy/block can be different from flat analysis. If the partition boundary analysis is important, distributed processing is not recommended.

Related Commands

- `report_power`

power_domain_based_clipping

```
power_domain_based_clipping {true | false}
```

Default: false

Data_type: Boolean

Determines the frequency to be used to clip propagated activity that is too high. When set to `false`, the frequency of the fastest clock in the design is used. If you set this attribute to `true`, the fastest clock domain on the net is used to clip propagated activity.

Related Commands

- `report_power`

power_dynamic_glitch_filter

```
power_dynamic_glitch_filter value
```

Default: -1.0

Data_type: Double

Sets the absolute threshold time (in ns) for filtering glitches in net toggles. When you specify a time value for this parameter, glitches in net toggles that are equal to or smaller in width than the specified time value are removed.

Related Commands

- `report_power`

power_enable_auto_queue

```
power_enable_auto_queue {true | false}
```

Default: false

Data_type: Boolean

Specifies to automatically determine the queue for power jobs. This attribute is used in the Voltus-XP distributed power flow with hierarchical DEFs. In the XP flow, partitions are divided into multiple queues depending on the number of CPUs (For example, Default queue: 1 CPU, Small queue: 2

CPUs, Medium queue: 4 CPUs, Large queue: 8 CPUs, and so on). By default, all power partitions run on the same queue. When the attribute is set to `true`, the blocks can run on different queues depending on the size (number of instances) of the blocks. The `power_enable_auto_queue` attribute will be beneficial when there are uneven power block partitions. For example, if there are some blocks with 20-30M instances and there are some with 3-5M instances. Using this attribute, bigger blocks will be automatically assigned to a bigger queue, resulting in better runtime.

Related Commands

- `report_power`

power_enable_auto_mapping

```
power_enable_auto_mapping {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to automatically perform instance name mapping between the RTL netlist and GATE level netlist. This attribute allows you to map the RTL vector to the Gate level netlist without a mapping file.

You can use the `read_activity_file -name_mapping_rule <filename>` command parameter to specify a file containing customized or design-specific rules.

Related Commands

- `report_power`

power_enable_disk_mapping

```
power_enable_disk_mapping {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to enable disk caching of the toggles. Therefore, the toggles are stored in the disk instead of keeping them in memory. This has made large vectors or long simulation duration runs possible even with lesser available memory. This attribute is only applicable to the event-based power analysis flow (`power_method event_based`) in the XP mode.

Example:

The following command specifies to enable disk caching of toggles:

```
set_db power_method event_based power_enable_disk_mapping true power_enable_xp true
```

Related Commands

- `report_power`

power_enable_duty_propagation_with_global_activity

```
power_enable_duty_propagation_with_global_activity {true | false}
```

Default: false

Data_type: Boolean

Specifies to propagate duty cycle with the global switching activity setting for all data nets in the dynamic vectorless analysis flow.

Related Commands

- `report_power`

power_enable_dynamic_current_slew_load_interpolation

```
power_enable_dynamic_current_slew_load_interpolation {true | false}
```

Default: false

Data_type: Boolean

Specifies to use the charge-based slew load interpolation. When this attribute is set to `true`, Power Analysis takes dynamic currents from the 4 nearest slew/load values and performs either a 4-way (lower slew, lower load, upper slew, upper load) or a 2-way (lower slew, upper slew OR lower load, upper load) interpolation to calculate the dynamic current waveform. The attribute allows you to derive a more realistic dynamic current waveform based on the exact slew load values from the .lib dynamic current table.

Related Commands

- `report_power`

power_dynamic_scale_clock_by_frequency

```
power_dynamic_scale_clock_by_frequency { freq_Afactor_Afreq_Bfactor_Bfreq_Cfactor_C... }
```

Default: ""

Data_type: String

Specifies the scale factors for the clock frequency values. This parameter allows you to perform frequency scaling for clocks running at relatively low frequencies, thereby ensuring higher toggle coverage for limited simulation period. The parameter is applicable only to the dynamic vectorless flow. The following units are supported for frequency: Hz, KHz, MHz, and GHz.

Related Commands

- `report_power`

power_dynamic_scale_clock_by_name

```
power_dynamic_scale_clock_by_name { clk_A factor_A clk_B factor_B clk_C factor_C ... }
```

Default: ""

Data_type: String

Specifies the scale factors for the low frequency clocks. This parameter allows you to perform frequency scaling for clocks running at relatively low frequencies, thereby ensuring higher toggle coverage for limited simulation period. The parameter is applicable only to the dynamic vectorless flow. The following units are supported for frequency: Hz, KHz, MHz, and GHz.

Related Commands

- `report_power`

power_enable_interactive_reports

```
power_enable_interactive_reports {true | false}
```

Default: true

Data_type: Boolean

Specifies to generate multiple dynamic reports based on the power analysis run done for a netlist. When this attribute is set to `true`, you can generate various types of power reports interactively

using the `report_power` command without the need to perform power calculation on the same netlist again. The `power_enable_interactive_reports` attribute is useful when you need to generate reports for the unaltered netlist within the same power analysis session.

This attribute is applicable only to the event-based power analysis flow with area-based partitioning (`power_use_physical_partition`) in the XP mode.

Example:

The following command specifies to enable interactive generation of power reports:

```
set_db power_method event_based power_use_physical_partition true power_partition_count  
2 power_enable_interactive_reports true power_enable_xp true
```

Related Commands

- `report_power`

power_enable_mt_state_propagation

```
power_enable_mt_state_propagation {true | false}
```

Default: `true`

Data_type: Boolean

Specifies to enable multi-threading in state propagation to help improve the runtime for large simulations. For example, when the attribute was specified for a design with 300 million instances and simulation duration of 120 microseconds, the runtime has improved from 21 hours to 1.5 hours.

This attribute is only applicable to the event-based power analysis flow (`power_method event_based`).

Related Commands

- `report_power`

power_equivalent_annotation

```
power_equivalent_annotation {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to enable equivalent annotation for Q or Qbar pins when only one of them is annotated.

Related Commands

- `set_power`

power_dynamic_vectorless_ranking_methods

```
power_dynamic_vectorless_ranking_methods {load | clock | vector_activity}
```

Default: ""

Data_type: String

Specifies to change the ranking method from the default to the following:

- `load` - Prioritizes scheduling cells with higher fanout
- `clock` - Prioritizes scheduling cells associated with faster clocks
- `vector_activity` - Prioritizes scheduling cells with higher activity

Related Commands

- `report_power`

power_enable_dynamic_scaling

```
power_enable_dynamic_scaling {true | false}
```

Default: false

Data_type: String

Enables `set_power` scaling for dynamic power analysis. The power and scaling specifications are used to scale dynamic current on both the power and ground currents of the specified instance/cell.

This attribute honors the user-defined average (static) power for leaf-level instances and hierarchies (instances/cells/modules) along with power net specification in the dynamic current scaling flows such that the average of the dynamic current is equal to the assigned average power. When you specify this attribute, the average of the dynamic current of these leaf-level instances will match the scaled static power numbers.

This attribute is not supported in the state-propagation-based vectorless flow (`power_enable_state_propagation true`). It is applicable only to the probability-based vectorless flow.

Related Commands

- `set_power`
- `report_power`

power_enable_flop_state_propagation

```
power_enable_flop_state_propagation {true | false}
```

Default: `false`

Data_type: Boolean

Enables accurate clock state propagation for sequential cells. When this attribute is set to `true`, flops propagate in a manner that toggles on the output pins are in sync with the clock. This attribute helps to align the output pins' toggles with the clock toggles.

Related Commands

- `set_power`
- `report_power`

power_enable_force_propagation

```
power_enable_force_propagation {true | false}
```

Default: `false`

Data_type: bool

Enables force propagation to ensure that the output toggles are according to the clock signal rather than the input pin states. This attribute assigns clock activity to outputs when input propagation is not feasible, thereby enhancing overall activity.

Related Commands

- `set_power`

power_enable_generated_clock

```
power_enable_generated_clock {true | false}
```

Default: `true`

Data_type: Boolean

Specifies to get generated clock frequency during activity propagation for static power analysis.

Related Commands

- `set_power`

power_enable_input_net_power

```
power_enable_input_net_power {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to calculate the switching power of input nets.

Related Commands

- `set_power`

power_enable_multithread_reports

```
power_enable_multithread_reports {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to enable multi-threading for generation of power calculation reports. When set to `true`, it improves the memory and runtime for power reporting in large designs with more than 100 million instances.

Related Commands

- `set_power`

power_enable_power_target_flow

```
power_enable_power_target_flow {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to enable the power target flow. When set to `true`, the tool allows you to specify power targets for full-chip power analysis. You can use the `set_power` command to specify the power targets for full-chip, physical block, logical block, and power/ground net. When these targets are specified, Voltus will automatically scale the toggle rates of clock gate ratio, input, macro, and sequential activities to meet the specified power targets.

Related Commands

- `set_power`

power_enable_pba_for_tempus_pi

```
power_enable_pba_for_tempus_pi {true | false}
```

Default: `true`

Data_type: Boolean

Specifies to enable the Path-Based Analysis (PBA) for slack calculation in the Tempus Power Integrity (Tempus PI) flow. By default, the Graph-Based Analysis (GBA) is used. This attribute is only applicable to the state-propagation-based vectorless flow (`set_db power_method dynamic_vectorless power_enable_state_propagation true`).

Related Commands

- `set_power`

power_enable_rtl_dynamic_vector_based

```
power_enable_rtl_dynamic_vector_based {true | false}
```

Default: false

Data_type: Boolean

Specifies to take an RTL or partial VCD/FSDb file as input and use that for dynamic vector-based flow. This attribute allows you to estimate current even for instances which are missing from the activity file.

You must use this attribute in conjunction with the `power_method dynamic_vectorbased` attribute.

Related Commands

- `set_power`

power_enable_scan_report

```
power_enable_scan_report {true | false}
```

Default: false

Data_type: Boolean

Specifies to generate all the scan mode analysis reports. The following reports will be generated:

- `voltus_power.scanff.duplicated.rpt` - flip-flops that are specified in multiple chains
- `voltus_power.scanff.missing.rpt` - flip-flops that are missing in the scan control file but exist in the design
- `voltus_power.scanff.nonassigned.rpt` - flip-flops that do not exist in the design and are dropped from the scan control file
- `voltus_power.scanff.nonscan.rpt` - assigned flip-flops that have no scan-out pin
- `voltus_power.scanff.nonswitched.rpt` - non-switching flip-flops in the scan control file
- `voltus_power.scanff.rpt` - detailed scan chain report containing global statistics for all scan chains and statistics for each scan chain in the design
- `voltus_power.scanff.stateconflict.rpt` - flip-flops that have conflicting state assignment

Related Commands

- `set_power`

power_enable_single_cycle_scheduling

```
power_enable_single_cycle_scheduling {true | false}
```

Default: `false`

Data_type: Boolean

Schedules macro instances in every dominant clock cycle in the dynamic vectorless flow. This attribute can be used achieve faster toggle coverage. After the scheduling of every instance and attaining the maximum toggle coverage, the scheduler cycles back to the same switch source element and then schedules it again with the opposite polarity. This attribute helps in achieving the same toggle coverage within half the simulation period compared to dual cycle scheduling.

When the attribute is set to `false`, the macros are scheduled every two cycles wherein the adjacent cycle is used for reversing the transition polarity.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `set_power`

power_enable_slew_based_ccs_pin_cap

```
power_enable_slew_based_ccs_pin_cap {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to use the Composite Current Source (CCS) capacitance value based on the slew given in the input pin section of the library cell in the Liberty file. If it is set to `true`, the tool honors the CCS pin capacitance; if it is set to `false`, it takes `max_capacitance` from `.lib`. This attribute can be used

when timing windows and/or a Tempus timing session is not available.

Related Commands

- `set_power`

power_enable_state_propagation

```
power_enable_state_propagation {true | false}
```

Default: false

Data_type: Boolean

Specifies to enable the vectorless state-propagation-based dynamic analysis flow.

For more information, refer to the *State-Propagation-Based Vectorless Methodology* section in the "*Dynamic Power and IRDrop Analysis*" chapter of the *Voltus User Guide*.

Related Commands

- `set_power`

power_enable_stable_clock_gating

```
power_enable_stable_clock_gating {true | false}
```

Default: false

Data_type: Boolean

Perform balanced scheduling of ICGs independent of the predecessor ICG state.

This attribute is used for the state-propagation-based vectorless flow (`set_db power_method dynamic_vectorless power_enable_state_propagation true`) only.

Related Commands

- `set_power`

power_enable_stable_flop_scheduling

```
power_enable_stable_flop_scheduling {true | false}
```

Default: false

Data_type: Boolean

Perform balanced scheduling of flops independent of the ECOs done on the design.

This attribute is used for the state-propagation-based vectorless flow (`set_db power_method dynamic_vectorless power_enable_state_propagation true`) only.

Related Commands

- `set_power`

power_enable_tempus_pi

```
power_enable_tempus_pi {true | false}
```

Default: false

Data_type: Boolean

Specifies to enable the Tempus PI analysis flow. When this attribute is set to `true`, data path scheduling is timing-aware, that is, scheduling is based on timing slacks. This attribute is only applicable to the state-propagation-based vectorless flow (`set_db power_method dynamic_vectorless power_enable_state_propagation true`).

For more information, refer to the **Tempus Power Integrity Analysis** chapter in the *Tempus User Guide*.

Related Commands

- `set_power`

power_enable_xp

```
power_enable_xp {true | false}
```

Default: false

Data_type: Boolean

Specifies to run standalone report generation in the XP mode.

Related Commands

- `set_power`

power_enhanced_blackbox_avg

```
power_enhanced_blackbox_avg {true | false}
```

Default: false

Data_type: Boolean

Specifies to use the average toggle rate of related inputs for enhanced blackbox propagation.

Related Commands

- `set_power`

power_enhanced_blackbox_max

```
power_enhanced_blackbox_max {true | false}
```

Default: false

Data_type: Boolean

Specifies to use the maximum toggle rate of related inputs for enhanced blackbox propagation.

Related Commands

- `set_power`

power_event_based_leakage_power

```
power_event_based_leakage_power {true | false}
```

Default: false

Data_type: Boolean

Specifies to calculate state-dependent leakage power. When this attribute is set to `true`, it calculates leakage power and current based on the state of an instance at each time point.

For example, at time 0ns, instance instA is in state !I ZN (0 on input, 1 on output), and at time 71ns, the instance is in state I !ZN (1 on input, 0 on output). The leakage power is calculated based on the different states of instA at different time points.

Related Commands

- `set_power`

power_external_load_config_file

```
power_external_load_config_file filename
```

Default: " "

Data_type: String

Specifies to honor the user-specified load capacitance values for power calculation. This attribute allows you to specify an external load capacitance file when you have a timing window file without the `EXTERNAL_LOAD` constructs.

File Format:

The format of the user-specified input file is:

```
INST|CELL      blockName      external_load_file
```

Here, the format of `external_load_file` is:

```
NET|PORT      name      load_value
```

Examples:

Configuration file:

```
INST          TOP          top_wcard.load
```

External load file (`top_wcard.load`):

```
UNIT pF
NET  result      0.02
NET  overflow    0.02
```

Related Commands

- `set_power`

power_extraction_tech_file

`power_extraction_tech_file filename`

Default: " "

Data_type: String

Specifies the extraction technology file to be used for top-level power-grid extraction.

Related Commands

- `set_power`

power_extractor_include

`power_extractor_include filename`

Default: " "

Data_type: String

Specifies the extractor (ZX) commands and variables in the form of an include file. This parameter is applicable only for the standalone report generation flow in the XP mode (`power_enable_xp true`).

Related Commands

- `set_power`

power_fanout_limit

`power_fanout_limit value`

Default: -1

Data_type: Integer

Specifies the fanout limit for nets. If an instance has nets more than or equal to the fanout limit, its

switching/internal/leakage power is taken to be zero.

Related Commands

- `set_power`

power_force_library_merging

```
power_force_library_merging {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to force merge PGVs even if there are some minor conflicts in the merging libraries. This parameter is applicable only for the standalone report generation flow in the XP mode (`power_enable_xp true`).

Related Commands

- `set_power`

power_from_x_transition_factor

```
power_from_x_transition_factor value
```

Default: `0.5`

Data_type: Double

Specifies how transitions from X to 0/1 are counted.

Related Commands

- `set_power`

power_from_z_transition_factor

```
power_from_z_transition_factor value
```

Default: `0.25`

Data_type: Double

Specifies how transitions from Z to 0/1 are counted.

Related Commands

- `set_power`

power_generate_flop_ranking_data

```
power_generate_flop_ranking_data directoryname
```

Default: ""

Data_type: String

Specifies the name of the directory where the flop ranking data is saved.

Related Commands

- `set_power`

power_generate_activity_mapping_report

```
power_generate_activity_mapping_report {true | false}
```

Default: false

Data_type: Boolean

Specifies to generate the `activity_mapping.rpt` report in the vector-based dynamic power flow. This report gives the list of signal name mappings between the Golden (RTL netlist - input name in the vector file) and Revised (Gate netlist - annotated design object name) netlists that were used in the flow. The report displays the list of annotated and unannotated objects and helps to debug the mapping file for unannotated design objects.

`activity_mapping.rpt` supports activity mapping done through the following methods:

- User-specified mapping file (`read_activity_map_file -two_column_mapping_file`)
- Conformal-generated RTL mapping file (`read_activity_map_file -rtl_to_gate`)
- Voltus auto-generated mapping file (`power_enable_auto_mapping`).

Related Commands

- `set_power`

power_generate_current_for_rail

```
power_generate_current_for_rail railnames
```

Default: ""

Data_type: String

Generates current files for the specified rail.

By default, the command generates current files for all power rails.

Related Commands

- `set_power`

power_generate_static_report_from_state_propagation

```
power_generate_static_report_from_state_propagation {true | false}
```

Default: false

Data_type: Boolean

Specifies to directly generate the static power analysis result based on the RTL-VCD input and state-propagation. When this attribute is specified, the RTL vector power analysis flow does not require the generation of a TCF/FSDB file. This attribute is applicable only to the RTL vector-based dynamic power analysis flow.

Related Commands

- `set_power`

power_generate_leakage_power_map_based_on_calculated_leakage

```
power_generate_leakage_power_map_based_on_calculated_leakage {true | false}
```

Default: false

Data_type: Boolean

Specifies to scale the instance leakage power by directly multiplying the user-specified scaling factor with the final calculated leakage power value, instead of scaling the leakage power value from the input dotlib.

If this parameter is not specified, the user-specified scaling factor will be applied on the leakage power values from the input dotlib.

Related Commands

- `set_power`

power_grid_libraries

```
power_grid_libraries library_list
```

Default: ""

Data_type: String

Specifies the name of the Cadence power cell libraries (.cl). You can also specify a list of defined power-grid libraries having decap cells tagged in them.

Note: When specifying a list of PGVs, you must specify the technology library as the first PGV library in the list, followed by the standard and macro PGV libraries. The selection between the standard and macro PGV library is order based. Therefore, if multiple power-grid libraries are specified for a cell, the power engine maps the cell with the first library after the technology library.

Related Commands

- `set_power`

power_handle_glitch

```
power_handle_glitch {true | false}
```

Default: false

Data_type: Boolean

A transition is defined as a glitch when the time difference between two sequential toggles are less than half of the rise transition time + fall transition time in the vector-based static power calculation.

If set to `true`, glitch power will be identified and calculated according to this definition and it will be reported in a separate column. Switching or internal power calculation does not include power due to glitches.

If set to `false`, glitch power will not be identified, and the respective power will be calculated as normal signal switches. Hence, the total power will be a little higher.

Related Commands

- `set_power`

power_handle_transport_glitch

```
power_handle_transport_glitch {true | false}
```

Default: `false`

Data_type: Boolean

Transport glitch is referred to as functional toggles for a net in a single clock period before it reaches a stable value. You can use this attribute to report the internal and switching power values for transport glitch.

The transport glitch power is reported in the following reports:

- Average Power Summary Report (`power.rpt.avgpwr`) - reported in the Total, Group, Clock, and Rail power sections.
- Instance Power Report (`power.rpt.instpwr`) - reported for all instances.
- Hierarchy Power Report (`power.rpt.hierpwr`) - reported for all hierarchies.

This attribute is automatically set to `true` with `power_handle_glitch true`. That is, reports both inertial and transport glitch power. When set to `false`, only the inertial glitch power is reported.

Related Commands

- `set_power`

power_handle_tristate

```
power_handle_tristate {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, all tristate device enable pin values will be taken into account when determining the propagation of the activity through tristate gates.

Related Commands

- `set_power`

power_hier_delimiter

`power_hier_delimiter character`

Default: " "

Data_type: String

Specifies the hierarchical delimiter in the DEF file. You can use this attribute to override the existing hierarchical delimiter.

Related Commands

- `set_power`

power_honor_combinational_logic_on_clock_net

`power_honor_combinational_logic_on_clock_net {true | false}`

Default: `true`

Data_type: Boolean

Specifies the behavior of combination logic on clock net.

- `true` (default) - specifies to honor activity propagation through combinational logic.
- `false` - specifies that the combination logic will be treated like a combinational clockgate.

Related Commands

- `set_power`

power_honor_default_switching_activity_in_scheduling

```
power_honor_default_switching_activity_in_scheduling {true | false}
```

Default: false

Data_type: bool

Controls whether default switching activity settings are applied during hierarchy/domain-based scheduling. This attribute must be used for switching activity assignment using the `set_default_switching_activity -pg_net` or `-hierarchy` parameters in the dynamic vectorless flow.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `set_power`

power_honor_negative_energy

```
power_honor_negative_energy {true | false}
```

Default: true

Data_type: Boolean

A value of `true` specifies that the software will keep negative internal energy numbers from the `.lib` internal power table. When set to `false`, negative power is treated as 0.

Related Commands

- `set_power`

power_honor_net_activity

```
power_honor_net_activity {true | false}
```

Default: `true`

Data_type: Boolean

Specifies to use activity from the `Net` section of the Switching Activity Interchange Format (SAIF) file for power calculation and optimization, and to ignore the pin-based state-dependent path delay (SDPD) activity. The default value of this attribute is `true`.

When this attribute is set to `false`, the power calculation engine can use both the net-based and pin-based activity. In this case, precedence will be given to activity from net or pin that appears later in the SAIF file.

Related Commands

- `set_power`

power_honor_non_mission_leakage

```
power_honor_non_mission_leakage {true | false}
```

Default: `false`

Data_type: Boolean

Supports modeling of well-to sub-leakage information when the circuit power supply is off for the level shifter cells. This is referred to as the non-mission mode leakage power calculation. The leakage power for a level shifter cell depends on the state of each PG pin. The Liberty file contains sections for PG pin conditions of each mode/state and non-mission mode leakage power for each PG pin of that mode.

When this attribute is set to `true`, the software identifies the `pg_pin` mode and assigns non-mission mode leakage power associated with the mode.

The use model of the non-mission mode leakage power calculation feature is:

```
power_honor_non_mission_leakage true  
power_off_pg_nets {VDD VDDO}
```

Example:

Let us consider a level shifter cell with the PG mode “`retained_VDDO_off`” and the following `pg_pin_condition`:

```
pg_setting_value (retained_VDDO_off) {
```

```
pg_pin_condition : "!VBB*VDD*VDDI ";  
pg_pin_active_state (VBB, V1);  
pg_pin_active_state (VDD, V2);  
pg_pin_active_state (VDDI, V2);
```

When you specify a list of PG nets to be turned off along with the `power_honor_non_mission_leakage` attribute, all cells/instances that have the PG state “retained_VDDO_off” associated with the specified PG pin condition, will use the corresponding non-mission leakage power.

Related Commands

- `set_power`

power_hybrid_analysis

```
power_hybrid_analysis {true | false}
```

Default: false

Data_type: Boolean

Specifies to perform both vector-based and vectorless power analysis in a single session.

Related Commands

- `set_power`

power_ignore_control_signals

```
power_ignore_control_signals {true | false}
```

Default: true

Data_type: Boolean

A value of `true` specifies that control signals will be ignored when propagating activity.

Related Commands

- `set_power`

power_ignore_data_phase_for_clock

```
power_ignore_data_phase_for_clock {true | false}
```

Default: false

Data_type: Boolean

Specifies to ignore data frequency in the clock path for the clock instances in the design. When this attribute is set to `true`, the software propagates only the fastest clock pin frequency instead of the data pin frequency for activity propagation through clock network. In the default mode, the software picks the fastest frequency between the data phase and clock phase on that pin.

Related Commands

- `set_power`

power_ignore_end_toggles_in_profile

```
power_ignore_end_toggles_in_profile {true | false}
```

Default: false

Data_type: Boolean

By default, the software profiles the toggles in the user-selected vector time window by steps including the toggles at the start/end boundary of the window. In each step, the toggles on the start boundary edge of the step is counted in the current step; the toggles on the end boundary edge is counted in the next step.

When this attribute is set to `false`, the toggles on the end boundary edge is also counted in the last step, as there is no next step to it. This may lead to double count of the power profile for the last step, especially for zero-delay vector profiling.

When this attribute is set to `true`, the software ignores the toggle on the end boundary of the last step.

Related Commands

- `set_power`

power_ignore_glitches_at_same_time_stamp

```
power_ignore_glitches_at_same_time_stamp {true | false}
```

Default: `true`

Data_type: Boolean

Specifies to ignore glitches at the same transient time in the vector-based static and dynamic power analysis flow. When this attribute is set to `true`, only the last signal value change is honored, and all the preceding value changes at the same time stamp are ignored. The last signal value will overwrite the previous value for the same signal at the same time stamp, so the last value will represent the state of the signal at that time stamp.

Related Commands

- `set_power`

power_ignore_inout_pin_cap

```
power_ignore_inout_pin_cap {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, ignores the bidirectional pin capacitances (`direction : inout`) defined for I/O cells in the .lib file, when calculating switching power and internal power; this also includes the default capacitance value for `inout` pins.

Related Commands

- `set_power`

power_ignore_macro_leakage_scale_for_temperature

```
power_ignore_macro_leakage_scale_for_temperature {true | false}
```

Default: `false`

Data_type: Boolean

The `power_leakage_scale_factor_for_temperature` attribute is used to perform linear scaling of

the leakage power for temperature in both the standard cell and macro libraries.

The `power_ignore_macro_leakage_scale_for_temperature` attribute allows you to selectively apply the scale factor only for standard cells, and not use the scale factor for macro cells.

Related Commands

- `set_power`

power_ignore_unconstraint_net_with_timing_data

```
power_ignore_unconstraint_net_with_timing_data {true | false}
```

Default: `false`

Specifies to ignore unconstrained nets/pins from toggle generation. These nets/pins do not have an associated clock but contain valid timing data with arrival windows in the TWF file. If set to `false`, the tool applies default frequency toggles to these unconstrained nets, which could cause false IR violation warnings due to high voltage drop.

Related Commands

- `set_power`

power_include_file

```
power_include_file file
```

Default: `" "`

Data_type: String

Specifies a power analysis include file. You can use this file to specify certain commands of the dynamic engine which do not have a Voltus equivalent. This attribute is used to pass the legacy power analysis commands and `report_power` command options only in the dynamic mode (vectorbased and vectorless). If the `power_method` attribute is set to `static`, the `power_include_file` attribute is ignored. The `power_include_file` attribute is required only if the `power_static_netlist` attribute is set to `def`.

Related Commands

- `set_power`

power_include_initial_x_transitions

```
power_include_initial_x_transitions {true | false}
```

Default: `true`

Data_type: Boolean

Controls how the initial X state toggle should be counted. The initial X state means the state at time 0.

When set to `true`, this means that the software counts the power caused by the `X->0` or `X->1` toggle. When set to `false`, the software does not count the power caused by the initial X state toggle.

Related Commands

- `set_power`

power_include_sequential_clock_pin_power

```
power_include_sequential_clock_pin_power {true | false}
```

Default: `false`

Data_type: Boolean

Reports the clock pin power of flip-flops as part of the clock network power.

Related Commands

- `set_power`

power_include_timing_in_current_file

```
power_include_timing_in_current_file {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to write timing bits for non-switching instances in the current files during the state-propagation-based flow. When specified, rail analysis will report effective instance voltage (EIV) for both switching and non-switching instances. If the parameter is set to `false`, rail analysis will report EIV for the switching instances only.

Related Commands

- `set_power`

power_ir_derated_timing_view

`power_ir_derated_timing_view view_name`

Default: " "

Data_type: String

Specifies the name of the timing view used to calculate timing slack and to rank the flops in the Tempus PI flow. This attribute must be specified if there are multiple views.

The `power_ir_derated_timing_view` attribute defines the timing view used to detect the IR-sensitive paths using a lower voltage than the actual analysis. This lower voltage simulates the IR-drop for each instance in the design. The software uses this view to prioritize the end-points that need to be prioritized for switching.

This attribute is only applicable to the state-propagation-based vectorless flow (`set_db power_method dynamic_vectorless power_enable_state_propagation true`).

Related Commands

- `set_power`

power_leakage_scale_factor_for_temperature

`power_leakage_scale_factor_for_temperature view_name`

Default: 1.0

Data_type: Double

Performs a linear scaling of the leakage power for temperature in all libraries. This replaces the k-factor for temperature in the .lib file. For process and voltage, the scaling will continue to be based on the corresponding k-factors in the .lib file. A value of 0.8 scales the leakage power in the library to 80 percent. A value of 1.2 scales the leakage power in the library to 120 percent.

Related Commands

- `set_power`

power_lib_files

```
power_lib_files in_file <file1[file2] ...
```

Default: ""

Data_type: String

Specifies the name of the timing library file(s) for dynamic power analysis. This attribute allows you to run the dynamic power analysis flow without reading a dotlib into design database to reduce background memory consumption. You must have an external TWF file to run this flow as the timing engine will not be invoked from within Voltus. This flow is recommended for large designs where front-end processing of timing libraries is high. Some of the database query commands will not work if this attribute is used.

Related Commands

- `set_power`

power_library_preference

```
power_library_preference {voltage | ecsm_ccsp}
```

Default: voltage

Data_type: String

Instances of the same cell can be attached to different power domains in a design. If you have specified different PVT libraries, then the software will by default bind each instance to the closest library definition.

You can use this attribute to select either the closest Effective Current Source Model (ECSM)/Composite Current Source-power (CCSP), or the closest voltage library for the instance:

- `voltage`: when specified, enables library binding to the closest voltage library.
- `ecsm_ccsp`: when specified, enables library binding to the closest ECSM/CCSP library.

Related Commands

- `set_power`

power_macro_pgv_leakage

```
power_macro_pgv_leakage {true | false}
```

Default: true

Data_type: Boolean

Controls leakage power reporting for cells that have power information available in both dotlib and PGV. When the attribute is set to `false`, the leakage current from Liberty is used and when the attribute is set to `true`, the leakage current from PGV is used.

Related Commands

- `set_power`

power_macro_toggle_pin_percentage

```
power_macro_toggle_pin_percentage percentage
```

Default: 0.0

Data_type: Double

Specifies the percentage of output data pins (ignoring the control pins) to be scheduled for a macro cell. This attribute is applicable to the state-propagation based vectorless flow. The default macro pin toggle percentage is 50%.

Related Commands

- `set_power`

power_match_state_for_logic_x

```
power_match_state_for_logic_x value
```

Default: x

Data_type: String

Controls how the logic X will be evaluated in the boolean function of the 'when' state of a power table.

The attribute uses the following arguments:

- 0 - regards X as 0
- 1 - regards X as 1
- x/X - regards X as neither 0 nor 1, that is, whenever there is any X logic in it, the boolean function will be evaluated 0 (false).

Related Commands

- `set_power`

power_memory_current_scaling_method

`power_memory_current_scaling_method {1 | 2 | 3}`

Default: 0

Data_type: Integer

Specifies to enable scaling of the NLPN-based currents by a user-specified scale factor for macro cells that are switching. The applicable arguments are 1, 2, and 3, which means enable a scaling factor of 0.5, 0.25, and 0.1 respectively.

Related Commands

- `set_power`

power_multibit_flop_toggle_behavior

`power_multibit_flop_toggle_behavior { simultaneous | independent | sbff }`

Default: independent

Data_type: Enum

Specifies the toggle behavior of multi-bit flip-flops (MBFF). The possible arguments are:

- `simultaneous`: The multiple bits of the MBFF will toggle or not toggle at the same time. The MBFF will be considered as one flip-flop.
- `independent`: The multiple bits of the MBFF will toggle independently. The MBFF will be considered as one flip-flop.

- `sbf`: The MBFF will be considered as multiple Single-Bit Flip-Flops (SBFFs). Each bit will be considered as one flip-flop, and will toggle independently.

Related Commands

- `set_power`

power_multibit_flop_toggle_pin_percentage

`power_multibit_flop_toggle_pin_percentage` *percentage*

Default: 0.0

Data_type: Double

Specifies the percentage of multi-bit flip-flops (MBFF) pins to be scheduled. This attribute is applicable to the state-propagation based vectorless flow. The attribute will only work when `power_multibit_flop_toggle_behavior` is set to `independent`.

Related Commands

- `set_power`

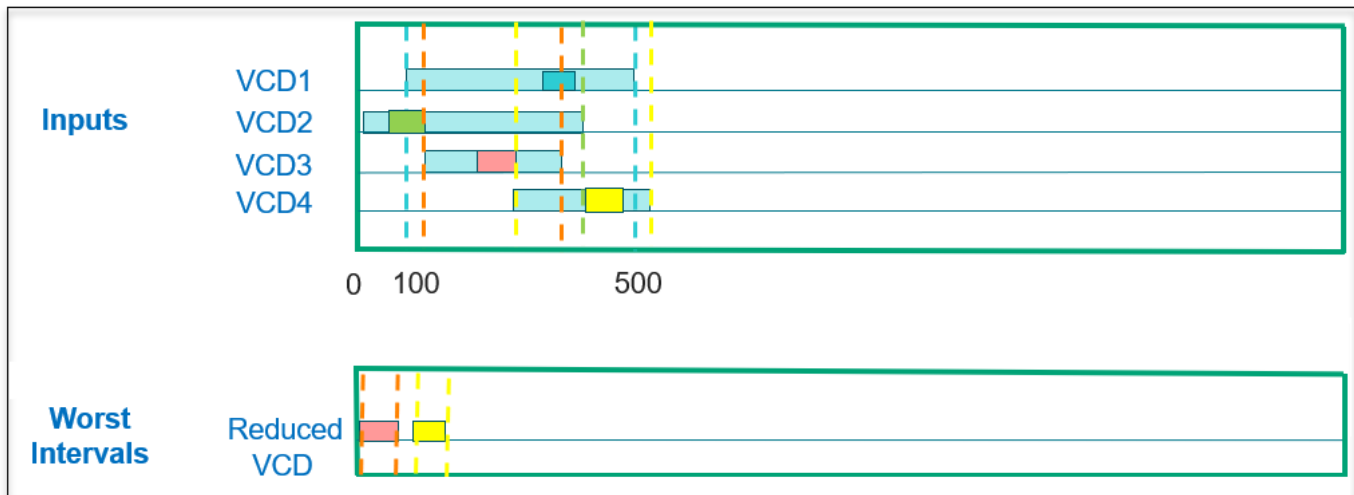
power_multi_scenario_simulation

`power_multi_scenario_simulation` {true | false}

Default: false

Data_type: Boolean

Specifies to create a long vector by combining multiple vectors and to identify the worst power intervals across the combined vector. The long vector includes all scenarios that are used sequentially and are non-overlapping.



This attribute allows you to perform power and current analysis using worst peak power interval across multiple simulation vectors.

Related Commands

- `set_power`

power_off_pg_bulk_leakage

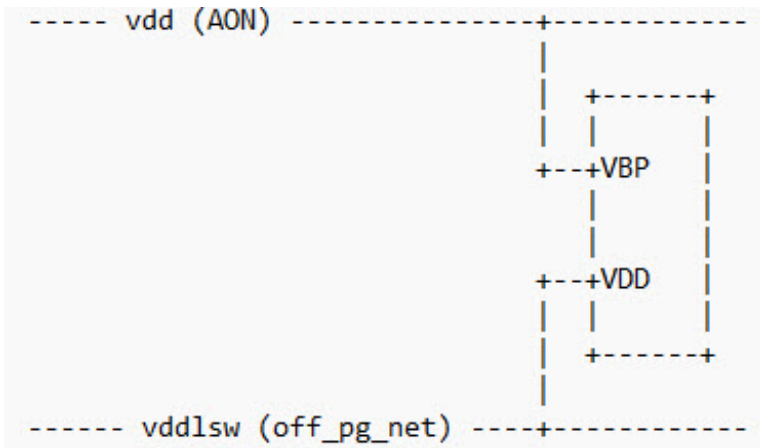
```
power_off_pg_bulk_leakage { true | false }
```

Default: false

Data_type: bool

Specifies to account for the leakage power through an instance's bulk pin when it is connected to a power-ground (PG) rail in the ON state.

The following diagram illustrated that the instance contributes bulk pin leakage to the ON state rail (**vdd**) through the *VBP* pin and zero leakage to the OFF state rail (*vddlsw*) through the VDD pin.



Related Commands

- `set_power`

power_merge_switched_net_currents

```
power_merge_switched_net_currents {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to merge switched net currents into always-on currents. For designs with a large number of switched nets, this attribute helps improve performance by merging switched net instance currents with always-on instance currents. However, when this attribute is set to `true`, you will not be able to use the generated current files to run power-up analysis.

Related Commands

- `set_power`

power_method

```
power_method { static | dynamic_vectorless | dynamic_vectorbased | dynamic_mixed_mode  
}
```

Default: `static`

Data_type: Enum

Specifies the type of analysis to be performed. You can perform two types of power analysis, namely static and dynamic.

- `static` - specifies to perform static power analysis to calculate average power.

Dynamic power analysis can be a vector-based or vectorless approach.

- `dynamic_vectorbased` - uses the full VCD output of a logic simulator to identify the instances that are switching and when they switch.
- `dynamic_vectorless` - uses the timing arrival window information from a static timing analysis tool to determine when instances switch. A timing arrival window describes when a signal can change within a clock cycle. An additional algorithm then determines the instances that switch.
- `dynamic_mixed_mode` - specifies to combine the vector-based and vectorless (mixed) dynamic analysis. This flow can be used in the following cases:
 - **case 1**: vectors are not available for every block of a full-chip design.
In this case, the software uses the vectorless methodology for blocks with incomplete vectors
 - **case 2**: blocks with vectors having unannotated combinational/ sequential logic.
In this case, the software performs state propagation for the missing combinational/ sequential logic. It uses the vectorless methodology for a logic cone that does not have the primary inputs or flops annotated from the vector, and then performs state propagation

To enable the dynamic mixed mode analysis flow, you must specify the following commands:

```
set_default_switching_activity -clock_gates_output 2.0 -global_activity  
<activity>  
set_db power_method dynamic_mixed_mode  
read_activity_file -scope <> -block <> -> (for available blocks)
```

Related Commands

- `set_power`

power_min_leaf_count

`power_min_leaf_count value`

Default: 0

Data_type: Integer

Specifies the minimum number of leaf instances required for writing the hierarchy data in the profiling database. It allows you to restrict the number of hierarchies for which waveforms are reported in the profiling database when using both the `power_write_profiling_db` attribute and `report_power -hierarchy` parameters.

Any hierarchy whose leaf instance count in its complete sub-tree is less than the `power_min_leaf_count` value will be skipped from getting reported in the profiling database. This attribute can be used to skip very small hierarchies and as a result save on the database writing time and disk space.

Related Commands

- `set_power`

power_off_pg_nets

`power_off_pg_nets net_list`

Default: ""

Data_type: String

The name of the power rails that will be turned off. The net name can contain instance hierarchy if appropriate. The software will take this data into account when figuring out the list of power gates which are switched off.

Note: A net is turned off if all connected power gates are turned off by control signals. When a net is off, all of the instances connected to the net will only contribute leakage current into the connecting nets.

Related Commands

- `set_power`

power_output_current_data_prefix

`power_output_current_data_prefix` *prefix*

Default: ""

Data_type: String

Specifies a prefix to the static or dynamic current files generated by the software. When power analysis is done in the multi-CPU mode, the output current data directory name and the underlying current files will also use the same prefix by default.

Related Commands

- `set_power`

power_output_dir

`power_output_dir` *directory*

Default: ""

Data_type: String

Specifies the name of the directory that power information will be output to.

Related Commands

- `set_power`

power_partition_count

`power_partition_count` *value*

Default: 0

Data_type: Integer

Specifies the number of physical partitions or jobs for a distributed run in the area-based partitioning flow. This attribute is only applicable to the event-based power analysis flow (`power_method event_based`) in the XP mode. The `power_partition_count` attribute must be specified with the `power_use_physical_partition` attribute.

Related Commands

- `set_power`

power_partition_spef

```
power_partition_spef {true | false}
```

Default: false

Specifies to generate block-level SPEF files from a flat SPEF file. This attribute allows you to process input SPEF data and automatically generate partition-level SPEF chunks. The attribute is supported only in the XP mode for full-chip designs using flat SPEF files.

The use model of this attribute is:

```
set_power_spef_files { flat.spef.gz }  
set_db power_partition_spef true
```

Related Commands

- `set_power`

power_partition_twf

```
power_partition_twf {true | false}
```

Default: false

Data_type: Boolean

Specifies to read the input TWF data (flat or hierarchical TWF files), and generate block-level TWFs. This attribute must be specified when the block/partition level TWF files are not available.

Related Commands

- `set_power`

power_pin_based_twf

```
power_pin_based_twf {true | false}
```

Default: false

Data_type: Boolean

Specifies whether to generate pin-based TWF or net-based TWF.

- When set to `false`, a net-based TWF file is generated. Net-based TWF file is smaller in size and therefore results in faster processing.
- When set to `true`, a pin-based TWF file is generated. Pin-based TWF file is bigger in size and requires longer processing time, however, is more accurate because the slews numbers are pin-based.

Note: This attribute must be used with the `read_sdc` command.

Related Commands

- `set_power`

power_precision

`power_precision value`

Default: 8

Data_type: Integer

Specifies precision of decimal range [1-8] to ensure consistent decimal places are displayed for each power component. This attribute is used for precision handling in `report_instance_power` and `report_power`.

Related Commands

- `report_power`

power_pre_simulation_empty_period

`power_pre_simulation_empty_period value`

Default: ""

Data_type: String

Specifies to add an empty pre-simulation period before simulation during IR drop analysis. Power analysis is not done for the pre-simulation period and flat dynamic currents are created.

Related Commands

- `report_power`

power_pre_simulation_period

`power_pre_simulation_period value`

Default: ""

Data_type: String

Specifies to set a pre-simulation period for IR drop analysis. Pre-simulation is used for generating power analysis reports and dynamic currents.

Related Commands

- `report_power`

power_pre_simulation_power_exclude_period

`power_pre_simulation_power_exclude_period value`

Default: ""

Data_type: String

Specifies to exclude power analysis during the pre-simulation period used for IR drop analysis. Pre-simulation is used only to create dynamic currents and not used for power reports.

Related Commands

- `report_power`

power_quit_on_activity_coverage_threshold

`power_quit_on_activity_coverage_threshold threshold_value`

Default: 0.0

Data_type: Double

Specifies the activity (VCD/FSDB/TCF) coverage threshold value. This attribute allows you to

control the acceptable activity coverage that you are expecting and enforce the software to quit when it does not meet the criteria. When you specify the threshold value, the software exits if the real activity coverage is below the threshold.

Related Commands

- `report_power`

power_read_rcdb

```
power_read_rcdb {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to write a SPEF file containing only the total C for signal nets and pass it to the Dynamic Power engine. Dynamic power analysis requires only the total C for signal nets. Therefore, instead of writing a large SPEF file (with distributed RC) from the RCDB, this attribute specifies to create and pass a SPEF file with only total C for all the signal nets. This significantly reduces the parasitic annotation time in the Dynamic Power engine.

Related Commands

- `report_power`

power_relax_arc_match

```
power_relax_arc_match {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to relax arc match requirements. This attribute is used only when the exact arc is not found in a .lib. When set to `true`, the internal energy is calculated as an average of multiple arcs with relaxed arc matching. If this attribute is set to `false`, relaxed arc search is not done and 0 internal energy is returned.

Related Commands

- `report_power`

power_report_library_usage

```
power_report_library_usage {true | false}
```

Default: false

Data_type: Boolean

Specifies to report library usage statistics in a text report. The report contains information such as, cell name, library format used (CCSP/ECSP/NLPM) for each cell, library name, and so on. This attribute allows you to perform library integrity checking for debugging purpose.

The format of the text report with sample content is given below:

Example:

#cell_name	net_name	op_voltage	lib_voltage	lib_format	missing_tables	
INV	VDD,VDDA	0.9	1.0	NLDM	leakage,timing	A
INV	VDDC	0.75	0.75	CCSP,NLDM	leakage	B

Related Commands

- `report_power`

power_report_missing_bulk_connectivity

```
power_report_missing_bulk_connectivity {true | false}
```

Default: false

Data_type: Boolean

Specifies to report missing bulk pins in the missing input report (`power_report_missing_input`).

Related Commands

- `report_power`

power_report_missing_input

```
power_report_missing_input {true | false}
```

Default: false

Data_type: Boolean

Specifies to report missing netlist information in input files, such as library, LEF/DEF, PGV, SPEF, TWF, and missing logical connectivity for a cell instance (PGNET).

The name of the report file is: `<prefix>_missingdata.<spef|lefdef|lib|pgv|twf|pgnet>`

Related Commands

- `report_power`

power_report_black_boxes

```
power_report_black_boxes {true | false}
```

Default: false

Data_type: Boolean

Specifies to report cells that are used as black boxes.

A black box is a cell which is either missing from the liberty file or has no functionality defined in the liberty file.

Related Commands

- `report_power`

power_report_clock_instance_switching_info

```
power_report_clock_instance_switching_info
```

Default: None

Data_type: String

Specifies to generate a report for clock instances that switched during the state-propagation based vectorless flow. The generated report contains the toggle time, instance name, input pin event, and

output pin event for each switching event of the clock instance.

Related Commands

- `report_power`

power_report_idle_instances

```
power_report_idle_instances {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to generate a report (`idleinstance.rpt`) that lists all the idle or non-switching instances.

The following is a snippet of the `idleinstance.rpt` report:

```
DFF22/DFF      F
DFF42/DFF      F
...
```

Here, the first column is the instance name and the second column is the type of instance (`F` indicates flop and `C` indicates combinational).

Related Commands

- `report_power`

power_report_instance_switching_info

```
power_report_instance_switching_info {all | output_logic | none}
```

Default: `none`

Data_type: Enum

Specifies to generate a report for combinational instance switched during state-propagation. In the output file, it contains the toggle time, instance name, input pin event, and output pin event for each switching event of the combination gate. The possible arguments of this attribute are:

- `all` - Specifies to report instance toggle information for both input and output pins. When this attribute is set to `all`, the switching of both the input and output pins of all the instances are reported in the `voltus_power.stateprop.switchinst` file and the switching of flops are

reported in the `voltus_power.stateprop.switchsrc` file.

- `output_logic` - Specifies to report instance toggle information for the output pins only. When this attribute is set to `output_logic`, the switching of the output pins of all the instances are reported in the `voltus_power.stateprop.switchinst` file and the switching of flops are reported in the `voltus_power.stateprop.switchsrc` file.
- `none` - Disables generation of instance toggle report. This is the default option. When this attribute is set to `none`, only switching of flops are reported in the `voltus_power.stateprop.switchsrc` file. The file does not report switching of instances.

This attribute is used for generating switching details of state-propagation and should be specified only for debugging purpose.

The following is a snippet of the `voltus_power.stateprop.switchinst` report:

```
*-----  
*Switch Time      Instance Name      Pin Name      Logic  
*-----  
*  
    0.097          B11/BUFF          I            rise  
    0.097          DFF11/DFF         Q            rise  
...  

```

Here,

- `Switch Time`: switching time of instances that can be either combinational or sequential
- `Instance Name`: instance name
- `Pin Name`: input/output pin of the instance
- `Logic`: present logic of the pin that can be either "rise" or "fall"

Related Commands

- `report_power`

power_report_instance_switching_list

`power_report_instance_switching_list filename`

Default: " "

Data_type: String

Specifies to generate a report (`voltus_power.stateprop.switchlist`) containing all the switching times for the specified instances when the `power_enable_state_propagation` attribute is set to `true`.

The format of the specified file is:

```
instname1
instname2
...
```

The following command is an example of the state-propagation-based dynamic vectorless flow, and also specifies to generate a report (`voltus_power.stateprop.switchlist`) containing all the switching times for the instances specified in the `inst_list` file:

```
set_db power_method dynamic_vectorless power_enable_state_propagation true
power_report_instance_switching_list inst_list
```

The following is the format of the `voltus_power.stateprop.switchlist` report:

```
*-----
---
*Instance Name    Pin Name    Pin Direction    Frequency    Switch Time
*-----
---
*
```

Here,

- `Instance Name`: instance name that can be either combinational or sequential
- `Pin Name`: name of the input/output pin of the instance
- `Pin Direction`: indicates whether the pin is an INPUT (I) or OUTPUT (O) pin
- `Frequency`: clock frequency
- `Switch Time`: gives the switching pattern for the instance, that is, `<time1> <logic> <time2> <logic>`. The logic of the pin can be "r" or "f". r indicates rise and f indicates fall.

The following is a snippet of the `voltus_power.stateprop.switchlist` report:

```
*-----
---
*Instance Name    Pin Name    Pin Direction    Frequency    Switch Time
*-----
---
*
    DFF11/DFF      QN          O                100          0.106 f 10.107 r
```

DFF11/DFF	Q	O	100	0.097 r 10.099 f
DFF11/DFF	CP	I	100	0.059 r 5.056 f 10.059 r 15.056
...				
...				

Related Commands

- `report_power`

power_report_missing_nets

```
power_report_missing_nets {true | false}
```

Default: false

Data_type: Boolean

Controls reporting of missing nets in both static and dynamic power analysis.

This attribute controls reporting for TWF and activity files in static analysis, and controls reporting for TWF/SDC/SPEF and activity files in dynamic analysis.

Related Commands

- `report_power`

power_report_scan_chain_statistics

```
power_report_scan_chain_statistics {true | false}
```

Default: false

Data_type: Boolean

Specifies to generate a report file called `voltus_power.scanchain.stat` during scan mode analysis that contains scan chain statistics.

The report has 2 parts:

- **Global Statistics:** this section has the cumulative statistics of all scan chains in the design
- **Chain Statistics:** this section has the statistics for each scan chain in the design

The Global and Chain Statistics sections have the following information:

- **Total**: total instances specified in the control file or in a scan chain
- **AssignedFF**: flip-flops defined in the scan control file which exist in the design
- **SwitchedFF**: flip-flops that have switching event at the output
- **Non-ScanFF**: flip-flops that have no scan-out pin
- **DuplicatedFF**: flip-flops that are specified in multiple chains
- **StateConflictFF**: flip-flops that have conflicting state assignment

Related Commands

- `report_power`

power_report_statistics

```
power_report_statistics {true | false}
```

Default: false

Data_type: Boolean

Reports a set of statistics on the instance power, instance power density, clock power, or transition density. This attribute is useful in viewing and debugging user defined activity factors.

Related Commands

- `report_power`

power_report_time_display_fraction_digits

```
power_report_time_display_fraction_digits value
```

Default: -1

Data_type: Integer

Sets the number of fractional digits or decimal places displayed for the instance switching time values in the `voltus_power.stateprop.switchinst` file. This attribute displays fractional digits that show the exact time difference if there are two events (rise/fall) of an instance switching at the same

time.

Related Commands

- `report_power`

power_report_twf_attributes

```
power_report_twf_attributes {summary| detailed}
```

Default: ""

Data_type: String

Controls the generation of the TWF attribute report based on the TWF attributes that are specified using the `set_twf_attribute` command.

The possible arguments are:

- `summary` - generates a summarized static timing analysis report
“`voltus_power.twf_attribute.summary`” that displays the number of user-defined TWF attributes (delay, slew, and slack values for both the rise and fall transitions) and the number of annotated TWF attributes for dynamic power analysis. The following is a snippet of the report:

```
STATISTICS:
           User Defined      Annotated      Annotated/Total (%)
shift_arrival_time         5              4             80
rise_slew                   2              2            100
fall_slew                   2              2            100
rise_delay                  5              4             80
fall_delay                  5              4             80
rise_slack                  2              2            100
fall_slack                  2              2            100
```

- `detailed` - generates a detailed static timing analysis report “`voltus_power.twf_attribute.detailed`” that includes the summary report, and a detailed report of the rise and fall transitions for the TWF attributes of each annotated instance/pin. It also includes the shift arrival time report.

This argument also allows you to generate a report `voltus_power.twf.rpt` containing the TWF statistics to highlight clock-frequency and delay used for the static and dynamic power calculation.

The TWF statistics report is generated only if:

- the `read_twf` command is specified before specifying the `power` category attributes.
- the `power_method static power_static_netlist def` or `power_method dynamic_vectorless | dynamic_vectorbased` command is specified.

The format of the `voltus_power.twf.rpt` report is:

```
# Voltus TWf Statistics
# Units: Freq (Hz), Delay (second), Slew (second)
# TWf File List:
#     TWf_file1
#     TWf_fileN

USER DEFINED TWf ATTRIBUTE STATISTICS
<Clock Freq> <Min Rise_Delay> <Rise Delay> <Max Rise Delay> <Min Fall Delay> <Fall
Delay> <Max Fall Delay> <Net>
```

The `USER DEFINED TWf ATTRIBUTE STATISTICS` section is included in the report only if the TWf attributes are defined using the `set_twf_attribute` command.

Example :

#check all instances of inst_C1 get shifted by 3ns

```
set_twf_attribute -pin inst_C1 -shift_arrival_time 3
```

#check all instances of inst_D1 get shifted by 3ns

```
set_twf_attribute -net inst_D1 -shift_arrival_time 3
```

```
set_db power_method dynamic_vectorless ; set_db power_write_db false ; set_db
power_disable_static false ; set_db power_enable_state_propagation true; set_db
power_current_generation_method peak ; set_db power_grid_libraries ../../library_pv.cl ;
set_db power_report_twf_attributes summary
```

Related Commands

- `report_power`

power_reuse_flop_ranking_data

```
power_reuse_flop_ranking_data directoryname
```

Default: ""

Data_type: String

Specifies the name of the directory where the flop ranking data to be reused is placed.

Related Commands

- `report_power`

power_reuse_flop_ranking_data_hier

```
power_reuse_flop_ranking_data_hier blockname
```

Default: ""

Data_type: String

Allows you to choose the block/hierarchy whose ranking data is to be used for the block-level analysis. This attribute is used when there are multiple instantiations of the same DEF block.

Related Commands

- `report_power`

power_scale_to_sdc_clock_frequency

```
power_scale_to_sdc_clock_frequency {true | false}
```

Default: false

Data_type: Boolean

Specifies to enable the VCD clock frequency to be scaled up to the SDC clock frequency. This allows you to match the VCD clock to the SDC clock to run analysis. By default, the clock frequency is scaled based on the activity file. When you set the `power_scale_to_sdc_clock_frequency` attribute to `true`, the power report and current waveforms apply the after-scaled clock frequency.

The default behavior for clock frequency scaling in the RTL and Gate level VCD flows is different, as described below:

- **RTL VCD** - By default, the clock frequency is scaled to the SDC clock frequency (`power_scale_to_sdc_clock_frequency true`) when the RTL VCD flow is enabled (`power_enable_rtl_dynamic_vector_based true`).
You can disable clock frequency scaling by setting the `power_scale_to_sdc_clock_frequency`

attribute to `false`.

- **Gate VCD** - By default, scaling of the clock frequency is disabled (`power_scale_to_sdc_clock_frequency false`).

You can enable the VCD clock frequency to be scaled up to the SDC clock frequency by setting the `power_scale_to_sdc_clock_frequency` attribute to `true`. This works only when `power_use_zero_delay_vector_file` is set to `true`, which means that the gate VCD is a zero-delay gate VCD.

Related Commands

- `report_power`

power_scan_chain_activity

```
power_scan_chain_activity {design_name chain_name activity_number} |  
{vectorless activity_number}
```

Default: ""

Data_type: String

Specifies to either provide vectorless activity for all scan chains or provide a specific activity for a specific chain. This attribute allows you to specify vectorless or user-defined activity to scan chains.

The following are the two use models:

- Generate the scan control file for a specific chain:
`set_db power_scan_chain_activity {design_name chain_name activity_number}`
- Generate the scan control file for all scan chains:
`set_db power_scan_chain_activity {vectorless activity_number}`

The `power_scan_chain_activity` attribute is supported only in the XP mode.

Related Commands

- `report_power`

power_scan_chain_name_pattern

```
power_scan_chain_name_pattern {design_name chain_name pattern}
```

Default: ""

Data_type: String

Specifies to automatically generate a scan-chain control file from a scan-chain based DEF file for the shift mode scan-chain analysis. This attribute allows you to create a control file for a large-sized DEF containing long chains or for multiple-DEF files. The generated scan-chain control file is saved in the power output directory.

The use model of the attribute is:

- `power_scan_chain_name_pattern {all pattern}` - generate scan control file for all scan chains.
Example: `power_scan_chain_name_pattern {all 11010}`
- `power_scan_chain_name_pattern {design_name chain_name pattern}` - generate scan control file for a specific chain.
Example: `power_scan_chain_name_pattern {counter AutoChain_1_seg1_clk_rising 10010}`

Using this attribute, you can generate the control file for both flat and hierarchical DEFs. For more information, refer to [Scan Control File Format](#).

Related Commands

- `report_power`

power_scan_control_file

```
power_scan_control_file filename
```

Default: ""

Data_type: String

Specifies the scan control file name required to run the Scan Mode Analysis flow.

The scan control file contains details about the scan analysis mode used for testing. For more information, refer to [Scan Control File Format](#).

Related Commands

- `report_power`

power_set_power_event_based

`power_set_power_event_based {true | false}`

Default: `false`

Data_type: Boolean

Specifies the instance-based power constraints that are to be taken into consideration for the power analysis flow. The attribute controls whether the power constraints are applied to the dynamic or static flow. The power constraints are specified using the `set_power` command.

The default value of `false` specifies to apply the power constraints in the static flow. When set to `true`, the constraints are applied to the dynamic flow.

Example:

The following command specifies to honor the instance-based power constraints in the dynamic power analysis flow:

```
set_db power_method event_based power_set_power_event_based true
set_power -ascii_power_file instPower.txt
report_power -out_file power.rpt
```

Related Commands

- `report_power`

power_settling_buffer

`power_settling_buffer value`

Default: " "

Data_type: String

Specifies the settling buffer time (in ps unit) between multiple windows of a VCD file. This attribute is used when you specify multiple pairs of the start time and end time for the non-overlapping multiple windows specified in an activity file (using the `read_activity_file` command).

Multiple current waveforms representing multiple VCD windows are merged into a single

continuous current waveform. To capture the waveform for signals at the edge of user windows, a buffer time is added between the windows being stitched. This buffer time ensures that the stitched waveform is contiguous. The generated waveform hence has the duration of user windows and the intervening settling buffers. The default settling buffer period is 400ps.

You must specify the buffer value without unit.

Related Commands

- `report_power`

power_show_cell_usage_stats

```
power_show_cell_usage_stats {true | false}
```

Default: false

Data_type: Boolean

Specifies to show the cell usage statistics in the power report. It provides cell usage statistics alongside power values, indicating the number of cells used from a specific library and the corresponding power percentage. This helps identify libraries with the greatest and least impact on power consumption.

You must ensure that this attribute is specified before using the `report_power` command, as shown below:

```
set_db power_show_cell_usage_stats true
report_power -out_file pl.rpt
```

Following is a snippet from the report:

```
Total leakage power = 0.00118471 mW
Cell usage statistics:
Library jlos_h , 88 cells ( 7.351713%) , 6.56434e-06 mW ( 0.554089% )
Library jlos_l , 123 cells ( 10.275689%) , 0.000799809 mW ( 67.511081% )
Library jlos , 986 cells ( 82.372597%) , 0.000378334 mW ( 31.934830% )
```

Related Commands

- `report_power`

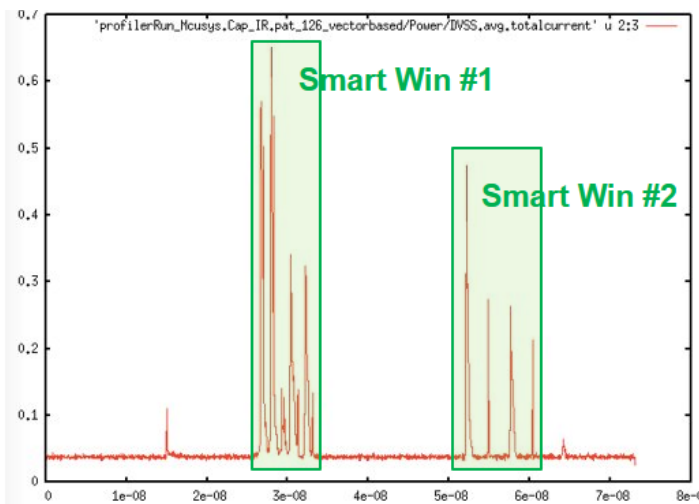
power_smart_window

```
power_smart_window {true | false}
```

Default: false

Data_type: Boolean

Specifies to enable the smart window feature to extract peak power scenarios from a vector-based activity file. A smart window is a variable size window to capture maximum regions of high power dissipation. An activity file can have multiple variable-sized regions that are ranked in the descending order of power metric.



The use model of the `power_smart_window` attribute is:

```
set_db power_smart_window true
set_db power_worst_step_size value
set_db power_worst_window_type [ power | activity | delta_power | delta_activity | ir |
critical_inst_power ]
```

Related Commands

- `report_power`

power_scan_multi_bit_flop_chain_type

```
power_scan_multi_bit_flop_chain_type type
```

Default: liberty

Data_type: String

Specifies the type of multi-bit scan flip-flop.

- `liberty` - specifies to use the syntax in Liberty to decide the multi-bit flip-flop type.
- `serial` - specifies to pick the first flop as the starting flop, and treat the scan chain as serial, regardless of `.lib`.
- `parallel` - specifies to treat the scan chain as parallel, regardless of `.lib`.

Related Commands

- `report_power`

power_split_bus_power

```
power_split_bus_power {true | false}
```

Default: `false`

Data_type: Boolean

A value of `false` applies the internal power number to each individual bit.

A value of `true` divides the internal power number by bus width, before applying it to the bus bit.

Related Commands

- `report_power`

power_start_time_alignment

```
power_start_time_alignment {true | false}
```

Default: `true`

Data_type: Boolean

Specifies to align all the signals of the activity file at time zero. When set to `true`, the simulation period is `max(endtime - starttime)`. When it is `false`, the simulation period is `max(endtime) - min(starttime)`.

Note: This attribute is not supported in the vector profiling flow. In the vector profiling flow, the start time will always be the `min(starttime)` and the end time will always be `max(endtime)`

Related Commands

- `report_power`

power_state_dependent_leakage

```
power_state_dependent_leakage {true | false}
```

Default: `true`

Data_type: Boolean

When set to `false`, power analysis performs state independent leakage power calculation.

Related Commands

- `report_power`

power_stateprop_ignore_unannot_pins

```
power_stateprop_ignore_unannot_pins {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to ignore unannotated pins in the functional logic of output pins for the RTL propagation flow. This attribute is set to `true` if there are unannotated input pins for instances and such pins are used in the functional logic of output pins that are propagated. For example output pin Q is propagated with functional logic A & B where A is unannotated and might cause wrong propagation for the Q pin.

Related Commands

- `report_power`

power_static_multi_mode_scenario_file

```
power_static_multi_mode_scenario_file filename
```

Default: `" "`

Data_type: String

Specifies a file with information on instance scaling scenario for the static multi-mode IR analysis flow. When this parameter is specified, Power Analysis creates the multi-mode static current file that is used during rail analysis. The format of the input file is:

INST_NAME	PIN_NAME	SCALE_FACTOR_1		SCALE_FACTOR_N
-----------	----------	----------------	-------	----------------

Here, the first column indicates the macro instance name, the second column indicates the pin name, followed by multiple current scaling factors. Each column of the scaling factor is referred to as a scenario. You must specify positive values only for scale factors.

The following is a snippet from the specified input file:

```
top/macroA    VDD      1      0.5    0.5    1
top/macroA    VDDM     0.5    0.5    0.5    1
top/macroB    VDD      0.5    1      0.5    0.5
```

If scale factors are not specified for certain scenarios, the tool will automatically use the instance's last stored scale factor value to complete current file generation.

In the following example, scale factor value 0.45 will be used in the 3rd and 4th scenario for instanceC, and scale factor value 0.35 will be used in the 4th scenario for instanceD:

```
instanceA VDD 0.45 0.55 0.65 0.31
instanceB VDD 0.31 0.5 0.67 0.21
instanceC VDD 0.25 0.45
instanceD VDD 0.31 0.25 0.35
```

Related Commands

- [report_power](#)

power_static_netlist

```
power_static_netlist {verilog | def}
```

Default: verilog

Data_type: String

Specifies to perform static power analysis using a Verilog or DEF only netlist.

By default, the static power analysis is performed using the Verilog netlist as the primary netlist. Using this attribute, you can specify DEF when a DEF netlist with logical connectivity is available, and use it as primary and not consider the Verilog netlist.

This attribute is also enabled when `set_rail_analysis_config -report_power_in_parallel` is

true for static power computation.

Related Commands

- `report_power`

power_switching_power_on_rise_only

```
power_switching_power_on_rise_only {true | false}
```

Default: false

Data_type: Boolean

Specifies to perform switching power analysis only for rise toggles.

Related Commands

- `report_power`

power_thermal_input_file

```
power_thermal_input_file file
```

Default: ""

Data_type: String

Specifies to read a power map file to perform thermal analysis. Thermal input file contains die area divided into tiles, and temperature value for each tile. This temperature of each tile is used to calculate power of all the instances that fall in that tile's bounding box.

The format of the thermal input file is:

```
DIE_TEMPERATURE_MAP
DIE_AREA    xmin(um) ymin(um) xmax(um) ymax(um) nx(int) ny(int)

NUMBER_OF_LAYERS    nLayers(int)
LAYER    layer_name(string)
TEMPERATURE

XCOORD    x0(0)    x0(1)    x0(2) ... ...x0(nx-1)
YCOORD    y0(0)    y0(1)    y0(2) ... ...y0(ny-1)
```

```

T0 (0,0)      T0 (1,0)      ...      T0 (nx-1,0)
T0 (0,1)      T0 (1,1)      ...      T0 (nx-1,1)

:
T0 (0,ny-1)   T0 (1,ny-1)           T0 (nx-1,ny-1)
END_TEMPERATURE
END_LAYER
END_DIE_TEMPERATURE_MAP

```

where,

- `DIE_AREA` specifies the coordinates of the four points of die outline as well as the tile number in X and Y direction.
- `NUMBER_OF_LAYERS` is the number of layers in each die instance
- `LAYER` specifies the LEF layer name
- `XCOORD` and `YCOORD` are the coordinates of each tile
- `T0 (0,0) ....` is the temperature value assigned to a tile at a specific coordinate

Related Commands

- `report_power`

power_thermal_leakage_temperature_scale_table_file

`power_thermal_leakage_temperature_scale_table_file` *file*

Default: ""

Data_type: String

Specifies a file containing the user-defined temperature values and temperature scale factors for each instance. This attribute allows you to use the current file (.ptiavg) with re-adjusted leakage power values to perform thermal-aware IR drop analysis.

The `power_thermal_leakage_temperature_scale_table_file` attribute must be specified with the `power_thermal_input_file` attribute to import the temperature map file generated by Celsius.

When the new attribute is specified, the generated power will include the scaled leakage power values.

The following is the format of the temperature-aware leakage scaling table:

```
#Type      Instance_Name      temp1      temp2      temp3
```

```
temp4
(Type: Lib/Cell/inst)
```

```
inst      inst_name1      scalefactor@temp1      scalefactor@temp2      scalefactor@temp3
scalefactor@temp4
inst      inst_name2      scalefactor@temp1      scalefactor@temp2      scalefactor@temp3
scalefactor@temp4
```

File Format Example:

#Type	Name	25	50	125	150
inst	ring/b213	1	2	5	10
inst	ring/b212	1	2	5	10
inst	ring/b211	1	2	5	10
inst	ring/b210	1	2	5	10

Related Commands

- `report_power`

power_to_x_transition_factor

```
power_to_x_transition_factor value
```

Default: 0.5

Data_type: Double

Specifies how 0/1 to X transitions are counted.

Related Commands

- `set_power`

power_to_z_transition_factor

```
power_to_z_transition_factor value
```

Default: 0.25

Data_type: Double

Specifies how 0/1 to Z transitions are counted.

Related Commands

- `set_power`

power_transition_factor_based_duty

```
power_transition_factor_based_duty {true | false}
```

Default: `false`

Data_type: Double

Specifies that duty for X and Z toggles is calculated using the X and Z transition factor of toggle specified using the following attributes:

- `power_from_x_transition_factor`
- `power_from_z_transition_factor`
- `power_to_x_transition_factor`
- `power_to_z_transition_factor`

The default value of `false` specifies to use the duty value given with the `set_default_switching_activity/set_switching_activity -duty` parameter.

Related Commands

- `report_power`

power_transition_time_method

```
power_transition_time_method {min | avg | max}
```

Default: `max`

Data_type: String

Specifies the minimum, maximum, or average transition time method that will be used with the integrated timer or the external TWF.

Related Commands

- `report_power`

power_twf_delay_annotation

```
power_twf_delay_annotation {min | avg | max}
```

Default: avg

Data_type: String

Allows you to choose between min, max, or avg arrival times from the timing window file (TWF). The arrival time is annotated to the vectors processed from the RTL VCD/FSDB Zero-Delay flows.

Related Commands

- `report_power`

power_twf_load_cap

```
power_twf_load_cap {min | avg | max}
```

Default: max

Data_type: String

Allows you to select minimum, maximum, or average value of TWF external load or capacitance for power calculation.

Related Commands

- `report_power`

power_unified_power_switch_flow

```
power_unified_power_switch_flow {true | false}
```

Default: true

Data_type: Boolean

Specifies to enable the unified analysis of the always-on and switch nets. When set to `true`, the power-up and always-on analyses are performed in a single flow, wherein the switch nets and always-on nets are simulated together resulting in improved performance and accuracy.

Related Commands

- `report_power`

power_use_cell_leakage_power_density

```
power_use_cell_leakage_power_density {true | false}
```

Default: true

Data_type: Boolean

Specifies to use library leakage density times area (`default_leakage_power_density`), if defined, instead of the library's default cell leakage power (`default_cell_leakage_power`). This is used when leakage power for cells are missing or not defined.

Related Commands

- `report_power`

power_use_fastest_clock_for_dynamic_scheduling

```
power_use_fastest_clock_for_dynamic_scheduling {true | false}
```

Default: false

Data_type: Boolean

Specifies to use only the respective fastest clock associated with each net or pin to schedule events for a given simulation period.

Related Commands

- `report_power`

power_use_lef_for_missing_cells

```
power_use_lef_for_missing_cells {true | false}
```

Default: false

Data_type: Boolean

Allows a combination of LEF and PGV in the dynamic analysis flow. If there are cells that are common to LEF and PGV, the PGV cell will get the precedence.

Related Commands

- `report_power`

power_use_only_ddv_current

```
power_use_only_ddv_current {true | false}
```

Default: `true`

Data_type: Boolean

Controls the use of static current of a detailed dynamic power-grid view (DDV PGV) cell for static power calculation. The parameter is set to `true` by default, that is, the DDV PGV static current is not used in the static power calculation flow. If you want to use the DDV PGV static current for static power calculation, set this parameter to `false`.

Related Commands

- `report_power`

power_worst_case_vector_activity

```
power_worst_case_vector_activity {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to use the worst activity value when multiple vectors are specified in the static power calculation flow.

Related Commands

- `report_power`

power_worst_step_size

`power_worst_step_size value`

Default: ""

Data_type: String

Specifies the step size or resolution for vector profiling. Activity/ Power profiling is carried out for every step size that you specify.

When this attribute is used with `power_worst_window_size`, the specified step size is the resolution at which window sliding is performed. The supported time units are s, ms, us, ns, and ps. If no explicit time unit is specified, the default ns will be applied.

Related Commands

- `report_power`

power_worst_window_count

`power_worst_window_count value`

Default: 1

Data_type: String

Specifies the number of worst windows from the last vector profile run to be used for dynamic power analysis. By default, the software implements a single worst window. This attribute provides n top windows with maximum activity/power/IR. This attribute is used when you need to combine dynamic power analysis and vector profiling for multiple worst-case windows in a single run.

Related Commands

- `report_power`

power_worst_window_size

`power_worst_window_size value`

Default: ""

Data_type: String

Specifies the duration of dynamic analysis. This attribute is used for sliding window vector profiling that involves sliding the profiling windows by using a very small resolution value to get the window with the worst current profile. `power_worst_window_size` must be specified with `power_worst_step_size`.

The supported time units are s, ms, us, ns, and ps. If no explicit time unit is specified, the default ns will be applied.

Related Commands

- `report_power`

power_worst_window_type

```
power_worst_window_type {power | activity | delta_power | delta_activity | ir |
critical_inst_power}
```

Default: ""

Data_type: String

Specifies the window type to be used for multi-window analysis. The possible arguments are:

- `power` - worst instance power
- `activity` - worst instance activity
- `delta_power` - worst dp/dt (rate of change of power)
- `delta_activity` - worst da/dt (rate of change of activity)
- `ir` - worst IR
- `critical_inst_power` - worst power for critical/IR-sensitive instance (worst timing and IR)

Related Commands

- `report_power`

power_worst_window_coverage

```
power_worst_window_coverage { top | tile }
```

Default: ""

Data_type: String

Specifies how the worst window type is applied for multi-window analysis in the vector profiling flow. The following are the possible options of this attribute:

- `top`: worst window selection for the full design
- `tile`: interval selection as per the worst tile power

Use Model:

```
set_db power_method event_based power_worst_window_coverage tile
report_power -out_file power.rpt -power_density_tiles 100
```

This flow should be run with `power_method event_based`.

When the `power_worst_window_coverage tile` attribute is specified, you should include any one of the following parameters to specify the total number of tiles:

- `report_power -power_density_tiles_size " 15 15 "`
- `report_power -power_density_tiles_row_column " 2 2 "`
- `report_power -power_density_tiles 100`

The software generates the `power.rpt.density` report that has the power density tiles profile report with an additional section "[Intervals covering unique tiles having worst power density](#)".

Related Commands

- [report_power](#)

power_worst_window_reports

```
power_worst_window_reports {full | worst | both}
```

Default: `full`

Data_type: String

Specifies to generate static power analysis reports for the specified worst window type. The possible arguments are:

- `full`: creates the static power report for the full simulation window specified with the `-start/-end` parameters of the `read_activity_file` command.
- `worst`: creates the static power report for the worst window only.

- `both`: creates the static power report for both the full simulation and worst windows.

The worst window report is created with `.1` added in the file name, as shown below:

- Static reports for full simulation: `power.rpt.avgpwr`, `power.rpt.instpwr`
- Static reports for worst window: `power.rpt.1.avgpwr`, `power.rpt.1.instpwr`

Related Commands

- `report_power`

power_write_dynamic_currents

```
power_write_dynamic_currents {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to create dynamic current files either for full simulation duration or for user-specified duration. This attribute is only applicable to the event-based power analysis flow (`power_method event_based`).

Example:

```
set_db power_method event_based power_write_dynamic_currents true
power_enable_state_propagation true
read_activity_file sim.vcd -format vcd
report_power -out_file power.rpt
```

Related Commands

- `report_power`

power_use_physical_partition

```
power_use_physical_partition {true | false}
```

Default: `false`

Data_type: Boolean

Enables tile or area-based partitioning of a design for distributed processing. When this attribute is

specified, the top-level design is divided into physical tiles, which are then grouped to ensure that each group or partition has the same instance count. Each partition is considered and processed as a separate job.

This attribute is only applicable to the event-based power analysis flow (`power_method event_based`) in the XP mode. The `power_use_physical_partition` attribute must be specified with the `power_partition_count` attribute.

Example:

The following command specifies to enable area-based partitioning, and distribute the event-based analysis into four jobs:

```
set_db power_method event_based power_enable_xp true power_use_physical_partition true
power_partition_count 4
report_power -distribute
```

Related Commands

- `report_power`

`power_use_twf_stable_randomized_arrival_delay`

```
power_use_twf_stable_randomized_arrival_delay {none | data_only | all}
```

Default: none

Data_type: String

Specifies to annotate randomized arrival time based on net names. The parameter ensures that when there is a change in design, the net delay is not impacted. This parameter is supported in both the static and dynamic flows.

The possible arguments are:

- none - disables randomized timing window annotation.
- all - performs randomized annotation for both the clock and data nets.
- data_only - performs randomized annotation only for the data nets.

Related Commands

- `report_power`

power_use_zero_delay_vector_file

```
power_use_zero_delay_vector_file {true | false}
```

Default: false

Data_type: Boolean

Enables zero delay mode in vector-based dynamic analysis to avoid pessimism in current estimation. When set to `true`, the software reads the cycle accurate (zero delay) VCD/FSDB file and adds the timing/delay information from the Timing Analyzer or external TWF file to the VCD/FSDB file.

Related Commands

- `report_power`

power_vector_based_multithread

```
power_vector_based_multithread {true | false}
```

Default: true

Data_type: Boolean

Specifies to enable multi-threading for the VCD/FSDB based dynamic vector-based flows. To enable multi-threading in the vector-based flows, set this attribute to `true`.

Related Commands

- `report_power`

power_vector_profile_mode

```
power_vector_profile_mode {activity | event_based | power_density | transient}
```

Default: event_based

Data_type: Enum

Specifies the method used to perform vector profiling. The possible arguments are:

- `activity` - performs activity profiling. This parameter generates only the worst activity report,

and does not generate the average power report. The name of the file is as specified using the `report_power -out_file` parameter.

- `event_based` - generates the average power report. This is the average power of all events in the user-specified window. This report includes a summary based on the power components, cell type, power rail, and clock network. The worst window report is captured in a file specified with the `report_power -out_file` parameter, and the report is suffixed by `'.avgpower'`. When `-out_file` is not specified, the `vectorprofile.report.avgpower` report file is created.
- `power_density` - performs area or power density profiling. For this method, power is reported for each tile of the design, where design can be considered to be physically divided into equal area tiles. By default, a design is divided in 10x10 tiles.
- `transient` - generates the event-based average power report and current files for the worst power window.

Related Commands

- `report_power`

power_write_gui_db

```
power_write_gui_db {true | false}
```

Default: `true`

Data_type: Boolean

Specifies to create a binary database of power results in the XP mode. When set to `true`, the tool creates sliced power database file for each rail partition and saves it to the `gui.db` sub-directory under the power output directory (example: `power_dir/part_*/gui.db/rail_part_*/power.x.db` (x = 1, 2, ..., 14)). The generated power database allows you to perform GUI interactive debugging using the *Power & Rail Plots* and *Power Debug* forms.

Related Commands

- `report_power`

power_write_bbox

```
power_write_bbox {true | false}
```

Default: false

Data_type: Boolean

Specifies to save the boundary box of each instance, its placement, and rotation at the top level. When set to `true`, generates the power database with each instance boundary box, placement, and rotation information at the top level.

Related Commands

- `report_power`

power_write_db

```
power_write_db {true | false}
```

Default: false

Data_type: Boolean

A value of `true` creates a binary database of power results.

Related Commands

- `report_power`

power_write_default_pti_files

```
power_write_default_pti_files {true | false}
```

Default: true

Data_type: Boolean

For instances that are not hooked to any power/ground rail, the current for these instances are reflected in the default static current files:

- `static_default_ground_rail.ptiavg`
- `static_default_power_rail.ptiavg`

Related Commands

- `report_power`

power_write_profiling_db

```
power_write_profiling_db {true | false}
```

Default: false

Data_type: Boolean

Writes out profiling databases which can later be viewed as histograms using the SimVision interface. You can specify this parameter during both the activity and power vector profiling. The profiling databases support instance level and power/ground net histograms. It creates an output file name with the *.trn extension.

Related Commands

- `report_power`

power_write_simulation_db

```
power_write_simulation_db {true | false}
```

Default: false

Data_type: Boolean

Specifies to save functional waveforms for each instance when parsing an activity file. When set to `true`, creates SimVision waveform database (SHM database) for toggle simulation used during power analysis, which can later be viewed as histograms using the SimVision interface.

The simulation database (`shmdb.trn`) is generated in the power analysis output directory.

Related Commands

- `report_power`

power_write_static_currents

```
power_write_static_currents {true | false}
```

Default: false

Data_type: Boolean

A value of `true` generates the current data files per net.

Note: When you perform static power calculation in the dynamic power analysis flow, you can use either the `power_disable_static` or `power_write_static_currents` attribute to enable static power calculation and write static current file. If only `power_disable_static` is set to false and `power_write_static_currents` is not set, the tool automatically sets `power_write_static_currents` to true. Similarly, if `power_write_static_currents` is set to true, and `power_disable_static` is not set, the tool automatically sets `power_disable_static` to false.

Related Commands

- `report_power`

power_x_transition_factor

```
power_x_transition_factor value
```

Default: 0.5

Data_type: Double

When using VCD/FSDB, one can specify how transitions to and from X are counted. These transitions are defined as shown in [Table 1](#) below. Transitions to and from the X state will be treated as full transitions multiplied by the specified factor.

Note: 0 -> X -> 1 or 1 -> X -> 0 count as full transitions and they are not controlled by the attribute.

Related Commands

- `report_power`

power_x_count_transition_using_3_states

```
power_x_count_transition_using_3_states {true | false}
```

Default: false

Data_type: Boolean

Specifies to calculate the toggle rate using 3 states, pre-previous, previous and current states.

Related Commands

- `report_power`

power_z_transition_factor

`power_z_transition_factor value`

Default: 0.25

Data_type: Double

When using VCD/FSDB, one can specify how transitions to and from Z are counted. These transitions are defined as shown in [Table 1](#) below. Transitions to and from the Z state will be treated as full transitions multiplied by the specified factor.

Note: 0 -> XZ-> 1 or 1 -> Z-> 0 count as full transitions and they are not controlled by the `attribute`.

Related Commands

- `report_power`

Table 1: Transitions from/to X and Z values and

Transition	Definition	Default
to/from X	[0 or 1 or Z] <-> X	0.5
to/from Z	[0 or 1] <-> Z	0.25

power_zero_delay_vector_toggle_shift

`power_zero_delay_vector_toggle_shift value`

Default: ""

Data_type: String

Specifies to shift the current waveform in the zero-delay VCD flow. This attribute is useful in removing the idle period for the current waveform result with a long idle period (small flat current) before the first high current peak because of delay annotation. The attribute allows to offset the idle period from the delay values.

The default value is 0ns. The supported time units are s, ms, us, ns, and ps. The default unit is ns. You can specify a negative or a positive value. A positive value means right shift in the timeline or add more delay to the original arrival time, and a negative value means left shift in the timeline or trim the unwanted idle period.

The `power_zero_delay_vector_toggle_shift` attribute enables you to make a more realistic current waveform that can be easily aligned with the other current waveforms.

Note: This attribute is similar to the `set_twf_attribute -shift_arrival_time` parameter, with the only difference that `power_zero_delay_vector_toggle_shift` supports both positive and negative values while `-shift_arrival_time` supports only a positive value.

Related Commands

- `report_power`

Scan Control File Format

The use model for the Scan Mode Analysis flow is:

```
set_db power_method dynamic_vectorbased power_scan_control_file filename
```

The format of the scan control file is:

```
MODE = [ SHIFT | VECTORLESS]
VECTORLESS_GLOBAL_ACTIVITY = <factor> [Between 0 - 1]
<chain_name> <flop_inst> <initial_state> <master_cell_name> <scale> <mode>
<scan_in_pin> <scan_out_pin> <scan_en_pin>
```

Note that `<chain_name>` is optional, `<initial_state>` is optional in vector-less mode, and `<master_cell_name>`, `<scale>`, `<mode>`, `<scan_in_pin>`, `<scan_out_pin>`, `<scan_en_pin>` are optional and only needed for the first internal scan flop inside one macro.

Where,

- Mode specifies the mode used for scan design analysis. The two modes are: `SHIFT` and `VECTORLESS`.

- `VECTORLESS_GLOBAL_ACTIVITY` specifies the activity specification used in the `VECTORLESS` mode.
- `chain_name` is the name of the scan chain.
- `initial_state` is the initial state of all scan flops.
- `master_cell_name` is the name of the macro.
- `scale` is the scaling factor applied to the current.
- `mode` is mode name of the scan mode in the PGV view.
- `scan_in_pin` is the macro's `scan_in` pin name.
- `scan_out_pin` is the macro's `scan_out` pin name.
- `scan_en_pin` is the macro's `scan_enable` pin name.

power_intent

The root attributes in the `power_intent` category are:

- `handle_old_lib_with_signal_level`
- `power_intent_allow_back_to_back_isolation`
- `power_intent_allow_nested_default_domain`
- `power_intent_allow_power_domain_min_gap_zero`
- `power_intent_allow_shifter_voltage_mismatch`
- `power_intent_assume_iso_enable_pin_is_always_on`
- `power_intent_check_all_nets_for_domain_crossing`
- `power_intent_do_not_use_top_domain_for_port_voltage`
- `power_intent_honor_power_domain_for_domain_crossing_route`
- `power_intent_honor_power_domain_for_intra_domain_route`
- `power_intent_include_dot_lib_related_pg_pin`
- `power_intent_use_cpf_global_connect_for_always_on_buffer`
- `power_intent_use_cpf_global_connect_for_shifter`
- `power_intent_use_effective_domain_for_iso_shifter_insertion`

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `power_intent` category, use the following command:

```
get_db -category power_intent
```

handle_old_lib_with_signal_level

```
handle_old_lib_with_signal_level {true | false}
```

Default: false

Data_type: Boolean

Allows using the old-style `.lib` with `signal_level` and `power_rail` in the MSV cell identification and term voltage.

power_intent_allow_back_to_back_isolation

```
power_intent_allow_back_to_back_isolation {true | false}
```

Default: false

Data_type: Boolean

Supports back-to-back isolation insertion along one cross-domain net based on the CPF ISO rule. Set true to enable.

power_intent_allow_nested_default_domain

```
power_intent_allow_nested_default_domain {true | false}
```

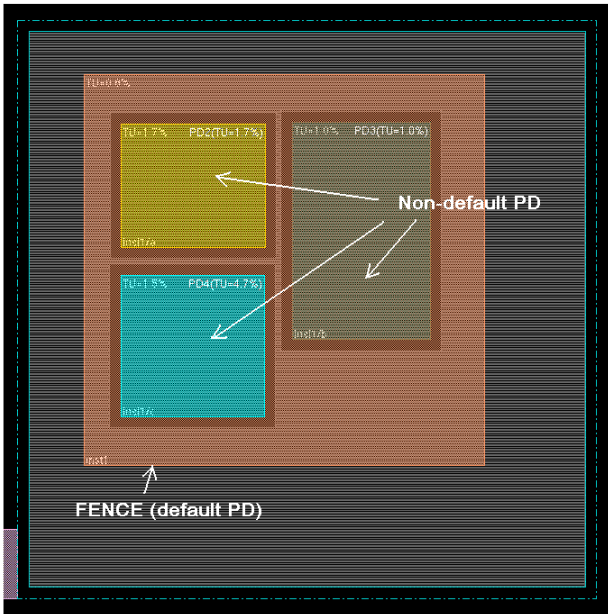
Default: true

Data_type: Boolean

Allows a non-default power domain member to be logically nested in a default power domain. This default power domain member has fence constraint.

By default, due to grouping, the fence constraint is lost for the non-default power domain member.

Set true to enable.



In the netlist: modules `HInst11` and `HInst12` are under `HInst1`; and `HInst1` is under the top.

You have the following:

```
create_power_domain -instances HInst1

create_power_domain -default -instances {HInst11 HInst12}...
```

That means `HInst11` and `HInst12` are cut from `HInst1` to default power domain. If there is a fence/region constraint for `HInst11` and `HInst12` in default power domain, then please turn this global on. Otherwise, the fence/region for `HInst11` and `HInst12` will be lost.

power_intent_allow_power_domain_min_gap_zero

```
power_intent_allow_power_domain_min_gap_zero {true | false}
```

Default: true

Data_type: Boolean

Allows users to set power domain zero minGap.

power_intent_allow_shifter_voltage_mismatch

```
power_intent_allow_shifter_voltage_mismatch {true | false}
```

Default: `true`

Data_type: Boolean

When set to `false`, it prevents insertion of level shifters by allowing voltage tolerance.

power_intent_assume_iso_enable_pin_is_always_on

```
power_intent_assume_iso_enable_pin_is_always_on {true | false}
```

Default: `true`

Data_type: Boolean

Insert ISO/LS for all the inter-domain nets using effective domain based on the CPF rules. Set `true` to enable.

power_intent_check_all_nets_for_domain_crossing

```
power_intent_check_all_nets_for_domain_crossing {true | false}
```

Default: `true`

Data_type: Boolean

Checks all the nets for domain crossings. Set `true` to enable.

power_intent_do_not_use_top_domain_for_port_voltage

```
power_intent_do_not_use_top_domain_for_port_voltage {true | false}
```

Default: `false`

Data_type: Boolean

Controls voltage using top fterm domain for IO pin voltage.

power_intent_honor_power_domain_for_domain_crossing_route

```
power_intent_honor_power_domain_for_domain_crossing_route {true | false}
```

Default: `true`

Data_type: Boolean

Routes nets belonging to more than one power domain so that the routing does not violate power domain boundaries, in addition to routing intra-power domain and global nets using the specifications of the power domains.

For example, when you specify

`power_intent_honor_power_domain_for_domain_crossing_route true`, if a net belongs to power domains PD1 and PD3, but not PD2, Early Global Route routes the net without going over the PD2 power domain boundary.

power_intent_honor_power_domain_for_intra_domain_route

```
power_intent_honor_power_domain_for_intra_domain_route {true | false}
```

Default: `true`

Data_type: Boolean

Routes intra-power domain and global nets using the specifications of the power domains. If you specify `power_intent_honor_power_domain_for_intra_domain_route true`, Early Global Route routes intra-power domain nets only within their power domains. Global nets that do not connect with a power domain do not get routed through that power domain.

power_intent_include_dot_lib_related_pg_pin

```
power_intent_include_dot_lib_related_pg_pin {true | false}
```

Default: `true`

Data_type: Boolean

Honors library related PG pins. Set `true` to enable.

power_intent_use_cpf_global_connect_for_always_on_buffer

```
power_intent_use_cpf_global_connect_for_always_on_buffer {true | false}
```

Default: `false`

Data_type: Boolean

Honors connection peps for AO buffer based on the CPF rules. Set `true` to enable.

power_intent_use_cpf_global_connect_for_shifter

```
power_intent_use_cpf_global_connect_for_shifter {true | false}
```

Default: false

Data_type: Boolean

Honors connection specs for shifters based on the CPF rules. Set `true` to enable.

power_intent_use_effective_domain_for_iso_shifter_insertion

```
power_intent_use_effective_domain_for_iso_shifter_insertion {true | false}
```

Default: true

Data_type: Boolean

Honors effective domains for ISO/LS insertions. Set `true` to enable.

precision

The root attributes in the `precision` category are:

- [enable_ui_precision](#)

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `precision` category, use the following command:

```
get_db -category precision
```

enable_ui_precision

```
enable_ui_precision {false | true}
```

Default: false

Data_type: bool, read/write

Enables UI precision reporting.

Example

```
set_db / enable_ui_precision true
```

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```
- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

program

The root attributes in the program category are:

- `cmd_file`
- `log_file`
- `logv_file`
- `program_major_version`
- `program_name`
- `program_short_name`
- `program_version`
- `selected`

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `program` category, use the following command:

```
get_db -category program
```

cmd_file

```
cmd_file fileName
```

Default: ""

Specifies the cmd log file name of the program.

log_file

```
log_file fileName
```

Default: ""

Specifies the log file name of the program.

logv_file

logv_file fileName

Default: ""

Specifies the verbose log file name of the program.

program_major_version

program_major_version

Default:

Returns the major version and one digit after the decimal point.

For example, the following command gives the major version of the software:

```
@innovus 1> get_db program_major_version  
20.1
```

program_name

program_name

Default:

Specifies the name of the program.

program_short_name

program_short_name

Default:

Specifies the short name of the program.

program_version

program_version

Default:

Specifies the version of the program.

selected

`selected list`

Default: ""

Specifies the list of currently selected objects. It can be set to a list of objects as long as they are selectable.

read_physical

The root attribute in the `read_physical` category is:

- [read_physical_extend_polygon_cell_shapes](#)
- [read_physical_check_antenna](#)
- [read_physical_check_colored_shape](#)
- [read_physical_merge_multi_port_to_single_port](#)
- [read_physical_must_join_all_ports](#)

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `read_physical` category, use the following command:

```
get_db -category read_physical
```

read_physical_extend_polygon_cell_shapes

```
read_physical_extend_polygon_cell_shapes { {layer extension_amount}... }
```

Default: ""

Data_type: string

Specifies a list of layer and a value for creating obs shape for 2D pin/obs shapes. Cells could have 2D objects, which require larger line-end spacing. You can use

`read_physical_extend_polygon_cell_shapes` to automatically add SPACING 0 OBS for routing during LEF in.

For any 2D shapes, you can extend edges that are perpendicular to the preferred routing direction of the rectilinear shape by specifying the `extension_amount` in

`read_physical_extend_polygon_cell_shapes`.

read_physical_check_antenna

```
read_physical_check_antenna {true | false}
```

Default: `true`

Data_type: `bool`

If you set `read_physical_check_antenna` to `true`, LEF or OA cells will have process antenna consistency checks done for LEF ANTENNA* data (or OA equivalent data) attached to their pins. Input pins with 0 gate-area, output pins with 0 diff-area, and inout pins with 0 gate-area or 0 diff-area values will generate warnings. If `false`, the tool will not perform consistency checking.

read_physical_check_colored_shape

```
read_physical_check_colored_shape {true | false}
```

Default: `false`

Data_type: `bool`

Specifies to check and ignore colored shapes on colorless layers while reading the LEF or DEF data. When set to `true`, the tool errors out if a colored shape is on a colorless layer. When set to `false`, the tool gives error message and ignores the color.

read_physical_merge_multi_port_to_single_port

```
read_physical_merge_multi_port_to_single_port {true | false}
```

Default: `false`

Data_type: `bool`

When set to `true`, multiple ports in a LEF PIN are merged into a single port. However, the pin should not have the `MUSTJOINALLPORTS` attribute as it specifies that all of the ports must be connected individually.

read_physical_must_join_all_ports

```
read_physical_must_join_all_ports {true | false}
```

Default: true

Data_type: bool

If set to `false`, the LEF_MUSTJOINALLPORTS property on a LEF PIN auto-creates MUSTJOIN pins for each PORT. If set to `true`, the MUSTJOIN pins are not created, but the router must connect to every port with a signal wire or signal via that physically overlaps all the ports.

Related Command

[read_physical](#)

report_obj

The root attributes in the `report_obj` category are:

- [report_obj_display_limit](#)

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `program` category, use the following command:

```
get_db -category program
```

report_obj_display_limit

```
report_obj_display_limit displayLimit
```

Default: 1000

Data_type: int, read/write

Controls the number of objects that will be displayed when using [report_obj](#). If the value is 10, 'report_obj insts' will only display 10 instance objects with all their non-default attributes.

route

route_detail_stub_routing_in_first_layerThe root attributes in the `route` category are:

- `extract_keep_fill_wires`
- `route_add_antenna_inst_prefix`
- `route_adjust_auto_via_weight`
- `route_allow_inst_overlaps`
- `route_allow_pin_as_feedthru`
- `route_antenna_cell_name`
- `route_antenna_diode_insertion`
- `route_concurrent_minimize_via_count_effort`
- `route_connect_to_bumps`
- `route_design_high_freq_bus_mixed_layer`
- `route_design_top_level_design`
- `route_detail_add_passive_fill_only_on_layers`
- `route_detail_allow_passive_fill_only_in_layers`
- `route_detail_antenna_eco_list_file`
- `route_detail_auto_stop`
- `route_detail_check_mar_on_cell_pin`
- `route_detail_end_iteration`
- `route_detail_fix_antenna`
- `route_detail_fix_antenna_on_secondary_pg_nets`
- `route_detail_fix_antenna_with_gate_array_filler_mode`
- `route_detail_merge_abutting_cut`
- `route_detail_min_length_for_spread_wire`
- `route_detail_min_length_for_widen_wire`
- `route_detail_min_slack_for_opt_wire`

- `route_detail_no_taper_in_layers`
- `route_detail_no_taper_on_output_pin`
- `route_detail_on_grid_only`
- `route_detail_post_route_litho_repair`
- `route_detail_post_route_spread_wire`
- `route_detail_post_route_swap_via`
- `route_detail_post_route_via_pillar_effort`
- `route_detail_post_route_wire_widen`
- `route_detail_post_route_wire_widen_rule`
- `route_detail_postroute_via_priority`
- `route_detail_search_and_repair`
- `route_detail_signoff_effort`
- `route_detail_stub_routing_in_first_layer`
- `route_detail_use_multi_cut_via_effort`
- `route_diode_insertion_for_clock_nets`
- `route_disable_route_rule_on_via_pillar_to_special_net_wire`
- `route_eco_ignore_existing_route`
- `route_enable_mixed_size_tsv_via`
- `route_enable_route_rule_si_limit_length`
- `route_enforce_route_rule_on_special_net_wire`
- `route_extra_via_enclosure`
- `route_fix_clock_nets`
- `route_high_freq_constraint_groups`
- `route_high_freq_match_report_file`
- `route_high_freq_num_reserved_layers`
- `route_high_freq_remove_floating_shield`
- `route_high_freq_search_repair`

- `route_high_freq_shield_trim_length`
- `route_honor_exclusive_region`
- `route_honor_power_domain`
- `route_ignore_antenna_top_cell_pin`
- `route_ignore_follow_pin_shapes`
- `route_interposer_allow_diagonal_trunk`
- `route_interposer_control_options`
- `route_interposer_interlayer_shielding_layers`
- `route_interposer_interlayer_shielding_nets`
- `route_interposer_interlayer_shielding_offsets`
- `route_interposer_interlayer_shielding_widths`
- `route_interposer_same_layer_shielding_net`
- `route_interposer_same_layer_shielding_width_spacing`
- `route_interposer_trunk_routing_layers`
- `route_interposer_trunk_routing_width_spacing`
- `route_number_fail_limit`
- `route_number_info_limit`
- `route_number_thread`
- `route_number_warning_limit`
- `route_process_node`
- `route_rc_extraction_corner`
- `route_relaxed_route_rule_spacing_to_power_ground_nets`
- `route_reserve_space_for_multi_cut`
- `route_reverse_direction`
- `route_route_clock_nets_first`
- `route_selected_net_only`
- `route_shield_crosstie_offset`

- [route_shield_length_threshold](#)
- [route_shield_report_skip_status](#)
- [route_shield_stripe_layer_range](#)
- [route_shield_tap_cell_insertion](#)
- [route_shield_tap_cell_name](#)
- [route_skip_analog](#)
- [route_strictly_honor_1d_routing](#)
- [route_strict_honor_route_rule](#)
- [route_stripe_layer_range](#)
- [route_third_party_data](#)
- [route_tieoff_to_shapes](#)
- [route_top_level_design](#)
- [route_trim_pull_back_distance_from_boundary](#)
- [route_trunk_with_cluster_target_size](#)
- [route_use_auto_via](#)
- [route_via_weight](#)
- [route_with_eco](#)
- [route_with_litho_driven](#)
- [route_with_si_driven](#)
- [route_with_timing_driven](#)
- [route_with_trim_metal](#)
- [route_with_via_in_pin](#)
- [route_with_via_only_for_block_cell_pin](#)
- [route_with_via_only_for_stdcell_pin](#)
- [route_trial_max_print_ignored_pad_nets](#)

To set the value of any root attribute, use the [set_db](#) command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `route` category, use the following command:

```
get_db -category route
```

extract_keep_fill_wires

```
extract_keep_fill_wires {true|false}
```

Default: `false`

Data_type: `bool`, read/write

When enabled, the NanoRoute Router imports metal fill wires so that RC extraction can calculate them.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_design`

route_add_antenna_inst_prefix

```
route_add_antenna_inst_prefix value
```

Default: `" "`

Data_type: `string`, read/write

Specifies the prefix that is added in the diode inst names. These diodes are added to fix antennas during routing.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:


```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_design`

route_adjust_auto_via_weight

```
route_adjust_auto_via_weight {true | false}
```

Default: true

Data_type: bool, read/write

Allows auto tuning of internal via weight. It adjusts auto via weight for via swapping.

- When set to `true`, it enables NanoRoute to auto tune internal via weight for double cut via ratio improvement.
- When set to `false`, it disables NanoRoute to auto tune internal via weight and the via weight specified using the `route_via_weight` attribute is honored for via swapping..

Note: NanoRoute does not swap the existing dcut via's to DFM via's.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_design`

route_allow_inst_overlaps

```
route_allow_inst_overlaps {true | false}
```

Default: `true`

Data_type: `bool`, `read/write`

Specifies that instance overlap check will be skipped when this attribute is set to `true`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_design`

route_allow_pin_as_feedthru

```
route_allow_pin_as_feedthru {true|TRUE|false|FALSE|none|NONE|output|input|inout, bottom  
LayerNum:topLayerNum}
```

Default: `true`

Data_type: `string`, `read/write`

Specifies that pins are allowed as feedthrough within the specified layer range.

- `true`: Specifies that input/output/inout pins can be used as feedthrough.
- `false`: Specifies that output/inout pins can be used as feedthrough.
- `none`: Specifies that no pin is allowed as feedthrough
- `output`: Specifies that output pins can be used as feedthrough.
- `input`: Specifies that input pins can be used as feedthrough.
- `inout`: Specifies that all inout pins can be used as feedthrough.
- `bottomLayerNum:topLayerNum`: Specifies the layer range that allows pins as feedthroughs.

For example, the following command allows feedthrough for M1 and M2 pins only.

```
set_db route_allow_pin_as_feedthrough "true, 1:2"
```

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_design`

route_antenna_cell_name

```
route_antenna_cell_name {"cell_name" | "list_of_cell_names"}
```

Default: ""

Data_type: string, read/write

Specifies antenna diode cells to use during postroute optimization. The cells must have the same LEF_{SITE} definition as the standard cells. Use this attribute with the `route_diode_insertion` attribute.

Note: If you do not specify a cell, the router uses a core macro of type ANTENNACELL specified in the LEF MACRO statement.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_design`

route_antenna_diode_insertion

```
route_antenna_diode_insertion {true | false}
```

Default: false

Data_type: bool, read/write

Enables or disables the insertion and placement of antenna diode cells when available placement locations exist.

By default, the NanoRoute router fixes antenna violations using layer changes (also called antenna stapling or layer hopping). However, it can also repair these violations by inserting antenna diode cells as close as possible to input gates to safely discharge accumulated charge.

When enabled, the router inserts antenna diode cells even if the filler cells are already placed. It can automatically swap filler cells with diode cells and adjust spacing if the diode cell size differs from the original filler cell.

To specify which diode cells to use, set the attribute:

```
route_antenna_cell_name.
```

Note: NanoRoute router can reclaim only those diode cells that it inserted itself. It does not reclaim user-inserted antenna diode cells.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- [route_design](#)

route_concurrent_minimize_via_count_effort

```
route_concurrent_minimize_via_count_effort value
```

Default: medium

Data_type: string, read/write

Specifies the effort level for reducing the total number of vias during routing. Use this attribute together with the `route_detail_use_multi_cut_via_effort` attribute to control the effort toward reducing the total number of vias and the effort toward increasing the ratio of double-cut vias to single-cut vias concurrently with routing.

If you specify `route_detail_use_multi_cut_via_effort` medium or high, the software sets the value of `route_concurrent_minimize_via_count_effort` to medium unless you

specify `route_concurrent_minimize_via_count_effort` yourself. To realize further savings, set `route_concurrent_minimize_via_count_effort` to high. This could further reduce the total via count up to 10 percent.

Note: If you specify the `flow_effort` attribute of the `init Category Attributes` to high, the software sets the value of `route_concurrent_minimize_via_count_effort` to high.

Note: Results might vary with your design.

Specify one of the following attributes:

- `low`: Reduces the total via count using low effort.
- `medium`: Reduces total via count by 15 percent, with negligible runtime overhead. This is the default setting.
- `high`: Reduces the total via count by 20 percent to 25 percent, with marginal runtime overhead.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `route_design`

route_connect_to_bumps

`route_connect_to_bumps {true | false}`

Default: `false`

Data_type: `bool`, read/write

Enables bump to bump routing.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_design`

route_design_high_freq_bus_mixed_layer

```
route_design_high_freq_bus_mixed_layer {false true auto only}
```

Default: false

Data_type: string, read/write

Enables mixed layer bus routing.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_detail`

route_design_top_level_design

```
route_design_top_level_design {true|false}
```

Default: false

Data_type: bool, read/write

Specifies whether the current design is treated as a top-level design during NanoRoute routing.

- When set to `true`, the router applies top-level routing behavior, accounting for design boundaries and block-level interfaces.

- When set to `false`, the design is treated as a block-level or hierarchical design.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_design`

route_detail_add_passive_fill_only_on_layers

```
route_detail_add_passive_fill_only_on_layers layers
```

Default: ""

Data_type: string, read/write

Specifies the layers where passive fill will be added.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_detail`

route_detail_allow_passive_fill_only_in_layers

```
route_detail_allow_passive_fill_only_in_layers layers
```

Default: false

Data_type: string, read/write

When specified, allows passive fill only in layers.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `route_detail`

route_detail_antenna_eco_list_file

`route_detail_antenna_eco_list_file file_name`

Default: ""

Data_type: string, read/write

Specifies the file generated during process antenna repair that contains the list of diodes that were inserted. If you insert diodes during more than one detailed routing pass, the NanoRoute router creates a separate file for each pass and gives it a unique name. For example, if you use the default filename, the router creates the following files:

- After the first detailed routing pass: `nano_ant_diode.list`
- After the second detailed routing pass: `nano_ant_diode.list1`
- After the third detailed routing pass: `nano_ant_diode.list2`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `route_design`

route_detail_auto_stop

`route_detail_auto_stop {true | false}`

Default: `true`

Data_type: `string`, `read/write`

Controls whether NanoRoute continues routing if there are many violations. In highly congested designs, NanoRoute stops if the number of violations is too high. Specify `false` to continue routing until there is no improvement.

Note: The `route_detail_auto_stop` attribute is set to `0` by default for 20nm designs and below.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `route_design`

route_detail_check_mar_on_cell_pin

`route_detail_check_mar_on_cell_pin {true|false}`

Default: `false`

Data_type: `bool`, `read/write`

Enables mar checking on used cell pins.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_design`

route_detail_end_iteration

```
route_detail_end_iteration pass_number
```

Default: auto

Data_type: string, read/write

Specifies the last pass in a detailed routing step. To run postroute optimization and repair remaining violations, including process antenna violations, set the attribute to its default value using the following command:

```
reset_db route_detail_end_iteration
```

When set to its default value, the router dynamically decides when to stop. However, when you specify `route_detail_end_iteration 0`, the end iteration will be at 0.

Note: Specifying a higher value than 20 for `route_detail_end_iteration` generally does not improve results.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_design`

route_detail_fix_antenna

```
route_detail_fix_antenna {auto true false}
```

Default: `auto`

Data_type: enum {`auto` `true` `false`}, `read/write`

When enabled, the tool automatically attempts to fix process antenna violations by either jumping between metal layers (layer hopping) or inserting diodes, if a diode list is provided. The tool ensures that all antenna violations are resolved without introducing new DRC errors. To achieve this, the router may need to make multiple passes, iterating until all violations are successfully repaired.

Note: You must set this option to `true` explicitly. The default value is `auto`, which behaves similar to `true` if you have antenna information defined in the LEF. However, in this mode, the router only fixes antenna violations on data nets during `route_design`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `route_design`

route_detail_fix_antenna_on_secondary_pg_nets

`route_detail_fix_antenna_on_secondary_pg_nets {true|false}`

Default: `false`

Data_type: `bool`, `read/write`

Fixes the antenna violation on secondary pg nets.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `route_design`

route_detail_fix_antenna_with_gate_array_filler_mode

`route_detail_fix_antenna_with_gate_array_filler_mode {true | false}`

Default: `false`

Data_type: `bool`, [read/write](#)

When enabled, the NanoRoute router detects the GA filler flow and uses the GA filler diode cells for antenna violation fixing.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

Related Commands

- `route_design`

route_detail_merge_abutting_cut

`route_detail_merge_abutting_cut {true|false}`

Default: `true`

Data_type: `bool`, [read/write](#)

Allows you to enable or disable the capability to merge cell cut shapes. When enabled, it merges two abutting cell cut shapes into a new one.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_design`

route_detail_min_length_for_spread_wire

```
route_detail_min_length_for_spread_wire { layer1 length1 ... layerN lengthN }
```

Default: 0

Data_type: string, read/write

Specifies the minimum length in microns for a wire segment of a net before the necessity for a jog to avoid OPC issues. Spreading wires further apart (creating jogs in the wires) helps avoid design rule violations, signal integrity problems, and yield loss caused by shorts in the wires.

This parameter supports the following values:

- `layer`: Specifies the metal layer. The supported metal layer name is Mx, they are Metal1, Metal2, Metal3, etc.
- `length`: Specifies the minimum length, in microns. The length value can be a floating number, default, or skip.
 - The default value is 0, it means wire spreading will determine the threshold automatically.
 - The skip value means there is no wire spreading on this layer.

Note: You should specify the value for each metal layer. To get the number of metal layers used in the design, use db command (dbGet head.layers.name).

Note: Turn wire spreading on or off with `route_detail_post_route_spread_wire`

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

For example, when you specify the following command, no wire spreading is done on layer

Metal1. For layer Metal2, the wire spreading value is set to 0 signifying that wire spreading will determine the threshold automatically. Similarly, for layers Metal3, Metal4, Metal5, and Metal6 the wire spreading length is set to 2.0.

```
set_db route_detail_min_length_for_spread_wire {Metal1 skip Metal2 0 Metal3 2.0 Metal4  
2.0 Metal5 2.0 Metal6 2.0 }
```

Related Commands

- `route_design`

route_detail_min_length_for_widen_wire

```
route_detail_min_length_for_widen_wire value
```

Default: 1.0

Data_type: double, read/write

Specifies the minimum length in microns for a wire segment of a net before the segment is a candidate for widening. Wires are widened by an amount defined by a nondefault rule. They are not widened unless they meeting the following criteria:

- Timing is not adversely affected
- Routing resources are available
- No overflow is created
- No design rule or process antenna violations are created

Widening wires helps avoid design rule violations, signal integrity problems, and yield loss.

Note: To enable this attribute, you must also include a wide wire nondefault rule in the LEF file and specify the `route_detail_post_route_widen_wire_rule 1` attribute.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```
- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_design`

route_detail_min_slack_for_opt_wire

`route_detail_min_slack_for_opt_wire value`

Default: 0.0

Data_type: double, read/write

Specifies the minimum slack on pins associated with nets that are candidates for wire spreading or wire widening. Using this parameter reduces the timing impact of wire spreading, wire widening, and via swapping, and helps reduce congestion and the yield loss due to shorts and opens.

Note: To enable this attribute, you must also specify `route_with_timing_driven true`.

Wire spreading and wire widening use the minimum slack value differently. While wire spreading only works for timing critical nets where the worst setup slack is less than the minimum slack value, wire widening only works for non-timing critical nets where the worst setup slack is greater than the minimum slack value.

You can use the `route_detail_min_slack_for_opt_wire` attribute to reduce the timing closure effort during post-route ECO. NanoRoute will replace via to multi-cut via on non-critical nets, if a violation does not occur.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

`set_db attribute_name value`

- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

Related Commands

- `route_design`

route_detail_no_taper_in_layers

`route_detail_no_taper_in_layers "bottom_layer_number:top_layer_number"`

Default: " "

Data_type: string, read/write

Prohibits the detailed router from tapering in the specified layer range, creating non-default-width wires in that range.

`bottom_layer_number:top_layer_number` signifies the layer range format.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- [route_design](#)

route_detail_no_taper_on_output_pin

```
route_detail_no_taper_on_output_pin {true|false|auto}
```

Default: false

Data_type: enum {false true auto}, read/write

Prohibits the detailed router from tapering at standard cells, macro cells, and block output pins.

Note: The default value of `route_detail_no_taper_on_output_pin` is `auto` for 32nm and below nodes.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- [route_design](#)

route_detail_on_grid_only

route_detail_on_grid_only none | false | true | all | {[wire [metall:metalh]] | [via [metall:metalh]]}

Default: none

Data_type: string, read/write

Controls the routing of the objects, wires, and vias in relation to the user-defined routing tracks.

- On grid wire means that the center of the wire is on the preferred routing track.
- On grid via means that the center of the via is on the intersection of preferred routing tracks.

Note: A preferred routing track is the track which has the same direction as that of the metal routing direction defined in the LAYER section in LEF.

- When there are off-grid objects on fully-routed data, NanoRoute skips those nets and issues a warning message.
- When there are fixed off-grid objects, NanoRoute skips those nets and issues a warning message.
- When there are non-fixed off-grid objects, NanoRoute deletes those nets and issues a warning message.

Specify one of the following arguments:

- **all:** Specifies that all routing objects (vias and wires) have to be on the preferred and non-preferred routing tracks. It forces all routing objects to be on grid.
- **none:** Specifies that the routing objects (vias and wires) can be off-grid. It does not force routing objects to be on grid. They can go off-grid depending on other restrictions.
- **via [metall:metalh]:** Specifies that the center of the vias must be at the intersection of preferred routing tracks. If `metall` and `metalh` are specified, all the vias between `metall` and `metalh` must be at the intersection of preferred routing tracks. For example, `via [1:3]` means all `via12` must be at the intersection of preferred routing tracks of layer `Metal1` and `Metal2` and all `via23` must be at the intersection of preferred routing tracks of layer `Metal2` and `Metal3`.
- **wire [metall:metalh]:** Specifies that the center of the wire must be on preferred routing tracks. When a wire makes a turn on the non-preferred direction, the center of the wire might not fall on any routing tracks. If `metall` and `metalh` are specified, all the center of the wires in between those metal layers `m1` and `m2` must be on the preferred routing tracks.

For example, wire [1:3] means the center of the Metal1, Metal2, and Metal3 wires must be on the preferred routing tracks of Metal1, Metal2, and Metal3.

- `false`: For backward compatibility purpose, the `route_detail_on_grid_only` argument also takes the value of `false`. This is equivalent to `'none'`.
- `true`: For backward compatibility purpose, the `route_detail_on_grid_only` argument also takes the value of `true`. This is equivalent to `'all'`.

Note: Block pins can be accessed by planar off-track wires.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `route_design`

route_detail_post_route_litho_repair

`route_detail_post_route_litho_repair {true | false}`

Default: `false`

Data_type: `bool`, `read/write`

Corrects bridging, necking, and line-end shortening problems caused by lithography. Takes the severity of the problems into account and fixes the high-severity problems first. Prior to running the router with this parameter, import a hotspot interface format (HIF) file with the `read_markers` command.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `route_design`

route_detail_post_route_spread_wire

`route_detail_post_route_spread_wire` {auto| true | false | 0 | 0.25| 0.5 | 1 }

Default: auto

Data_type: string, read/write

Spreads routing, post route, by the percentage of the track. The router moves the wires without moving the vias. Specifying false for this attribute signifies no spreading, while specifying true signifies 0.5 / 1 track spreading by default for all nodes. The use of `true` and `false` is not needed as a value greater than 0 and less than equal to 1 ($0 < x \leq 1$) activates the spreading. Specifying this attribute will not increase the overall routing runtime much but helps avoid yield loss caused by shorts and improve the timing as well.

Specify one of the following:

- `auto`: Spreading is on if the timing effort is set to high (`flow_effort` attribute to high in the `init` Category Attributes). Otherwise it is off.
- `true`: 0.5 / 1 track spreading. Set `route_detail_post_route_spread_wire` to `true` if you want to spread.
- `false`: no spreading
- `0.25`: 0.25 / 0.5 / 1 track spreading
- `0.5`: 0.5 / 1 track spreading
- `1`: 1 track spreading

Note: To turn off this feature, set `route_detail_post_route_spread_wire` to `false` explicitly.

To call wire spreading explicitly, set `route_detail_post_route_spread_wire` to `true` and call `route_design -wire_opt`.

Post route wire spreading automatically avoids fractional spreading for DPT designs. If `route_detail_post_route_spread_wire` is enabled in 20nm designs and below, only 1 track wire spreading is done on DPT layers.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_design`

route_detail_post_route_swap_via

```
route_detail_post_route_swap_via {true | multicut | false | none}
```

Default: false

Data_type: string, read/write

Swaps single-cut vias for multiple-cut vias or reverses swapping on critical nets in a fully routed design, so that multiple-cut vias are swapped for single-cut vias on those nets. It swaps single-cut vias to multi-cut vias including vias of non-default nets. However, while swapping multi-cut vias to single cut vias, NanoRoute does not swap the vias of non-default rule nets. In timing-driven routing, does not swap vias if doing so would negatively impact timing.

A net with a pin on a critical path is considered a critical net if the pin slack, plus the margin defined by the `route_detail_min_slack_for_opt_wire` attribute, is less than or equal to 0. Use this attribute with the `route_design` command. You do not have to specify a value for this parameter when you run the following command, because the software does so automatically:

```
route_design -via_opt
```

Note: This attribute is not enabled concurrently with postroute wire widening or wire spreading. In addition, the software does not perform via swapping if `route_detail_post_route_litho_repair true` is specified.

Specify one of the following:

- `true`: Swaps all nets. The nets with attributes will get double cut and DFM precedence for others. Its weight can be specified with the `route_via_weight` attribute.
- `multicut`: All nets will be swapped with dcut precedence.
- `none`: Does not swap vias.
- `false`: Only nets with attributes are swapped.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- [route_design](#)

route_detail_post_route_via_pillar_effort

```
route_detail_post_route_via_pillar_effort {none | low | medium | high}
```

Default: high

Data_type: string, [read/write](#)

Specifies the effort levels for adding via pillars. Specify one of the following:

- `none`: Adds [via pillars only](#) during routing that are required for EM constraints.
- `low`: Adds via pillars during timing-driven global routing where they help in timing and do no impact congestion
- `medium`: Adds via pillars more aggressively during routing.
- `high`: Adds via pillars throughout routing to maximize via pillar impact on timing and area.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- [route_design](#)

route_detail_post_route_wire_widen

```
route_detail_post_route_wire_widen {widen | shrink | none}
```

Default: none

Data_type: string, read/write

Applies the rule specified by the `route_detail_post_route_wire_widen_rule` attribute to widen or shrink wires. Use this parameter with the `route_design` command.

Note: This attribute is not enabled concurrently with postroute via swapping.

Note: Wire widening is not performed on layer Metal1.

Specify one of the following:

- `widen`: During timing-driven routing, widens only non-critical nets; otherwise widens as many nets as possible that are covered by the nondefault rule for widening wires.
- `shrink`: During timing-driven routing, shrinks only critical nets; otherwise shrinks as many nets as possible that are covered by the nondefault rule for widening wires.
- `none`: Turns off this attribute, so the router does not widen or shrink wires.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```
- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_design`

route_detail_post_route_wire_widen_rule

```
route_detail_post_route_wire_widen_rule rule_name
```

Default: ""

Data_type: string, read/write

Specifies the non-default rule for widening wires. The rule is specified in the LEF file. Use this

attribute with the `route_design` command.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_design`

route_detail_postroute_via_priority

```
route_detail_postroute_via_priority {auto | allNets | criticalNetsfirst |  
nonCriticalNetOnly}
```

Default: auto

Data_type: string, read/write

Sets the net priority for postRoute via swapping based on timing.

- `auto`: Specifies the current behavior of the software, which is to detect the node and timing files to determine whether to do `criticalNetsFirst` or `nonCriticalNetOnly`. This is the default behavior.
- `allNets`: Swaps single-cut vias for multiple-cut vias on all nets.
- `criticalNetsFirst`: Specifies that swapping should be preformed first on timing critical nets and then on other nets. This option is recommended for process nodes below 32nm.
- `nonCriticalNetonly`: Specifies that swapping should not be performed on timing critical nets. This option is recommended for 32 nm and above process nodes.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_design`

route_detail_search_and_repair

`route_detail_search_and_repair {true | false}`

Default: `true`

Data_type: `bool`, `read/write`

Runs a search-and-repair step after the initial detailed routing (end iteration 20).

Tip: Specify `0` for this attribute and run detailed routing to determine quickly if the design is routable.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

Related Commands

- `route_design`

route_detail_signoff_effort

`route_detail_signoff_effort {high | medium | low | auto | n}`

Default: `high`

Data_type: `string`, `read/write`

Specifies when to skip fixing violations related to fixed objects.

- `high`: In this mode, the software will ignore violations related to fixed objects after 20 detail route iterations and will continue with post-route operations.
- `medium`: When set to `medium`, the software will ignore any violations related to fixed objects after detail route iteration number 15.

- **low:** When set to `low`, the software will ignore any violations related to fixed objects after detail route iteration number 5 and it will also skip post-route operations.
- **auto:** When set to `auto`, the software will automatically detect when to ignore violations related to fixed objects and it will also skip post-route operations.
- **n:** You can also specify the number of iterations or "n". When the iteration number is specified, the software will ignore violations related to fixed objects after the specified number of detail route iteration and will also skip post-route operations.

Note: n must be between 3 and 20.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `route_design`

route_detail_stub_routing_in_first_layer

`route_detail_stub_routing_in_first_layer {true | false}`

Default: `false`

Data_type: string, read/write

Specifies that minimum routing should be done on the first routing layer defined in the LEF.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `route_design`

route_detail_use_multi_cut_via_effort

`route_detail_use_multi_cut_via_effort {low | medium | high}`

Default: `low`

Data_type: string, read/write

Specifies the effort level toward increasing the ratio of double-cut vias to single-cut vias concurrently with routing. Increasing the effort level increases the double-cut via ratio and decreases the total number of vias in the design, which means that it also decreases the number of single-cut vias. The lower the number of single-cut vias, the better the yield will be. After routing with this parameter specified, you can add more double-cut vias or larger overhang vias by using the `route_design` command.

Specify one of the following:

- `low`: Specifies normal routing, so the router uses single-cut vias only.
- `medium`: Balances the need for a high double-cut viaration with run time and congestion. Specifying this parameter increases the run time somewhat compared with the low parameter. The router inserts some double-cut vias, although not as many as if the high attribute were specified.
- `high`: Specifies the highest yield possible for vias, as the router puts its best effort toward achieving the highest possible double-cut via ratio at the expense of run time and congestion.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `route_design`

route_diode_insertion_for_clock_nets

`route_diode_insertion_for_clock_nets {true | false}`

Default: `false`

Data_type: `bool`, [read/write](#)

Inserts diodes to repair process antenna violations on clock nets that are in the regular net section of the DEF file.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

`set_db attribute_name value`

- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

Related Commands

- [route_design](#)

route_disable_route_rule_on_via_pillar_to_special_net_wire

`route_disable_route_rule_on_via_pillar_to_special_net_wire value`

Default: `false`

Data_type: `string`, [read/write](#)

Disables the non-default rule spacing checks between via pillars and PG nets while continuing to honor the non-default spacing rules for via pillars to any signal nets, clock nets, and wires/cuts.

Note: This option is not honored by the [check_drc](#) command. You must filter these type of violations in the browser.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

`set_db attribute_name value`

- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

Related Commands

- `route_design`

route_eco_ignore_existing_route

`route_eco_ignore_existing_route value`

Default: ""

Data_type: string, read/write

When specified, the selected net will be routed with highest priority by ignoring existing routes in post-route mode.

- `all`: Ignores all other nets in the design that are not selected, including clock, secondary PG, and signal nets.
- `signal`: Ignores only other signal nets that are not selected, but honors clock and secondary PG nets.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

For example, the following commands select and route a list of nets by ignoring only other signal nets, and considers secondary PG and clock nets during routing. After that, a second `route_eco` is run to fix DRC violations:

```
select_obj [get_db nets {net1 net2 net3}]
set_db route_selected_net_only true
set_db route_eco_ignore_existing_route signal
route_eco

reset_db -category route
reset_db route_selected_net_only
reset_db route_eco_ignore_existing_route
verify_drc -limit 0
route_eco
```

Related Commands

- `route_design`

route_enable_mixed_size_tsv_via

`route_enable_mixed_size_tsv_via {true|false}`

Default: `false`

Data_type: `bool`, `read/write`

When enabled, this option allows the use of mixed-size TSV (Through-Silicon Via) vias during routing.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

Related Commands

- `route_design`

route_enable_route_rule_si_limit_length

`route_enable_route_rule_si_limit_length {true | false}`

Default: `false`

Data_type: `string`, `read/write`

Controls how NanoRoute evaluates PRL-dependent non-default rule (NDR) metal spacing checks. When enabled, the router uses the minimum-area-rule (MAR) derived wire length as the effective parallel run length (PRL) threshold when checking NDR spacing violations. This setting applies only to NDR-based spacing checks.

Note: This attribute does not enable, disable, or modify signal-integrity (SI) analysis or SI-driven routing.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_design`

route_enforce_route_rule_on_special_net_wire

```
route_enforce_route_rule_on_special_net_wire {false | true | special_net_name_list}
```

Default: false

Data_type: string, read/write

Specifies that non-default rule will be enforced on all special wire segments of specified nets.

Specify one of the following parameters:

- `false`: When specified, the non-default rule is not enforced on any wire segment of any special net. It is the default value.
- `true`: When specified, the non-default rule is enforced on all wire segments of all special nets (with non default rules attached).
- `specified_special_net_list`: When specified, the non-default rule is enforced on all wire segments of those specified special nets.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_design`

route_extra_via_enclosure

`route_extra_via_enclosure distance`

Default: 0.0

Data_type: double, [read/write](#)

Specifies an extra via enclosure to use when connecting to block pins and special wires. To center vias for via access, specify a value that is equal to one-half the width of the largest pin or wire. An extra enclosure of this size forces NanoRoute to connect tie-high and tie-low nets to the center of power routes.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

`set_db attribute_name value`

- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

Related Commands

- [route_design](#)

route_fix_clock_nets

`route_fix_clock_nets {true | false}`

Default: false

Data_type: bool, [read/write](#)

Specifies whether to maintain the current status of clock nets when the `route_design` command runs or change the status to ROUTED.

- If you specify true for this parameter, `route_design` does not change the status of clock nets.
- If you do not specify this parameter (or you specify false for this parameter), `route_design` changes the status of clock nets changes to ROUTED.

Note: The parameter applies to the `route_design` command only.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_design`

route_high_freq_constraint_groups

```
route_high_freq_constraint_groups {order net match bus pair shield}
```

Default: " "

Data_type: string, read/write

Routes only the specified constraint groups. The routing constraint groups can be - `net match bus pair shield`.

- `net`: Constraint group for a simple net.
- `match`: Constraint group for matched nets.
- `bus`: Constraint group for bus nets.
- `pair` - Constraint group for differential pair nets.
- `shield` - Constraint group for shielding a single net.

Note: You can add "order" in front to control the route order. If order is present, the NanoRoute high frequency router will honor the order, if not, the router will determine the order.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_design`

route_high_freq_match_report_file

`route_high_freq_match_report_file filename`

Default: " "

Data_type: string, read/write

Specifies the name of the match report file.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

`set_db attribute_name value`

- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

Related Commands

- [route_design](#)

route_high_freq_num_reserved_layers

`route_high_freq_num_reserved_layers value`

Default: 1

Data_type: int, read/write

Specifies the number of metal layers to reserve for high frequency routing above the standard cell area so that routing has more resources to access standard cell pins.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

`set_db attribute_name value`

- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

Related Commands

- [route_design](#)

route_high_freq_remove_floating_shield

```
route_high_freq_remove_floating_shield {true|false}
```

Default: false

Data_type: bool, read/write

Removes floating shield segments for high frequency nets.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `route_design`

route_high_freq_search_repair

```
route_high_freq_search_repair {auto | false | true | only}
```

Default: auto

Data_type: string, read/write

Runs search and repair to remove violations.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `route_design`

route_high_freq_shield_trim_length

`route_high_freq_shield_trim_length value`

Default: 0.0

Data_type: double, [read/write](#)

Specifies the shielding wire length. All shielding wires longer than the specified value are preserved by NanoRoute for high frequency nets.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

`set_db attribute_name value`

- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

Related Commands

- [route_design](#)

route_honor_exclusive_region

`route_honor_exclusive_region {true | false}`

Default: true

Data_type: bool, [read/write](#)

When set to true, it enables the NanoRoute router to honor exclusive region routing.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

`set_db attribute_name value`

- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

Related Commands

- [route_design](#)

route_honor_power_domain

`route_honor_power_domain {true | false}`

Default: `false`

Data_type: `bool`, [read/write](#)

Routes nets without crossing power domains.

- If a net connects pins that all belong to one power domain, the software tries to route the net within that power domain. If completing the routing is not possible without breaking the rules, due to congestion for example, completes the routing and issues a warning message.
- If a net connects pins that belong to different power domains, the software tries to route the net within the involved power domains, without crossing over into other power domains that do not include any of the pins in the net. If completing the routing is not possible without breaking this rule, completes the routing and issues a warning message.

Note: You must specify a power domain before using this attribute.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- [route_design](#)

route_ignore_antenna_top_cell_pin

`route_ignore_antenna_top_cell_pin {true | false}`

Default: `true`

Data_type: `bool`, [read/write](#)

If set to true, ignores any process antenna data from external I/O pins (e.g. DEF PIN ANTENNA* data), so only the routing and antenna data from cells inside the design are used to find and repair local antenna violations. This is most commonly used with methodologies that insert diodes at the

boundary pins (either inside the block or at the next level up), or in bottom-up flows where the block is routed, updated antenna data is created with `check_antenna`, and then passed up to the top-level in an updated block abstract for the top-level to resolve any violations with added diodes or layer-hops.

If set to 0, the external IO antenna data is used to find and repair violations. This option will add protection diodes if needed to any external output/inout IOs that have antenna data that requires protection (e.g. have gate area), and will include the effects of external antenna data when finding and fixing local antenna violations. This is used when the user has “antenna budgets” added to the external IOs that estimate the worst-case top-level routing and gate-areas that needs protection inside this block. Currently these types of antenna budgets can only be added with `defln` or `oaln`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_design`

route_ignore_follow_pin_shapes

```
route_ignore_follow_pin_shapes {true | false}
```

Default: `false`

Data_type: `bool`, `read/write`

Specifies that follow pin via shapes will be ignored when this attribute is set to true.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_design`

route_interposer_allow_diagonal_trunk

`route_interposer_allow_diagonal_trunk {auto | true | false}`

Default: " "

Data_type: string, read/write

Allows you to control the trunk patterns in interposer routing.

- `auto`: Allows the routing to decide the diagonal or straight trunk patterns based on the routing resources.
- `true`: Enables diagonal trunk in interposer routing.
- `false`: Disables diagonal trunk in interposer routing.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `route_design`

route_interposer_control_options

`route_interposer_control_options value`

Default: " "

Data_type: string, read/write

Specifies the interposer control options.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_design`

route_interposer_interlayer_shielding_layers

```
route_interposer_interlayer_shielding_layers {layer_name  
direction layer_name1:layer_name2 direction ...}
```

Default: " "

Data_type: string, read/write

Specifies the signal layers that require shielding along with the direction - top, bottom, both. By default, all routing layers (used by current routing command) are shielded from bottom.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

For example, the following command specifies that the M2/M3/M4 signal layers will be shielded from the top, M7 from the bottom, and M9 will be shielded from both sides:

```
set_db route_interposer_interlayer_shielding_layers {M2:M4 top M7 bottom M9 both}
```

Related Commands

- `route_design`

route_interposer_interlayer_shielding_nets

```
route_interposer_interlayer_shielding_nets {layer_name1:layer_nameN net_name ...}
```

Default: " "

Data_type: string, read/write

Specifies the inter-layer shielding nets.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

For example, the following command specifies that the M3/M4/M5/M6 shielding net name is VSS and the M8/M9/M10 shielding net name is VDD:

```
set_db route_interposer_interlayer_shielding_nets {M3:M6 VSS M8:M10 VDD}
```

Related Commands

- [route_design](#)

route_interposer_interlayer_shielding_offsets

```
route_interposer_interlayer_shielding_offsets {layer_name  
offset layer_name1:layer_name2 offset ...}
```

Default: " "

Data_type: string, read/write

Specifies the center-to-center offset values between the shielding and signal wires. By default, all routing layers (used by current routing command) have an offset value of 0.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

For example, the following command specifies that the offset value is 1.1 um between the M1 shielding wire and signal wire and the offset value is 2.2 um between M2:M3 shielding wire and signal wire:


```
set_db route_interposer_interlayer_shielding_offsets {M1 1.1 M2:M3 2.2}
```

Related Commands

- [route_design](#)

route_interposer_interlayer_shielding_widths

```
route_interposer_interlayer_shielding_widths {layer_name  
width layer_name1:layer_name2 width ...}
```

Default: " "

Data_type: string, [read/write](#)

Specifies the inter-layer shielding widths. By default, all routing layers (used by current routing command) are shielded by the default width wires.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

For example, the following command specifies that the shielding M1 wires width is 1.1 um, shielding M2/M3 wires width is 2.2 um:

```
set_db route_interposer_interlayer_shielding_widths {M1 1.1 M2:M3 2.2}
```

Related Commands

- [route_design](#)

route_interposer_same_layer_shielding_net

```
route_interposer_same_layer_shielding_net {net_name}
```

Default: " "

Data_type: string, [read/write](#)

Specifies the same-layer shielding nets.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

For example, the following commands specifies that the M1/M3/M5 shielding net name is VSS:

```
set_db route_interposer_trunk_routing_layers {M1 M3 M5}
```

```
set_db route_interposer_same_layer_shielding_net {VSS}
```

Related Commands

- `route_design`

route_interposer_same_layer_shielding_width_spacing

```
route_interposer_same_layer_shielding_width_spacing {width spacing}
```

Default: " "

Data_type: string, read/write

Specifies the same-layer shielding width and spacing.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

For example, the following command specifies that the shielding width is 0.6 and spacing is 1.2:

```
set_db route_interposer_same_layer_shielding_width_spacing {0.6 1.2}
```

Related Commands

- `route_design`

route_interposer_trunk_routing_layers

`route_interposer_trunk_routing_layers {layers}`

Default: " "

Data_type: string, read/write

Specifies the trunk routing layers.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

`set_db attribute_name value`

- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

For examples, the following command sets layers M1, M3, and M5 as the interposer trunk routing layers.

```
set_db route_interposer_trunk_routing_layers {M1 M3 M5}
```

Related Commands

- [route_design](#)

route_interposer_trunk_routing_width_spacing

`route_interposer_trunk_routing_width_spacing {width spacing}`

Default: " "

Data_type: string, read/write

Specifies the trunk routing signal width and spacing.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

`set_db attribute_name value`

- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

For example, the following command sets the trunk routing signal width to 1 and spacing to 0.7.

```
set_db route_interposer_trunk_routing_width_spacing {1.0 0.7}
```

Related Commands

- `route_design`

route_number_fail_limit

```
route_number_fail_limit integer
```

Default: 0

Data_type: int, read/write

Specifies the maximum number of command failures allowed.

- In interactive mode, when a command fails, the router allows you to continue with the next command.
- In batch mode, a failed command normally stops the script. However, increasing the fail limit allows the script to continue executing even after encountering failures.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```
- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_design`

route_number_info_limit

```
route_number_info_limit integer
```

Default: 0

Data_type: int, read/write

Specifies the maximum number of informational messages that NanoRoute is allowed to generate.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- [route_design](#)

route_number_thread

```
route_number_thread numberProcessors
```

Default: 1

Data_type: int, [read/write](#)

Note: This attribute is included for backward compatibility. Cadence recommends that you use the `set_multi_cpu_usage` command to specify the number of threads in multiple-CPU processing.

Specifies the maximum number of threads to use in multi-threading. Generally, using more threads decreases the run time. Specify this attribute before running any routing commands. There is no absolute limit to the number of threads the router can use, however, Cadence recommends that you do not specify more threads than the number of CPUs available. The router does not necessarily use the number of threads you specify – it determines the optimal number up to the value you specify. A small design generally uses fewer threads than a large design.

Note: To enable a value greater than 1 for this attribute, you must have licenses available for the additional threads.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_design`

route_number_warning_limit

`route_number_warning_limit integer`

Default: 0

Data_type: int, read/write

Specifies the maximum number of warning messages that can be reported to the log file for a single command. Once this limit is reached, the router continues processing but stops reporting additional warnings for that command.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
- To get the current value of any root attribute, use the `get_db` command:

`set_db attribute_name value`

`get_db attribute_name`

Related Commands

- `route_design`

route_process_node

`route_process_node value`

Default: " "

Data_type: string, read/write

Specifies the process node. This attribute is an umbrella option to set all process-related routing settings.

Note: It supports a wide variety of process technologies across multiple semiconductor foundries. The supported values are S2, N2, N2P, N2X, S3, N3, N3E, S4, I5, N6, C7, G7, I7, N7, S7, I10, N10, SU10, S11, A12, G12, N12, A14, G14, A16, and N22.

Note: Some advanced process nodes include minimum-width pins that are smaller than the default

widths defined in the LEF. In such cases, the NanoRoute Router automatically tapers the width from the LEF default to the pin's minimum width.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- [route_design](#)

route_rc_extraction_corner

```
route_rc_extraction_corner value
```

Default: " "

Data_type: string, read/write

Specifies the RC extraction corner to use in routing. This helps in better [resistance calculation](#).

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- [route_design](#)

route_relaxed_route_rule_spacing_to_power_ground_nets

```
route_relaxed_route_rule_spacing_to_power_ground_nets {layer:spacing layer:spacing}
```

Default: none

Data_type: string, read/write

Relaxes the spacing requirement from NDR spacing for the layers. *Layer_spacing* signifies a distance in microns (per layer) of how close any net with NDR can get to the power/ground stripes.

For example,

```
set_db route_strict_honor_route_rule true
set_db route_relaxed_route_rule_spacing_to_power_ground_nets {metal1:0.04 metal2:0.04
metal3:0.04 metal4:0.04 metal5:0.04}
```

It only works for PG usage net and relaxes the NDR requirement to the specified spacing. The specified spacing should be greater than the `min_spacing` and less than the `hard_ndr_spacing`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `route_design`

route_reserve_space_for_multi_cut

```
route_reserve_space_for_multi_cut {true | false}
```

Default: false

Data_type: bool, read/write

Reserves space to insert multicut vias in postroute stage. This option has to be set before routing. After routing with this parameter specified, you can add double-cut vias or larger overhang vias by using the `route_design -via_opt` command.

Note: `route_reserve_space_for_multi_cut` is `true` by default for 32nm designs and below.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_design`

route_reverse_direction

```
route_reverse_direction "(x1 y1 x2 y2) (x3 y3 x4 y4) ...."
```

Default: ""

Data_type: string, read/write

Reverses the preferred routing direction in the specified areas. You can specify one or more areas.

Note: To route a specified net in the non preferred direction in the specified area, you must also use the following commands:

```
select_obj [get_db nets net1]  
route_selected_net_only 1
```

You can also use this parameter to increase the utilization of routing channels where some routing layers are blocked by power routing. For example, in a narrow routing channel, the long way usually has more traffic than the crosswise direction. If the preferred direction for power routing is the long way, the power routing might use too many routing resources and cause congestion. Reversing the preferred routing direction in this channel, that is, using routing layers without power wires for long way routing, might increase available routing resources.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_design`

route_route_clock_nets_first

`route_route_clock_nets_first {true | false}`

Default: `true`

Data_type: `bool`, [read/write](#)

Detects whether clock nets require ECO routing after an `route_design` super command checks whether the existing clock nets were fully or partially routed with the NanoRoute router and reroutes them before routing the remaining nets. If the existing clock nets were not routed with NanoRoute, `route_design` skips routing on those nets.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- [route_design](#)

route_selected_net_only

`route_selected_net_only {true | false}`

Default: `false`

Data_type: `bool`, [read/write](#)

Specifies whether NanoRoute routes all nets at once or routes selected nets only. To route critical nets as short as possible, select the critical nets and set this attribute to `true`. NanoRoute does the following:

- Removes incomplete nets unless they are marked + `FIXED` or have a `-skip_routing net` attribute.
- Routes the remaining selected nets.

Note: To select a net, use the `select_obj` command or click the net in the design display window.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_design`

route_shield_crosstie_offset

```
route_shield_crosstie_offset {layerName:numTrack1 layerName2:numTrack2...}
```

Default: " "

Data_type: string, read/write

Specifies the offset in terms of number of tracks for adding crossties. It enables you to control the starting track for adding crossties. Default is 0 offset for all layers.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_design`

route_shield_length_threshold

```
route_shield_length_threshold value
```

Default: -1.0

Data_type: double, read/write

Specifies the minimum route segment length below which shielding is not required.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_design`

route_shield_report_skip_status

```
route_shield_report_skip_status {true|false}
```

Default: `false`

Data_type: `bool`, read/write

Generates a report file containing a list of nets that have been skipped for shielding.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_design`

route_shield_stripe_layer_range

```
route_shield_stripe_layer_range "bottom_layer_name1:top_layer_name1  
bottom_layer_name2:top_layer_name2"
```

Default: `" "`

Data_type: `string`

Specifies the target layer range of stripes for tie net connection. It accepts different top and bottom layer range for both frontside and backside.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- [route_design](#)

route_shield_tap_cell_insertion

```
route_shield_tap_cell_insertion {true | false}
```

Default: false

Data_type: bool

When set to true, it enables the NanoRoute router to insert tap cells for shielding.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- [route_design](#)

route_shield_tap_cell_name

```
route_shield_tap_cell_name string
```

Default: " "

Data_type: string

Specifies the tap cell names for shielding.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_design`

route_skip_analog

```
route_skip_analog {true | false}
```

Default: false

Data_type: bool, read/write

Skips routing nets or pins marked `+ USE ANALOG` in the DEF file or where `signalType` modified to `analog` in the OA DB. Specify this parameter for NanoRoute to skip the routing of nets with Analog Signal Type.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_design`

route_strictly_honor_1d_routing

`route_strictly_honor_1d_routing {true | false | bottomLayerNum:topLayerNum}`

Default: `false`

Data_type: string, read/write

When specified, the NanoRoute router strictly honors 1d routing. This enables you to turn off jog routing on a per layer basis.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- [route_design](#)

route_strict_honor_route_rule

`route_strict_honor_route_rule {true | false | bottom_layer_num:top_layer_num |
wire bottom_layer_num:top_layer_num}`

Default: `false`

Data_type: string, read/write

Uses nondefault spacing rules for nondefault wires.

Note: The router tapers wires and uses default vias even when this parameter is set to `true`.

You define nondefault routing rules in the `NONDEFAULTRULE` statement of the LEF file. Specify one of the following arguments:

- `false`: Specifies that the nondefault rule (NDR) spacing is a soft rule. In congested areas NanoRoute may need to use the extra NDR spacing to meet the minimum foundry rule set.
- `true`: Specifies that the nondefault rule spacing is a hard rule. NanoRoute will attempt to honor the requested NDR spacing irrespective of resource availability.
- `bottom_layer_num:top_layer_num`: Specifies that the nondefault rule spacing is a hard rule for

all the layers specified in the layer range.

- `wire bottom_layer_num:top_layer_num`: Specifies that the nondefault rule spacing is a hard rule for the wire but not the via metal to enable better DFM via optimization.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

For example,

- `set_db route_strict_honor_route_rule 3:6`

Specifies that the NDR spacing will be used in layers Metal3, Metal4, Metal5, and Metal6 for wire and via metal.

- `set_db route_strict_honor_route_rule "wire 3:6"`

Specifies that the NDR spacing will be used in Metal3, Metal4, Metal5, and Metal6 for wire and not via metal.

Note: Setting the rule to `false` will attempt to honor the NDR rule and allow short hot spots without detouring. If you set the rule to `true` (hard rule) we recommend you work more on minimizing the congested areas to preserve the NDR topology.

Related Commands

- `route_design`

route_stripe_layer_range

```
route_stripe_layer_range "bottomLayerNum:topLayerNum"
```

Default: ""

Data_type: string, read/write

Specifies the range of routing layers for stripes.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:


```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_design`

route_third_party_data

```
route_third_party_data {true | false}
```

Default: false

Data_type: bool, read/write

When enabled, NanoRoute supports third party data as input. It accepts non-DEF connectivity models, multiple NDRs per net, and other third party structures in order to enable ECO routing on non-native data.

Note: Since the third party data uses DEF data types for different purposes and is formed for a completely different use model, it is important to translate it into Cadence types. With proper setup Innovus accepts third party data for ECO Routing.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_design`

route_tieoff_to_shapes

```
route_tieoff_to_shapes "auto | stripe | ring | followpin | powergroundpin"
```

Default: auto

Data_type: string, read/write

Specifies the target to special wire shapes or target instance pin for tie net connection.

Specify one of the following arguments:

- `auto`: Specify `auto` to target special wire shapes, including stripe, ring, and standard cell followpin.
- `stripe`: Specify `stripe` to target stripes only. Stripes are the special wiring used for creating the power mesh.
- `ring`: Specify `ring` to target only rings. The power mesh along the macro boundary forms a ring. If it is in a macro placed in the design, only exposes power ring-pin shapes. When power stripes are added across the macro, they connect to each side of the ring-pin but do not go inside the ring.
- `followpin`: Specify `followpin` to target standard cell followpin only. Followpin is the special wiring used for connecting standard cells to power mesh.
- `powergroundpin`: Specify `powergroundpin` to target the power and ground pin of the instances only.
- `auto powergroundpin`: Specifies the target to special wire shapes or target instance pin for tie net.

Note: The target shapes can be specified together. The value `auto` can only be specified with `powergroundpin`, since the other values `stripe`, `ring`, and `followpin` are included in `auto`.

For example,

- `route_tieoff_to_shapes "followpin stripe"` signifies that it can tie to followpin or stripe.
- `route_tieoff_to_shapes "followpin powergroundpin"` signifies that it can tie to followpin or power ground pin
- `route_tieoff_to_shapes "stripe powergroundpin"` signifies that it can tie to stripe or power ground pin

Note: Using `"auto stripe"` is not recommended as `stripe` is included by `auto`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_design`

route_top_level_design

```
route_top_level_design {true|false}
```

Default: false

Data_type: bool, read/write

Specifies whether the current design is treated as a top-level design during NanoRoute routing.

- When set to `true`, the router applies top-level routing behavior, accounting for design boundaries and block-level interfaces.
- When set to `false`, the design is treated as a block-level or hierarchical design.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_design`

route_trim_pull_back_distance_from_boundary

```
route_trim_pull_back_distance_from_boundary {layer:value ...}
```

Default: " "

Data_type: string, read/write

Specifies the boundary pullback value per layer for trim metal layers. The pullback value signifies

how far the trim metal router brings signals and fill metal to a boundary (for example, block edge) in the absence of any obstructions.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- [route_design](#)

route_trunk_with_cluster_target_size

```
route_trunk_with_cluster_target_size integer
```

Default: 1

Data_type: int, read/write

Controls the routing pattern of nets with the trunk pattern attribute. By default, the router connects each pin in nets on which you have applied `set_attribute -pattern trunk` directly to the trunk of the special net; use this parameter to change the routing pattern for those nets:

- Specify 0 to route nets with the trunk pattern specified with a Steiner tree pattern instead of a trunk pattern.
- Specify an integer greater than 1 to route the nets with a fishbone pattern. In a fishbone pattern, several pins in a column are clustered together before connecting to the special net. The value of this attribute represents the approximate number of pins in each cluster.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_design`

route_use_auto_via

`route_use_auto_via value`

Default: `auto`

Data_type: `string, read/write`

When specified, it allows the use of internal generated vias. In new advance nodes, it enables you to specify some custom via's names from the LEF for the router to use. This helps to provide better pin access. The following is the syntax of the supported values:

```
{true | false| auto | {-rule ruleName1 viaName viaName} {-rule ruleName2 viaName  
viaName} {-prefix_name viaPrefix }}
```

Specify one of the following arguments:

- `true`: Specify `true` to turn on auto via generation.
- `false`: Specify `false` to turn off auto via generation.
- `auto`: Specify `auto` to enable via generation based on process nodes and allow Innovus to decide to turn on or off.
- `-rule ruleName1 viaName viaName`: Specify `-rule` to enable Innovus to use the provided via list for the specified rule.
- `-prefix_name`: Specify `-prefix_name` to add a prefix to the auto generated via names. By default, `NR_` prefix is used by the tool.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

For example,

- `set_db route_use_auto_via "-rule default VIA0_1 VIA0_2"`
signifies that M0 pins use VIA0_1 and VIA0_2 for default routing.

- `set_db route_use_auto_via {-rule HV_NDR HV_NDR_VIA4_2x1 HV_NDR_VIA5_2x1 HV_NDR_VIA6_2x1 HV_NDR_VIA7_2x1}`

signifies that for nets with HV_NDR rule, use the specified list via's only for those layers. For other layers Innovus can choose other via's defined in the NDR rule section or it can auto generate. Only 5 via's per layer per rule can be defined.

- **Note:** The NanoRoute router treats VIA and USEVIA definitions as soft constraints when the `route_use_auto_via` attribute is enabled (default is enabled). The vias specified with VIA or USEVIA LEF rules are considered as target vias along with other auto-generated vias. To specify a hard constraint for the vias to use, use the USEVIACUTCLASS rule. Alternatively, specify the following:

```
set_db route_use_auto_via "-rule ndr_rule_name viaName"
```

- `set_db route_use_auto_via "-prefix_name blockA"`

Signifies that the NanoRoute router will use blockA as the prefix for auto generated vias names instead of NR_.

Related Commands

- `route_design`

route_via_weight

```
route_via_weight {via_name via_weight ...}
```

Default: ""

Data_type: string, read/write

Specifies the priority or the order of vias for post route via swap to use. It sets the via weight for the specified via. The bigger the number, the higher priority the via is given. A negative value means the via will not be used.

Range: -1 to 256

You can use wildcards for the *viaName*.

For example:

```
route_via_weight {*FBD* 8 *FBS* 7 *PBD* 6 *PBS* 5 *2cut_p1* 4 *2cut_p2* 3 *2cut_p3* 2 *FAT* 1}
```

Note: Via name can be a predefined class of vias such as @1cut and @2cut. Here, @1cut stands

for single cut vias, @2cut stands for double cut vias.

In case of duplicated value setting of via, the latest value will be used.

For example:

```
route_via_weight { @1cut 2 @2cut 4 VIA12_1cut 1 }
```

Here, VIA12_1cut was first set weight to 2, and then set to 1, the final weight of VIA12_1cut is 1.

When you specify this option to do post route via swap, it's important to set the related options to improve the DFM and double cut via ratio.

- Specify `route_reserve_space_for_multi_cut true` before detail route to reserve space for inserting multicut vias in postroute via swap stage
- Specify `route_with_timing_driven false` and `route_detail_post_route_swap_via multiCut` to swap the single cut via to DFM and double cut via as much as possible

Note: If you have two NDR rules for which you want to specify a list of via's then it should be specified as one string.

```
set_db route_use_auto_via { { -rule rule_name via_name via_name } { -rule ruleName2 viaName2 } }  
route_design
```

Note: The NanoRoute router treats VIA and USEVIA definitions as soft constraints when the `route_use_auto_via` attribute is enabled (default is enabled). The vias specified with VIA or USEVIA LEF rules are considered as target vias along with other auto-generated vias. To specify a hard constraint for the vias to use, use the USEVIACUTCLASS rule. Alternatively, specify the following:

```
set_db route_use_auto_via -rule ndr_rule_name via_name
```

The following command signifies that the NanoRoute router will use blockA as the prefix for auto generated vias names instead of NR_.

```
set_db route_use_auto_via "-prefix_name blockA"
```

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```
- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_design`

route_with_eco

`route_with_eco {true | false}`

Default: `false`

Data_type: `bool`, `read/write`

Routes in engineering change order (ECO) mode. During ECO mode, NanoRoute completes partial routes with added logic, while maintaining the existing wire segments as much as possible.

Note: The Innovus software also has an `route_eco` command.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

Related Commands

- `route_design`

route_with_litho_driven

`route_with_litho_driven {true | false}`

Default: `false`

Data_type: `bool`, `read/write`

Avoids lithography problems during routing by avoiding certain routing patterns that might lead to the creation of lithography hotspots.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

`set_db attribute_name value`

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_design`

route_with_si_driven

```
route_with_si_driven {true | false}
```

Default: `false`

Data_type: `bool`, `read/write`

Prevents or reduces crosstalk. Works in conjunction with timing-driven routing.

- When timing-driven routing is specified, uses SMART routing to identify victim nets and minimize crosstalk by wire spacing, layer hopping, net ordering, and minimizing the use of long parallel wires.
- When timing-driven routing is not specified, uses an older signal integrity engine to minimize crosstalk by preventing or reducing the use of long parallel wires.

Specify `route_with_timing_driven` and `route_with_si_driven` to run SMART routing.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_design`

route_with_timing_driven

```
route_with_timing_driven {true | false}
```

Default: false

Data_type: bool, read/write

Minimizes timing violations by analyzing the timing slack for each path, the drive strengths of each cell in the library, and the maximum capacitance and maximum transition limits. During timing-driven routing, NanoRoute routes multi-pin nets to the most critical sink first, performs wire optimization by reducing resistance and coupling, and continually adjusts detouring.

When this attribute is set to 1, the router automatically generates a timing information file named `.timingfile.tif` in the working directory. It uses this file when it runs timing driven routing.

Note: Specify the following commands and to skip the CTE call and timing file generation step and use the specified file:

```
write_design_timing  fileName
set_db route_with_timing_driven true
```

Specify `route_with_timing_driven` and `route_with_si_driven` to run SMART routing.

You can use the `route_with_timing_driven` attribute to reduce the timing closure effort during post-route ECO. NanoRoute will replace via to multi-cut via on non-critical nets, if a violation does not occur.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `route_design`

route_with_trim_metal

```
route_with_trim_metal value
```

Default: " "

Data_type: string, read/write

Specifies the preferred trim metal grid for NanoRoute Router to use. You can specify the *value* in

the following format:

```
{-layer layer_number -mask1 {-pitch value -core_offset value} -mask1 { -pitch value ... } -mask2 { -pitch value ... } -mask2 {-pitch value ... }}
```

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

For example, the following command specifies the preferred trim metal grid on layer 2.

```
set_db route_with_trim_metal {-layer 2 -mask1 {-pitch 0.300 -core_offset 0.025 } \
-mask1 {-pitch 0.300 -core_offset 0.195 } \
-mask2 {-pitch 0.300 -core_offset 0.065 } \
-mask2 {-pitch 0.300 -core_offset 0.235  }}
```

Note: The width value is taken from the LEF.

Related Commands

- `route_design`

route_with_via_in_pin

```
route_with_via_in_pin {true | false | bottomLayerNum:topLayerNum}
```

Default: `false`

Data_type: string, read/write

Encloses via geometries completely inside standard cell pins. Specify `true` for this parameter to avoid `MINSTEP` violations for standard cell pin access.

Note: To avoid violations, specify the same value for this parameter throughout the entire design flow.

- `true`: Enclose vias inside standard cell pins on all layers.
- `false`: The router is not required to enclose vias inside standard cell pins.
- `bottomLayerNum:topLayerNum`: Enclose vias inside standard cell pins on layers in the specified layer range.

- 1:1: Specifies that Metal1 pins must enclose vias.
- 2:5: Specifies that pins on Metal2, Metal3, Metal4, and Metal5 must enclose vias. Pins on other layers are not required to enclose vias.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- [route_design](#)

route_with_via_only_for_block_cell_pin

```
route_with_via_only_for_block_cell_pin {true|false}
```

Default: false

Data_type: string, read/write

Encloses via in pin shape for macro cell pins.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- [route_design](#)

route_with_via_only_for_stdcell_pin

```
route_with_via_only_for_stdcell_pin {true | false | bottomLayerNum:topLayerNum }
```

Default: `false`

Data_type: `string`, `read/write`

Allows standard cell pins to be accessed by vias only. Prohibits planar access.

- `true`: Always use vias for standard cell pin access on all layers.
- `false`: Use vias or planar access for standard cell pins on all layers.
- `bottomLayerNum:topLayerNum`: Always use vias for pin access in the specified layer range.
 - `1:1`: Always use vias for access to Metal1 standard cell pins. Metal2 and above can use vias or planar access.
 - `2:5`: Always use vias for access to standard cell pins on Metal2, Metal3, Metal4, and Metal5. Metal1 and Metal6 (and above) can use vias or planar access.

Note: Set this option before any routing or verification because it affects the pin access preparation steps. In addition, after you set this option, the access remains for same the entire Innovus session. Do not change the setting, even for future routing sessions that load saved data from the database or a saved DEF file.

For example, if a design with pins on Metal1 and Metal2 is routed with this option set to `1:1`, the router uses vias for access to all standard cell pins on Metal1, and vias or planar access to standard cell pins on Metal2. If you save the design after routing, then import the design in a different Innovus session and route it again with this option set to `true` (that is, forcing the router to use vias to access all standard cell pins on all layers), you might have planar connections (from the previous routing session) to pins on Metal2. All the planar connections to those pins become violations, because they are not allowed when this option is set to `true`.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
`set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
`get_db attribute_name`

Related Commands

- `route_design`

route_trial_max_print_ignored_pad_nets

`route_trial_max_print_ignored_pad_nets` *integer*

Default: 0

Data_type: int, [read/write](#)

Number of nets to be printed connecting a pad term to a fterm without geometry.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

`set_db attribute_name value`

- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

Related Commands

- [route_design](#)

route_special

The root attributes in the `route_special` category are:

- `route_special_allow_non_preferred_direction_route`
- `route_special_avoid_over_core_row_layer`
- `route_special_block_pin_connect_ring_pin_corners`
- `route_special_block_pin_route_with_pin_width`
- `route_special_connect_broken_core_pin`
- `route_special_core_pin_ignore_obs`
- `route_special_core_pin_merge_limit`
- `route_special_core_pin_length`
- `route_special_core_pin_length_as_inst`
- `route_special_core_pin_max_via_scale`
- `route_special_core_pin_refer_to_follow_pin`
- `route_special_core_pin_reference_macro`
- `route_special_core_pin_site_rail_width`
- `route_special_core_pin_snap_to`
- `route_special_core_pin_stop_route`
- `route_special_extend_nearest_target`
- `route_special_jog_threshold_ratio`
- `route_special_layer_preferred_direction_cost`
- `route_special_layer_non_preferred_direction_cost`
- `route_special_pad_pin_min_via_size`
- `route_special_pad_pin_split`
- `route_special_pad_pin_split_from_center`

- [route_special_pad_ring_use_lef](#)
- [route_special_secondary_pin_max_gap](#)
- [route_special_secondary_pin_rail_width](#)
- [route_special_signal_pin_as_pg](#)
- [route_special_split_long_via](#)
- [route_special_pg_pin_as_signal](#)
- [route_special_target_number](#)
- [route_special_target_search_distance](#)
- [route_special_time_limit](#)
- [route_special_endcap_as_core](#)
- [route_special_via_through_to_closest_ring](#)
- [route_special_via_connect_to_shape](#)
- [route_special_welltap_as_endcap](#)

To set the value of any root attribute, use the [set_db](#) command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the [get_db](#) command:

```
get_db attribute_name
```

To get the current value of all attributes in the `route_special` category, use the following command:

```
get_db -category route_special
```

route_special_allow_non_preferred_direction_route

```
route_special_allow_non_preferred_direction_route {true | false}
```

Default: false

Data_type: Boolean

Allows wrong way routing.

Note: If the metal1 preferred routing direction is vertical and standard cells have their VDD and VSS pins on layer metal1, route will route metal1 wires on horizontal direction to continue extending the followpin connections to core rings by this option

route_special_avoid_over_core_row_layer

`route_special_avoid_over_core_row_layer layer_name`

Default: max layer of standard cell pin

Data_type: String

Specifies the number of layers to avoid routing of block pin and pad pin over the core row area.

route_special_block_pin_connect_ring_pin_corners

`route_special_block_pin_connect_ring_pin_corners {true | false}`

Default: false

Data_type: Boolean

Specifies that the block wire used for a connection is to have the same width as the pin from which it connects.

route_special_block_pin_route_with_pin_width

`route_special_block_pin_route_with_pin_width {true | false}`

Default: false

Data_type: Boolean

Specifies that the block wire used for a connection is to have the same width as the pin from which it connects.

route_special_connect_broken_core_pin

`route_special_connect_broken_core_pin {true | false}`

Default: false

Data_type: Boolean

When set to 1, `sroute` checks the signal pin, second power pin, and obs, and then breaks the followpin to avoid violations. When set to 0, `sroute` does not check internal geometries and does not break the followpin.

route_special_core_pin_ignore_obs

```
route_special_core_pin_ignore_obs {none | placement_blockage | overlap_obs | block_halo  
}
```

Default: none

Data_type: Enum

Ignores the specified blockage while connecting the follow pin.

route_special_core_pin_merge_limit

```
route_special_core_pin_merge_limit value
```

Default: 0.0

Data_type: Double

Joins follow pin rail gaps smaller than this variable, bigger gaps will remain open. The option is of type *float*.

route_special_core_pin_length

```
route_special_core_pin_length value
```

Default: 0.0

Data_type: Double

Connects core pins of specified length. The option is of type *float*.

route_special_core_pin_length_as_inst

```
route_special_core_pin_length_as_inst {true | false}
```

Default: false

Data_type: Boolean

Connects core pins whose length equals or is greater than the length of instance.

route_special_core_pin_max_via_scale

```
route_special_core_pin_max_via_scale widthPct heightPct
```

Default: ""

Data_type: String

Controls the length and width of vias at the crossover of core pin rails and stripes. Specify this value as percentages of the width of the via at the top of the stack. Smaller vias leave room for more routing tracks on layers between the vias.

route_special_core_pin_refer_to_follow_pin

```
route_special_core_pin_refer_to_follow_pin {true | false}
```

Default: false

Data_type: Boolean

If set to `true`, parallel metal followpins are generated based on the existing standard cell pins, [when there is no other metal followpin on the standard cell pins](#).

route_special_core_pin_reference_macro

```
route_special_core_pin_reference_macro macro
```

Default: ""

Data_type: String

Uses specified macros as references for followpin creation. The option is of type *string*.

route_special_core_pin_site_rail_width

```
route_special_core_pin_site_rail_width {{site1 layer11 Width11 layer12 Width12 ...}  
{site2 layer21 Width21 layer22 Width22 ...} ...}
```

Default: ""

Data_type: String

Defines the followpin layer width for the specified site rail for followpin generation. The followpin width defined by this attribute has higher priority than the width of the same layer defined in the macro LEF, but lower than that defined by the `route_special -core_pin_width` parameter.

Following are the values used with this attribute:

- `site`: Specifies the site name defined in the LEF file.
- `layer`: Specifies the name of the layer defined in the LEF file.
- `width`: Specifies the followpin width on the top and bottom sides of the defined site.

route_special_core_pin_snap_to

```
route_special_core_pin_snap_to {M1_pin | opt_routing_track | grid | half_grid}
```

Default: M1_pin

Data_type: Enum

Snaps M2 followpins in the location to save routing tracks in a preventive manner. When set to:

- `M1_pin`: Generated metal2 followpins are snapped to metal1 stdcell pin center.
- `opt_routing_track`: Generated metal2 followpins are snapped to track or half track to save routing tracks.
- `grid`: Generated M2 followpins are snapped to the nearest routing track.
- `half_grid`: Generated M2 followpins are snapped to the nearest half routing track.

route_special_core_pin_stop_route

```
route_special_core_pin_stop_route {atRowEnd atCellEnd}
```

Default: RowEnd

Data_type: Enum

Extends the follow pin wire either to the row-end or cell-end.

route_special_extend_nearest_target

```
route_special_extend_nearest_target {true | false}
```

Default: false

Data_type: Boolean

Extends the nearest target so that it can be connected. The option is of type *bool*.

route_special_jog_threshold_ratio

`route_special_jog_threshold_ratio value`

Default: 10.0

Data_type: Double

Defines the distance ratio between a jogging target and a straight target. The option is of type *float*.

route_special_layer_preferred_direction_cost

`route_special_layer_preferred_direction_cost layer1 cost1 layer2 cost2...`

Default: ""

Data_type: String

Specifies layer cost of preferred direction. The option is of type *string*.

route_special_layer_non_preferred_direction_cost

`route_special_layer_non_preferred_direction_cost layer1 cost1 layer2 cost2...`

Default: ""

Data_type: String

Specifies layer cost of non-preferred direction.

route_special_pad_pin_min_via_size

`route_special_pad_pin_min_via_size viaSizePercent`

Default: 20.0

Data_type: Double

Permits the software to reduce the size of vias at pad pins when changing layers, if there is not enough room for a standard size via.

route_special_pad_pin_split

`route_special_pad_pin_split width spacing`

Default: ""

Data_type: String

Splits the width and spacing for wide pad pins.

route_special_pad_pin_split_from_center

```
route_special_pad_pin_split_from_center {true | false}
```

Default: false

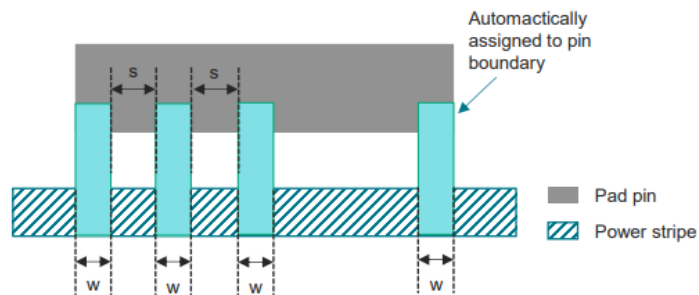
Data_type: Boolean

Splits the width and spacing specified by the `route_special_pad_pin_split` attribute to enable connections to the center of wide pad pins. This parameter must be used with the `route_special_pad_pin_split` attribute. See the examples below:

- ```
set_db route_special_pad_pin_split {w s}
```

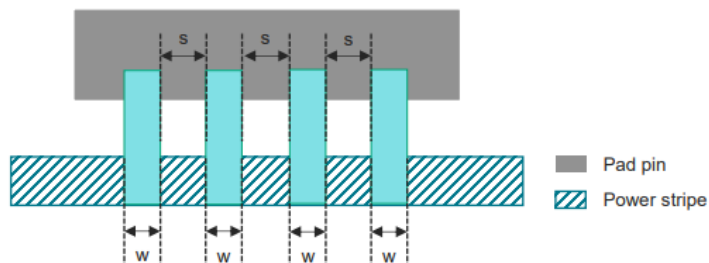
```
set_db route_special_pad_pin_split_from_center false
```



- ```
set_db route_special_pad_pin_split {w s}
```



```
set_db route_special_pad_pin_split_from_center true
```



route_special_pad_ring_use_lef

```
route_special_pad_ring_use_lef {true | false}
```

Default: `true`

Data_type: Boolean

Splits the width and spacing for wide pad pins.

route_special_secondary_pin_max_gap

`route_special_secondary_pin_max_gap value`

Default: `0.0`

Data_type: Double

Specifies variable value for gaps bigger than this value cause level shifter rail to break. The option type is *float*.

route_special_secondary_pin_rail_width

`route_special_secondary_pin_rail_width value`

Default: `0.0`

Data_type: Double

Specifies width of followpin rail. The option type is *float*.

route_special_signal_pin_as_pg

`route_special_signal_pin_as_pg {true | false}`

Default: `false`

Data_type: Boolean

Specifies signal pins as PG pins in pad ring creation. The option type is *bool*.

route_special_split_long_via

`route_special_split_long_via {threshold_value step_value offset_value height_value}`

Default: `0 0 -1 -1`

Data_type: String

Splits vias longer than the specified length (`threshold_value`, in micrometers) into smaller vias with specified center-to-center spacing (`step_value`, in micrometers), left/bottom end offset (`offset_value`, in micrometers) and vertical/horizontal height (`height_value`, in micrometers) values. This is independent of the option `-split_via`.

Note: If the `offset_value` is given as negative, the vias will be placed symmetrically on both ends. The default value of `offset_value` is `-1`.

route_special_pg_pin_as_signal

```
route_special_pg_pin_as_signal {cellName: pinName}
```

Default: ""

Data_type: String

Lists the cell/pin information NOT extracted from CPF.

route_special_target_number

```
route_special_target_number value
```

Default: 0

Data_type: Integer

Specify number of target to connect. The option type is *int*.

route_special_target_search_distance

```
route_special_target_search_distance dist
```

Default: 0.0

Data_type: Double

Specifies target search distance. The option type is *float*.

route_special_time_limit

```
route_special_time_limit value
```

Default: 0.0

Data_type: Double

Specify time limit (in seconds) that the router is allowed in routing each port. The option type is *float*.

route_special_endcap_as_core

```
route_special_endcap_as_core {true | false}
```

Default: false

Data_type: Boolean

Specifies to treat Endcap macro as core macro. The option type is *bool*.

route_special_via_through_to_closest_ring

```
route_special_via_through_to_closest_ring {true | false}
```

Default: false

Data_type: Boolean

Specifies that a via should connect only to the closest ring layer when multiple rings overlap.

route_special_via_connect_to_shape

```
route_special_via_connect_to_shape {padding|ring|stripe|blockring|blockpin|coverpin|no-shape|blockwire|corewire|followpin|iowire}
```

Default: padding ring stripe blockring blockpin coverpin no-shape blockwire corewire followpin iowire

Data_type: Enum

Specifies which shapes vias can connect to.

route_special_welltap_as_endcap

```
route_special_welltap_as_endcap string
```

Default: ""

Data_type: String

When used, `route_special` treats the `Welltap` cells as the `Endcap` cells. Additionally,

the `ENDCAP` class cells are ignored by default, when connecting the `followpin` wires.

safety

The root attributes in the `safety` category are:

- `safety_dcls_isolate_halo_type`
- `safety_dcls_isolate_type`
- `safety_dcls_route_types`
- `safety_flow_enable`
- `safety_midas_enable`
- `safety_ser_cells`
- `safety_ser_type`
- `safety_tmr_isolate_clock`
- `safety_tmr_isolate_reset`
- `safety_tmr_isolate_scan_enable`
- `safety_tmr_spacing`
- `safety_tmr_spacing_x`
- `safety_tmr_spacing_y`
- `safety_tmr_well_tap_cells`

To set the value of a read/write root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `safety` category, use the following command:

```
get_db -category safety
```

safety_dcls_isolate_halo_type

safety_dcls_isolate_halo_type {soft | hard}

Default: soft

Data_type: enum {soft hard}, read-only

Specifies the type of isolation halo required for the Dual Core Lock Step (DCLS) groups. This attribute supports the following values:

- **hard:** Specifies that the isolation cells on a group's input/output nets should always be placed inside the group's isolation halo boundary.
- **soft:** Works the same as **hard**, but can also move the isolation cells inside the group boundary, if required, to improve timing.

This is a root attribute.

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `read_usf`

safety_dcls_isolate_type

safety_dcls_isolate_type {soft | hard}

Default: soft

Data_type: enum {soft hard}, read-only

Specifies the type of isolation required for the Dual Core Lock Step (DCLS) groups. This attribute supports the following values:

- **soft:** Specifies that the isolation networks (`insts` and `nets`) need not avoid the boundary of the `exclusive_groups` of the groups that they isolate.
- **hard:** Specifies that the isolation networks (`insts` and `nets`) must avoid the boundary of the `exclusive_groups` of the groups that they isolate.

This is a root attribute.

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- [read_usf](#)

safety_dcls_route_types

```
safety_dcls_route_types {internal | interface | common | top}
```

Default:

Data_type: enum* {internal interface common top}, [read-only](#)

Specifies the type of nets on which routing rules need to be enforced for the Dual Core Lock Step (DCLS) groups. This attribute supports the following values:

- `none`: Specifies that routing rules will not be enforced on any nets for DCLS.
- `internal`: Specifies the nets whose all the pins are logically inside a single DCLS group. These net cannot route outside of the group boundary.
- `interface`: Specifies the net that has pins logically inside and outside a DCLS group, and outside of the `exclusive_groups`. This net cannot route over the boundary of its `exclusive_groups`.
- `common`: Specifies the net that has pins logically inside two or more DCLS `exclusive_groups`. This net can route inside the boundary of those groups to complete the connections, but a violation should be reported.
- `top`: Specifies the nets whose pins are logically outside of DCLS groups. This net can route over the boundary of any DCLS group, but cannot route over the boundary of two or more `exclusive_groups`.

This is a root attribute.

- To get the current value of any root attribute, use the [get_db](#) command:

```
get_db attribute_name
```

Related Commands

- [read_usf](#)

safety_flow_enable

`safety_flow_enable {true | false}`

Default: `false`

Data_type: `bool`, `read-only`

Specifies the master switch to enable the implementation and analysis of the safety-mechanisms defined in the USF file.

The software sets the `safety_flow_enable` attribute automatically, once the `read_usf` command run is completed successfully.

This is a root attribute.

- To get the current value of any root attribute, use the `get_db` command:

`get_db attribute_name`

Related Commands

- `read_usf`

safety_midas_enable

`safety_midas_enable {none | fmeda | physical}`

Default: `none`

Data_type: `enum {none fmeda physical}`, `read/write`

Enables the Unified Safety Format (USF) technology. The supported values of this attribute are:

- `none`: Does not enable this attribute.
- `fmeda`: Enables the USF command set for the failure mode effect and diagnostic analysis.
Note: This value is supported only in Genus.
- `physical`: Enables the USF command set for physical implementation of safety mechanisms.
Note: This value is supported in both Innovus and Genus.

To enable USF commands (such as `read_usf`, `write_usf`, and `check_safety_mechanism`) in Innovus/Genus, set `safety_midas_enable` to `physical`:

```
set_db safety_midas_enable physical
```

You will see a message like this:

```
Cadence Midas USF - Physical
```

```
Copyright 2022 Cadence Design Systems, Inc. All rights reserved worldwide.
```

```
Cadence and the Cadence logo are registered trademarks of Cadence Design Systems Inc.  
in the United States and other countries.
```

```
Midas USF - Physical Version: 22.03.001
```

```
physical
```

 Once USF Physical is loaded, it is not possible to disable or reload it in the current session.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `read_usf`

safety_ser_cells

```
safety_ser_cells string
```

Default: ""

Data_type: base_cell*, read-only

Specifies the list of `base_cells` allowed for those insts, which have a Soft Error Resilience (SER) safety mechanism.

This is a root attribute.

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `read_usf`

safety_ser_type

`safety_ser_type {hard | soft}`

Default: `hard`

Data_type: enum {hard soft}, read-only

Specifies the type of Soft Error Resilience (SER) safety mechanism to be used for failure mode mapping between the `insts` to `ser` cells. The supported values of this attribute include:

- `hard`: Always uses SER cells for instances that have an SER safety mechanism.
- `soft`: Uses SER cells for instances that have an SER safety mechanism, except in cases where using a non-SER cell can improve timing.

This is a root attribute.

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `read_usf`

safety_tmr_isolate_clock

`safety_tmr_isolate_clock {none | split_net | split_network}`

Default: `none`

Data_type: enum {none split_net split_network}, read-only

Specifies the type of clock isolation required for Triple Modular Redundancy (TMR) flops. The supported values of this attribute include:

- `none`: Specifies that no isolation is required.
- `split_net`: Specifies that the net directly driving a TMR flop cannot directly drive either of the other two flops from the same TMR group.

- `split_network`: Specifies that the network driving the TMR flops should split at the last combinational cell (excluding buffering) on that network. Each branch can drive multiple TMR and non-TMR flops, but can only drive one of the three flops in a TMR group.

This is a root attribute.

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

safety_tmr_isolate_reset

```
safety_tmr_isolate_reset {none | split_network | split_net}
```

Default: none

Data_type: enum {none split_network split_net}, read-only

Specifies the type of isolation required for the reset signals driving the Triple Modular Redundancy (TMR) flops. The values by this attribute include:

- `none`: Specifies that no isolation is required.
- `split_net`: Specifies that the net directly driving a TMR flop cannot directly drive either of the other two flops from the same TMR group.
- `split_network`: Specifies that the network driving the TMR flops will split at the last combinational cell (excluding buffering) on that network. Additionally, each branch can drive multiple TMR and non-TMR flops, but only one of the three flops in a TMR group.

This is a root attribute.

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `read_usf`

safety_tmr_isolate_scan_enable

```
safety_tmr_isolate_scan_enable {none | split_network}
```

Default: none

Data_type: enum {none split_network split_net}, read-only

Specifies the type of isolation required for the `scan_enable` signals driving the Triple Modular Redundancy (TMR) flops. The values by this attribute include:

- `none`: Specifies that no isolation is required.
- `split_net`: Specifies that the net directly driving a TMR flop cannot directly drive either of the other two flops from the same TMR group.
- `split_network`: Specifies that the network driving the TMR flops will split at the last combinational cell (excluding buffering) on that network. Additionally, each branch can drive multiple TMR and non-TMR flops, but only one of the three flops in a TMR group.

This is a root attribute.

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `read_usf`

safety_tmr_spacing

`safety_tmr_spacing` *microns*

Default: 0.0

Data_type: coord, read-only

Specifies the radial spacing in any direction that should be maintained between the three flops in a TMR group. The value of this attribute is measured in microns.

Note: The `safety_tmr_spacing_x`, `safety_tmr_spacing_y`, and `safety_tmr_spacing` attributes are mutually exclusive.

This is a root attribute.

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- [read_usf](#)

safety_tmr_spacing_x

`safety_tmr_spacing_x` *microns*

Default: 0.0

Data_type: coord, read-only

Specifies the horizontal spacing that should be maintained between the three flops in a TMR group. The value of this attribute is measured in microns. Only one value of either `safety_tmr_spacing_x` or `safety_tmr_spacing_y` needs to be met.

Note: The `safety_tmr_spacing_x`, `safety_tmr_spacing_y`, and `safety_tmr_spacing` attributes are mutually exclusive.

This is a root attribute.

- To get the current value of any root attribute, use the [get_db](#) command:

```
get_db attribute_name
```

Related Commands

- [read_usf](#)

safety_tmr_spacing_y

`safety_tmr_spacing_y` *microns*

Default: 0.0

Data_type: coord, read-only

Specifies the vertical spacing that should be maintained between the three flops in a TMR group. The value of this attribute is measured in microns. Only one value of either `safety_tmr_spacing_x` or `safety_tmr_spacing_y` needs to be met.

Note: The `safety_tmr_spacing_x`, `safety_tmr_spacing_y`, and `safety_tmr_spacing` attributes are mutually exclusive.

This is a root attribute.

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `read_usf`

safety_tmr_well_tap_cells

```
safety_tmr_well_tap_cells cells
```

Default: {}

Data_type: base_cell*, read-only

Specifies the well-tap cells to be placed next to the left and right edges of the Triple Modular Redundancy (TMR) flops.

This attribute can be used for the designs without power domains.

This is a root attribute.

- To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

Related Commands

- `read_usf`

si

The root attributes in the **si** category are:

- `si_accumulated_small_aggressor_factor`
- `si_accumulated_small_aggressor_mode`
- `si_accumulated_small_aggressor_threshold`
- `si_aggressor_alignment`

- [si_clock_synchronicity](#)
- [si_delay_clock_delta_threshold](#)
- [si_delay_delta_threshold](#)
- [si_delay_delta_annotation_mode](#)
- [si_delay_enable_double_clocking_check](#)
- [si_delay_enable_logical_correlation](#)
- [si_delay_enable_report](#)
- [si_delay_only](#)
- [si_delay_separate_on_data](#)
- [si_delay_double_switching](#)
- [si_enable_bus_attacker_correlation](#)
- [si_enable_drv_with_delta_slew](#)
- [si_enable_glitch_overshoot_undershoot](#)
- [si_enable_glitch_propagation](#)
- [si_enable_glitch_propagation_spice_deck](#)
- [si_enable_lle_glitch](#)
- [si_enable_two_stage_driver_weakening](#)
- [si_max_virtual_attacker_constituents](#)
- [si_enable_user_defined_bus_merging](#)
- [si_enable_virtual_attacker_constituent_report](#)
- [si_glitch_accuracy_level](#)
- [si_glitch_analysis_type](#)
- [si_glitch_constrained_input_threshold_failure_point](#)
- [si_glitch_enable_dynamic_receiver_peak_limits](#)
- [si_glitch_enable_report](#)
- [si_glitch_input_full_propagation_threshold](#)
- [si_glitch_input_full_propagation_voltage_high_threshold](#)

- [si_glitch_input_full_propagation_voltage_low_threshold](#)
- [si_glitch_input_threshold](#)
- [si_glitch_input_voltage_high_threshold](#)
- [si_glitch_input_voltage_low_threshold](#)
- [si_glitch_receiver_clock_peak_limit](#)
- [si_glitch_receiver_latch_peak_limit](#)
- [si_glitch_receiver_peak_limit](#)
- [si_individual_aggressor_simulation_filter](#)
- [si_individual_aggressor_threshold](#)
- [si_individual_aggressor_clock_threshold](#)
- [si_nonlinear_aggressor_slew](#)
- [si_num_iteration](#)
- [si_reselection](#)
- [si_reselection_delay_threshold](#)
- [si_reselection_hold_slack](#)
- [si_reselection_setup_slack](#)
- [si_secondary_attacker_decoupling_factor](#)
- [si_skip_noise_model_check](#)
- [si_skip_timing_window](#)
- [si_switch_probability](#)
- [si_unconstrained_net_use_infinite_timing_window](#)
- [si_use_infinite_timing_window](#)
- [si_virtual_inter_die_attacker_slew](#)

To set the value of any root attribute, use the [set_db](#) command:

- `set_db attribute_name value`

To get the current value of any root attribute, use the [get_db](#) command:

- `get_db attribute_name`

To get the current value of all attributes in the `assign_pins` category, use the following command:

- `get_db -category si`

The SI attributes can be reported by using the following command:

- `get_db si_*`

si_accumulated_small_aggressor_factor

`si_accumulated_small_aggressor_factor double`

Default: 1.0

Data_type: Double

Specifies the multiplication factor that controls the effect of the cap mode accumulated small attacker. An accumulated small attacker is an imaginary attacker that represents the sum of all the weak attackers on a specified net. To reduce the pessimism of using the accumulated small attacker, adjust this value to anything between 0 and 1. A value of 0 means that the cap mode accumulated small attacker will not be used.

Note: This option does not work with the current mode accumulated small attacker, which is default for process sizes 45nm and below.

si_accumulated_small_aggressor_mode

`si_accumulated_small_aggressor_mode {cap | current | zero_mean | numerical}`

Default: {}

Data_type: Enum

Specifies whether to use the current matching based method for accumulated small attackers. You can specify the following options:

- `cap`: Uses the accumulated small attacker method, which matches the total coupling capacitance of an attacker with the small attackers that are being virtualized. This is the default mode for all process nodes above 45nm.
- `current`: Uses the current matching based method for accumulated small attackers. With this method, the transitions on virtual attackers are modeled using Piece Wise Linear (PWL) waveforms. The PWL waveforms are generated in a way that the current induced by the transition matches the total current induced by all accumulated small attacker components.

- **zero_mean:** Uses the accumulation method where series of small attackers (that is, attackers with less than the individual attacker threshold value), are assumed to transition in opposing directions and cancel one another in aggregate.

Note: For 45nm and below process nodes, the default value is `current`.

si_accumulated_small_aggressor_threshold

`si_accumulated_small_aggressor_threshold double`

Default: 10.01

Data_type: Double

Sets the accumulated small attacker glitch threshold. If the virtual attacker produces a glitch whose peak is below this threshold, the component coupling capacitors of the accumulated small attacker are split into two ground capacitors and there is no accumulated small attacker for the victim. The value is specified as a ratio of VDD. To disable the accumulated small attacker, you must set a value greater than 1.

si_aggressor_alignment

`si_aggressor_alignment {path | path_overlap | timing_aware_edge | input_arrival}`

Default: `input_arrival`

Data_type: Enum

Specifies the attacker alignment. You can specify the following options:

- **path:** The victim worst arrival edge is used while determining the attackers. Crosstalk impact is computed according to the latest/earliest arrival edge of victim for max/min analysis.
- **path_overlap:** The full timing window of victim is considered while determining the attackers. Crosstalk impact is calculated according the victim edge, lying anywhere in timing window, which may experience maximum impact.
- **timing_aware_edge:** Realistic analysis similar to path-based analysis is performed. Each attacker's impact is annotated on victim transition where attacker timing window edge aligns with the victim. However, to ensure GBA bounds PBA analysis, the timing-aware edge mode handles some nets based on their timing properties in GBA analysis.
- **input_arrival:** The attackers use timing windows formed by 2-sigma arrivals regardless of the analysis sigma multiplier. The victim driver input arrivals will be used for SI computation.

Also, the mean victim timing window and edge will be used for SI computation in GBA and PBA modes, respectively. This behavior allows pessimism reduction in GBA SI analysis with narrower timing windows. And provides a more accurate PBA SI analysis, with no dependency of driver cell GBA delays.

For more information, refer to the [Attacker Selection Based on Alignment](#) section in the *Tempus Stylus User Guide*.

si_clock_synchronicity

`si_clock_synchronicity {asynchronous | synchronous}`

Default: synchronous

Data_type: Enum

Treats the clocks either synchronous or asynchronous to each other.

si_delay_clock_delta_threshold

`si_delay_clock_delta_threshold double`

Default: -1.0

Data_type: Double

Specifies the clock delta delay analysis threshold value.

si_delay_delta_threshold

`si_delay_delta_threshold double`

Default: -1.0

Data_type: Double

Specifies the delta delay analysis threshold value.

si_delay_delta_annotation_mode

`si_delay_delta_annotation_mode {lumpedOnNet | arc}`

Default: arc

Data_type: Enum

Calculates the delay for each arc (cell arc and net arc) in a path. By default, AAE calculates the delay, including SI for each arc and if the user chooses to view the SI delay separately, it would show annotated to each arc.

Alternatively, you can choose to annotate the SI delay solely to the net. In this case, the worst-case arc is used to calculate the SI delay and it is annotated solely to the net.

- `lumpedOnNet`: Lumps the delta delay on a the net, if it is broken out.
- `arc`: Separates the delta delay on each cell arc and net arc, as calculated.

si_delay_enable_double_clocking_check

```
si_delay_enable_double_clocking_check {true | false}
```

Default: false

Data_type: Boolean

Enables double clocking check in AAE.

This option enables double clocking analysis and `report_double_clocking` command can be run to print a report.

si_delay_enable_logical_correlation

```
si_delay_enable_logical_correlation {true | false}
```

Default: false

Data_type: Boolean

Enables the AAE-SI logical correlation. Logical correlation refers to correlation of logic between (aggressor) nets which are connected to each other. A simple case would be input and output nets of an inverter never switch in the same direction, so cannot be attackers to the same net simultaneously.

si_delay_enable_report

```
si_delay_enable_report {true | false}
```

Default: false

Data_type: Boolean

Enables SI delay reports.

si_delay_only

```
si_delay_only {true | false}
```

Default: false

Data_type: Boolean

Performs noise-on-delay analysis exclusively. This means that when you use this parameter, the software does not perform glitch analysis and only performs noise-on-delay analysis. In addition, this parameter performs SI analysis.

si_delay_separate_on_data

```
si_delay_separate_on_data {true | false}
```

Default: false

Data_type: Boolean

Separates incremental delays on signal nets.

si_delay_double_switching

```
si_delay_double_switching {none | clock | data_sequential | all}
```

Default: none

Data_type: Enum

Specifies the type of data (clock, data sequential, or all) for which double switching analysis will be performed.

si_enable_bus_attacker_correlation

```
si_enable_bus_attacker_correlation {true | false}
```

Default: false

Data_type: Boolean

Enables the software to treat bits of bus as correlated attackers.

When set to `true`, the bits of bus type nets are grouped together based on their cumulative weight.

The bus bits that act as attackers to a specific victim are treated as a group. All the attackers are grouped into different bundles where normal (non-bus) attackers form one group and bits of the same bus form multiple other groups. In case of a victim that has a number of small couplings to bits of a bus, the combined effect of a full bus can be used to decide whether an attacker is significant.

si_enable_drv_with_delta_slew

```
si_enable_drv_with_delta_slew {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, the `max_transition` violations are reported based on the SI slews. Also, this allows SI slew reporting/fixing by the `opt_signoff` command and is applied prior running the `write_eco_opt_db` command.

si_enable_glitch_overshoot_undershoot

```
si_enable_glitch_overshoot_undershoot {true | false}
```

Default: `false`

Data_type: Boolean

Enables glitch overshoot/undershoot.

si_enable_glitch_propagation

```
si_enable_glitch_propagation {true | false}
```

Default: `false`

Data_type: Boolean

Enables glitch propagation through single stage cells and driver weakening through single stage and dual stage cells. When this attribute is set to `true`, the software does not delete the glitch waveform saved on instance input pins. The waveform is later used by spice deck to propagate glitch at the next stage receiver pin.

si_enable_glitch_propagation_spice_deck

```
si_enable_glitch_propagation_spice_deck {true | false}
```

Default: false

Data_type: Boolean

Enables the driver input glitch waveform to be printed in the SPICE deck such that SPICE can also perform single stage glitch propagation. When this option is set to `true`, the software does not delete the glitch waveform saved on instance input pins. The waveform is later used by spice deck to propagate glitch at the next stage receiver pin.

si_enable_lle_glitch

```
si_enable_lle_glitch {true | false}
```

Default: false

Data_type: Boolean

Enables SI glitch analysis for LLE.

si_enable_two_stage_driver_weakening

```
si_enable_two_stage_driver_weakening {true | false}
```

Default: true

Data_type: Boolean

Enables the two-stage driver weakening mode.

A non-failing significant glitch at the input of a two-stage driver can propagate through the first stage and weaken the second stage. This might cause the crosstalk-induced glitch to amplify at the next stage receiver (though it may not be significant enough to propagate). When two-stage driver weakening is enabled, the software considers weakened holding resistance for the two-stage driver for glitch analysis.

The dual stage driver weakening honors the glitch propagation threshold criteria.

Note: This feature is supported with CCS-N models only.

si_max_virtual_attacker_constituents

```
si_max_virtual_attacker_constituents integer
```

Default: 5

Data_type: Integer

Specifies the maximum number of virtual attacker constituents to report.

si_enable_user_defined_bus_merging

```
si_enable_user_defined_bus_merging {true | false}
```

Default: false

Data_type: Boolean

Enables the bus merging feature. If a net is in two groups, then the groups will be merged.

si_enable_virtual_attacker_constituent_report

```
si_enable_virtual_attacker_constituent_report {true | false}
```

Default: false

Data_type: Boolean

Enables virtual attacker constituent reporting.

The software provides support for virtual attacker constituent reporting (when the bus attacker correlation feature is enabled using the `si_enable_bus_attacker_correlation` attribute).

The virtual attacker constituent reporting feature is useful for identifying significant buses and their bits - this information can be used for glitch fixing.

si_glitch_accuracy_level

```
si_glitch_accuracy_level int
```

Default: 0

Data_type: Integer

Specifies the accuracy levels for glitch analysis.

si_glitch_analysis_type

```
si_glitch_analysis_type {source | full_propagation}
```

Default: `source`

Data_type: Enum

Specifies the glitch analysis type - `source` or `full_propagation`.

- `full_propagation`: Propagates glitch noise through single and 2-stage cells with CCSN (for example, `inv` or `buf`), and stops glitch propagation at any sequential cell, or 3 or more stage cell input.
- `source`: Propagates glitch through 1-stage cells only, and performs driver weakening through 2-stage cells with CCSN.

si_glitch_constrained_input_threshold_failure_point

```
si_glitch_constrained_input_threshold_failure_point {input | both}
```

Default: `input`

Data_type: Enum

Allows you to control the glitch failure criteria when a user specifies an object-specific RIP threshold.

When set to `input`, the software skips ROP analysis and performs RIP-based failure check on the receiver pins for cases where object-specific RIP threshold is set using the `set_glitch_threshold -failure_point input -net` (or `-pin`) command.

When set to `both`, the software performs both RIP and ROP failure checks, wherever possible. If user-specified RIP failure criteria is violated, then it will be flagged as a RIP violation. If RIP criteria passes and receiver noise models exist, then the ROP is computed and checked against the ROP failure threshold for failure determination.

For the following cases where the object-specific RIP thresholds are not specified, the attribute behavior remains unchanged:

- For receiver pins with noise models present, ROP-based failure check is performed. The RIP-based failure criteria is not checked at all.
- For receiver pins with noise models missing, the RIP-based failure criteria is checked for violations.

si_glitch_enable_dynamic_receiver_peak_limits

```
si_glitch_enable_dynamic_receiver_peak_limits {true | false}
```

Default: `false`

Data_type: Boolean

Enables automatic computation of ROP threshold value of two-stage cells, using the CCS-noise data on timing arcs.

When set to `true`, the software computes the noise threshold of the last stage from the table of DC currents.

si_glitch_enable_report

`si_glitch_enable_report {true | false}`

Default: `false`

Data_type: Boolean

Enables the storage of glitch information for further debug using the `report_noise` command.

When set to `false`, the `time_design` command will still report glitches in the `timingReports/*.SI_Glitches.rpt` file but the `report_noise` command will not be available for use.

When set to `true`, the `report_noise` command may be used and glitch information will be more aligned to Tempus signoff values.

si_glitch_input_full_propagation_threshold

`si_glitch_input_full_propagation_threshold double`

Default: `0.75`

Data_type: Double

Specifies the receiver input glitch threshold beyond which Tempus will not propagate the glitch in full-propagation mode, and an RIP violation will be flagged.

si_glitch_input_full_propagation_voltage_high_threshold

`si_glitch_input_full_propagation_voltage_high_threshold double`

Default: `0.75`

Data_type: Double

Specifies the receiver VH input glitch threshold beyond which Tempus will not propagate the glitch

in full-propagation mode, and an RIP violation will be flagged.

This attribute takes precedence over `si_glitch_input_full_propagation_threshold`.

si_glitch_input_full_propagation_voltage_low_threshold

`si_glitch_input_full_propagation_voltage_low_threshold double`

Default: 0.75

Data_type: Double

Specifies the receiver VL input glitch threshold beyond which Tempus will not propagate the glitch in full-propagation mode, and an RIP violation will be flagged.

This option takes precedence over `si_glitch_input_full_propagation_threshold`.

These attributes are applicable only when the full propagation mode is enabled (using the `si_glitch_analysis_type full_propagation` attribute).

si_glitch_input_threshold

`si_glitch_input_threshold double`

Default: 0.4

Data_type: Double

Specifies the input glitch failure threshold value during AAE analysis.

si_glitch_input_voltage_high_threshold

`si_glitch_input_voltage_high_threshold double`

Default: 0.4

Data_type: Double

Specifies the input glitch vh failure threshold value during AAE analysis. The value is the fraction of local VDD – if VDD is 0.9V and the threshold is 0.4, then the criteria for failure is $0.4 * 0.9 = 0.36V$.

During timing window re-convergence, only certain nets are reselected for glitch analysis in each iteration (if any of the above input glitch threshold parameters is specified). The criteria for glitch reselection is based on the specified input glitch threshold value, if this value is smaller than 30% of VDD. If not, then all the glitches larger than 30% of VDD are selected for re-computation in each iteration.

si_glitch_input_voltage_low_threshold

si_glitch_input_voltage_low_threshold double

Default: 0.4

Data_type: Double

Specifies the input glitch vl lower threshold value during AAE analysis. The value is a fraction of the local VDD - if VDD is 0.9V and the threshold value is 0.4, then the criteria for failure is $0.4 * 0.9 = 0.36V$.

During timing window re-convergence, only certain nets are re-selected for glitch analysis in each iteration (if any of the above input glitch threshold parameters is specified). The criteria for glitch re-selection is based on the specified input glitch threshold value, if this value is smaller than 30% of VDD. If not, then all the glitches larger than 30% of VDD are selected for re-computation in each iteration.

si_glitch_receiver_clock_peak_limit

si_glitch_receiver_clock_peak_limit double

Default: 0.05

Data_type: Double

Defines the glitch check limit for latch inputs in receiver peak mode. This parameter is specified as a ratio of VDD.

si_glitch_receiver_latch_peak_limit

si_glitch_receiver_latch_peak_limit double

Default: 0.1

Data_type: Double

Defines the receiver peak/glitch peak limit at all nodes. The value specified for this parameter is a ratio of VDD.

si_glitch_receiver_peak_limit

si_glitch_receiver_peak_limit double

Default: 0.15

Data_type: Double

Defines the receiver peak/glitch peak limit.

si_individual_aggressor_simulation_filter

`si_individual_aggressor_simulation_filter {true | false}`

Default: true

Data_type: Boolean

Uses electrical filtering to reduce pessimism due to small attackers being preserved when their simulated value is under the individual attacker threshold value.

Note: This option can only be used when the accumulated small attacker is turned off (–`accumulated_small_attacker_threshold` is 1.00 or larger).

si_individual_aggressor_threshold

`si_individual_aggressor_threshold double`

Default: 1.5% or 0.015

Data_type: Double

Sets the glitch tolerance for individual attacker nets on the victim net. Tempus estimates the noise contributed by an individual attacker on the victim. If the peak of this noise exceeds the value of `individual_attacker_threshold`, the attacker is significant and is individually reported in the constituents section of the report. Otherwise, this aggressor becomes eligible for the virtual attacker. If the virtual attacker peak does not exceed the value of `accumulated_small_attacker_threshold`, the aggressor nets in the virtual attacker are grounded and there is no virtual attacker.

Note: Minimum allowed individual attacker to be kept is 0.1% of VDD, which is equivalent to an `individual_attacker_threshold` of 0.001.

si_individual_aggressor_clock_threshold

`si_individual_aggressor_clock_threshold double`

Default: 0.015

Data_type: Double

Defines the AAE virtual attacker clock threshold.

si_nonlinear_aggressor_slew

si_nonlinear_aggressor_slew {true | false}

Default: true

Data_type: Boolean

Uses a non-linear attacker. If set to false, the software uses a linear attacker.

si_num_iteration

si_num_iteration int

Default: 2

Data_type: Integer

Specifies the maximum number of timing window iterations that should be performed during SI delay analysis. Performing delay analysis results in changes to the timing windows, which can potentially change the victim/attacker net combinations. Timing window iterations enable the software to reach to a point where the delay changes are negligible.

Note: The minimum allowed timing window iterations is 2 for performing SI analysis.

si_reselection

si_reselection {delta_delay | slack | xcap_ratio}

Default: slack

Data_type: Enum

Specifies the SI reselection criteria during timing window iterations.

- **delta_delay:** Reselects a net for calculation if the delta delay on the net is greater than the delay value.
- **slack:** Reselects nets that have failing slacks during timing window iterations.
- **xcap_ratio:** Specifies the xcap ratio.

si_reselection_delay_threshold

si_reselection_delay_threshold double

Default: 1e-11 (seconds)

Data_type: Double

Allows you to change the delay reselection criteria.

si_reselection_hold_slack

`si_reselection_hold_slack double`

Default: -1e-12 (seconds)

Data_type: Double

Specifies the slack reselection threshold for hold.

si_reselection_setup_slack

`si_reselection_setup_slack double`

Default: -1e-12 (seconds)

Data_type: Double

Specifies the slack reselection threshold for setup.

si_secondary_attacker_decoupling_factor

`si_secondary_attacker_decoupling_factor double`

Default: 1.0

Data_type: Double

Specifies decoupling factor for secondary aggressors.

si_skip_noise_model_check

`si_skip_noise_model_check string`

Default: " "

Data_type: String

Specifies the list of pins for which noise model check will not take place.

si_skip_timing_window

`si_skip_timing_window` *string*

Default: " "

Data_type: String

Specifies the list of nets for which timing window is to be skipped. The timing window becomes infinite for the net when it is both a victim and aggressor.

When specified, only the last setting is retained.

si_switch_probability

`si_switch_probability` *double*

Default: 0.3

Data_type: Double

Sets the attacker switching probability. If you specify a value less than 0.0 or more than 1.0 , the switching probability is automatically set to the default value of 0.3.

si_unconstrained_net_use_infinite_timing_window

`si_unconstrained_net_use_infinite_timing_window` {true | false}

Default: true

Data_type: Boolean

Specifies that unconstrained nets will have an infinite timing window. To turn this off and use a constrained timing window, set this to `false`.

si_use_infinite_timing_window

`si_use_infinite_timing_window` {true | false}

Default: false

Data_type: Boolean

Uses infinite timing windows in a single SI run.

si_virtual_inter_die_attacker_slew

`si_virtual_inter_die_attacker_slew value`

Default: 2.5e-11

Data_type: Float

Specifies the virtual inter-die attacker slew value.

ssyn

The root attribute in the `ssyn` category is:

- [pbs_db_directory](#)
- [pbs_mim_flow](#)

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `ssyn` category, use the following command:

```
get_db -category ssyn
```

pbs_db_directory

`pbs_db_directory string`

Description

Default: .

Data_type: string, read/write

Specifies the directory where partition-based synthesis can write temporary files.

pbs_mim_flow

```
pbs_mim_flow {true | false}
```

Description

Default: false

Data_type: bool, read/write

Enables the Multiple instantiated Modules (MiM) flow inside Partition-Based Synthesis (PBS). Instead of creating a set of unique instances for each module at the start of the flow (by an internal call to the 'uniquify' command), the PBS steps inside the flow, optimizes a single instance for each module, and copies the result to all other instances. So for any module with N instances, the tool optimizes that logic just once instead of N times. This results in a faster runtime for generic and mapped optimization flow steps. There are however a few prerequisites that these MiM have to fulfill. The timing, activity and design constraints must be the same across all instances for a module. If the tool detects differences like unique timing exceptions or dont_touch/dont_scan attributes that do not match, then it uniquifies such instances.

system

The root attributes in the system category are:

- [auto_file_dir](#)
- [auto_file_prefix](#)
- [distributed_mmmc_disable_reports_auto_redirection](#)
- [gui_verbose](#)
- [jedai_catalog_token](#)
- [jedai_catalog_url](#)
- [log_verbose_type](#)
- [read_db_directory](#)
- [read_db_file_check](#)
- [read_db_stop_at_design_in_memory](#)
- [read_db_tool_name](#)
- [read_db_version](#)
- [resolve_show_tcl_command_with_value](#)
- [script_search_path](#)
- [soft_stack_size_limit](#)
- [source_echo_filename](#)
- [source_continue_on_error](#)
- [set_db_verbose](#)
- [source_verbose](#)
- [source_verbose_line_length_limit](#)
- [tcl_error_info](#)
- [tcl_report_header_with_value](#)
- [ui_precision](#)
- [ui_precision_capacitance](#)

- [ui_precision_derating](#)
- [ui_precision_power](#)
- [ui_precision_sensitivities](#)
- [ui_precision_timing](#)
- [write_db_auto_save_user_globals](#)
- [write_db_restrict_under_batch_mode](#)
- [write_db_restrict_under_eco_batch_mode](#)

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `system` category, use the following command:

```
get_db -category system
```

auto_file_dir

```
auto_file_dir directoryName
```

Default: `./`

Specifies the directory to store files/sub-directories 'auto-generated' by the tool. This includes report files or other output files with default names that are automatically created by some commands in the current run directory. Default names that have been overridden with a name chosen by the user, are not affected. This allows multiple simultaneous runs from the same run directory to direct output files to different result directories to avoid collisions.

auto_file_prefix

```
auto_file_dir fileName
```

Default: `""`

Specifies a prefix to be applied to all files 'auto-generated' by the tool. This includes report files or other output files with default names that are automatically created by some commands in the current run directory. So a default name like "cmd.report" will become "<prefix>cmd.report". Default names that have been overridden with a name chosen by the user, are not affected. This allows

multiple simultaneous runs from the same run directory to direct default output files to different locations to avoid collisions.

distributed_mmmc_disable_reports_auto_redirection

```
distributed_mmmc_disable_reports_auto_redirection {true| false}
```

Default: false

Allows reports to be generated in the specified location.

When set to `true`, the software does not allow the reports for different views to get overwritten. When set to `false`, the reports are generated automatically in the DMMMC view directory structure.

gui_verbose

```
gui_verbose {all db none}
```

Default: db

Controls which GUI actions are logged. You can select:

- `all`: Logs all GUI actions.
- `db`: Logs the GUI actions that affect the DB, such as `gui_select`, but does not capture display-only actions such as `gui_dim_foreground`.
- `none`: Does not log any GUI actions. So, the `.log` and `.cmd` files are incomplete.

jedai_catalog_token

```
jedai_catalog_token token
```

Default: ""

Data_type: string

Authenticates and provides access to the OpenAI services.

jedai_catalog_url

```
jedai_catalog_url <jedai_server_url>
```

Default: ""

Data_type: string

Enables the AI chat box in the Tempus GUI. You need to set the following setting in

`tempus_startup.tcl`.

```
#set server url
```

```
set_db jedai_catalog_url <jedai_server_url>
```

log_verbose_type

```
log_verbose_type {pre post both}
```

Default: pre

When verbose logging is enabled, the default logging of each commands is pre-Tcl expansion. In some cases it is helpful to see post-Tcl expansion, or even both pre-Tcl and post-Tcl expansion when doing detailed debugging. You can switch between settings in the middle of a script.

Related topics

- [set_proc_verbose](#)

read_db_directory

```
read_db_directory directory_name
```

Default: ""

Specifies the absolute path of the database, that is used to restore the session. In case of a non-restored session, this variable returns a blank value.

The value of this global variable is set internally by the software.

read_db_file_check

```
read_db_file_check {true | false}
```

Default: true

By default, `read_db` will check all the files inside the saved DB directory. If any file is edited, renamed, or deleted, the tool will give an error. However, `read_db` will not perform this check if `read_db_file_check` is set to `false`.

read_db_stop_at_design_in_memory

```
read_db_stop_at_design_in_memory {true | false}
```

Default: true

By default, tool gives an error and stops if you call `read_db` more than once in the same session unless `read_db_stop_at_design_in_memory` is set to `false`. Using `read_db` twice in one session is not recommended as settings from the first database may not be cleaned up properly and can cause problems in the second database.

read_db_tool_name

```
read_db_tool_name
```

Default: ""

Indicates the tool name of the tool that created the restored db on the disk database.

read_db_version

```
read_db_version
```

Default: ""

Indicates the version of the tool that generates an on disk database.

resolve_show_tcl_command_with_value

```
resolve_show_tcl_command_with_value {true | false}
```

Default: false

Data_type: Boolean, read/write

When set to `true`, it prints the Tcl variable value in the resolve log.

script_search_path

```
script_search_path search_directory_path
```

Default: "."

Provides a set of directories for the software to search for files that are sourced using the

Tcl `source` command. The software will search in each search path directory for the specified file.

soft_stack_size_limit

```
soft_stack_size_limit size_limit
```

Default: 8

Specifies the soft stack size limit in mbytes. When the tool is launched, the soft stack size limit is auto-enlarged to 0.2% RAM. If 0.2%RAM is larger than the hard limit, the tool sets it to the hard limit.

source_echo_filename

```
source_echo_filename {true| false}
```

Default: false

Displays the file name being sourced before the actual sourcing.

If this is true, and verbose 'source' command logging is off (e.g. `source_verbose` is false), any 'source' command will be logged with 'Sourcing <filename>'. This can be useful to find which source files are calling other source files, or to log all the startup Tcl files that are loaded even though verbose logging is off.

source_continue_on_error

```
source_continue_on_error {true| false}
```

Default: false

Enables or disables `source` to continue the script on `TCL_ERROR`. If `true`, the `source` command will ignore Tcl errors, and continue processing the script files. If `false`, a Tcl error will halt the script.

set_db_verbose

```
set_db_verbose {true| false}
```

Default: false

Specifies additional messages are issued describing what attribute is being set on what object.

source_verbose

`source_verbose {true| false}`

Default: `true`

Data_type: `bool`

Controls whether scripts are sourced in quiet or verbose mode:

- By default (`true`), files are sourced in verbose mode. This is equivalent to `source filename -verbose`.
- If you set this variable to `false`, files are sourced in quiet mode.

source_verbose_line_length_limit

`source_verbose_line_length_limit value`

Default: `-1`

Data_type: `int`

Limits the command printed to stdout and the .log file to the specified number of characters for verbose script source. This is normally only needed if you log commands "post Tcl expansion" when long Tcl lists may occur in the command arguments.

tcl_error_info

`tcl_error_info {true| false}`

Default: `false`

Data_type: `bool`, read/write

When set to `true`, the `tcl_error_info` prints the error information if there is an issue in the Tcl script.

tcl_report_header_with_value

`tcl_report_header_with_value {true| false}`

Default: `false`

Data_type: `boolean`, read/write

When set to `true`, prints the Tcl variable value in the report header.

Example

In the following example, `tcl_report_header_with_value` is set to `true`, the report shows the value, `myct`, set for the variable in the report header.

```
[DEV]tempus 8> report_analysis_coverage -check_type \
+      $myct

#####
#  Generated by:      Cadence Innovus 24.10-dev.jinchuan
#  OS:               Linux x86_64 (Host ID sjfib902)
#  Generated on:      Wed Jan 17 23:22:11 2024
#  Design:           DTMF_CHIP
#  Command:          report_analysis_coverage -check_type \
#      $myct
#####
+-----+
|          TIMING CHECK COVERAGE SUMMARY          |
+-----+
| Check | No. of |    Met    | Violated | Untested |
| Type  | Checks |           |           |           |
+-----+-----+-----+-----+-----+
| Setup |  1821  | 219 (12%) |  0 (0%)  | 1602 (87%) |
+-----+-----+-----+-----+-----+
```

If the `tcl_report_header_with_value` is set to `false`, the report will show the name of the variable in the report and not its value.

```
[DEV]tempus 9> report_analysis_coverage -check_type \
+      $myct
#####
#  Generated by:      Cadence Innovus 24.10-dev.jinchuan
#  OS:               Linux x86_64 (Host ID sjfib902)
#  Generated on:      Wed Jan 17 23:22:20 2024
#  Design:           DTMF_CHIP
#  Command:          report_analysis_coverage -check_type \
#      setup
#####
```

ui_precision

`ui_precision` *number*

Default: 3

Specifies the number of significant digits to be displayed in the absence of an explicit precision for that type.

ui_precision_capacitance

`ui_precision_capacitance` *number*

Default: 3

Specifies the number of significant digits to be displayed for quantities of type capacitance.

ui_precision_derating

`ui_precision_derating` *number*

Default: 3

Specifies the number of significant digits to be displayed for derating factors such as those specified via `set_timing_derate` constraints or AOCV derating libraries.

ui_precision_power

`ui_precision_power` *number*

Default: 3

Specifies the number of significant digits to be displayed for quantities of type power.

ui_precision_sensitivities

`ui_precision_sensitivities` *number*

Default: 3

Specifies the number of significant digits to be displayed for statistical sensitivity values and SOCV sigma values.

ui_precision_timing

`ui_precision_timing` *number*

Default: 3

Specifies the number of significant digits to be displayed for delay type values including cell and net delays, transitions, arrival and required times, and slacks.

write_db_auto_save_user_globals

`write_db_auto_save_user_globals` {true| false}

Default: true

If true, `write_db` automatically saves the Tcl global variables created by the user. User globals are defined as any “info globals” names that do not exist at startup (such as created by the user executed code).

You can use the `define_variables` command to add or delete specific Tcl variables from the automatic save list.

write_db_restrict_under_batch_mode

`write_db_restrict_under_batch_mode` {0| 1}

Default: 0

Data_type: bool, read/write

Controls whether to save a database under the batch mode. By default, a new database is saved under the batch mode. However, if `write_db_restrict_under_batch_mode` is set to true, the tool gives an error.

write_db_restrict_under_eco_batch_mode

`write_db_restrict_under_eco_batch_mode` {true | false}

Default: false

Data_type: bool, read/write

Controls whether to restrict saving the database in the ECO batch mode.

tech

The root attributes in the `tech` category are:

- `tech_db_units`
- `tech_finfet_grid_direction`
- `tech_finfet_grid_offset`
- `tech_finfet_grid_pitch`
- `tech_inst_mask_shift_layers`
- `tech_mfg_grid`

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `tech` category, use the following command:

```
get_db -category tech
```

tech_db_units

```
tech_db_units int
```

Default: ""

Data_type: Instance

Specifies the database units per micron from LEF UNITS statement or OA techfile, unless `init_min_dbu_per_micron` is larger and overrides the LEF/OA value.

tech_finfet_grid_direction

```
tech_finfet_grid_direction enum
```

Default: vertical

Data_type: Enum

Specifies an enum value indicating the direction of the finfet grid from the LEF FINFET statement or OA.

tech_finfet_grid_offset

`tech_finfet_grid_offset coord`

Default: `no_value`

Data_type: [Coordinate](#)

Specifies the finfet grid offset from 0 in microns from the LEF FINFET statement or OA.

tech_finfet_grid_pitch

`tech_finfet_grid_pitch coord`

Default: `no_value`

Data_type: [Coordinate](#)

Specifies the finfet grid pitch in microns from the LEF FINFET statement or OA.

tech_inst_mask_shift_layers

`tech_inst_mask_shift_layers layer*`

Default: `" "`

Data_type: [Layer](#)

Specifies the ordered list of layer pointers to the layers that allow instance mask shifting. This is the list of layers that show up in the DEF COMPONENTMASK statement for layers that can have mask values shifted in standard cells.

tech_mfg_grid

`tech_mfg_grid coord`

Default: `" "`

Data_type: [Coordinate](#)

Specifies the manufacturing grid from LEF MANUFACTURINGGRID statement or OA techfile.

Note: This is the default value for each layer, but it can be overridden for specific layers. In some technologies, the `mfg_grid` for higher metal layers is coarser than for the lower layers.

timing

- [timing_all_registers_filter_clock_pins_by_clock](#)
- [timing_all_registers_identify_macros_include_is_macro_cell](#)
- [timing_all_registers_include_icg_cells](#)
- [timing_allow_input_delay_on_clock_source](#)
- [timing_analysis_aocv](#)
- [timing_analysis_aging](#)
- [timing_analysis_async_checks](#)
- [timing_analysis_check_type](#)
- [timing_analysis_clock_gating](#)
- [timing_analysis_clock_propagation_mode](#)
- [timing_analysis_clock_source_paths](#)
- [timing_analysis_cppr](#)
- [timing_analysis_engine](#)
- [timing_analysis_enable_inter_power_domain](#)
- [timing_analysis_enable_robustness_mode](#)
- [timing_analysis_graph_pba_mode](#)
- [timing_analysis_multi_input_switching_mode](#)
- [timing_analysis_precision_ps](#)
- [timing_analysis_self_loops_paths_no_skew](#)
- [timing_analysis_socv](#)
- [timing_analysis_type](#)
- [timing_aocv_analysis_mode](#)
- [timing_aocv_chip_size](#)

- `timing_aocv_core_size`
- `timing_aocv_derate_mode`
- `timing_aocv_stage_count_update_on_timing_reset`
- `timing_apply_check_derate_to_external_output_delay`
- `timing_apply_default_primary_input_assertion`
- `timing_apply_exceptions_to_data_check_related_pin`
- `timing_apply_setup_hold_exceptions_to_data_check_related_pin`
- `timing_cap_unit`
- `timing_case_analysis_for_icg_propagation`
- `timing_case_analysis_for_sequential_propagation`
- `timing_case_analysis_propagation`
- `timing_check_timing_report_all_checks`
- `timing_check_timing_signal_level_high_to_low_threshold`
- `timing_check_timing_signal_level_low_to_high_threshold`
- `timing_clock_phase_propagation`
- `timing_clock_source_paths_unconstrained_mark_clock_used_as_data`
- `timing_clock_source_use_driving_cell`
- `timing_clock_uncertainty_from_to_precedence`
- `timing_collection_all_fanin_fanout_traversal_mode`
- `timing_collection_result_display_limit`
- `timing_collection_variable_assignment_compatibility`
- `timing_context_apply_default_primary_input_assertion`
- `timing_context_apply_port_sdc_exceptions`
- `timing_context_clock_mapping_percentage_tolerance`
- `timing_context_clock_mapping_check_waveform`
- `timing_context_continue_on_invalid_module_list`
- `timing_context_clock_phase_based_clock_mapping`

- `timing_context_data_phase_based_clock_mapping`
- `timing_context_port_based_clock_mapping`
- `timing_consider_late_arrival_for_ssi_derate`
- `timing_constraint_disable_min_max_input_delay_worst_casing`
- `timing_constraint_enable_detailed_report_invalid_begin_end_points`
- `timing_constraint_enable_report_invalid_begin_end_points`
- `timing_constraint_enable_search_path`
- `timing_constraint_enable_separate_multicycle_data_checks`
- `timing_constraint_enable_logging`
- `timing_constraint_load_minimal_set_for_automated_eco`
- `timing_constraint_path_delay_exclude_check_delay_from_ignore_clock_latency`
- `timing_constraint_path_delay_exclude_io_delay_from_ignore_clock_latency`
- `timing_constraint_path_delay_exclude_unconstrained_endpoints`
- `timing_constraint_path_delay_include_clock_pin_endpoints`
- `timing_constraint_warn_for_timing_derate_exceeding_max_limit`
- `timing_continue_on_error`
- `timing_constraint_update_io_latency_averaging_mode`
- `timing_cpnr_opposite_edge_mean_scale_factor`
- `timing_cpnr_opposite_edge_sigma_scale_factor`
- `timing_cpnr_opposite_edge_sigma_scale_factor_net`
- `timing_cpnr_opposite_edge_sigma_scale_factor_cell`
- `timing_cpnr_propagate_thru_latches`
- `timing_cpnr_remove_clock_to_data_pessimism`
- `timing_cpnr_self_loop_mode`
- `timing_cpnr_skip_clock_reconvergence`
- `timing_cpnr_skip_clock_reconvergence_for_unmatched_clocks`
- `timing_cpnr_threshold_ps`

- [timing_cppr_transition_sense](#)
- [timing_create_clock_default_propagated](#)
- [timing_default_opcond_per_lib](#)
- [timing_defer_mmmc_obj_updates](#)
- [timing_derate_aocv_dynamic_delays](#)
- [timing_derate_aocv_reference_point](#)
- [timing_derate_dynamic_compatibility](#)
- [timing_derate_incremental_adjust_additive_mode](#)
- [timing_derate_enable_incremental_multiplication_mode](#)
- [timing_derate_incremental_multiply_accumulative_mode](#)
- [Examples](#)
- [timing_derate_ocv_reference_point](#)
- [timing_derate_negative_delay_backward_compatibility](#)
- [timing_derate_spatial_distance_unit](#)
- [timing_derate_voltage_scaling_mode](#)
- [timing_disable_bus_contention_check](#)
- [timing_disable_clock_period_checks](#)
- [timing_disable_constant_propagation_for_sequential_cells](#)
- [timing_disable_drv_report_on_constant_nets](#)
- [timing_disable_floating_bus_check](#)
- [timing_disable_get_clocks_for_current_instance](#)
- [timing_disable_genclk_combinational_blocking](#)
- [timing_disable_inferred_clock_gating_checks](#)
- [timing_disable_inout_output_side_timing_checks](#)
- [timing_disable_internal_inout_cell_paths](#)
- [timing_disable_internal_inout_net_arcs](#)
- [timing_disable_lib_pulse_width_checks](#)

- [timing_disable_library_data_to_data_checks](#)
- [timing_disable_library_tieoffs](#)
- [timing_disable_netlist_constants](#)
- [timing_disable_model_parasitics](#)
- [timing_disable_nochange_checks](#)
- [timing_disable_non_sequential_checks](#)
- [timing_disable_output_as_clock_port](#)
- [timing_disable_parallel_arcs](#)
- [timing_disable_pulse_width_same_edge_si_cpvr_mode](#)
- [timing_disable_report_header_info](#)
- [timing_disable_retime_clock_path_slew_propagation](#)
- [timing_disable_sdf_retain_arc_merging](#)
- [timing_disable_skew_checks](#)
- [timing_disable_test_signal_arc](#)
- [timing_max_transition_use_si_transition](#)
- [timing_model_netlist_exclude_dummy](#)
- [timing_disable_timing_model_latch_inferencing](#)
- [timing_disable_tristate_disable_arcs](#)
- [timing_disable_user_data_to_data_checks](#)
- [timing_driving_cell_override_library](#)
- [timing_drv_analysis_ignore_latency_phases](#)
- [timing_enable_aocv_slack_based](#)
- [timing_enable_case_analysis_conflict_warning](#)
- [timing_enable_clock_to_clock_clock_gating_check](#)
- [timing_enable_clock_phase_based_rise_fall_derating](#)
- [timing_enable_data_through_clock_gating](#)
- [timing_enable_db_attribute_query](#)

- timing_enable_derating_for_pulse_width_checks
- timing_enable_early_late_data_slews_for_setuphold_mode_checks
- timing_enable_genclk_divide_by_inherit_parent_duty_cycle
- timing_enable_genclk_source_path_register_limit
- timing_enable_generated_clock_edge_based_source_latency
- timing_enable_get_obj_escaped_name_backward_compatible
- timing_enable_get_objects_regexp_compatibility
- timing_enable_hier_context_unmapped_clock_analysis
- timing_enable_hierarchical_get_nets_support
- timing_enable_individual_distance_inter_die_analysis
- timing_enable_latch_thru_mode
- timing_enable_latency_through_clock_gating
- timing_enable_mmmc_loop_breaking
- timing_enable_model_constraint_warnings
- timing_enable_multi_drive_net_reduction_with_assertions
- timing_enable_multi_frequency_latch_analysis
- timing_enable_multicycle_data_check_compatibility
- timing_enable_path_delay_to_unconstrained_endpoints_compatibility
- timing_enable_pessimistic_cpvr_for_reconvergent_clock_paths
- timing_enable_power_ground_constants
- timing_enable_preset_clear_arcs
- timing_enable_pulse_latch
- timing_enable_si_cpvr
- timing_enable_simultaneous_setup_hold_mode
- timing_enable_spatial_derate_mode
- timing_enable_timing_window_pessimism_removal
- timing_enable_tristate_clock_gating

- [timing_enable_uncertainty_for_clock_checks](#)
- [timing_enable_uncertainty_for_pulse_width_checks](#)
- [timing_enable_vtskew_derate_mode](#)
- [timing_enable_wire_variation](#)
- [timing_enable_zero_delay_analysis_mode](#)
- [timing_extract_model_aocv_mode](#)
- [timing_extract_model_case_analysis_in_library](#)
- [timing_extract_model_check_arcs_as_lvf](#)
- [timing_extract_model_consider_design_level_drv](#)
- [timing_extract_model_disable_cycle_adjustment](#)
- [timing_extract_model_enable_combinational_arc_to_clock_source_on_output_ports](#)
- [timing_extract_model_enable_output_pin_attributes](#)
- [timing_extract_model_enable_unconstrained_port_modelling](#)
- [timing_extract_model_exhaustive_validation_dir](#)
- [timing_extract_model_exhaustive_validation_mode](#)
- [timing_extract_model_gating_as_nochange_arc](#)
- [timing_extract_model_half_cycle_non_interface_paths](#)
- [timing_extract_model_ideal_clock_latency_arc](#)
- [timing_extract_model_include_applied_load_in_characterization_range](#)
- [timing_extract_model_include_applied_slew_in_characterization_range](#)
- [timing_extract_model_non_borrowing_latch_path_as_setup](#)
- [timing_extract_model_slew_propagation_mode](#)
- [timing_extract_model_synchronous_clocks_as_skew_checks](#)
- [timing_extract_model_write_clock_checks_as_arc](#)
- [timing_extract_model_write_clock_checks_as_scalar_tables](#)
- [timing_extract_model_write_function_for_combinational_arcs](#)
- [timing_extract_model_write_min_max_clock_tree_path](#)

- `timing_extract_model_write_lvf`
- `timing_full_context_flow`
- `timing_generate_normalized_driver_waveform`
- `timing_generated_clocks_inherit_ideal_latency`
- `timing_generated_clocks_allow_nested_assertions`
- `timing_get_of_objects_hier_compatibility`
- `timing_hierarchical_context_continue_on_top_block_clock_mismatch`
- `timing_hier_obj_name_compatibility`
- `timing_ignore_clock_based_path_delay_on_data_checks`
- `timing_ignore_lumped_rc_assertions`
- `timing_io_use_clock_network_latency`
- `timing_ipd_allow_paths_for_asynchronous_clocks`
- `timing_ipd_xdomain_path_filtering_threshold`
- `timing_library_build_async_deassert_arc`
- `timing_library_convert_async_setuphold_to_recrem`
- `timing_library_generated_clock_use_group_name`
- `timing_library_hold_constraint_corner_sigma_multiplier`
- `timing_library_hold_sigma_multiplier`
- `timing_library_ignore_combinational_arc_mismatch_tables`
- `timing_library_infer_async_pins_from_timing_arcs`
- `timing_library_infer_cap_range_from_ccs_receiver_model`
- `timing_library_infer_cap_range_from_ecsm_receiver_model`
- `timing_library_interpolate_drv_values`
- `timing_library_infer_socv_from_aocv`
- `timing_library_read_ccs_noise_data`
- `timing_library_read_without_ecsm`
- `timing_library_read_without_power`

- `timing_library_read_without_sensitivity`
- `timing_library_scale_aocv_to_socv_to_n_sigma`
- `timing_library_setup_constraint_corner_sigma_multiplier`
- `timing_library_setup_sigma_multiplier`
- `timing_library_term_voltage_from_lib_pin`
- `timing_library_use_two_piece_receiver_cap`
- `timing_library_zero_negative_timing_check_arcs`
- `timing_multi_frequency_clock_rounding_factor`
- `timing_normalized_driver_waveform_clip_linear_part`
- `timing_normalized_driver_waveform_weight_factor`
- `timing_nsigma_multiplier`
- `timing_null_collection_return_compatibility`
- `timing_path_based_enable_bounding_box_for_io_paths`
- `timing_path_based_enable_closest_common_pin_driver_for_bounding_box`
- `timing_path_based_epba_bounded_mode`
- `timing_path_based_enable_report_launch_clock_path`
- `timing_path_based_enable_verbose_mode`
- `timing_path_based_enable_exhaustive_depth_bounded_by_gba`
- `timing_path_based_exhaustive_enable_design_coverage`
- `timing_path_based_exhaustive_max_paths_limit`
- `timing_path_based_ipba_endpoints_max_limit`
- `timing_pba_exhaustive_path_nworst_limit`
- `timing_path_based_report_analysis_summary_max_paths_limit`
- `timing_pll_clock_use_driving_cell`
- `timing_prefix_module_name_with_library_generated_clock`
- `timing_propagate_latch_data_uncertainty`
- `timing_property_arrival_window_enable_tcl_dict_format`

- `timing_property_arrival_clocks_consider_clock_source_data_phase`
- `timing_property_arrival_clocks_consider_data_phase`
- `timing_property_arrival_clocks_consider_latency_phase`
- `timing_property_return_null_collection_with_quiet`
- `timing_rail_swing_checks_high_voltage_threshold`
- `timing_rail_swing_checks_low_voltage_threshold`
- `timing_rcdb_allow_mismatch_option`
- `timing_recompute_sdf_in_setuphold_mode`
- `timing_reduce_multi_drive_net_arcs`
- `timing_reduce_multi_drive_net_arcs_threshold`
- `timing_report_analysis_summary_csv_extended_new_format`
- `timing_report_arrival_property_worstcase_mode`
- `timing_report_begin_end_pair_max_path_limit`
- `timing_report_check_timing_unconstrained_endpoints_due_to_constants`
- `timing_report_clock_pin_as_begin_point`
- `timing_report_constraint_extended_cell_em_flow`
- `timing_report_constraint_enable_extended_drv_format`
- `timing_report_constraint_format`
- `timing_report_constraint_rise_fall_clock_period_check`
- `timing_report_constraint_report_all_drv_violation`
- `timing_report_constraint_use_infinity_slack_for_unconstrained`
- `timing_report_default_frequency_for_unconstrained_nets`
- `timing_report_disable_max_paths_per_group`
- `timing_report_drv_enable_clock_source_as_clock`
- `timing_report_drv_enable_frequency_per_view`
- `timing_report_drv_per_frequency_per_input_slew`
- `timing_report_drv_enable_ghz_notation_for_drv_fields`

- timing_report_drv_enable_slew_threshold_scaling
- timing_report_drv_per_frequency
- timing_report_drv_use_worst_timing_slack
- timing_report_enable_cppr_point
- timing_report_enable_em_index_clipping_report
- timing_report_enable_flag_field_symbols
- timing_report_enable_clock_to_for_unconstrained_paths
- timing_report_enable_lead_trail_to_rise_fall_map
- timing_report_enable_markers
- timing_report_enable_max_capacitance_drv_for_constant_nets
- timing_report_enable_max_path_limit_warning
- timing_report_enable_qpin_as_start_for_begin_end_pair
- timing_report_enable_report_clock_timing_across_clock_pin
- timing_report_enable_si_debug
- timing_report_enable_verbose_ssta_mode
- timing_report_enable_unique_pins_multiple_capture_clock_paths
- timing_report_enable_write_twf_third_party_for_input_pins
- timing_report_fields
- timing_report_generated_clock_info
- timing_report_group_based_mode
- timing_report_limit_max_path_count
- timing_report_max_transition_check_using_nsigma_slew
- timing_report_precision
- timing_report_property_fastest_clock_consider_data_phase
- timing_report_pulse_width_matching_launch_capture_paths
- timing_report_redirect_message_types
- timing_report_retime_formatting_mode

- [timing_report_skip_constraint_loop_check](#)
- [timing_report_skip_icg_from_all_fanin_fanout_collection](#)
- [timing_report_socv_summary_mean_sigma](#)
- [timing_report_timing_header_detail_info](#)
- [timing_report_unconstrained_paths](#)
- [timing_report_unconstrained_path_early_late_header](#)
- [timing_report_use_receiver_model_capacitance](#)
- [timing_report_use_worst_parallel_cell_arc](#)
- [timing_resolve_driver_conflicts](#)
- [timing_scaling_for_negative_checks](#)
- [timing_scaling_for_negative_delays](#)
- [timing_sdf_adjust_negative_setuphold](#)
- [timing_sdf_enable_setuphold_scond_ccond](#)
- [timing_self_loop_paths_no_skew_max_depth](#)
- [timing_self_loop_paths_no_skew_max_slack](#)
- [timing_set_clock_source_to_output_as_data](#)
- [timing_socv_preserve_variation_with_annotations](#)
- [timing_socv_rc_variation_mode](#)
- [timing_socv_sigma_spatial_multiplier](#)
- [timing_socv_view_based_nsigma_multiplier_mode](#)
- [timing_spatial_derate_chip_size](#)
- [timing_spatial_derate_distance_mode](#)
- [timing_suppress_escape_characters](#)
- [timing_suppress_ilm_constraint_mismatches](#)
- [timing_time_unit](#)
- [timing_use_clock_pin_attribute_for_clock_net_marking](#)
- [timing_use_latch_early_launch_edge](#)

- [timing_use_incremental_si_transition](#)
- [timing_use_latch_time_borrow](#)
- [timing_use_verilog_for_model_netlist](#)
- [timing_waveform_aware_pulse_width_checks_high_voltage_level](#)
- [timing_waveform_aware_pulse_width_checks_low_voltage_level](#)
- [timing_write_sdf_no_escape_backslash](#)

To set the value of any root attribute, use the [set_db](#) command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the [get_db](#) command:

```
get_db attribute_name
```

To get the current value of all attributes in the [check_drc](#) category, use the following command:

```
get_db -category check_drc
```

timing_all_registers_filter_clock_pins_by_clock

```
timing_all_registers_filter_clock_pins_by_clock {true | false}
```

Default: false

Data_type: Boolean

Reports clock pins filtered based on specified clocks.

When set to `true`, the software will filter and return only those clock pins where at least one clock is from the specified list of clocks.

This attribute applies only when any of the [all_registers](#) command parameters, `-clock`, `-rise_clock`, `-fall_clock`, is specified.

timing_all_registers_identify_macros_include_is_macro_cell

```
timing_all_registers_identify_macros_include_is_macro_cell {true | false}
```

Default: true

Data_type: Boolean

Enables to include cells as macro cells.

When set to `true`, the software includes cells as macro cells if the `is_macro_cell` property (see [get_property](#)) is set to `true` in the library.

When set to `false`, the `all_registers` command will not return cells for which the `is_macro_cell` property is `true`.

timing_all_registers_include_icg_cells

```
timing_all_registers_include_icg_cells {true | false}
```

Default: `true`

Data_type: Boolean

Includes integrated clock gating (ICG) cell information from the `all_registers` report output.

When set to `false`, the attribute does not allow the `all_registers` command report output to display ICG cell details.

timing_allow_input_delay_on_clock_source

```
timing_allow_input_delay_on_clock_source {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, this attribute allows the software to apply input delay constraints on a pin where a clock was previously asserted.

timing_analysis_aocv

```
timing_analysis_aocv {true | false}
```

Default: `false`

Data_type: Boolean

Sets the analysis mode to advanced on-chip variation (AOCV) analysis.

timing_analysis_aging

```
timing_analysis_aging {true | false}
```

Default: false

Data_type: Boolean

Enables timing-aware aging analysis.

The aging feature must be enabled before the timing updates are done.

Note: You cannot disable this feature in the same session later.

timing_analysis_async_checks

```
timing_analysis_async_checks {async | no_async | async_only | auto}
```

Default: auto

Data_type: Enum

Reports timing violations for asynchronous timing checks, including recovery and removal, and other checks on pins defined as preset or clear in the Liberty file.

The following options are supported:

- **async**: Reports timing checks on pins that are defined as preset or clear in the Liberty file.
- **no_async**: Reports timing violations while ignoring the two time arc attributes.
- **async_only**: Reports only recovery and removal checks.

The following setting corresponds to `set timing_disable_recovery_removal_checks true` in Tempus legacy.

```
set_db timing_analysis_async_checks no_async
```

The following setting corresponds to `set timing_disable_recovery_removal_checks false` in Tempus legacy.

```
set_db timing_analysis_async_checks async
```

timing_analysis_check_type

```
timing_analysis_check_type {setup | hold}
```

Default: setup

Data_type: Enum

Checks the design for setup or hold violations in the current analysis.

Note: When simultaneous setup/hold mode analysis is on, this setting does not have any impact. You can use the `report_timing -late/-early` parameter to report setup/hold timing reports.

timing_analysis_clock_gating

```
timing_analysis_clock_gating {true | false}
```

Default: true

Data_type: Boolean

Enables clock gating checks.

timing_analysis_clock_propagation_mode

```
timing_analysis_clock_propagation_mode {sdc_control | forced_ideal}
```

Default: sdc_control

Data_type: Enum

Specifies how to determine whether clock end points are interpreted as having either ideal or propagated clock timing.

- `sdc_control`: Reports the timing violations considering clock tree and enables propagating clocks to report clock latency. You must set the `set_propagated_clock` constraint to propagate the clocks.

Note: By default, CTS will instruct the timer to make all of the clock roots propagated. In this

case, you do not need to set the `set_propagated_clock` constraint.

- `forced_ideal`: Does not report the timing violations considering the clock tree.

timing_analysis_clock_source_paths

```
timing_analysis_clock_source_paths {true | false}
```

Default: `true`

Data_type: Boolean

Reports combinational paths from the clock source to the timing checks on data pins.

When set to `false`, the clock portion of the path is considered as ideal (due to the SDC constraints or the `timing_analysis_clock_propagation_mode forced_ideal` attribute setting), the delay calculation software treats the clock nets as ideal nets when calculating the delays and slews along the data path.

timing_analysis_cppr

```
timing_analysis_cppr {both | none | setup | hold}
```

Default: `none`

Data_type: Enum

Removes pessimism from clock paths that have a portion of the clock network in common between the clock source and clock destination paths.

The pessimism is introduced when the timing analysis tools assume that the common path has different delay values for two different paths in case of on-chip variation. The pessimism can also be because of clock reconvergence in the clock network. The pessimism effects all analysis modes including single, best case-worst case (bcwc), and on-chip variation.

- `none`: Disables removal of clock reconvergence pessimism.
- `both`: Enables removal of clock reconvergence pessimism for both setup and hold modes.
- `setup`: Enables removal of clock reconvergence pessimism in setup mode only.
- `hold`: Enables removal of clock reconvergence pessimism in hold mode only.

timing_analysis_engine

```
timing_analysis_engine {statistical | static}
```

Default: static

Data_type: Enum

Allows you to specify the analysis engine - Statistical or Static - for performing timing analysis.

timing_analysis_enable_inter_power_domain

```
timing_analysis_enable_inter_power_domain {true | false}
```

Default: false

Data_type: Boolean

Enables IPD analysis.

For more information, refer to the [Inter Power Domain \(IPD\) Analysis](#) chapter in the *Tempus Stylus User Guide*.

timing_analysis_enable_robustness_mode

```
timing_analysis_enable_robustness_mode {true | false}
```

Default: false

Data_type: Boolean

Enables PTR analysis.

The Parametric Timing Robustness (PTR) analysis calculates the probability of an entire chip meeting timing requirements, instead of relying on the probabilities on individual paths to compute the robustness of a chip.

timing_analysis_graph_pba_mode

```
timing_analysis_graph_pba_mode {true | false}
```

Default: false

Data_type: Boolean

When specified, the software performs partial PBA analysis during GBA to reduce GBA pessimism.

timing_analysis_multi_input_switching_mode

timing_analysis_multi_input_switching_mode *int*

Default: 0

Data_type: Integer

Sets the multiple-input switching (MIS) mode.

You can specify one of the following options below:

- 0: Simultaneous input switching is not considered.
- 1: MIS analysis considers cells for which multiple inputs of a cell are tied together (tied-inputs).
- 2: MIS analysis considers both the tied-inputs and inputs of cells switching independent to each other.
- 3: Enables derate-based MIS. When MIS-derate is enabled, Tempus will automatically disable AMIS-simulation.

timing_analysis_precision_ps

timing_analysis_precision_ps {0.1 | 0.01}

Default: 0.01

Data_type: Double

Allows you to set the delay precision value (in 0.1ps or 0.01ps) for the `write_sdf` command.

The `write_sdf` command shows a double digit value after the decimal point by default. You can use this attribute for more accuracy.

timing_analysis_self_loops_paths_no_skew

```
timing_analysis_self_loops_paths_no_skew {true | false}
```

Default: false

Data_type: Boolean

Checks for self loop paths without skew.

When set to `true`, the software eliminates clock skew due to clock uncertainty for a path starting and ending at the same register. If the clock skew is not eliminated, the timing for such paths is pessimistic.

timing_analysis_socv

```
timing_analysis_socv {true | false}
```

Default: false

Data_type: Boolean

Enables SOCV Analysis.

timing_analysis_type

```
timing_analysis_type {single | best_case_worst_case | ocv}
```

Default: ocv

Data_type: Enum

Sets the timing analysis type to single, best case worst case or on-chip variation.

In this mode, the software calculates the delay for one path based on maximum operating condition while calculating the delay for another path based on minimum operating condition for setup or hold checks.

- `single`: Scales the delay values based on one operating condition.
- `best_case_worst_case`: Checks the design for two extreme operating conditions. The software uses the maximum delays for all paths during setup checks and minimum delays for all paths

during hold checks.

- `ocv`: Calculates the delay for one path based on maximum operating condition while calculating the delay for another path based on minimum operating condition for setup or hold checks.

If you read in one library set and do not specify any timing analysis type, the software uses `single` by default. If you read in two library sets and do not specify any timing analysis type, the software uses `best_case_worst_case` by default.

For more information, see [Timing Analysis Modes](#) in the *Tempus User Guide*.

timing_aocv_analysis_mode

```
timing_aocv_analysis_mode {launch_capture | clock_only | separate_data_clock |  
combine_launch_capture}
```

Default: `launch_capture`

Data_type: Enum

Controls handling of AOCV analysis modes.

- `launch_capture`: AOCV analysis will be performed for both data and clock paths, and the AOCV derating factor will be calculated based on both clock and data path stages.
- `clock_only`: AOCV analysis will be performed for clock paths only, and the AOCV derating factor will be calculated based on the clock path stage depth only.
- `separate_data_clock`: The software calculates the AOCV stage count for clock paths and data paths separately. Both clock and data paths have related AOCV derating factors.
- `combine_launch_capture`: The software calculates AOCV stage count using combined depth for launch and capture paths. This is achieved by summation of path depth on the launch and capture paths.

timing_aocv_chip_size

```
timing_aocv_chip_size double
```

Default: `1e+30`

Data_type: double

Specifies the diagonal length of the chip, in microns. The software will automatically calculate the length from the SPEF file, if it is loaded.

The chip and core sizes specified with the `timing_aocv_chip_size` and `timing_aocv_core_size` attributes take priority over chip and core size values derived from the SPEF file, or through placement. When the SPEF file is not loaded, or placement information is not available, then the following cases are considered:

- If you specify `timing_aocv_chip_size` but not `timing_aocv_core_size`, the software considers the core size to be equal to the `timing_aocv_chip_size` value.
- If you specify `timing_aocv_core_size` but not `timing_aocv_chip_size`, the software ends the AOCV analysis and generates an error message.
- If both `timing_aocv_chip_size` and `timing_aocv_core_size` are not specified, the software ends the AOCV analysis and generates an error message.

timing_aocv_core_size

`timing_aocv_core_size` *real_number*

Default: 1e+30

Data_type: double

Specifies the diagonal length of the core area, in microns. The software will automatically calculate the length from the SPEF file, if it is loaded.

The chip and core sizes specified with the `timing_aocv_chip_size` and `timing_aocv_core_size` attributes take priority over chip and core size values derived from the SPEF file, or through placement. When the SPEF file is not loaded, or placement information is not available, then the following cases are considered:

- If you specify `timing_aocv_chip_size` but not `timing_aocv_core_size`, the software considers the core size to be equal to the `timing_aocv_chip_size` value.
- If you specify `timing_aocv_core_size` but not `timing_aocv_chip_size`, the software ends the AOCV analysis and generates an error message.
- If both `timing_aocv_chip_size` and `timing_aocv_core_size` are not specified, the software ends the AOCV analysis and generates an error message.

timing_aocv_derate_mode

```
timing_aocv_derate_mode {aocv_multiplicative | aocv_additive}
```

Default: aocv_multiplicative

Data_type: Enum

Controls the AOCV derating mode.

When set to aocv_multiplicative, the derating factor will be calculated as AOCV derating * OCV derating, which is set using the [set_timing_derate](#) command.

When set to aocv_additive, the derating factor will be calculated as AOCV derating + OCV derating values.

When you use this attribute, the [report_timing](#) command shows the `total_derate` column in the timing report output, which allows you to view and cross-check the calculated total derate factor.

timing_aocv_stage_count_update_on_timing_reset

```
timing_aocv_stage_count_update_on_timing_reset {true | false}
```

Default: false

Data_type: Boolean

When set to true, the AOCV stage counts are re-calculated when timing is reset.

timing_apply_check_derate_to_external_output_delay

```
timing_apply_check_derate_to_external_output_delay {true | false}
```

Default: false

Data_type: Boolean

Applies check derate to output external delay.

The [set_timing_derate](#) -cell_check parameter by default does not apply check derate to output external delay (using the [set_output_delay](#) command). To enable this feature, you can set the `timing_apply_check_derate_to_external_output_delay` timing attribute to `true`. The attribute

applies cell (check) derate to the top boundary port.

timing_apply_default_primary_input_assertion

```
timing_apply_default_primary_input_assertion {true | false}
```

Default: true

Data_type: Boolean

When set to `true`, primary input and bidirectional ports that do not have an explicit arrival time specified (through the [set_input_delay](#) constraint), are provided a default arrival time. The arrival is referenced to an internal default clock that is indicated by the @ sign in various timing reports.

If a `set_input_delay` constraint is assigned to the input port, but the `-clock` parameter was not specified, the software assumes the constraint to be with reference to the default clock.

When set to `false`, inputs ports with no explicit `set_input_delay` are unconstrained for sequential timing analysis.

timing_apply_exceptions_to_data_check_related_pin

```
timing_apply_exceptions_to_data_check_related_pin {true | false}
```

Default: true

Data_type: Boolean

Controls if false path assertions which block the data path to the related (-from) reference pin of the data-to-data check disable the data-to-data check.

When set to `false`, the data-to-data remains valid when a false path assertion blocks the data path to the related (-from) pin.

When set to `true`, any false path assertion which blocks the data path to the related (-from) pin of the data-to-data check also causes the check to be disabled.

This attribute has no effect on the data-to-data checks with a reference clock, where an exception blocks the clock reference signal at the reference pin. The data-to-data check will always be disabled in this situation.

Examples

- In the following example, the data-to-data check is disabled:

```
set timing_apply_exceptions_to_data_check_related_pin true
set_data_check -from portA -to portB
set_false_path -to portA
```

- In the following example, the data-to-data check is valid. This is the default situation:

```
set timing_apply_exceptions_to_data_check_related_pin false
set_data_check -from portA -to portB
set_false_path -to portA
```

- In this case, the data check is always disabled and is not affected by this attributes' settings:

```
set_data_check -from portA -to portB
set_false_path -to [get_clocks ClkA]
```

where the path to `portA` is clocked by `ClkA`.

timing_apply_setup_hold_exceptions_to_data_check_related_pin

```
timing_apply_setup_hold_exceptions_to_data_check_related_pin {true | false}
```

Default: `true`

Data_type: Boolean

Disables the early signal data-to-data paths, when false path is applied on the late reference path.

When set to `true`, the `set_false_path -setup` parameter settings on the reference path disables early signal d2d path, and vice versa. If both the `-setup` and `-hold` parameters are set, the signal early/late d2d path is disabled, irrespective of this attribute's settings.

When set to `false`, the `set_false_path` exceptions set on d2d reference path do not affect data path slack. When the `set_false_path` command is set on the reference d2d path with only `-setup` (or only `-hold`) parameter, the signal path of d2d check is not affected by these exceptions.

timing_cap_unit

```
timing_cap_unit string
```

Default: `" "`

Data_type: string

Specifies the capacitance units to be used for timing constraints and timing reports. The default unit for capacitance is inferred from the timing libraries. You can override this inference by explicitly setting the value to 1pf or 1ff. Setting this variable explicitly is the same as executing the `set_library_unit -cap` command to set the capacitance units.

timing_case_analysis_for_icg_propagation

```
timing_case_analysis_for_icg_propagation {false | require_seq_prop | always | case_only  
| constant_only}
```

Default: false

Data_type: Enum

Controls whether constants, which propagate to the inputs of integrated clock gating (ICG) cells, are further propagated to the output pins of these cells. The ICG cells typically contain an internal register or latch structure which synchronizes the devices' enable signal. If a constant propagates to this enable, it can be selectively propagated through the internal sequential element to the related output pins.

This attribute setting can uniquely control the handling of sequential propagation through ICG cells, or it can work in conjunction with the more general `timing_case_analysis_for_sequential_propagation` attribute settings.

The constants can be present in the analysis due to several different reasons, broadly breaking down to netlist constants and case analysis. The netlist constants come from connections to tie-hi or tie-lo cells, a 1'b0/1'b1 assignment in the netlist, or explicit power rail connections. The constants derived from these sources are considered to be immutable. The constants due to case analysis come strictly from the `set_case_analysis` constraint and are considered to be temporary - configuring the analysis to align to a certain mode. You may choose to handle constants from any source in a similar manner or customize the handling based on the constant type.

If you choose to allow propagation of only a specific type of constant, the output pin's state function must resolve to a constant by only considering inputs pins with the required constant type. Otherwise, the constant will not be propagated.

The propagation of constants to the output of a sequential device may affect how the timing checks on related input pins are analyzed. If all the possible transitions on a given input pin are unable to change the constant state of the output pin, the timing checks on the input pin will be disabled by default.

Changing the state of this attribute will change where constants are present in the analysis, typically

resulting in sets of timing arcs and timing checks being enabled or disabled. As a result, a full timing update, including a fresh delay calculation, will be initiated when this setting is toggled.

You can use one of the following options:

Option	Description
false	Disables sequential propagation from any constant source (default setting).
require_seq_prop	When specified, the software will defer ICG sequential propagation handling to what is more generally set for all sequential devices, via either the <code>timing_case_analysis_for_sequential_propagation</code> attribute setting. For all the other options, the state of these external settings is not consequential.
always	Allows sequential propagation from any constant source - netlist or constraint-based.
constant_only	Allows sequential propagation only for constants that are due to the propagation of netlist-based constants.
case_only	Allows sequential propagation only for constants that are due to the propagation of constraint-based constants from <code>set_case_analysis</code> settings.

timing_case_analysis_for_sequential_propagation

```
timing_case_analysis_for_sequential_propagation {false | true | case_only |  
constant_only}
```

Default: false

Data_type: Enum

Controls whether constants, which propagate to the inputs of sequential devices like registers or flip-flops, are propagated to the output pins of those devices. This attribute will also affect the propagation of constants through integrated clock gating (ICG) cells - depending on the setting of the related attribute `timing_case_analysis_for_icg_propagation`. Refer to the documentation for this attribute for more information on ICG handling.

The constants can be present in the analysis due to several different reasons - broadly breaking down to netlist constants and case analysis. The netlist constants come from connections to tie-hi

or tie-lo cells, a 1'b0/1'b1 assignment in the netlist, or explicit power rail connections. The constants derived from these sources are considered to be immutable. The constants due to case analysis come strictly from the `set_case_analysis` constraint and are considered to be temporary - configuring the analysis to align to a certain mode. You may choose to handle constants from any source in a similar manner or customize the handling based on the constant type.

If you choose to allow propagation of only a specific type of constant, the output pin's state function must resolve to a constant by only considering inputs pins with the required constant type. Otherwise, the constant will not be propagated. The propagation of constants to the output of a sequential device may affect how the timing checks on related input pins are analyzed. If all the possible transitions on a given input pin are unable to change the constant state of the output pin, the timing checks on the input pin will be disabled by default.

Changing the state of this attribute will change where constants are present in the analysis, typically resulting in sets of timing arcs and timing checks being enabled or disabled. As a result, a full timing update, including a fresh delay calculation, will be initiated when this setting is toggled. You can use one of the following options:

Option	Description
false	Disables sequential propagation from any constant source (default setting).
true	Allows sequential propagation from any constant source - netlist or constraint-based.
constant_only	Allows sequential propagation only for constants that are due to the propagation of netlist-based constants.
case_only	Allows sequential propagation only for constants that are due to the propagation of constraint-based constants from the <code>set_case_analysis</code> command.

timing_case_analysis_propagation

```
timing_case_analysis_propagation {true | false}
```

Default: true

Data_type: Boolean

Enables case analysis propagation.

The `timing_case_analysis_propagation` attribute is the top-level control for enabling or disabling all case propagation in the current session. When set to `true`, case propagation will be enabled and other attributes can then be used to selectively control case propagation in specific circumstances. The `timing_case_analysis_for_sequential_propagation` attribute variable can be used to control whether case is propagated across sequential devices. Propagation through integrated clock-gating (ICG) cells can be controlled with the `timing_case_analysis_for_icg_propagation` attribute. When set to `false`, the software disables case analysis propagation.

timing_check_timing_report_all_checks

```
timing_check_timing_report_all_checks {true | false}
```

Default: `false`

Data_type: Boolean

Enables reporting of all the checks performed by the `check_timing` command.

When set to `true`, the `check_timing` command reports all the checks performed, even if there is no violation(s) for the corresponding check type.

When set to `false`, only the check types that have any violations will be reported.

timing_check_timing_signal_level_high_to_low_threshold

```
timing_check_timing_signal_level_high_to_low_threshold double
```

Default: `0.0`

Data_type: Double

Sets the threshold value for warning the signal level difference between driver and receiver voltages, when the driver is at a higher voltage than the receiver voltage.

The software shows the signal level warnings (`check_timing signal_level`) that report the net driver/receiver pin pairs for which the pin voltage does not match. The signal level warning messages can be filtered based on these threshold values.

timing_check_timing_signal_level_low_to_high_threshold

timing_check_timing_signal_level_low_to_high_threshold double

Default: 0.0

Data_type: Double

Sets the threshold value for warning the signal level difference between driver and receiver voltages, when the driver is at a lower voltage than the receiver voltage.

The software shows the signal level warnings ([check_timing signal_level](#)) that report the net driver/receiver pin pairs for which the pin voltage does not match. The signal level warning messages can be filtered based on these threshold values.

timing_clock_phase_propagation

timing_clock_phase_propagation {positive | negative | both}

Default: both

Data_type: Enum

Allows you select the appropriate clock phases at register clock pins when both positive and negative phases of the same clock signal are seen in the clock network. For example, a register whose clock pin is fed by a two-input XOR gate where one input is a clock port and the other input is a data port. When the value of the data signal of the XOR gate is unknown, the XOR gate can see both positive and negative phases of the clock signal.

both	Selects both positive and negative clock phases at the register clock pins.
positive	Selects positive clock phases.
negative	Selects negative clock phases.

timing_clock_source_paths_unconstrained_mark_clock_used_as_data

timing_clock_source_paths_unconstrained_mark_clock_used_as_data {true | false}

Default: true

Data_type: Boolean

The `is_clock_used_as_data` property (by using the `get_property` command) returns a value of `true` for unconstrained clock source network.

timing_clock_source_use_driving_cell

```
timing_clock_source_use_driving_cell {true | false}
```

Default: `true`

Data_type: Boolean

When set to `false`, delay calculation in propagated clock mode will use an idealized driver model at the `create_clock` root pin or port when it calculates delays for the first stage of the clock tree. The idealized model is a ramp input with the transition time set by the system default transition, which is normally 0ns.

When this variable is set to `true`, the drive characteristics of the cell output pin that `create_clock` is asserted on will be used to calculate the first stage clock tree delays and slews. The input slew used for this driver model is whatever slew has been propagated to the cell's input pins from upstream logic. The arrival time of the clock at the root pin is not modified in any way.

If you wish to specify the input slews to be used for the clock driver cell, that is, not use the upstream propagated slews - you can use the `set_annotated_transition` constraint on the cell's input pins to control this. Setting an input transition of 0ns on the input pins can be useful when trying to correlate analysis results with other applications which do not use the full propagated transition times.

timing_clock_uncertainty_from_to_precedence

```
timing_clock_uncertainty_from_to_precedence {true | false}
```

Default: `false`

Data_type: Boolean

Sets the clock uncertainty precedence.

When set to `true`, the software gives higher precedence to clock-to-clock uncertainty.

timing_collection_all_fanin_fanout_traversal_mode

`timing_collection_all_fanin_fanout_traversal_mode {hierarchical | flat}`

Default: flat

Data_type: Enum

Controls how the software processes netlists during `all_fanin/all_fanout` command processing.

timing_collection_result_display_limit

`timing_collection_result_display_limit integer`

Default: 100

Limits the number of objects of a collection to be displayed in the output report.

When set to an integer value, all the commands, such as, `get_cells`, `add_to_collection`, and `query_objects` commands, that display collection objects in the output report will be impacted.

When set to -1, all the elements in the collection will be displayed in the output.

timing_collection_variable_assignment_compatibility

`timing_collection_variable_assignment_compatibility {true | false}`

Default: true

Data_type: Boolean

Disables printing the names of the output of `get_*` commands.

When set to `false`, the software prints the `get_*` command output on the shell. This attribute should be used for debugging purposes only.

timing_context_apply_default_primary_input_assertion

`timing_context_apply_default_primary_input_assertion {true | false}`

Default: `false`

Data_type: Boolean

Propagates the phase at block level for unmapped clocks.

When set to `true`, the nets in the fanout of data ports with unmapped clocks are considered to have infinite timing windows during SI context analysis.

When set to `false`, the unmapped top clock arrivals on data ports are dropped during context analysis.

timing_context_apply_port_sdc_exceptions

```
timing_context_apply_port_sdc_exceptions {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, the software will consider path exceptions on block constraints in the hierarchical context flow.

timing_context_clock_mapping_percentage_tolerance

```
timing_context_clock_mapping_percentage_tolerance double
```

Default: `0.0`

Data_type: Double

Controls the tolerance value of period and waveform in clock mapping.

timing_context_clock_mapping_check_waveform

```
timing_context_clock_mapping_check_waveform {true | false}
```

Default: `true`

Data_type: Boolean

Controls the waveform checking.

timing_context_continue_on_invalid_module_list

```
timing_context_continue_on_invalid_module_list {true | false}
```

Default: false

Data_type: Boolean

Enables to continue timing analysis without generating timing context.

When set to `true`, the software will continue with timing analysis without generating timing context, when all the specified modules/instances are invalid.

timing_context_clock_phase_based_clock_mapping

```
timing_context_clock_phase_based_clock_mapping  
{true | false}
```

Default: false

Data_type: Boolean

When set to `true`, the port-based clock mapping search is enhanced by additionally attempting to map originally unmapped top clocks.

Such unmapped top clock on data port is to be mapped to block clock if top clock was mapped to block clock as a clock on clock port.

Note: This attribute can be used when `timing_context_port_based_clock_mapping` is set to `true`.

timing_context_data_phase_based_clock_mapping

```
timing_context_data_phase_based_clock_mapping {true | false}
```

Default: false

Data_type: Boolean

When set to `true`, the port-based clock mapping search () is enhanced by additionally trying to map originally unmapped top clocks.

Such unmapped top clock on data port is to be mapped to block clock if top clock was mapped to block clock on other data ports.

Note: This attribute can be applied when `timing_context_port_based_clock_mapping` is set to `true`.

timing_context_port_based_clock_mapping

timing_context_port_based_clock_mapping {true | false}

Default: false

Data_type: Boolean

Enables clock mapping based on block ports.

When set to `true`, clock mapping is done depending solely on clock property of the top and block clock for each clock and data port separately. For enabling N-to-1 or 1-to-N clock mapping, this attribute should be set `true`.

When set to `false`, only 1-to-1 clock mapping is done. In such cases, the following rules are used to determine clock mapping:

- Rule1: Clocks with matching property (period/duty cycle)
- Rule2: Clocks with matching name
- Rule3: Clocks present on maximum number common ports
- Rule4: Any one of the matching clocks

For example,

- Consider the following illustration:



When set to `true`, the following mapping will be seen:

Block Port	Block Clock	Mapped top clock
P1	CLK	CLK, TCLK

When set to `false`, CLK will be mapped to CLK according to Rule 2.

Notes:

- The `context_data` property of block port can be accessed only when this attribute is set `true`.

- When set to `true`, the software may increase the pessimism as the worst latency/arrival will be picked for block context analysis.
- If there are multiple block clocks that can be mapped to same top clock on the same block port, then Tempus does not perform auto mapping (when set to `true`). Whereas for the `false` setting, clock mapping is done depending on the above mentioned rules.

timing_consider_late_arrival_for_ssi_derate

```
timing_consider_late_arrival_for_ssi_derate {true | false}
```

Default: `false`

Data_type: Boolean

Controls constant propagation across sequential cells.

When set to `true`, the software will use the late arrival time for window overlapping evaluation during simultaneous switching input (SSI) analysis.

timing_constraint_disable_min_max_input_delay_worst_casing

```
timing_constraint_disable_min_max_input_delay_worst_casing {true | false}
```

Default: `false`

Data_type: Boolean

Disables the override of the maximum value (set using the `set_input_delay -max` parameter) from the specified `-min` value.

timing_constraint_enable_detailed_report_invalid_begin_end_points

```
timing_constraint_enable_detailed_report_invalid_begin_end_points {true | false}
```

Default: `false`

Data_type: Boolean

Enables detailed reporting of invalid begin/end points specified with the `set_false_path`/`set_multicycle_path` constraints.

When set to `false`, the software checks only the structural end points.

timing_constraint_enable_report_invalid_begin_end_points

```
timing_constraint_enable_report_invalid_begin_end_points {true | false}
```

Default: `false`

Data_type: Boolean

Enables reporting of invalid begin points/end points specified with the `set_false_path` or `set_multicycle_path` constraint.

timing_constraint_enable_search_path

```
timing_constraint_enable_search_path {true | false}
```

Default: `false`

Data_type: Boolean

Enables search of constraint files from the search path.

timing_constraint_enable_separate_multicycle_data_checks

```
timing_constraint_enable_separate_multicycle_data_checks {true | false}
```

Default: `false`

Data_type: Boolean

Controls multicycle path constraint application on data checks.

timing_constraint_enable_logging

```
timing_constraint_enable_logging {true | false}
```


Default: false

Data_type: Boolean

Enables dumping of constraint commands in the log file.

Note: The impact of this attribute is visible only when the constraints are loaded using the `read_sdc` command or the view definition file.

timing_constraint_load_minimal_set_for_automated_eco

```
timing_constraint_load_minimal_set_for_automated_eco {true | false}
```

Default: false

Data_type: Boolean

Skips loading the SDC constraints that are not required in the Tempus session, when the `opt_signoff` command is used. The timing data is obtained from the ECO timing database - this attribute reduces the TAT for designs with complex constraint mode files.

timing_constraint_path_delay_exclude_check_delay_from_ignore_clock_latency

```
timing_constraint_path_delay_exclude_check_delay_from_ignore_clock_latency {true | false}
```

Default: false

Data_type: Boolean

Excludes check arc delays from slack computation.

When set to `true`, the software excludes check arc delays from slack computation for paths constrained by the `set_max_delay/set_min_delay -ignore_clock_latency` command.

timing_constraint_path_delay_exclude_io_delay_from_ignore_clock_latency

```
timing_constraint_path_delay_exclude_io_delay_from_ignore_clock_latency {true | false}
```

Default: false

Data_type: Boolean

Excludes the input/output delays from slack computation.

When set to `true`, the software excludes the input/output delays from slack computation for paths constrained by the `set_max_delay/set_min_delay -ignore_clock_latency` command.

timing_constraint_path_delay_exclude_unconstrained_endpoints

```
timing_constraint_path_delay_exclude_unconstrained_endpoints {true | false}
```

Default: `false`

Data_type: Boolean

When path delay exceptions are asserted (using the `set_min_delay` or `set_max_delay` commands) with a complete `-from` and `-to` specification, the exception can be applied between any two data path pins. If the constraint assertion includes only a `-from` and/or `-through` argument, then a set of valid endpoints must be established to define the constrained paths. This attribute allows you to configure the set of valid endpoints (inferred `-to` pins) that are selected.

When set to `false`, the set of allowed endpoints is simply all of the endpoints in the combinational fanout of the `-from` and/or `-through` arguments - essentially, wherever the path tracing stops. The path delay constraint will be applied as if there were a `-to` argument specified for each endpoint.

When set to `true`, the software uses different rules to determine the valid set of inferred `-to` endpoints. Only constrained pins in the fanout of `-from` and/or `-through` arguments are considered as valid `-to` endpoints for the constraint. A constrained endpoint will have some type of timing check - such as Setup or Hold already in force on that pin. Additionally, each transition at the endpoints will be examined to determine if each is individually constrained. The path exception will only be applied to the edges that are constrained. As an example, if a register preset or a clear pin was a potential endpoint, then only the edge constrained by the recovery or removal check will have the path delay applied. To apply the path delay constraint to unconstrained endpoints while in this mode, you must specify them explicitly with the `-to`, `-rise_to`, or `-fall_to` parameters of the `set_min_delay` and `set_max_delay` constraints.

timing_constraint_path_delay_include_clock_pin_endpoints

```
timing_constraint_path_delay_include_clock_pin_endpoints {true | false}
```

Default: `false`

Data_type: Boolean

Considers register clock pins as end-points for path delay constraints applied upstream.

When set to `true`, the `set_max_delay` command considers unconstrained paths ending at the register clock pins.

timing_constraint_warn_for_timing_derate_exceeding_max_limit

```
timing_constraint_warn_for_timing_derate_exceeding_max_limit double
```

Default: 100.0

Value Range: 0 to 100

Data_type: Double

Sets a limit on derate values. If a larger derate value is applied, the derates are still processed. However, the software issues a warning.

Note: It is recommended to specify derate values within the acceptable value range, because large derate values may cause errors during processing.

timing_continue_on_error

```
timing_continue_on_error {true | false}
```

Default: false

Data_type: Boolean

When set to `true`, the software continues to process if an error occurs when any constraints are used, or during timing analysis, or when a library is read. This attribute does not have any effect on other commands, except constraints or commands related to timing analysis.

When set to `false`, if an error occurs, during timing analysis, library reading, or defined constraints, the software stops processing and exits.

timing_constraint_update_io_latency_averaging_mode

```
timing_constraint_update_io_latency_averaging_mode {rise_fall | rise_only}
```

Default: rise_fall

Data_type: Enum

Controls the analysis to update IO latency for both rise and fall, or for rise only. You can set one of the options - `rise_fall`, `rise_only`.

timing_cppr_opposite_edge_mean_scale_factor

`timing_cppr_opposite_edge_mean_scale_factor double`

Default: 1.0

Value Range: 0 to 1

Data_type: Double

Controls the mean delay component for CPPR credit for opposite transition edges at a common pin.

When this attribute is specified, the software provides a finer level configuration of the CPPR credit for reducing Common Path Pessimism Removal (CPPR) in timing analysis. The software allows a certain amount of CPPR credit in the case of opposite edge transitions to be scaled to a user-specified value, that is between zero credit or full credit provided by the existing controls.

You can use `timing_cppr_opposite_edge_sigma_scale_factor` attribute for adjustment of the sigma component of the CPPR credit. In SOCV analysis mode, you can use both of these attributes to separately control the adjustment of the mean and sigma components of the CPPR credit. In regular static-timing analysis (STA) mode (without SOCV), only the mean scale factor is considered for CPPR adjustment scaling.

timing_cppr_opposite_edge_sigma_scale_factor

`timing_cppr_opposite_edge_sigma_scale_factor double`

Default: 1.0

Value Range: 0 to 1

Data_type: Double

Controls sigma delay component for CPPR credit for opposite transition edges at common pin.

timing_cppr_opposite_edge_sigma_scale_factor_net

`timing_cppr_opposite_edge_sigma_scale_factor_net double`

Default: 1.0

Data_type: Double

Controls the net sigma delay component for CPPR credit for opposite transition edges at the common pin.

The scale factors are applicable only when the common pin has mismatched transitions for the launch and capture paths.

timing_cppr_opposite_edge_sigma_scale_factor_cell

`timing_cppr_opposite_edge_sigma_scale_factor_cell double`

Default: 1.0

Data_type: Double

Controls the cell sigma delay component for CPPR credit for opposite transition edges at the common pin.

The scale factors are applicable only when the common pin has mismatched transitions for the launch and capture paths.

timing_cppr_propagate_thru_latches

`timing_cppr_propagate_thru_latches {true | false}`

Default: false

Data_type: Boolean

When set to `true`, the CPPR of a timing path crossing a latch (or latches), for example, ff1 -> lat1 -> ff2, is calculated using the common point of the origin device (ff1) and the capture device at the end of the path (ff2).

When set to `false`, the segment-based CPPR calculation is performed, considering the common point of latch (lat1) and the capture device (ff2). This CPPR setting is considered only when the latch thru analysis mode is enabled and a latch thru path exists.

timing_cppr_remove_clock_to_data_pessimism

`timing_cppr_remove_clock_to_data_pessimism {true | false}`

Default: `false`

Data_type: Boolean

When set to `true`, removes clock re-convergence pessimism (CRP) for clock source paths.

timing_cppr_self_loop_mode

`timing_cppr_self_loop_mode {true | false}`

Default: `true`

Data_type: Boolean

When set to `true` in case of self-loop paths, computes CPPR adjustment by taking the difference between early and late clock arrival time of the common point; the clock pin of the flop. A self-loop path is a path that goes from a clock pin of a flop and ends at the data pin of the same flop. However detecting self-loop paths requires runtime, and is design dependant.

When set to `false`, does not detect self-loop paths. In this case, the software computes CPPR adjustment by using the common point as the output pin driving the clock pin of the flop. This might result in better runtime, but less accurate results. The loss of accuracy is the difference between early and late delay of the interconnect portion driving the clock pin.

timing_cppr_skip_clock_reconvergence

`timing_cppr_skip_clock_reconvergence {true | false}`

Default: `false`

Data_type: Boolean

Specifies the branch point to use for computing clock path pessimism removal (CPPR) adjustment when there is reconvergence in the clock tree.

When set to `false`, the software uses the branch point closest to the register clock pins where the clocks reconverge. When set to `true`, the software uses the farthest branch point from the register clock pins (that is, the branch point closest to the clock root pin where the clocks diverge).

timing_cppr_skip_clock_reconvergence_for_unmatched_clocks

```
timing_cppr_skip_clock_reconvergence_for_unmatched_clocks {true | false}
```

Default: false

Data_type: Boolean

Allows the software to skip re-convergence of unmatched clocks.

When set to `true`, the software uses the farthest branch point from the register clock pins when the launching and capturing clocks are different - the re-converge points are ignored and not searched.

timing_cppr_threshold_ps

```
timing_cppr_threshold_ps double
```

Default: 20.0 ps

Data_type: Double

Specifies the maximum amount of pessimism that clock path pessimism removal (CPPR) analysis is allowed to leave in the path. The unit is pico seconds. Setting this attribute to a specified value means that all the paths might be reported without having their pessimism removed. This attribute can be used to speed up CPPR analysis run time. The larger the value of the attribute, the faster the run time.

timing_cppr_transition_sense

```
timing_cppr_transition_sense {normal | same_transition | same_transition_expanded}
```

Default: normal

Data_type: Enum

Specifies the transition sense of the launching and capturing clocks at the common node, to calculate clock path pessimism removal (CPPR).

- `normal`: The CPPR analysis computes the CPPR value even if the launching and capturing clock transitions at the common node are in opposite directions. If the transition sense at the common node are in opposite directions, the pessimism is the minimum of the difference

between late and early arrival times for rise and fall clock transitions.

- `same_transition`: The CPPR analysis computes the CPPR value only if the launching clock transition is same as the capturing clock transition at the common node. If the transitions are different, the CPPR value is zero.
- `same_transition_expanded`: The CPPR analysis computes the CPPR value only if the launching clock transition is the same as the capturing clock transition at the common node. If the transitions are different, the software searches until it finds a common point where the transitions are the same, and calculates the CPPR value from that point.

timing_create_clock_default_propagated

```
timing_create_clock_default_propagated {true | false}
```

Default: `false`

Data_type: Boolean

Enables clocks to be created in propagated mode.

By default, all clocks, created using `create_clock` and `create_generated_clock` commands, are in ideal mode.

When the attribute is set to `true`, all subsequently created regular clocks, derived clocks, and library generated clocks are in propagated mode.

There is no effect on the functionality of `set_propagated_clock` and `set_clock_latency` commands.

timing_default_opcond_per_lib

```
timing_default_opcond_per_lib {true | false}
```

Default: `true`

Data_type: Boolean

Determines the default operating condition behavior when there is no explicit user operating condition.

When set to `true`, each timing library uses its own internally-defined default operating condition specification.

When set to `false`, the default operating condition of the master library (that is, the first library in the library search list) is used to set the PVT operation point for the entire design. All timing libraries are accessed and derated to that operating point.

timing_defer_mmmc_obj_updates

```
timing_defer_mmmc_obj_updates {true | false}
```

Default: `false`

Data_type: Boolean

By default, the multi-mode multi-corner (MMMC) update commands, `update_constraint_mode` and `update_analysis_view`, actively load new data upon execution. Since all the timing information is reloaded with each update command, executing a sequence of updates can result in multiple reloads and a subsequent degradation of performance and turn-around-time.

When set to `true`, the software allows a sequence of MMMC updates to be accumulated before new data is loaded and analyzed.

If the `set_analysis_view -update_timing` parameter is specified, the software indicates that the MMMC object updates are complete and new configuration can be loaded and analyzed.

For example:

```
update_constraint_mode -name ioTiming -sdc_files {postCtsIO.sdc}
update_constraint_mode -name coreTiming -sdc_files {postCtsCore.sdc}
set_analysis_view -update_timing
```

timing_derate_aocv_dynamic_delays

```
timing_derate_aocv_dynamic_delays {true | false}
```

Default: `true`

Data_type: Boolean

Controls AOCV derating of SI-induced delays.

When set to `false`, AOCV derates will not be applied on dynamic, SI-induced delays, that is, a

value of `false` ensures AOCV derating is only applied to the static component of the delay. When set to `true`, AOCV derating is applied to both static and dynamic arc delay components.

timing_derate_aocv_reference_point

```
timing_derate_aocv_reference_point {true | false}
```

Default: 1

Data_type: Integer

Allows specification of reference point for AOCV factors. As per the current AOCV library definitions, the AOCV reference point is always expected to be set at 1.0.

timing_derate_dynamic_compatibility

```
timing_derate_dynamic_compatibility {true | false}
```

Default: true

Data_type: Boolean

Controls how the `set_timing_derate` command factors that are not specified, using either `-static` or `-dynamic` options, are applied to the delay.

When this attribute is set to 1, the behavior will be compatible with previous releases. In this mode, the static and dynamic components will be summed up before applying derating.

When set to 0, the static and dynamic components will be derated separately and then combined. A setting of 0 is equivalent to using separate derate assertions with the `set_timing_derate -dynamic` and `-static` options explicitly specified.

timing_derate_incremental_adjust_additive_mode

```
timing_derate_incremental_adjust_additive_mode {true | false}
```

Default: false

Data_type: Boolean

Allows the software to accumulate incremental derate values instead of overwriting them.

When set to `false`, the successive application of incremental derating will overwrite any existing derate value with the new value.

When set to `true`, the successive applications of the incremental derates will be accumulated into a final derate value.

timing_derate_enable_incremental_multiplication_mode

```
timing_derate_enable_incremental_multiplication_mode {true | false}
```

Default: `false`

Controls the overwrite or accumulative mode settings when the `set_timing_derate -multiply` parameter is specified.

When set to `false`, the latest `set_timing_derate -multiply` command will overwrite the previous constraints at the same level.

When set to `true`, all the values in history will be accumulatively multiplied, when multiple `set_timing_derate -multiply` settings are made on the same object.

timing_derate_incremental_multiply_accumulative_mode

```
timing_derate_incremental_multiply_accumulative_mode {true | false}
```

Default: `false`

Data_type: Boolean

Controls the overwrite or accumulative mode settings when the `set_timing_derate -multiply` parameter is specified. You can use this attribute when

`timing_derate_enable_incremental_multiplication_mode` is set to `true` (that is, the `-multiply` setting must be enabled).

When set to `false`, the last specified `set_timing_derate -multiply` command will overwrite the previous constraints at the same level.

When set to `true`, all the values saved in history will be accumulatively multiplied, when multiple `-multiply` settings are made on the same object.

Examples

- The following example shows that in overwrite mode, the multiplicative adjustment will be updated to 0.5:

```
set_global timing_derate_enable_incremental_multiplication_mode true
set_global timing_derate_incremental_multiply_accumulative_mode false
set_timing_derate -multiply 1.2
set_timing_derate -multiply 0.5
```
- The following example shows that in accumulative mode, the multiplicative adjustment will be updated to $1.2 \times 0.5 = 0.6$:

```
set_global timing_derate_enable_incremental_multiplication_mode true
set_global timing_derate_incremental_multiply_accumulative_mode true
set_timing_derate -multiply 1.2
set_timing_derate -multiply 0.5
```

timing_derate_ocv_reference_point

```
timing_derate_ocv_reference_point {true | false}
```

Default: 1

Data_type: Integer

Allows specification of reference point for OCV factors. As per the current OCV library definitions, the OCV reference point is always expected to be set at 1.0.

timing_derate_negative_delay_backward_compatibility

```
timing_derate_negative_delay_backward_compatibility {true | false}
```

Default: true

Data_type: Boolean

Controls the effective derate calculation for negative delays. When effective derate is requested for negative delays, the value will be converted according to the negative delay scaling rule, e.g., 2 -

positiveVal.

timing_derate_spatial_distance_unit

```
timing_derate_spatial_distance_unit {default | 1um | 1nm}
```

Default: default

Data_type: Enum

Specifies distance units in the SOCV and AOCV library formats.

This attribute supports 3 distance units - default, 1um, and 1nm. The default value is the default units that are derived from the current system units for distance (µm/nm overrides the default).

Note: The timing_derate_spatial_distance_unit attribute should be set before a design is loaded.

timing_derate_voltage_scaling_mode

```
timing_derate_voltage_scaling_mode first_library | snap_to_nearest | interpolate
```

Default: first_library

Data_type: Enum

Controls how timing derates are computed when the analysis voltage is not at the characterization voltage of any of the currently loaded timing libraries. When the timing analysis environment is configured for interpolation, the MMMC library set will contain a list of timing libraries at different voltage characterizations points.

You can specify the following options:

first_library	Derates associated with the first library in the timing library list will be used.
snap_to_nearest	Allows the software to choose one set of derates from the library closest to the required analysis voltage.
interpolate	Allows the timer to perform interpolation of the derates associated with the two bounding reference libraries.

Consider the following example:

```
create_library_set -name ls_interp_1.50V_1.80V  
-timing [list stdcell_1.50V.lib stdcell_1.75V.lib stdcell_1.80V.lib]
```

In this example, by default (`first_library`) the derates associated with the library `stdcell_1.50V.lib` will be used.

When set to `snap_to_nearest`, if the analysis voltage is 1.70V, the derates from the library `stdcell_1p75V.lib` will be used.

When set to `interpolate`, if the analysis voltage is 1.60V, then interpolation of derates from the libraries `stdcell_1.50V.lib` and `stdcell_1.75V.lib` will be used.

If interpolation mode is selected and the analysis voltage is beyond either the smallest or largest voltage, then the library derates will not be interpolated. The derates associated with the nearest bounding library will be used. If the analysis voltage is 1.90V, then only the derates from `stdcell_1.80V.lib` will be used.

If interpolation mode is enabled, but not all the bounding libraries have derate values, a default derate value of 1.0 will be assumed for the missing derates and used for interpolation. A TA-960 warning message will also be issued.

timing_disable_bus_contention_check

```
timing_disable_bus_contention_check {true | false}
```

Default: false

Data_type: Boolean

When set to `true`, disables propagation of maximum delay along three state disable timing arcs and minimum delay along three state enable arcs. If you know that bus contention conditions do not occur in your design, disable these checks by setting these attributes to `true`.

Checks for setup and hold violations in three-state bus designs. Checks the worst delay path to ensure that the latched signals are stable and not unknown values - X. Ensures proper timing checks for bus contention.

timing_disable_clock_period_checks

```
timing_disable_clock_period_checks {true | false}
```

Default: false

Data_type: Boolean

When set to `true`, disables timing model clock period checks during timing analysis.

timing_disable_constant_propagation_for_sequential_cells

```
timing_disable_constant_propagation_for_sequential_cells {true | false}
```

Default: false

Data_type: Boolean

Controls constant propagation across sequential cells.

When set to `true`, the software disables constant propagation across sequential cells.

When set to `false`, the software enables constant propagation across sequential cells.

timing_disable_drv_report_on_constant_nets

```
timing_disable_drv_report_on_constant_nets {true | false}
```

Default: false

Data_type: Boolean

Disables DRV checks for nets that are disabled by constant propagation:

When the attribute is set to `true`, the `report_constraint` command does not report DRV violations for constant nets.

timing_disable_floating_bus_check

```
timing_disable_floating_bus_check {true | false}
```

Default: false

Data_type: Boolean

When set to `true`, disables propagation of minimum delay through three state disable timing arcs

and maximum delay through three state enable arcs. These checks are only valid during floating bus conditions.

timing_disable_get_clocks_for_current_instance

```
timing_disable_get_clocks_for_current_instance {true | false}
```

Default: false

Data_type: Boolean

When set to `true`, the software disables the `current_instance` command support on `get_clocks`.

timing_disable_genclk_combinational_blocking

```
timing_disable_genclk_combinational_blocking  
{true | false}
```

Default: false

Data_type: Boolean

Disables blocking of the latency path for a generated clock.

When set to `true`, allows the latency path to traverse through the upstream generated clock.

When set to `false`, if two back-to-back generated clocks are on the same path, and if the downstream generated clock is set using the `create_generated_clock -combinational` parameter, then the latency path for this generated clock is blocked by the upstream clock. The downstream generated clock is created with zero latency.

timing_disable_inferred_clock_gating_checks

```
timing_disable_inferred_clock_gating_checks {true | false}
```

Default: false

Data_type: Boolean

When set to `true`, disables clock gating checks that are inferred on combinational elements in the clock path. Explicit clock gating checks that are described in the timing library are not affected by this attribute.

timing_disable_inout_output_side_timing_checks

```
timing_disable_inout_output_side_timing_checks {true | false}
```

Default: `false`

Data_type: Boolean

Disables timing checks on both the input and output sides of the bidirectional pin.

When set to `false`, the timing checks on both the input and output sides of the bidirectional pin are analyzed.

timing_disable_internal_inout_cell_paths

```
timing_disable_internal_inout_cell_paths {true | false}
```

Default: `true`

Data_type: Boolean

When set to `false`, enables internal bidirectional feedback paths that are completely contained in one instance.

timing_disable_internal_inout_net_arcs

```
timing_disable_internal_inout_net_arcs {true | false}
```

Default: `true`

Data_type: Boolean

When set to `true`, this attribute disables internal bidirectional feedback paths that span multiple instances.

When set to `false`, this attribute enables internal bidirectional feedback paths, which are feedback

paths that you might not want to use during analysis and optimization.

This attribute applies only to paths that span multiple instances. The attribute has no impact on internal feedback paths that are completely contained in one instance.

timing_disable_lib_pulse_width_checks

```
timing_disable_lib_pulse_width_checks {true | false}
```

Default: false

Data_type: Boolean

When set to `true`, disables timing model pulse width checks during timing analysis.

timing_disable_library_data_to_data_checks

```
timing_disable_library_data_to_data_checks {true | false}
```

Default: false

Data_type: Boolean

When set to `true`, the software disables the timing arcs between any data-to-clock or clock-to-clock checks - when `timing_type` attribute is set to one of the following:

- `non_seq_setup_rising`
- `non_seq_setup_falling`
- `non_seq_hold_rising`
- `non_seq_hold_falling`

This attribute affects non-sequential checks only, where a reference pin of the timing check is a clock pin. The data-to-data checks (where the reference pin of the timing check is a data pin) are not affected by this setting. To disable data-to-data checks you can use the

`timing_disable_library_data_to_data_checks` attribute.

timing_disable_library_tieoffs

```
timing_disable_library_tieoffs {true | false}
```

Default: false

Data_type: Boolean

Disables constant functions in a library.

When set to `false`, the software considers constant propagation from library tie high/low cells prior to those from logical netlist and constraints. There may be cases where programmable tie cells are used whose value can be changed later in the flow, assuming the programmable tie cells are modelled with constant output functions.

When set to `true`, the library constants are not propagated during timing analysis or timing optimization.

timing_disable_netlist_constants

```
timing_disable_netlist_constants {true | false}
```

Default: false

Data_type: Boolean

When set to `true`, ignores constants defined in the Verilog netlist. This includes all uses of the in-place constant notation, such as `1'b0`, and the use of power and ground nets. Constants from `set_case_analysis` assertions and from tie-hi and tie-lo cells are still recognized.

When set to `false`, the software recognizes all constants.

timing_disable_model_parasitics

```
timing_disable_model_parasitics {true | false}
```

Default: 0

Data_type: int

Disables saving parasitics data into a model.

When set to `true`, the software will not save parasitics data into the boundary model, so the generated boundary models will occupy a smaller disk space. However, while reading this boundary model at the top level, the original block parasitics (which was used while generating the model) should be read along with the top level parasitics.

When set to `false`, the parasitic data is not stored in the boundary models. Hence, the user does not need to read the block parasitics, since it is read from the boundary models. By default, this attribute is set to `false`.

timing_disable_nochange_checks

```
timing_disable_nochange_checks {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, disables no change timing model checks during timing analysis.

timing_disable_non_sequential_checks

```
timing_disable_non_sequential_checks {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, the software disables the timing arcs between any data-to-clock or clock-to-clock checks.

This attribute affects non-sequential checks only, where a reference pin of the timing check is a clock pin. The data-to-data checks (where the reference pin of the timing check is a data pin) are not affected by this setting. To disable data-to-data checks you can use the

`timing_disable_library_data_to_data_checks` attribute.

timing_disable_output_as_clock_port

```
timing_disable_output_as_clock_port{true | false}
```

Default: `false`

Data_type: Boolean

Allows a clock path to be considered as a data path.

When set to `true`, the software allows a clock source path, leading to an output or bidi port, to be treated as a data path, irrespective of constraints, such as, `set_output_delay`, `set_data_check`, or `set_max_delay`, asserted on that port.

When set to `false`, all the clock source paths, leading to output ports, are considered as valid clock end points. Any unconstrained output port may not be considered as data end point.

timing_disable_parallel_arcs

```
timing_disable_parallel_arcs {true | false}
```

Default: `true`

Data_type: Boolean

Enables the software to automatically perform reduction of parallel delay arcs and check arcs to improve efficiency.

When set to `false`, parallel delay and check arc reduction will be disabled.

timing_disable_pulse_width_same_edge_si_cppr_mode

```
timing_disable_pulse_width_same_edge_si_cppr_mode {true | false}
```

Default: `true`

Data_type: Boolean

Enables SI CPPR credit up to the last divergent pin for pulse width checks for zero width pulse clocks. By default, the software credits minimum of rise and fall transition CPPR up to the common pin as the transitions mismatch at the common pin.

When set to `true`, the software computes base delay CPPR up to the common pin as CPPR credit for the path. The software then finds an alternate common timing pin in the clock network where the transition for launch and capture match. If a match is found, the software includes the SI delay up to this pin in the CPPR credit as well.

timing_disable_report_header_info

`timing_disable_report_header_info {true | false}`

Default: `false`

Data_type: Boolean

Controls whether timing reports are generated using a common report header.

When set to `false`, the software generates timing reports with a report header that includes the following information:

- Tempus software version number
- Platform on which the software is being run
- Time when the report was generated
- Command used to generate the report

Any report-specific banner information is included in the same header box, but listed after the common header information.

When set to `true`, the software generates backward-compatible timing reports without the additional header information.

timing_disable_retime_clock_path_slew_propagation

`timing_disable_retime_clock_path_slew_propagation {true | false}`

Default: `true`

Data_type: Boolean

When set to `true`, path based analysis is performed on data paths only and the clock segments are not considered.

When set to `false`, analysis is performed on both data and clock paths.

timing_disable_sdf_retain_arc_merging

`timing_disable_sdf_retain_arc_merging {true | false}`

Default: `false`

Data_type: Boolean

Controls handling of retain arcs in SDF2.1.

When set to `false`, the Retain information is merged into SDF 2.1, as Retain represents a valid minimum path timing for a cell.

When set to `true`, the Retain arc merging will be disabled.

timing_disable_skew_checks

```
timing_disable_skew_checks {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, disables library skew checks for timing analysis.

timing_disable_test_signal_arc

```
timing_disable_test_signal_arc {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, timing analysis will not analyze the signal arcs coming from or going to the test pin. Such pins are marked in the timing library with either the `test_scan_in`, the `test_scan_enable` attribute for the input pin, the `test_scan_clock` attribute for the input clock pin, or the `test_output_only` attribute for the output pin.

The following are the equivalent Liberty and TLF pin attributes:

Liberty and TLF Pin Attributes

Type	Liberty	TLF
enable	<code>test_scan_enable</code>	SCAN_PINTYPE (ENABLE)
input	<code>test_scan_in</code>	SCAN_PINTYPE (INPUT)
input clock	<code>test_scan_clock</code>	SCAN_PINTYPE (CLOCK)

output	test_output_only	SCAN_PINTYPE (OUTPUT)
--------	------------------	-----------------------

timing_max_transition_use_si_transition

timing_max_transition_use_si_transition {true | false}

Default: false

Data_type: Boolean

Enables reporting of noisy slew in DRV flows.

timing_model_netlist_exclude_dummy

timing_model_netlist_exclude_dummy {true | false}

Default: false

Data_type: Boolean

Excludes the dummy cell definitions that are written to the model netlist.

timing_disable_timing_model_latch_inferencing

timing_disable_timing_model_latch_inferencing {true | false}

Default: true

Data_type: Boolean

Controls whether latch behavior is inferred for cell descriptions tagged with the Liberty `timing_model_type` attribute, including all values: `abstracted`, `extracted`, or `qtm`.

When set to `false`, the software infers latch behavior from the timing check and timing delay arc relationships in the model's description.

When set to `true`, latch inferencing does not happen for cells with the `timing_model_type` attribute. Explicit Liberty latch modelling constructs must be coded into the description in order to model latch behavior.

This attribute does not affect Liberty timing model cells that do not contain

the `timing_model_type` attribute. Standard cell core libraries generally do not have this attribute specified; therefore, they typically are not be affected by this attribute setting.

timing_disable_tristate_disable_arcs

```
timing_disable_tristate_disable_arcs {true | false}
```

Default: false

Data_type: Boolean

When set to `true`, disables all 0/1->Z transitions of tristate arcs during timing analysis. By default, all tristate arcs are enabled.

timing_disable_user_data_to_data_checks

```
timing_disable_user_data_to_data_checks {true | false}
```

Default: false

Data_type: Boolean

When set to `true`, disables data to data checks that are created by the `set_data_check` command. This attribute does not affect non-sequential timing checks described in the timing library.

timing_driving_cell_override_library

```
timing_driving_cell_override_library {true | false}
```

Default: false

Data_type: Boolean

Controls the selection of library cells specified using the `set_driving_cell` command.

When set to `true`, a driving cell will be picked up from the Max Library for max drive cell adjustment and Min Library for min drive cell adjustment, irrespective of the library specified with the `set_driving_cell -library` parameter.

When set to `false`, the libraries are selected as specified with the `set_driving_cell` command.

timing_drv_analysis_ignore_latency_phases

```
timing_drv_analysis_ignore_latency_phases {true | false}
```

Default: false

Data_type: Boolean

Enables ignoring latency phases in DRV analysis.

timing_enable_aocv_slack_based

```
timing_enable_aocv_slack_based {0 | 1}
```

Default: false

Data_type: Boolean

Enables AOCV analysis using slack-based method.

When set to `true`, the software will try to remove excess pessimism from the graph-based analysis (GBA) AOCV stage counts. The paths to endpoints, which are under the specified slack criteria (default 0.0 ns), are considered when computing stage counts. This allows the stage counts and subsequent AOCV derates to be aligned with the failing paths – and not unnecessarily punished by derates coming from related short paths that meet timing requirements.

Note: This setting is not recommended for signoff timing analysis in Tempus. For signoff, path-based analysis (PBA) is the recommended flow for removing excess GBA pessimism.

timing_enable_case_analysis_conflict_warning

```
timing_enable_case_analysis_conflict_warning {true | false}
```

Default: true

Data_type: Boolean

Displays a warning when propagated and asserted constant values are not the same when a full timing update is performed.

When set to `true`, a report file, named `CTE_constant_mismatch.rpt`, is generated that contains details of pins and the reason for constant mismatch. This report is over-written each time a full timing update takes place.

timing_enable_clock_to_clock_clock_gating_check

```
timing_enable_clock_to_clock_clock_gating_check {true | false}
```

Default: `false`

Data_type: Boolean

Enables clock-to-clock clock gating checks.

When set to `true`, the software performs clock to clock gating checks at gating elements. For clock to clock gating checks, the frequency of a clock arriving at the reference side of a check should be equal to or greater than the frequency of the clock arriving at the signal side of the check.

timing_enable_clock_phase_based_rise_fall_derating

```
timing_enable_clock_phase_based_rise_fall_derating {true | false}
```

Default: `false`

Data_type: Boolean

Determines the interpretation of edge-specific derating factors for clock paths, specified using the `set_timing_derate -rise/-fall` parameters.

When set to `false`, the `set_timing_derate -rise/-fall` parameter refer to the direction of the transition from the perspective of each individual output pin along the clock path.

When set to `true`, the `-rise/-fall` parameter refers to which edge (or phase) of the clock is triggering the transition as viewed from the source of the clock where the clock waveform is defined.

timing_enable_data_through_clock_gating

```
timing_enable_data_through_clock_gating {true | false}
```

Default: `true`

Data_type: Boolean

When set to `false`, blocks signals arriving on the enable of the clock-gating check.

When set to `true`, allows both data and clock signals at the enable pin of a clock-gating check to propagate forward past the check.

timing_enable_db_attribute_query

```
timing_enable_db_attribute_query {true | false}
```

Default: `true`

Data_type: Boolean

Allows to query DB attributes with the `get_property` command.

timing_enable_derating_for_pulse_width_checks

```
timing_enable_derating_for_pulse_width_checks {true | false}
```

Default: `false`

Data_type: Boolean

Controls whether the cell check derates, set by `set_timing_derate -cell_check` command, can be applied for pulse width checks.

When set to `true`, the pulse width checks honor cell check derate factors.

timing_enable_early_late_data_slews_for_setuphold_mode_checks

```
timing_enable_early_late_data_slews_for_setuphold_mode_checks {true | false}
```

Default: `false`

Data_type: Boolean

Controls calculated delays.

When set to `true`, the software allows use of both the minimum and maximum data slews for setup and hold checks. By default, the minimum or maximum slew values for clock pins are considered, and for data pins only the maximum slew value is considered while calculating the setup/hold timing check values.

The `timing_enable_early_late_data_slews_for_setuphold_mode_checks` attribute enables an eight-way timing check for:

- Early/late libraries
- Clock/data pins
- Min/max slews

You can use this attribute in simultaneous setup or hold mode only. To enable this mode, you need to set `timing_enable_simultaneous_setup_hold_mode` attribute to `true`. In this mode, you can use the `report_delay_calculation` command (`-from` and `-to` parameters) to report all the setup/hold check values corresponding to minimum/maximum combination of clock and data slews.

timing_enable_genclk_divide_by_inherit_parent_duty_cycle

```
timing_enable_genclk_divide_by_inherit_parent_duty_cycle{true | false}
```

Default: `false`

Data_type: Boolean

Controls inheritance of duty cycle from master clock for generated clocks with divide by option.

timing_enable_genclk_source_path_register_limit

```
timing_enable_genclk_source_path_register_limit {true | false}
```

Default: `false`

Data_type: Boolean

Limits generated clock source latency path to traverse across one register only.

When set to `false`, the software allows master clock latency path to traverse across any number of sequential devices to reach the generated clock target pin.

When set to `true`, the software restricts the master clock latency path for a generated clock to

traverse exactly one sequential device.

timing_enable_generated_clock_edge_based_source_latency

```
timing_enable_generated_clock_edge_based_source_latency {true | false}
```

Default: true

Data_type: Boolean

Controls how the software chooses generated clock source latency paths. Certain specifications of the `create_generated_clock` constraint imply a cause-and-effect relationship between the edges of the master clock and those of the generated clock.

When set to `true`, the software checks whether a path being analyzed has the correct cause-effect relationship consistent with the specified constraint. Paths that are found to be inconsistent are ignored. Only the paths that are in agreement with the required edge relationship are considered for source latency. If no such path exists, the software uses a source latency of 0 ns. This selection is done independently for each edge of the generated clock; therefore the result might be different for each edge.

The edge-to-edge sufficiency checks are performed assuming a Rise-to-Rise, Rise-to-Fall relationship between the master clock and the generated clock with an even number – `divide_by` factor. For the odd `-divide_by` factors, the sufficiency check is performed assuming a Rise-to-Rise, Fall-to-Fall relationship. The sufficiency checking is not performed for – `multiply_by` options.

When set to `false`, the software does not check paths for the correct cause-effect relationship.

timing_enable_get_obj_escaped_name_backward_compatible

```
timing_enable_get_obj_escaped_name_backward_compatible {true | false}
```

Default: false

Data_type: Boolean

Enables printing of escape characters in the `get_object_name` command output.

timing_enable_get_objects_regexp_compatibility

timing_enable_get_objects_regexp_compatibility {true | false}

Default: false

Data_type: Boolean

Defines the pattern search scope to boundaries deduced from hierarchy separation characters specified in the search pattern.

When set to `false`, the commands with `-regexp` parameter will search at the top level only (unless used with the `-hier` parameter).

When set to `true`, the commands will search the specified pattern within the hierarchies based upon the given pattern hierarchy (similar to pattern search without the `-regexp` parameter). The `-hier` along with the `-regexp` parameter shows the same behavior similar to when the command is used without the `-regexp` option - only the leaf level object name is compared to the pattern match.

Examples

Command behavior when this attribute is set to `false`:

```
get_pins -regexp seg2/./QN
**WARN: (TCLCMD-513): The software could not find a matching object of the specified
type for the pattern './.*/'
**ERROR: (TCLCMD-917): Cannot find 'pins' that match 'seg2/./QN'

get_pins -regexp seg2/./QN -hier
seg2/u3/QN seg2/u9/QN seg2/u14/QN

get_pins -regexp ./QN -hier
seg1/u3/QN seg1/u9/QN seg1/u14/QN seg2/u3/QN seg2/u9/QN seg2/u14/QN seg3/u3/QN
seg3/u9/QN seg3/u14/QN u2/QN u5/QN u8/QN
```

Command behavior when this attribute is set to `true`:

```
get_pins -regexp seg2/./QN
seg2/u3/QN seg2/u9/QN seg2/u14/QN

get_pins -regexp seg2/./QN -hier
**WARN: (TCLCMD-513): The software could not find a matching object of the specified
type for the pattern './.*/'
**ERROR: (TCLCMD-917): Cannot find 'pins' that match 'seg2/./QN'

get_pins -regexp ./QN -hier
seg1/u3/QN seg1/u9/QN seg1/u14/QN seg2/u3/QN seg2/u9/QN seg2/u14/QN seg3/u3/QN
seg3/u9/QN seg3/u14/QN u2/QN u5/QN u8/QN
```

timing_enable_hier_context_unmapped_clock_analysis

```
timing_enable_hier_context_unmapped_clock_analysis {true | false}
```

Default: false

Data_type: Boolean

When set to `true`, for unmapped block clocks, the software enables timing analysis based on the block SDC constraints set on block ports.

timing_enable_hierarchical_get_nets_support

```
timing_enable_hierarchical_get_nets_support {true | false}
```

Default: false

Data_type: Boolean

Enables the hierarchical segment support for nets that are specified using the `get_nets` command.

When set to `false`, the `get_nets` command will report only flat nets.

timing_enable_individual_distance_inter_die_analysis

```
timing_enable_individual_distance_inter_die_analysis {true | false}
```

Default: false

Data_type: Boolean

Enables individual inter-die calculation in 3DIC analysis.

timing_enable_latch_thru_mode

```
timing_enable_latch_thru_mode {true | false}
```


Default: `false`

Data_type: Boolean

Enables the latch through analysis mode in which timing paths can propagate across latches, depending on their arrival time with respect to the latch transparency window.

When set to `true`, the arrival and required times will be propagated across a latch if it is in transparent mode. The reporting will show the entire flow through the path, rather than individual segments. This allows the overall path slack to be seen through upstream borrowing latch stages. The timing analysis, timing optimization, and reporting will consider the sequence of borrowing latch stages as a single path. As a result, the timing optimization reaches the target slack by viewing the entire path.

This feature provides better visibility into the overall context of latch path timing issues - for both the stage and path-level slacks. Additionally, this improves usability for debugging and manual ECOs, and shows better quality of result for Tempus ECO timing closure.

timing_enable_latency_through_clock_gating

```
timing_enable_latency_through_clock_gating {true | false}
```

Default: `true`

Data_type: Boolean

Allows propagation of latency phases across clock gating elements.

When set to `true`, the software allows propagation of clock latency phase across clock gating elements.

When set to `false`, the software suppresses propagation of clock latency across clock gating elements from the enable pin. The enable pin of the clock gating element should not be in the clock network for any other clocks. This is not recommended because the enable path (through the 'inferred' clock gates, such as, AND, NAND, OR) is either for clock latency or data paths. If the gated clock check is met, then the enable should not pass through to the output.

In case of ICGs, there is no arc from the EN to the CLKOUT on ICGs that have an incoming enable signal latched. In case of ICG/AND, using the enable path may not be correct for determining latency paths. If no latency paths are found, then the generated clocks definition will need to be re-checked.

timing_enable_mmmc_loop_breaking

timing_enable_mmmc_loop_breaking {true | false}

Default: true

Data_type: Boolean

When set to `true`, loop breaking is handled independently per analysis view, according to the following rules in order of precedence:

- If an analysis view does not contain a combinational loop, it does not inherit any loop breaking done for other views.
- If a loop exists, and the user has supplied specific case analysis and `set_disable_timing` constraints to break the loop at a certain place, the specified loop break is honored.
- If a loop is still detected, the software automatically selects a point to break the loop, and applies the loop break consistently to any views where the loop exists.

When set to `false`, if a combinational loop is detected in any analysis view, the timing system breaks the loop by disabling an arc along the looping path. These combinational loop breaks are reflected in all analysis views, even if those views by themselves do not contain these looping paths.

Note: Cadence recommends that all combinational loops detected by the software be evaluated, to ensure that the loop is broken at the most ideal location. Automatic loop breaking decisions are made based on how the design is traversed. Small changes in the design can influence where loop breaking decisions are made.

To reduce risk and the need to re-evaluate loop-breaking decisions, it is highly recommended that once combinational loop breaks have been evaluated, the user add constraints to ensure that the loop is always broken in that location.

timing_enable_model_constraint_warnings

timing_enable_model_constraint_warnings integer

Default: 0

Data_type: Integer

Allows the software to report ignored warnings generated from boundary modeled blocks while processing constraints. These warnings can be reviewed later for debugging purposes.

When set to 0, the software suppresses the constraint warnings.

timing_enable_multi_drive_net_reduction_with_assertions

```
timing_enable_multi_drive_net_reduction_with_assertions {none | delay | all}
```

Default: none

Data_type: Enum

Allows reduction of multi-drive networks when constraint assertions are present on the network elements:

The following options are available:

- **none:** Reduction is not performed on portions of the network where constraints have been asserted.
- **delay:** Reduction of the multi-drive network will still be applied if only `set_annotated_delay` and `set_annotated_transition` constraints have been applied. Other constraints present on the multi-drive network can still prevent full reduction.
- **all:** The software ignores the presence of any assertions on the multi-drive network and forces reduction to occur.

timing_enable_multi_frequency_latch_analysis

```
timing_enable_multi_frequency_latch_analysis {true | false}
```

Default: false

Data_type: Boolean

Enables multi-frequency latch timing analysis of the latch time borrowing for when a data signal coming to a latch is controlled by a clock with a frequency different to a clock of the latch enabling signal.

timing_enable_multicycle_data_check_compatibility

```
timing_enable_multicycle_data_check_compatibility {true | false}
```

Default: false

Data_type: Boolean

Enables checking for SDC compatible data checks when `set_multicycle_path -start` parameter is specified.

timing_enable_path_delay_to_unconstrained_endpoints_compatibility

```
timing_enable_path_delay_to_unconstrained_endpoints_compatibility {true | false}
```

Default: false

Data_type: Boolean

When set to `true`, the software does not constrain paths with path delay - if the path ends at an unconnected input pin of a combination cell.

timing_enable_pessimistic_cppr_for_reconvergent_clock_paths

```
timing_enable_pessimistic_cppr_for_reconvergent_clock_paths{true | false}
```

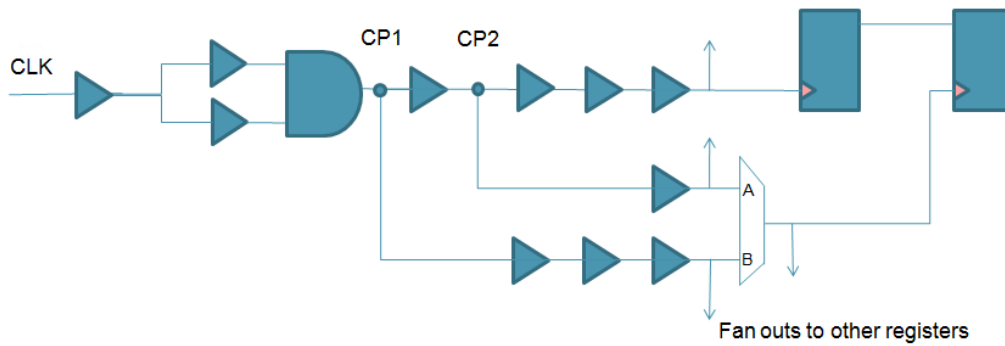
Default: false

Data_type: Boolean

When set to `true`, the software will consider all combinations of launch and capture clock path pairs to identify all possible common pins, and then selects the pin with minimum CPPR adjustment over all such common pins as the common pin.

When set to `false`, the default, the software will choose the CPPR common pin based on the commonality of the worst launch and worst capture path only.

Consider the following example,



In the above diagram, both capture paths are considered along with the single launch path, and the common pin is chosen as the pin with the least CPPR adjustment. The pin CP1 is chosen as a common pin and CPPR is reported up to this pin. The same common pin, CP1, will be used for CPPR adjustment whether the capture clock path goes through the mux A pin or the mux B pin. When this attribute is set to false, CPPR common pin is chosen by the commonality in the worst-case launch and worst-case capture paths. so CP2 is chosen as the common pin and CPPR is reported up to this pin.

timing_enable_power_ground_constants

```
timing_enable_power_ground_constants {true | false}
```

Default: false

Data_type: Boolean

Determines if case analysis is inferred from power and ground rail connections/nets, e.g., VDD and GND.

When set to `true`, the software recognizes the constants that come from power and ground rail connections.

When set to `false`, the software ignores these constants. This provides the most intuitive behavior with logical style netlist (and other tools that do not read physical data).

timing_enable_preset_clear_arcs

```
timing_enable_preset_clear_arcs {true | false}
```

Default: false

Data_type: Boolean

Determines whether the timing arcs are created to model the transition to active state (assertion) of the preset or clear pin, and the subsequent transition of the output to a controlled state.

When set to `true`, the software builds the arcs and the timer automatically analyzes paths through them (enable clock propagation through preset clear pins).

When set to `false`, the software does not build these arcs, because they are associated with analyzing the reset mode timing paths. Filtering them prevents unwanted interaction with functional mode timing analysis.

Note: The `timing_enable_preset_clear_arcs` attribute can also impact delay calculation results of a design. When an arc is disabled, the software will calculate delay using the linear table lookup. When an arc is not disabled, the delay calculations are made based on the quadratic table lookup.

Note: This attribute should be set to `true` when reset checks are performed on a dedicated reset SDC.

timing_enable_pulse_latch

```
timing_enable_pulse_latch {true | false}
```

Default: `true`

Data_type: Boolean

Enables pulse-latch analysis.

When set to `true`, the pulse-latch analysis mode provides several additional pulse-clock specific constraint specifications and reporting capabilities.

When in pulse-latch mode, you can use the following pulse clock constraints:

- `set_pulse_clock_max_width`
- `set_pulse_clock_min_width`
- `set_pulse_clock_max_transition`
- `set_pulse_clock_min_transition`

Additionally, in this mode the `report_constraint` command supports the following pulse clock check types:

- `pulse_clock_max_width`

- pulse_clock_min_width
- pulse_clock_max_transition
- pulse_clock_min_transition

timing_enable_si_cppr

timing_enable_si_cppr {true | false}

Default: true

Data_type: Boolean

Enables more accurate CPPR analysis when incremental delays are present.

When set to `true`, the incremental delay is not included in the CPPR calculation during setup analysis, but it is included in the CPPR calculation during hold analysis.

When set to `false`, the incremental delay is included in both setup and hold CPPR calculations.

timing_enable_simultaneous_setup_hold_mode

timing_enable_simultaneous_setup_hold_mode {true | false}

Default: true

Data_type: Boolean

Controls whether setup and hold checks are analyzed separately, or together on the same timing graph.

When set to `false`, switches the system from analyzing setup slacks to analyzing hold slacks.

When set to `true`, setup and hold slacks are analyzed concurrently. As a consequence, both `-early` and `-late` queries for reporting commands such as `report_timing` can be satisfied immediately, without additional calculation. Simultaneous mode is useful for immediately understanding the consequences to both setup and hold slacks from what-if operations and manual ECOs to the design.

Note: Simultaneous setup and hold mode can be used only for timing analysis; it cannot be used for any physical implementation steps.

Note: This attribute must always be true while running SI analysis.

timing_enable_spatial_derate_mode

```
timing_enable_spatial_derate_mode {true | false}
```

Default: true

Data_type: Boolean

Enables the spatial derate mode in SOCV analysis.

timing_enable_timing_window_pessimism_removal

```
timing_enable_timing_window_pessimism_removal{true | false}
```

Default: false

Data_type: Boolean

Removes pessimism in SI impact calculation. As a result of elongated arrival windows of attackers and victims, there may be pessimism in SI impact calculation due to a difference in the minimum-maximum arrivals of common clock paths shared between attackers and victims. This pessimism can be removed by adjusting attacker/victim arrival window with the difference in minimum-maximum delay of common clock path between victim and attacker.

When set to `true` (when CPPR is enabled), the software adjusts the timing windows of attackers/victims when the victim is located either on a data path or a clock path during path-based analysis.

For adjustment on victim nets lying on clock paths, full path path-based analysis can be enabled by setting the `timing_disable_retime_clock_path_slew_propagation` attribute to `false`.

timing_enable_tristate_clock_gating

```
timing_enable_tristate_clock_gating {true | false}
```

Default: false

Data_type: Boolean

The timing system infers gated-clock checks in the design where a data and clock path intersect at a combinational device. When this attribute is set to `true`, inferred gated-clock checks are added when clock and data signals converge through the tristate enable and data input of tristate buffers. By default (a setting of `false`), gated-clock checks are not added for these situations.

timing_enable_uncertainty_for_clock_checks

```
timing_enable_uncertainty_for_clock_checks {true | all | false | none | pulse_width |  
period | skew}
```

Default: `true`

Data_type: Boolean

Considers clock uncertainty when performing clock checks.

When set to `true`, the `set_clock_uncertainty` command supports clock uncertainty for period, pulse width, skew, and separation checks.

Note: This attribute overrides the `timing_enable_uncertainty_for_pulse_width_checks` setting.

timing_enable_uncertainty_for_pulse_width_checks

```
timing_enable_uncertainty_for_pulse_width_checks {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, considers clock uncertainty when performing minimum pulse width checks. Clock uncertainty is considered during checks of both user-defined minimum pulse widths (using the `.lib`).

Note: Specifying a value of `true` affects the following commands that take minimum pulse width checks into account:

- `report_timing -check_type pulse_width`
- `report_timing -check_clocks`
- `report_min_pulse_width`
- `report_analysis_coverage`

timing_enable_vtskew_derate_mode

```
timing_enable_vtskew_derate_mode {true | false}
```

Default: false

Data_type: Boolean

Enables voltage threshold (VT) skew analysis.

When set to `true`, the software analyzes the effects of VT skew in a multi-VT environment.

Due to variations in the manufacturing process, the voltage threshold (Vth) of a given Vth class (e.g., SVT, LVT, ULVT) can show some variations that might result in cells of that class operating either faster or slower than the nominal value. The Vth skew analysis automates the process of selecting which Vth permutations need to be run, which ensures a robust analysis and optimizes the software to run faster and efficiently.

timing_enable_wire_variation

```
timing_enable_wire_variation {true | false}
```

Default: false

Data_type: Boolean

Enables wire variation analysis in path-based analysis mode.

timing_enable_zero_delay_analysis_mode

```
timing_enable_zero_delay_analysis_mode  
{true | false}
```

Default: false

Data_type: Boolean

When set to `true`, the software will use zero delays during timing analysis.

timing_extract_model_aocv_mode

```
timing_extract_model_aocv_mode {graph_based | path_based | none}
```

Default: none

Data_type: Enum

Allows you to specify AOCV derating mode.

The valid values are described below:

- `graph_based`: Specifies that the delays in the timing model are derated using the graph-based stage counts.
- `path_based`: Specifies that the delays in the timing model are derated using the total path stage count of worst path between those pin pairs.
- `none`: Specifies that the delays in the timing model are under-rated base delays.

Note: Before setting this attribute to `graph_based` or `path_based` mode, it is mandatory that the AOCV libraries are read in and AOCV analysis is enabled.

timing_extract_model_case_analysis_in_library

```
timing_extract_model_case_analysis_in_library {true | false}
```

Default: true

Data_type: Boolean

Enables the software to model the constants as a function in the extracted timing model.

When set to `true`, the software models constants as functions in the library if these propagate to the output port.

When set to `false`, instead of modeling constants as functions inside the library, these are written as `set_case_analysis` in the constraints file (*.asrt).

timing_extract_model_check_arcs_as_lvf

```
timing_extract_model_check_arcs_as_lvf {true | false}
```

Default: true

Data_type: Boolean

Enables check arcs to be modeled as LVF arcs. By default, the check arcs will be written as nSigma numbers.

The software allows you to write mean+/-nSigma values for constraint arcs, while writing separate mean and sigma values for delay and transition data. This can be enabled by setting `timing_extract_model_check_arc_as_lvf` attribute to `false` and `timing_extract_model_write_lvf` attribute to `true`.

timing_extract_model_consider_design_level_drv

```
timing_extract_model_consider_design_level_drv{true | false}
```

Default: `true`

Data_type: Boolean

Considers `set_max_capacitance` and `set_max_transition` DRV constraints, that have been applied on the current design during model extraction.

When set to `true`, these constraints applied at the design level will be considered during model extraction. However, the maximum capacitance and maximum transition from various DRV constraints applied/inferred on the ports is considered.

When set to `false`, the software extracts the worse-case DRV values and the constraints applied on the current design will be ignored.

timing_extract_model_disable_cycle_adjustment

```
timing_extract_model_disable_cycle_adjustment {true | false}
```

Default: `false`

Data_type: Boolean

Controls cycle adjustment for multi-cycle paths.

When set to `false`, if the multi-cycle paths specified in the SDC at the block-level cannot be pushed out to the ports (since there are other paths going through these paths), then the Tempus software adjusts the delay/check numbers in the ETM to take into account the cycle adjustment.

When set to `true`, the cycle adjustment for the multi-cycle paths in ETM arc delay are disabled.

timing_extract_model_enable_combinational_arc_to_clock_source_on_output_ports

```
timing_extract_model_enable_combinational_arc_to_clock_source_on_output_ports {true | false}
```

Default: false

Data_type: Boolean

When set to `true`, the software models the combinational arc ending on an end point with clock asserted on it.

timing_extract_model_enable_output_pin_attributes

```
timing_extract_model_enable_output_pin_attributes String
```

Default: none

Data_type: String

Specifies the attributes attributes (e.g., `always_on`, `is_isolated`) to enable on output pins during ETM generation. You can use this global variable when backward compatibility with older ETM behavior is required. By default, this global variable is set to `none`.

timing_extract_model_enable_unconstrained_port_modelling

```
timing_extract_model_enable_unconstrained_port_modelling
```

```
{true | false}
```

Default: false

Data_type: Boolean

Enables extraction of unconstrained paths to the output ports.

When set to `false`, for unconstrained output ports, which have no inbound timing, there will be no arcs modelled for the paths.

When set to `true`, the software will extract dummy arcs inbound to unconstrained output ports that will show the following characteristics:

- Early delay/slew of last stage connected to the output ports will be modelled.
- Related pins will be specified as unique unconstrained internal pins.

- Extracted slew values will be modelled as unidimensional *_transition tables that are dependent on the output capacitance and real propagated slews inside a block.
- Extracted unidimensional delays will be displayed in the cell_* tables.

For more information, refer to "[Timing Model Generation](#)" chapter in the *Tempus Stylus User Guide*.

timing_extract_model_exhaustive_validation_dir

```
timing_extract_model_exhaustive_validation_dir {true | false}
```

Default: {}

Data_type: String

Controls the location of files to be stored in the ETM Extremity Validation mode.

When set to `false`, additional files generated during ETM Extremity Validation (but not generated during normal validation) will be stored in the current working directory.

When set to `true`, the location to place these files can be user-specified.

timing_extract_model_exhaustive_validation_mode

```
timing_extract_model_exhaustive_validation_mode{true | false}
```

Default: `false`

Data_type: Boolean

Enables extracted timing model (ETM) validation at the minimum and maximum indices of the slew load indices.

When set to `false`, the ETM validation is done only for the input slews/output-loads that have been asserted on the design during timing model extraction.

When set to `true`, the software uses the “Extremity-Validation” (EV-ETM) methodology that allows ETM to model the timing arcs in the design for a range of discrete input slews/output-loads, thus providing a more comprehensive validation.

timing_extract_model_gating_as_nochange_arc

```
timing_extract_model_gating_as_nochange_arc {true | false}
```

Default: true

Data_type: Boolean

Controls the conversion of clock gating checks to nochange arcs.

When set to `false`, the software disables conversion of clock gating checks to nochange arcs in the extracted timing model.

timing_extract_model_half_cycle_non_interface_paths

```
timing_extract_model_half_cycle_non_interface_paths {true | false}
```

Default: false

Data_type: Boolean

When set to `true`, the software models half cycle non-interface paths as MPW (minimum pulse width) checks in ETM.

timing_extract_model_ideal_clock_latency_arc

```
timing_extract_model_ideal_clock_latency_arc{true | false}
```

Default: true

Data_type: Boolean

When set to `true`, the latency arcs for generated clocks that are defined ideal are extracted with reference as master clock, during ETM extraction.

When set to `false`, arcs are not extracted for generated clocks that are defined ideal during ETM extractions.

timing_extract_model_include_applied_load_in_characterization_range

```
timing_extract_model_include_applied_load_in_characterization_range {true | false}
```

Default: true

Data_type: Boolean

Specifies that the actual load visible to the timer that will be included in the load characterization range for path end points.

timing_extract_model_include_applied_slew_in_characterization_range

```
timing_extract_model_include_applied_slew_in_characterization_range {true | false}
```

Default: true

Data_type: Boolean

Specifies that the actual timer slew must be included in the characterization range timing model.

timing_extract_model_non_borrowing_latch_path_as_setup

```
timing_extract_model_non_borrowing_latch_path_as_setup {true | false}
```

Default: false

Data_type: Boolean

Models the non-borrowing interface latch paths as setup checks.

timing_extract_model_slew_propagation_mode

```
timing_extract_model_slew_propagation_mode {worst_slew | path_based_slew}
```

Default: worst_slew

Data_type: Boolean

Specifies the type of slew propagation to use for generating extracted timing model (ETM).

- `worst_slew`: Propagates the worst slew of all the incoming arcs at a converging point for extracting the arcs.
- `path_based_slew`: Propagates the actual slew for the path elements for extracting the arcs.

timing_extract_model_synchronous_clocks_as_skew_checks

```
timing_extract_model_synchronous_clocks_as_skew_checks {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, the software additionally captures constraints in ETM for reg-to-reg paths, which are launched and captured by different synchronous clocks within the block. The software converts the slack of such paths into a skew-rising/falling constraint requirement between the two clock pins at the IP boundary.

timing_extract_model_write_clock_checks_as_arc

```
timing_extract_model_write_clock_checks_as_arc{true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, the clock checks (`min_period` and `min_pulse_width`) are modelled as arcs in the extracted timing models (ETM). Clock check arcs will be modelled as a single dimensional table with constrained and reference pin as clock pin itself.

When set to `false`, clock checks are modeled as scalar numbers in the extracted ETM.

timing_extract_model_write_clock_checks_as_scalar_tables

```
timing_extract_model_write_clock_checks_as_scalar_tables {true | false}
```

Default: `false`

Data_type: Boolean

Allows the extracted timing model to write the minimum pulse width or minimum period checks as scalar arcs.

timing_extract_model_write_function_for_combinational_arcs

```
timing_extract_model_write_function_for_combinational_arcs {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, the software writes the function attribute on ports with combinational arcs.

timing_extract_model_write_min_max_clock_tree_path

```
timing_extract_model_write_min_max_clock_tree_path {true | false}
```

Default: `true`

Data_type: Boolean

Writes the minimum/maximum worst latency for each clock source.

timing_extract_model_write_lvf

```
timing_extract_model_write_lvf {true | false}
```

Default: `false`

Data_type: Boolean

Generates extracted timing model (ETM) containing Liberty Variation Format (LVF) model.

When set to `true`, this attribute enables writing separate mean and variation tables for all the delay/constraint and transition tables.

When set to `false`, the variation data will be merged with the mean values. The mean +/- nsigma values of delay/constraint/transition data will be written in the ETM.

timing_full_context_flow

`timing_full_context_flow integer`

Default: 0

Data_type: Integer

Enables the full chip timing context mode. This variable captures the estimated slews and timing windows as part of the timing context in the boundary model.

When set to 1, the full chip timing analysis (STA or DSTA) run saves the timing context that is used by the block scope timing later in the flow.

timing_generate_normalized_driver_waveform

`timing_generate_normalized_driver_waveform {0 | 1}`

Default: true

Data_type: Boolean

Generates the analytical normalized driver waveform for timing library if the waveform is not specified in the library. The waveform is generated by weighed mean of linear and exponential waveform.

timing_generated_clocks_inherit_ideal_latency

`timing_generated_clocks_inherit_ideal_latency {false | true |
actual_partial_latency_path | ideal_partial_latency_path}`

Default: false

Data_type: Boolean

When the `set_clock_latency` command is used to define network latency for ideal clock mode at intermediate pins of a clock network, Tempus considers the latency. The

`timing_generated_clocks_inherit_ideal_latency` global variable allows including the closest applied network latency for ideal inheritance also. You can use the following options:

- `false`: Uses only the source latency of generated clocks, if it exists.
- `true`: Allows the generated clocks to inherit the ideal network latency of the parent clock.
- `actual_partial_latency_path`: Uses the network latency of the master clock asserted closest to the target pin and propagated delays for the path from this point to the assertion point of the generated clock.
- `ideal_partial_latency_path`: Uses the network latency of the master clock asserted closest to the target pin and ideal delays for the path from this point to the assertion point of the generated clock.

Note: If the `set_clock_latency -source` parameter is used to specify latency of a generated clock, the software will completely override any inheritance.

timing_generated_clocks_allow_nested_assertions

```
timing_generated_clocks_allow_nested_assertions {true | false}
```

Default: `false`

Data_type: Boolean

Controls clock latency handling for nested generated clocks that are created on the same timing pin as the master clock.

When set to `true`, the software handles generation of multiple nested generated clocks that are asserted on the same pin, such that the child clock inherits the clock latency value of its parent clock.

When set to `false`, the nested generated clocks asserted on the same pin do not inherit the latency values and are created with 0ns master clock latency. Any full clock timing path reported using these clocks will show the clock path up to the target pin of the generated clock.

timing_get_of_objects_hier_compatibility

```
timing_get_of_objects_hier_compatibility {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, the `get_pins -of_objects` parameter will return leaf pins connected to the specified objects.

When set to `false`, the `get_pins -of_objects` parameter will return hierarchical pins and leaf pins connected to the specified objects. In order to return leaf pins only when the attribute is set to `false`, the `-leaf` option must be specified along with the `get_pins -of_objects` parameter.

When set to `true`, the `get_cells -objects` parameter returns cells connected to the net at the same level of hierarchy as the net itself. Hierarchical ports will be returned if the net is connected to a hierarchical module.

timing_hierarchical_context_continue_on_top_block_clock_mismatch

```
timing_hierarchical_context_continue_on_top_block_clock_mismatch {true | false}
```

Default: `false`

Data_type: Boolean

Allows loading of the timing context model with unmapped clocks.

When set to `true`, the software allows the script to process even if there are any mismatches between the top and block-level clocks. The mismatches are reported in the *timing_context_clock_mapping_data.rpt* file, which shows data that allows you to determine if corrections or additions are required to the clock mapping file.

When set to `false`, the software will show an error if any of the top-level clocks cannot be matched with a block-level clock.

Note: This attribute should not be used for signoff.

timing_hier_obj_name_compatibility

```
timing_hier_obj_name_compatibility {true | false}
```

Default: `true`

Data_type: Boolean

Controls how hierarchical delimiters are interpreted in the search pattern, when the `-hier` parameter is used with the `get_*collection` commands.

When set to `true`, the software interprets the hierarchical delimiter as part of the search pattern

itself, and tries to match names that include the delimiter. For example, if you specify:

```
get_cells -hier L1/R*
```

The software searches at every level of the hierarchy for instances with the name `L1/R*`, completely ignoring the hierarchy separator:

When set to `false`, hierarchical delimiters are interpreted as hierarchy separators. For example, if you specify:

```
get_cells -hier L1/R*
```

The software searched for instances with names matching `R*` in the hierarchy level below `L1`:

```
TL2/L1/RX1 TL2/L1/RX2
```

timing_ignore_clock_based_path_delay_on_data_checks

```
timing_ignore_clock_based_path_delay_on_data_checks {true | false}
```

Default: `false`

Data_type: Boolean

Disables clock-based path delay application on data checks.

When set to `true`, the Tempus software enables the data checks to be reported by disabling the applied path delay. The "Data Check Setup" details are displayed in the timing report output.

When set to `false`, the timing reports display the "Path Delay" details. By default, this attribute is set to `false`.

timing_ignore_lumped_rc_assertions

```
timing_ignore_lumped_rc_assertions {true | false}
```

Default: `false`

Data_type: Boolean

When set to `false`, the `set_load` and `set_resistance` values override the actual extracted representation. A warning message is issued stating that the specified constraints on internal nets will override extracted parasitics on those nets.

When set to `true`, instructs the software to ignore `set_load` and `set_resistance` assertions on internal nets in the design. Parasitic data for internal nets normally should be provided by Tempus RC extraction, or external SPEF annotation.

Using `set_load` to specify I/O boundary loading on ports is supported by either setting of this attribute.

timing_io_use_clock_network_latency

```
timing_io_use_clock_network_latency {always | ideal}
```

Default: `ideal`

Data_type: Enum

Controls whether network latency of a reference clock is added or not to the data arrival time on the port. When set to `ideal`, the `set_input_delay` and the `set_output_delay` constraints will not add the network latency of the reference clock to the data arrival time on the port if the clock is in `propagated` mode.

When this attribute is set to `always`, network latency is added to the data arrival time on the port regardless of the clock propagation mode.

timing_ipd_allow_paths_for_asynchronous_clocks

```
timing_ipd_allow_paths_for_asynchronous_clocks {true | false}
```

Default: `true`

Data_type: Boolean

When set to `true`, the software will ensure that the paths between the asynchronous clocks are allowed to be treated as IPD (inter-power domain) paths.

For more information, refer to the "Inter Power Domain (IPD) Analysis" chapter in the *Tempus Stylus User Guide*.

timing_ipd_xdomain_path_filtering_threshold

```
timing_ipd_xdomain_path_filtering_threshold double
```

Default: 0.003

Data_type: Double

When this attribute is used, the specified threshold (in ns) will be used in the path filtering criteria during native inter-power domain (xdomain) solution.

timing_library_build_async_deassert_arc

```
timing_library_build_async_deassert_arc {true | false}
```

Default: true

Data_type: Boolean

Controls whether input to output arcs from the pre-set or clear pins transitioning to an active state are included, when the timing system is initialized.

timing_library_convert_async_setuphold_to_recrem

```
timing_library_convert_async_setuphold_to_recrem {true | false}
```

Default: 1

Data_type: Integer

Controls whether single-edged setup and hold checks on asynchronous pins in the Liberty library are inferred as recovery and removal checks.

This attribute provides support for older modeling styles in legacy libraries.

timing_library_generated_clock_use_group_name

```
timing_library_generated_clock_use_group_name {true | false}
```

Default: false

Data_type: Boolean

When set to `true`, the software uses the `generated_clock` group name when creating a generated clock from a library-generated clock group. When set to `false`, the software uses the pin name when creating the generated clock.

For example, assume the cell for instance `a/b` in the timing library contained the following generated clock group:

```
generated_clock(genclk2) {  
  clock_pin : clk1;  
  master_pin : "int/clk";  
  multiplied_by : 2;  
  duty_cycle : 50.0;  
}
```

By default (`false`), the software creates a generated clock with the following name:

`a/b/clk1`

If the `timing_prefix_module_name_with_library_generated_clock` attribute is set to `true`, and you set this attribute to `true`, the software creates a generated clock with the following name:

`a/b/genclk2`

If the `timing_prefix_module_name_with_library_generated_clock` attribute is set to `false`, and you set this attribute to `true`, the software creates a generated clock with the following name:

`genclk2`

timing_library_hold_constraint_corner_sigma_multiplier

`timing_library_hold_constraint_corner_sigma_multiplier double`

Default: 0.0

Data_type: Double

Applies user control multiplier to generate a constraint table using sigma values applied for hold arc.

timing_library_hold_sigma_multiplier

`timing_library_hold_sigma_multiplier double`

Default: 0.0

Data_type: Double

User control multiplier to generate constraint table applied for hold arc.

timing_library_ignore_combinational_arc_mismatch_tables

`timing_library_ignore_combinational_arc_mismatch_tables`

`{true | false}`

Default: false

Data_type: Boolean

When set to `true`, the software will give preference to `timing_type` over arc tables. This will change infer arcs based on the `timing_type` (liberty format) or timing tables settings. A warning message will be displayed when the preference (`timing_type` or arc tables) is changed.

timing_library_infer_async_pins_from_timing_arcs

`timing_library_infer_async_pins_from_timing_arcs {true | false}`

Default: false

Data_type: Boolean

When set to `true`, a pin is marked clear if it has timing arcs with type “clear” to output pins. The sequential block is not required to be present in the cell for marking a pin as clear.

timing_library_infer_cap_range_from_ccs_receiver_model

`timing_library_infer_cap_range_from_ccs_receiver_model {true | false}`

Default: false

Data_type: Boolean

Infers the rise or fall of capacitance range values from CCS receiver capacitance model.

When set to `false`, the rise/fall CCS capacitance range values are not inferred. The NLDM pin capacitance range values defined in the library are honored.

timing_library_infer_cap_range_from_ecsm_receiver_model

```
timing_library_infer_cap_range_from_ecsm_receiver_model {true | false}
```

Default: `false`

Data_type: Boolean

Infers the rise or fall of NLDM pin capacitance range from the ECSM capacitance range while calculating the total loading capacitance during switching power computation. If the attribute is set to `true` and the capacitance range is not defined in the library, the rise/fall capacitance range is inferred from the ECSM receiver capacitance models. The capacitance range values are derived either from the pin-level ECSM cap models or the timing-level ECSM cap models.

When this attribute is set to `false`, the rise/fall capacitance range values are not inferred, and the library values are honored.

timing_library_interpolate_drv_values

```
timing_library_interpolate_drv_values {true | false}
```

Default: `false`

Data_type: Boolean

Allows the software to use a range of trilib DRV values for performing delay calculations.

timing_library_infer_socv_from_aocv

```
timing_library_infer_socv_from_aocv {true | false}
```

Default: `false`

Data_type: Boolean

Controls the transformation of AOCV data.

This attribute infers sensitivity data from AOCV libraries, and must be set before loading any timing library data.

timing_library_read_ccs_noise_data

```
timing_library_read_ccs_noise_data{true | false}
```

Default: true

Data_type: Boolean

This attribute can be used when the CCS library contains noise models, and SI glitch analysis is being performed.

timing_library_read_without_ecsm

```
timing_library_read_without_ecsm {true | false}
```

Default: false

Data_type: Boolean

Disables loading ECSM data.

When set to `true`, the software allows you to change the mode of use (of ECSM timing model) if the timing library contains NLDM, ECSM, or CCS data in the same file.

Note: This attribute does not disable CCS data loading.

timing_library_read_without_power

```
timing_library_read_without_power {true | false}
```

Default: false

Data_type: Boolean

When set to `true`, this attribute allows the software to read timing libraries without power information.

When set to `false`, the software reads power data also.

Note: The `get_db` command shows the default value as `false`. However, this global variable is set to `true` by default in Tempus.

timing_library_read_without_sensitivity

```
timing_library_read_without_sensitivity {true | false}
```

Default: `false`

Data_type: Boolean

Disables loading of ECSM timing sensitivity data.

timing_library_scale_aocv_to_socv_to_n_sigma

```
timing_library_scale_aocv_to_socv_to_n_sigma <n_sigma value>
```

Default: `3.0`

Data_type: Double

AOCV derates are expected to be derived based on 3-sigma variation.

timing_library_setup_constraint_corner_sigma_multiplier

```
timing_library_setup_constraint_corner_sigma_multiplier double
```

Default: `0.0`

Data_type: Double

Applies user control multiplier to generate a constraint table using sigma values applied for setup arc.

timing_library_setup_sigma_multiplier

`timing_library_setup_sigma_multiplier double`

Default: 0.0

Data_type: Double

Applies user control multiplier to generate constraint table using sigma values applied for setup arc.

timing_library_term_voltage_from_lib_pin

`timing_library_term_voltage_from_lib_pin int`

Default: 1

Data_type: Integer

Enables the use of library pin voltage for reporting the term voltage.

timing_library_use_two_piece_receiver_cap

`timing_library_use_two_piece_receiver_cap {true | false}`

Default: false

Data_type: Boolean

Enables the use of 2-piece receiver capacitance constructs, if a library contains both the N-piece and 2-piece receiver capacitance construct.

When set to `true`, if a library has 2-piece and multi-piece pin capacitance, then the software gives priority to 2-piece models.

When set to `false`, the software reads the N-piece receiver capacitance constructs.

timing_library_zero_negative_timing_check_arcs

`timing_library_zero_negative_timing_check_arcs {true | false}`

Default: false

Data_type: Boolean

When set to `true`, the software checks for negative values in the timing arcs.

timing_multi_frequency_clock_rounding_factor

`timing_multi_frequency_clock_rounding_factor double`

Default: `1e-05`

Data_type: Double

Rounds off irrational numbers so that they can be read appropriately.

timing_normalized_driver_waveform_clip_linear_part

`timing_normalized_driver_waveform_clip_linear_part {0 | 1}`

Default: `false`

Data_type: Boolean

When set to 1, the linear ramp component in the normalized driver waveform is saturated to the maximum value, while generating the normalized driver waveform.

timing_normalized_driver_waveform_weight_factor

`timing_normalized_driver_waveform_weight_factor double`

Default: `0.5`

Data_type: Double

Defines the value of the weight of linear components in the algorithm for generation of normalized driver waveform.

When set to 1, the waveform will be purely linear. When set to 0, the waveform will be purely exponential. You can specify values within the range of 0 to 1.

timing_nsigma_multiplier

`timing_nsigma_multiplier` *double*

Default: 3.0

Data_type: Double

Controls the sigma multiplier to be used in SOCV mode.

timing_null_collection_return_compatibility

`timing_null_collection_return_compatibility` {true | false}

Default: false

Data_type: Boolean

Allows you to migrate previous release scripts to the new use model.

When the attribute is set to `true`, the software returns "NA" values that are consistent with the previous release. The nil return values are returned as an empty collection. This allows the return value to be used directly by collection iterators, such as `foreach_in_collection`.

The default value is `false`, which is consistent with the current release.

Note: This compatibility option may be removed from future releases. You should update your scripts to the new use model to avoid future problems.

timing_path_based_enable_bounding_box_for_io_paths

`timing_path_based_enable_bounding_box_for_io_paths` {true | false}

Default: false

Data_type: Boolean

Enables the bounding box mode for input-output paths.

timing_path_based_enable_closest_common_pin_driver_for_bounding_box

timing_path_based_enable_closest_common_pin_driver_for_bounding_box {true | false}

Default: false

Data_type: Boolean

Enables bounding box calculation using closest common pin driver from the clock mesh.

timing_path_based_epba_bounded_mode

timing_path_based_epba_bounded_mode {true | false}

Default: false

Data_type: Boolean

Enables the IPBA mode with exhaustive PBA analysis (run using the [report_timing - retime_mode exhaustive](#) command).

When set to `true`, the software ensures that all the paths to the violating endpoints are covered for PBA, and the slack is the worst PBA slack reported across all the paths to the endpoint. For any endpoint that is considered for IPBA analysis, the complete fanin cone (all the timing paths ending at that endpoint) is evaluated for any possible PBA violation.

timing_path_based_enable_report_launch_clock_path

timing_path_based_enable_report_launch_clock_path {true | false}

Default: true

Data_type: Boolean

When set to `false`, the software skips the reporting launch clock path in PBA path_type full report.

timing_path_based_enable_verbose_mode

timing_path_based_enable_verbose_mode value

Default: 1

Data_type: Enum

Controls printing of verbose messages during and at the end of path-based analysis (PBA).

You can specify one of the three values - 0, 1, and 2. These are described below:

- 0: Verbose messages are not reported during exhaustive PBA analysis. For default PBA analysis, the path count will be reported.
- 1: Reports the final message for PBA path count and max depth reached during exhaustive PBA. Progress messages are not reported.
- 2: Reports multiple progress messages after regular intervals (in addition to the final message for PBA path count and max depth).

This attribute impacts messages when the `report_timing` and `report_constraint` commands are used in PBA mode.

For example,

- The following commands print messages during and after PBA analysis:

```
set_db timing_path_based_enable_verbose_mode 2
report_timing -retime path_slew_propagation -retime_mode exhaustive -max_paths
500000 -nworst 1
```

The following progress messages are displayed during PBA analysis:

```
INFO: Path Based Analysis (PBA) currently performed on '108455' paths
INFO: Path Based Analysis (PBA) currently performed on '217574' paths
INFO: Path Based Analysis (PBA) currently performed on '327511' paths
INFO: Path Based Analysis (PBA) currently performed on '437472' paths
....
```

The following messages are displayed at the end of PBA analysis mentioning the PBA path count and max depth reached during exhaustive PBA:

```
INFO: Path Based Analysis (PBA) performed on total '2660526' paths
INFO: Max depth 9460 reached for end point 'top/hier1/hier2/flop/d' during Path
Based Analysis
```

timing_path_based_enable_exhaustive_depth_bounded_by_gba

```
timing_path_based_enable_exhaustive_depth_bounded_by_gba  
{true | false}
```

Default: true

Data_type: Boolean

Enables the timing report to bound PBA results by GBA slacks of the respective leftover nworst paths, whenever exhaustive PBA depth for an endpoint is exhausted.

When set to `true`, during exhaustive PBA, if you specify exhaustive depth as N, and an endpoint has nworst paths greater than N, then the timing report will show the worst path corresponding to worst slack out of worst PBA slack (out of first N paths) and GBA slack (for the (N+1)th path). Here, the variable N refers to exhaustive PBA depth and can be set using the `timing_pba_exhaustive_path_nworst_limit` attribute.

When set to `false`, during exhaustive PBA, if you specify exhaustive depth as N, and an endpoint has nworst paths greater than N, then the software will ignore the paths with depth greater than N, which may result in reporting an optimistic PBA slack for the endpoint reaching the depth.

timing_path_based_exhaustive_enable_design_coverage

```
timing_path_based_exhaustive_enable_design_coverage  
{true | false}
```

Default: false

Data_type: Boolean

Enables exhaustive path-based analysis (PBA) to automatically pick up and analyze all the violating endpoints in the design, irrespective of the specified maximum paths.

When set to `true`, exhaustive PBA analysis automatically picks up and analyzes the violating design endpoints instead of N endpoints (specified with the `report_timing -max_paths` parameter). After analyzing the violating endpoints, the software reports the top N worst endpoints - depending on the specified value of N.

When set to `false`, exhaustive PBA analysis restricts analysis (using the `-max_paths` parameter) to the specified number of N endpoints.

This attribute is valid for both EPBA as well as [infinite depth path-based analysis \(IPBA\)](#).

Note: When this attribute is enabled, there may be an increase in run time because the software will analyze all the violating endpoints and analysis will not be restricted to the specified N endpoints.

timing_path_based_exhaustive_max_paths_limit

timing_path_based_exhaustive_max_paths_limit *integer*

Default: 2000000

Data_type: Integer

Specifies the maximum number of paths that can be internally analyzed during exhaustive path-based analysis (PBA).

If the number of candidate paths during exhaustive PBA is greater than the specified max paths limit, then the software will skip path-based analysis of these paths beyond this limit. The paths exceeding the specified max paths limit will then be included as GBA in the final report output so that real violations are not missed.

timing_path_based_ipba_endpoints_max_limit

timing_path_based_ipba_endpoints_max_limit *integer*

Default: 100000

Data_type: Integer

Specifies the maximum number of worst critical GBA endpoints, which will be picked for IPBA analysis. The rest of the GBA endpoints will report GBA slack.

This attribute should be set to a number higher than the total number of GBA violating endpoints, in order to cover all the violating endpoints for IPBA.

To ensure that all the GBA violations are considered for IPBA analysis, the [report_timing - max_paths](#) parameter should be set equal to or greater than the total number of GBA violating endpoints.

timing_pba_exhaustive_path_nworst_limit

timing_pba_exhaustive_path_nworst_limit *<value>*

Default: 10000

Data_type: Integer

Specifies the maximum number of paths per endpoint (nworst) that will be re-timed in exhaustive path-based analysis (PBA).

In exhaustive PBA mode, the value specified with the `report_timing -nworst` parameter is used only during the final reporting. The value specified with the `timing_pba_exhaustive_path_nworst_limit` attribute (and not the `report_timing -nworst` value) is used to specify the maximum number of paths to be re-timed per endpoint.

If the limit specified by `timing_pba_exhaustive_path_nworst_limit` is reached and `timing_path_based_enable_exhaustive_depth_bounded_by_gba` attribute is set to:

- `true`: The worst of the current PBA slack is used and N+1th GBA path is reported.
- `false`: The current PBA slack is reported - which may be optimistic.

timing_path_based_report_analysis_summary_max_paths_limit

`timing_path_based_report_analysis_summary_max_paths_limit integer`

Default: 2000000

Data_type: Integer

Specifies the limit on the maximum paths that can be reported in PBA mode with the `report_analysis_summary` command.

timing_pll_clock_use_driving_cell

`timing_pll_clock_use_driving_cell {true | false}`

Default: false

Data_type: Boolean

Uses delay calculation of all the enabled arcs to the PLL clock output to compute the transition.

When set to `true`, the delay calculation of all enabled arcs to the PLL clock output will be used to compute the transition. If no driving arcs are available in the model or if they are disabled, the same idealized transition will be used.

When set to `false`, the software considers the PLL clock output to be electrically isolated from the input-side reference clock, and creates the output clock with an idealized transition. You can use

the `set_annotated_transition` command to customize the output clock transition.

timing_prefix_module_name_with_library_generated_clock

```
timing_prefix_module_name_with_library_generated_clock {true | false}
```

Default: true

Data_type: Boolean

When set to `true`, the software appends the instance name to the clock pin name when creating a generated clock. When set to `false`, the software uses only the clock pin name when creating a generated clock.

For example, assume the cell for instance `a/b` in the timing library contained the following generated clock group:

```
generated_clock(genclk2) {  
  clock_pin : clk1;  
  master_pin : "int/clk";  
  multiplied_by : 2;  
  duty_cycle : 50.0;  
}
```

By default (`true`), the software creates a generated clock with the following name:

```
a/b/int/clk
```

If you set the attribute to `false`, the software creates a generated clock with the following name:

```
int/clk
```

If you set this attribute to `false`, and

the `timing_library_generated_clock_use_group_name` attribute is set to `true`, the software creates a generated clock with the following name:

```
genclk2
```

timing_propagate_latch_data_uncertainty

```
timing_propagate_latch_data_uncertainty {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, uses the clock phase associated with a flush latch's data pin as the "from" clock phase for downstream uncertainty timing calculations

When set to `false`, the downstream uncertainty calculations are based on the clock phase of the flush latch's clock signal.

timing_property_arrival_window_enable_tcl_dict_format

```
timing_property_arrival_window_enable_tcl_dict_format {true | false}
```

Default: `false`

Data_type: Boolean

Allows the software to print the new format of the `arrival_window` property (see [get_property](#) command).

timing_property_arrival_clocks_consider_clock_source_data_phase

```
timing_property_arrival_clocks_consider_clock_source_data_phase {true | false}
```

Default: `true`

Data_type: Boolean

Considers the clock source data phases.

The [get_property](#) and [report_property](#) commands support the `arrival_clocks` collection property on pin objects. The `arrival_clocks` property considers data phases, clock source data phases, and latency phases.

timing_property_arrival_clocks_consider_data_phase

```
timing_property_arrival_clocks_consider_data_phase {true | false}
```

Default: `true`

Data_type: Boolean

Returns the pure data phase.

The `get_property` and `report_property` commands support the `arrival_clocks` collection property on pin objects. The `arrival_clocks` property considers data phases, clock source data phases, and latency phases.

timing_property_arrival_clocks_consider_latency_phase

```
timing_property_arrival_clocks_consider_latency_phase {true | false}
```

Default: `true`

Data_type: Boolean

Returns the latency phase.

The `get_property` and `report_property` commands support the `arrival_clocks` collection property on pin objects. The `arrival_clocks` property considers data phases, clock source data phases, and latency phases.

timing_property_return_null_collection_with_quiet

```
timing_property_return_null_collection_with_quiet {true | false}
```

Default: `false`

Data_type: Boolean

Allows you to query values of all the properties.

When set to `true`, the software enables returning values of properties of all the objects, when the `get_property -quiet` parameter is specified.

timing_rail_swing_checks_high_voltage_threshold

```
timing_rail_swing_checks_high_voltage_threshold double
```

Default: `0.95`

Data_type: Double

Specifies the voltage threshold values for high rail swing checks.

timing_rail_swing_checks_low_voltage_threshold

timing_rail_swing_checks_low_voltage_threshold double

Default: 0.05

Data_type: Double

Specifies the voltage threshold values for low rail swing checks.

timing_rcdb_allow_mismatch_option

timing_rcdb_allow_mismatch_option {true | false}

Default: true

Data_type: Boolean

Disables errors while reading boundary models.

When set to `true`, the software disables errors when reading the boundary model RCBD for cases where the contents do not match the boundary model netlist.

timing_recompute_sdf_in_setuphold_mode

timing_recompute_sdf_in_setuphold_mode {true | false}

Default: false

Data_type: Boolean

Controls the recomputing of SDF delays when the software is in simultaneous setup and hold analysis mode.

When the software is not in simultaneous setup and hold analysis mode, only one corner of data is stored at a time; therefore additional delay calculation is required for `write_sdf` command. When the software is in simultaneous setup and hold analysis mode, both sets of data are in memory at the same time, and additional delay calculation is not necessary.

If set to `false`, `write_sdf` will write out the delays from memory, instead of calling delay calculation.

Note: This attribute must be used with `timing_enable_simultaneous_setup_hold_mode true`.

timing_reduce_multi_drive_net_arcs

`timing_reduce_multi_drive_net_arcs {true | false}`

Default: `true`

Data_type: Boolean

Controls whether multi-drive or mesh networks are reduced to simpler, single driver models.

When set to `true`, the nets that meet the drivers' receivers arc threshold (defined using `timing_reduce_multi_drive_net_arcs_threshold` attribute), are reduced. The net arcs are created only from one driver or from the drivers for which you have set the constraints.

If you specify constraints on the net driven by parallel buffers, it might result in increased run time and memory degradation. By specifying timing constraints through nets, the constraints are moved to the drivers. This leads to the creation of net arcs from all the drivers, thus disabling net arc reduction. In such cases, you can select one driver of the net and use that driver to specify the constraints.

When set to `false`, the number of net arcs created for timing analysis for nets driven by parallel buffers are not reduced. If a net is driven by n parallel buffers and is connected to m sink pins, then the total number of net arcs created is $n*m$. For example, if n is 100 and m is 20000, the number of net arcs created for timing analysis will be two million. This may result in increased run time and memory usage.

timing_reduce_multi_drive_net_arcs_threshold

`timing_reduce_multi_drive_net_arcs_threshold int`

Default: 10000

Data_type: Integer

Sets a threshold number that is used to trigger the reduction of timing arcs of nets driven by parallel buffers. The reduction takes place if the number of arcs is greater than the threshold value and if the `timing_reduce_multi_drive_net_arcs` attribute is set to `true`.

timing_report_analysis_summary_csv_extended_new_format

`timing_report_analysis_summary_csv_extended_new_format {true | false}`

Default: `false`

Data_type: Boolean

Keeps the analysis summary CSV extended report in sync with the summary timing report. These reports are available when the respective `report_timing` parameters are specified:

- `-analysis_summary_csv_extended_file`
- `-analysis_summary_file`

When set to `true` (and when the `report_timing -analysis_summary_csv_extended_file` parameter is specified), the column name sequence will be from `fromClk` to `toClk`, that is, `fromClk` will be printed first, and then `toClk`. Also, the timing report will show "-", if `toClk` or `fromClk` is not present (and when `report_analysis_summary -expanded_summary_clocks` is set to `false`).

When set to `false`, the missing `toClk` or `fromClk` column will be represented as "**".

timing_report_arrival_property_worstcase_mode

`timing_report_arrival_property_worstcase_mode Integer`

Default: `0`

Data_type: Integer

Differentiates the clock and data phases based on the `arrival_*` properties (see `get_property` command) . The accepted values are 0, 1, 2, or 3.

timing_report_begin_end_pair_max_path_limit

`timing_report_begin_end_pair_max_path_limit integer`

Default: `2000000`

Data_type: Integer

Sets a limit on the maximum number of paths that can be reported when the `report_timing -begin_end_pair` parameter is specified.

Note: Higher maximum paths limit may lead to high memory usage and runtime.

timing_report_check_timing_unconstrained_endpoints_due_to_constants

```
timing_report_check_timing_unconstrained_endpoints_due_to_constants {true | false}
```

Default: false

Data_type: Boolean

Enables the software to flag unconstrained signals arriving at the end points due to case analysis.

When set to `false`, the `check_timing` command output will show warnings for the unconstrained endpoints.

When set to `true`, the command output will also show warnings for unconstrained endpoints due to case analysis under the subcategory `uncons_endpoints_due_to_case_analysis`. For more details, see [check_timing](#).

timing_report_clock_pin_as_begin_point

```
timing_report_clock_pin_as_begin_point {true | false}
```

Default: false

Data_type: Boolean

Allows the software to report clock pins as begin points.

timing_report_constraint_extended_cell_em_flow

```
timing_report_constraint_extended_cell_em_flow {true | false}
```

Default: false

Data_type: Boolean

When set to `true`, the software enables extended Cell EM flow.

timing_report_constraint_enable_extended_drv_format

```
timing_report_constraint_enable_extended_drv_format {true | false}
```

Default: false

Data_type: Boolean

Enables the software to report data as per clock/data and rise/fall phases.

timing_report_constraint_format

```
timing_report_constraint_format string
```

Default: " "

Data_type: String

Specifies the DRV fields that will be reported with the `report_constraint` command. For valid format column options, refer to `report_constraint -drv_fields`.

If DRV fields are not specified with this attribute (or with `report_constraint -drv_fields`), then the default set of DRV fields will be displayed. By default, the constraint report will display the pin names, the required and actual DRC values, and the view names.

Note: The reporting fields specified with the `report_constraint -drv_fields` parameter will override the attribute settings.

timing_report_constraint_rise_fall_clock_period_check

```
timing_report_constraint_rise_fall_clock_period_check {true | false}
```

Default: false

Data_type: Boolean

Enables reporting of rise-to-rise and fall-to-fall clock period checks separately for the `report_constraint` command.

When set to `false`, the software reports the rise-to-rise and fall-to-fall clock period checks under the same table in the report. With the `false` setting, when the `report_constraint -check_type clock_period` option is specified, the software reports violations that are the worst amongst all the rise-fall transitions for each point.

timing_report_constraint_report_all_drv_violation

```
timing_report_constraint_report_all_drv_violation {true | false}
```

Default: true

Data_type: Boolean

Enables reporting of all the design rule violations (DRV).

When set to `false`, the software reports the worst 1000 DRV violations per DRV check (specified with the `report_constraint -all_violators -drv_violation_type` command).

When set to `true`, the software will report all the DRV violations. You can use the `report_constraint -max_pins_per_drv` parameter to specify the maximum number of DRV violations to be reported.

Note: The `-max_pins_per_drv` parameter has higher precedence over the `timing_report_constraint_report_all_drv_violation` attribute setting.

timing_report_constraint_use_infinity_slack_for_unconstrained

```
timing_report_constraint_use_infinity_slack_for_unconstrained {true | false}
```

Default: false

Data_type: Boolean

Reports the timing slacks of unconstrained paths through the DRV violating pin as "INFINITY". By default, slacks of unconstrained paths are reported as * (asterisk).

timing_report_default_frequency_for_unconstrained_nets

```
timing_report_default_frequency_for_unconstrained_nets value
```

Default: 1e+09

Defines the frequency for unconstrained nets to be used in maximum capacitance DRV reporting.

timing_report_disable_max_paths_per_group

```
timing_report_disable_max_paths_per_group {true | false}
```

Default: false

Data_type: Boolean

Controls the interpretation of the `report_timing -max_paths` parameter when you request reporting across a set of path groups. By default (false setting), the specified `-max_paths` will be reported per path group. With the `true` setting, a total of the specified `-max_paths` will be reported across all of the specified path groups.

For Stylus flow cross-tool compatibility – you should specify a setting of `true`. For backward compatibility with the legacy UI flow – you should use a value of `false`.

For example,

When set to `false`:

Legacy UI: `report_timing -path_group {reg2reg in2reg} -nworst 2 ...`

Stylus UI: `report_timing -group {reg2reg in2reg} -nworst 2 ...`

The `report_timing` command will return up to 2 paths for the `reg2reg` group and also up to 2 paths for the `in2reg` group.

When set to `true`:

Legacy UI: `set timing_report_disable_max_paths_per_group true`

Stylus UI: `set_db timing_report_disable_max_paths_per_group true`

A total of 2 paths will be reported across both path groups. The paths may come entirely from one or the other group – or both, based on the slack value.

timing_report_drv_enable_clock_source_as_clock

```
timing_report_drv_enable_clock_source_as_clock {true | false}
```

Default: false

Data_type: Boolean

Enables the clock DRV for pure clock source path instead of data DRV.

timing_report_drv_enable_frequency_per_view

```
timing_report_drv_enable_frequency_per_view {true | false}
```

Default: true

Data_type: Boolean

Enables view-based frequency DRV flow.

When set to `true`, in the MMMC DRV flow, the software performs frequency computation for design pins as per the clock period of specific analysis views reaching a pin. In case of non-activity based flow, the frequency is computed as per the fastest clock of all the views.

When set to `false`, in the propagate activity flow, the software performs frequency computation for design pins from propagated activity on that pin.

For other flows. the software performs frequency computation for design pins as per the smallest clock period (largest clock frequency) from all the active analysis views.

timing_report_drv_per_frequency_per_input_slew

```
timing_report_drv_per_frequency_per_input_slew {true | false}
```

Default: false

Data_type: Boolean

Enables frequency dependent DRV lookup from a library.

When set to `true`, the software will enable the `report_constraint` command to honor `max_transition` or `max_capacitance` limit after worst-casing due to the following cases:

- One-dimensional frequency DRV commands, such as, `set_max_transition_per_freq` or `set_max_cap_per_freq`.
- Two-dimensional maximum transition or capacitance frequency/electro-migration tables.
- Other frequency independent DRV limit sources, e.g., `set_max_transition` from the SDC or library limit.

timing_report_drv_enable_ghz_notation_for_drv_fields

```
timing_report_drv_enable_ghz_notation_for_drv_fields {true | false}
```

Default: false

Data_type: Boolean

Enables reporting of clock frequency in GigaHertz (Ghz). The default frequency unit is MegaHertz (MHz).

timing_report_drv_enable_slew_threshold_scaling

```
timing_report_drv_enable_slew_threshold_scaling {true | false}
```

Default: false

Data_type: Boolean

Enables the transition constraints scaling as per the slew threshold value.

timing_report_drv_per_frequency

```
timing_report_drv_per_frequency  
{true | false}
```

Default: true

Data_type: Boolean

Improves reporting of maximum transition violations as per the worst limit defined using the `set_max_transition` and `set_max_transition_per_freq` commands, or the library specifications. This behavior applies to minimum/maximum transition and capacitance limits.

In MMMC designs, for reporting transition and capacitance violations, the software uses the worst transition limit specified in the transition limit sources, for example, library specifications, or `set_max_transition` constraints.

For frequency-based transition specifications, the transition violations are reported only on the basis of the transition limit specified with frequency-based specifications. The `set_max_transition` specifications are ignored by the software, if the values are pessimistic.

timing_report_drv_use_worst_timing_slack

```
timing_report_drv_use_worst_timing_slack  
{true | false}
```

Default: false

Data_type: Boolean

Enables reporting of the worst DRV slack across all the views for configurable DRV report.

When set to `true`, the software shows a non-default special case report, where timing slacks for MMC flow is the worst among all active views.

timing_report_enable_cpvr_point

```
timing_report_enable_cpvr_point { true | false}
```

Default: false

Data_type: Boolean

Enables the timing report to output CPVR point details.

When set to `true`, the `report_timing` command output shows the CPVR point field.

timing_report_enable_em_index_clipping_report

```
timing_report_enable_em_index_clipping_report {true | false}
```

Default: false

Data_type: Boolean

Enables index clipping reporting for EM (electro-migration) DRV (design rule violation) reports.

For more information, refer to the [Cell EM Analysis and ECO](#) chapter in the *Tempus Stylus User Guide*.

timing_report_enable_flag_field_symbols

`timing_report_enable_flag_field_symbols {true | false}`

Default: `false`

Data_type: Boolean

Enables reporting of Instance/Net related bits in the `report_timing` Flags column.

timing_report_enable_clock_to_for_unconstrained_paths

`timing_report_enable_clock_to_for_unconstrained_paths {true | false}`

Default: `false`

Data_type: Boolean

Enables unconstrained path reporting when the `report_timing -clock_to` parameter is specified.

timing_report_enable_lead_trail_to_rise_fall_map

`timing_report_enable_lead_trail_to_rise_fall_map {true | false}`

Default: `false`

Data_type: Boolean

Enables reporting of rise/fall edge times, instead of lead/trail edge times of a clock (reported using the `report_clocks` command).

timing_report_enable_markers

`timing_report_enable_markers{true | false}`

Default: `true`

Data_type: Boolean

Enables display of markers in reports.

When set to `true`, the `report_timing` output format displays markers that identify pins specified

using the `-from`, `-to`, or `-through` parameters. The markers, displayed as ‘->’, show up in the ‘timing point’ or ‘hpin’ column (if the ‘timing point’ column is not available) of the timing report. This feature provides better readability. When set to `false`, the timing report does not display the markers.

timing_report_enable_max_capacitance_drv_for_constant_nets

```
timing_report_enable_max_capacitance_drv_for_constant_nets {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, this attribute will perform max_capacitance design rule violation check on constant elements as well.

timing_report_enable_max_path_limit_warning

```
timing_report_enable_max_path_limit_warning {true | false}
```

Default: `false`

Data_type: Boolean

Determines if the number of reported paths for a particular clock group equals the specified maximum paths limit.

When set to `true`, the software will issue a warning message in case the number of reported paths for a clock group is equal to the given maximum paths limit. This applies to both graph- and path-based reporting modes.

This attribute is supported in clock group-based non-statistical reporting flows only. The clock group-based reporting mode can be enabled by setting `timing_report_group_based_mode` attribute to `true`.

timing_report_enable_qpin_as_start_for_begin_end_pair

```
timing_report_enable_qpin_as_start_for_begin_end_pair  
{true | false}
```

Default: `false`

Data_type: Boolean

Enables reporting of multiple paths with different q-pins having the same clock pin.

timing_report_enable_report_clock_timing_across_clock_pin

```
timing_report_enable_report_clock_timing_across_clock_pin {true | false}
```

Default: false

Data_type: Boolean

Enables fanout tracing across clock pins if any combinational arc is present.

timing_report_enable_si_debug

```
timing_report_enable_si_debug {true | false}
```

Default: false

Data_type: Boolean

Allows the software to capture signal integrity (SI) related details for each stage of a path being analyzed during path-based analysis in the form of attributes.

timing_report_enable_verbose_ssta_mode

```
timing_report_enable_verbose_ssta_mode {true | false}
```

Default: false

Data_type: Boolean

Enables reporting of sigma/multiplier for the timing report ([report_timing](#) command) header section in SSTA or SOCV mode.

timing_report_enable_unique_pins_multiple_capture_clock_paths

```
timing_report_enable_unique_pins_multiple_capture_clock_paths {true | false}
```

Default: false

Data_type: Boolean

Enables reporting of different paths with respect to the reference clocks in the unique pins.

timing_report_enable_write_twf_third_party_for_input_pins

```
timing_report_enable_write_twf_third_party_for_input_pins {true | false}
```

Default: false

Data_type: Boolean

When set to `true`, the software writes timing information for input pins in the third-party TWF file.

timing_report_fields

```
timing_report_fields string
```

Default: {instance arc cell delay arrival required}

Data_type: String

Specifies a `report_timing` format. See [report_timing -fields](#) for more information.

Valid format columns are:

adjustment, abs_delay_err, abs_transition_err, annotation, aocv_adj_stages, aocv_derate, aocv_weight, arc, arrival, arrival_mean, arrival_sigma, cell, delay, delay_mean, delay_sigma, direction, edge, fanin, fanout, flags, hpin, incr_delay, instance, instance_location, latch_window, load, module, net, pct_delay_err, pct_slew_err, phase, phys_info, pin, pin_load, pin_location, power_domain, required, retime_delay, retime_delay_mean, retime_delay_sigma, retime_mis_delta, retime_slew, retime_transition_mean, retime_transition_sigma, stage_count, transition, transition_mean, transition_sigma, socv_derate, spice_arrival, spice_delay, spice_slew, stolen, timing_point, total_derate, user_derate, voltage, vt_skew_derate, when_cond, wire_load, wlmodel

timing_report_generated_clock_info

`timing_report_generated_clock_info {true | false}`

Default: `true`

Data_type: Boolean

When set to `true` (the default), generated clock information is automatically added to the report if generated clocks are encountered in either the launching or latching clock paths. The generated clock information column is displayed if the 'Delay' field exists. If the 'Delay' field does not exist, then the generated clock adjustment column is added to the report only when explicitly specified using the `report_timing -fields` command.

timing_report_group_based_mode

`timing_report_group_based_mode {true | false}`

Default: `false`

Data_type: Boolean

Groups paths by clock domain.

When set to `true`, the behavior of the following reporting commands will change as follows:

report_timing

The behavior of the following `report_timing` parameters changes as follows:

- `-max_points`: In this mode you are not required to specify this parameter. If specified, the parameter is mapped to `-max_paths` option.
- `-max_paths M`: Reports `M` paths per clock group.
- `-nworst N`: Limits the number of paths per endpoint per clock group. By default, the value of `nworst` is 1.

The `report_timing` command will have the following four types of groups (if design does not have user-defined path groups):

- Asynch group: Recovery/removal checks.
- Clock gating group: Inferred and library clock gating checks.

- Default group: Combinational max delay or min delay paths.
- Clock Groups: Groups for all design clocks. For a design with three clocks, three groups will be reported, one group for each clock. Endpoints sharing the same reference clock will be part of the same group. If an endpoint belongs to multiple groups then it will be reported separately in each group.

Note: If user-defined path groups are specified, the software will override the path groups described above.

report_min_pulse_width

The `report_min_pulse_width` command reports pulse width slacks of all the clocks for a pin. In summary mode, the "clock" column will be displayed. In verbose mode (with `path_type` option), the behavior will be similar to the `report_timing` output in group-based mode.

report_constraint

The `report_constraint` command will report the following checks:

- For max_delay/setup and min_delay/hold type checks paths will be reported for each clock group separately.
- The behavior of pulse width check types will be the same as `report_min_pulse_width` command.

report_analysis_coverage

The `report_analysis_coverage` command will report a check at end points for all the clocks. In verbose mode, the "clock" column will be displayed.

timing_report_limit_max_path_count

`timing_report_limit_max_path_count value`

Default: 2000000

Data_type: integer

Sets the maximum number of violations (of data/clock checks) to be reported.

timing_report_max_transition_check_using_nsigma_slew


```
timing_report_max_transition_check_using_nsigma_slew {true | false}
```

Default: false

Data_type: Boolean

Enables the `report_constraint` command to use mean and sigma slew values for reporting transition values in DRV checks.

When set to `true`, in the SOCV flow the `report_constraint` command uses the mean plus n-sigma slew values for DRV checks.

When set to `false`, the `report_constraint` command will report only the mean slews.

timing_report_precision

```
timing_report_precision integer
```

Default: 3

Data_type: Integer

Controls the number of digits appearing after the decimal point in the `report_timing`, `report_nets`, `report_cell_instance_timing`, and `report_clocks` reports.

timing_report_property_fastest_clock_consider_data_phase

```
timing_report_property_fastest_clock_consider_data_phase
```

```
{true | false}
```

Default: false

Data_type: Boolean

Considers the data signals while determining the fastest clock arriving on any given pin.

When set to `false`, the clocks arriving in clock phase form will be considered for determining the fastest clock.

When set to `true`, the clocks arriving in clock phase as well as data phase form will be considered.

timing_report_pulse_width_matching_launch_capture_paths

```
timing_report_pulse_width_matching_launch_capture_paths {true | false}
```

Default: false

Data_type: Boolean

Reports pulse width paths that have matching launch and capture paths - the count of pins and the pins must match exactly. The paths that do not meet this condition will not be reported.

When set to `true`, the software will impact the minimum pulse width report that is generated with the `report_min_pulse_width` command along with the `-path_type short` and `-path_type full_clock` options.

timing_report_redirect_message_types

```
timing_report_redirect_message_types  
{none | info}
```

Default: none

Data_type: Enum

Redirects reporting messages to a report file.

When set to `info`, the path count and max depth (displayed when the `report_timing` command is run in PBA mode) will be appended to the final report output.

When set to `none`, the reporting messages are not redirected to a report file.

timing_report_retime_formatting_mode

```
timing_report_retime_formatting_mode {manual | retime_compare | retime_replace}
```

Default: manual

Data_type: Enum

Manages the re-timing fields in the reporting format. Based on the given settings, this attribute automatically replaces (or adds) the default columns with the respective re-timing columns. For

example, in case of delay column, 'Retime Delay' will be added or replaced automatically.

timing_report_skip_constraint_loop_check

```
timing_report_skip_constraint_loop_check {true | false}
```

Default: true

Data_type: Boolean

Allows you to skip loop checking when the `check_timing` command is used.

When set to `true`, the software skips reporting of loops in case the timing graph inside a loop is broken. This may happen if some of the commands, such as, `create_clock`, `set_input_delay`, or `set_output_delay` are defined inside a loop.

Note: When this attribute is set to `true`, the `check_timing` command will report lesser loops than the ones reported in the CTE_loops.rpt report - which reports all the loops.

timing_report_skip_icg_from_all_fanin_fanout_collection

```
timing_report_skip_icg_from_all_fanin_fanout_collection {true | false}
```

Default: false

Data_type: Boolean

When set to `true`, the software skips ICG clock pins from the `all_fanin` (or `all_fanout`) command collection, when queried with startpoints/endpoints.

timing_report_socv_summary_mean_sigma

```
timing_report_socv_summary_mean_sigma {true | false}
```

Default: false

Data_type: Boolean

Prints the report header in new format when the `report_timing` command is run in SOCV mode. The new header contains nsigma, mean, and sigma details.

timing_report_timing_header_detail_info

timing_report_timing_header_detail_info {default | extended}

Default: default

Data_type: Enum

Controls the timing report (generated using the [report_timing](#) command) header format. You can specify one of the following options:

- When set to `default`, the software generates timing reports with a report header that includes the following information:
 - Tempus software version number
 - Platform on which the software is being run
 - Time when the report was generated
 - Command used to generate the report
- When set to `extended`, the software generates timing reports with additional header information given below:
 - The analysis mode - best case worst case or on chip variation
 - The active views (displayed only in MMMC mode)
 - The clock reconvergence pessimism (CPPR)
 - The latch slack mode
 - The specified clock propagation value
 - The specified delay calculation engine

Note: You can use this attribute when `timing_disable_report_header_info` is set to `false`.

timing_report_unconstrained_paths

timing_report_unconstrained_paths {true | false}

Default: `false`

Data_type: Boolean

When set to `true`, the `report_timing` command reports unconstrained paths if the software cannot find any constrained paths to report.

When set to `false`, the `report_timing` command reports only the constrained paths.

Note: The behavior of `timing_report_unconstrained_paths` attribute is not the same as the behavior of `report_timing -unconstrained` command. If there are several paths to the same endpoint from different begin points, including constrained and unconstrained paths, the `report_timing -unconstrained` parameter always reports unconstrained paths, even if a constrained path can be reported. Whereas the `timing_report_unconstrained_paths` attribute reports constrained paths but not unconstrained paths.

timing_report_unconstrained_path_early_late_header

```
timing_report_unconstrained_path_early_late_header {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, this software provides information in the timing report header that indicates whether an unconstrained path is a late or early path.

timing_report_use_receiver_model_capacitance

```
timing_report_use_receiver_model_capacitance {true | false}
```

Default: `false`

Data_type: Boolean

Enables the max_capacitance violations based on dynamic pin capacitance.

When set to `true`, the software checks for both the input level and timing arc level dynamic pin capacitance tables for worst-case pin capacitance table for given input slew. If conditional pin capacitance tables (or `timing_arcs`) exist, the worst-case is used among all the conditions.

This attribute performs static pin capacitance inference after delay calculation, so the delay

computation is not impacted. The pin capacitance is used only for reporting and max_cap violations.

timing_report_use_worst_parallel_cell_arc

```
timing_report_use_worst_parallel_cell_arc {true | false}
```

Default: false

Data_type: Boolean

When set to `false`, setting the `-nworst` parameter reports several paths by selecting parallel arcs in the library cell between two pins.

When set to `true`, only a single worst path is reported by selecting the worst delay arc among all parallel arcs of a cell. For example, for reporting timing through A to Y pins (where the cell is a 2-input cell), the parallel arc is applicable for the same input and output pin. So the worst arc for A->Y is chosen from all arcs of A->Y.

timing_resolve_driver_conflicts

```
timing_resolve_driver_conflicts {conservative | aggressive}
```

Default: aggressive

Data_type: Enum

Resolves constant propagation results on nets with multiple drivers.

When the attribute is set to `conservative` mode, the net values are as follows:

- If all the drivers are 1, the net will be driven to value 1
- If all the drivers are 0, the net will be driven to value 0
- In other cases, the net value will be `Undefined`

When the attribute is set to `aggressive` mode, the net values are as follows:

- If some of the drivers are 0, the net value will be 0
- If some of the drivers are 1, but none are 0, then the net value will be 1
- In other cases, the net value will be `Undefined`

timing_scaling_for_negative_checks

```
timing_scaling_for_negative_checks {default | divider | multiplier}
```

Default: default

Data_type: Enum

Modifies the scaling of the negative timing analysis check value. This variable does not affect the positive checks. You can specify the following options:

- **default:** Negative check is multiplied by (2.0 - derating factor), where the derating factor is the value that you set using the `set_timing_derate` command.
- **divider:** Divides the negative check value by the derating factor.
- **multiplier:** Multiplies the negative check value by the derating factor.

timing_scaling_for_negative_delays

```
timing_scaling_for_negative_delays {default | divider | multiplier}
```

Default: default

Data_type: Enum

Modifies the scaling of the negative delay value to be used during timing analysis. This attribute does not affect the positive delay. You can specify the following options:

- **default:** The negative delay is multiplied by (2.0 - derating factor), where the derating factor is the value that you set using the `set_timing_derate` command.
- **divider:** The software divides the negative delay value by the derating factor.
- **multiplier:** The negative delay value is multiplied by the derating factor.

Currently, the constraints are used during timing in the flattened mode, so the internal ILM instances are seen instead of the LEF pins of the ILMs. Therefore, reading the bounding box constraints can cause errors without this variable set. If you want to see the LEF pins of the ILM in GUI, the design must be in the unflattened mode.

timing_sdf_adjust_negative_setuphold

```
timing_sdf_adjust_negative_setuphold {true | false}
```

Default: false

Data_type: Boolean

Controls how the setup and hold check values are adjusted when both are initially negative in the SDF file.

When set to `true`, the software sets the smaller check delay value to a negated larger check delay value. For example, if the initial setup/hold value is equal to -1/-2. then the value -2 from the setup is smaller and it is reset to a negated value of the larger one (-1 from hold). So the adjusted setup/hold value is equal to -1/1.

When set to `false`, both the negative setup and hold check values are set to 0.

The command order for using this attribute is as follows:

- `read_sdf`
- `timing_sdf_adjust_negative_setuphold {true | false}`
- `write_sdf`

The timing report (`report_timing`) results are not impacted.

timing_sdf_enable_setuphold_scond_ccond

```
timing_sdf_enable_setuphold_scond_ccond {true | false}
```

Default: false

Data_type: Boolean

When set to `true`, enables proper generation of SDF (Standard Delay Format) with `scond` and `ccond` qualifiers on `SETUPHOLD` and `RECREM` timing checks.

Since some SDF readers do not accept `scond` and `ccond` qualifiers this attribute is set to `false` by default.

timing_self_loop_paths_no_skew_max_depth

timing_self_loop_paths_no_skew_max_depth *int*

Default: 10

Data_type: Integer

Identifies self-loop paths at the specified depth.

For example:

```
set_db timing_self_loop_paths_no_skew_max_depth 20
```

Allows the timer traverse 20 timing arcs (10 stages of logic) before abandoning the search for a self loop.

timing_self_loop_paths_no_skew_max_slack

timing_self_loop_paths_no_skew_max_slack *value*

Default: 0.0

Data_type: Double

Identifies self-loop paths at the specified threshold.

For example:

```
set_db timing_self_loop_paths_no_skew_max_slack 1.0
```

In this case, all timing endpoints with a worst slack less than 1.0 are considered for self loop skew removal.

timing_set_clock_source_to_output_as_data

timing_set_clock_source_to_output_as_data {true | false}

Default: false

Data_type: Boolean

When set to `true`, causes a clock source path leading to an output or bidi port to be treated as a

data path if there is `set_output_delay` or `set_data_check` assertion on the port.

timing_socv_preserve_variation_with_annotations

```
timing_socv_preserve_variation_with_annotations {true | false}
```

Default: false

Data_type: Boolean

Preserves the sigma value computed by delay calculation along with the annotation from SDC.

In the clock mesh flow, it is possible to preserve sigma values even after applying annotation on slew/delay through SDC.

When this attribute is set to `true`, the software accepts the sigma value computed by delay calculation (for nets that have delay/slew annotations from SDC, e.g., clock-mesh), along with the annotation from SDC.

timing_socv_rc_variation_mode

```
timing_socv_rc_variation_mode {true | false}
```

Default: true

Data_type: Boolean

Enables interconnect variation mode in SOCV mode.

timing_socv_sigma_spatial_multiplier

```
timing_socv_sigma_spatial_multiplier {true | false}
```

Default: 1.0

Data_type: Double

Provides a fractional percentage multiplier, which allows you to select some (or all, none) of the

spatial derate to be applied.

timing_socv_view_based_nsigma_multiplier_mode

```
timing_socv_view_based_nsigma_multiplier_mode {true | false}
```

Default: true

Data_type: Boolean

Enables setting of view-based separate setup and hold sigma multipliers.

By default or with an explicit true setting, the timing system is configured to support different N-sigma multiplier values for Setup vs. Hold – and for different analysis views. The specific N-sigma multipliers can be set using the `set_socv_reporting_nsigma_multiplier` command.

timing_spatial_derate_chip_size

```
timing_spatial_derate_chip_size value_in_microns
```

Default: 1e+30

Data_type: Double

Allows you to define the chip size.

The chip size is the diagonal length of a rectangle that is formed by the chip's length and width. You can obtain the diagonal distance by calculating the square-root of the $\text{Length}^2 + \text{Width}^2$. The diagonal distance should always be specified in units of microns.

The chip size can be picked in the following ways (in order of highest to lowest precedence):

- When set to a value other than 1e+30, the specified value will have the highest precedence.
- If the physical floorplan information is available, the chip size will be derived automatically from the floorplan.
- If the SPEF parasitic files containing the coordinate information are loaded, then the chip size will be automatically derived.

timing_spatial_derate_distance_mode

```
timing_spatial_derate_distance_mode {chip_size}
```

Default: chip_size

Data_type: Enum

Allows you to select the graph-based analysis (GBA) spatial mode - chip_size.

This attribute can be used when the spatial derate mode is enabled using the `timing_enable_spatial_derate_mode true` setting.

timing_suppress_escape_characters

```
timing_suppress_escape_characters {true | false}
```

Default: true

Data_type: Boolean

Suppresses the reporting of escape characters in timing object names.

When the attribute is set to `true`, all the reporting and collection commands report the timing objects names (e.g., pins, ports, or instances) without the escape characters.

timing_suppress_ilm_constraint_mismatches

```
timing_suppress_ilm_constraint_mismatches {true | false}
```

Default: false

Data_type: Boolean

When set to `true`, the software suppresses all error and warning messages related to objects not found when loading SDC constraint files for the ILM flow.

In the current ILM flow, SDC constraints originally specified against the complete flat netlist for the design do not get filtered when being applied against the ILM view of the design. Constraints that reference parts of the design that were pruned cause warnings and errors during constraint loading.

timing_time_unit

`timing_time_unit` *string*

Default: none

Data_type: String

Specifies the timing units to be used for timing constraints and timing reports. The default units for timing are inferred from the timing libraries. You can override this inference by explicitly setting the value to `none`, `1ns`, `1ps`, `10ps`, or `100ps`. Setting this variable explicitly is the same as executing the `set_library_unit -time` command to set the time units.

timing_use_clock_pin_attribute_for_clock_net_marking

`timing_use_clock_pin_attribute_for_clock_net_marking` {true | false}

Default: false

Data_type: Boolean

Allows the propagation of clocks to pins with clock attributes regardless of the presence of check arcs or trigger arcs at the pins.

timing_use_latch_early_launch_edge

`timing_use_latch_early_launch_edge` {true | false}

Default: true

Data_type: Boolean

Enables the latch use early launch edge feature.

During timing analysis when time borrowing and clock path pessimism removal (CPPR) is enabled, the given time is calculated using both borrow time and CPPR time. This may result in unnecessary timing violations. Since CPPR is an accurate means of pessimism removal, you are required to set the `timing_use_latch_early_launch_edge` attribute to `false` so that CPPR is applied to all the paths in the design.

When set to `true`, the launch clock path of timing paths starting at a latch is timed in early mode for

setup analysis, and for late mode for hold analysis. No CPPR credit is given to the timing path.

When set to `false`, the behavior is like for any other path. The launch clock path of timing paths starting at a latch is timed in late mode for setup analysis, and early mode for hold analysis. The CPPR credit is given to the path according to the attribute CPPR settings.

timing_use_incremental_si_transition

```
timing_use_incremental_si_transition {true | false}
```

Default: `false`

Data_type: Boolean

Enables usage of incremental slew during DRV violation reporting.

timing_use_latch_time_borrow

```
timing_use_latch_time_borrow {true | false}
```

Default: `true`

Data_type: Boolean

When set to `false`, does not consider time borrowing during timing analysis. Time borrowing is the amount of time borrowed by a previous logic.

timing_use_verilog_for_model_netlist

```
timing_use_verilog_for_model_netlist int
```

Default: `0`

Data_type: integer

Allows you to choose whether to write the scope and boundary model netlists in Verilog format instead of binary.

When set to `1`, the software will use Verilog format for saving model nets.

timing_waveform_aware_pulse_width_checks_high_voltage_level

timing_waveform_aware_pulse_width_checks_high_voltage_level *double*

Default: 0.95

Data_type: Double

Specifies the high voltage level to be used for waveform-aware pulse width checks.

timing_waveform_aware_pulse_width_checks_low_voltage_level

timing_waveform_aware_pulse_width_checks_low_voltage_level *double*

Default: 0.05

Data_type: Double

Specifies the low voltage level to be used for waveform-aware pulse width checks.

timing_write_sdf_no_escape_backslash

timing_write_sdf_no_escape_backslash {true | false}

Default: false

Data_type: Boolean

Supports various patterns of the escape character.

For example, consider the following input Verilog expression:

```
\a\b\c\d
```

When the attribute is set to `true`, the `write_sdf` command removes the first escape character and outputs `a/b\c\d` in the SDF file.

When set to `false`, the command removes all the escape characters and outputs `a/b/c/d`.

timing_library

The root attributes in the `timing_library` category are:

- `aocv_library`
- `bank_based_multibit_inferencing`
- `disable_when_checks`
- `enable_library_pins_sorting_in_mmmc`
- `honor_valid_location`
- `ignore_pin_error_in_test_cell_function`
- `ignore_scan_combinational_arcs`
- `ignore_sigma_arc_mismatch`
- `init_design_mmmc_skip_inactive`
- `large_cell_arc_threshold`
- `link_library`
- `mark_async_pin_using_timing_arcs`
- `mark_macro_as_power_switch_cell`
- `mark_valid_lp_cell_as_usable`
- `multibit_allow_async_phase_map`
- `multibit_auto_exclude_registers_with_exceptions`
- `multibit_cells_from_different_busses`
- `multibit_preserve_inferred_instances`
- `multibit_preserved_net_check`
- `multibit_seqs_instance_naming_style`
- `multibit_seqs_members_naming_style`
- `multibit_split_string`
- `operating_conditions`
- `opt_multibit_combinational_name_concatenate_string`

- `opt_multibit_seqs_name_concatenate_string`
- `opt_use_default_related_pg_pin_for_always_on`
- `opt_use_next_state_type_only_to_assign_sync_control`
- `override_library_max_drc`
- `parse_lib_moments_table`
- `reload_when_for_macro_cell`
- `socv_library`
- `support_internal_pg_pins`
- `support_master_slave_flop`
- `support_multi_seq_elements`
- `support_multi_seq_scan_latch`
- `syn_exact_match_sequential_sync_control`
- `target_library`
- `timing_enable_set_reset_latch_preset_clear_arcs`
- `timing_library_allow_invalid_primary_power_pins_lib_cell`
- `timing_library_allow_multiple_sync_control`
- `timing_library_ignore_attribute_check_during_pin_convert`
- `timing_library_infer_cap_range_from_c1cn_dynamic_pin_cap_model`
- `timing_library_infer_cap_range_from_dynamic_pin_cap_model`
- `timing_library_limit_messages`
- `timing_library_look_up_drv_per_frequency`
- `timing_library_multibit_force_merge_combinational_cells`
- `timing_library_multibit_force_merge_iso_cells`
- `timing_library_multibit_force_merge_sequential_cells`
- `timing_library_multibit_use_combinational_cells`
- `timing_library_respect_async_control_priority`
- `timing_library_sequential_in_out_phase_opt`

- [timing_library_support_3d_table_power_arc](#)
- [timing_library_support_combinational_clock](#)
- [timing_library_support_merge_libs](#)
- [timing_library_support_serial_scan_in_multibit_cell](#)
- [timing_library_support_transparent_latch_group](#)
- [timing_library_suppress_library_messages](#)
- [timing_library_use_sequential_test_cell](#)
- [treat_non_seq_arc_cell_as_unusable](#)
- [turbo_lib_loading](#)
- [use_area_from_lef](#)
- [use_compatibility_based_grouping](#)
- [use_main_cell_output_function_for_test_cell](#)
- [use_multibit_cells](#)
- [use_multibit_iso_cells](#)
- [use_multibit_seq_and_tristate_cells](#)
- [use_scan_seqs_for_non_dft](#)

To set the value of any root attribute, use the [set_db](#) command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the [get_db](#) command:

```
get_db attribute_name
```

To get the current value of all attributes in the `timing_library` category, use the following command:

```
get_db -category timing_library
```

aocv_library

```
aocv_library string
```

Description

Default: ""

Data_type: string, read/write

Specifies the list of advanced on-chip variation (AOCV) libraries.

bank_based_multibit_inferencing

```
bank_based_multibit_inferencing {false | true}
```

Description

Default: false

Data_type: bool, read/write

Enables predefined multibit cell inferencing (MBCI) without limiting the multibit mapping to specific multibit library cells. The tool automatically finds a suitable multibit cell. As this attribute enables *predefined* multibit cell inferencing, it implies a forced type of mapping and is therefore not QoR-driven.

Related Information

- [synthesize_design](#)

disable_when_checks

```
disable_when_checks {true| false}
```

Default: true

Data_type: bool, read/write

Disables the processing of long when condition having bus operands.

enable_library_pins_sorting_in_mmmc

```
enable_library_pins_sorting_in_mmmc {true| false}
```

Description

Default: `true`

Data_type: `bool`, read/write

Enables library pins sorting in MMMC flow.

honor_valid_location

```
honor_valid_location {true| false}
```

Description

Default: `false`

Data_type: `bool`, read/write

Controls whether or not to honor the valid location defined on the power intent library cells. By default, the valid location defined in the library will be ignored. If you set this attribute to `true`, the valid location defined in the library will be honored and passed on to downstream tools.

Related Information

- `commit_power_intent`

ignore_pin_error_in_test_cell_function

```
ignore_pin_error_in_test_cell_function {true| false}
```

Description

Default: `false`

Data_type: `bool`, read/write

Controls whether to ignore the pin errors in the test cell function. This attribute must be set before you read in the libraries.

Related Information

- `timing_library_use_sequential_test_cell`

ignore_scan_combinational_arcs

`ignore_scan_combinational_arcs {false | true}`

Description

Default: `true`

Data_type: `bool`, read/write

Controls whether to ignore the combinational arcs from scan input pins to output pins of scan flip-flops during timing analysis and optimization.

Example

```
set_db / .ignore_scan_combinational_arcs false
```

Related Commands

- `report_timing`

ignore_sigma_arc_mismatch

`ignore_sigma_arc_mismatch {true | false}`

Description

Default: `true`

Data_type: bool, read only

Indicates whether sigma arc inconsistency is ignored across views during MMMC consistency check.

init_design_mmmc_skip_inactive

```
init_design_mmmc_skip_inactive {true | false}
```

Description

Default: false

Data_type: bool, read/write

Controls the establishment of library and skipping the inactive library domain in MMMC flow for `init_design` speedup.

large_cell_arc_threshold

```
large_cell_arc_threshold integer
```

Description

Default: 200

Data_type: int, read/write

Specifies the threshold arc count to create a separate binary decision diagram (BDD) for the cell. Cells with leakage arc count greater than this value have a separate BDD for power computation.

link_library

```
link_library string
```

Description

Default: ""

Data_type: string, read/write

Specifies the list of link libraries. Link libraries are technology libraries which the tool uses to resolve cell references. The link libraries contain the descriptions of cells (macros like RAMS, pads, PLLs, and so on) in a mapped netlist.

Related Information

- [target_library](#)

mark_async_pin_using_timing_arcs

```
mark_async_pin_using_timing_arcs {true | false}
```

Description

Default: true

Data_type: bool, read/write

Marks library pin with recovery or removal timing arcs as asynchronous.

mark_macro_as_power_switch_cell

```
mark_macro_as_power_switch_cell {true | false}
```

Description

Default: false

Data_type: bool, read/write

Marks the power switch cell defined as macro as the power switch cell.

mark_valid_lp_cell_as_usable

```
mark_valid_lp_cell_as_usable {true | false}
```

Description

Default: false

Data_type: bool, read/write

Marks valid low power cells as usable.

multibit_allow_async_phase_map

multibit_allow_async_phase_map {inherited | true | false}

Description

Default: true (root), inherited (inst and module)

Data_type: bool (root), enum (inst and module), read/write

Controls whether asynchronous pins can be interchanged during multibit cell inferencing. By default, asynchronous pin phase mapping will be enabled.

The values are:

inherited	Inherits the value from the multibit_allow_async_phase_map root attribute.
false	Prevents phase inversion on the module during multibit mapping.
true	Allows phase inversion on the module during multibit mapping.

Examples

Consider the following netlist after mapping and before incremental optimization:

```
module test(resetn, clk, in, out);
  input resetn, clk;
  input [3:0] in;
  output [3:0] out;
  DFSF cnt_0_reg[0] (.SDN (resetn), .CP (clk), .D(in[0]), .Q (out[0] ));
  DFCF cnt_0_reg[1] (.CDN (resetn), .CP (clk), .D(in[1]), .Q (out[1] ));
  DFCF cnt_0_reg[2] (.CDN (resetn), .CP (clk), .D(in[2]), .Q (out[2] ));
  DFSF cnt_0_reg[3] (.SDN (resetn), .CP (clk), .D(in[3]), .Q (out[3] ));
endmodule
```


The netlist contains two clear sequential libcells and two preset sequential libcells.

- When the `multibit_allow_async_phase_map` attribute is set to `true` (default), you allow the tool to map clear sequential cells to a preset sequential cells while inverting the D input and Q output if this allows multibit inferencing. For the previous netlist, that means that cells `DFSF` and `DFCF` can be combined in a multibit register `DUALDFSF`. The D inputs and the Q outputs of the `DFCF` cell are inverted.

```
module test(resetn, clk, in, out);
    input resetn, clk;
    input [3:0] in;
    output [3:0] out;
    wire resetn, clk;
    wire [3:0] in;
    wire [3:0] out;
    wire n_31, n_32, n_43, n_44;
    DUALDFSF \CDN_MBIT_cnt_0_reg[0_1] (.SDN (resetn), .CP (clk), .D1(in[0]),
        .D2 (n_31), .Q1 (out[0]), .Q2 (n_32));
    INV g5(.I (in[1]), .ZN (n_31));
    INV g6(.I (n_32), .ZN (out[1]));
    DUALDFSF \CDN_MBIT_cnt_0_reg[2_3] (.SDN (resetn), .CP (clk), .D1(n_43),
        .D2 (in[3]), .Q1 (n_44), .Q2 (out[3]));
    INV g23(.I (in[2]), .ZN (n_43));
    INV g24(.I (n_44), .ZN (out[2]));
endmodule
```

- When the `multibit_allow_async_phase_map` attribute is set to `false`, the clear sequential cells cannot be mapped to preset ones, and consequently the `DFCF` cells cannot be replaced with a multibit register in this case, while the two `DFSF` cells are mapped to a multibit register.

```
module test(resetn, clk, in, out);
    input resetn, clk;
    input [3:0] in;
    output [3:0] out;
    wire resetn, clk;
    wire [3:0] in;
    wire [3:0] out;
    DFCF cnt_0_reg[1] (.CDN (resetn), .CP (clk), .D (in[1]),
        .Q (out[1]));
    DFCF cnt_0_reg[2] (.CDN (resetn), .CP (clk), .D (in[2]),
        .Q (out[2]));
    DUALDFSF \CDN_MBIT_cnt_0_reg[0_3] (.SDN (resetn), .CP (clk),
        .D1(in[0]), .D2 (in[3]), .Q1 (out[0]), .Q2 (out[3]));
endmodule
```

Applies to:

- `inst`
- `module`
- `root`

Related Information

- [synthesize_design](#)

multibit_auto_exclude_registers_with_exceptions

`multibit_auto_exclude_registers_with_exceptions {true | false}`

Default: `false`

Data_type: `bool`, `read/write`

Controls whether timing exception checks should be done on sequential instances to exclude them from multibit merging. By default, the exception checks are skipped and merging happens for flops with identical exceptions only when all exceptions are transferable to the multibit cell that is created. Enabling this attribute will automatically apply `dont_merge_multibit` instance attribute on all sequential instances with exceptions.

Example

Assume two single bit instances `reg1` and `reg2` have the same timing exceptions. The library has a two-bit multibit cell.

- If `multibit_auto_exclude_registers_with_exceptions` is set to `true`, then `reg1` and `reg2` will not be considered as candidates for multibit merging.
- If `multibit_auto_exclude_registers_with_exceptions` is set to `false`, then `reg1` and `reg2` will be merged into a multibit libcell.

Related Information

- `dont_merge_multibit` in [inst](#) in *Stylus Common UI Database Object Information Guide*
- [synthesize_design](#)

multibit_cells_from_different_busses

`multibit_cells_from_different_busses {true | false}`

Description

Default: `true`

Data_type: `bool`, read/write

If this attribute remains `false`, only the instances with the same basename are merged together. If this attribute is set to `true`, the tool can merge single-bit cells from different banks (also called busses) into a single, multibit cell. For example, four single-bit flops below:

`p_reg[4]`, `p_reg[5]`, `q_reg[8]` and `q_reg[9]`

can be merged into one four-bit flop:

`p_reg[4_5]_q_reg[8_9]`

The single-bit cells affected by this attribute include: flip-flops, latches, tristate buffers, and combinational cells.

Related Information

- [multibit_seqs_members_naming_style](#)
- [synthesize_design](#)

multibit_preserve_inferred_instances

`multibit_preserve_inferred_instances {false | true}`

Description

Default: `false`

Data_type: bool, read/write

Specifies whether to preserve multibit instances that are inferred during incremental optimization. If set to `true`, the `preserve` attribute for all multibit instances is set to `size_delete_ok`. This preserve setting prevents any optimization step down the flow from breaking the multibit cells. For example, multibit tristate and combinational cells can get optimized in many places.

Related Information

- [synthesize_design](#)

multibit_preserved_net_check

```
multibit_preserved_net_check {false | true}
```

Description

Default: false

Data_type: bool, read/write

Controls whether instances can be merged into multibit instances if the connected nets are marked preserved. Set this attribute to `true` to prevent multibit merging when any of the nets connected to the instances are marked preserved.

Related Information

- [synthesize_design](#)

multibit_seqs_instance_naming_style

```
multibit_seqs_instance_naming_style {concat | auto | short}
```

Description

Default: concat

Data_type: string, read/write

Controls the naming of inferred multibit (flops, latches and tristates) instances.

This attribute can have the following values:

auto	Creates the name of the multibit instance based on busses of merged instances. • If the instances belong to the same bus, the name of the multibit instance starts with the bus name followed by the range of the merged bit-indices. • If the instances belong to different busses, the name of the multibit instance contains each bus name followed by the range of corresponding bit-indices.
concat	Creates the name of the multibit instance by concatenating the names of merged instances.
short	Creates the name of the multibit instance by using the bus name for only the first index of a multibit cell if all instances belong to the same bus. If the multibit instance contains single bits from different busses, the <code>short</code> naming style has no effect and the <code>concat</code> style is followed.

Examples

- Consider the following instances:

```
SCFFQXC1 I3256 (.CLK(CLK), .DATA(D1), .Q(Q1));
SCFFQXC1 I3267 (.CLK(CLK), .DATA(D2), .Q(Q2));
SCFFQXC1 I3312 (.CLK(CLK), .DATA(D3), .Q(Q3));
SCFFQXC1 I3439 (.CLK(CLK), .DATA(D4), .Q(Q4));
SCFFQXC1 I3572 (.CLK(CLK), .DATA(D5), .Q(Q5));
SCFFQXC1 I4188 (.CLK(CLK), .DATA(D6), .Q(Q6));
SCFFQXC1 I4357 (.CLK(CLK), .DATA(D7), .Q(Q7));
SCFFQXC1 I4567 (.CLK(CLK), .DATA(D8), .Q(Q8));
```

- Using the `auto` setting:

```
set_db / .multibit_seqs_instance_naming_style auto
```

The multibit instances are named as follows:

```
SCCSGFF4QXC1 I3256_3439(.CLK (CLK), .DATA0 (D1), .DATA1 (D2),
    .DATA2 (D3), .DATA3 (D4), .Q0 (Q1), .Q1 (Q2), .Q2 (Q3), .Q3 (Q4));
SCCSGFF4QXC1 I3572_4567(.CLK (CLK), .DATA0 (D5), .DATA1 (D6),
    .DATA2 (D7), .DATA3 (D8), .Q0 (Q5), .Q1 (Q6), .Q2 (Q7), .Q3 (Q8));
```

- Using the `concat` setting:

```
set_db / .multibit_seqs_instance_naming_style concat
```

The multibit instances are named as follows:

```
SCCSGFF4QXC1 I3256_I3267_I3312_I3439(.CLK (CLK), .DATA0 (D1), .DATA1 (D2),  
    .DATA2 (D3), .DATA3 (D4), .Q0 (Q1), .Q1 (Q2), .Q2 (Q3), .Q3 (Q4));  
SCCSGFF4QXC1 I3572_I4188_I4357_I4567(.CLK (CLK), .DATA0 (D5), .DATA1 (D6),  
    .DATA2 (D7), .DATA3 (D8), .Q0 (Q5), .Q1 (Q6), .Q2 (Q7), .Q3 (Q8))
```

- Consider the following instances:

```
a_reg[2]  
a_reg[1]  
b_reg[2]  
b_reg[1]  
a_reg[0]  
b_reg[0]
```

- Using the `short` setting:

```
set_db / .multibit_seqs_instance_naming_style short
```

The multibit instances are named as follows:

```
CDN_MBIT_a_reg[0]_MB_a_reg[1]_MB_a_reg[2]_MB_b_reg[0]  
CDN_MBIT_b_reg[1]_MB_[2]
```

The `short` naming style was not used for the first multibit instance because it contains single bits from two different busses. The `short` naming style was used for the second multibit instance because both single bits belong to the same bus. The bus name is only shown for the first bit.

Related Information

- [synthesize_design](#)

multibit_seqs_members_naming_style

```
multibit_seqs_members_naming_style {list_of_strings}
```

Description

Default: "%s%d" "%s%d_reg" "%s_%d_" "%s_%d_reg" {"%s[%d]} {"s[%d]_reg}"

Data_type: string, read/write

Controls which single-bit sequential instances can be merged into a multibit sequential instance during synthesis. The attribute value can contain a list of naming styles of the single-bit instances for which merging can be done.

Examples

- To combine the single bit cells `a1` and `a2`, the following naming style must be part of the attribute value:
`set_db / .multibit_seqs_members_naming_style "%s%d"`
- To combine the single bit cells `a1_reg` and `a2_reg`, the following naming style must be part of the attribute value:
`set_db / .multibit_seqs_members_naming_style "%s%d_reg"`
- To combine the single bit cells `a[1]` and `a[2]`, the following naming style must be part of the attribute value:
`set_db / .multibit_seqs_members_naming_style {%s[%d]}`
- To combine the single bit cells, `a[1]_reg` and `a[2]_reg`, the following naming style must be part of the attribute value:
`set_db / .multibit_seqs_members_naming_style {%s[%d]_reg}`

Related Information

- [multibit_cells_from_different_busses](#)
- [synthesize_design](#)

multibit_split_string

`multibit_split_string` *string*

Description

Default: `_split_`

Data_type: string, read/write

Specifies the string to be used to name single bit instances when the netlist that was read in already

contains multibit cells.

When the tool merges single bit instances into a multibit instance, it keeps track of the original names during the current session. If the tool splits the multibit cell back into single bits in the same session, it can reuse the original names. If the merging and splitting occur in different sessions, the original names are not stored and the tool uses this attribute value to create the single bit names.

Example

Assume the following netlist was read in during the current session:

```
module PIPO_REGISTER_REGDIMENSION9(CLK, RSN, Inpbus, Outbus);
    input CLK, RSN;
    input [9:0] Inpbus;
    output [9:0] Outbus;

    HS65V_GSH_4SDFPQX18 \OutbusintCK_reg[0_3] (.CP (CLK), .\D[0]
        (Inpbus[0]), .\D[1] (Inpbus[1]), .\D[2] (Inpbus[2]), .\D[3]
        (Inpbus[3]), .TI (Outbus[0]), .TE (1'b0), .\Q[0] (Outbus[0]),
        .\Q[1] (Outbus[1]), .\Q[2] (Outbus[2]), .\Q[3] (Outbus[3]));
endmodule
```

While performing incremental optimization, the multibit cell is split again into single bit instances. The tool uses the value of the `multibit_split_string` attribute to create the single bit names.

Assume the attribute was set to `_user_`, the netlist written out would look as follows:

```
module PIPO_REGISTER_REGDIMENSION9(CLK, RSN, Inpbus, Outbus);
    input CLK, RSN;
    input [9:0] Inpbus;
    output [9:0] Outbus;
    wire CLK, RSN;
    wire [9:0] Inpbus;
    wire [9:0] Outbus;
    wire UNCONNECTED, UNCONNECTED0, UNCONNECTED1, UNCONNECTED2;
    SDDFFX4 \OutbusintCK_reg[0_3]_user_0 (.CK (CLK), .D (Inpbus[0]), .SI
        (Outbus[0]), .SE (1'b0), .Q (Outbus[0]), .QN (UNCONNECTED));
    SDDFFX4 \OutbusintCK_reg[0_3]_user_1 (.CK (CLK), .D (Inpbus[1]), .SI
        (Outbus[1]), .SE (1'b0), .Q (Outbus[1]), .QN (UNCONNECTED0));
    SDDFFX4 \OutbusintCK_reg[0_3]_user_2 (.CK (CLK), .D (Inpbus[2]), .SI
        (Outbus[2]), .SE (1'b0), .Q (Outbus[2]), .QN (UNCONNECTED1));
    SDDFFX4 \OutbusintCK_reg[0_3]_user_3 (.CK (CLK), .D (Inpbus[3]), .SI
        (Outbus[3]), .SE (1'b0), .Q (Outbus[3]), .QN (UNCONNECTED2));
endmodule
```


Related Information

- `bit_width` in `lib_cell` in *Stylus Common UI Database Object Information Guide*
- `elaborate_design`
- `multibit_seqs_members_naming_style`
- `use_multibit_cells`

operating_conditions

`operating_conditions` *operating_conditions*

Description

Default: ""

Data_type: `string`, `library` (read only) `library_domain` and `root` (read/write)

Specifies the operating conditions to use for timing. You must specify the path to an `operating_conditions` object in the library. The `operating_conditions` attribute does not need to have a value if you want to use the default operating conditions.

Specify this attribute before you use the libraries in the design.

Examples

- The following command sets the operating condition for library domain `srpg` to `1p08`:
`set_db library_domains/srpg .operating_condition 1p08`
- The following command sets the operating conditions to `worst_case`:
`set_db operating_conditions worst_case`

Applies to:

- `library`
- `library_domain`
- `root`

opt_multibit_combinational_name_concatenate_string

opt_multibit_combinational_name_concatenate_string *string*

Description

Default: `_MB_`

Data_type: `string`, `read/write`

Specifies the separator string to be used in the name of the multibit combinational and special low power cells. The recommended values for the verification flow are `_MB_` and `_mb_`.

Related Information

- [synthesize_design](#)

opt_multibit_seqs_name_concatenate_string

opt_multibit_seqs_name_concatenate_string *string*

Description

Default: `_MB_`

Data_type: `string`, `read/write`

Specifies the separator string to be used for concatenation when naming multibit sequential instances. The recommended values for the verification flow are `_MB_` and `_mb_`.

Example

```
set_db / .opt_multibit_seqs_name_concatenate_string _mb_
```

Related Information

- [synthesize_design](#)

opt_use_default_related_pg_pin_for_always_on

```
opt_use_default_related_pg_pin_for_always_on {true| false}
```

Description

Default: true

Data_type: bool, read/write

Specifies whether to use the default power or ground pin for the input or output pin's related_power_pin or related_ground_pin of always-on buffer or inverter cell, respectively.

opt_use_next_state_type_only_to_assign_sync_control

```
opt_use_next_state_type_only_to_assign_sync_control {true| false}
```

Description

Default: false

Data_type: bool, read/write

Forces the tool to use the nextstate_type .lib pin attribute (when available in the library) to assign synchronous control pins.

By default, the tool uses the nextstate_type .lib pin attribute along with some heuristics based on pin names to assign synchronous control pins.

Related Information

- [synthesize_design](#)

override_library_max_drc

```
override_library_max_drc {true | false}
```

Description

Default: `false`

Data_type: `bool`, `read/write`

Specifies whether you can relax the design rule constraints set on the library. Set this attribute to `true` to relax the `max_capacitance`, `max_fanout`, and `max_transition` constraints. By default, you can only tighten the constraint values.

Related Information

- [design](#) in *Stylus Common UI Database Object Information Guide*
 - `max_capacitance`
 - `max_fanout`
 - `max_transition`
- [report_constraint](#)
- [synthesize_design](#)

parse_lib_moments_table

```
parse_lib_moments_table {true | false}
```

Description

Default: `false`

Data_type: `bool`, `read/write`

Parses the library moments table while loading the library file.

reload_when_for_macro_cell

```
reload_when_for_macro_cell {false | true}
```

Description

Default: `true`

Data_type: `bool`, read/write

Disables the processing of the when condition redundantly for the macro library cell during library characterization.

socv_library

`socv_library string`

Description

Default:

Data_type: `string`, read/write

Specifies the list of Statistical On-Chip Variation (SOCV) libraries.

Related Information

- `read_db`

support_internal_pg_pins

`support_internal_pg_pins {true| false}`

Description

Default: `true`

Data_type: `bool`, read/write

Supports internal power and ground pins.

support_master_slave_flop

```
support_master_slave_flop {true| false}
```

Description

Default: `false`

Data_type: `bool`, read/write

Support master-slave flop library cell.

support_multi_seq_elements

```
support_multi_seq_elements {true| false}
```

Description

Default: `true`

Data_type: `bool`, read/write

Controls the conversion of a state-retention cell with multiple sequential elements, such as multiple `ff` or `latch` groups, into a single `ff` or `latch` function.

Any outgoing timing arcs on the power gating pin of the state-retention cell are enabled by default.

You must set this attribute before you read the libraries.

support_multi_seq_scan_latch

```
support_multi_seq_scan_latch {true| false}
```

Description

Default: `false`

Data_type: bool, read/write

Supports scan latch library cell defined using two latches.

syn_exact_match_sequential_sync_control

`syn_exact_match_sequential_sync_control {true| false}`

Description

Default: false

Data_type: bool, read/write

If set to `true`, the tool will try to force map the synchronous flops inferred from RTL to complex library flip-flops with corresponding synchronous inputs regardless of QoR. If the library does not have the appropriate synchronous control libcells, simple flip-flops and combinational logic will be used.

It is recommended to set this attribute before reading the libraries to reduce the runtime of processing the libraries.

This attribute may have a positive impact on clock gating coverage due to the explicit enable.

Related Information

- [synthesize_design](#)
- [Set and Reset Synthesis Pragmas](#) in *Innovus Synthesis HDL Modeling Guide*

target_library

`target_library string`

Description

Default:

Data_type: string, read/write

Specifies the list of target libraries. Target libraries are technology libraries that are used by the

Synthesis engine during mapping and optimization. The target libraries contain the cells used to generate the netlist and definitions for the design's operating conditions.

Related Information

- [link_library](#)

timing_enable_set_reset_latch_preset_clear_arcs

`timing_enable_set_reset_latch_preset_clear_arcs {true| false}`

Description

Default: `false`

Data_type: `bool`, read/write

Controls the enabling of the timing arcs through clear and preset pins of set-reset (SR) latches.

Related Information

- [report_timing](#)
- [synthesize_design](#)

timing_library_allow_invalid_primary_power_pins_lib_cell

`timing_library_allow_invalid_primary_power_pins_lib_cell {true| false}`

Description

Default: `false`

Data_type: `bool`, read/write

Treats cells with invalid primary power pins as unusable for mapping and optimization. This includes:

- Cells with more than one primary_power pin with none of them marked as


```
std_cell_main_rail=true.
```

- Cells with more than one primary_power pin and more than one of them marked as

```
std_cell_main_rail=true.
```

This does not apply to level-shifter cells. Level-shifter cells with invalid primary_power pins are treated as usable.

This attribute must be set before synthesis.

Related Information

- [synthesize_design](#)

timing_library_allow_multiple_sync_control

```
timing_library_allow_multiple_sync_control {true| false}
```

Description

Default: false

Data_type: bool, read/write

Allow multiple synchronous clear and preset pins.

timing_library_ignore_attribute_check_during_pin_convert

```
timing_library_ignore_attribute_check_during_pin_convert {true| false}
```

Description

Default: true

Data_type: bool, read/write

Ignores the liberty attributes check while converting the library's logical pin defined as power or ground in LEF.

timing_library_infer_cap_range_from_c1cn_dynamic_pin_cap_model

```
timing_library_infer_cap_range_from_c1cn_dynamic_pin_cap_model {true| false}
```

Description

Default: false

Data_type: bool, read/write

Enables Syntech support to infer cap range from n-piece dynamic pincap.

timing_library_infer_cap_range_from_dynamic_pin_cap_model

```
timing_library_infer_cap_range_from_dynamic_pin_cap_model {true| false}
```

Description

Default: false

Data_type: bool, read/write

Enables the tool to override any static capacitance from the library with an inferred capacitance.

timing_library_limit_messages

```
timing_library_limit_messages {true| false}
```

Description

Default: true

Data_type: bool, read/write

Controls the printing of the LBR messages. By default, each LBR message will be printed maximum twenty times for each library that is being read. If for any LBR message, the `max_print` message object attribute has a lower limit, this limit will take precedence. If you set this attribute to `false`, all LBR messages will be printed to the log file.

Related Information

- [message](#) in *Stylus Common UI Database Object Information Guide*
 - `max_print`

timing_library_look_up_drv_per_frequency

`timing_library_look_up_drv_per_frequency {worst | linear}`

Description

Default: `worst`

Data_type: `enum`, `read/write`

When `max_cap` lookup tables are present in the library, this attribute determines whether the maximum capacitance values used for design rule verification during incremental optimization should be the worst value for all frequencies or the interpolated value for the maximum frequency.

In the library, the maximum capacitance can be specified as follows:

- at the library level using the `default_max_capacitance` attribute
- at the pin level using the `max_capacitance` attribute
- at the pin level using the `max_cap` group (a table which specifies the maximum capacitance in function of the frequency and the input transition time).

Currently, the tool only supports one dimensional `max_cap` lookup tables in function of the frequency.

If you set this attribute to `linear`, the tool always determines the maximum pin capacitance by doing a linear interpolation of the pin's `max_cap` lookup table in function of the maximum frequency constraint that the timing engine returns across all paths through the given pin.

If you set this attribute to `worst`, the tool uses a worst value for all frequencies determined as following:

default_max_capacitance (lib level)	max_capacitance (pin level)	max_cap_group (pin level)	Action if attribute set to worst
yes	yes	yes	Selects the worst of the <code>max_capacitance</code> pin attribute and the interpolated value of the <code>max_cap</code> group.
no	yes	yes	Selects the worst of the <code>max_capacitance</code> pin attribute and the interpolated value of the <code>max_cap</code> group.
yes	no	yes	Selects the worst of the <code>default_max_capacitance</code> lib level attribute and the interpolated value of the <code>max_cap</code> group.
no	no	yes	Interpolates the value of the <code>max_cap</code> group.

This attribute is taken into account only if the `use_max_cap_lut` attribute is set to `true`.

Related Information

- [report_timing](#)
- [synthesize_design](#)
- [report_constraint](#)
- `timing_library_use_max_cap_look_up_table` in [opt Category Attributes](#)

timing_library_multibit_force_merge_combinational_cells

```
timing_library_multibit_force_merge_combinational_cells {false | true}
```

Description

Default: `false`

Data_type: `bool`, `read/write`

Merges single-bit combinational instances into an appropriate multibit combinational instance independent of the impact on the QoR. When enabled, multibit cell inferencing will occur even if it degrades the delay, power, or area QoR of the design. This is useful for increasing multibit coverage but might negatively impact the QoR.

Example

Assume two single bit instances `g1` and `2` can be merged into a multibit instance. The library has three suitable multibit cells: `2g1x1`, `2g1x2`, and `2g1x3`. Consider only

`timing_library_multibit_force_merge_combinational_cells` **enabled**:

```
set_db timing_library_multibit_force_merge_combinational_cells true
```

In this case, the tool computes the total cost with timing for each of the possible multibit `lib_cell` candidates. The best candidate for multibit cell merging will be chosen, irrespective of whether it improves or degrades the delay QoR of the design.

Related Information

- `synthesize_design`
- `timing_library_multibit_force_merge_iso_cells`
- `timing_library_multibit_force_merge_sequential_cells`
- `use_multibit_cells`

timing_library_multibit_force_merge_iso_cells

```
timing_library_multibit_force_merge_iso_cells {true | false}
```

Description

Default: `false`

Data_type: bool, read/write

Merges single-bit isolation instances into an appropriate multibit isolation instance independent of the impact on the QoR. When enabled, multibit cell inferencing will occur even if it degrades the delay, power, or area QoR of the design. This is useful for increasing multibit coverage but might negatively impact the QoR.

Example

Assume two single bit instances `g1` and `2` can be merged into a multibit instance. The library has three suitable multibit cells: `2g1x1`, `2g1x2`, and `2g1x3`. Consider only `timing_library_multibit_force_merge_iso_cells` enabled:

```
set_db timing_library_multibit_force_merge_iso_cells true
```

In this case, the tool computes the total cost with timing for each of the possible multibit `lib_cell` candidates. The best candidate for multibit cell merging is chosen, irrespective of whether it improves or degrades the delay QoR of the design.

Related Information

- [synthesize_design](#)
- [timing_library_multibit_force_merge_combinational_cells](#)
- [timing_library_multibit_force_merge_sequential_cells](#)
- [use_multibit_cells](#)

timing_library_multibit_force_merge_sequential_cells

```
force_merge_seqs_into_multibit_cells {false | true}
```

Description

Default: false

Data_type: bool, read/write

Merges single-bit sequential instances into an appropriate multibit sequential instance independent of the impact on the QoR. When enabled, multibit cell inferencing will occur even if it degrades the

delay, power, or area QOR of the design. This is useful for increasing multibit coverage but might negatively impact the QoR.

Example

Assume two single bit instances `reg1` and `reg2` can be merged into a multibit instance. The library has three suitable multibit cells: `dual1`, `dual2`, and `dual3` and

`force_merge_seqs_into_multibit_cells` enabled:

```
set_db / .force_merge_seqs_into_multibit_cells true
```

In this case, the tool computes the total cost with timing for each of the possible multibit `lib_cell` candidates. The best candidate for multibit cell merging will be chosen irrespective of whether it improves or degrades the delay QoR of the design.

Related Information

- [synthesize_design](#)
- [timing_library_multibit_force_merge_combinational_cells](#)
- [timing_library_multibit_force_merge_iso_cells](#)
- [use_multibit_cells](#)

timing_library_multibit_use_combinational_cells

```
timing_library_multibit_use_combinational_cells {false | true}
```

Description

Default: `false`

Data_type: `bool`, read/write

Controls the mapping of single-bit combinational cells to appropriate multibit combinational cells from the technology library.

By default, this attribute inherits the last setting of the `use_multibit_cells` attribute, but you can explicitly override this setting to enable or disable combinational cell mapping.

Multibit cell inferencing is performed during incremental optimization rather than during the mapping stage. While cells that are marked unusable by the tool cannot be used during mapping, they can be used during incremental optimization.

Related Information

- `bit_width` in `lib_cell` in *Stylus Common UI Database Object Information Guide*
- `synthesize_design`
- `use_multibit_cells`

timing_library_respect_async_control_priority

`timing_library_respect_async_control_priority {true| false}`

Description

Default: `true`

Data_type: `bool`, read/write

Instructs the library parser to consider the asynchronous control pins priority description of sequential libcells from the Liberty files. This enables the implicit implementation of the priority logic by the sequential library cells during mapping.

Set this attribute to `false` if you want the priority logic to be implemented explicitly by combinational gates. It is recommended to set this attribute before reading the libraries to reduce the runtime of processing the libraries.

Related Information

- `synthesize_design`

timing_library_sequential_in_out_phase_opt

`timing_library_sequential_in_out_phase_opt {true| false}`

Description

Default: `false`

Data_type: `bool`, `read/write`

Enables the tool to perform the following transformations during synthesis if they result in a QoR improvement.

- If the original netlist has no inverters at the `d` input and the `q` output, the tool can add inverters at this input and this output during mapping.
A simple flip-flop can be converted to a flop with inversion at the `d` input and the `q` output
- If the original netlist has an inverter at the `d` input and the `q` output, the tool can remove both inverters from this input and this output during mapping.
A flop with inversion at the `d` input and the `q` output can be converted to a simple flop by removing the inversions.
- If the original netlist has an inverter at the `d` input, the tool can move the inverter from the `d` input to the `q` output of the sequential cell.
This can be referred to as propagating an inverter through the sequential cell, or also bubble pushing.

Verification of these transforms requires compute-expensive phase mapping in Conformal. The affected key points will be reported as inverted equivalents.

- Inverting the `d` input and `q` output also inverts the sense of the preset (set) and clear (reset) signals (both synchronous and asynchronous). A clear sequential libcell with inverted `d` input and `q` output becomes a preset libcell. And a preset libcell becomes a clear one. The preset/clear swapping can only happen if we invert the `d` and `q` phases.
This is useful when the RTL functionality requires a preset function but the technology library has only a reset flop (or vice-versa).

It is recommended to set this attribute before reading the libraries to reduce the runtime of processing the libraries.

By default, these operations are not performed because they can impact verification of the design.

Example

```
module invert_d_q (q, q1, d, clk, rstn, pstn);
    input clk, rstn, pstn, d;  output q, q1;  reg q, q1;
    always @(posedge clk)
        begin
            if (!rstn) q = 1'b0;
            else q = d;
            if (!pstn) q1 = 1'b1;
            else q1 = d;
        end
endmodule
```

- By default, this RTL description is synthesized into a reset flop and a preset flop.
- If the attribute is set to `true` and if the library has only reset flops, the preset flop will be mapped to a reset flop.
- If the attribute is set to `true` and if the library has only preset flops, the reset flop will be mapped to a preset flop.

Related Information

- `syn_exact_match_sequential_async_control` in [opt Category Attributes](#)
- `synthesize_design`
- `write_do_lec`
- [Translating Phase Mapping](#) in *Innovus Synthesis Conformal Interface Guide*

timing_library_support_3d_table_power_arc

```
timing_library_support_3d_table_power_arc {true| false}
```

Description

Default: `false`

Data_type: `bool`, read/write

Controls support for three-dimensional power arc.

timing_library_support_combinational_clock

timing_library_support_combinational_clock {true| false}

Description

Default: false

Data_type: bool, read/write

When you set this attribute to `true`, non-sequential cells that have a `clock` attribute set to `true` on one of the input pins will be treated as sequential cells.

Related Information

- [lib_cell](#) in *Stylus Common UI Database Object Information Guide*
 - `is_combinational`
 - `is_sequential`
 - `is_usable`

timing_library_support_merge_libs

timing_library_support_merge_libs {true| false}

Description

Default: false

Data_type: bool, read/write

Controls whether or not you can append two or more libraries that are defined in different `.lib` files and that have the same library name and the same PVT conditions but contain different libcells.

Related Information

- [synthesize_design](#)

timing_library_support_serial_scan_in_multibit_cell

```
timing_library_support_serial_scan_in_multibit_cell {true| false}
```

Description

Default: true

Data_type: bool, read/write

Determines whether to support a multi-bit serial scan cell as multi-bit cell when only the scan input of the first bit is specified as a scan input pin (test_scan_in) while the scan input of all other bits are internal nodes.

timing_library_support_transparent_latch_group

```
timing_library_support_transparent_latch_group {true| false}
```

Description

Default: true

Data_type: bool, read/write

Supports library cells having the transparent latch (tlatch) group.

timing_library_suppress_library_messages

```
timing_library_suppress_library_messages {true| false}
```

Description

Default: false

Data_type: bool, read/write

Controls the printing of the LBR parse messages.

timing_library_use_sequential_test_cell

timing_library_use_sequential_test_cell {true | false}

Description

Default: false

Data_type: bool, read/write

Controls whether or not to use the test cell function if the main cell function is unusable.

This attribute must be set before you read in the libraries.

Related Information

- [ignore_pin_error_in_test_cell_function](#)
- [read_db](#)

treat_non_seq_arc_cell_as_unusable

treat_non_seq_arc_cell_as_unusable {true| false}

Description

Default: true

Data_type: bool, read/write

Treats the lib cells having non sequential setup arc as unusable if

timing_disable_non_sequential_checks is set to false.

Related Information

- [timing_disable_non_sequential_checks](#) in [timing Category Attributes](#)

turbo_lib_loading

turbo_lib_loading {0 | 1 | 2}

Description

Default: 2

Data_type: int, read/write

Populates timing arcs, power arcs, and library establishment data only on need basis for runtime speedup.

use_area_from_lef

```
use_area_from_lef {false | true | auto}
```

Description

Default: false

Data_type: string, read/write

Specifies whether to use the cell area from the LEF libraries. If you set this attribute to `auto`, the cell area will be read from the technology area unless the LEF library is available and it has at least one macro definition. By default, the cell area will not be read from the LEF library.

0 and `false` are equivalent, and 1 and `true` are equivalent.

Related Commands

- `synthesize_design`

use_compatibility_based_grouping

```
use_compatibility_based_grouping {false | true}
```

Description

Default: true

Data_type: bool, read/write

Controls the mapping of single-bit sequential, combinational, and tristate cells into multibit cells based on common control signals.

By default, the tool will create groups of compatible single-bit cells that have common control signals. Merging is only done within a group of compatible cells.

Related Information

- `bit_width` in [lib_cell](#) in *Stylus Common UI Database Object Information Guide*
- `multibit_seqs_members_naming_style`
- `synthesize_design`
- `timing_library_multibit_force_merge_combinational_cells`
- `timing_library_multibit_force_merge_sequential_cells`
- `use_multibit_combo_cells`
- `use_multibit_seq_and_tristate_cells`

use_main_cell_output_function_for_test_cell

`use_main_cell_output_function_for_test_cell {true | false}`

Description

Default: true

Data_type: bool, read/write

Controls whether or not to ignore the `test_cell` functionality and use the main cell functionality for DFT sequential cells when there is a discrepancy between the main-cell function and the `test_cell` function or when the `test_cell` function is missing. When the attribute is disabled and the `test_cell` group has no function, only the main-cell function is used and the cell is treated as a non-scan cell.

use_multibit_cells

```
use_multibit_cells {false | true}
```

Description

Default: false

Data_type: bool, read/write

If set to `true`, single-bit cells are mapped to an appropriate multibit cell from the technology library.

For example, the following four single-bit flops:

```
ff_reg[0] ff_reg[1] ff_reg[2] ff_reg[3]
```

can be merged into one four-bit flop as shown below:

```
ff_reg[0_3]
```

Single-bit cells that can be merged include: flip-flops, latches, tristate buffers, combinatorial cells, isolation cells and level-shifter cells.

When merging flops, the shared input is the clock line.

When merging latches, the shared input is the gate line.

When merging tri-state cells, the shared input is the enable line.

Multibit cell inferencing is performed during the mapping stage. While cells that are marked unusable by the tool cannot be used during mapping, they can be used during incremental optimization.

This attribute controls both logical and physical-aware multibit mapping. If the design is placed, physical-aware multibit mapping is performed, otherwise logical multibit mapping is performed.

Related Information

- `bit_width` in `lib_cell` in *Stylus Common UI Database Object Information Guide*
- `multibit_cells_from_different_busses`
- `multibit_seqs_members_naming_style`
- `synthesize_design`
- `timing_library_multibit_force_merge_combinational_cells`
- `timing_library_multibit_force_merge_sequential_cells`
- `timing_library_multibit_use_combinational_cells`
- `use_multibit_seq_and_tristate_cells`

use_multibit_iso_cells

```
use_multibit_iso_cells {true | false}
```

Description

Default: `false`

Data_type: `bool`, read/write

Controls the mapping of single-bit isolation cells to appropriate multibit isolation cells from the technology library.

By default, this attribute inherits the last setting of the `use_multibit_cells` attribute, but you can explicitly override this setting to enable or disable isolation cell mapping.

Multibit cell inferencing is performed during incremental optimization rather than during the mapping stage. While cells that are marked unusable by the tool cannot be used during mapping, they can be used during incremental optimization.

Examples

```
get_db use_multibit_cells  
false
```

```
get_db use_multibit_iso_cells
false

set_db use_multibit_cells true
    Setting attribute of root '/': 'use_multibit_cells' = true

get_db use_multibit_iso_cells
true

set_db use_multibit_iso_cells false
    Setting attribute of root '/': 'use_multibit_iso_cells' = false

set_db use_multibit_cells true
    Setting attribute of root '/': 'use_multibit_cells' = true

get_db use_multibit_iso_cells
true
```

Related Information

- `bit_width` in [lib_cell](#) in *Stylus Common UI Database Object Information Guide*
- [synthesize_design](#)
- [use_multibit_cells](#)

use_multibit_seq_and_tristate_cells

```
use_multibit_seq_and_tristate_cells {true | false}
```

Description

Default: `false`

Data_type: `bool`, `read/write`

Controls the mapping of single-bit sequential and tristate cells to appropriate multibit cells from the technology library.

By default, this attribute inherits the last setting of the `use_multibit_cells` attribute, but you can explicitly override this setting to enable or disable sequential and tristate cell mapping.

Multibit cell inferencing is performed during incremental optimization rather than during the mapping stage. While cells that are marked unusable by the tool cannot be used during mapping, they can be used during incremental optimization.

Related Information

- `bit_width` in [lib_cell](#) in *Stylus Common UI Database Object Information Guide*
- `synthesize_design`
- `use_multibit_cells`

use_scan_seqs_for_non_dft

`use_scan_seqs_for_non_dft {true | false | degenerated_only}`

Description

Default: `true`

Data_type: `enum`, `read/write`

Controls the mapping of registers to scan flip-flops for functional purposes.

This attribute can have the following values:

<code>degenerated_only</code>	Allows mapping to scan flip-flops with the scan-related pins tied off (also known as degenerated scan flops). This value is typically used when only scan flop library cells are available for use during technology mapping.
<code>false</code>	Avoids mapping to scan flip-flops for functional use. This value is typically used when the flip-flops will be mapped to scan for DFT by setting the <code>dft_scan_map_mode</code> design object attribute to either <code>force_all</code> or <code>tdrc_pass</code>.
<code>true</code>	Allows registers not targeted for DFT to be mapped to scan flip-flops for functional use.

A flop can only be mapped to scan for DFT if you run the `check_dft_rules` command and the flop passes the DFT rules check or if prior to mapping, you set the `dft_scan_map_mode` design attribute to `force_all`.

Related Information

- [synthesize_design](#)

write_db

The root attributes in the `write_db` category are:

- [write_db_create_read_file](#)
- [write_db_include_metal_fill_rules](#)

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `write_db` category, use the following command:

```
get_db -category write_db
```

write_db_create_read_file

```
write_db_create_read_file {true | false}
```

Default: false

Data_type: bool

When set to `true`, '`write_db my_db`' will also create a '`my_db.read`' file with a '`read_db my_db`' command inside it. This is a Tcl file that can be passed to the Linux executable to make it easy to load the DB at startup like this:

```
'tempus -files my_db.read'.
```

For OA usage, it would have the appropriate '`read_db -oa_lib_cell_view {<lib><cell> <view>}`' command inside it.

write_db_include_metal_fill_rules

```
write_db_include_metal_fill_rules {0 | 1}
```

Default: 0

Data_type: int

Specifies whether metal fill settings are to be saved in the `.init` file when saving the design. Metal fill

settings include `set_metal_fill`, `set_metal_fill_spacing_table`, and `set_via_fill` settings. By default, these metal fill settings are lost when you exit Tempus. To enable Tempus to record metal fill settings in the .init file, you must set the `write_db_include_metal_fill_rules` to 1 before saving the design. You can easily reload your settings by restoring the saved design:

```
set write_db_include_metal_fill_rules 1
set_metal_fill -iterationName iter1 ...

write_db

read_db

#The previous metal fill settings are reloaded.
```

Related Commands

- [write_db](#)
- [read_db](#)

write_def

The root attribute in the `write_def` category is:

- [write_def_lef_out_version](#)

To set the value of any root attribute, use the [set_db](#) command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the [get_db](#) command:

```
get_db attribute_name
```

To get the current value of all attributes in the `write_def` category, use the following command:

```
get_db -category write_def
```

write_def_lef_out_version

```
write_def_lef_out_version version_num
```

Default: 5.8

Data_type: enum {5.5 5.6 5.7 5.8 6.0}

Specifies LEF/DEF output version. Possible string values are 5.3, 5.4, 5.5, 5.6, 5.7, and 5.8. The default is 5.8.

write_netlist

The root attributes in the write_netlist category are:

- [write_netlist_full_pin_out](#)
- [write_netlist_port_association_style](#)

To set the value of any root attribute, use the [set_db](#) command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the [get_db](#) command:

```
get_db attribute_name
```

To get the current value of all attributes in the `write_netlist` category, use the following command:

```
get_db -category write_netlist
```

write_netlist_full_pin_out

```
write_netlist_full_pin_out {true | false}
```

Default: false

Data_type: bool

Specifies whether to write out every instance port connection, including unconnected ports unused in the input netlist.

write_netlist_port_association_style

```
write_netlist_port_association_style {true | false}
```

Default: false

Data_type: bool

Writes out the netlist with implicit port mapping (that is, ordered port connections or position-based port connections) for hierarchical instances.

Rule Attributes

- [rule_filter](#)
- [rule_instance](#)
- [rule_occurrence](#)

rule_filter

The attributes in the rule_occurrence category are:

- [base_name](#)
- [name](#)
- [obj_type](#)

base_name

`base_name string`

Default: ""

Data_type: string, read/only

Base name of the rule.

name

`name string`

Default: ""

Data_type: string, read/only

Name of the rule.

obj_type

```
obj_type { design base_cell module inst root net port base_pin pg_base_pin pin layer
net_group hinst hpin hnet boundary density_shape shape
        route_rule via_def_rule stack_via_rule pg_pin bus port_bus hport_bus
hpin_bus power_domain hport layer_rule pin_blockage pin_group pin_guide
        resize_blockage row sdp track_pattern antenna_model route_type group via_def
io_constraint antenna_data bump bump_pin bus_guide marker text
        trim_grid special_via special_wire port_shape package_component
package_object gcell_grid gcell foreign_cell gui_line gui_rect gui_text
        gui_polygon site via wire patch_wire virtual_wire layer_shape shape_via
place_blockage route_blockage partition physical_pin resistor
        inst_obs_shape message bus_sink_group tech bump_grid def_scan_chain
```

tap_distance_rule tap_layer_distance interposer_route_rule flow_step flow
attribute isolation_rule level_shifter_rule retention_rule power_switch_rule
opcond timing_point path_group lib_arc lib_pin clock arc lib_cell
library delay_noise_obj timing_path rc_corner library_set timing_condition
analysis_view constraint_mode delay_corner seq_function context_data
context_top_clock what_if_wire what_if_via rule_instance rule_occurrence
rule_filter const_hier_exc1 const_comp_exc1 const_hier_exc2
const_comp_exc2 const_hier_exc3 const_comp_exc3 const_hier_exc4
const_comp_exc4 const_hier_exc5 const_comp_exc5 const_hier_exc6 const_comp_exc6
const_hier_exc7 const_comp_exc7 const_hier_exc10 const_comp_exc10
const_hier_exc11 const_comp_exc11 const_hier_exc12 const_comp_exc12
const_hier_exc13 const_comp_exc13 const_hier_exc14 const_comp_exc14
const_hier_exc15 const_comp_exc15 const_hier_exc16 const_comp_exc16
const_hier_exc17 const_comp_exc17 const_hier_clk2a const_comp_clk2a
const_hier_clk2b const_comp_clk2b const_hier_clk2c const_comp_clk2c
const_hier_clk1 const_comp_clk1 const_hier_clk3 const_comp_clk3
const_hier_clk2d const_comp_clk2d const_hier_clk2e const_comp_clk2e
const_hier_clk2f const_comp_clk2f const_hier_clk4 const_comp_clk4
const_hier_clk7 const_comp_clk7 const_hier_clk8 const_comp_clk8
const_hier_io2 const_comp_io2 const_hier_io4 const_comp_io4 const_hier_sca1
const_comp_sca1 const_hier_sdt1 const_comp_sdt1 const_hier_grp1
const_comp_grp1 const_hier_cg1 const_comp_cg1 const_hier_io5i
const_comp_io5i const_hier_io5o const_comp_io5o const_hier_io6i const_comp_io6i
const_hier_io6o const_comp_io6o const_hier_io7i const_comp_io7i
const_hier_io7o const_comp_io7o const_hier_io8i const_comp_io8i const_hier_io8o
const_comp_io8o const_hier_ctrl1 const_comp_ctrl1 const_hier_cg2
const_comp_cg2 const_hier_unc1 const_comp_unc1 const_hier_dat1 const_comp_dat1
const_hier_pls1 const_comp_pls1 const_hier_iol1i const_comp_iol1i
const_hier_iol1o const_comp_iol1o const_hier_lat1 const_comp_lat1
const_hier_iol2i const_comp_iol2i const_hier_iol2o const_comp_iol2o
const_hier_old1 const_comp_old1 const_hier_hfn2 const_comp_hfn2
const_hier_tim2 const_comp_tim2 const_hier_clk6 const_comp_clk6
const_hier_io1 const_comp_io1 const_hier_io3 const_comp_io3 const_hier_io9
const_hier_io10 const_lint_cg1 const_lint_scope2 const_lint_cg2
const_lint_clk1 const_lint_clk2 const_lint_clk3 const_lint_clk5 const_lint_clk4
const_lint_clk6 const_lint_clk7 const_lint_clk8 const_lint_clk9
const_lint_cmd1 const_lint_ctr1 const_lint_ctr2 const_lint_ctr3 const_lint_ctr4
const_lint_ctr5 const_lint_ctr7 const_lint_exc1 const_lint_exc2
const_lint_flp1 const_lint_hfn1 const_lint_hfn2 const_lint_idl6 const_lint_idl7
const_lint_inf1 const_lint_itr2 const_lint_itr1 const_lint_itr3
const_lint_itr4 const_lint_lat1 const_lint_lat2 const_lint_lat3
const_lint_lat4 const_lint_mcp1 const_lint_odl1 const_lint_odl2
const_lint_odl3 const_lint_odl4 const_lint_odl5 const_lint_odl6 const_lint_odl7

const_lint_odl8 const_lint_old1 const_lint_old2 const_lint_ref1
const_lint_ref2 const_lint_ref4 const_lint_sca1 const_lint_unc1 const_lint_unc2
const_lint_idl1 const_lint_idl2 const_lint_idl3 const_lint_idl4
const_lint_idl5 const_lint_dat1 const_lintcfg_cmd2 const_lintcfg_cmd3
const_lintcfg_cmd4 const_lintcfg_opt10 const_lintcfg_opt2 const_lintcfg_opt3
const_lintcfg_opt4 const_lintcfg_opt5 const_lintcfg_opt6
const_lintcfg_opt7 const_lintcfg_opt8 const_lintcfg_ref4 const_lintcfg_ref5
const_lintcfg_ref7 const_lintcfg_val3 const_lintcfg_val4
const_sca_chk1 const_sca_chk2 const_clk_ctr1 const_clk_ctr2 const_clk_ctr4
const_clk_ctr6 const_clk_ctr9 const_clk_ctr10 const_clk_ctr12
const_clk_ctr13 const_clk_def1 const_clk_def3 const_clk_def4 const_clk_def2
const_clk_def6 const_clk_def10 const_clk_def11 const_clk_def12
const_clk_def13 const_clk_def23 const_clk_def26 const_clk_def49
const_clk_def32 const_clk_def33 const_clk_def34 const_clk_def35 const_clk_def36
const_clk_def37 const_clk_def38 const_clk_def39 const_clk_def48
const_clk_def47 const_clk_def40 const_clk_def41 const_clk_def42 const_clk_def43
const_clk_def44 const_clk_def45 const_clk_def46 const_clk_grp1
const_clk_grp2 const_clk_grp3 const_clk_grp4 const_clk_grp5 const_clk_grp7
const_clk_lat1 const_clk_lat3 const_clk_lat4 const_clk_lat6 const_clk_lat7
const_clk_lat8 const_clk_lat11 const_clk_lat12 const_clk_lat13
const_clk_unc1 const_clk_unc3 const_clk_unc4 const_clk_unc6 const_clk_unc7
const_clk_unc10 const_dat_chk1 const_dat_chk2 const_dat_chk3
const_dat_chk4 const_clk_unc9 const_dgn_cmb1 const_dgn_cmb2 const_dgn_cmb3
const_dgn_cmb5 const_dgn_cmb6 const_dgn_cmb7 const_exc_clk1
const_exc_flp1 const_exc_flp7 const_exc_flp8 const_exc_flp9 const_exc_mcp8
const_exc_smd3 const_exc_smd4 const_exc_smd5 const_exc_smd6
const_exc_smd7 const_exc_flp10 const_exc_flp11 const_exc_mcp1 const_exc_mcp5
const_exc_mcp6 const_exc_mcp7 const_exc_olp2 const_exc_olp1a
const_exc_olp1b const_exc_olp1c const_exc_olp1d const_exc_olp1e
const_exc_olp1f const_exc_smd1 const_exc_smd2 const_io_idl1 const_io_old1
const_io_old3 const_io_old4 const_io_drc1 const_io_drc2 const_io_drc3
const_io_idl4 const_io_idl5 const_io_idl6 const_io_idl13 const_io_idl14
const_io_idl15 const_io_odl5 const_io_odl6 const_io_odl13 const_io_odl14
const_io_odl15 const_io_itr1 const_io_itr2 const_io_itr3
const_io_itr10 const_io_odl1 const_io_odl4 const_cg_chk1 const_cg_chk2
const_cg_chk3 const_dgn_tim1 const_clk_def21 const_clk_def7
const_hfn_chk1a const_hfn_chk1b const_hfn_chk2 const_hfn_chk3
const_lint_unc3 const_clk_grp6 const_lint_grp2 const_lint_grp3 const_cg_chk5
const_cg_chk6 const_cg_chk7 const_dat_chk5 const_dgn_tim2 const_exc_clk2
const_cov_cg const_cov_cp const_cov_cs const_cov_data const_cov_pw
const_cov_rr const_cov_sh const_cov_skew clock_tree skew_group
clock_tree_source_group flexible_htree clock_spine preferred_cell_stripe }

Default: ""

Data_type: enum, read/only

Object type name

rule_instance

The attributes in the rule_instance category are:

- [base_name](#)
- [ltms_category](#)
- [ltms_continue_on_error](#)
- [ltms_desc](#)
- [ltms_is_run](#)
- [ltms_is_virtual](#)
- [ltms_rule_filters](#)
- [ltms_rule_occurrences](#)
- [ltms_severity](#)
- [ltms_status](#)
- [ltms_table_header](#)
- [name](#)
- [obj_type](#)

base_name

`base_name` *string*

Default: ""

Data_type: string, read/only

Name of the rule.

ltms_category

ltms_category *string*

Default: ""

Data_type: string, read/only

Category of the rule.

ltms_continue_on_error

ltms_continue_on_error {true|false}

Default: ""

Data_type: boolean, read/only

Check whether we should continue checking this rule in case of an error.

ltms_desc

ltms_desc *string*

Default: ""

Data_type: string, read/only

Description of the rule.

ltms_is_run

ltms_is_run {true|false}

Default: ""

Data_type: boolean, read/only

Check whether the rule_instance is run.

ltms_is_virtual

ltms_is_virtual {true|false}

Default: ""

Data_type: boolean, read/only

Check whether the rule instance is virtual.

ltms_rule_filters

`ltms_rule_filters rule_filter`

Default: ""

Data_type: object, read/only

List of filters of the rule instance.

ltms_rule_occurrences

`ltms_rule_occurrences rule_occurrence`

Default: ""

Data_type: object, read/only

List of occurrences of the rule instance.

ltms_severity

`ltms_severity string`

Default: ""

Data_type: string, read/only

Severity of the rule.

ltms_status

`ltms_status string`

Default: ""

Data_type: string, read/only

Status of the rule.

ltms_table_header

ltms_table_header *string*

Default: ""

Data_type: string, read/only

Table header of the rule.

name

name *string*

Default: ""

Data_type: string, read/only

Full name of the rule.

obj_type

obj_type { design base_cell module inst root net port base_pin pg_base_pin pin layer
net_group hinst hpin hnet boundary density_shape shape route_rule via_def_rule
stack_via_rule
pg_pin bus port_bus hport_bus hpin_bus power_domain hport layer_rule
pin_blockage pin_group pin_guide resize_blockage row sdp track_pattern antenna_model
route_type
group via_def io_constraint antenna_data bump bump_pin bus_guide marker
text trim_grid special_via special_wire port_shape package_component package_object
gcell_grid
gcell foreign_cell gui_line gui_rect gui_text gui_polygon site via wire
patch_wire virtual_wire layer_shape shape_via place_blockage route_blockage partition
physical_pin resistor
inst_obs_shape message bus_sink_group tech bump_grid def_scan_chain
tap_distance_rule tap_layer_distance interposer_route_rule flow_step flow attribute
isolation_rule
level_shifter_rule retention_rule power_switch_rule opcond timing_point
path_group lib_arc lib_pin clock arc lib_cell library delay_noise_obj timing_path
rc_corner library_set
timing_condition analysis_view constraint_mode delay_corner seq_function
context_data context_top_clock what_if_wire what_if_via rule_instance rule_occurrence
rule_filter

const_hier_exc1 const_comp_exc1 const_hier_exc2 const_comp_exc2
const_hier_exc3 const_comp_exc3 const_hier_exc4 const_comp_exc4 const_hier_exc5
const_comp_exc5 const_hier_exc6 const_comp_exc6 const_hier_exc7
const_comp_exc7 const_hier_exc10 const_comp_exc10 const_hier_exc11 const_comp_exc11
const_hier_exc12 const_comp_exc12 const_hier_exc13 const_comp_exc13
const_hier_exc14 const_comp_exc14 const_hier_exc15 const_comp_exc15 const_hier_exc16
const_comp_exc16 const_hier_exc17 const_comp_exc17 const_hier_clk2a
const_comp_clk2a const_hier_clk2b const_comp_clk2b const_hier_clk2c const_comp_clk2c
const_hier_clk1 const_comp_clk1 const_hier_clk3 const_comp_clk3
const_hier_clk2d const_comp_clk2d const_hier_clk2e const_comp_clk2e const_hier_clk2f
const_comp_clk2f

const_hier_clk4 const_comp_clk4 const_hier_clk7 const_comp_clk7
const_hier_clk8 const_comp_clk8 const_hier_io2 const_comp_io2 const_hier_io4
const_comp_io4

const_hier_sca1 const_comp_sca1 const_hier_sdt1 const_comp_sdt1
const_hier_grp1 const_comp_grp1 const_hier_cg1 const_comp_cg1 const_hier_io5i
const_comp_io5i

const_hier_io5o const_comp_io5o const_hier_io6i const_comp_io6i
const_hier_io6o const_comp_io6o const_hier_io7i const_comp_io7i const_hier_io7o
const_comp_io7o

const_hier_io8i const_comp_io8i const_hier_io8o const_comp_io8o
const_hier_ctrl const_comp_ctrl const_hier_cg2 const_comp_cg2 const_hier_unc1
const_comp_unc1

const_hier_dat1 const_comp_dat1 const_hier_pls1 const_comp_pls1
const_hier_iol1i const_comp_iol1i const_hier_iol1o const_comp_iol1o const_hier_lat1
const_comp_lat1

const_hier_iol2i const_comp_iol2i const_hier_iol2o const_comp_iol2o
const_hier_old1 const_comp_old1 const_hier_hfn2 const_comp_hfn2 const_hier_tim2
const_comp_tim2

const_hier_clk6 const_comp_clk6 const_hier_io1 const_comp_io1 const_hier_io3
const_comp_io3 const_hier_io9 const_hier_io10 const_lint_cg1 const_lint_scope2
const_lint_cg2 const_lint_clk1 const_lint_clk2 const_lint_clk3
const_lint_clk5 const_lint_clk4 const_lint_clk6 const_lint_clk7 const_lint_clk8
const_lint_clk9 const_lint_cmd1

const_lint_ctrl const_lint_ctr2 const_lint_ctr3 const_lint_ctr4
const_lint_ctr5 const_lint_ctr7 const_lint_exc1 const_lint_exc2 const_lint_flp1
const_lint_hfn1 const_lint_hfn2

const_lint_idl6 const_lint_idl7 const_lint_info1 const_lint_itr2
const_lint_itr1 const_lint_itr3 const_lint_itr4 const_lint_lat1 const_lint_lat2
const_lint_lat3 const_lint_lat4

const_lint_mcpl const_lint_odl1 const_lint_odl2 const_lint_odl3

const_lint_odl4 const_lint_odl5 const_lint_odl6 const_lint_odl7 const_lint_odl8
const_lint_old1 const_lint_old2
const_lint_ref1 const_lint_ref2 const_lint_ref4 const_lint_sca1
const_lint_unc1 const_lint_unc2 const_lint_idl1 const_lint_idl2 const_lint_idl3
const_lint_idl4 const_lint_idl5
const_lint_dat1 const_lintcfg_cmd2 const_lintcfg_cmd3 const_lintcfg_cmd4
const_lintcfg_opt10 const_lintcfg_opt2 const_lintcfg_opt3 const_lintcfg_opt4
const_lintcfg_opt5
const_lintcfg_opt6 const_lintcfg_opt7 const_lintcfg_opt8 const_lintcfg_ref4
const_lintcfg_ref5 const_lintcfg_ref7 const_lintcfg_val3 const_lintcfg_val4
const_sca_chk1
const_sca_chk2 const_clk_ctr1 const_clk_ctr2 const_clk_ctr4 const_clk_ctr6
const_clk_ctr9 const_clk_ctr10 const_clk_ctr12 const_clk_ctr13 const_clk_def1
const_clk_def3
const_clk_def4 const_clk_def2 const_clk_def6 const_clk_def10 const_clk_def11
const_clk_def12 const_clk_def13 const_clk_def23 const_clk_def26 const_clk_def49
const_clk_def32 const_clk_def33 const_clk_def34 const_clk_def35
const_clk_def36 const_clk_def37 const_clk_def38 const_clk_def39 const_clk_def48
const_clk_def47
const_clk_def40 const_clk_def41 const_clk_def42 const_clk_def43
const_clk_def44 const_clk_def45 const_clk_def46 const_clk_grp1 const_clk_grp2
const_clk_grp3
const_clk_grp4 const_clk_grp5 const_clk_grp7 const_clk_lat1 const_clk_lat3
const_clk_lat4 const_clk_lat6 const_clk_lat7 const_clk_lat8 const_clk_lat11
const_clk_lat12
const_clk_lat13 const_clk_unc1 const_clk_unc3 const_clk_unc4 const_clk_unc6
const_clk_unc7 const_clk_unc10 const_dat_chk1 const_dat_chk2 const_dat_chk3
const_dat_chk4 const_clk_unc9 const_dgn_cmb1 const_dgn_cmb2 const_dgn_cmb3
const_dgn_cmb5 const_dgn_cmb6 const_dgn_cmb7 const_exc_clk1 const_exc_flp1
const_exc_flp7 const_exc_flp8 const_exc_flp9 const_exc_mcp8 const_exc_smd3
const_exc_smd4 const_exc_smd5 const_exc_smd6 const_exc_smd7 const_exc_flp10
const_exc_flp11 const_exc_mcp1 const_exc_mcp5 const_exc_mcp6 const_exc_mcp7
const_exc_olp2 const_exc_olpla const_exc_olplb const_exc_olplc const_exc_olpld
const_exc_olple const_exc_olplf const_exc_smd1 const_exc_smd2 const_io_idl1
const_io_old1 const_io_old3 const_io_old4 const_io_drc1 const_io_drc2 const_io_drc3
const_io_idl4 const_io_idl5 const_io_idl6 const_io_idl13 const_io_idl14
const_io_idl15 const_io_odl5 const_io_odl6 const_io_odl13 const_io_odl14 const_io_odl15
const_io_itr1
const_io_itr2 const_io_itr3 const_io_itr10 const_io_odl1 const_io_odl4
const_cg_chk1 const_cg_chk2 const_cg_chk3 const_dgn_tim1 const_clk_def21 const_clk_def7
const_hfn_chk1a const_hfn_chk1b const_hfn_chk2 const_hfn_chk3

```
const_lint_unc3 const_clk_grp6 const_lint_grp2 const_lint_grp3 const_cg_chk5
const_cg_chk6
    const_cg_chk7 const_dat_chk5 const_dgn_tim2 const_exc_clk2 const_cov_cg
const_cov_cp const_cov_cs const_cov_data const_cov_pw const_cov_rr const_cov_sh
    const_cov_skew clock_tree skew_group clock_tree_source_group flexible_htree
clock_spine preferred_cell_stripe }
```

Default: ""

Data_type: enum, read/only

Object type name

rule_occurrence

The attributes in the rule_occurrence category are:

- [base_name](#)
- [ltms_filtered_by](#)
- [ltms_is_filtered](#)
- [ltms_message](#)
- [ltms_rule_instance](#)
- [ltms_severity](#)
- [ltms_status](#)
- [name](#)
- [obj_type](#)

base_name

```
base_name string
```

Default: ""

Data_type: string, read/only

Name of the rule.

ltms_filtered_by

`ltms_filtered_by rule_filter`

Default: ""

Data_type: rule_filter, read/only

Filter applied to the occurrence.

ltms_is_filtered

`ltms_is_filtered {true|false}`

Default: ""

Data_type: boolean, read/only

Check whether the rule_occurrence is filtered out.

ltms_message

`ltms_message string`

Default: ""

Data_type: boolean, read/only

Message of the rule occurrence.

ltms_rule_instance

`ltms_rule_instance rule_instance`

Default: ""

Data_type: string, read/only

Rule instance to which this occurrence belongs.

ltms_severity

`ltms_severity string`

Default: ""

Data_type: string, read/only

Severity of the rule.

ltms_status

ltms_status *string*

Default: ""

Data_type: string, read/only

Status of the rule.

name

name *string*

Default: ""

Data_type: string, read/only

Full name of the rule.

obj_type

```
obj_type { design base_cell module inst root net port base_pin pg_base_pin pin layer
net_group hinst hpin hnet boundary density_shape shape
        route_rule via_def_rule stack_via_rule pg_pin bus port_bus hport_bus
hpin_bus power_domain hport layer_rule pin_blockage pin_group
        pin_guide  resize_blockage row sdp track_pattern antenna_model route_type
group via_def io_constraint antenna_data bump bump_pin
        bus_guide marker text  trim_grid special_via special_wire port_shape
package_component package_object gcell_grid gcell foreign_cell
        gui_line gui_rect gui_text  gui_polygon site via wire patch_wire
virtual_wire layer_shape shape_via place_blockage route_blockage
        partition physical_pin resistor  inst_obs_shape message bus_sink_group tech
bump_grid def_scan_chain tap_distance_rule tap_layer_distance
        interposer_route_rule flow_step flow attribute isolation_rule
level_shifter_rule retention_rule power_switch_rule opcond timing_point
        path_group lib_arc lib_pin clock arc lib_cell library delay_noise_obj
timing_path rc_corner library_set timing_condition analysis_view
        constraint_mode delay_corner seq_function context_data  context_top_clock
```

what_if_wire what_if_via rule_instance rule_occurrence
rule_filter const_hier_exc1 const_comp_exc1 const_hier_exc2 const_comp_exc2
const_hier_exc3 const_comp_exc3 const_hier_exc4 const_comp_exc4
const_hier_exc5 const_comp_exc5 const_hier_exc6 const_comp_exc6
const_hier_exc7 const_comp_exc7 const_hier_exc10 const_comp_exc10
const_hier_exc11 const_comp_exc11 const_hier_exc12 const_comp_exc12
const_hier_exc13 const_comp_exc13 const_hier_exc14 const_comp_exc14
const_hier_exc15 const_comp_exc15 const_hier_exc16 const_comp_exc16
const_hier_exc17 const_comp_exc17 const_hier_clk2a const_comp_clk2a
const_hier_clk2b const_comp_clk2b const_hier_clk2c const_comp_clk2c
const_hier_clk1 const_comp_clk1 const_hier_clk3 const_comp_clk3
const_hier_clk2d const_comp_clk2d const_hier_clk2e const_comp_clk2e
const_hier_clk2f const_comp_clk2f const_hier_clk4 const_comp_clk4
const_hier_clk7 const_comp_clk7 const_hier_clk8 const_comp_clk8
const_hier_io2 const_comp_io2 const_hier_io4 const_comp_io4 const_hier_sca1
const_comp_sca1 const_hier_sdt1 const_comp_sdt1 const_hier_grp1
const_comp_grp1 const_hier_cg1 const_comp_cg1 const_hier_io5i const_comp_io5i
const_hier_io5o const_comp_io5o const_hier_io6i const_comp_io6i
const_hier_io6o const_comp_io6o const_hier_io7i const_comp_io7i const_hier_io7o
const_comp_io7o const_hier_io8i const_comp_io8i const_hier_io8o
const_comp_io8o const_hier_ctr1 const_comp_ctr1 const_hier_cg2 const_comp_cg2
const_hier_unc1 const_comp_unc1 const_hier_dat1 const_comp_dat1
const_hier_pls1 const_comp_pls1 const_hier_io11i const_comp_io11i const_hier_io11o
const_comp_io11o const_hier_lat1 const_comp_lat1 const_hier_io12i
const_comp_io12i const_hier_io12o const_comp_io12o const_hier_old1 const_comp_old1
const_hier_hfn2 const_comp_hfn2 const_hier_tim2 const_comp_tim2
const_hier_clk6 const_comp_clk6 const_hier_io1 const_comp_io1 const_hier_io3
const_comp_io3 const_hier_io9 const_hier_io10 const_lint_cg1
const_lint_scope2 const_lint_cg2 const_lint_clk1 const_lint_clk2 const_lint_clk3
const_lint_clk5 const_lint_clk4 const_lint_clk6 const_lint_clk7
const_lint_clk8 const_lint_clk9 const_lint_cmd1 const_lint_ctr1 const_lint_ctr2
const_lint_ctr3 const_lint_ctr4 const_lint_ctr5 const_lint_ctr7
const_lint_exc1 const_lint_exc2 const_lint_flp1 const_lint_hfn1 const_lint_hfn2
const_lint_idl6 const_lint_idl7 const_lint_info1 const_lint_itr2
const_lint_itr1 const_lint_itr3 const_lint_itr4 const_lint_lat1 const_lint_lat2
const_lint_lat3 const_lint_lat4 const_lint_mcp1 const_lint_odl1
const_lint_odl2 const_lint_odl3 const_lint_odl4 const_lint_odl5 const_lint_odl6
const_lint_odl7 const_lint_odl8 const_lint_old1 const_lint_old2
const_lint_ref1 const_lint_ref2 const_lint_ref4 const_lint_sca1 const_lint_unc1
const_lint_unc2 const_lint_idl1 const_lint_idl2 const_lint_idl3
const_lint_idl4 const_lint_idl5 const_lint_dat1 const_lintcfg_cmd2 const_lintcfg_cmd3
const_lintcfg_cmd4 const_lintcfg_opt10 const_lintcfg_opt2 const_lintcfg_opt3
const_lintcfg_opt4 const_lintcfg_opt5 const_lintcfg_opt6 const_lintcfg_opt7

```

const_lintcfg_opt8 const_lintcfg_ref4 const_lintcfg_ref5 const_lintcfg_ref7
const_lintcfg_val3 const_lintcfg_val4 const_sca_chk1 const_sca_chk2 const_clk_ctrl1
const_clk_ctr2 const_clk_ctr4 const_clk_ctr6 const_clk_ctr9 const_clk_ctr10
const_clk_ctr12 const_clk_ctr13 const_clk_def1 const_clk_def3 const_clk_def4
const_clk_def2 const_clk_def6 const_clk_def10 const_clk_def11
const_clk_def12 const_clk_def13 const_clk_def23 const_clk_def26 const_clk_def49
const_clk_def32
const_clk_def33 const_clk_def34 const_clk_def35 const_clk_def36
const_clk_def37 const_clk_def38 const_clk_def39 const_clk_def48 const_clk_def47
const_clk_def40
const_clk_def41 const_clk_def42 const_clk_def43 const_clk_def44
const_clk_def45 const_clk_def46 const_clk_grp1 const_clk_grp2 const_clk_grp3
const_clk_grp4
const_clk_grp5 const_clk_grp7 const_clk_lat1 const_clk_lat3 const_clk_lat4
const_clk_lat6 const_clk_lat7 const_clk_lat8 const_clk_lat11 const_clk_lat12
const_clk_lat13 const_clk_unc1 const_clk_unc3 const_clk_unc4 const_clk_unc6
const_clk_unc7 const_clk_unc10 const_dat_chk1 const_dat_chk2 const_dat_chk3
const_dat_chk4 const_clk_unc9 const_dgn_cmb1 const_dgn_cmb2 const_dgn_cmb3
const_dgn_cmb5 const_dgn_cmb6 const_dgn_cmb7 const_exc_clk1 const_exc_flp1
const_exc_flp7 const_exc_flp8 const_exc_flp9 const_exc_mcp8 const_exc_smd3
const_exc_smd4 const_exc_smd5 const_exc_smd6 const_exc_smd7 const_exc_flp10
const_exc_flp11 const_exc_mcp1 const_exc_mcp5 const_exc_mcp6 const_exc_mcp7
const_exc_olp2 const_exc_olp1a const_exc_olp1b const_exc_olp1c const_exc_olp1d
const_exc_olp1e const_exc_olp1f const_exc_smd1 const_exc_smd2 const_io_idl1
const_io_old1 const_io_old3 const_io_old4 const_io_drc1 const_io_drc2 const_io_drc3
const_io_idl4 const_io_idl5 const_io_idl6 const_io_idl13 const_io_idl14
const_io_idl15 const_io_odl5 const_io_odl6 const_io_odl13 const_io_odl14 const_io_odl15
const_io_itr1 const_io_itr2 const_io_itr3 const_io_itr10 const_io_odl1
const_io_odl4 const_cg_chk1 const_cg_chk2 const_cg_chk3 const_dgn_tim1 const_clk_def21
const_clk_def7 const_hfn_chk1a const_hfn_chk1b const_hfn_chk2 const_hfn_chk3
const_lint_unc3 const_clk_grp6 const_lint_grp2 const_lint_grp3 const_cg_chk5
const_cg_chk6 const_cg_chk7 const_dat_chk5 const_dgn_tim2 const_exc_clk2
const_cov_cg const_cov_cp const_cov_cs const_cov_data const_cov_pw const_cov_rr
const_cov_sh const_cov_skew clock_tree skew_group clock_tree_source_group
flexible_htree clock_spine preferred_cell_stripe }

```

Default: ""

Data_type: enum, read/only

Object type name

SVA Attributes

- [ltms_sva_assertion_in_module_limit](#)
- [ltms_sva_assertion_limit](#)
- [ltms_sva_assertion_prefix](#)
- [ltms_sva_assertion_suffix](#)
- [ltms_sva_cgc_file_prefix](#)
- [ltms_sva_cgc_file_suffix](#)
- [ltms_sva_cgc_use_edge_sampling](#)
- [ltms_sva_clock_selection](#)
- [ltms_sva_disable_sdc_comment_processing](#)
- [ltms_sva_file_prefix](#)
- [ltms_sva_file_suffix](#)
- [ltms_sva_generate_comment](#)
- [ltms_sva_include_per_property_comment](#)
- [ltms_sva_inc_buf](#)
- [ltms_sva_mcp_assertion_limit](#)
- [ltms_sva_mcp_assertion_prefix](#)
- [ltms_sva_mcp_assertion_suffix](#)
- [ltms_sva_mcp_file_prefix](#)
- [ltms_sva_mcp_file_suffix](#)
- [ltms_sva_mcp_module_prefix](#)
- [ltms_sva_mcp_module_suffix](#)
- [ltms_sva_mcp_reset_signal_name](#)

- ltms_sva_mcp_skip_single_cycle
- ltms_sva_module_prefix
- ltms_sva_module_suffix
- ltms_sva_num_paths_per_exception
- ltms_sva_path_based_sampling_exhaustive
- ltms_sva_path_based_sampling_opt_level
- ltms_sva_reset_signal_name
- ltms_sva_sca_clock_define
- ltms_sva_sca_clock_name
- ltms_sva_sca_config_pin_exclude
- ltms_sva_sca_config_pin_include
- ltms_sva_sca_enable_sca_text
- ltms_sva_sca_fail_string
- ltms_sva_sca_file_extension
- ltms_sva_sca_file_prefix
- ltms_sva_sca_file_suffix
- ltms_sva_sca_id
- ltms_sva_sca_module_name
- ltms_sva_sca_reset_define
- ltms_sva_sca_reset_polarity_define
- ltms_sva_sca_reset_polarity_value
- ltms_sva_sca_reset_signal_name
- ltms_sva_sca_tag
- ltms_sva_sca_top_inst_define
- ltms_sva_sca_top_inst_name
- ltms_sva_sca_use_user_defined_clock
- ltms_sva_sca_valid_string

- [ltms_sva_sdt_clock_define](#)
- [ltms_sva_sdt_clock_name](#)
- [ltms_sva_sdt_file_prefix](#)
- [ltms_sva_sdt_file_suffix](#)
- [ltms_sva_sdt_id](#)
- [ltms_sva_sdt_label](#)
- [ltms_sva_sdt_module_name](#)
- [ltms_sva_sdt_property_macro_name](#)
- [ltms_sva_sdt_reset_define](#)
- [ltms_sva_sdt_tag](#)
- [ltms_sva_sdt_top_define](#)
- [ltms_sva_sdt_top_inst_name](#)
- [ltms_sva_sdt_use_user_defined_clock](#)
- [ltms_sva_sfp_assertion_limit](#)
- [ltms_sva_sfp_assertion_prefix](#)
- [ltms_sva_sfp_assertion_suffix](#)
- [ltms_sva_sfp_cycle](#)
- [ltms_sva_sfp_file_prefix](#)
- [ltms_sva_sfp_file_suffix](#)
- [ltms_sva_sfp_module_prefix](#)
- [ltms_sva_sfp_module_suffix](#)
- [ltms_sva_sfp_reset_signal_name](#)
- [ltms_sva_traverse_through_mux_select_pin](#)

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `sva` category, use the following command:

```
get_db -category sva
```

ltms_sva_assertion_in_module_limit

```
ltms_sva_assertion_in_module_limit int
```

Default: 1000

Data_type: Integer

Defines the maximum number of assertion generated per module.

ltms_sva_assertion_limit

```
ltms_sva_assertion_limit intltms_sva_sdt_tag
```

Default: 0 (no limit)

Data_type: Integer

Defines the maximum number of SVA assertions to generate.

ltms_sva_assertion_prefix

```
ltms_sva_assertion_prefix string
```

Default: " "

Data_type: String

Defines the prefix for false-path and multi-cycle-path assertions.

ltms_sva_assertion_suffix

```
ltms_sva_assertion_suffix string
```

Default: " "

Data_type: String

Defines the suffix for false-path and multi-cycle-path assertions.

ltms_sva_cgc_file_prefix

```
ltms_sva_cgc_file_prefix string
```

Default: " "

Data_type: String

Defines the prefix for `create_generated_clock` SVA file. If not defined, `ltms_sva_file_prefix` is used.

ltms_sva_cgc_file_suffix

`ltms_sva_cgc_file_suffix` *string*

Default: " "

Data_type: String

Defines the suffix for `create_generated_clock` SVA file. If not defined, `ltms_sva_file_suffix` is used.

ltms_sva_cgc_use_edge_sampling

`ltms_sva_cgc_use_edge_sampling` *bool*

Default: false

Data_type: Boolean

Setting this to 'true', enables the edge sampling mode of SVA generation for CGC (`create_generated_clock`) and disables the toggle based method (Default).

ltms_sva_clock_selection

`ltms_sva_clock_selection` *string*

Default: all_propagated

Data_type: String

Defines the clock(s) to be used for assertions: `fast` | `fast_propagated` | `slow` | `slow_propagated` | `all` | `all_propagated` | `<clock_name>`

ltms_sva_disable_sdc_comment_processing

`ltms_sva_disable_sdc_comment_processing` *bool*

Default: true

Data_type: Boolean

Setting this to 'true', disables the feature of filtering exceptions based on comments in SDC command. Attribute ltms_sva_generate_comment is not honored and exceptions category is not picked from SDC comment when -use_category is specified with generate_sva.

ltms_sva_file_prefix

ltms_sva_file_prefix *string*

Default: " "

Data_type: String

Defines the prefix for false-path, multi-cycle-path and create_generated_clock SVA files.

ltms_sva_file_suffix

ltms_sva_file_suffix *string*

Default: " "

Data_type: String

Defines the suffix for false-path, multi-cycle-path and create_generated_clock SVA files.

ltms_sva_generate_comment

ltms_sva_generate_comment *string*

Default: " "

Data_type: String

Defines the comment to be added to generated assertions.

ltms_sva_include_per_property_comment

ltms_sva_include_per_property_comment *bool*

Default: false

Data_type: Boolean

When set to 'false', comments (// From|Through|To/Clock) for each SVA property are not written in output SVA file.

ltms_sva_inc_buf

ltms_sva_inc_buf *bool*

Default: false

Data_type: Boolean

Include buffer in SFP/MCP path analysis.

ltms_sva_mcp_assertion_limit

ltms_sva_mcp_assertion_limit *int*

Default: -1

Data_type: Integer

Defines the maximum number of SVA assertions to generate for multi_cycle_path constraint type.

0 = no limit

If unset, ltms_sva_assertion_limit is used.

ltms_sva_mcp_assertion_prefix

ltms_sva_mcp_assertion_prefix *string*

Default: " "

Data_type: String

Defines the prefix for multi-cycle-path assertions. If not defined, ltms_sva_assertion_prefix is used.

ltms_sva_mcp_assertion_suffix

ltms_sva_mcp_assertion_suffix *string*

Default: " "

Data_type: String

Defines the suffix for multi-cycle-path assertions. If not defined, ltms_sva_assertion_suffix is used.

ltms_sva_mcp_file_prefix

ltms_sva_mcp_file_prefix *string*

Default: " "

Data_type: String

Defines the prefix for multi-cycle-path SVA file. If not defined, ltms_sva_file_prefix is used.

ltms_sva_mcp_file_suffix

ltms_sva_mcp_file_suffix *string*

Default: " "

Data_type: String

Defines the suffix for multi-cycle-path SVA file. If not defined, ltms_sva_file_suffix is used.

ltms_sva_mcp_module_prefix

ltms_sva_mcp_module_prefix *string*

Default: " "

Data_type: String

Defines the prefix for multi-cycle-path module. If not defined, ltms_sva_module_prefix is used.

ltms_sva_mcp_module_suffix

ltms_sva_mcp_module_suffix *string*

Default: " "

Data_type: String

Defines the suffix for multi-cycle-path module. If not defined, ltms_sva_module_suffix is used.

ltms_sva_mcp_reset_signal_name

ltms_sva_mcp_reset_signal_name *string*

Default: " "

Data_type: String

Defines the reset signal to use for multi-cycle-path assertions: unique | <signal_name>

ltms_sva_mcp_skip_single_cycle

`ltms_sva_mcp_skip_single_cycle bool`

Default: true

Data_type: Boolean

When set to 'true' tool does not generate any SVA corresponding to the MCP exceptions with cycle value of 1.

ltms_sva_module_prefix

`ltms_sva_module_prefix string`

Default: " "

Data_type: String

Defines the prefix for false-path and multi-cycle-path module.

ltms_sva_module_suffix

`ltms_sva_module_prefix string`

Default: " "

Data_type: String

Defines the suffix for false-path and multi-cycle-path module.

ltms_sva_num_paths_per_exception

`ltms_sva_num_paths_per_exception int`

Default: 10000

Data_type: Integer

Define the maximum number of paths to be processed per exception SDC command. Default:10000

ltms_sva_path_based_sampling_exhaustive

`ltms_sva_path_based_sampling_exhaustive bool`

Default: false

Data_type: Boolean

When set to 'true' tool exhaustively processes all paths corresponding to MCP/SFP exceptions, applies only to -sfp / -mcp options, causes higher runtime for SVA generation.

ltms_sva_path_based_sampling_opt_level

ltms_sva_path_based_sampling_opt_level *enum*

Default: 1

Data_type: Enum

Define level of optimization for Path Based SVA for MCP/FP: 1 | 2 | 3

- 1 = Mode1 (default)
- 2 = Mode2
- 3 = Basic Mode

ltms_sva_reset_signal_name

ltms_sva_reset_signal_name *string*

Default: 1'b0

Data_type: String

Defines the reset signal to use for all assertions: unique | <signal_name>

ltms_sva_sca_clock_define

ltms_sva_sca_clock_define *string*

Default: SDC_CLOCK_CASE

Data_type: String

Specify tickdefine for clock in testbench.

ltms_sva_sca_clock_name

ltms_sva_sca_clock_name *string*

Default: tb.dut.clk

Data_type: String

Specify name of clock to be used in SCA testbench.

ltms_sva_sca_config_pin_exclude

ltms_sva_sca_config_pin_exclude *string*

Default: " "

Data_type: String

Specify file containing pin list to be excluded from SCA testbench.

ltms_sva_sca_config_pin_include

ltms_sva_sca_config_pin_include *string*

Default: " "

Data_type: String

Specify file containing pin list to be included in SCA testbench.

ltms_sva_sca_enable_sca_text

ltms_sva_sca_enable_sca_text *bool*

Default: false

Data_type: Boolean

Specify whether to print original set_case_anaysis command in testbench messages. When set to true, original command from SDC / constraint file is used. This usually leads to increased runtime for testbench generation.

ltms_sva_sca_fail_string

ltms_sva_sca_fail_string *string*

Default: [FAIL]

Data_type: String

Define string to display fail events in SCA testbench.

ltms_sva_sca_file_extension

ltms_sva_sca_file_extension *string*

Default: sv

Data_type: String

Define extension for set_case_analysis SVA/testbench file.

ltms_sva_sca_file_prefix

`ltms_sva_sca_file_prefix string`

Default: " "

Data_type: String

Defines the prefix for set_case_analysis SVA files.

ltms_sva_sca_file_suffix

`ltms_sva_sca_file_suffix string`

Default: " "

Data_type: String

Defines the suffix for set_case_analysis SVA files. If not defined, ltms_sva_file_suffix is used.

ltms_sva_sca_id

`ltms_sva_sca_id string`

Default: CASE_ID

Data_type: String

Define ID for SCA testbench display messages.

ltms_sva_sca_module_name

`ltms_sva_sca_module_name string`

Default: tb_set_case_analysis

Data_type: String

Defines the name for set_case_analysis SVA module.

ltms_sva_sca_reset_define

`ltms_sva_sca_reset_define string`

Default: SDC_DISABLE_CASE

Data_type: String

Specify tickdefine for reset signal in testbench.

ltms_sva_sca_reset_polarity_define

`ltms_sva_sca_reset_polarity_define string`

Default: SDC_DISABLE_CASE_POLARITY

Data_type: String

Specify tickdefine for reset signal polarity in testbench.

ltms_sva_sca_reset_polarity_value

`ltms_sva_sca_reset_polarity_value string`

Default: 1

Data_type: String

Specify polarity of reset signal to be used in SCA testbench.

ltms_sva_sca_reset_signal_name

`ltms_sva_sca_reset_signal_name string`

Default: tb.dut.reset

Data_type: String

Specify name of reset signal to be used in SCA testbench.

ltms_sva_sca_tag

`ltms_sva_sca_tag string`

Default: [SDC_CASE_ANALYSIS]

Data_type: String

Defines the TAG for SCA testbench display messages.

ltms_sva_sca_top_inst_define

ltms_sva_sca_top_inst_define *string*

Default: TOP_INST

Data_type: String

Specify tickdefine of testbench top inst name to be used in SVA, when providing a tickdefines files.

ltms_sva_sca_top_inst_name

ltms_sva_sca_top_inst_name *string*

Default: tb.dut

Data_type: String

Specify name of top inst to be used in SCA testbench.

ltms_sva_sca_use_user_defined_clock

ltms_sva_sca_use_user_defined_clock *bool*

Default: false

Data_type: Boolean

Specify whether to use user defined clock in SCA testbench. When set to true, attribute ltms_sva_sca_clock_define and ltms_sva_sca_clock_name are used.

ltms_sva_sca_valid_string

ltms_sva_sca_valid_string *string*

Default: [VALID]

Data_type: String

Define string to display valid / pass events in SCA testbench.

ltms_sva_sdt_clock_define

ltms_sva_sdt_clock_define *string*

Default: ltms_sva_sdt_CLK

Data_type: String

Specify literal to be used define clock signal in SVA SDT testbench.

ltms_sva_sdt_clock_name

`ltms_sva_sdt_clock_name` *string*

Default: TB.clk

Data_type: String

Specify literal to be used define clock signal in SVA SDT testbench.

ltms_sva_sdt_file_prefix

`ltms_sva_sdt_file_prefix` *string*

Default: " "

Data_type: String

Defines the prefix for set_disable_timing SVA files.

ltms_sva_sdt_file_suffix

`ltms_sva_sdt_file_suffix` *string*

Default: " "

Data_type: String

Defines the suffix for set_disable_timing SVA files. If not defined, ltms_sva_file_suffix is used.

ltms_sva_sdt_id

`ltms_sva_sdt_id` *string*

Default: DISABLE_ID

Data_type: String

Defines ID for SDT testbench display messages.

ltms_sva_sdt_label

`ltms_sva_sdt_label` *string*

Default: a_set_disable_timing

Data_type: String

Specify string to be used as label prefix for SDT SVAs.

ltms_sva_sdt_module_name

`ltms_sva_sdt_module_name` *string*

Default: `tb_set_disable_timing`

Data_type: String

Defines the name for `set_disable_timing` SVA module. If not defined, 'tb_set_disable_timing' is used.

ltms_sva_sdt_property_macro_name

`ltms_sva_sdt_property_macro_name` *string*

Default: `p_set_disable_timing_arc`

Data_type: String

Specifies string to be used for SDT property macro signal in SDT SVA.

ltms_sva_sdt_reset_define

`ltms_sva_sdt_reset_define` *string*

Default: `ltms_sva_sdt_RESET`

Data_type: String

Specifies literal to be used define reset signal in SVA SDT testbench.

ltms_sva_sdt_tag

`ltms_sva_sdt_tag` *string*

Default: `[SDC_DISABLE_TIMING]`

Data_type: String

Defines TAG to use for SDT testbench display messages.

ltms_sva_sdt_top_define

ltms_sva_sdt_top_define *string*

Default: TOP_SDT

Data_type: String

Specifies literal to be used as top define be used in SVA SDT testbench.

ltms_sva_sdt_top_inst_name

ltms_sva_sdt_top_inst_name *string*

Default: tb.dut

Data_type: String

Specifies name of top inst to be used in SVA SDT testbench.

ltms_sva_sdt_use_user_defined_clock

ltms_sva_sdt_use_user_defined_clock *bool*

Default: false

Data_type: Boolean

Specifies whether to use user defined clock in SDT testbench. When set to true, attribute ltms_sva_sdt_clock_define is used.

ltms_sva_sfp_assertion_limit

ltms_sva_sfp_assertion_limit *int*

Default: -1

Data_type: Integer

Defines the maximum number of SVA assertions to generate for false-path constraint type.

0 = no limit

If unset, ltms_sva_assertion_limit is used.

ltms_sva_sfp_assertion_prefix

`ltms_sva_sfp_assertion_prefix string`

Default: " "

Data_type: String

Defines the prefix for false-path assertions. If not defined, ltms_sva_assertion_prefix is used.

ltms_sva_sfp_assertion_suffix

`ltms_sva_sfp_assertion_suffix string`

Default: " "

Data_type: String

Defines the suffix for false-path assertions. If not defined, ltms_sva_assertion_suffix is used.

ltms_sva_sfp_cycle

`ltms_sva_sfp_cycle int`

Default: 10

Data_type: Integer

Defines the number of clock cycles false-path SVA should check for signal stability.

ltms_sva_sfp_file_prefix

`ltms_sva_sfp_file_prefix string`

Default: " "

Data_type: String

Defines the prefix for false-path SVA file. If not defined, ltms_sva_file_prefix is used.

ltms_sva_sfp_file_suffix

`ltms_sva_sfp_file_suffix string`

Default: " "

Data_type: String

Defines the suffix for false-path SVA file. If not defined, ltms_sva_file_suffix is used.

ltms_sva_sfp_module_prefix

`ltms_sva_sfp_module_prefix` *string*

Default: " "

Data_type: String

Defines the prefix for false-path module. If not defined, ltms_sva_module_prefix is used.

ltms_sva_sfp_module_suffix

`ltms_sva_sfp_module_suffix` *string*

Default: " "

Data_type: String

Defines the suffix for false-path module. If not defined, ltms_sva_module_suffix is used.

ltms_sva_sfp_reset_signal_name

`ltms_sva_sfp_reset_signal_name` *string*

Default: " "

Data_type: String

Defines the reset signal to use for false-path assertions: `unique | <signal_name>`

ltms_sva_traverse_through_mux_select_pin

`ltms_sva_traverse_through_mux_select_pin` *bool*

Default: false

Data_type: Boolean

Traverse through mux select pin in SVA path analysis.

CPC Attributes

The attributes in the 'constraint promotion control' category are:

- `timing_disable_promotion_clock_prefix`
- `timing_disable_promotion_clock_group_prefix`
- `timing_enable_constant_promotion_to_top_driver`
- `timing_promote_boundary_path_exceptions`
- `timing_promote_cgc_enable_source_invert_and_multiply_by`
- `timing_promote_cgc_source_pin_with_master`
- `timing_promote_clock_to_top_port`
- `timing_promote_derived_clocks_for_exceptions`
- `timing_promote_global_block_constraints`
- `timing_promote_in_out_delay_over_combinational_cone`
- `timing_promote_inherit_clock_relationships`
- `timing_promote_input_transition_as_annotated_transition`
- `timing_promote_internal_set_input_delay`
- `timing_promote_internal_set_output_delay`
- `timing_promote_inverted_clock_pair_to_top`
- `timing_promote_set_load_over_combinational_cone`

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

timing_disable_promotion_clock_prefix

`timing_disable_promotion_clock_prefix {true | false}`

Default: false

Data_type: Boolean

Removes instance name prefix from clock names if there is no collision.

timing_disable_promotion_clock_group_prefix

`timing_disable_promotion_clock_group_prefix {true | false}`

Default: false

Data_type: Boolean

Disables the automatic addition of instance name prefixes to clock group names after promotion.

timing_enable_constant_promotion_to_top_driver

`timing_enable_constant_promotion_to_top_driver {true | false}`

Default: true

Data_type: Boolean

Enables promotion of block constants to top driver for input type.

timing_promote_boundary_path_exceptions

`timing_promote_boundary_path_exceptions {true | false}`

Default: true

Data_type: Boolean

Enables promotion of block exceptions with block ports.

timing_promote_cgc_enable_source_invert_and_multiply_by

`timing_promote_cgc_enable_source_invert_and_multiply_by {true | false}`

Default: true

Data_type: Boolean

Enables the `-source_invert` and `-multiply_by` options in promoted block clocks as cgc.

timing_promote_cgc_source_pin_with_master

```
timing_promote_cgc_source_pin_with_master {true | false}
```

Default: false

Data_type: Boolean

Replaces the CGC source of the promoted block clock with the promoted master root, when the value is set to 'true'.

timing_promote_clock_to_top_port

```
timing_promote_clock_to_top_port {true | false}
```

Default: false

Data_type: Boolean

Promotes unreachable block clocks to the top, otherwise the block clock is left at the same pin with reference to the top, when the value is set to 'true'.

timing_promote_derived_clocks_for_exceptions

```
timing_promote_derived_clocks_for_exceptions {true | false}
```

Default: true

Data_type: Boolean

Enables promotion derived clock for exceptions differences.

timing_promote_global_block_constraints

```
timing_promote_global_block_constraints {true | false}
```

Default: false

Data_type: Boolean

Promotes global block constraint to the top design, when the value is set to 'true'.

timing_promote_in_out_delay_over_combinational_cone

`timing_promote_in_out_delay_over_combinational_cone {true | false}`

Default: true

Data_type: Boolean

Enables promotion SIOD over combinational cone.

timing_promote_inherit_clock_relationships

`timing_promote_inherit_clock_relationships {true | false}`

Default: false

Data_type: Boolean

Enables promoted CGC to inherit the master clock relationship by using the `set_clock_group` commands.

timing_promote_input_transition_as_annotated_transition

`timing_promote_input_transition_as_annotated_transition {true | false}`

Default: false

Data_type: Boolean

Promotes the input transition applied on the block port as annotated transition on top pins if the block port is not directly connected to top port (driver), when the value is set to 'true'.

timing_promote_internal_set_input_delay

`timing_promote_internal_set_input_delay {true | false}`

Default: false

Data_type: Boolean

Promotes `set_input_delay` inside the blocks, when the value is set to 'true'.

timing_promote_internal_set_output_delay

`timing_promote_internal_set_output_delay {true | false}`

Default: false

Data_type: Boolean

Promotes `set_output_delay` inside the blocks, when the value is set to 'true'.

timing_promote_inverted_clock_pair_to_top

`timing_promote_inverted_clock_pair_to_top {true | false}`

Default: false

Data_type: Boolean

Promotes the clock if phase inversion is found in the path and clock is inverted, when the value is set to 'true'.

timing_promote_set_load_over_combinational_cone

`timing_promote_set_load_over_combinational_cone {true | false}`

Default: false

Data_type: Boolean

Enables promotion of set load over combinational cone.